

Volume II | 現代電腦圖學：2D／3D繪圖、建模、材質、光線追蹤與動畫

幾何與成像 × 建模與材質 × 光線追蹤 × 動畫 × 養殖數位分身

30章 · 111,551 中文字

模型審稿完成，仍待人工審定。

模型審稿不等於人工數學驗證；所有養殖案例為合成教學資料。

內容版本：2026-10-03T02:27:18.821961

HTML／PDF 圖文出版版

Volume II：從幾何到數位分身

幾何與座標 → 相機與光柵化



網格、曲線與程序化建模



線性光、材質與貼圖



射線、光傳輸與抽樣



動畫、合成資料與唯讀agent

目錄

1. 導讀：從公式到可檢查的圖學實作
2. 第 1 章 從像素到養殖場數位分身
3. 第 2 章 向量、外積與幾何判定
4. 第03章 2D／3D變換與齊次座標
5. 第04章 相機、座標系與視圖矩陣
6. 第05章 透視投影、裁切與深度
7. 第 6 章 三角形光柵化與插值
8. 第 7 章 三角網格與拓撲資料結構
9. 第08章 法線、切線與逆轉置
10. 第09章 Bézier曲線與樣條
11. 第 10 章 參數曲面與曲面離散化
12. 第 11 章 程序化建模與幾何品質
13. 第 12 章 資產交換與可重現場景
14. 第13章 色彩、線性光與影像取樣
15. 第14章 UV參數化與貼圖座標
16. 第15章 局部照明與BRDF基礎
17. 第16章 物理材質與微表面模型
18. 第 17 章 法線貼圖與表面細節
19. 第18章 著色器與GPU管線橋接
20. 第19章 射線、交點與數值穩健性
21. 第20章 包圍盒與BVH加速
22. 第 21 章 光傳輸積分與 Monte Carlo
23. 第 22 章 路徑追蹤與重要性取樣
24. 第23章 水面反射、折射與介質
25. 第24章 渲染誤差、降噪與效能驗證

- 26. 第25章 場景階層與關鍵影格
- 27. 第 26 章 四元數、旋轉與SLERP
- 28. 第 27 章 骨架、綁定姿勢與蒙皮
- 29. 第28章 逆向運動學與限制
- 30. 第29章 魚群動畫與合成資料標註
- 31. 第 30 章 整合專題：可稽核養殖數位分身
- 32. 附錄：實作契約與驗收清單
- 33. 參考來源

導讀：從公式到可檢查的圖學實作

本卷是Volume II，主題為現代2D／3D圖學。Volume I的線性代數是先備工具，不代表讀者已學過相機、材質、渲染或動畫。每章先建立幾何與數學，再提供手算、程式、測試、除錯與習題。

全書五部、三十章，初步目標約十三萬五千中文字，含附錄及修訂仍不得超過二十萬字。篇幅用來補足推導和實作，不要求湊滿。核心實驗以CPU、小型合成資料與Python／NumPy為主；Blender、GPU著色器是延伸，不是完成所有基礎練習的必要條件。

共同案例是一座虛構養殖場數位分身：幾何池體、簡化魚體、相機、光源、水面與動畫。合成影像可用於理解多模態觀測與agent的唯讀查詢，但不能憑圖像真實感推論物理精確或真實魚群行為，也不連接現場設備。

閱讀路線

第一部先處理向量、座標、相機與像素。第二部建立可交換的幾何資產。第三部處理顏色、材質與著色。第四部從求交走到光傳輸、取樣與誤差。第五部建立動畫、蒙皮、逆運動學及整合專題。

微積分、機率、輻射度量在首次使用時橋接；更完整的理論可銜接後續Volume III～V。學習目標不是只會調用軟體，而是能說清座標、推導、數值結果與失效條件。

驗證標籤

- 「預期輸出」：依推導預測，尚未實際執行。
- 「模型審稿」：不同模型核對稿件，不等於人工審定。
- 「已執行」：只限明列腳本、環境及紀錄的測試，不可推廣成全書已驗證。

起步程式 `examples/graphics_lab.py` 由主編另行撰寫，與模型生成片段分開。後續章內程式經靜態檢查及人工檢視後，才另行安排執行，避免自動執行未審查程式。

第 1 章 從像素到養殖場數位分身

學習目標與先備知識

本章是第二卷的入口。讀完之後，你應該能夠：

1. 說出圖學管線從物件座標到像素的五個階段，並指出每個階段哪些量是離散、哪些量是連續。
2. 明確陳述本章所採的右手世界系座標軸方向、相機朝向與三角形正面繞序，並說明若改用左手系需要同步調整哪些慣例。
3. 寫出一段不依賴任何影像套件的程式，把 16×16 的 RGB 陣列輸出成合法 PPM 檔，並驗證像素索引與值域。
4. 說明為什麼「合成資料集」在養殖場數位分身裡是必要的中介，而不是最終答案。

先備知識：Volume I 的向量、矩陣、內積、齊次座標；Python 3.10+ 基本語法；能在自己的環境使用 NumPy 2.2.6（本卷不代讀者安裝套件）。微積分只需要導數與極限的直覺，機率在第三部才大量使用，本章不依賴它。

本卷統一採用**右手世界系**： $+X$ 向右、 $+Y$ 向上、 $+Z$ 由畫面向觀者。相機局部看向 $-Z$ ，且 $\text{up} = +Y$ 。位置記為 $p_h = (x, y, z, 1)^T$ （column vector，直向），方向記為 $v_h = (x, y, z, 0)^T$ 。矩陣乘法慣例為

$$M_{world} = M_{parent} M_{local}, \quad p_{world} = M_{world} p_{local}.$$

三角形「正面」定義為：從外向法線方向看下去，三個頂點是逆時針。像素原點在左上角，像素中心位於 $(u + 0.5, v + 0.5)$ ，寬 W 高 H 。影像的 Y 軸向下、世界的 Y 軸向上，這條規則會在後續所有貼圖與取樣章節反覆出現，本章先把它釘住。

問題與直覺

養殖場的數位分身不是一張漂亮的 3D 圖。它是一組可以被查詢、被稽核的資料結構：池體幾何、魚群位置、時間軸、標註與不確定性。要做到這件事，最低限度必須先能把「連續的物理場景」轉成「離散的像素陣列」，並且能反過來說明每個像素代表什麼。

一條魚游過鏡頭，現實世界的光是連續分佈與連續時間。相機感測器把它切成 $W \times H$ 個感光單元，每個單元在曝光時間內累積光量，量化成整數強度。圖學管線做的其實是同一件事的另一種版本：我們用程式描述幾何、材質、光源，再把它「取樣」到像素。

這就是本章的兩個關鍵字：**取樣與量化**。取樣把連續的空間與時間切成有限格點；量化把實數強度切成有限階的整數。解析度不足、曝光不當、色彩空間搞錯，都會讓數位分身帶著系統性偏差，而這些偏差往往在下游被誤讀為「魚的異常行為」。

離散影像最常見的兩個儲存慣例是：

- 像素座標 (u, v) ：整數索引， u 從左到右、 v 從上到下。
- 像素中心 $(u + 0.5, v + 0.5)$ ：浮點座標，用於把世界點投影到影像平面。

後續所有投影、光柵化、貼圖都會用到第二種座標。若把兩者混用，會出現「整張圖偏移半個像素」或「接縫錯位一格」這類難以察覺的錯誤。

數學與幾何推導

從場景到像素

設一個場景點 $p_{obj} \in \mathbb{R}^3$ 。管線的固定順序是：

$$p_{clip} = P V M p_{obj,h}, \quad p_{ndc} = \frac{p_{clip}}{p_{clip,w}}.$$

其中 M 是模型矩陣（物件到世界）、 V 是視圖矩陣（世界到相機）、 P 是投影矩陣（相機到裁切空間）。 $P V M$ 是一個 4×4 矩陣乘一個 4 維齊次向量。透視除法用第四分量 $p_{clip,w}$ 歸一化，在正交投影下這個分量恆為 1。

本卷主線採 OpenGL 式 NDC： $x, y, z \in [-1, 1]$ 。視窗映射到像素座標為

$$u = \frac{x_{ndc} + 1}{2} W, \quad v = \frac{1 - y_{ndc}}{2} H.$$

此映射將 NDC 區間 $[-1, 1]$ 線性對應到影像座標區間 $[0, W]$ 與 $[0, H]$ 。像素索引 i 覆蓋連續區間 $[i, i + 1)$ ，其中心位於 $i + 0.5$ 。當 $x_{ndc} = -1$ 時， $u = 0$ ，對應第 0 號像素的左邊界；當 $x_{ndc} = +1$ 時， $u = W$ ，對應影像右邊界。此處 u, v 為連續影像座標，整數索引另記為 u_{idx}, v_{idx} 。本章採半開影像域 $0 \leq u < W$ ， $0 \leq v < H$ ；只有域內點才取 floor 寫入像素。 $u = W$ 或 $v = H$ 是外邊界，不是合法索引，不把域外點直接夾入影像。此慣例與後續章節保持一致，避免半像素偏移。

深度緩衝值由

$$z_{buf} = \frac{z_{ndc} + 1}{2} \in [0, 1]$$

取得。這是「OpenGL 式」的定義；WebGPU 與 Direct3D 的 z_{ndc} 範圍不同（ $[0, 1]$ ），深度轉換也不同，**不能混用**。後續章節只要出現投影，就會先宣告是哪一種。

離散影像與色彩

一張 RGB 影像可以用整數陣列 $I[u, v] \in \{0, \dots, 255\}^3$ 描述。這是線性 RGB 經過量化後的結果，但真實感測器與顯示器通常不是線性的。第 13 章會建立完整的 sRGB 轉換；本章只需要知道兩條規則：

1. 所有光度與 BRDF 計算在**線性 RGB** 進行。
2. 輸出到螢幕或檔案前才做 sRGB 編碼；normal、depth、roughness 等**資料貼圖**不做此轉換。

PPM 的 P6 變體是一種最簡單的無損 RGB 格式，檔頭為

```
P6
W H
255
```

後面接 $W \cdot H \cdot 3$ 個位元組，逐像素、由左到右、由上到下排列。它不需要任何函式庫，是本章與後續章節的首選離線輸出格式。

解析度、可見性與合成資料

解析度就是每單位影像平面上有多少個取樣點。若視野角為 θ 、影像寬為 W ，則單一像素在距離 d 處覆蓋的橫向長度約為

$$\Delta x \approx \frac{2d \tan(\theta/2)}{W}.$$

魚離相機 2 m、水平視野 60° 、寬 1280 像素時， $\Delta x \approx 1.8$ mm。如果要量測魚體曲率或體表病灶，這個尺度必須與目標特徵比較，而不是與相機規格比較。

可見性指的是哪一個表面被某個像素看到。在光柵化管線裡，可見性由深度測試決定；本章只需建立「近平面距離 $near$ 、遠平面距離 far 、 $far > near > 0$ 」的慣例。若把 $near$ 設成 0 或負值，透視投影矩陣會退化，後面會出現不是幾何誤差的數值錯誤。

合成資料的限制必須講清楚：合成影像只能驗證**管線是否自治**，不能證明管線對應真實光學或真實生態。下面每個案例結尾都會標明「合成」、「實測」或「待驗證」。

逐步手算例題

例題一：16×16 全紅影像的像素索引

設 $W = H = 16$ ，全部像素為 (255, 0, 0)。

1. 像素總數為 $16 \times 16 = 256$ 。
2. 位元組數為 $256 \times 3 = 768$ 。
3. P6 檔頭為 `P6\n16 16\n255\n`，共 $2 + 1 + 5 + 1 + 3 + 1 = 13$ 個位元組（`P6`、`\n`、`16 16`、`\n`、`255`、`\n`）。
4. 檔案總長 $13 + 768 = 781$ 位元組。
5. 像素 ($u = 3, v = 5$) 在資料區的位元組位移為 $(5 \cdot 16 + 3) \cdot 3 = 83 \cdot 3 = 249$ 。它是第 84 個像素（從 0 起算為 83）。

這條算式之後會在測試裡逐位元組比對。

例題二：把一個場景點映射到像素

相機位於原點，看向 $-Z$ ， $up = +Y$ 。視野角 90° ， $W = H = 16$ 。設世界點 $p = (0.1, 0.0, -1.0)$ 。

先判斷哪些座標需要投影。視圖矩陣 V 為單位矩陣（相機在慣例位置與朝向），所以 $p_{cam} = (0.1, 0, -1)$ 。透視投影的近似寫法是

$$x_{ndc} = \frac{x_{cam}}{-z_{cam} \tan(\theta/2)}.$$

代入 $\tan(45^\circ) = 1$ ， $z_{cam} = -1$ ：

$$x_{ndc} = \frac{0.1}{1} = 0.1, \quad y_{ndc} = \frac{0}{1} = 0.$$

再映射到像素：

$$u = \frac{0.1 + 1}{2} \cdot 16 = 8.8, \quad v = \frac{1 - 0}{2} \cdot 16 = 8.0.$$

所以這個點的連續影像座標為 $(u_{cont}, v_{cont}) = (8.8, 8.0)$ 。確認位於半開影像域內後，整數索引為 $(u_{idx}, v_{idx}) = (\lfloor 8.8 \rfloor, \lfloor 8.0 \rfloor) = (8, 8)$ 。 $x_{cam} > 0$ 對應影像右側； $y_{cam} = 0$ 對應影像中線 $v_{cont} = 8.0$ ，恰在索引7與8的像素交界。本章的半開區間規則將它分給索引8；該像素中心是8.5，不把中線與像素中心混稱。這條推導已經給出下一節程式的語意：**像素陣列與 NDC 之間只差一個仿射映射**。

實作與程式

下面的程式只用 Python 標準庫，寫出一個 16×16 的 PPM，並內建三項自我檢查：檔頭長度、像素索引、RGB 值域。它也可以接受一個可選的 NumPy 路徑，但為保持「零依賴即可執行」，預設只使用 `array` 與 `struct`。

```
"""ppm16.py — 產生並檢查 16x16 PPM 影像。只使用標準庫。"""
from array import array
import sys

def make_rgb16(width=16, height=16, fn=None):
    """回傳 bytes，內容是 RGB 樣本的 packed 序列（每像素 3 位元組）。"""
    if fn is None:
        fn = lambda u, v: (255, 0, 0) # 全紅
    buf = bytearray()
    for v in range(height):
        # v: 由上到下
        for u in range(width):
            # u: 由左到右
            r, g, b = fn(u, v)
            if not all(0 <= c <= 255 for c in (r, g, b)):
                raise ValueError(f"RGB 值域越界: {(r, g, b)} at ({u},{v})")
            buf += bytes((r, g, b))
    return bytes(buf)
```



```

def write_ppm(path, width, height, body):
    header = f"P6\n{width} {height}\n255\n".encode("ascii")
    with open(path, "wb") as f:
        f.write(header)
        f.write(body)
    return len(header) + len(body)

def index_of(u, v, width):
    """資料區內的位元組位移；用於驗證像素索引。"""
    return (v * width + u) * 3

def self_check(path="out16.ppm"):
    W = H = 16
    body = make_rgb16(W, H)
    total = write_ppm(path, W, H, body)
    # 檢查檔頭長度
    assert total == 13 + W * H * 3, f"檔案長度不對：{total}"
    # 檢查像素 (3,5) 位移
    off = index_of(3, 5, W)
    assert off == 249, f"像素位移不對：{off}"
    assert body[off:off+3] == bytes((255, 0, 0)), "像素值不對"
    # 檢查值域
    assert max(body) == 255 and min(body) == 0, "值域異常"
    return {"path": path, "bytes": total, "ok": True}

if __name__ == "__main__":
    print(self_check(*sys.argv[1:]) if len(sys.argv) > 1 else self_check())

```

三件事值得注意。第一，`fn(u, v)` 的簽章固定為「像素索引 → RGB」，這樣後續換成 checker、UV、或投影結果時不必改動輸出層。第二，`index_of` 是獨立的純函式，因為它會在之後的貼圖、mipmap、讀寫測試中被重複使用。第三，`self_check` 的斷言就是本章的驗收條件：位元組長度、特定像素位移、值域上下界。

若你的環境有 NumPy 2.2.6，可以用下面的一行等價版本產生同一份資料，但本章不要求：

```

# 可選：NumPy 版本；未安裝時略過。
import numpy as np
arr = np.zeros((16, 16, 3), dtype=np.uint8)
arr[..., 0] = 255          # 紅通道
body = arr.tobytes(order="C") # 由上到下、由左到右

```

NumPy 預設 `order="C"` 與 PPM 的列優先順序一致，但只要之後改動 `arr` 的形狀或轉置，順序就可能不再是 PPM 要的，這是常見錯誤之一。

測試與預期結果

執行 `python ppm16.py` 後，預期輸出類似：

```
{'path': 'out16.ppm', 'bytes': 781, 'ok': True}
```

預期產生的 `out16.ppm` 檔頭為 `P6\n16 16\n255\n`，資料區 768 個位元組全為 `\xff\x00\x00`。可用系統工具檢查（本卷不執行）：

```
xxd out16.ppm | head -n 2
```

預期第一行會看到 `5036 0a31 3620 3136 0a32 3535 0a` 對應 `P6\n16 16\n255\n`，接著是紅色位元組。若不符，先看檔頭，再看資料區。

以上輸出皆為以直讀程式碼得到的預期結果；本卷不聲稱任何工具或環境已被執行。

除錯與常見陷阱

1. **索引與中心混用**：投影得到的是連續影像座標 u, v ；寫入索引為 $\lfloor u \rfloor, \lfloor v \rfloor$ ；像素中心是索引 $+0.5$ 。
若把連續座標直接當中心，或反之，會偏半格。此慣例與 Ch5、Ch6 一致。
2. **Y 方向錯置**：世界 $+Y$ 向上、影像 v 向下。把世界 Y 直接當成 v 會得到上下顛倒影像。
3. **值域未來**：線性計算容易產生 < 0 或 > 255 的浮點，寫入前必須明確夾到 $[0, 255]$ 再量化。
4. **色彩空間搞混**：對 normal 貼圖做 sRGB 轉換，會讓法線方向整體偏移。
5. **PPM 檔頭多餘空白**：`P6\n16 16\n255\n` 中每個 `\n` 都是一個位元組；用 `\r\n` 或省略會讀者端解析失敗。
6. **NumPy 轉置後忘了 `tobytes` 順序**：`arr.T.tobytes()` 是另一個資料布局，不會給你轉置後的圖像。

養殖數位分身案例

想像一座循環水養殖池的數位分身骨架：

- 池體幾何： $6\text{ m} \times 3\text{ m} \times 1.2\text{ m}$ 的長方體水體。
- 相機：安裝在水面斜上方，視野 $60^\circ \times 45^\circ$ ， 1280×960 ， $near = 0.1\text{ m}$ 、 $far = 10\text{ m}$ 。
- 魚：以簡化魚體網格（後續章節建立）代表每條魚的位置與朝向。
- 標註：每條魚在影像上的像素座標、深度、識別碼。

在本章階段，這個分身只做到「把一個 16×16 的測試圖正確輸出」。這聽起來很基本，但下游所有觀測（魚體偵測、水色分析、遮擋判定）都依賴同一條管線的像素索引與色彩慣例是否一致。

必須明確標註：本章的案例是**合成場景**，不是實測影像。數位分身可以用它驗證資料流與標註管線，**不能**用來證明任何生物行為或水質結論。真實判斷仍需實際量測、實驗設計與生物／水產專業。

習題

習題 1 (手算) 設 $W = 8$ 、 $H = 8$ 、視野 90° ，相機在原點看向 $-Z$ ， $up = +Y$ 。世界點 $p = (0.05, -0.05, -0.5)$ 。求 x_{ndc}, y_{ndc} ，以及該點落入的像素 (u, v) ，並寫出它在 PPM 資料區的位元組位移。

習題 2 (程式測試) 改寫 `make_rgb16`，讓它接收一個 `checker` 函式，產生棋盤格：當 $(u + v)$ 為偶數時輸出 `(255, 255, 255)`，否則輸出 `(0, 0, 0)`。用 `self_check` 的結構寫出至少三條斷言：像素 $(0, 0)$ 、像素 $(0, 1)$ 、以及整體位元組數。

習題 3 (反例／除錯) 有人把 `index_of` 寫成 `return (u * height + v) * 3`。請指出這個函式在 16×16 時仍給出 249 的座標條件，並說明為什麼在多數情況下會與 PPM 的列優先慣例衝突。至少舉一個具體 (u, v) 例子。

習題 4 (整合應用) 設計一份「池體表面色塊圖」的流程圖：輸入池體六面、一個簡化的 Lambert 反照率，輸出 16×16 的 PPM。流程圖必須包含：模型矩陣、視圖矩陣、投影矩陣、像素映射、量化與寫檔六個階段，並標明哪一步使用線性 RGB。這個練習不需要實作，只需寫出各階段輸入／輸出與單位。

習題解答

習題 1 解答 由 $\tan(45^\circ) = 1$ ， $z_{cam} = -0.5$ ：

$$x_{ndc} = \frac{0.05}{0.5} = 0.1, \quad y_{ndc} = \frac{-0.05}{0.5} = -0.1.$$

像素：

$$u = \frac{0.1 + 1}{2} \cdot 8 = 4.4 \Rightarrow u = 4,$$

$$v = \frac{1 - (-0.1)}{2} \cdot 8 = 4.4 \Rightarrow v = 4.$$

位元組位移為 $(4 \cdot 8 + 4) \cdot 3 = 36 \cdot 3 = 108$ 。注意 $y_{cam} < 0$ 導致 v 大於影像中央（即偏下），符合影像 v 向下、世界 Y 向上的分離。

習題 2 解答 修改如下：

```
def checker(u, v):
    return (255, 255, 255) if (u + v) % 2 == 0 else (0, 0, 0)

def self_check2(path="checker16.ppm"):
    W = H = 16
    body = make_rgb16(W, H, fn=checker)
    total = write_ppm(path, W, H, body)
    assert total == 13 + W * H * 3
    assert body[0:3] == bytes((255, 255, 255))      # (0,0) 白
    assert body[3:6] == bytes((0, 0, 0))            # (0,1) 黑
```

```
assert body[index_of(2, 3, W):index_of(2, 3, W)+3] == bytes((0, 0, 0)) # 2+3=5 奇→黑
return total
```

三個斷言分別檢查左上角、右鄰像素與一個非邊界位置，足以捕捉索引方向與奇偶規則的錯誤。

習題 3 解答 令 $(u \cdot 16 + v) \cdot 3 = (v \cdot 16 + u) \cdot 3$ ，兩邊同除 3 並整理： $15u = 15v$ ，即 $u = v$ 。所以在對角線 $u = v$ 上（例如 $(5, 5)$ ）兩種寫法結果相同。但對 $(u, v) = (1, 0)$ ，PPM 正確位移為 3，錯誤版為 $(1 \cdot 16 + 0) \cdot 3 = 48$ ，指向索引 16 的像素（ $v = 1, u = 0$ ）的紅色分量，完全不同的位置。只要 $u \neq v$ 就會出錯；維度不是正方形時連對角線都不會對齊，錯得更明顯。這正是「行列優先」與「欄優先」混淆的典型症狀。

習題 4 解答（流程圖文字版） 假設池體六面已在物件空間以公尺為單位，水面反照率為線性 RGB 三通道值 $\rho \in [0, 1]^3$ 。

1. 模型矩陣 M ：物件 $\rightarrow$ 世界，方向/平移，無旋轉（此例）。單位公尺，齊次 4×4 。
2. 視圖矩陣 V ：世界 $\rightarrow$ 相機，由 look-at 決定。單位公尺。
3. 投影矩陣 P ：相機 $\rightarrow$ 裁切空間，由 fov 、 $aspect$ 、 $near$ 、 far 決定。無單位。
4. 像素映射： $x_{ndc}, y_{ndc} \rightarrow u, v$ ，中間量為 NDC，最後得到離散整數索引。
5. 量化：線性 RGB 反照率乘以簡化光照項 E （單位 $W \cdot m^{-2}$ 縮放後），夾到 $[0, 1]$ ，乘 255 並四捨五入。**線性 RGB 只在這個階段之前使用；夾取與量化之後即為顯示值。**
6. 寫檔：PPM P6，顯式宣告 255，資料列優先。

這份流程圖的每個階段都有明確的輸入與輸出型別；後續章節會逐一補上矩陣的精確定義。

本章小結

本章建立了本卷的四條骨架。第一，**管線是固定的**： PVM 與透視除法不可任意交換或插入步驟。第二，**離散與連續要分清**：像素索引 (u, v) 是整數，像素中心 $(u + 0.5, v + 0.5)$ 是浮點，兩者不能混用。第三，**色彩與資料要分清**：線性 RGB 用於計算，sRGB 用於輸出，normal/depth 不套 sRGB。第四，**合成資料有邊界**：它能驗證管線自治，不能代替實測與生物判斷。

同時， 16×16 PPM 這個極小的實驗，已經把「取樣—量化—寫檔」的完整鏈路走過一遍。後面所有幾何、光學、動畫、標註，都只是把這條鏈路換成更複雜的輸入而已。

參考來源

- [G1] PBRT 4：Transformations，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G2] PBRT 4：Reflection Models，https://pbr-book.org/4ed/Reflection_Models
- [G3] PBRT 4：The Light Transport Equation，https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- [G4] Ray Tracing in One Weekend，<https://raytracing.github.io/books/RayTracingInOneWeekend.html>

- [G5] LearnOpenGL：Transformations，<https://learnopengl.com/Getting-started/Transformations>
- [G6] Blender Manual：Skinning Introduction，<https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>
- [G7] NumPy 線性代數參考，<https://numpy.org/doc/stable/reference/routines.linalg.html>
- [G8] Khronos glTF 2.0 規格，<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

以上連結僅作為讀者延伸查閱的入口；本章的推導與數值皆為獨立撰寫，未逐條對應來源論點，也未聲稱已跑過官方範例。Blender 與 GLSL 在本卷為可選橋接，不宣稱特定版本 UI 或硬體已測。

第 2 章 向量、外積與幾何判定

第一部 | 幾何、座標與成像

學習目標與先備知識

本章把 Volume I 的向量與內積知識，轉化為可直接用於圖學管線的幾何工具。完成本章後，讀者應能：

1. 用內積計算長度、夾角、投影及判斷方向關係。
2. 用外積計算三角形面積、表面法線與頂點繞序。
3. 從三個非共線點建立平面方程。
4. 計算點到平面的帶號距離與無號距離。
5. 辨認零長向量、退化三角形及近退化幾何。
6. 區分長度、面積與無因次容差。
7. 實作並測試 `dot`、`cross`、`normalize` 與點到平面距離。

本書採右手世界座標系： $+X$ 向右、 $+Y$ 向上、 $+Z$ 由畫面指向觀者。數學向量皆為 column vector。位置與位移的長度單位為公尺，面積單位為平方公尺。

本程式使用 Python 3.10+ 與 NumPy 2.2.6 相容寫法。程式僅供讀者自行執行；列出的輸出均標為「預期」，不宣稱已在特定平台執行。

問題與直覺

養殖場數位分身不只是把池體與魚畫成彩色像素。幾何處理與渲染程式會反覆回答：

- 池壁朝向池內，還是池外？
- 光線從表面的正面還是背面射入？
- 某位置位於水面上方還是下方？
- 三個頂點是否真的形成三角形？
- 魚鰭上的兩條邊幾乎平行時，法線是否可靠？
- 相鄰網格面是否使用一致的頂點繞序？

這些問題的共同核心是向量。內積把兩個向量縮成純量，可回答「有多同向」；外積把兩個三維向量轉成垂直方向，可回答「它們張成的平面朝哪裡」以及「張成多大面積」。

理想數學與浮點運算之間仍有差距。理論上，平行向量的外積等於零；實際模型卻可能含有量化誤差、程序化建模誤差或浮點捨入。另一方面，任意寫死 `1e-6` 也不安全：若拿帶長度單位的門檻去比較面積，量綱便不一致；同一個門檻用在毫米級魚鰭與百公尺池體，也可能得到完全不同的判定。

因此，本章會區分：

- 精確幾何條件與浮點近似判定；
 - 絕對容差與相對容差；
 - 長度量、面積量與無因次量；
 - 「數學上不退化」與「工程上品質足夠」兩種要求。
-

數學與幾何推導

1. 向量、點與位移

令三維向量為

$$\mathbf{a} = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix}.$$

若 $\mathbf{p}$ 、 $\mathbf{q}$ 是位置，則

$$\mathbf{v} = \mathbf{q} - \mathbf{p}$$

是從 $\mathbf{p}$ 指向 $\mathbf{q}$ 的位移。位置可以相減得到位移，位置加位移則得到另一位置。

向量長度為

$$\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2}.$$

若座標以公尺表示，位移與其長度的單位都是公尺。長度具有非負性：

$$\|\mathbf{a}\| \geq 0,$$

而且只有零向量才滿足 $\|\mathbf{a}\| = 0$ 。

2. 內積

內積定義為

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z.$$

其幾何形式是

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta,$$

其中 $\theta \in [0, \pi]$ 是兩向量夾角。兩向量皆非零時，

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

因此：

- $\mathbf{a} \cdot \mathbf{b} > 0$ ：夾角小於 90° ；
- $\mathbf{a} \cdot \mathbf{b} = 0$ ：理論上互相垂直；
- $\mathbf{a} \cdot \mathbf{b} < 0$ ：夾角大於 90° 。

內積也給出長度平方：

$$\|\mathbf{a}\|^2 = \mathbf{a} \cdot \mathbf{a}.$$

若 $\hat{\mathbf{b}}$ 是單位向量， $\mathbf{a}$ 沿其方向的純量投影為

$$s = \mathbf{a} \cdot \hat{\mathbf{b}},$$

向量投影為

$$\text{proj}_{\hat{\mathbf{b}}}(\mathbf{a}) = (\mathbf{a} \cdot \hat{\mathbf{b}})\hat{\mathbf{b}}.$$

與 $\hat{\mathbf{b}}$ 垂直的剩餘分量是

$$\mathbf{a}_\perp = \mathbf{a} - (\mathbf{a} \cdot \hat{\mathbf{b}})\hat{\mathbf{b}}.$$

若 $\mathbf{a}$ 、 $\mathbf{b}$ 都是以公尺表示的位移，內積單位為平方公尺。只有兩者都是無因次單位方向時，內積才可直接解讀為 $\cos \theta$ 。

3. 正規化與零長向量

非零向量的正規化為

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|}.$$

結果滿足 $\|\hat{\mathbf{a}}\| = 1$ 。若 $\mathbf{a} = \mathbf{0}$ ，方向不存在，不能正規化。把零向量直接替換成某個固定方向，可能掩蓋重複頂點、退化面或錯誤輸入。

核心幾何函式通常應採取以下策略之一：

1. 拒絕零長或過短向量；
2. 回傳明確的失敗狀態；
3. 由呼叫端依應用語意提供備援方向。

本程式採第一種策略。預設門檻只是示範值；正式場景應依資料尺度傳入 `eps_length`，不可把 `1e-12` 視為通用幾何標準。

4. 三維外積

外積定義為

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}.$$

它具有反交換性：

$$\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a}),$$

且同時垂直於兩個輸入：

$$\mathbf{a} \cdot (\mathbf{a} \times \mathbf{b}) = 0, \quad \mathbf{b} \cdot (\mathbf{a} \times \mathbf{b}) = 0.$$

方向由右手定則決定：右手手指由 $\mathbf{a}$ 捲向 $\mathbf{b}$ ，拇指方向即為 $\mathbf{a} \times \mathbf{b}$ 。

外積長度為

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \|\mathbf{b}\| |\sin \theta|.$$

這是兩向量張成之平行四邊形的面積。若兩向量單位為公尺，外積各分量及其長度的單位皆為平方公尺。

5. 三角形面積、法線與繞序

令三角形頂點為 $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ ，定義

$$\mathbf{e}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{e}_2 = \mathbf{p}_2 - \mathbf{p}_0.$$

未正規化法線為

$$\mathbf{n}_{\text{raw}} = \mathbf{e}_1 \times \mathbf{e}_2.$$

其長度是三角形雙倍面積，因此

$$A_{\triangle} = \frac{1}{2} \|\mathbf{n}_{\text{raw}}\|.$$

若 $\mathbf{n}_{\text{raw}} \neq \mathbf{0}$ ，單位法線為

$$\hat{\mathbf{n}} = \frac{\mathbf{n}_{\text{raw}}}{\|\mathbf{n}_{\text{raw}}\|}.$$

交換 $\mathbf{p}_1$ 、 $\mathbf{p}_2$ 會反轉法線：

$$(\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_1 - \mathbf{p}_0) = -\mathbf{n}_{\text{raw}}.$$

依本書約定，從外向法線方向觀察時，三角形前面為逆時針。外積只依頂點順序產生方向，不知道何處是池內、池外或物體表面；這些語意必須由模型約定提供。

6. 退化與近退化三角形

精確數學中，若

$$\|\mathbf{e}_1 \times \mathbf{e}_2\| = 0,$$

則三點共線，或至少有兩點重合，三角形面積為零。此時不存在唯一表面法線。

浮點運算中還要辨認近退化狀態。一種尺度相對判定是

$$\|\mathbf{e}_1 \times \mathbf{e}_2\| \leq \tau_{\text{rel}} \|\mathbf{e}_1\| \|\mathbf{e}_2\|,$$

其中 $\tau_{\text{rel}} \geq 0$ 是無因次門檻。當兩邊皆非零時，比例

$$\rho = \frac{\|\mathbf{e}_1 \times \mathbf{e}_2\|}{\|\mathbf{e}_1\| \|\mathbf{e}_2\|} = |\sin \theta|$$

描述兩邊有多接近平行。 ρ 越小，兩邊越接近平行；在邊長與頂點誤差尺度相近的條件下，法線方向通常較不穩定，但這不是完整的條件數分析。

程式還應另設長度門檻 ε_L ，拒絕

$$\|\mathbf{e}_1\| \leq \varepsilon_L \quad \text{或} \quad \|\mathbf{e}_2\| \leq \varepsilon_L.$$

ε_L 與座標具有相同單位。相對門檻不能取代長度門檻：一個邊長僅 10^{-15} 公尺、形狀卻接近直角的三角形，角度品質可能良好，但對公尺級場景仍可能小到沒有工程意義。

即使使用者令 $\tau_{\text{rel}} = 0$ ，程式仍必須拒絕精確零面積，否則後續會除以零。零相對容差表示「不額外拒絕非零的近共線三角形」，而不是允許零法線。

7. 絕對面積容差與尺度選擇

相對容差只描述形狀，不描述三角形的絕對大小。有些流程還需要絕對面積門檻 ε_A ：

$$A_{\Delta} \leq \varepsilon_A.$$

其中 ε_A 的單位是平方公尺。它適用於：

- 匯入模型時排除小於製作解析度的碎片面；
- 避免極小面造成不穩定的法線加權；
- 依合成場景的最小可見特徵清理資料。

三種門檻解決不同問題：

門檻	單位	用途
ε_L	m	拒絕過短邊或重合頂點
ε_A	m ²	拒絕絕對面積太小的面
τ_{rel}	無因次	拒絕形狀過度狹長、近共線的面

例如同樣是直角等腰三角形，邊長 1 公尺與 10^{-5} 公尺的 ρ 都是 1，但後者可能低於模型製作解析度。反之，一個面積不小但長寬比極端的三角形，可能通過 ε_A ，卻因 ρ 太小而不適合後續計算。

若模型尺度約為 L_{scene} ，可先根據資料來源決定最小有意義長度 L_{min} ，再令 ε_L 接近該尺度，而非單純取機器精度。面積門檻可依最小面片需求設定；相對門檻則依演算法對狹長面的容忍程度設定。這些都是幾何品質規則，不是養殖設備的物理安全閾值。

8. 平面方程

通過點 $\mathbf{p}_0$ 、法線為非零向量 $\mathbf{n}$ 的平面滿足

$$\mathbf{n} \cdot (\mathbf{x} - \mathbf{p}_0) = 0.$$

展開可得

$$\mathbf{n} \cdot \mathbf{x} + d = 0, \qquad d = -\mathbf{n} \cdot \mathbf{p}_0.$$

若 $\mathbf{n} = (a, b, c)^T$ ，則

$$ax + by + cz + d = 0.$$

同一平面的四個係數可同乘任何非零常數，因此表示並不唯一。若 $\mathbf{n}$ 是單位法線，係數具有較直接的距離意義。

三個點可建立平面的必要條件是它們不共線。計算順序為：

1. 建立 $\mathbf{e}_1$ 、 $\mathbf{e}_2$ ；
2. 檢查短邊與退化；
3. 計算並正規化外積；
4. 令 $d = -\hat{\mathbf{n}} \cdot \mathbf{p}_0$ 。

若退化檢查失敗，不能用任意備援法線假裝三點定義了平面。

9. 點到平面的距離與半空間判定

令查詢點為 $\mathbf{q}$ 。若 $\hat{\mathbf{n}}$ 是單位法線，帶號距離為

$$s = \hat{\mathbf{n}} \cdot (\mathbf{q} - \mathbf{p}_0).$$

- $s > 0$ ：位於法線所指一側；
- $s = 0$ ：位於平面上；
- $s < 0$ ：位於相反一側。

無號距離為

$$D = |s|.$$

若使用未正規化法線，則必須除以其長度：

$$s = \frac{\mathbf{n} \cdot (\mathbf{q} - \mathbf{p}_0)}{\|\mathbf{n}\|}.$$

點在平面上的正交投影為

$$\mathbf{q}_{\text{proj}} = \mathbf{q} - s\hat{\mathbf{n}}.$$

浮點數中不宜直接用 `s == 0.0` 判斷「位於平面」。若應用允許距離誤差 ε_D ，可分類為

$$\begin{cases} s > \varepsilon_D & \text{正側,} \\ s < -\varepsilon_D & \text{負側,} \\ |s| \leq \varepsilon_D & \text{邊界帶.} \end{cases}$$

ε_D 是長度量。邊界帶能避免一個幾乎在平面上的點因微小捨入誤差在正負兩側反覆切換。

逐步手算例題

例題一：池底三角形的面積與法線

某池底局部三角形頂點為

$$\mathbf{p}_0 = (0, 0, 0)^T, \quad \mathbf{p}_1 = (2, 0, 0)^T, \quad \mathbf{p}_2 = (0, 0, 3)^T,$$

單位為公尺。兩條邊是

$$\mathbf{e}_1 = (2, 0, 0)^T, \quad \mathbf{e}_2 = (0, 0, 3)^T.$$

外積為

$$\mathbf{e}_1 \times \mathbf{e}_2 = \begin{pmatrix} 0 \\ -6 \\ 0 \end{pmatrix} = (0, -6, 0)^T \text{ m}^2.$$

所以面積為

$$A_{\Delta} = \frac{1}{2} \sqrt{(-6)^2} = 3 \text{ m}^2.$$

單位法線是

$$\hat{\mathbf{n}} = (0, -1, 0)^T.$$

此法線朝下。若池底外向法線應朝上，須交換後兩點，改用順序 $(\mathbf{p}_0, \mathbf{p}_2, \mathbf{p}_1)$ 。面積不受繞序影響，但法線方向會反轉。

例題二：點到斜平面的距離

平面通過

$$\mathbf{p}_0 = (0, 1, 0)^T$$

且法線為

$$\mathbf{n} = (0, 2, 2)^T.$$

查詢點是

$$\mathbf{q} = (0, 4, 1)^T.$$

法線長度為

$$\|\mathbf{n}\| = 2\sqrt{2},$$

故單位法線為

$$\hat{\mathbf{n}} = \left(0, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)^T.$$

位移是

$$\mathbf{q} - \mathbf{p}_0 = (0, 3, 1)^T.$$

帶號距離為

$$s = \hat{\mathbf{n}} \cdot (\mathbf{q} - \mathbf{p}_0) = \frac{3}{\sqrt{2}} + \frac{1}{\sqrt{2}} = 2\sqrt{2} \text{ m}.$$

因 $s > 0$ ，查詢點位於法線所指一側。投影點為

$$\mathbf{q}_{\text{proj}} = \mathbf{q} - s\hat{\mathbf{n}} = (0, 4, 1) - (0, 2, 2) = (0, 2, -1)^T.$$

檢查：

$$\mathbf{n} \cdot (\mathbf{q}_{\text{proj}} - \mathbf{p}_0) = (0, 2, 2) \cdot (0, 1, -1) = 0.$$

所以投影點確實位於平面上。

例題三：形狀容差與絕對面積

令

$$\mathbf{e}_1 = (1000, 0, 0)^T, \quad \mathbf{e}_2 = (1000, 0.001, 0)^T.$$

外積為

$$\mathbf{e}_1 \times \mathbf{e}_2 = (0, 0, 1)^T,$$

雙倍面積為 1 m^2 ，三角形面積為 0.5 m^2 。其形狀比例為

$$\rho = \frac{1}{1000\sqrt{1000^2 + 0.001^2}}.$$

因

$$\sqrt{1000^2 + 0.001^2} = \sqrt{1000000.000001} \approx 1000.0000000005,$$

所以

$$\rho \approx 9.99999999995 \times 10^{-7}.$$

也就是說， ρ 的量級約為 10^{-6} ，但並非精確等於 10^{-6} 。面積並不極小，形狀卻非常狹長。若 $\tau_{\text{rel}} = 10^{-5}$ ，它會被判為近退化；若 $\tau_{\text{rel}} = 10^{-8}$ ，則通過形狀檢查。

相反地，邊向量

$$\mathbf{u} = (10^{-5}, 0, 0)^T, \quad \mathbf{v} = (0, 10^{-5}, 0)^T$$

形成完好的直角，但面積僅

$$A = \frac{1}{2} \times 10^{-10} = 5 \times 10^{-11} \text{ m}^2.$$

它的 $\rho = 1$ ，不近共線；是否接受，應由長度或絕對面積規則決定。這兩例說明相對與絕對門檻不可互相取代。

實作與程式

以下程式完整定義本章函式。輸入統一轉成 `float64`、`shape (3,)` 的 NumPy 陣列。

`triangle_geometry` 先明確拒絕零面積，再套用相對門檻，因此即使 `rel_sine_tol=0`，也不會接受零法線或發生除以零。

```
import numpy as np

def vec3(value, name="vector"):
    """轉成 shape (3,) 的有限 float64 向量。"""
    result = np.asarray(value, dtype=np.float64)
    if result.shape != (3,):
        raise ValueError(
            f"{name} 必須是 shape (3,)，目前為 {result.shape}"
        )
    if not np.all(np.isfinite(result)):
        raise ValueError(f"{name} 含有 NaN 或無限值")
    return result

def dot(a, b):
    a = vec3(a, "a")
    b = vec3(b, "b")
    return float(np.dot(a, b))

def cross(a, b):
    a = vec3(a, "a")
    b = vec3(b, "b")
    return np.cross(a, b)

def length(v):
    v = vec3(v, "v")
    return float(np.linalg.norm(v))

def normalize(v, eps_length=1e-12):
    """
    正規化三維向量。
    """
```

```

eps_length 與 v 的單位相同；若 v 是無因次方向，
eps_length 也無因次。正式應用應依資料尺度設定。
"""
v = vec3(v, "v")
if not np.isfinite(eps_length) or eps_length < 0.0:
    raise ValueError("eps_length 必須是有限非負數")

magnitude = length(v)
if magnitude <= eps_length:
    raise ValueError("無法正規化零長或過短向量")
return v / magnitude

def triangle_geometry(
    p0,
    p1,
    p2,
    eps_length=1e-12,
    rel_sine_tol=1e-10,
    eps_area=0.0,
):
    """
    回傳三角形面積、單位法線、未正規化法線與形狀比例。

    eps_length：長度門檻，單位與座標相同。
    rel_sine_tol：無因次近共線門檻，可為 0。
    eps_area：面積門檻，單位為座標單位的平方，可為 0。
    """
    p0 = vec3(p0, "p0")
    p1 = vec3(p1, "p1")
    p2 = vec3(p2, "p2")

    tolerances = (eps_length, rel_sine_tol, eps_area)
    if not all(np.isfinite(x) and x >= 0.0 for x in tolerances):
        raise ValueError("所有容差必須是有限非負數")

    e1 = p1 - p0
    e2 = p2 - p0
    l1 = length(e1)
    l2 = length(e2)

    if l1 <= eps_length or l2 <= eps_length:
        raise ValueError("三角形含有重合或過近的頂點")

    raw_normal = cross(e1, e2)
    double_area = length(raw_normal)

    # 即使 rel_sine_tol == 0，也必須拒絕精確零面積。
    if double_area == 0.0:
        raise ValueError("三角形精確退化，無法定義法線")

    sine_ratio = double_area / (l1 * l2)
    if sine_ratio <= rel_sine_tol:
        raise ValueError("三角形近共線，法線不可靠")

    area = 0.5 * double_area
    if area <= eps_area:
        raise ValueError("三角形面積低於絕對面積門檻")

```



```

    return {
        "area": area,
        "normal": raw_normal / double_area,
        "raw_normal": raw_normal,
        "sine_ratio": sine_ratio,
    }

def signed_point_plane_distance(
    point,
    plane_point,
    plane_normal,
    eps_normal=1e-12,
):
    """
    計算點到平面的帶號距離。
    point 與 plane_point 若以公尺表示，結果也是公尺。
    """
    point = vec3(point, "point")
    plane_point = vec3(plane_point, "plane_point")
    unit_normal = normalize(plane_normal, eps_normal)
    return dot(unit_normal, point - plane_point)

def classify_point_to_plane(
    point,
    plane_point,
    plane_normal,
    eps_distance,
    eps_normal=1e-12,
):
    """回傳 'positive'、'negative' 或 'boundary'。

    eps_distance 與座標同單位；eps_normal 與 plane_normal 同單位。
    正式應用應依資料尺度設定這兩個門檻，不要依賴預設值。
    """
    if not np.isfinite(eps_distance) or eps_distance < 0.0:
        raise ValueError("eps_distance 必須是有限非負數")

    distance = signed_point_plane_distance(
        point, plane_point, plane_normal, eps_normal=eps_normal
    )
    if distance > eps_distance:
        return "positive"
    if distance < -eps_distance:
        return "negative"
    return "boundary"

def project_point_to_plane(
    point,
    plane_point,
    plane_normal,
    eps_normal=1e-12,
):
    point = vec3(point, "point")
    plane_point = vec3(plane_point, "plane_point")
    unit_normal = normalize(plane_normal, eps_normal)

```

```

distance = dot(unit_normal, point - plane_point)
return point - distance * unit_normal

def main():
    print("dot =", dot([1, 2, 3], [4, -1, 2]))
    print("cross =", cross([1, 0, 0], [0, 1, 0]))
    print("normalize =", normalize([0, 3, 4]))

    tri = triangle_geometry(
        [0, 0, 0],
        [2, 0, 0],
        [0, 0, 3],
    )
    print("area =", tri["area"])
    print("normal =", tri["normal"])
    print("sine ratio =", tri["sine_ratio"])

    distance = signed_point_plane_distance(
        point=[0, 4, 1],
        plane_point=[0, 1, 0],
        plane_normal=[0, 2, 2],
    )
    print("signed distance =", distance)

    projected = project_point_to_plane(
        point=[0, 4, 1],
        plane_point=[0, 1, 0],
        plane_normal=[0, 2, 2],
    )
    print("projected =", projected)

    for label, action in [
        ("zero vector", lambda: normalize([0, 0, 0])),
        (
            "degenerate triangle",
            lambda: triangle_geometry(
                [0, 0, 0],
                [1, 0, 0],
                [2, 0, 0],
                rel_sine_tol=0.0,
            ),
        ),
    ]:
        try:
            action()
        except ValueError as error:
            print(label + ":", error)

if __name__ == "__main__":
    main()

```

這裡使用 `double_area == 0.0` 並不是拿精確比較取代容差，而是建立不可跨越的除零防線。非零但品質差的三角形仍由 `rel_sine_tol`、`eps_length` 與 `eps_area` 分別處理。

這份入門實作假設座標尺度適中，使 `np.linalg.norm`、外積及 `l1 * l2` 不會溢位或下溢。若座標接近浮點格式的極端範圍，單靠 `epsilon` 並不足以保證穩健；應先將局部幾何縮放到合理範圍，或採用經尺度化的範數與謂詞演算法。公尺級池體與一般合成模型不應故意使用接近 `float64` 上下限的座標。

測試與預期結果

上列程式的**預期**重點如下；浮點顯示格式可能略有差異。

```
dot = 8.0
cross = [0. 0. 1.]
normalize = [0. 0.6 0.8]
area = 3.0
normal = [ 0. -1.  0.]
sine ratio = 1.0
signed distance = 2.828427124746...
projected = [ 0.  2. -1.]
zero vector: 無法正規化零長或過短向量
degenerate triangle: 三角形精確退化，無法定義法線
```

可加入以下斷言。比較容差明確寫出，以免把 NumPy 預設值誤認為場景規格。預期會失敗的呼叫採 `try`、`except`、`else` 結構：只有收到指定的 `ValueError` 才算通過；沒有拋出例外便在 `else` 中明確使測試失敗。

```
def run_assertions():
    rtol = 1e-12
    atol = 1e-12

    assert np.isclose(
        dot([1, 2, 3], [4, -1, 2]),
        8.0,
        rtol=rtol,
        atol=atol,
    )

    assert np.allclose(
        cross([1, 0, 0], [0, 1, 0]),
        [0, 0, 1],
        rtol=rtol,
        atol=atol,
    )

    assert np.allclose(
        normalize([0, 3, 4]),
        [0, 0.6, 0.8],
        rtol=rtol,
        atol=atol,
    )

    tri = triangle_geometry(
        [0, 0, 0], [2, 0, 0], [0, 0, 3]
    )
    assert np.isclose(
```

```

        tri["area"], 3.0, rtol=rtol, atol=atol
    )
    assert np.allclose(
        tri["normal"], [0, -1, 0],
        rtol=rtol, atol=atol
    )

    d = signed_point_plane_distance(
        [0, 4, 1], [0, 1, 0], [0, 2, 2]
    )
    assert np.isclose(
        d, 2.0 * np.sqrt(2.0),
        rtol=rtol, atol=atol
    )

    try:
        normalize([0, 0, 0])
    except ValueError:
        pass
    else:
        raise AssertionError("零向量測試應該失敗")

# 零相對容差仍不得接受精確共線三角形。
try:
    triangle_geometry(
        [0, 0, 0],
        [1, 0, 0],
        [2, 0, 0],
        rel_sine_tol=0.0,
    )
except ValueError:
    pass
else:
    raise AssertionError("精確共線三角形應該失敗")

# 錯誤維度必須被拒絕。
try:
    dot([1, 2, 3], [1, 2, 3])
except ValueError:
    pass
else:
    raise AssertionError("shape (1, 3) 應該失敗")

# 交換繞序只反轉法線，不改變面積。
reversed_tri = triangle_geometry(
    [0, 0, 0], [0, 0, 3], [2, 0, 0]
)
assert np.isclose(
    reversed_tri["area"], tri["area"],
    rtol=rtol, atol=atol
)
assert np.allclose(
    reversed_tri["normal"], -tri["normal"],
    rtol=rtol, atol=atol
)

# 法線縮放不應改變幾何距離。
d1 = signed_point_plane_distance(
    [0, 3, 0], [0, 1, 0], [0, 1, 0]

```

```
)
d2 = signed_point_plane_distance(
    [0, 3, 0], [0, 1, 0], [0, 10, 0]
)
assert np.isclose(d1, d2, rtol=rtol, atol=atol)
```

測試至少應涵蓋正常案例、符號反轉、零長向量、精確退化、近退化及容差邊界。只測一個成功案例，無法證明錯誤輸入會被安全拒絕。

除錯與常見陷阱

1. 外積順序反了

`cross(e1, e2)` 與 `cross(e2, e1)` 大小相同、方向相反。若整個池壁朝內，應優先檢查頂點繞序與外積順序。

2. 把未正規化法線直接當距離

下式通常不是幾何距離：

$$\mathbf{n} \cdot (\mathbf{q} - \mathbf{p}_0).$$

除非 $\|\mathbf{n}\| = 1$ ，否則必須除以 $\|\mathbf{n}\|$ 。法線放大十倍時，真實距離不應跟著變成十倍。

3. 對零向量正規化

重複頂點、共線三角形及相同位置相減都可能產生零向量。不要讓 `NaN` 流入後續著色、裁切或求交；應在幾何資料進入流程時拒絕。

4. 把零容差理解成允許退化

`rel_sine_tol=0` 只能表示不拒絕非零的近共線三角形。精確零面積仍然沒有法線，必須單獨拒絕。否則 `raw_normal / double_area` 會除以零。

5. 容差單位不一致

`length(e1)` 是長度，`length(cross(e1, e2))` 是面積，不能直接與同一個帶單位常數比較。應分別使用長度、面積與無因次形狀門檻。

6. 只檢查從同一頂點出發的兩條邊

若只檢查 $\|\mathbf{p}_1 - \mathbf{p}_0\|$ 與 $\|\mathbf{p}_2 - \mathbf{p}_0\|$ ，第三條邊 $\|\mathbf{p}_2 - \mathbf{p}_1\|$ 仍可能非常短。外積通常會揭露面積問題，但若應用要完整診斷重複頂點，應檢查三條邊並回報具體頂點對。

7. 直接用 $\arccos$ 判角度

浮點誤差可能使餘弦略超出 $[-1, 1]$ 。若確實需要角度，先做：

```
cos_theta = np.clip(cos_theta, -1.0, 1.0)
theta = np.arccos(cos_theta)
```

若只需判斷正面、背面或近垂直，通常直接比較內積即可。

8. 混淆位置與方向

平面法線與邊向量是方向，不應受平移影響。後續齊次座標會把位置寫成 $w = 1$ ，方向寫成 $w = 0$ ；本章的幾何語意應先保持清楚。

9. 修正法線卻不修正網格繞序

若只將法線乘以 -1 ，但不反轉三角形索引，後續背面剔除、陰影與相鄰面拓撲仍可能互相矛盾。資料問題應優先在網格層修正，不能只改顯示結果。

10. 將容差當成浮點穩健性的完整解法

合理容差能表達應用接受範圍，但不能防止所有溢位、下溢或消去誤差。例如極大向量的外積可能先溢位，程式尚未比較容差便已得到無限值。實務上應採合理單位與局部座標，避免把模型放在遠超場景需求的數值範圍。

養殖數位分身案例

考慮用兩個三角形表示矩形水面，高度為 $y = 1.5$ 公尺，希望法線朝上：

$$\hat{\mathbf{n}} = (0, 1, 0)^T.$$

四個角點為

$$\mathbf{p}_{00} = (-2, 1.5, -3)^T, \quad \mathbf{p}_{10} = (2, 1.5, -3)^T,$$

$$\mathbf{p}_{01} = (-2, 1.5, 3)^T, \quad \mathbf{p}_{11} = (2, 1.5, 3)^T.$$

第一個三角形使用

$$(\mathbf{p}_{00}, \mathbf{p}_{01}, \mathbf{p}_{10}).$$

兩條邊為

$$\mathbf{e}_1 = (0, 0, 6)^T, \quad \mathbf{e}_2 = (4, 0, 0)^T,$$

所以

$$\mathbf{e}_1 \times \mathbf{e}_2 = (0, 24, 0)^T.$$

法線朝 +Y。第二個三角形使用

$$(\mathbf{p}_{10}, \mathbf{p}_{01}, \mathbf{p}_{11}),$$

其法線也朝 +Y。每個三角形面積為 12 m^2 ，合計水面面積為 24 m^2 。

假設一條魚的合成參考位置為

$$\mathbf{f} = (0, 1.1, 0)^T.$$

以水面點與上向法線計算：

$$s = (0, 1, 0) \cdot (\mathbf{f} - \mathbf{p}_{00}) = 1.1 - 1.5 = -0.4 \text{ m}.$$

該位置位於水面下方 0.4 公尺。若分類門檻為 $\varepsilon_D = 0.01$ 公尺，則：

- $s > 0.01$ ：水面上方；
- $s < -0.01$ ：水面下方；
- $|s| \leq 0.01$ ：水面邊界帶。

網格匯入流程還可逐面執行：

1. 檢查三個索引合法且頂點有限；
2. 計算三條邊並找出重複頂點；
3. 檢查面積與形狀比例；
4. 計算法線；
5. 與預期外向方向做內積；
6. 若內積為負，反轉該面的頂點繞序；
7. 重新計算法線並留下修正紀錄。

對水平水面，可使用預期方向 $(0, 1, 0)^T$ 。若

$$\hat{\mathbf{n}}_{\text{face}} \cdot (0, 1, 0) < 0,$$

表示該面朝下。但對曲面魚體，不能假設所有法線都朝 $+Y$ ；應使用模型中心到面中心的方向、封閉網格方向規則或拓撲一致性判定。

這些結果只描述合成幾何位置與網格品質，不代表魚隻健康、水質安全或真實量測。波浪水面也不能由單一水平平面完整表示。

習題

習題 1：手算

給定

$$\mathbf{a} = (1, 2, -2)^T, \quad \mathbf{b} = (2, 0, 1)^T.$$

1. 計算 $\mathbf{a} \cdot \mathbf{b}$ 。
2. 計算 $\mathbf{a} \times \mathbf{b}$ 。
3. 驗證外積垂直於兩輸入向量。
4. 求兩向量張成之三角形面積。

習題 2：程式測試

補上測試，驗證：

1. `cross(a, b) == -cross(b, a)`；
2. `normalize([3, 0, 4])` 長度接近 1；
3. 法線由 `[0, 1, 0]` 改成 `[0, 10, 0]` 時距離不變；
4. `shape (1, 3)` 會被拒絕；
5. `rel_sine_tol=0` 時，精確共線三角形仍會被拒絕。

習題 3：反例與除錯

某程式以

```
raw = np.cross(p1 - p0, p2 - p0)
if np.linalg.norm(raw) < 1e-6:
    degenerate = True
```

判斷退化。指出至少兩個問題，並給出一個「形狀相同但整體縮放後判定改變」的反例。再說明何時應加入絕對面積門檻。

習題 4：整合應用

某垂直池壁由三點構成：

$$\mathbf{p}_0 = (2, 0, -1)^T, \quad \mathbf{p}_1 = (2, 2, -1)^T, \quad \mathbf{p}_2 = (2, 0, 3)^T.$$

1. 求面積與單位法線。
2. 求 $\mathbf{q} = (1.25, 1, 0)^T$ 到平面的帶號距離。
3. 若希望法線朝池內的 $-X$ ，目前繞序是否正確？
4. 求 $\mathbf{q}$ 的平面投影。

習題 5：容差設計

某合成場景以公尺為單位，最短有意義邊長為 0.5 毫米，最小保留面積為 0.2 平方毫米。請把這兩個值換成公尺與平方公尺，並寫出適合傳給 `triangle_geometry` 的 `eps_length`、`eps_area`。說明為何仍需另選 `rel_sine_tol`。

習題解答

習題 1 解答

內積：

$$\mathbf{a} \cdot \mathbf{b} = 1(2) + 2(0) + (-2)(1) = 0.$$

外積：

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} 2 \\ -5 \\ -4 \end{pmatrix}.$$

驗證：

$$\mathbf{a} \cdot (\mathbf{a} \times \mathbf{b}) = 2 - 10 + 8 = 0,$$

$$\mathbf{b} \cdot (\mathbf{a} \times \mathbf{b}) = 4 - 4 = 0.$$

外積長度為

$$\sqrt{2^2 + (-5)^2 + (-4)^2} = 3\sqrt{5}.$$

三角形面積為

$$A_{\triangle} = \frac{3\sqrt{5}}{2}.$$

習題 2 解答

以下寫法不依賴額外測試框架。預期失敗的函式呼叫放在 `try` 區塊中；若沒有收到 `ValueError`，便由 `else` 明確使測試失敗。

```
def exercise_tests():
    a = np.array([1.0, 2.0, 3.0])
    b = np.array([-2.0, 1.0, 4.0])

    assert np.allclose(cross(a, b), -cross(b, a))

    unit = normalize([3, 0, 4])
    assert np.isclose(length(unit), 1.0)

    d1 = signed_point_plane_distance(
        [0, 3, 0], [0, 1, 0], [0, 1, 0]
    )
    d2 = signed_point_plane_distance(
        [0, 3, 0], [0, 1, 0], [0, 10, 0]
    )
    assert np.isclose(d1, 2.0)
    assert np.isclose(d1, d2)

    try:
        dot([[1, 2, 3]], [1, 2, 3])
    except ValueError:
        pass
    else:
        raise AssertionError("shape (1, 3) 應被拒絕")

    try:
        triangle_geometry(
            [0, 0, 0],
            [1, 0, 0],
            [2, 0, 0],
            rel_sine_tol=0.0,
        )
    except ValueError:
        pass
    else:
        raise AssertionError("共線三角形應被拒絕")
```

最後一項是容差邊界測試：零相對門檻仍不能使零面積法線合法化。

習題 3 解答

第一，`1e-6` 的單位不明。外積長度是雙倍面積；若座標為公尺，其單位是平方公尺。

第二，絕對面積門檻隨縮放改變判定。令

$$\mathbf{p}_0 = (0, 0, 0), \quad \mathbf{p}_1 = (1, 0, 0), \quad \mathbf{p}_2 = (0, 1, 0).$$

雙倍面積為 1。將全部座標乘上 10^{-4} 後，形狀仍為直角等腰三角形，但雙倍面積變成 10^{-8} ，會被原程式判為退化。

改進方式是：

1. 用 `eps_length` 檢查短邊；
2. 用無因次比例

$$\frac{\|\mathbf{e}_1 \times \mathbf{e}_2\|}{\|\mathbf{e}_1\| \|\mathbf{e}_2\|}$$

檢查近共線；3. 先明確拒絕零面積，避免除以零；4. 若應用確實不保留極小碎片面，再另設帶面積單位的 `eps_area`。

絕對面積門檻適合表示製作解析度或資料清理需求，但不能單獨代表形狀品質。

習題 4 解答

兩條邊為

$$\mathbf{e}_1 = (0, 2, 0)^T, \quad \mathbf{e}_2 = (0, 0, 4)^T.$$

外積：

$$\mathbf{e}_1 \times \mathbf{e}_2 = (8, 0, 0)^T.$$

所以

$$A_{\triangle} = 4 \text{ m}^2, \quad \hat{\mathbf{n}} = (1, 0, 0)^T.$$

查詢點位移為

$$\mathbf{q} - \mathbf{p}_0 = (-0.75, 1, 1)^T.$$

帶號距離：

$$s = (1, 0, 0) \cdot (-0.75, 1, 1) = -0.75 \text{ m}.$$

目前法線朝 $+X$ ，不符合所需的 $-X$ ，應交換 $\mathbf{p}_1$ 、 $\mathbf{p}_2$ 。

投影點為

$$\mathbf{q}_{\text{proj}} = \mathbf{q} - s\hat{\mathbf{n}} = (1.25, 1, 0) - (-0.75)(1, 0, 0) = (2, 1, 0)^T.$$

習題 5 解答

因

$$1 \text{ mm} = 10^{-3} \text{ m},$$

所以

$$0.5 \text{ mm} = 5 \times 10^{-4} \text{ m}.$$

又因

$$1 \text{ mm}^2 = 10^{-6} \text{ m}^2,$$

所以

$$0.2 \text{ mm}^2 = 2 \times 10^{-7} \text{ m}^2.$$

可設定：

```
eps_length = 5e-4  
eps_area = 2e-7
```

邊長與面積門檻仍不能判斷三角形是否過度狹長。例如一個面積高於 `eps_area` 的細長三角形，法線仍可能對頂點誤差十分敏感，因此還要依演算法需求選擇無因次的 `rel_sine_tol`。

本章小結

內積描述向量的方向關係，可用於長度、投影、垂直與正反側判定。外積產生垂直於兩輸入向量的方向，其長度等於平行四邊形面積，因此能建立三角形面積與法線。

三角形法線依賴頂點繞序；交換兩個頂點會反轉法線。零面積三角形沒有合法法線，即使相對容差設為零也必須拒絕。近退化幾何則應分別使用長度門檻、面積門檻與無因次形狀門檻，不能用單一 `epsilon` 混合處理。

平面可寫成

$$\mathbf{n} \cdot (\mathbf{x} - \mathbf{p}_0) = 0.$$

使用單位法線後，點代入所得值就是帶號距離，也可用來建立含容差的半空間分類。這些工具將成為後續變換、網格、光柵化、法線處理與射線求交的共同基礎。

參考來源

- [G1] *Physically Based Rendering, 4th ed.* : Transformations
https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G4] *Ray Tracing in One Weekend*
<https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- [G7] NumPy 線性代數參考
<https://numpy.org/doc/stable/reference/routines.linalg.html>

以上來源供讀者回查向量、幾何與數值 API 背景；本章公式、案例與程式依本書座標、單位及錯誤處理約定整理。

第03章 2D／3D變換與齊次座標

學習目標與先備知識

本章結束後，你應能將二維及三維的位置、方向寫成齊次座標，推導平移、旋轉、縮放矩陣，解釋矩陣乘法為何不能任意換序，並實作可逆的 4×4 TRS 變換與往返測試。

先備知識為向量、矩陣乘法、內積及基本 Python。全書採**右手世界座標系**： $+X$ 向右、 $+Y$ 向上、 $+Z$ 由畫面指向觀者。角度以弧度計算，正角依右手定則旋轉；數學向量一律寫成**縱向量**，矩陣從左側作用。場景長度以公尺計，縮放係數沒有單位。

問題與直覺

設想一尾用合成網格表示的魚。建模時，魚身中心位於局部原點，魚頭指向局部 $+X$ 。把魚放進池體場景，可能要先調整長寬比例、再轉向、最後移到池中的位置。如果每次都改寫網格全部頂點，既容易弄錯，也難以還原原始模型；用一個矩陣記錄這三個動作更方便。

位置與方向卻不能完全同等對待。把魚移動三公尺，魚頭的**位置**應移動，指向魚頭的**方向**不應因平移改變。另一方面，魚先原地轉身再前進，與先前進再繞池體原點轉動，是兩種運動。齊次座標處理第一個差異；矩陣乘法的次序描述第二個差異。

數學與幾何推導

二維：為平移增加一個分量

在二維，令位置 $p = (x, y)^T$ 、位移 $t = (a, b)^T$ 。平移公式 $p' = p + t$ 不能只用一個 2×2 矩陣表示，因為任何 2×2 線性變換都把零向量映到零向量，平移卻把原點移到 t 。

把位置擴充為 $p_h = (x, y, 1)^T$ ，便可寫成：

$$T_2(a, b)p_h = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + a \\ y + b \\ 1 \end{bmatrix}.$$

若擴充的是方向 $v_h = (v_x, v_y, 0)^T$ ，相同矩陣的最後一欄乘上零，於是 $T_2 v_h = v_h$ 。這個末分量 w 標示了此處的兩種幾何用途：**位置用** $w = 1$ ，**方向用** $w = 0$ 。它不是第四個或第三個空間方向。

二維繞原點逆時針旋轉 θ ，以及沿兩軸縮放，可分別寫為：

$$R_2(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad S_2(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

旋轉矩陣的前兩個縱行 (column)，分別是原來的單位 X 、 Y 方向旋轉後的座標。這也提供檢查正負號的方法： $\theta = \pi/2$ 時， $(1, 0)$ 應轉到 $(0, 1)$ 。

本章用 $w = 1$ 表示位置，是方便組合仿射變換的選擇，並不是說所有齊次向量都只能取 0 或 1。例如 $(2x, 2y, 2)^T$ 與 $(x, y, 1)^T$ 表示同一個二維歐氏位置：當末分量非零時，將其餘分量除以 w ，即可得到歐氏座標； $w = 0$ 的方向則不能如此相除。後續透視投影會產生不固定為 1 的 w ，屆時必須在適當階段做透視除法。本章的程式刻意只建立 $w \in \{0, 1\}$ 的位置與方向，不是通用的投影座標程式。

三維：把同一規則擴充到 4×4

三維位置與方向定義為

$$p_h = (x, y, z, 1)^T, \quad v_h = (v_x, v_y, v_z, 0)^T.$$

對平移 $t = (t_x, t_y, t_z)^T$ 及無單位縮放 $s = (s_x, s_y, s_z)$ ，相應矩陣是

$$T(t) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(s) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

直接相乘可見 $T(t)p_h = (p + t, 1)^T$ ，而 $T(t)v_h = (v, 0)^T$ 。方向不受平移影響，但仍會被旋轉、縮放；縮放後的方向也未必維持單位長度。

繞 $+Z$ 軸作主動旋轉——即座標系不動、物體轉動——有

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

為方便處理不只在水平面轉動的物件，其餘兩軸的右手旋轉為

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

旋轉可以相乘，但必須說明次序。例如 $R_z R_y R_x p_h$ 是 R_x 最先作用，不能只含糊地稱作「三軸旋轉」。本章實驗只需繞 Z 軸的 R_z 。

上述 T 、 S 、 R 的最後一個**橫列 (row) **均為 $(0, 0, 0, 1)$ ；它們相乘後仍保有此形式。因此位置輸入的 $w = 1$ 、方向輸入的 $w = 0$ ，變換後也各自不變。一般投影矩陣的末橫列不必是這個形式，可能使輸出的 w 隨位置而變；那時不能再直接把前三個分量當作歐氏位置。這是仿射模型變換與後續投影變換之間的重要界線。

TRS 的次序與逆

把縮放、旋轉、平移組合，定義局部到世界的模型矩陣：

$$M = TRS, \quad p_{\text{world}} = Mp_{\text{local}}.$$

因為使用縱向量，右側矩陣先作用：**先沿局部軸縮放，再旋轉，最後沿世界軸平移**。若有父子節點，則 $M_{\text{world}} = M_{\text{parent}}M_{\text{local}}$ ，仍然由右向左作用。

次序不是排版選擇。一般 $TR \neq RT$ ： RT 會旋轉平移量；非均勻縮放時通常也有 $RS \neq SR$ 。只有在特定輸入或特殊變換下，兩種次序的結果才可能碰巧相同。

縮放與旋轉不交換也能直接算出來。取局部方向 $v = (1, 0, 0, 0)^T$ 、 $S = S(2, 1, 1)$ 、 $R = R_z(\pi/4)$ 。在 RSv 中，先縮放得 $(2, 0, 0, 0)^T$ ，再旋轉得 $(\sqrt{2}, \sqrt{2}, 0, 0)^T$ 。在 SRv 中，先旋轉得 $(\sqrt{2}/2, \sqrt{2}/2, 0, 0)^T$ ，再沿固定座標軸縮放得 $(\sqrt{2}, \sqrt{2}/2, 0, 0)^T$ 。兩個結果的 Y 分量不同。這正是「沿魚的局部長軸拉長後再轉向」與「轉向後沿世界 X 軸拉長」的差別；只看變換名稱而忽略乘法次序，無法知道實際縮放的是哪組軸。

若 s_x, s_y, s_z 全部非零，且 R 為純旋轉， S 、 R 、 T 都可逆。由矩陣乘法的逆序法則，

$$M^{-1} = (TRS)^{-1} = S^{-1}R^{-1}T^{-1} = S^{-1}R^T T(-t).$$

所以從世界座標回到局部座標，要先扣除平移，再反向旋轉，最後逐軸除以縮放係數。零縮放會壓掉某軸資訊，無法唯一還原。極小但非零的縮放雖然可逆，也可能讓浮點計算對誤差非常敏感。

以上推導針對位置與方向。非均勻縮放下，表面法線不能直接當成一般方向乘以 RS ；法線需使用可逆線性區塊的逆轉置，再正規化。負縮放則可能形成鏡射，改變三角形繞序與手性；本章容許其參與位置計算，但使用它建網格時必須另查表面朝向。

逐步手算例題

例一：魚體頂點的 TRS 與還原。 取局部位置 $p = (1, 1, 0)$ 公尺，縮放 $s = (2, 1, 1)$ ，繞 $+Z$ 旋轉 $\pi/2$ ，平移 $t = (3, 4, 0)$ 公尺。

1. 先縮放： $(1, 1, 0) \rightarrow (2, 1, 0)$ 。
2. 正向旋轉九十度： $(2, 1, 0) \rightarrow (-1, 2, 0)$ 。
3. 最後平移： $(-1, 2, 0) \rightarrow (2, 6, 0)$ 公尺。

現在反推： $(2, 6, 0) - (3, 4, 0) = (-1, 2, 0)$ ；旋轉 $-\pi/2$ 得 $(2, 1, 0)$ ；各軸除以 $(2, 1, 1)$ ，還原為 $(1, 1, 0)$ 。若輸入改為局部方向 $(1, 0, 0, 0)^T$ ，同一 M 會把它變成 $(0, 2, 0, 0)^T$ ：它旋轉且變長，卻不會被加上 $(3, 4, 0)$ 。

例二：同樣的數字，不同的次序。 設 $p = (1, 0, 0, 1)^T$ 、 $t = (2, 0, 0)$ 、 $R = R_z(\pi/2)$ 。對 TRp ，先旋轉得 $(0, 1, 0, 1)^T$ ，再平移得 $(2, 1, 0, 1)^T$ 。對 RTp ，先平移得 $(3, 0, 0, 1)^T$ ，再旋轉得 $(0, 3, 0, 1)^T$ 。兩個答案都符合各自的操作，不是其中一個精度較差。

實作與程式

下列是可獨立閱讀的 Python 程式，使用 NumPy 建立本章所需的 4×4 TRS，並把手算結果寫成測試。它不讀寫檔案，也不需要 GPU。NumPy 陣列的記憶體儲存順序，與我們用縱向量時的數學乘法次序是兩回事。

```
import numpy as np

def translation(t):
    t = np.asarray(t, dtype=float)
    if t.shape != (3,) or not np.all(np.isfinite(t)):
        raise ValueError("平移須為三個有限數")
    m = np.eye(4)
    m[:3, 3] = t
    return m

def scaling(s):
    s = np.asarray(s, dtype=float)
    if s.shape != (3,) or not np.all(np.isfinite(s)):
        raise ValueError("縮放須為三個有限數")
    if np.any(s == 0):
        raise ValueError("本實驗要求可逆，縮放不可為零")
    m = np.eye(4)
    m[0, 0], m[1, 1], m[2, 2] = s
    return m

def rotation_z(theta):
    if not np.isfinite(theta):
        raise ValueError("角度須為有限弧度數")
    c, s = np.cos(theta), np.sin(theta)
    m = np.eye(4)
    m[:2, :2] = [[c, -s], [s, c]]
    return m

def homogeneous(xyz, w):
    xyz = np.asarray(xyz, dtype=float)
    if xyz.shape != (3,) or not np.all(np.isfinite(xyz)):
        raise ValueError("座標須為三個有限數")
    if w not in (0, 1):
        raise ValueError("本實驗只建立位置 w=1 或方向 w=0")
    return np.array([xyz[0], xyz[1], xyz[2], float(w)])

def trs(t, theta, s):
    return translation(t) @ rotation_z(theta) @ scaling(s)
```

```

m = trs((3, 4, 0), np.pi / 2, (2, 1, 1))
p = homogeneous((1, 1, 0), 1)
direction = homogeneous((1, 0, 0), 0)

assert np.allclose(m @ p, (2, 6, 0, 1), atol=1e-12)
assert np.allclose(m @ direction, (0, 2, 0, 0), atol=1e-12)
assert np.allclose(np.linalg.inv(m) @ (m @ p), p, atol=1e-12)

q = homogeneous((1, 0, 0), 1)
t = translation((2, 0, 0))
r = rotation_z(np.pi / 2)
assert np.allclose((t @ r) @ q, (2, 1, 0, 1), atol=1e-12)
assert np.allclose((r @ t) @ q, (0, 3, 0, 1), atol=1e-12)

try:
    scaling((1, 0, 1))
except ValueError:
    pass
else:
    raise AssertionError("零縮放應遭拒絕")

```

NumPy 的 `m[i, j]` 指第 i 個橫列、第 j 個縱行；`m[:3, 3] = t` 因此把平移放在最後一個縱行的前三個位置。`@` 執行矩陣乘法，`*` 則是逐元素相乘，不能用後者取代 TRS 。陣列採何種記憶體儲存順序，不會把 `m @ p` 自動改成橫向量慣例；檢查程式時，應看索引和乘法式，而非從儲存方式猜測幾何意義。

此處以 `np.linalg.inv(m)` 作為**獨立的數值往返檢查**；手算時仍應理解 $S^{-1}R^{-1}T^{-1}$ 的順序，而非把求逆函式當作次序的解釋。程式拒絕零縮放，是因為本實驗要測逆轉換；某些只需把模型壓成平面的圖學操作可以使用零縮放，但不能再要求唯一反推原座標。對接近零的縮放，雖然 `inv` 在數學上仍有定義，逆矩陣卻會放大該軸的輸入誤差；實務上須按模型尺度檢查數值條件，不能只依「非零」便認定往返一定可靠。

測試與預期結果

若在既有 NumPy 環境中執行，**預期**上述斷言全部通過，程式沒有文字輸出。測試依序檢查：位置的 TRS 結果、方向不受平移、逆矩陣往返、 TR 與 RT 的不同結果，以及零縮放遭拒。這些是可重現的預期，並非本章已執行程式的紀錄。

`atol=1e-12` 只針對此處尺度約數公尺、運算很短的數值例子。因為 $\cos(\pi/2)$ 的浮點結果可能不是精確的零，不宜對旋轉後分量使用完全相等判斷。若場景尺度、矩陣串接次數或縮放條件改變，應重新選擇合理容差；數值容差不能當成任何物理安全閾值。

單一頂點的往返通過，也不代表任意輸入都沒有錯誤。例如某個點剛好在旋轉軸上，即使旋轉角寫錯，其位置也可能不變。因此測試同時選用不在旋轉軸上的位置、方向向量及兩種乘法次序。擴充程式時，還可測試原點經 M 變換後是否正好位於 t ，以及不同局部點之差是否和對應世界點之差一致；後者的 $w = 0$ ，正好能抓出方向誤受平移影響的實作錯誤。

除錯與常見陷阱

- **角度或正向弄反：** `np.sin`、`np.cos` 接收弧度。先用 $R_z(\pi/2)(1, 0, 0, 0)^T = (0, 1, 0, 0)^T$ 檢查，不要只憑畫面直覺改負號。
- **把位置當方向，或把方向當位置：** 檢查 w 。同一個平移矩陣應移動 $(0, 0, 0, 1)^T$ ，但不應移動 $(1, 0, 0, 0)^T$ 。
- **旋轉中心不對：** 物件若繞世界原點公轉，而不是繞自己的局部原點轉動，檢查矩陣是否誤寫成 RTS ，以及網格局部原點設在哪裡。
- **逆矩陣乘法順序不對：** $M = TRS$ 的逆不是 $T^{-1}R^{-1}S^{-1}$ 。先用例一逐步還原，再檢查程式。
- **混淆畫面與世界座標：** 世界 $+Y$ 向上；本書後續使用的影像像素索引則向下增加。影像翻轉應在對應的座標轉換中處理，不要偷偷改本章的右手旋轉定義。
- **鏡射後表面不見了：** 負縮放可能翻轉三角形繞序，使以背面剔除繪製的表面消失；位置計算正確，不代表繞序與法線也已處理。

養殖數位分身案例

在一個純合成場景中，先以公尺建立池體及魚身網格。魚身局部原點放在中心，魚頭朝局部 $+X$ ；記錄其平移 t 、轉向角 θ 和縮放 s ，即可用 $M = TR_z(\theta)S(s)$ 把魚身頂點放到世界座標。例一的局部頂點 $(1, 1, 0)$ 因而位於世界 $(2, 6, 0)$ 公尺；要查詢這個世界位置落在原網格何處，使用 M^{-1} 。

若池體是父節點、魚體是子節點，記錄 $M_{\text{world}} = M_{\text{pool}}M_{\text{fish}}$ ，可以先在池體局部座標擺魚，再把整座池體安放至世界。每一層都應記清楚**輸入與輸出屬於哪個座標系**；相同的三個數字，在魚局部系與世界系不代表同一位置。縮放係數無單位，但魚網格與池體尺寸須採相同長度單位，否則數學測試可以通過，場景比例仍可能錯誤。這些合成座標只證明幾何配置可重現，不證明魚的姿態或池體尺寸符合實際養殖觀測。

習題

1. **手算：** 設局部位置 $p = (1, 2, 0)$ 公尺， $S = S(2, 3, 1)$ 、 $R = R_z(\pi/2)$ 、 $T = T(4, -1, 0)$ 公尺。求 $TRSp_h$ ，再依正確次序反推 p 。
2. **程式測試：** 在本程式末尾增加測試：任選有限的平移量，確認純平移不改變方向 $(0, 1, 0, 0)^T$ ；再確認「位置加方向」變換後，等於兩者分別變換後相加。寫出斷言並解釋末分量。
3. **反例與除錯：** 某程式員寫 `scaling((2, 1, 1)) @ rotation_z(np.pi / 2)`，卻說它會「先沿魚的局部 X 軸拉長兩倍，再旋轉」。分別求程式及其所述操作對方向 $(1, 0, 0, 0)^T$ 的結果，指出改法。
4. **整合應用：** 池體父節點平移 $(10, 0, 0)$ 公尺。魚體子節點依序在其局部系均勻縮放 2、繞 Z 軸轉 $\pi/2$ 、平移 $(0, 2, 0)$ 公尺。求魚體局部位置 $(1, 0, 0)$ 的世界位置，以及局部方向 $(1, 0, 0)$ 的世界方向；寫出世界位置轉回魚體局部所需的矩陣順序。

習題解答

1. 縮放後是 $(2, 6, 0)$ ，旋轉後是 $(-6, 2, 0)$ ，平移後是 $(-2, 1, 0)$ 公尺，即 $(-2, 1, 0, 1)^T$ 。逆向先扣除 $(4, -1, 0)$ ，得 $(-6, 2, 0)$ ；再轉 $-\pi/2$ ，得 $(2, 6, 0)$ ；最後逐軸除以 $(2, 3, 1)$ ，還原 $(1, 2, 0)$ 。
2. 可在原程式後加入：

```
shift = translation((7, -3, 2))
point = homogeneous((1, 2, 3), 1)
vector = homogeneous((0, 1, 0), 0)
assert np.allclose(shift @ vector, vector)
assert np.allclose(
    shift @ (point + vector),
    (shift @ point) + (shift @ vector)
)
```

`point + vector` 的 $w = 1 + 0 = 1$ ，仍表示位置；等式也可直接由矩陣對齊次向量相加的分配律得到。預期兩個斷言通過。

3. 程式的乘積是 SR ，先旋轉 $(1, 0, 0)$ 得 $(0, 1, 0)$ ，再沿 X 縮放，結果仍是 $(0, 1, 0, 0)^T$ 。敘述所需的 RS 則先得 $(2, 0, 0)$ ，再旋轉成 $(0, 2, 0, 0)^T$ 。應改寫為 `rotation_z(np.pi / 2) @ scaling((2, 1, 1))`。
4. 魚體局部位置依序變成 $(2, 0, 0)$ 、 $(0, 2, 0)$ 、 $(0, 4, 0)$ ，最後經父節點平移，世界位置是 $(10, 4, 0)$ 公尺。方向略過兩次平移，世界方向是 $(0, 2, 0)$ 。令

$$M_{\text{parent}} = T(10, 0, 0), \quad M_{\text{local}} = T(0, 2, 0)R_z(\pi/2)S(2, 2, 2).$$

則世界到魚體局部使用 $(M_{\text{parent}}M_{\text{local}})^{-1} = M_{\text{local}}^{-1}M_{\text{parent}}^{-1}$ 。先撤銷父節點，再撤銷魚體局部變換，所得位置為 $(1, 0, 0, 1)^T$ 。

本章小結

齊次座標讓平移、旋轉、縮放都能用矩陣組合，同時藉 $w = 1$ 與 $w = 0$ 區分位置和方向。在縱向量慣例下， TRS 從右往左作用；改變順序通常會改變結果。反變換則以相反次序撤銷各步驟，並要求縮放非零。建立場景時，先用可手算的位置與方向測試次序，再把矩陣套用到整個網格及父子節點。

參考來源

- [G1] *PBRT 4 : Transformations*，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] *LearnOpenGL : Transformations*，<https://learnopengl.com/Getting-started/Transformations>
- [G7] *NumPy 線性代數參考*，<https://numpy.org/doc/stable/reference/routines.linalg.html>

以上提供變換概念及線性代數函式的延伸閱讀；閱讀其他材料時，仍須先核對其使用的是縱向量或橫向量，以及矩陣的作用次序。

第04章 相機、座標系與視圖矩陣

學習目標與先備知識

本章建立從世界座標系到相機座標系的映射能力。讀者需具備第3章的齊次座標與 4×4 變換矩陣基礎，並熟悉外積的右手定則判定。本節定義全書通用符號：

- **位置**： $p_h = (x, y, z, 1)^T$ 。
- **方向**： $v_h = (x, y, z, 0)^T$ 。
- **世界系**：右手系， $+X$ 向右， $+Y$ 向上， $+Z$ 由畫面向觀者。
- **相機系**：右手系，視點位於原點，視線方向為 $-Z_c$ ，上方為 $+Y_c$ ，右方為 $+X_c$ 。

我們區分「主動變換」（移動物體）與「被動變換」（改變座標系）。視圖矩陣 V 是被動變換：將世界中的點轉到相機座標系，使相機位於原點且看向 $-Z_c$ 。以相機位置、目標點與建議上方向建立此矩陣的方法稱為 look-at；它是建立世界到相機外參的一種方式，而非所有外參的同義詞 [G1]。外參描述兩個座標系的相對姿態，不描述鏡頭焦距、影像尺寸或投影方式。同一個世界點即使沒有移動，只要相機姿態改變，其相機座標就會改變。

本章以 column vector 表示數學向量，使用 $p' = Mp$ 的形式實作變換；即矩陣在左、向量在右。NumPy 的 `M @ p` 直接對應此數學約定，記憶體儲存順序不改變數學結果。

問題與直覺

養殖場監控相機固定在水池上方。若相機旋轉拍攝，物體位置不變，但其在相機座標中的數值改變。我們需要一個函式 `look_at(eye, center, up)`，輸入相機位置 `eye`、目標點 `center` 與建議上方向 `up`，輸出視圖矩陣 V 。

直覺上， V 由三個基向量定義：

1. **視線方向**：從 `eye` 指向 `center`，對應相機 $-Z_c$ 軸。
2. **右方向**：垂直於視線與上方向，對應相機 $+X_c$ 軸。
3. **真正上方向**：垂直於視線與右方向，對應相機 $+Y_c$ 軸。

視圖矩陣將世界點 p 轉為相機點 p_c ，滿足 $p_c = Vp$ 。此矩陣將世界中的相機位置 e 映到相機座標原點，並以相機局部基底表示其餘世界點 [G5]。亦即 $V(e, 1)^T = (0, 0, 0, 1)^T$ ；世界原點一般不會映到相機原點。此處談的是同一個幾何點換用另一組座標描述，而不是把池體頂點真的移動。

數學與幾何推導

設相機位置 e ，目標點 c ，上方向 u （不要求單位向量）。

1. **視線方向**： $f = \frac{c-e}{\|c-e\|}$ 。這是相機看的方向，即 $-Z_c$ 。因此相機的 Z 軸為 $z_c = -f$ 。
2. **右方向**： $x_c = \frac{f \times u}{\|f \times u\|}$ 。利用右手定則， $f \times u$ 指向相機右側：例如 $f = (0, 0, -1)$ 、 $u = (0, 1, 0)$ 時， $f \times u = (1, 0, 0)$ 。正規化後的 x_c 垂直於 f 。輸入的 u 只是建議方向；若它不是與視線垂直，最後的 y_c 會是在其垂直視線平面上的方向，而非原封不動的 u 。當 u 與 f 平行時，外積為零，單靠這三項輸入無法唯一決定相機繞視線的轉角。
3. **真正上方向**： $y_c = x_c \times f$ 。確保正交且符合右手系。

相機基向量矩陣 R （旋轉部分）由這三個單位向量組成。由於是column vector約定， R 的**橫列**（row）分別為 x_c, y_c, z_c ：

$$R = \begin{bmatrix} x_c^T & 0 & & \\ y_c^T & 0 & & \\ z_c^T & 0 & & \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x_{cx} & x_{cy} & x_{cz} & 0 \\ y_{cx} & y_{cy} & y_{cz} & 0 \\ z_{cx} & z_{cy} & z_{cz} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

採用 column vector 且由右先作用，視圖矩陣**先平移、再旋轉**：先求世界點相對於相機位置的位移，然後把位移在三個相機軸上取內積。令 $B = [x_c \ y_c \ z_c]$ 為以這三個三維向量作為縱列（column）的 3×3 矩陣，則 B^T 的三個橫列（row）正是 x_c^T, y_c^T, z_c^T ，因此

$$V = RT(-e) = \begin{bmatrix} B^T & -B^T e \\ 0 \ 0 \ 0 & 1 \end{bmatrix}.$$

這也釐清了齊次向量的維度： $p_h = (p_x, p_y, p_z, 1)^T$ 是四維， p 與 e 才是三維，不能直接寫 $p_h - e$ 。正確的分步結果為

$$T(-e)p_h = \begin{bmatrix} p - e \\ 1 \end{bmatrix}, \quad Vp_h = \begin{bmatrix} B^T(p - e) \\ 1 \end{bmatrix}.$$

換言之，相機座標的三個分量依次是 $x_c \cdot (p - e)$ 、 $y_c \cdot (p - e)$ 與 $z_c \cdot (p - e)$ 。平移欄 $-B^T e$ 是由矩陣乘法得來，不宜在相機旋轉時僅把 $-e$ 填入 V 最後一欄。若輸入為方向 $v_h = (v_x, v_y, v_z, 0)^T$ ，平移不起作用，所得方向為 $(B^T v, 0)^T$ 。位置與方向的差異正是齊次分量 w 的用途。

被動轉換與反變換： V 是剛體仿射變換；其左上角 3×3 部分 B^T 為正交矩陣且行列式為 $+1$ ，含平移的整個 4×4 矩陣則不能稱為正交矩陣。相機到世界矩陣 C 把相機局部座標轉回世界座標 [G4]。其旋轉部分 B 的三個縱列（column），依序是相機右、上、後方軸在世界座標中的表示：

$$C = \begin{bmatrix} B & e \\ 0 \ 0 \ 0 & 1 \end{bmatrix}, \quad B = [x_c \ y_c \ z_c].$$

由三軸互相正交且為單位向量可得 $B^T B = I$ 。對相機座標 q ，先旋轉回世界方向再加上相機位置，即 $p = Bq + e$ ；反解得 $q = B^T(p - e)$ ，所以

$$V = C^{-1} = \begin{bmatrix} B^T & -B^T e \\ 0 \ 0 \ 0 & 1 \end{bmatrix}.$$

這個反推也提供實作檢查： VC 及 CV 都應接近單位矩陣。將相機本身在世界中的姿態看作主動放置相機，與把固定世界點改用相機系描述的被動視圖轉換，會得到互為反矩陣的數值關係；兩種敘述不可混成同一個作用方向。

逐步手算例題

案例1：簡易相機 世界系： $e = (0, 0, 5), c = (0, 0, 0), u = (0, 1, 0)$ 。

1. $f = \frac{(0,0,0)-(0,0,5)}{5} = (0, 0, -1)$ 。
2. $z_c = -f = (0, 0, 1)$ 。
3. $x_c = \frac{f \times u}{\|f \times u\|} \circ f \times u = (0, 0, -1) \times (0, 1, 0) = (1, 0, 0) \circ \|f \times u\| = 1 \circ x_c = (1, 0, 0)$ 。
4. $y_c = x_c \times f = (1, 0, 0) \times (0, 0, -1) = (0, 1, 0)$ 。檢查： x_c, y_c, z_c 兩兩正交且為單位向量。
 $\det([x_c, y_c, z_c]) = 1$ 。

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \circ T(-e) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \circ V = RT(-e) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix}。$$

測試點：世界原點 $p = (0, 0, 0)$ 。 $Vp = (0, 0, -5, 1)^T = (0, 0, -5)$ 。在相機座標中，原點在 $z = -5$ ，即相機前方5公尺。正確。測試點：世界點 $p = (1, 0, 0)$ 。 $Vp = (1, 0, -5, 1)^T$ 。 $x_{cam} = 1$ ，符合右手系（右方為 $+X_c$ ）。

案例2：平行up向量陷阱與Fallback 若 u 平行於 f （例如相機垂直往下看，且 u 也是垂直）， $f \times u = 0$ 。解法：先確認視點與目標點有足夠距離、 up 不是零向量，再檢測 $\|f \times u_n\| < \epsilon_{angle}$ ，其中 u_n 是正規化的 up ，而又積長度是無因次的夾角正弦。若過小，採事先公布的備援上方向，例如先試 $(0, 0, 1)$ 、仍平行才試 $(1, 0, 0)$ 。備援軸只是決定未指定的相機 roll，並不能推知拍攝者原本想讓畫面哪一側朝上；動畫相機若要求姿態連續，還需另設延續上一影格方向的政策。數值案例：設 $e = (0, 0, 0), c = (0, 1, 0), u = (0, 1, 0)$ 。 $f = (0, 1, 0)$ 。 $u_n = (0, 1, 0)$ 。 $f \times u_n = 0$ 。Fallback $u' = (0, 0, 1)$ 。 $x_c = \frac{f \times u'}{\|f \times u'\|} = \frac{(0,1,0) \times (0,0,1)}{1} =$

$$(1, 0, 0) \circ y_c = x_c \times f = (1, 0, 0) \times (0, 1, 0) = (0, 0, 1) \circ z_c = -f = (0, -1, 0) \circ V = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

。此矩陣將世界 $+Y$ 軸映到相機 $-Z$ 軸（視線），世界 $+Z$ 軸映到相機 $+Y$ 軸（上方向）。符合預期。

實作與程式

以下程式供讀者在既有的 Python 3.10+、NumPy 環境使用，不須 GPU。`eye` 與 `center` 以公尺表示，`up` 只指定方向；若使用倍率不變性測試，選取的倍率必須讓 `up` 長度仍高於 `direction_eps`。`position_eps` 應依場景尺度設定，不能把一個容差同時當作公尺、輸入方向長度及角度的門檻。輸入座標雖可為整數，函式會轉為浮點陣列。程式分別拒絕錯誤 shape、非有限輸入、視點與目標過近及零長上方向；這些是建立相機基底的必要條件，不是物理安全閾值。

用矩陣前三個橫列存放相機軸，是因為每個相機座標分量都要對世界位移取一次內積。函式最後的 $R @ T$ 代表先平移後旋轉，不是 NumPy 陣列在記憶體中的儲存次序。備援上方向是明訂的確定性政策；它能讓完全平行的輸入有結果，卻可能使接近平行處的畫面旋轉突然改變。需要連續動畫時，不宜僅以此靜態函式保證鏡頭運動平滑。

```
import numpy as np

def look_at(eye, center, up,
            position_eps=1e-9, direction_eps=1e-12,
            angular_eps=1e-6):
    """建立世界到相機的 (4, 4) 視圖矩陣。

    position_eps 與 eye、center 同長度單位（本書為公尺）；
    direction_eps 與輸入 up 的數值單位相同；
    angular_eps 為正規化方向叉積的無因次容差。
    """
    def as_vec3(value, name):
        value = np.asarray(value, dtype=np.float64)
        if value.shape != (3,):
            raise ValueError(f"{name} must have shape (3,)")
        if not np.all(np.isfinite(value)):
            raise ValueError(f"{name} must contain finite values")
        return value

    eye = as_vec3(eye, "eye")
    center = as_vec3(center, "center")
    up = as_vec3(up, "up")
    for name, value in (("position_eps", position_eps),
                        ("direction_eps", direction_eps)):
        if (not np.isscalar(value) or not np.isreal(value)
            or not np.isfinite(value) or value <= 0):
            raise ValueError(f"{name} must be positive and finite")
    if (not np.isscalar(angular_eps) or not np.isreal(angular_eps)
        or not np.isfinite(angular_eps)
        or not (0 < angular_eps <= 1)):
        raise ValueError("angular_eps must be in (0, 1]")

    displacement = center - eye
    distance = np.linalg.norm(displacement)
    if not np.isfinite(distance) or distance < position_eps:
        raise ValueError("eye and center are too close or out of range")
    f = displacement / distance

    up_len = np.linalg.norm(up)
    if not np.isfinite(up_len) or up_len < direction_eps:
        raise ValueError("up is too short or out of range")
    up_n = up / up_len

    right = np.cross(f, up_n)
    right_len = np.linalg.norm(right)
    if right_len < angular_eps:
        # 備援政策：先世界 +Z；若平行，再世界 +X。
        for candidate in ((0.0, 0.0, 1.0), (1.0, 0.0, 0.0)):
            right = np.cross(f, candidate)
            right_len = np.linalg.norm(right)
            if right_len >= angular_eps:
                break
```

```

    else:
        raise ValueError("cannot determine camera right axis")
    x_c = right / right_len
    y_c = np.cross(x_c, f)
    z_c = -f

    R = np.eye(4)
    R[0, :3] = x_c
    R[1, :3] = y_c
    R[2, :3] = z_c
    T = np.eye(4)
    T[:3, 3] = -eye
    return R @ T

```

測試與預期結果

以下是**預期**通過的讀者端測試，並非聲稱作者已執行。除了核對矩陣數字，還要檢查變換的幾何不變量：相機位置應落在原點；目標應落在負 Z_c 軸；旋轉部分各軸長度為一、互相正交且行列式為+1。只測最後一欄容易漏掉外積次序或矩陣橫列配置錯誤。基本案例把世界原點送到(0, 0, -5)，恰好也能區分「相機位置映到原點」與「世界原點映到原點」這兩種敘述。

若要檢查逆轉換，可在下方測試後另加 `C = np.linalg.inv(V)`、`assert np.allclose(V @ C, np.eye(4))`，再令 `p = np.array([1., 2., 0., 1.])`，檢查 `C @ (V @ p)` 是否接近 `p`。這裡的逆矩陣只用於讀者端核對；前述推導已說明為何它存在。浮點數應使用合理容差比較，不應要求兩次矩陣乘法的結果逐位相等。

1. 基本測試與相機位置驗證：

```

eye = np.array([0.0, 0.0, 5.0])
V = look_at(eye, [0,0,0], [0,1,0])
# Expected V:
# [[1, 0, 0, 0],
#  [0, 1, 0, 0],
#  [0, 0, 1, -5],
#  [0, 0, 0, 1]]
assert np.allclose(V[2, 3], -5.0)

# 驗證相機位置映到原點
eye_cam = V @ np.array([0.0, 0.0, 5.0, 1.0])
assert np.allclose(eye_cam, [0.0, 0.0, 0.0, 1.0])

# 驗證中心點位於負 Z 軸
center_cam = V @ np.array([0.0, 0.0, 0.0, 1.0])
assert np.allclose(center_cam[:2], [0.0, 0.0])
assert center_cam[2] < 0

```

2. 平行up測試：

```

# Eye (0,0,0), Center (0,1,0), Up (0,1,0) -> Parallel
V = look_at([0,0,0], [0,1,0], [0,1,0])
# Expected x_c = (1,0,0), y_c = (0,0,1), z_c = (0,-1,0)

```

```
assert np.allclose(V[0, :3], [1, 0, 0])
assert np.allclose(V[1, :3], [0, 0, 1])
assert np.allclose(V[2, :3], [0, -1, 0])
```

3. 右手系與行列式檢查：

```
V = look_at([1,2,3], [0,0,0], [0,1,0])
R3 = V[:3, :3]
assert np.allclose(R3 @ R3.T, np.eye(3))
assert np.allclose(np.linalg.det(R3), 1.0)
assert np.allclose(np.cross(R3[0], R3[1]), R3[2])
```

4. Up 倍率不變性：

```
V1 = look_at([1, 2, 3], [0, 0, 0], [0, 1, 0])
V2 = look_at([1, 2, 3], [0, 0, 0], [0, 100, 0])
assert np.allclose(V1, V2)
```

5. Eye == Center 錯誤處理：

```
raised = False
try:
    look_at([0, 0, 0], [0, 0, 0], [0, 1, 0])
except ValueError:
    raised = True
assert raised
```

除錯與常見陷阱

當畫面左右顛倒時，先用世界+ X 方向的測試點檢查相機+ X_c ，再檢查 $x_c \times y_c = z_c$ ，不要只看目標是否落在負 Z_c 。目標落在正確方向，仍可能伴隨錯誤的上方向或鏡射。若相機在非原點，亦應核對 $V(e, 1)^T$ ；只在 $e = (0, 0, 0)$ 測試，無法發現平移欄漏乘旋轉矩陣。

容差失敗須分辨三種量：視點與目標距離有公尺單位；未正規化的 `up` 長度依呼叫者提供的數值尺度而定；兩個單位方向的叉積長度則無因次。平行與近乎平行都可能使右軸不穩定，但提高角度容差也會讓更多姿態進入備援分支，應依用途測試邊界。即使某個頂點在近平面之外，也不能單憑它刪除整個跨越近平面的三角形；頂點篩選不等於三角形裁切。

1. **up向量平行**：必須處理cross product為零的情況。正規化後的叉積長度代表夾角的正弦值，`eps` 是無因次角度退化容差。
2. **左乘右乘混淆**：本採用 $p' = Mp$ 。若使用行向量，需轉置。
3. **z_c 方向**：確保 $z_c = -f$ ，使得視點看向 $-Z_c$ 。若誤用 $z_c = f$ ，相機會看向 $+Z$ ，導致所有物體在 $+Z$ 軸，與後續投影矩陣不符。
4. **外積次序**： $x_c = \text{normalize}(f \times u)$ 而非 $u \times f$ 。 $u \times f$ 會導致左手系或鏡射（行列式為-1）。
5. **非單位向量**：輸入的 `up` 不一定要單位長度，實作中會先正規化以確保尺度不變性，但 `f` 必須正規化以確保 z_c 是單位向量。

6. **近平面判定**：在相機看向 $-Z_c$ 的慣例下，近平面為 $z_{cam} = -near$ ，遠平面為 $z_{cam} = -far$ 。僅考慮前後裁切時，保留範圍為 $-far \leq z_{cam} \leq -near$ 。因此 $z_{cam} > -near$ 表示位於相機後方或過於接近相機； $z_{cam} < -far$ 表示超過遠平面。這只是深度方向測試，不代表點通過左右、上下視錐或遮擋測試。

養殖數位分身案例

水池頂點 $P_{pool} = (10, 0, 5)$ 。相機 $e = (0, 10, 10)$, $c = (0, 0, 0)$, $u = (0, 1, 0)$ 。計算 V 後，將 P_{pool} 轉到相機座標。 $f = \frac{(0,0,0)-(0,10,10)}{\sqrt{200}} = (0, -1/\sqrt{2}, -1/\sqrt{2})$ 。令 $a = 1/\sqrt{2}$ 。 $z_c = -f = (0, a, a)$ 。 $x_c = \frac{f \times u}{\|f \times u\|}$ 。 $f \times u = (0, -a, -a) \times (0, 1, 0) = (a, 0, 0)$ 。正規化後 $x_c = (1, 0, 0)$ 。 $y_c = x_c \times f = (1, 0, 0) \times (0, -a, -a) = (0, a, -a)$ 。

旋轉矩陣 R ：

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & a & -a & 0 \\ 0 & a & a & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

平移矩陣 $T(-e)$ ，其中 $-e = (0, -10, -10)$ ：

$$T(-e) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -10 \\ 0 & 0 & 1 & -10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

視圖矩陣 $V = RT(-e)$ 。平移分量計算： $V[0, 3] = (1, 0, 0) \cdot (0, -10, -10) = 0$ 。 $V[1, 3] = (0, a, -a) \cdot (0, -10, -10) = -10a + 10a = 0$ 。 $V[2, 3] = (0, a, a) \cdot (0, -10, -10) = -10a - 10a = -20a = -10\sqrt{2}$ 。

$P_{pool} - e = (10, -10, -5)$ 。 $x_{cam} = (1, 0, 0) \cdot (10, -10, -5) = 10$ 。 $y_{cam} = (0, a, -a) \cdot (10, -10, -5) = -10a + 5a = -5a = -5/\sqrt{2}$ 。 $z_{cam} = (0, a, a) \cdot (10, -10, -5) = -10a - 5a = -15a = -15/\sqrt{2}$ 。

$P_{pool}^{cam} = (10, -5/\sqrt{2}, -15/\sqrt{2}, 1)^T \approx (10, -3.536, -10.607, 1)^T$ 。 $z_{cam} \approx -10.607$ 公尺。若 $near = 0.1$ 公尺且 $far > 10.607$ 公尺，此點通過前後深度範圍測試；仍未檢查左右、上下視錐及遮擋，因此不能僅憑負的 z_{cam} 斷言頂點會出現在影像內。此處的池體頂點與相機配置皆為合成數值。

習題

- 手算：給定 $e = (1, 1, 1)$, $c = (0, 0, 0)$, $u = (0, 1, 0)$ ，計算 x_c, y_c, z_c 並構建 V 的前三行及平移列。
- 程式測試：現有 `look_at` 已接受未正規化的 `up`。請撰寫測試，驗證將非零 `up` 乘以正的有限倍率後，在長度仍高於 `direction_eps` 時，視圖矩陣與原輸入一致；另測零長 `up` 會被拒絕。

3. 反例/除錯：若 up 平行於 f ，目前的fallback策略是否可能導致 y_c 與預期上方向相反？如何修正roll角？
4. 整合應用：寫一個函式，輸入一個三角網格（頂點列表）和相機參數，輸出在相機座標中 $z > -near$ （即相機背後或太近）的頂點索引列表，用於簡單剔除。注意：這僅為頂點層級剔除，三角形裁切需額外處理。三個頂點不全在同一外側半空間時，不得直接刪除三角形。

習題解答

1. $f = \frac{(-1,-1,-1)}{\sqrt{3}} \circ z_c = \frac{(1,1,1)}{\sqrt{3}} \circ x_c = \frac{f \times u}{\|f \times u\|} \circ f \times u = \frac{1}{\sqrt{3}}(-1,-1,-1) \times (0,1,0) = \frac{1}{\sqrt{3}}(1,0,-1)$ 。
Norm is $1/\sqrt{3} \cdot \sqrt{2} \circ x_c = \frac{1}{\sqrt{2}}(1,0,-1)$ 。依外積各分量計算， $(1,0,-1) \times (-1,-1,-1) = (-1,2,-1)$ ，故 $y_c = x_c \times f = \frac{1}{\sqrt{6}}(-1,2,-1)$ 。它與 x_c 、 z_c 互相正交；將三軸填入 V 的前三個橫列後，才計算平移欄。

Translation: $-e = (-1,-1,-1) \circ V[0,3] = x_c \cdot (-1,-1,-1) = \frac{1}{\sqrt{2}}(1(-1) + 0 + -1(-1)) = 0$ 。
 $V[1,3] = y_c \cdot (-1,-1,-1) = \frac{1}{\sqrt{6}}(-1(-1) + 2(-1) + -1(-1)) = 0$ 。
 $V[2,3] = z_c \cdot (-1,-1,-1) = \frac{1}{\sqrt{3}}(1(-1) + 1(-1) + 1(-1)) = -\sqrt{3}$ 。

$$V = \begin{bmatrix} \frac{1}{\sqrt{2}} & 0 & -\frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & 0 \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & -\sqrt{3} \\ 0 & 0 & 0 & 1 \end{bmatrix}。$$

2. 例如選 $e = (1,2,3)$ 、 $c = (0,0,0)$ 、 $u = (0,2,0)$ ，可用 `assert np.allclose(look_at(e, c, u), look_at(e, c, np.array(u) * 50.0))` 檢查正倍率不變性；也可與 `look_at(e, c, np.array(u) / np.linalg.norm(u))` 比較。零向量測試則用 `try` 呼叫 `look_at(e, c, [0,0,0])` 並在捕獲 `ValueError` 後確認確有拋錯。負倍率會反轉建議上方向，不屬於本題的不變性。
3. 當 up 與 f' 平行，輸入不足以唯一決定繞視線的 $roll$ ；所有可用的真正上方向都垂直於 f ，不能靠「與原 up 夾角最小」選出唯一答案。可明訂先選世界+ Z 、仍平行再選世界+ X 的政策；若是逐影格動畫，也可增加上一影格相機右軸作為額外輸入，以盡量保持時間連續性。固定備援軸雖可重現，接近切換方向時仍可能出現不連續翻轉。
4. 以下函式回傳**頂點索引**，不刪除任何三角形。它檢查 `near` 為正且有限、頂點為有限的三維座標；空網格回傳空列表。等於近平面 $z = -near$ 的頂點不列入外側，因為題目使用嚴格大於。

```
def vertices_before_near(vertices, eye, center, up, near):
    if not np.isscalar(near) or not np.isfinite(near) or near <= 0:
        raise ValueError("near must be positive and finite")
    pts = np.asarray(vertices, dtype=np.float64)
    if pts.shape == (0,):
        pts = np.empty((0, 3), dtype=np.float64)
    if pts.ndim != 2 or pts.shape[1] != 3:
        raise ValueError("vertices must have shape (N, 3)")
    if not np.all(np.isfinite(pts)):
        raise ValueError("vertices must be finite")
    V = look_at(eye, center, up)
```

```
return [i for i, p in enumerate(pts)
        if (V @ np.array([p[0], p[1], p[2], 1.0]))[2] > -near]
```

例如令 `eye=[0,0,5]` 、 `center=[0,0,0]` 、 `up=[0,1,0]` 、 `near=1` ，世界 $z = 4$ 的頂點恰在近平面，預期不列入；世界 $z = 4.5$ 的頂點預期列入，世界 $z = 3$ 則不列入。可以用 `assert vertices_before_near([[0,0,4], [0,0,4.5], [0,0,3]], [0,0,5], [0,0,0], [0,1,0], 1) == [1]` 檢查。對三角形而言，只有整個三角形都在同一外側半空間時，才能據此作整體剔除；跨越近平面的面需要裁切。

本章小結

本章介紹了視圖矩陣的構造，強調了被動變換的概念和右手系的一致性。`look_at` 函式是3D圖學的核心，必須嚴格處理邊界情況（如`up`平行於`f`）並確保外積次序正確以維持右手系。正確建立相機座標是後續投影和剔除的基礎。

參考來源

- G1: PBRT 4 Transformations
- G4: Ray Tracing in One Weekend (Camera setup)
- G5: LearnOpenGL Transformations

第05章 透視投影、裁切與深度

學習目標與先備知識

讀完本章後，你應能：

- 說明透視投影與正交投影如何把相機座標映射到剪裁座標。
- 推導右手相機、視線沿 $-Z$ 、OpenGL 式 NDC 深度範圍 $[-1, 1]$ 的透視投影矩陣。
- 區分剪裁座標、NDC、視窗座標與深度緩衝值，並解釋透視除法。
- 以近平面條件裁切線段或三角形，並說明深度精度受到近平面、遠平面及緩衝格式的影響。

先備知識是向量、矩陣乘法與齊次座標。本文採用直向量與右手座標系；相機位於原點時看向 $-Z$ ，相機上方為 $+Y$ 。長度以公尺表示，角度運算使用弧度。

本章主線採 OpenGL 式剪裁與 NDC 慣例。其他圖形 API 可能使用不同的深度範圍或矩陣慣例；不能把它們的投影矩陣與本章的裁切及深度映射規則混用。

問題與直覺

三維養殖池場景必須顯示在有限大小的影像上。相機投影將場景位置轉成剪裁座標，裁切程序移除不可見部分，最後才映射到 NDC 與像素範圍。

透視投影符合近大遠小：同一物體距離相機越遠，投影尺寸越小。正交投影則不隨深度縮放，適合工程或量測視圖。兩種投影都需要裁切；差異主要是投影如何處理深度與大小。

要正確處理可見性，必須分清相機空間的 z 、剪裁座標的 z_c 、透視除法後的 z_{ndc} ，以及深度緩衝值。它們不是同一個量，也不能不經轉換就互相替代。

數學與幾何推導

從相似三角形到剪裁座標

令相機座標中的點為

$$p_{\text{cam}} = (x, y, z, 1)^T,$$

相機前方的點滿足 $z < 0$ 。近平面與遠平面距離分別為 n 、 f ，且 $0 < n < f$ ；兩平面位於 $z = -n$ 與 $z = -f$ 。

令垂直視野角為 θ ，影像長寬比為 $a = W/H$ 。近平面邊界為

$$t = n \tan \frac{\theta}{2}, \quad b = -t, \quad r = at, \quad l = -r.$$

由相似三角形，三維點投影到近平面的座標是

$$x_{\text{near}} = n \frac{x}{-z}, \quad y_{\text{near}} = n \frac{y}{-z}.$$

由於相機前方 $z < 0$ ，分母 $-z$ 為正。遠處點的 $|z|$ 較大，所以投影位置的絕對值較小，這就是透視縮小。

把近平面橫向座標映射到 NDC 的 $[-1, 1]$ ，其線性映射為

$$x_{\text{ndc}} = \frac{2x_{\text{near}} - (r + l)}{r - l}.$$

代入 $x_{\text{near}} = nx/(-z)$ ，並令剪裁座標的齊次分量 $w_c = -z$ ，可寫成

$$x_{\text{ndc}} = \frac{\frac{2n}{r-l}x + \frac{r+l}{r-l}z}{-z}.$$

因此剪裁座標第一分量應為

$$x_c = \frac{2n}{r-l}x + \frac{r+l}{r-l}z.$$

同理，

$$y_c = \frac{2n}{t-b}y + \frac{t+b}{t-b}z.$$

第三分量需要把 $z = -n$ 映至 $z_{\text{ndc}} = -1$ ，並把 $z = -f$ 映至 $z_{\text{ndc}} = 1$ 。設

$$z_c = Az + B, \quad w_c = -z,$$

則兩個端點條件分別給出

$$\frac{-An + B}{n} = -1, \quad \frac{-Af + B}{f} = 1.$$

第一式乘以 n 得 $-An + B = -n$ ，第二式乘以 f 得 $-Af + B = f$ 。兩式相減：

$$-A(f - n) = f + n,$$

所以

$$A = -\frac{f + n}{f - n}.$$

代回任一端點式，例如 $-An + B = -n$ ，得到

$$B = -n + An = -n - \frac{n(f+n)}{f-n} = -\frac{2fn}{f-n}.$$

因此

$$z_c = -\frac{f+n}{f-n}z - \frac{2fn}{f-n}, \quad w_c = -z.$$

合併三個分量，透視投影矩陣為

$$P = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}.$$

對稱視錐滿足 $l = -r$ 、 $b = -t$ ，因此

$$P = \begin{bmatrix} \frac{1}{a \tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta/2)} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}.$$

剪裁座標為

$$p_{\text{clip}} = Pp_{\text{cam}} = (x_c, y_c, z_c, w_c)^T.$$

裁切、NDC 與深度緩衝

相機前方的點有 $w_c = -z > 0$ 。透視除法得到

$$x_{\text{ndc}} = \frac{x_c}{w_c}, \quad y_{\text{ndc}} = \frac{y_c}{w_c}, \quad z_{\text{ndc}} = \frac{z_c}{w_c}.$$

點在剪裁體積內的條件為

$$-w_c \leq x_c \leq w_c, \quad -w_c \leq y_c \leq w_c, \quad -w_c \leq z_c \leq w_c, \quad w_c > 0.$$

在 $w_c > 0$ 的前提下，除以 w_c 後，條件等價於三個 NDC 分量都在 $[-1, 1]$ 。若 $w_c < 0$ ，除法會反轉不等式方向；因此不能忽略正 w_c 條件，也不能把相機後方的點照一般 NDC 範圍判斷為可見。

第三列係數的推導保證 $z = -n$ 映到 $z_{\text{ndc}} = -1$ ， $z = -f$ 映到 $z_{\text{ndc}} = 1$ 。本章深度緩衝值為

$$d = \frac{z_{\text{ndc}} + 1}{2},$$

因此近平面深度為 0、遠平面深度為 1。由投影矩陣可得

$$z_{\text{ndc}} = \frac{f+n}{f-n} + \frac{2fn}{(f-n)z}.$$

深度與相機距離並非線性關係。近平面附近通常分配到較多深度精度；近平面設得過小、遠平面設得過大，會令遠處深度更容易出現精度不足。實際精度也取決於深度緩衝的格式與位元數。

正交投影

正交投影不依深度縮小物體。若範圍為 $[l, r] \times [b, t]$ ，深度區間為 $z = -n$ 至 $z = -f$ ，可用

$$P_{\text{ortho}} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

其 $w_c = 1$ ，物體投影大小不隨深度改變。它仍有裁切範圍，但不產生透視的近大遠小。

NDC 到影像座標

對寬 W 、高 H 的影像，若像素原點位於左上角，連續影像座標為

$$u = \frac{x_{\text{ndc}} + 1}{2}W, \quad v = \frac{1 - y_{\text{ndc}}}{2}H.$$

v 軸反向，是因 NDC 的 Y 向上而影像座標的 v 向下。像素索引 (i, j) 的中心在 $(i + 0.5, j + 0.5)$ ；連續座標不一定是像素索引，實際涵蓋與取樣方式由光柵化規則決定。

逐步手算例題

例一：近平面、遠平面與中間深度

取 $n = 1$ 公尺、 $f = 10$ 公尺。深度係數為

$$A = -\frac{f+n}{f-n} = -\frac{11}{9}, \quad B = -\frac{2fn}{f-n} = -\frac{20}{9}.$$

由 $z_c = Az + B$ 、 $w_c = -z$ ：

在近平面 $z = -1$ ，

$$z_c = -\frac{11}{9}(-1) - \frac{20}{9} = -1, \quad w_c = 1.$$

所以 $z_{\text{ndc}} = -1$ 、深度 $d = 0$ 。

在遠平面 $z = -10$ ，

$$z_c = -\frac{11}{9}(-10) - \frac{20}{9} = 10, \quad w_c = 10.$$

所以 $z_{\text{ndc}} = 1$ 、深度 $d = 1$ 。

在 $z = -2$ ，

$$z_c = -\frac{11}{9}(-2) - \frac{20}{9} = \frac{2}{9}, \quad w_c = 2.$$

所以 $z_{\text{ndc}} = 1/9$ 、深度 $d = 5/9$ 。這不是線性的距離比例 $1/9$ 。

例二：由相機座標算到影像位置

令 $n = 1$ 、 $f = 10$ 、垂直視野角 $\theta = 90^\circ$ 、長寬比 $a = 2$ 。此時 $\tan(\theta/2) = 1$ ，投影矩陣的 x, y 縮放係數分別是 $1/2$ 與 1 。

取 $p_{\text{cam}} = (1, 0, -2, 1)^T$ ，得到剪裁座標

$$(x_c, y_c, z_c, w_c) = \left(\frac{1}{2}, 0, \frac{2}{9}, 2\right).$$

透視除法後

$$(x_{\text{ndc}}, y_{\text{ndc}}, z_{\text{ndc}}) = \left(\frac{1}{4}, 0, \frac{1}{9}\right).$$

深度緩衝值為 $5/9$ 。若影像尺寸為 800×400 ，連續影像位置為

$$u = \frac{1 + 1/4}{2}(800) = 500, \quad v = \frac{1 - 0}{2}(400) = 200.$$

結果是連續座標 $(500, 200)$ ，不是指定的像素索引。要確定哪個像素取樣此點，還須套用像素中心與光柵化規則。

近平面線段裁切

令 $n = 1$ ，線段端點深度為 $z_A = -0.5$ 、 $z_B = -2$ 。以相機前方距離 $s = -z$ 表示，交點內插參數為

$$\lambda = \frac{n - s_A}{s_B - s_A} = \frac{1 - 0.5}{2 - 0.5} = \frac{1}{3}.$$

對三維端點向量 A, B ，交點為

$$C = A + \lambda(B - A).$$

三角形逐邊測試近平面時，在跨越平面的邊上建立交點，再保留可見側的多邊形。輸出通常是三角形、四邊形或空集合；頂點恰落在平面上時，實作也可能產生重複頂點或退化面，需在後續處理中辨識。若頂點帶有 UV、顏色等屬性，也要用同一個邊參數內插。實際渲染器通常在剪裁空間處理所有剪裁平面，不先對近平面外的頂點做透視除法。

實作與程式

以下完整範例只使用 Python 標準庫。它建立右手 OpenGL 式透視矩陣，檢查近平面與遠平面的深度端點、剪裁範圍，以及線段的近平面裁切。矩陣以巢狀串列儲存，數學慣例仍是矩陣乘直向量。

```
import math

def perspective_rh_opengl(fovy, aspect, near, far):
    """右手相機看向 -Z; NDC z 範圍為 [-1, 1]。"""
    if not (0.0 < fovy < math.pi):
        raise ValueError("fovy 必須介於 0 與 pi 之間")
    if aspect <= 0.0 or near <= 0.0 or far <= near:
        raise ValueError("aspect、near、far 不符合條件")

    q = 1.0 / math.tan(fovy / 2.0)
    return [
        [q / aspect, 0.0, 0.0, 0.0],
        [0.0, q, 0.0, 0.0],
        [0.0, 0.0, -(far + near) / (far - near),
         -(2.0 * far * near) / (far - near)],
        [0.0, 0.0, -1.0, 0.0],
    ]

def mat_vec(matrix, vector):
    return [
        sum(matrix[r][c] * vector[c] for c in range(4))
        for r in range(4)
    ]

def ndc_and_depth(matrix, point):
    clip = mat_vec(matrix, [point[0], point[1], point[2], 1.0])
    w = clip[3]
    if w <= 0.0:
        raise ValueError("點不在相機前方，不能做本例透視除法")
    ndc = tuple(clip[i] / w for i in range(3))
    depth = (ndc[2] + 1.0) / 2.0
    return clip, ndc, depth

def inside_clip(clip, eps=1e-12):
    x, y, z, w = clip
    return (
        w > 0.0
```

```

        and -w - eps <= x <= w + eps
        and -w - eps <= y <= w + eps
        and -w - eps <= z <= w + eps
    )

def clip_segment_near(a, b, near):
    """輸入端點 (x, y, z)，保留 z <= -near 的線段部分。"""
    a_in = a[2] <= -near
    b_in = b[2] <= -near

    if not a_in and not b_in:
        return None
    if a_in and b_in:
        return a, b

    t = (-near - a[2]) / (b[2] - a[2])
    c = tuple(a[i] + t * (b[i] - a[i]) for i in range(3))
    return (a, c) if a_in else (c, b)

def main():
    near, far = 1.0, 10.0
    matrix = perspective_rh_opengl(math.pi / 2.0, 2.0, near, far)

    for z, expected_depth in [(-near, 0.0), (-far, 1.0)]:
        clip, ndc, depth = ndc_and_depth(matrix, (0.0, 0.0, z))
        assert inside_clip(clip)
        assert abs(depth - expected_depth) < 1e-12
        assert abs(ndc[2] - (2.0 * expected_depth - 1.0)) < 1e-12

    outside = mat_vec(matrix, [20.0, 0.0, -2.0, 1.0])
    assert not inside_clip(outside)

    clipped = clip_segment_near(
        (0.0, 0.0, -0.5), (1.5, 0.0, -2.0), near
    )
    assert clipped is not None
    assert abs(clipped[0][2] + near) < 1e-12
    assert clipped[1] == (1.5, 0.0, -2.0)

    assert clip_segment_near(
        (0.0, 0.0, -0.2), (1.0, 0.0, -0.7), near
    ) is None

    print("near/far, side clipping, and near-plane tests passed")

if __name__ == "__main__":
    main()

```

線段函式只處理近平面，不是完整三角形裁切器。視錐的邊界其實是由六個半空間共同定義：近平面、遠平面，以及左右上下四個側面。程式中的 `inside_clip` 把這些條件寫成三組不等式

$$-w_c \leq x_c \leq w_c, \quad -w_c \leq y_c \leq w_c, \quad -w_c \leq z_c \leq w_c.$$

其中 $x_c \geq -w_c$ 對應左平面、 $x_c \leq w_c$ 對應右平面、 $y_c \geq -w_c$ 對應下平面、 $y_c \leq w_c$ 對應上平面、 $z_c \geq -w_c$ 對應近平面、 $z_c \leq w_c$ 對應遠平面。任一條不等式不成立，該點就在視錐外。這些比較都建立在 $w_c > 0$ 的前提下；若 $w_c \leq 0$ ，點位於相機後方或與相機共平面，整個判斷失去意義，必須先排除。

本章範例刻意只示範近平面這一個條件的裁切，因此即使線段兩端都通過近平面測試，它仍可能落在左右、上下或遠平面之外——下一節的 `outside` 斷言就在說明這件事：`inside_clip` 會拒絕側向超出的點，但 `clip_segment_near` 不會。完整渲染流程還須對其餘五個半空間逐一裁切，並同步內插交點上的所有頂點屬性（UV、法線、顏色等），且在頂點恰落於平面上時處理重複或退化情形。程式使用的 10^{-12} 是此簡單案例的容差，不是通用常數；場景尺度與浮點格式改變時，應重新選擇容差。

測試與預期結果

程式斷言檢查：

- 近平面深度映到 0，遠平面深度映到 1。
- 超出視錐側邊的點不通過剪裁條件。
- 跨越近平面的線段交點位於 $z = -n$ 。
- 兩端都在近平面外的線段裁切結果為空。

若讀者執行程式，預期最後一行會印出：

```
near/far, side clipping, and near-plane tests passed
```

這是依公式與程式可推得的預期結果，不表示作者已執行程式或驗證特定環境。

除錯與常見陷阱

- **相機前方符號錯誤**：本章相機看向 $-Z$ ，可見點的 $z < 0$ ，因此 $w_c = -z > 0$ 。
- **忘記透視除法**：剪裁座標還不是 NDC。直接以 x_c, y_c 映射像素會造成比例錯誤。
- **混用深度慣例**：本章 NDC 深度範圍是 $[-1, 1]$ 。改用另一 API 時，投影矩陣、裁切條件與深度映射都須配套調整。
- **未裁切就對近平面外點做除法**：三角形跨越近平面時，先裁切，再透視除法，避免不穩定的投影座標。
- **忽略 w_c 的正負**：當 $w_c < 0$ ，除法會反轉不等式方向；不能直接用相機前方點的判斷方式處理。
- **把深度當距離**：深度是非線性投影量。要還原相機距離，必須依同一組投影參數反算。
- **忽略緩衝格式**：深度精度也受深度緩衝格式與位元數影響。
- **影像上下顛倒**：NDC 的 Y 向上，影像 v 向下，映射時要反轉。
- **連續影像座標誤當像素索引**：需依像素中心與光柵化規則決定取樣位置。

養殖數位分身案例

假設合成池景長 8 公尺、寬 4 公尺，魚群活動深度約 0.5 至 3 公尺。將池體與魚群轉到相機座標後，依可見範圍選擇視野角。近平面要小於最近需要成像物體的距離，但不應無故接近零；遠平面需涵蓋最遠的池體構件或背景物件，不必無限制延伸。

三角形若跨越近平面，先算交點、裁切並內插 UV 等頂點屬性，再做透視除法與光柵化。深度緩衝值可用於遮擋比較，但不是以公尺為單位的距離。

若輸出合成深度標註，資料應記錄 NDC 慣例、近平面、遠平面、深度緩衝格式，以及深度值是否已反算成距離。單獨保存灰階深度圖而不保存這些設定，無法確定其尺度與意義。結果只描述合成場景及相機設定，不是對真實魚體位置或水下光學現象的驗證。

習題

1. 手算：深度映射

取 $n = 0.5$ 公尺、 $f = 8$ 公尺。求 $z = -0.5$ 、 $z = -8$ 、 $z = -2$ 時的 z_{ndc} 與深度緩衝值。

2. 程式測試：端點與近平面線段

新增測試，確認 $z = -n$ 與 $z = -f$ 的點通過剪裁範圍檢查，深度分別為 0、1。再測兩端都滿足 $z > -n$ 的線段，確認裁切結果為空。

3. 反例與除錯：除法後檢查 NDC

相機前方的點位於 $z = -0.5$ ，近平面為 $n = 1$ 。計算其 z_{ndc} ，說明為何不可能同時讓三個 NDC 分量都在 $[-1, 1]$ 。再說明當 $w_c < 0$ 時，只看除法後 NDC 的風險。

4. 整合應用：選擇近平面

合成池景中最近魚距相機約 0.8 公尺，最遠池壁約 12 公尺。提出一組合理的 n, f ，說明設定原則；再說明把 n 改成 0.001 公尺可能造成什麼影響。

習題解答

1. 手算：深度映射

使用

$$z_{\text{ndc}} = \frac{f+n}{f-n} + \frac{2fn}{(f-n)z}, \quad d = \frac{z_{\text{ndc}} + 1}{2}.$$

本題 $(f+n)/(f-n) = 17/15$ ， $2fn/(f-n) = 16/15$ 。

- $z = -0.5$ ： $z_{\text{ndc}} = -1$ ， $d = 0$ 。
- $z = -8$ ： $z_{\text{ndc}} = 1$ ， $d = 1$ 。
- $z = -2$ ： $z_{\text{ndc}} = 17/15 - 8/15 = 3/5$ ， $d = 4/5$ 。

2. 程式測試：端點與近平面線段

用程式中的 `ndc_and_depth` 與 `inside_clip` 測試 $z = -n, -f$ ；預期深度分別為 0, 1。當 $n = 1$ ，線段端點可用 $(0, 0, -0.2)$ 與 $(1, 0, -0.7)$ ；兩端都滿足 $z > -1$ ，所以 `clip_segment_near` 預期回傳 `None`。這些測試涵蓋近平面與程式明列的剪裁條件，不代表已測完整多邊形裁切器。

3. 反例與除錯：除法後檢查 NDC

投影矩陣給出

$$z_{\text{ndc}} = \frac{f+n}{f-n} + \frac{2fn}{(f-n)z}.$$

令 $z = -0.5, n = 1$ ，則

$$z_{\text{ndc}} = \frac{f+1}{f-1} - \frac{4f}{f-1} = \frac{-3f+1}{f-1}.$$

對 $f > 1$ ，此值小於 -1 ，所以三個 NDC 分量不可能同時落在 $[-1, 1]$ 。對 $w_c > 0$ 且使用同一投影矩陣，NDC 範圍檢查與齊次剪裁不等式等價；不能說它會漏掉本例的近平面條件。

若 $w_c < 0$ ，除以負值會反轉不等式方向。若忽略 $w_c > 0$ 、直接套用可見點的判斷方式，可能把相機後方的點誤判為可見。

4. 整合應用：選擇近平面

例如取 $n = 0.5$ 公尺、 $f = 12$ 公尺。最近魚在近平面之後，最遠池壁在遠平面內；若相機會移動，可依場景保留合理餘量，但不必把近平面設得接近零。

將 n 降到 0.001 公尺會讓深度分布更不均勻；在相同緩衝格式下，遠處較容易遇到深度精度不足與遮擋次序錯誤。實際影響仍取決於深度緩衝格式與整個場景的深度範圍。

本章小結

透視投影以 $w_c = -z$ 產生近大遠小，透視除法將剪裁座標轉為 NDC。本章 OpenGL 式 NDC 深度為 $[-1, 1]$ ，深度緩衝值為 $(z_{\text{ndc}} + 1)/2$ 。近平面與遠平面的兩個端點條件可解出投影矩陣深度列的係數。幾何跨越近平面時，應在透視除法前裁切並同步內插屬性。正交投影則令 $w_c = 1$ ，不依深度縮放物體。深度不是距離的線性編碼；其精度受投影範圍與緩衝格式影響。

參考來源

以下列出延伸閱讀；列出來源不表示本章已逐條外部驗證所有論點。

- [G1] *Physically Based Rendering, Fourth Edition*，Transformations：https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] LearnOpenGL，Transformations：<https://learnopengl.com/Getting-started/Transformations>

第 6 章 三角形光柵化與插值

學習目標與先備知識

讀完本章，你應該能：

1. 用**邊函數**（edge function）在離散像素格上判定一個點是否落在三角形內部，並說明為何它與有號面積等價。
2. 由邊函數推導**重心座標** $\lambda_A, \lambda_B, \lambda_C$ ，並用它們在三角形內插值純量屬性（深度、UV、顏色、法線）。
3. 說明 **top-left 規則**在共享邊上的作用：兩個相鄰三角形對同一條邊僅有一方把該邊算作內部，避免重複塗寫或空隙。
4. 用 **z-buffer** 做逐像素可見性判定，並區分 NDC 深度 $z_{\text{ndc}} \in [-1, 1]$ 與深度緩衝值 $z_w = (z_{\text{ndc}} + 1)/2$ 。
5. 區分兩種插值：**螢幕空間線性插值**（適用於 z_{ndc} 這類本身已是 $1/w$ 仿射函數的量）與**透視校正插值** $\frac{\sum_i \lambda_i a_i / w_i}{\sum_i \lambda_i / w_i}$ （適用於視空間線性量如 UV、顏色、法線）。

先備（Volume I）：向量、內積、外積、 2×2 行列式、矩陣乘法次序。本章沿用共同約定：

- 右手世界系， $+X$ 右、 $+Y$ 上、 $+Z$ 朝觀者；相機看向 $-Z$ 。
- 像素原點在左上，像素 (u, v) 中心 $(u + 0.5, v + 0.5)$ ，螢幕 Y 向下（與世界 Y 向上分開）。
- 主線： $p_{\text{clip}} = P V M p$ ，NDC 用 OpenGL 式 $z \in [-1, 1]$ 、 $\text{near} > 0$ 、 $\text{far} > \text{near}$ 。

我們先處理「螢幕空間」（screen space）的 2D 問題：給定三角形三個頂點的像素座標、 z_{ndc} 、透視除法因子 w （即 clip 空間的 w ）與各頂點屬性，決定每個像素是否被覆蓋、以及該像素的插值屬性是什麼。

問題與直覺

3D 幾何要進入一張離散的 $W \times H$ 畫素陣列，必經兩步：先變換到 NDC，再做視埠映射，得到以像素為單位的頂點座標；接著把「覆蓋面積有限但連續」的三角形離散化成有限像素集合。這個離散化的核心問題是：

- **哪些像素屬於這個三角形？** 若用「三角形重心與點關係」直接做浮點比較，會產生誤差與邊界不一致。
- **每個像素內插到什麼屬性？** 深度、UV、顏色在三角形內是連續的，必須由三頂點值按位置比例內插。
- **為什麼不能一律用螢幕比例？** 透視投影把近處壓縮得更小，螢幕上等距的兩點在 3D 中不等距；UV、顏色、法線這類「視空間線性」屬性必須補償這個非均勻。

三個問題分別對應本章核心：**邊函數 + 重心座標**、**top-left 規則**、**深度線性插值 vs. 其他屬性的透視校正插值**；再加上 z-buffer 完成可見性。

數學與幾何推導

有號面積與邊函數

設螢幕座標下三頂點 $A = (x_A, y_A)$ 、 B 、 C ，定義

$$\text{area2}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

這是三角形有號面積的兩倍，對應 Volume I 的 2D 外積。對任意點 P ，依序對三條有向邊 $A \rightarrow B$ 、 $B \rightarrow C$ 、 $C \rightarrow A$ 定義邊函數

$$e_{AB}(P) = (B_x - A_x)(P_y - A_y) - (B_y - A_y)(P_x - A_x),$$

$e_{BC}(P)$ 、 $e_{CA}(P)$ 類同。在螢幕座標 Y 向下時，定義正繞序為 $\text{area2} > 0$ 的頂點順序（此對應數學座標 Y 向上時的逆時針，視覺上為順時針）。在此正繞序下，內部所有點的三個邊函數皆 ≥ 0 。邊函數是 P 的仿射函數（線性部分+常數），因此可作為三角形上的重心係數。

重心座標

面積比即重心座標：

$$\lambda_A = \frac{e_{BC}(P)}{\text{area2}(A, B, C)}, \quad \lambda_B = \frac{e_{CA}(P)}{\text{area2}(A, B, C)}, \quad \lambda_C = \frac{e_{AB}(P)}{\text{area2}(A, B, C)}.$$

由定義立得 $\lambda_A + \lambda_B + \lambda_C = 1$ 。這組 λ_i 是屏幕空間上的量。

Top-left 規則

逐像素比較浮點邊函數，兩個相鄰三角形在共享邊上會得到「同時微小正」或「同時微小負」；只要有任一方把邊界像素算入，就會有重複；若雙方都不算，就會留縫。標準解法：只把「左上邊」的 $e = 0$ 當作內部。

在螢幕座標（Y 向下）下，定義邊 $a \rightarrow b$ 為**左上邊**當且僅當

$$(b_y - a_y) < 0 \quad \text{或} \quad (b_y = a_y \wedge b_x > a_x).$$

即「向上」或「水平向右」。若相鄰兩三角形在共享邊的方向相反（它們必須如此，才能各自為正繞序），則這條邊只會在一方被判定為左上邊，像素恰好歸屬其中一方。

Z-buffer

每個像素保留目前最小的**深度緩衝值** z_w 。NDC 深度 z_{ndc} 與 z_w 的關係依管線而定；本章採 OpenGL 式 NDC $z \in [-1, 1]$ ，遠平面映射到 1、近平面到 -1 ， z_{ndc} 越小越近：

$$z_w = \frac{z_{\text{ndc}} + 1}{2}, \quad z_w \in [0, 1], \text{ 越小越近.}$$

像素通過深度測試時 ($z_w < z_{\text{buffer}}[v][u]$)，寫入新 z_w 並覆蓋顏色。**關鍵性質**： z_{ndc} 是 $1/w$ 的仿射函數（推導見下小節），所以 z_{ndc} 在螢幕空間以 λ_i 線性插值即為正確值，不需要再做透視校正。

透視校正插值與深度屬性的特殊地位

設三頂點在 clip 空間的 $w_i > 0$ （即 OpenGL 右手相機前方點 $-z_{\text{view},i}$ ）。視空間線性屬性 a 的正確插值為

$$a_P = \frac{\lambda_A a_A / w_A + \lambda_B a_B / w_B + \lambda_C a_C / w_C}{\lambda_A / w_A + \lambda_B / w_B + \lambda_C / w_C}$$

其中 λ_i 為螢幕空間重心座標。這是 Volume I 尚未觸及的結論，但可由「視空間位置在螢幕上的投影是 λ 的透視方式」直接導出。特別地，代入 $a_i = 1$ 得

$$1 = \frac{\sum \lambda_i / w_i}{\sum \lambda_i / w_i},$$

而代入 $a = 1/w$ 得 $\sum \lambda_i / w_i$ 為其分子分母之分子。更直接的等式是：

$$\frac{1}{w_P} = \lambda_A \frac{1}{w_A} + \lambda_B \frac{1}{w_B} + \lambda_C \frac{1}{w_C}.$$

也就是說， $\frac{1}{w}$ 的螢幕空間重心插值即為 $\frac{1}{w_P}$ 。

投影矩陣的 NDC 深度是 $1/w$ 的仿射函數。以 $\text{near} = n$ 、 $\text{far} = f$ 的 OpenGL 右手投影為例：

$$z_{\text{ndc}} = \frac{f + n}{f - n} - \frac{2fn}{f - n} \cdot \frac{1}{w},$$

代入 $w = n$ 得 $z_{\text{ndc}} = -1$ ，代入 $w = f$ 得 $z_{\text{ndc}} = 1$ 。因為 z_{ndc} 是 $1/w$ 的仿射函數，而 $1/w$ 本身以 $\sum \lambda_i / w_i$ 作螢幕空間線性插值，故

$$z_{\text{ndc},P} = \lambda_A z_{\text{ndc},A} + \lambda_B z_{\text{ndc},B} + \lambda_C z_{\text{ndc},C}.$$

結論：螢幕空間重心座標 λ_i 對 UV、顏色、法線等視空間線性屬性要帶 $1/w$ 分母做**透視校正插值**；對 z_{ndc} （或 z_w ）則**直接線性插值即為正確**。實作時若對 z_{ndc} 誤用透視校正公式，會得到偏離正確值的深度，在斜面或大 w 差場景下遮擋會翻轉。

逐步手算例題

例 1（邊函數與重心座標）。螢幕座標 Y 向下，三角形 $A = (0, 0)$ 、 $B = (4, 0)$ 、 $C = (0, 4)$ ，像素中心 $P = (1.5, 1.5)$ 。

$$\text{area2}(A, B, C) = (4)(4) - (0)(0) = 16 > 0。$$

$$e_{BC}(P) = (C_x - B_x)(P_y - B_y) - (C_y - B_y)(P_x - B_x) = (-4)(1.5) - (4)(-2.5) = 4,$$

$$e_{CA}(P) = (A_x - C_x)(P_y - C_y) - (A_y - C_y)(P_x - C_x) = (0)(-2.5) - (-4)(1.5) = 6,$$

$$e_{AB}(P) = (4)(1.5) - (0)(1.5) = 6.$$

三者皆 > 0 ，故 P 在內部。 $\lambda_A = 4/16 = 0.25$ ， $\lambda_B = 6/16 = 0.375$ ， $\lambda_C = 6/16 = 0.375$ ，和為 1。若頂點 UV 為 $A : (0, 0)$ 、 $B : (1, 0)$ 、 $C : (0, 1)$ ，則 P 處 $u = 0.375$ 、 $v = 0.375$ 。

例 2（屬性透視校正插值）。令 $w_A = 2$ 、 $w_B = 4$ 、 $w_C = 8$ ；螢幕重心 $\lambda = (0.25, 0.375, 0.375)$ ；屬性 $a_A = 1$ 、 $a_B = 2$ 、 $a_C = 4$ 。

分子： $0.25 \cdot 1/2 + 0.375 \cdot 2/4 + 0.375 \cdot 4/8 = 0.125 + 0.1875 + 0.1875 = 0.5$ 。

分母： $0.25 \cdot 0.5 + 0.375 \cdot 0.25 + 0.375 \cdot 0.125 = 0.125 + 0.09375 + 0.046875 = 0.265625$ 。

$$a_P = 0.5/0.265625 \approx 1.88235。$$

純螢幕線性（錯誤做法）： $0.25 \cdot 1 + 0.375 \cdot 2 + 0.375 \cdot 4 = 2.5$ 。兩者相差約 0.62，對 UV 或顏色都是肉眼可見的偏移。

例 3（ z_{ndc} 的插值：線性 vs. 透視校正）。設 $\text{near} = 1$ 、 $\text{far} = 2$ ， $w_A = 1$ 、 $w_B = 2$ ， $\lambda_A = \lambda_B = 0.5$ 。則 $z_{\text{ndc},A} = -1$ 、 $z_{\text{ndc},B} = 1$ 。

- 螢幕線性（正確）： $0.5 \cdot (-1) + 0.5 \cdot 1 = 0$ 。
- 透視校正（錯誤用法）： $\frac{0.5 \cdot (-1)/1 + 0.5 \cdot 1/2}{0.5/1 + 0.5/2} = \frac{-0.25}{0.75} \approx -0.333$ 。

以 $1/w$ 檢核： $1/w_P = 0.5/1 + 0.5/2 = 0.75$ ， $w_P = 4/3$ ；由投影公式 $z_{\text{ndc}} = 3 - 4/w_P = 3 - 3 = 0$ ，與線性插值一致。錯誤用法把中間像素誤判為更靠近相機，在斜面或大 w 差場景會造成遮擋錯誤。

實作與程式

以下程式只依賴 **Python 3.10+ 標準庫**，讀者可自行複製、執行。

```
# 檔案：raster.py 只依賴標準庫
import math

def edge(ax, ay, bx, by, px, py):
    """邊函數：正值為有號面積正側；Y向下時視覺上位於有向邊右側。"""
    return (bx - ax) * (py - ay) - (by - ay) * (px - ax)

def is_top_left(ax, ay, bx, by):
    """是否為左上邊：朝上，或朝右且水平。"""
    dy, dx = by - ay, bx - ax
    return (dy < 0.0) or (dy == 0.0 and dx > 0.0)
```

```

class Framebuffer:
    """attr_dim 為屬性向量維度；write_ppm 只取前三個分量當 RGB。"""
    def __init__(self, W, H, attr_dim=5):
        self.W, self.H, self.attr_dim = W, H, attr_dim
        self.attr = [[[0.0] * attr_dim for _ in range(W)] for _ in range(H)]
        self.depth = [[1.0] * W for _ in range(H)] # 越小越近

    def write_ppm(self, path):
        with open(path, "wb") as f:
            f.write(f"P6\n{self.W} {self.H}\n255\n".encode("ascii"))
            for v in range(self.H):
                for u in range(self.W):
                    px = self.attr[v][u]
                    for c in (0, 1, 2):
                        x = max(0.0, min(1.0, px[c]))
                        # Note: 1/2.2 is an approximation, not precise sRGB; see Ch 13
                        f.write(bytes([int(round(255.0 * x ** (1.0 / 2.2))))))

def raster_triangle(fb, verts):
    """verts = [(x_px, y_px, z_ndc, w_clip, [attrs...]), ...] 共 3 項。

    attrs 為視空間線性量，用透視校正插值；
    z_ndc 為 1/w 的仿射函數，用螢幕線性插值即為正確。
    """
    (x0, y0, z0, w0, a0), (x1, y1, z1, w1, a1), (x2, y2, z2, w2, a2) = verts

    area2 = edge(x0, y0, x1, y1, x2, y2)
    if area2 == 0.0:
        return
    if area2 < 0.0:
        (x0, y0, z0, w0, a0), (x1, y1, z1, w1, a1) = \
            (x1, y1, z1, w1, a1), (x0, y0, z0, w0, a0)
        area2 = -area2

    if w0 <= 0.0 or w1 <= 0.0 or w2 <= 0.0:
        return # 近平面之後須先裁切

    minx = max(0, math.floor(min(x0, x1, x2)))
    maxx = min(fb.W - 1, math.ceil(max(x0, x1, x2)))
    miny = max(0, math.floor(min(y0, y1, y2)))
    maxy = min(fb.H - 1, math.ceil(max(y0, y1, y2)))

    inv0, inv1, inv2 = 1.0 / w0, 1.0 / w1, 1.0 / w2
    na = len(a0)

    for v in range(miny, maxy + 1):
        py = v + 0.5
        row_a, row_d = fb.attr[v], fb.depth[v]
        for u in range(minx, maxx + 1):
            px = u + 0.5
            e0 = edge(x1, y1, x2, y2, px, py) # 對 BC 邊
            e1 = edge(x2, y2, x0, y0, px, py) # 對 CA 邊
            e2 = edge(x0, y0, x1, y1, px, py) # 對 AB 邊

            inside = e0 > 0.0 and e1 > 0.0 and e2 > 0.0
            if not inside:
                if e0 < 0.0 or e1 < 0.0 or e2 < 0.0:
                    continue

```

```

        if e0 == 0.0 and not is_top_left(x1, y1, x2, y2):
            continue
        if e1 == 0.0 and not is_top_left(x2, y2, x0, y0):
            continue
        if e2 == 0.0 and not is_top_left(x0, y0, x1, y1):
            continue

    la, lb, lc = e0 / area2, e1 / area2, e2 / area2

    # 深度：螢幕空間線性插值 (z_ndc 是 1/w 的仿射函數)
    z_ndc = la * z0 + lb * z1 + lc * z2
    z_w = 0.5 * (z_ndc + 1.0)
    if z_w >= row_d[u]:
        continue

    # 其他屬性：透視校正插值
    denom = la * inv0 + lb * inv1 + lc * inv2
    if denom == 0.0:
        continue
    row_d[u] = z_w
    pa = row_a[u]
    for k in range(na):
        pa[k] = (la * a0[k] * inv0 + lb * a1[k] * inv1
                + lc * a2[k] * inv2) / denom

```

重點：深度與其他屬性的插值公式不同。 z_{ndc} 已是 $1/w$ 的仿射函數，螢幕線性插值即正確；UV、顏色、法線是視空間線性量，必須用帶分母的透視校正公式。若為避免每像素除法，可在頂點階段把屬性預乘 $1/w$ ，讓片段階段只做兩次線性插值再相除，結果等價。

測試與預期結果

測試 1：共享邊無重複無空隙。在 16×16 framebuffer 上，畫兩個共斜邊三角形

$$T_1: A_1 = (0, 0), B_1 = (16, 0), C_1 = (0, 16); \quad T_2: A_2 = (0, 16), B_2 = (16, 0), C_2 = (16, 16).$$

兩者共享邊 $(16, 0)-(0, 16)$ （直線 $x + y = 16$ ）。在覆蓋判定通過後、深度測試之前對每個像素計數，預期所有像素計數 ≤ 1 ，且 $x + y < 16$ 與 $x + y > 16$ 的像素各被其中一方覆蓋一次、邊上的像素亦只被一方覆蓋一次。若關掉 top-left 條件改成「 $e \geq 0$ 即內部」，共享邊上的像素會被兩個三角形各寫一次，計數變 2。

測試 2：遮擋順序。兩個交疊三角形 T_{near} 、 T_{far} ， z_{ndc} 分別為 -0.5 與 $+0.5$ ，先畫遠再畫近。預期重疊區最終顏色為近者；顛倒繪製順序結果相同。若把近者錯用透視校正公式插值 z_{ndc} ，斜面或大 w 差時遮擋會翻轉（見例 3）。

測試 3：斜面 UV 透視校正。三角形一頂點 $w = 1$ 、另兩頂點 $w = 10$ ，UV 對應到較遠端；分別用校正與未校正公式算同一像素的 UV。預期校正後 UV 偏向遠端的真實 3D 位置，差異隨 w 比值增大。

測試 4：退化輸入。三頂點共線或兩點重合時 $\text{area2}==0$ 應直接返回、不寫任何像素；頂點 $w \leq 0$ 亦直接丟棄（已在近平面之後，須先裁切）。

以上預期數字或比例需讀者自行實作後量測，本章未執行。

除錯與常見陷阱

- **像素中心取整方式**：若用整數 (u, v) 當像素中心而非 $(u + 0.5, v + 0.5)$ ，覆蓋區域會偏向一邊；連續排兩個三角形時縫隙或重疊會出現在整數點上。
- **Y 軸方向混淆**：世界 Y 向上、影像 Y 向下。務必在邊函數符號約定前先固定，否則三角形的內外判斷會整個反向。
- **Top-left 條件寫反**：以本節定義，一條邊的兩方向只有一方滿足左上條件；驗證方法是用測試 1。
- **深度插值公式選錯**： z_{ndc} 在螢幕空間對 λ_i 線性；對 UV 等視空間量才需透視校正。把兩者混用是最常見的深度錯誤來源。
- **z 值種類混淆**： $z_{\text{ndc}} \in [-1, 1]$ 、 $z_w \in [0, 1]$ 、 $1/w$ 、 $\log z$ 是四種不同表達，混用會導致遮擋方向顛倒或遠近翻轉。務必在 `Framebuffer` 內只使用一種，其餘在寫入前轉換。
- **透視校正的分母**：若頂點屬性已預乘 $1/w$ ，分母仍需一併插值。漏掉分母會變成「透視校正只做一半」的結果，錯誤量與未校正同量級，不易一眼察覺。
- **退化三角形**：頂點共線或極接近共線時 `area2` 可能為浮點零或極小，導致重心座標爆掉。需要 ϵ 或提前丟棄。 ϵ 必須依場景尺度與像素單位設定，不能當作物理閾值。
- **插值精度累積**：對大三角形， $e_{AB}(P)$ 用每像素重新計算比從第一像素遞增更穩定；遞增做法須注意浮點漂移，必要時每列重算。
- **裁切前光柵化**：本章只處理 NDC 內三角形。若三角形有頂點在近平面之後 ($w \leq 0$)，必須先做近平面裁切（第 5 章），否則 $1/w$ 會發散。

養殖數位分身案例

養殖池的視覺化常以低多邊形魚體表示。一個魚身側面最小可只用兩個共邊三角形圍出。假設魚身側面三頂點在螢幕座標與 clip 空間 w 為

$$T_1: A_1 = (20, 10), B_1 = (140, 40), C_1 = (30, 60); \quad w = (4.0, 6.0, 4.5),$$

$$T_2: A_2 = (140, 40), B_2 = (150, 70), C_2 = (30, 60); \quad w = (6.0, 6.5, 4.5).$$

屬性維度 5： (R, G, B, U, V) ，UV 在 T_1 為 $(0, 0) \rightarrow (1, 0) \rightarrow (0, 1)$ 、在 T_2 為 $(1, 0) \rightarrow (1, 1) \rightarrow (0, 1)$ ，前三個分量放對應的 checker 顏色。

當魚身靠近相機、 w 差異大時，若不使用透視校正，靠近相機一側的條紋會被拉長；若使用校正，條紋在螢幕上的間距符合近大遠小的直觀。這個例子說明：**幾何正確但插值公式選錯的渲染，仍會給出誤導性的視覺證據**；魚群尺寸、位置與姿態的量化分析須建立在插值正確的管線上。合成資料並非生物學量測；UV 與顏色只是外觀，不代表魚體健康、行為或任何生態結論。

模型魚身可以只用少數三角形；細節靠 UV 與法線貼圖增加。三角形數量、UV 接縫、z-buffer 精度三者是同一個權衡：越多三角形越精細，但共享邊與深度精度問題越需謹慎處理。

習題

習題 1 (手算)。螢幕座標 Y 向下。三角形 $A = (2, 1)$ 、 $B = (7, 1)$ 、 $C = (2, 6)$ 。(a) 求 $\text{area2}(A, B, C)$ 。(b) 對像素中心 $P = (3.5, 2.5)$ 求三個邊函數與重心座標，判斷是否落在內部。(c) 若頂點 UV 為 $A : (0, 0)$ 、 $B : (1, 0)$ 、 $C : (0, 1)$ ，求 P 的螢幕線性 UV；再用 $w_A = 1$ 、 $w_B = 3$ 、 $w_C = 5$ 求透視校正 UV，並報告兩者差異。

習題 2 (程式測試)。將 `raster_triangle` 包一層計數器，繪製下列三個三角形於 32×32 framebuffer (皆以 $w = 1$)： $T_1 = (0, 0), (32, 0), (0, 32)$ ； $T_2 = (0, 0), (0, 32), (32, 32)$ ； $T_3 = (16, 0), (32, 16), (16, 32)$ 。將計數器放在覆蓋判定通過後、深度測試之前；本題計的是覆蓋次數，不是最終顏色寫入次數。報告 (a) 每個像素覆蓋次數的最大值；(b) 覆蓋像素集合的形狀與次數分布；(c) 若移掉 top-left 條件改成「 $e \geq 0$ 即內部」，最大值與分布如何改變。三角形內部可重疊，不能用共享邊規則推論每個像素最多覆蓋一次。

習題 3 (反例／除錯)。以下片段聲稱能正確插值深度：

```
z = (la * z0 / w0 + lb * z1 / w1 + lc * z2 / w2) / (la/w0 + lb/w1 + lc/w2)
if z < depth[u]:
    depth[u] = z_w_from(z)
```

(a) 指出它在什麼條件下出錯。(b) 給一個最小反例：視空間深度 w 差距很大時，螢幕上某像素的 z_{ndc} 被算錯，導致遮擋判斷偏離正確幾何。(c) 修正為正確版本。

習題 4 (整合應用)。用本章 `Framebuffer`、`raster_triangle` 與任一簡單寫 PPM 函式，繪製 16×16 的兩三角形魚身輪廓（見養殖案例），屬性維度 5（前三位為 checker 顏色、後兩位為 UV），把 UV 寫入第二張 PPM 的 R、G 通道（可用 `attr_dim=5` 但把 `write_ppm` 改成取 channel 3、4 再補零寫 B）。設計一個驗證步驟：抽樣兩條共享邊法線方向三格像素，檢查 UV 在兩側是否連續（差值小於 $1/8$ ）？說明若未做透視校正，這項檢查會在何處失敗。

習題解答

習題 1。(a) $\text{area2} = (7 - 2)(6 - 1) - (1 - 1)(2 - 2) = 5 \cdot 5 = 25 > 0$ 。

(b) $e_{BC}(P) = (2 - 7)(2.5 - 1) - (6 - 1)(3.5 - 7) = (-5)(1.5) - (5)(-3.5) = -7.5 + 17.5 = 10$ 。
 $e_{CA}(P) = (2 - 2)(2.5 - 6) - (1 - 6)(3.5 - 2) = 0 - (-5)(1.5) = 7.5$ 。 $e_{AB}(P) = (7 - 2)(2.5 - 1) - (1 - 1)(3.5 - 2) = 5 \cdot 1.5 = 7.5$ 。和 $= 25 = \text{area2}$ 。 $\lambda_A = 10/25 = 0.4$ ， $\lambda_B = 7.5/25 = 0.3$ ， $\lambda_C = 0.3$ 。三者皆 ≥ 0 ， P 在內部。

(c) 螢幕線性： $u = 0.3$ ， $v = 0.3$ 。分母 $= 0.4/1 + 0.3/3 + 0.3/5 = 0.4 + 0.1 + 0.06 = 0.56$ 。 u 分子 $= 0.4 \cdot 0/1 + 0.3 \cdot 1/3 + 0.3 \cdot 0/5 = 0.1$ ， $u = 0.1/0.56 \approx 0.1786$ 。 v 分子 $= 0.4 \cdot 0/1 + 0.3 \cdot 0/3 + 0.3 \cdot 1/5 = 0.06$ ， $v = 0.06/0.56 \approx 0.1071$ 。差異： u 差約 0.121， v 差約 0.193。

習題 2。(a) 最大覆蓋次數為 2。這裡計數的樣本不是所有連續平面點，而是 1024 個像素中心

$$(x, y) = (i + 0.5, j + 0.5), \quad 0 \leq i, j < 32.$$

三個連續三角形確實在幾何點 (16, 16) 相交，但像素中心的兩個座標皆為半整數，因此該點不是本題的取樣位置。Top-left 只決定邊界樣本歸屬，不能消除不同三角形內部的重疊。若在深度測試後才計數，相同深度的後續片段可能被拒絕，便測不到本題要求的覆蓋重疊。

(b) 在正方形內， T_1 的連續區域為 $x + y \leq 32$ ， T_2 為 $y \geq x$ ， T_3 為 $x \geq 16$ ， $x - 16 \leq y \leq 48 - x$ ，邊界是否納入另依 top-left 規則決定。三者並非無重疊的三角剖分： T_1 與 T_2 在 $x < 16$ 有面積不為零的交集； T_3 則在 $x > 16$ 分別與另兩者重疊。

依本章 top-left 規則，把像素中心代入可化為整數條件：

$$T_1 : i + j \leq 30, \quad T_2 : j > i,$$

$$T_3 : 16 \leq i \leq 30, \quad i - 15 \leq j \leq 46 - i.$$

因此三者各覆蓋 496、496、240 個像素；兩兩交集 $T_1 \cap T_2$ 、 $T_1 \cap T_3$ 、 $T_2 \cap T_3$ 分別有 240、56、56 個像素。 $T_1 \cap T_2$ 只可能出現在 $i \leq 15$ ，而 T_3 要求 $i \geq 16$ ，所以沒有三重覆蓋。由容斥原理，聯集為

$$496 + 496 + 240 - 240 - 56 - 56 = 880.$$

故覆蓋 2 次者有 $240 + 56 + 56 = 352$ 個；覆蓋 1 次者有 528 個；覆蓋 0 次者有 $1024 - 880 = 144$ 個。

(c) 若移除 top-left 規則並把所有 $e = 0$ 都算作內部，整數條件改為包含三條幾何邊界。此時三個三角形各覆蓋 528、528、272 個像素，兩兩交集分別為 272、72、72 個，仍無像素中心落在三重交點。因此最大值仍為 2；覆蓋 0、1、2 次分別為 112、496、416 個，聯集共 912 個。

幾何點 (16, 16) 的連續覆蓋次數是 3，但本題問的是像素中心的離散覆蓋次數，兩者不可混為一談。這也說明只比較最大值不足以驗證 top-left 規則，還要比較完整分布；前面的兩三角形共享斜邊測試才是單純的共享邊歸屬案例。

習題 3。 (a) 錯在把 z_{ndc} 當成視空間線性屬性。 z_{ndc} 本身已經是 $1/w$ 的仿射函數，螢幕線性插值即正確；用透視校正公式反而算出偏離正確幾何的值。

(b) 例 3：near= 1、far= 2、 $w_A = 1$ 、 $w_B = 2$ 、 $\lambda = (0.5, 0.5)$ 。正確 $z_{\text{ndc},P} = 0$ ；錯誤公式得 -0.333 。若像素恰在 T_{far} 之前一個近三角形邊界附近，這種偏差足以讓遠三角形通過深度測試，遮擋翻轉。

(c) 修正：直接對 z_{ndc} 做螢幕線性插值。

```
z_ndc = la * z0 + lb * z1 + lc * z2
z_w = 0.5 * (z_ndc + 1.0)
if z_w < depth[u]:
    depth[u] = z_w
```

習題 4。以本章 `raster_triangle` 傳入 `[x, y, z_ndc, w, [r, g, b, u, v]]` 形式的頂點即可。第一張 PPM 用前三個分量當 RGB 輸出；第二張 PPM 取 channel 3、4 拷貝到輸出的 R、G，並補 B=0。驗證共享邊時抽樣對角線兩側三格像素，計算 UV 差值：正確做法下差值 $< 1/8$ ；若未對 UV 做透視校正（用螢幕線性），當兩端 w 差 10 倍以上時，跨越共享邊的 UV 會出現階差。這正是養殖視覺化中「魚體皮膚在近端出現接縫狀色塊」的來源。

本章小結

- 邊函數是螢幕空間的仿射函數，直接作為重心座標的分子；三者同號（依繞序）代表像素落在三角形內。
- Top-left 規則以有向邊方向決定 $e = 0$ 的歸屬，是共享邊不重複、不留縫的關鍵。
- Z-buffer 以 $z_w \in [0, 1]$ 儲存最小深度值。 z_{ndc} 是 $1/w$ 的仿射函數，因此螢幕線性插值即為正確；其他視空間線性屬性（UV、顏色、法線）必須用透視校正公式 $a_P = \frac{\sum \lambda_i a_i / w_i}{\sum \lambda_i / w_i}$ 。
- 兩種插值公式的差異在斜面或大 w 差場景最顯著；深度公式選錯會讓遮擋判斷與真實幾何不一致。
- 三角形光柵化把連續幾何離散化為像素棧，是後續貼圖、法線貼圖、光照與路徑追蹤等所有像素層級計算的入口。

參考來源

- G1：PBRT 4 *Transformations*，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- G2：PBRT 4 *Reflection Models*，https://pbr-book.org/4ed/Reflection_Models
- G3：PBRT 4 *The Light Transport Equation*，https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- G4：Ray Tracing in One Weekend，<https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- G5：LearnOpenGL *Transformations*，<https://learnopengl.com/Getting-started/Transformations>
- G6：Blender Manual *Skinning Introduction*，<https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>
- G7：NumPy 線性代數參考，<https://numpy.org/doc/stable/reference/routines.linalg.html>
- G8：Khronos glTF 2.0 規格，<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

上述來源用於建構本章主題知識；本章文字與程式為自行撰寫，未逐字重製，亦未執行官方範例。GLSL、Blender 與 GPU 相關內容在後續章節作為可選橋接，本章實驗全部可在純 CPU 標準庫完成。

第 7 章 三角網格與拓撲資料結構

學習目標與先備知識

三角形是即時光柵化、離線渲染與幾何處理最常見的表面基本元件。本章把單一三角形推進到由大量三角形組成的**三角網格**，並處理幾何座標以外的拓撲問題。

完成本章後，讀者應能：

1. 以頂點陣列與索引三角形表示網格。
2. 由三角形索引建立無向邊及面鄰接關係。
3. 依右手系與繞序計算面法線。
4. 判斷邊界邊、內部邊與非流形邊。
5. 辨識非法索引、重複面、退化三角形與錯誤繞序。
6. 理解「每條邊最多連兩個面」不足以保證頂點流形。
7. 建立低面數開口池體，驗證其面積、法線與邊界。

先備知識包括三維向量、內積、外積、向量長度，以及 Python 與 NumPy 的基本操作。本章長度單位採公尺，面積單位為平方公尺。

問題與直覺

只保存一堆彼此獨立的三角形座標雖然可以繪圖，卻不容易回答下列問題：

- 哪兩個三角形共享同一條邊？
- 某條邊位於物體內部還是開口邊界？
- 表面是否破洞？
- 相鄰面的繞序是否一致？
- 某頂點附近是否形成正常的圓盤或半圓盤？
- 修改一個角點時，所有相鄰三角形是否會一起更新？

因此通常把位置與連接關係分開保存。令頂點位置陣列為

$$V = (\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{n-1}), \quad \mathbf{p}_i \in \mathbb{R}^3,$$

三角形索引陣列為

$$F = ((i_0, j_0, k_0), (i_1, j_1, k_1), \dots).$$

三角形 (i, j, k) 使用 $\mathbf{p}_i, \mathbf{p}_j, \mathbf{p}_k$ ，而不是再次複製座標。這種**索引網格**能共享頂點、節省空間，並讓鄰接分析成為離散索引問題。

但「位置相同」不必然代表「拓撲上是同一頂點」。例如 UV 接縫或硬邊可能保留兩筆相同位置、不同屬性的頂點。反過來，若本來應該共享的角點被重複建立，幾何看似密合，拓撲上仍可能有裂縫。

數學與幾何推導

1. 面的繞序、面積與法線

對索引三角形 $f = (i, j, k)$ ，定義兩條邊向量

$$\mathbf{e}_1 = \mathbf{p}_j - \mathbf{p}_i, \quad \mathbf{e}_2 = \mathbf{p}_k - \mathbf{p}_i.$$

未正規化法線為

$$\mathbf{c}_f = \mathbf{e}_1 \times \mathbf{e}_2.$$

其方向由右手規則決定。從法線方向觀看時，本書約定三角形前面為逆時針。三角形面積為

$$A_f = \frac{1}{2} \|\mathbf{c}_f\|.$$

若 $\|\mathbf{c}_f\| = 0$ ，三點共線或有頂點重複，三角形退化，無法定義唯一單位法線。非退化時

$$\mathbf{n}_f = \frac{\mathbf{c}_f}{\|\mathbf{c}_f\|}.$$

浮點計算不應只測試是否恰等於零。若場景代表尺度為 $L > 0$ 公尺，則 $\|\mathbf{c}_f\|$ 的單位為 m^2 ，可用

$$\|\mathbf{c}_f\| \leq \varepsilon_A, \quad \varepsilon_A = \tau L^2$$

判為近退化，其中 τ 是無因次相對容差，例如 10^{-12} 。

本程式以網格軸對齊包圍盒的對角線長度作為代表尺度：

$$L = \|\mathbf{p}_{\max} - \mathbf{p}_{\min}\|.$$

若所有頂點重合，則 $L = 0$ ，所有三角形的外積也必為零；此時直接以零外積判為退化。若座標含 NaN 或無限值，尺度與法線皆沒有可用意義，必須拒絕輸入。這些容差是數值品質門檻，不是物理安全閾值。

2. 有向邊與無向邊

三角形 (i, j, k) 產生三條有向邊：

$$(i, j), \quad (j, k), \quad (k, i).$$

建立鄰接表時，通常把

$$\{a, b\} \mapsto (\min(a, b), \max(a, b))$$

作為無向邊的標準鍵。令 $d(e)$ 為使用無向邊 e 的面數：

- $d(e) = 1$ ：邊界邊；
- $d(e) = 2$ ：一般內部邊；
- $d(e) > 2$ ：非流形邊；
- $d(e) = 0$ ：不會出現在由面建立的邊表中。

對一致定向的可定向表面，共享邊在兩個面的有向表示必須相反。例如一面含 (a, b) ，另一面應含 (b, a) 。若兩面都沿 (a, b) ，至少有一面的繞序錯誤。

3. 面鄰接

若兩個不同三角形共享一條無向邊，便稱它們為**邊鄰接**。設

$$E(e) = \{f \mid f \text{ 使用邊 } e\},$$

當 $E(e) = \{f_r, f_s\}$ 時，可在面鄰接圖中加入連線 $f_r \leftrightarrow f_s$ 。

只共享一個頂點的兩面不算邊鄰接。這項區別會影響洪水填充、連通元件、法線傳播及網格簡化。

4. 流形與邊界

直觀而言，二維流形表面的每個局部區域都應像一小片圓盤；位於邊界上的點則像半圓盤。

對三角網格，必要條件包括：

1. 每條邊最多接兩個三角形。
2. 內部邊接兩面，邊界邊接一面。
3. 每個頂點周圍的入射面形成單一扇形，而不是數個只在該點相碰的扇形。

第三項不能由邊數條件取代。兩個四面體若只共用一個頂點，所有邊仍可能各接兩面，但共同頂點的鄰域分裂成兩組，形成「蝴蝶結」式非流形頂點。

可用頂點的 **link** 判定局部拓撲。對每個含頂點 v 的三角形 (v, a, b) ，在 link 圖加入邊 (a, b) ：

- 內部流形頂點的 link 是一個環，每個 link 頂點度數皆為 2。
- 邊界流形頂點的 link 是一條路徑，恰有兩個度數為 1 的端點，其餘度數為 2。
- link 不連通、出現分支或其他度數，表示非流形或重複結構。

5. Euler 特徵

若網格有 $|V|$ 個實際使用的頂點、 $|E|$ 條無向邊與 $|F|$ 個面，Euler 特徵定義為

$$\chi = |V| - |E| + |F|.$$

單一封閉、無洞且與球面同拓撲的可定向表面有 $\chi = 2$ ；與圓盤同拓撲的單一開口表面有 $\chi = 1$ 。Euler 特徵是有用的整體檢查，但不能單獨證明網格正確：不同錯誤可能互相抵消。

6. 退化面與重複面

常見無效面包括：

- 索引超出 $[0, n - 1]$ ；
- 三個索引中有重複值；
- 三個不同頂點幾乎共線；
- 同一組頂點被重複列出；
- 同一幾何面以相反繞序重複，形成重疊雙面。

重複面可先用排序後的三元組

$$(\min(i, j, k), \text{mid}(i, j, k), \max(i, j, k))$$

作為忽略繞序的鍵。這只能找出索引完全相同的重複面，不能發現由不同索引表示但座標重合的重疊面。

非法索引會妨礙幾何計算；面內重複索引會污染邊表與頂點 link；完全重複面會重複增加邊的入射次數，而以集合表示的 link 又可能掩蓋這項重複。因此本章採用下列檢查順序：

1. 驗證頂點與索引格式。
2. 檢查索引範圍。
3. 計算面積與幾何退化資訊。
4. 若有面內重複索引，停止拓撲分析。
5. 若有重複面，停止拓撲分析。
6. 對結構合法且無重複的面建立邊表與 link。

這種策略刻意區分「發現輸入錯誤」與「分析有效表面」。如果想研究錯誤資料造成的原始邊入射數，可以另建診斷工具，但不可把結果誤稱為有效網格拓撲。

逐步手算例題

例題一：面積、法線與繞序

設

$$\mathbf{p}_0 = (0, 0, 0), \quad \mathbf{p}_1 = (2, 0, 0), \quad \mathbf{p}_2 = (0, 0, 3).$$

三角形索引為 $(0, 2, 1)$ 。先求

$$\mathbf{e}_1 = \mathbf{p}_2 - \mathbf{p}_0 = (0, 0, 3),$$

$$\mathbf{e}_2 = \mathbf{p}_1 - \mathbf{p}_0 = (2, 0, 0).$$

外積為

$$\mathbf{e}_1 \times \mathbf{e}_2 = (0, 0, 3) \times (2, 0, 0) = (0, 6, 0).$$

因此

$$A = \frac{1}{2} \sqrt{0^2 + 6^2 + 0^2} = 3 \text{ m}^2, \quad \mathbf{n} = (0, 1, 0).$$

若改成 $(0, 1, 2)$ ，外積變為 $(0, -6, 0)$ 。面積不變，但法線反向。由此可見，繞序是幾何方向的一部分，不只是儲存格式細節。

例題二：四邊形三角化後的鄰接

令四個頂點為 $0, 1, 2, 3$ ，兩面為

$$f_0 = (0, 1, 2), \quad f_1 = (0, 2, 3).$$

f_0 的有向邊為

$$(0, 1), (1, 2), (2, 0),$$

f_1 的有向邊為

$$(0, 2), (2, 3), (3, 0).$$

標準化後，無向邊 $(0, 2)$ 出現兩次，因此是內部邊；其餘四條只出現一次，因此是邊界：

$$(0, 1), (1, 2), (2, 3), (0, 3).$$

共享邊在 f_0 中為 $(2, 0)$ ，在 f_1 中為 $(0, 2)$ ，方向相反，故兩面繞序一致。總數為

$$|V| = 4, \quad |E| = 5, \quad |F| = 2,$$

所以

$$\chi = 4 - 5 + 2 = 1,$$

符合圓盤拓撲。

例題三：開口池體的計數

考慮長 4 m、寬 3 m、深 1 m 的開口矩形池。底面與四側壁各拆成兩個三角形，因此

$$|F| = 2 + 4 \times 2 = 10.$$

使用上下各四個角點，共 $|V| = 8$ 。頂部四條邊沒有頂蓋，因此是邊界邊。由三角形邊關係

$$3|F| = 2|E_{\text{int}}| + |E_{\partial}|$$

得

$$30 = 2|E_{\text{int}}| + 4, \quad |E_{\text{int}}| = 13.$$

總邊數為

$$|E| = 13 + 4 = 17,$$

所以

$$\chi = 8 - 17 + 10 = 1.$$

此池體表面與圓盤同拓撲；頂部開口並不代表有一個「把手」或貫穿洞。

實作與程式

以下程式只使用 Python 3.10+ 與 NumPy。它建立法線朝向池內的開口池體，檢查索引、退化面、重複面、邊界、非流形邊、共享邊繞序與頂點 link。

```
import numpy as np
from collections import defaultdict, deque

def make_open_tank():
```

```

# 單位：公尺；Y 向上；表面法線朝向池內。
vertices = np.array([
    [-2.0, 0.0, -1.5], # 0 bottom
    [ 2.0, 0.0, -1.5], # 1
    [ 2.0, 0.0,  1.5], # 2
    [-2.0, 0.0,  1.5], # 3
    [-2.0, 1.0, -1.5], # 4 top
    [ 2.0, 1.0, -1.5], # 5
    [ 2.0, 1.0,  1.5], # 6
    [-2.0, 1.0,  1.5], # 7
], dtype=float)

faces = np.array([
    [0, 2, 1], [0, 3, 2], # floor: +Y
    [0, 1, 5], [0, 5, 4], # z = -1.5: +Z
    [1, 2, 6], [1, 6, 5], # x = +2: -X
    [2, 3, 7], [2, 7, 6], # z = +1.5: -Z
    [3, 0, 4], [3, 4, 7], # x = -2: +X
], dtype=int)
return vertices, faces

def face_geometry(vertices, faces, relative_tol=1e-12):
    if vertices.ndim != 2 or vertices.shape[1] != 3:
        raise ValueError("vertices 必須具有形狀 (N, 3)")
    if len(vertices) == 0:
        raise ValueError("vertices 不可為空")
    if not np.all(np.isfinite(vertices)):
        raise ValueError("頂點座標不可含 NaN 或無限值")
    if relative_tol < 0 or not np.isfinite(relative_tol):
        raise ValueError("relative_tol 必須是有限非負數")

    # 代表尺度 L：軸對齊包围盒對角線長度。
    extent = np.ptp(vertices, axis=0)
    scale = float(np.linalg.norm(extent))
    cross_tol = relative_tol * scale * scale
    # scale == 0 時 cross_tol == 0；所有面必有零外積。

    normals = np.zeros((len(faces), 3), dtype=float)
    areas = np.zeros(len(faces), dtype=float)
    degenerate = []

    for fi, (i, j, k) in enumerate(faces):
        c = np.cross(
            vertices[j] - vertices[i],
            vertices[k] - vertices[i]
        )
        length = float(np.linalg.norm(c))
        areas[fi] = 0.5 * length

        if length <= cross_tol:
            degenerate.append(fi)
        else:
            normals[fi] = c / length

    return normals, areas, degenerate, scale, cross_tol

def build_edge_table(faces):

```

```

# 每筆值保存 (面編號, 原始有向邊)。
table = defaultdict(list)
for fi, (i, j, k) in enumerate(faces):
    for a, b in ((i, j), (j, k), (k, i)):
        key = (min(a, b), max(a, b))
        table[key].append((fi, (a, b)))
return table

def vertex_link_issues(vertex_count, faces):
    links = [defaultdict(set) for _ in range(vertex_count)]

    for i, j, k in faces:
        links[i][j].add(k)
        links[i][k].add(j)
        links[j][i].add(k)
        links[j][k].add(i)
        links[k][i].add(j)
        links[k][j].add(i)

    issues = []
    for v, graph in enumerate(links):
        if not graph: # 未使用頂點另行報告
            continue

        nodes = set(graph)
        start = next(iter(nodes))
        seen = {start}
        queue = deque([start])

        while queue:
            a = queue.popleft()
            for b in graph[a]:
                if b not in seen:
                    seen.add(b)
                    queue.append(b)

        degrees = [len(graph[a]) for a in nodes]
        degree_one = sum(d == 1 for d in degrees)
        valid_cycle = all(d == 2 for d in degrees)
        valid_path = (
            degree_one == 2
            and all(d in (1, 2) for d in degrees)
        )

        if seen != nodes or not (valid_cycle or valid_path):
            issues.append(v)

    return issues

def inspect_mesh(vertices, faces, relative_tol=1e-12):
    vertices = np.asarray(vertices, dtype=float)
    faces_array = np.asarray(faces)

    if vertices.ndim != 2 or vertices.shape[1] != 3:
        raise ValueError("vertices 必須具有形狀 (N, 3)")
    if len(vertices) == 0:
        raise ValueError("vertices 不可為空")

```

```

if not np.all(np.isfinite(vertices)):
    raise ValueError("頂點座標不可含 NaN 或無限值")
if faces_array.ndim != 2 or faces_array.shape[1] != 3:
    raise ValueError("faces 必須具有形狀 (M, 3)")

result = {
    "invalid_faces": [],
    "repeated_index_faces": [],
    "duplicate_faces": [],
    "topology_status": "not_checked",
}

n = len(vertices)
clean_faces = []
seen_faces = {}

# 逐值檢查，避免把 1.5 靜默轉成整數 1。
for fi, face in enumerate(faces_array):
    values = []
    valid = True

    for x in face:
        try:
            xf = float(x)
        except (TypeError, ValueError):
            valid = False
            break

        if not np.isfinite(xf) or not xf.is_integer():
            valid = False
            break
        values.append(int(xf))

    if not valid or not all(0 <= x < n for x in values):
        # 立即記錄非法面，不加入 clean_faces，避免以虛構面
        # 汙染後續幾何或拓撲分析。迴圈結束後會直接返回。
        result["invalid_faces"].append(fi)
        continue

    i, j, k = values
    clean_faces.append((i, j, k))

    if len({i, j, k}) < 3:
        result["repeated_index_faces"].append(fi)

    key = tuple(sorted((i, j, k)))
    if key in seen_faces:
        result["duplicate_faces"].append(
            (seen_faces[key], fi)
        )
    else:
        seen_faces[key] = fi

# 非法索引無法安全進行幾何計算。
if result["invalid_faces"]:
    result["topology_status"] = "skipped_invalid_index"
    return result

clean_faces = np.asarray(clean_faces, dtype=int)

```

```

normals, areas, degenerate, scale, cross_tol = face_geometry(
    vertices, clean_faces, relative_tol
)

result.update({
    "normals": normals,
    "areas": areas,
    "total_area": float(np.sum(areas)),
    "degenerate_faces": degenerate,
    "representative_scale": scale,
    "cross_tolerance": cross_tol,
})

# 面內重複索引會製造自環邊並污染 link。
if result["repeated_index_faces"]:
    result["topology_status"] = "skipped_repeated_index"
    return result

# 完全重複面會重複增加邊入射數，而集合式 link
# 可能掩蓋重複，因此不把後續結果視為有效拓撲。
if result["duplicate_faces"]:
    result["topology_status"] = "skipped_duplicate_face"
    return result

edge_table = build_edge_table(clean_faces)

boundary = sorted(
    edge
    for edge, uses in edge_table.items()
    if len(uses) == 1
)
nonmanifold = sorted(
    edge
    for edge, uses in edge_table.items()
    if len(uses) > 2
)

orientation_errors = []
adjacency = [set() for _ in range(len(clean_faces))]

for edge, uses in edge_table.items():
    if len(uses) == 2:
        (f0, d0), (f1, d1) = uses
        adjacency[f0].add(f1)
        adjacency[f1].add(f0)

        if d0 == d1:
            orientation_errors.append(edge)

used = set(map(int, clean_faces.ravel()))

result.update({
    "boundary_edges": boundary,
    "nonmanifold_edges": nonmanifold,
    "orientation_errors": sorted(orientation_errors),
    "vertex_link_issues":
        vertex_link_issues(n, clean_faces),
    "unused_vertices":
        sorted(set(range(n)) - used),
})

```

```

        "edge_count": len(edge_table),
        "euler_characteristic":
            len(used) - len(edge_table) + len(clean_faces),
        "adjacency":
            [sorted(neighbors) for neighbors in adjacency],
        "topology_status": "checked",
    })
    return result

if __name__ == "__main__":
    vertices, faces = make_open_tank()
    report = inspect_mesh(vertices, faces)

    print("faces:", len(faces))
    print("edges:", report["edge_count"])
    print("boundary:", report["boundary_edges"])
    print("nonmanifold:", report["nonmanifold_edges"])
    print("orientation errors:", report["orientation_errors"])
    print("vertex link issues:",
          report["vertex_link_issues"])
    print("degenerate:", report["degenerate_faces"])
    print("area:", report["total_area"])
    print("Euler characteristic:",
          report["euler_characteristic"])

    assert report["topology_status"] == "checked"
    assert report["invalid_faces"] == []
    assert report["repeated_index_faces"] == []
    assert report["duplicate_faces"] == []
    assert report["boundary_edges"] == [
        (4, 5), (4, 7), (5, 6), (6, 7)
    ]
    assert report["nonmanifold_edges"] == []
    assert report["orientation_errors"] == []
    assert report["vertex_link_issues"] == []
    assert report["degenerate_faces"] == []
    assert np.isclose(report["total_area"], 26.0)
    assert report["euler_characteristic"] == 1

```

`vertex_link_issues` 適用於此處的三角形複形檢查。若資料含面內重複索引或完全重複面，檢查器會保留已完成的格式與幾何診斷，但不建立邊表或解讀 link。修正錯誤面後，應重新執行完整拓撲檢查。

測試與預期結果

上述程式未在此處執行。依資料與推導，原始池體的**預期**結果如下：

- 面數：10。
- 無向邊數：17。
- 邊界邊：(4,5)、(5,6)、(6,7)、(4,7)。
- 非流形邊：無。
- 共享邊繞序錯誤：無。

- 非流形頂點：無。
- 退化面：無。
- 重複面：無。
- 未使用頂點：無。
- 總面積：26 m²。
- Euler 特徵：1。
- `topology_status`："checked"。

本例包圍盒尺寸為 (4, 1, 3)，所以代表尺度為

$$L = \sqrt{4^2 + 1^2 + 3^2} = \sqrt{26} \text{ m}.$$

若 $\tau = 10^{-12}$ ，程式採用的外積長度容差為

$$\varepsilon_A = 10^{-12} L^2 = 2.6 \times 10^{-11} \text{ m}^2.$$

總面積也可獨立核對：

$$A = 4 \times 3 + 2(4 \times 1) + 2(3 \times 1) = 12 + 8 + 6 = 26 \text{ m}^2.$$

故障注入測試

下列案例應各自由原始正確網格重新開始。

1. 反轉一個底面

把 [0, 2, 1] 改成 [0, 1, 2]。預期底面法線反向，且它與相鄰面共享的部分邊出現繞序錯誤。

2. 加入重複面

加入 [0, 2, 1] 的副本。預期 `duplicate_faces` 記錄原面與新增面的編號，並得到：

```
topology_status == "skipped_duplicate_face"
```

檢查器不建立邊表、link 或 Euler 特徵。若忽略保護而直接建立原始邊表，重複面確實會增加三條邊的入射次數；但那只是錯誤資料的診斷現象，不是有效表面的拓撲結果。

3. 加入面內重複索引

加入 [0, 0, 1]。預期同時列入 `repeated_index_faces` 與 `degenerate_faces`，並得到：

```
topology_status == "skipped_repeated_index"
```

4. 加入非法索引

加入 `[0, 1, 99]`。預期列入 `invalid_faces`，並得到：

```
topology_status == "skipped_invalid_index"
```

程式不會嘗試以索引 99 存取座標，也不會回傳 `areas`、`normals` 或 `degenerate_faces`：這些鍵在該分支並不存在，呼叫端不應假設它們存在。修正或移除非合法索引後，應重新執行完整檢查並要求 `topology_status == "checked"`。

第三、第四項都是錯誤輸入示範。修正或移除錯誤面後，才應重新執行並解讀邊界、流形及 Euler 特徵。

除錯與常見陷阱

把法線錯誤當成相機錯誤

若開啟背面剔除後某些牆消失，先檢查頂點繞序與座標系，不要立刻關閉剔除。雙面顯示可能暫時掩蓋模型錯誤。

只比較頂點座標，不比較索引

兩個頂點座標完全相同，但索引不同時，拓撲上仍是兩個頂點。這可能形成零寬裂縫、重複邊或不連通元件。是否合併頂點還須考慮 UV、材質、法線與硬邊需求。

認為每條邊最多兩面就一定是流形

這只能排除非流形邊，不能排除非流形頂點。數個封閉扇形可只在一點相接，因此還要檢查頂點 link 是否為單一環或單一路徑。

把重複面當成普通非流形邊

重複面本身就是輸入結構錯誤。若直接送入邊表，它可能使原本的內部邊看似接了三個面；若 link 使用集合，又可能消除重複資訊。應先報告並移除重複面，再進行一般流形判定。

以固定 epsilon 處理所有尺度

若模型的所有座標乘上比例 s ，邊長乘上 $|s|$ ，外積長度與面積則乘上 s^2 。因此退化容差也應按 s^2 縮放。

不要為了避免零尺度而把所有小於 1 m 的模型尺度強制設成 1 m；那會破壞相似模型的尺度相對性。零尺度應獨立處理，非有限座標則應拒絕。

在錯誤面存在時解讀拓撲

面 $[0,0,1]$ 會產生自環式邊 $(0,0)$ 。若仍把它送入一般邊表，邊界數與 link 度數可能有形式上的輸出，卻沒有可靠的二維表面意義。應先清除結構錯誤，再分析拓撲。

忘記內表面與外表面的語意

池體可建成：

- 法線朝外的封閉容器外殼；
- 法線朝水體的內壁表面；
- 同時具有厚度的實體牆。

本例只有零厚度內壁，法線朝池內。它適合合成場景中的水下可見表面，但不是可製造的完整實體模型。

用 Euler 特徵取代局部檢查

χ 正確不代表沒有重複面、錯誤繞序、自相交或不連通的頂點鄰域。Euler 特徵應與邊入射數、頂點 link、幾何退化及連通性共同使用。

養殖數位分身案例

低面數池體可作為養殖數位分身中的基礎幾何，其座標範圍為

$$x \in [-2, 2], \quad y \in [0, 1], \quad z \in [-1.5, 1.5].$$

這裡 $y = 0$ 是池底， $y = 1$ 是池緣。模型只含池底與四面內壁，頂部保持開放。內向法線的語意如下：

- 池底： $+\mathbf{y}$ ；
- $z = -1.5$ 側壁： $+\mathbf{z}$ ；
- $z = 1.5$ 側壁： $-\mathbf{z}$ ；
- $x = -2$ 側壁： $+\mathbf{x}$ ；
- $x = 2$ 側壁： $-\mathbf{x}$ 。

四條頂部邊界可用於後續操作，例如生成池緣、建立水面四邊形或驗證模型是否意外封頂。不要僅以「有四條邊界邊」判定正確；還應確認它們形成單一閉合環：

$$4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 4.$$

這裡的「邊界」是網格的拓撲邊界，不保證在某個相機位置必然可見。邊界可能被其他物件遮住，也可能因雙面繪製、背面剔除或材質設定而呈現不同外觀。

若要加入水面，可用獨立網格表示。水面若完全封住池頂，整體拓撲可能變成封閉表面，但水面法線、透明材質與介質邊界有不同語意，不宜只為消除邊界而與池壁混成同一材質面。

此模型是合成幾何，不代表真實池壁厚度、磨損、排水口或結構安全。實務資產還應保存單位、座標系、面材質與版本資訊，以便後續渲染和標註重現。

習題

習題 1：手算

給定頂點

$$\mathbf{p}_0 = (0, 0, 0), \quad \mathbf{p}_1 = (1, 0, 0), \quad \mathbf{p}_2 = (1, 1, 0), \quad \mathbf{p}_3 = (0, 1, 0),$$

以及面 $(0, 1, 2)$ 、 $(0, 2, 3)$ 。

1. 求兩面的單位法線與總面積。
2. 列出邊界邊及內部邊。
3. 求 Euler 特徵。

習題 2：程式測試

在池體程式中加入三角形 `[4, 5, 6]`。不執行程式，推導哪些既有邊的入射數會改變，判斷是否形成完整頂蓋，並檢查其繞序。

習題 3：反例與除錯

有人提出：「只要所有邊都接一面或兩面，網格就是帶邊界的二維流形。」請給出反例，並說明邊表為何無法發現問題。

習題 4：整合應用

要在池頂加入由兩個三角形構成的拓撲頂蓋，頂點沿用 4, 5, 6, 7，並與法線朝池內的池壁一致定向，因此頂蓋法線應朝 $-\mathbf{y}$ 。這不是獨立朝上的水面。

1. 寫出一組正確索引。
2. 加入後共有多少面與多少無向邊？
3. 若把頂蓋視為池體同一拓撲網格，Euler 特徵為何？
4. 此網格是否仍有邊界？

習題 5：退化判定

設場景代表尺度 $L = 10 \text{ m}$ ，相對容差 $\tau = 10^{-12}$ 。某三角形的外積長度為 $5 \times 10^{-11} \text{ m}^2$ 。依本章規則，它是否判為近退化？

習題解答

解答 1

第一面：

$$(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0) = (1, 0, 0) \times (1, 1, 0) = (0, 0, 1).$$

第二面：

$$(\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_3 - \mathbf{p}_0) = (1, 1, 0) \times (0, 1, 0) = (0, 0, 1).$$

兩者單位法線皆為 $(0, 0, 1)$ 。每面面積為 $1/2 \text{ m}^2$ ，總面積為 1 m^2 。

內部邊為 $(0, 2)$ ；邊界邊為

$$(0, 1), (1, 2), (2, 3), (0, 3).$$

共有 4 個頂點、5 條邊、2 個面，因此

$$\chi = 4 - 5 + 2 = 1.$$

解答 2

新增面 `[4,5,6]` 使用無向邊

$$(4, 5), \quad (5, 6), \quad (4, 6).$$

原本 $(4, 5)$ 與 $(5, 6)$ 各只接一面，新增後各接兩面，從邊界變成內部邊。 $(4, 6)$ 是新邊，只接新增面，因此成為邊界邊。

原本另外兩條頂部邊 $(6, 7)$ 與 $(4, 7)$ 仍是邊界。邊界形成三角形缺口

$$4 \rightarrow 6 \rightarrow 7 \rightarrow 4,$$

所以尚未形成完整頂蓋。

此外，新增面 $[4, 5, 6]$ 沿頂邊為 $4 \rightarrow 5$ 、 $5 \rightarrow 6$ ；相鄰池壁沿相同邊則為 $5 \rightarrow 4$ 、 $6 \rightarrow 5$ ，方向相反，故共享邊繞序一致。此面法線朝 $-\mathbf{y}$ ，符合整個池殼朝內的定向。改成 $[4, 6, 5]$ 雖使法線朝 $+\mathbf{y}$ ，卻會造成與池壁共享邊同向；若要做朝上的水面，應作獨立表面，不宜稱與池壁共同一致定向。

解答 3

取兩個彼此分離的四面體表面，將其中各一個頂點合併成同一索引，但不合併任何邊。每條邊仍只接兩個面，因此邊入射數完全正常。

然而共同頂點附近存在兩個互不連通的三角形扇形。該頂點的 link 是兩個分離的環，而不是單一環，因此不具圓盤鄰域，是非流形頂點。只記錄每條邊接幾個面無法發現 link 的不連通性。

解答 4

在 xz 平面中，要與朝內池壁一致定向，頂蓋法線應朝 $-\mathbf{y}$ ，可使用

$(4, 5, 6)$
 $(4, 6, 7)$

例如第一面中

$$(\mathbf{p}_5 - \mathbf{p}_4) \times (\mathbf{p}_6 - \mathbf{p}_4) = (4, 0, 0) \times (4, 0, 3) = (0, -12, 0).$$

加入兩面後，面數為

$$10 + 2 = 12.$$

新增對角線 $(4, 6)$ ，原本四條邊界邊變成內部邊，因此總邊數為

$$17 + 1 = 18.$$

Euler 特徵為

$$\chi = 8 - 18 + 12 = 2.$$

所有頂部邊都接兩面，對角線也接兩個頂蓋三角形，因此沒有邊界；每條共享邊的兩次使用方向相反，定向也一致。拓撲上與球面相同。

幾何語意上，這只是零厚度封閉殼。若水面與池壁屬於不同介質或材質，仍宜分開保存材質與表面角色。

解答 5

外積長度容差為

$$\varepsilon_A = \tau L^2 = 10^{-12} \times 10^2 = 10^{-10} \text{ m}^2.$$

因為

$$5 \times 10^{-11} < 10^{-10},$$

所以判為近退化。外積長度是三角形面積的兩倍，但程式直接以外積長度和 `cross_tol` 比較，因此容差定義也針對外積長度，而不是直接針對面積。

本章小結

三角網格不只是三角形座標集合，也包含由索引定義的拓撲。三角形繞序決定法線方向；無向邊表可找出面鄰接、邊界與非流形邊；共享邊的有向方向可檢查局部定向一致性。

有效網格至少應檢查：

- 頂點座標是否有限；
- 索引是否為整數且位於合法範圍；
- 面內索引是否重複；
- 面積是否小於尺度相關容差；
- 是否有重複面；
- 邊是否接超過兩個面；
- 共享邊方向是否相反；
- 頂點 link 是否為單一環或路徑；
- Euler 特徵與預期拓撲是否一致。

重複面與面內重複索引應在一般拓撲分析之前處理，不能把受污染的邊入射數或 link 當成有效表面的判定結果。

本章的開口池體有 8 個頂點、17 條邊、10 個三角形、4 條頂部邊界邊，總面積為 26 m^2 ，Euler 特徵為 1。這些可重現數量提供了後續法線、材質、光線求交與場景資產交換的基礎驗收條件。

參考來源

1. PBRT 4，〈Transformations〉：幾何表示、座標與變換背景。
https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
2. NumPy 線性代數參考：向量、範數與陣列運算介面。
<https://numpy.org/doc/stable/reference/routines.linalg.html>

3. Khronos Group，glTF 2.0 Specification：索引幾何、頂點屬性與資產交換概念。

<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

以上來源供延伸閱讀，不表示本章各項拓撲定義已由所列來源逐條查證。池體例子、推導與程式以本章明示的座標、索引及容差定義為準。

第08章 法線、切線與逆轉置

學習目標與先備知識

完成本章後，你應能：

- 區分由三角形決定的**幾何法線**與供平滑著色使用的**著色法線**。
- 推導一般線性變換下的法線矩陣，並說明為何非均勻縮放不能直接作用於法線。
- 由位置與 UV 求切線，建立正交的切線空間基底 TBN，處理鏡射造成的手性變化。
- 用數值測試檢查變換後的垂直關係，並拒絕無法建立法線矩陣的奇異縮放。

先備知識是三維向量、內積、外積、矩陣乘法與基本 Python。本章長度單位為公尺；UV 與單位法線沒有單位。採右手世界座標：+X 向右、+Y 向上、+Z 朝向觀者。所有向量均視為直向量，三角形從外向法線方向看為逆時針。

問題與直覺

設想一片合成魚鰭網格：三角形的位置被拉長、旋轉，以配合魚身，但照明仍要知道魚鰭「朝哪裡」。頂點位置是點，切線是沿表面的**方向**，法線則描述一個約束：它必須與表面的所有切向垂直。三者不能一律乘同一個矩陣。

平移不會改變方向。旋轉及一致縮放較不易暴露錯誤；非均勻縮放則會改變兩個方向的夾角。若把原法線像頂點一樣乘上縮放矩陣，所得向量通常不再垂直表面。更重要的是，這個錯誤可能只表現為光照明暗不對，而非網格形狀明顯破損。

本章把幾何、著色與材質分開處理。幾何法線由實際三角形邊決定，可用來判斷面朝向。著色法線可由相鄰面的法線平均而成，使低面數魚身看起來平滑；它不會改變三角形輪廓，也不保證與每一片三角形完全垂直。

數學與幾何推導

從三角形取得法線

給定同一座標空間中的三角形頂點 $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2 \in \mathbb{R}^3$ ，定義邊向量

$$\mathbf{e}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{e}_2 = \mathbf{p}_2 - \mathbf{p}_0.$$

外積 $\mathbf{c} = \mathbf{e}_1 \times \mathbf{e}_2$ 垂直兩條邊，長度為平行四邊形面積，故三角形面積為 $\|\mathbf{c}\|/2$ 。當 $\|\mathbf{c}\| > 0$ 時，單位幾何法線為

$$\mathbf{n}_g = \frac{\mathbf{c}}{\|\mathbf{c}\|}.$$

交換 $\mathbf{p}_1$ 、 $\mathbf{p}_2$ 會反轉其方向。若面積接近零，法線無法可靠定義；實作時須按模型尺度設定容差，而不是替退化三角形指定一個任意的「正確」法線。

頂點著色法線 $\mathbf{n}_s$ 可以來自數片相鄰面的加權平均。跨越魚鰭銳利邊緣時，通常要為同位置建立不同頂點法線，保留折角。著色法線是照明資料；幾何面朝向、遮擋與三角形面積仍由幾何決定。若把法線作插值，使用前應再次正規化。

為何是逆轉置

令物件到世界的仿射變換為

$$\mathbf{p}' = A\mathbf{p} + \mathbf{t},$$

其中 $A \in \mathbb{R}^{3 \times 3}$ 是可逆的線性部分， $\mathbf{t}$ 是平移。兩點相減後平移抵消，因此切向 $\mathbf{v}$ 變成 $A\mathbf{v}$ 。原法線滿足 $\mathbf{n}^T \mathbf{v} = 0$ ；我們要求變換後的法線 $\mathbf{n}'$ 仍滿足

$$\mathbf{n}'^T (A\mathbf{v}) = 0.$$

令 $\mathbf{n}' = A^{-T} \mathbf{n}$ ，則

$$(A^{-T} \mathbf{n})^T A\mathbf{v} = \mathbf{n}^T A^{-1} A\mathbf{v} = \mathbf{n}^T \mathbf{v} = 0.$$

因此法線矩陣是 $A^{-T} = (A^{-1})^T$ ，而實際使用的單位法線為

$$\widehat{\mathbf{n}'} = \frac{A^{-T} \mathbf{n}}{\|A^{-T} \mathbf{n}\|}.$$

此推導適用於幾何法線，也適用於作為表面方向資料的頂點著色法線。它要求 A 可逆：縮放某軸至零會壓扁三維空間， A^{-1} 不存在。此時應拒絕這項法線變換，或重新定義壓扁後的幾何及其法線；不能默默以零向量代替。

若 $A = RS$ ， R 是旋轉， $S = \text{diag}(s_x, s_y, s_z)$ ，則

$$A^{-T} = R \text{diag}(1/s_x, 1/s_y, 1/s_z).$$

可見非均勻縮放對法線採用各軸的**倒數**，而非原縮放值。平移不進入法線矩陣。在本書的直向量慣例下，物件變換 $M = TRS$ 由右先作用；從 4×4 的 M 取左上角 3×3 作為 A 即可。

切線、UV 與 TBN

UV 座標記作 (u, v) ，其中 v 向上。對三角形，令 $\Delta \mathbf{uv}_i = (\Delta u_i, \Delta v_i)$ 表示從頂點 0 到頂點 i 的 UV 差。假設位置在三角形內隨 UV 作線性變化，則

$$\mathbf{e}_1 = \mathbf{T} \Delta u_1 + \mathbf{B} \Delta v_1, \quad \mathbf{e}_2 = \mathbf{T} \Delta u_2 + \mathbf{B} \Delta v_2.$$

令 $D = \Delta u_1 \Delta v_2 - \Delta v_1 \Delta u_2$ 。若 $D \neq 0$ ，解此二元線性系統可得未正規化的切線與副切線：

$$\mathbf{T} = \frac{\mathbf{e}_1 \Delta v_2 - \mathbf{e}_2 \Delta v_1}{D}, \quad \mathbf{B} = \frac{\mathbf{e}_2 \Delta u_1 - \mathbf{e}_1 \Delta u_2}{D}.$$

若 D 接近零，UV 三角形退化，不能用它求唯一的 TBN。即使位置三角形未退化，UV 仍可能退化，兩種檢查不可互相取代。

頂點著色法線與插值切線未必精確垂直。給定單位著色法線 $\mathbf{N}$ ，可用 Gram-Schmidt 步驟正交化切線：

$$\hat{\mathbf{T}} = \frac{\mathbf{T} - \mathbf{N}(\mathbf{N} \cdot \mathbf{T})}{\|\mathbf{T} - \mathbf{N}(\mathbf{N} \cdot \mathbf{T})\|}.$$

再保存手性 $h \in \{-1, +1\}$ ，定義 $\hat{\mathbf{B}} = h(\mathbf{N} \times \hat{\mathbf{T}})$ 。於是以 $\hat{\mathbf{T}}, \hat{\mathbf{B}}, \mathbf{N}$ 為欄組成的 3×3 TBN 矩陣，可把切線空間方向轉至該法線所在的座標空間。法線貼圖的解碼及使用將於後續章節處理。

鏡射須分清兩件事。其一，若 $\det A < 0$ ，空間變換反轉手性；直接以變換後頂點重新計算的三角形外積，方向相對於 $A^{-T} \mathbf{n}_g$ 會多一個負號：

$$(\mathbf{A} \mathbf{e}_1) \times (\mathbf{A} \mathbf{e}_2) = \det(A) A^{-T} (\mathbf{e}_1 \times \mathbf{e}_2).$$

因此保留原索引繞序的鏡射模型，需要明確決定是否交換三角形兩個索引，使面朝向符合預期。其二，UV 鏡射會改變切線框架的手性；若只儲存 $\mathbf{N}$ 與 $\mathbf{T}$ ，仍須儲存 h ，不能永遠令 $\mathbf{B} = \mathbf{N} \times \mathbf{T}$ 。面繞序修正與 UV 手性修正分別服務於幾何及材質，不可混為同一步。

逐步手算例題

例一：非均勻縮放後仍垂直

取位於 xy 平面斜面上的兩條切向 $\mathbf{v}_1 = (1, 1, 0)^T$ 、 $\mathbf{v}_2 = (0, 0, 1)^T$ 。一個原單位法線是 $\mathbf{n} = (1, -1, 0)^T / \sqrt{2}$ ；檢查可得 $\mathbf{n} \cdot \mathbf{v}_1 = \mathbf{n} \cdot \mathbf{v}_2 = 0$ 。

令 $A = \text{diag}(2, 1, 1)$ 。變換後 $\mathbf{v}'_1 = (2, 1, 0)^T$ 。錯誤作法給出 $A(1, -1, 0)^T = (2, -1, 0)^T$ ，與 $\mathbf{v}'_1$ 的內積是 $4 - 1 = 3$ ，不垂直。正確方向為

$$A^{-T}(1, -1, 0)^T = (1/2, -1, 0)^T,$$

其內積為 $(1/2) \cdot 2 + (-1) \cdot 1 = 0$ 。正規化前後均維持垂直；正規化只改長度，不修復錯誤方向。

例二：鏡射的面朝向與 TBN 手性

取 $\mathbf{e}_1 = (1, 0, 0)^T$ 、 $\mathbf{e}_2 = (0, 1, 0)^T$ ，則 $\mathbf{n}_g = (0, 0, 1)^T$ 。令 $A = \text{diag}(-1, 1, 1)$ ，表示沿 X 鏡射。逆轉置作用於原法線仍得到 $(0, 0, 1)^T$ ；但由新邊計算的外積為

$$(-1, 0, 0)^T \times (0, 1, 0)^T = (0, 0, -1)^T.$$

兩者相反，因為 $\det A = -1$ 。若設計上希望外表面繼續朝 $+Z$ ，可在輸出鏡射後的網格時交換每個三角形的第二、第三個索引；這會反轉外積方向。若不交換，必須接受並一致處理新繞序與面朝向。

再看同一位置三角形的 UV：設 $(u_0, v_0) = (0, 0)$ 、 $(u_1, v_1) = (0, 1)$ 、 $(u_2, v_2) = (1, 0)$ 。此時 $D = -1$ ，解得 $\mathbf{T} = (0, 1, 0)^T$ 、 $\mathbf{B} = (1, 0, 0)^T$ 。由於 $\mathbf{N} \times \mathbf{T} = (-1, 0, 0)^T$ ，要還原 $\mathbf{B}$ 必須取 $h = -1$ 。這是 UV 方向造成的手性，不能僅憑模型有沒有被鏡射來猜測。

實作與程式

下列程式僅使用 Python 3.10+ 與 NumPy 2.2.6 相容寫法。它建立一個三角形，計算幾何法線與 UV 切線框架，然後以非均勻縮放檢查垂直關係。程式碼供讀者在既有環境自行執行；本書未執行它。

```
import numpy as np

def unit(v, eps=1e-12):
    v = np.asarray(v, dtype=float)
    length = np.linalg.norm(v)
    if not np.isfinite(length) or length <= eps:
        raise ValueError("零長度或無效方向")
    return v / length

def geometry_normal(p0, p1, p2, area_eps=1e-12):
    e1 = np.asarray(p1, dtype=float) - np.asarray(p0, dtype=float)
    e2 = np.asarray(p2, dtype=float) - np.asarray(p0, dtype=float)
    c = np.cross(e1, e2)
    if np.linalg.norm(c) <= area_eps:
        raise ValueError("退化位置三角形")
    return unit(c)

def normal_matrix(A, det_eps=1e-12):
    A = np.asarray(A, dtype=float)
    if A.shape != (3, 3) or not np.all(np.isfinite(A)):
        raise ValueError("A 必須是有限的 3x3 矩陣")
    if abs(np.linalg.det(A)) <= det_eps:
        raise ValueError("奇異或接近奇異的線性變換")
    return np.linalg.inv(A).T
```

```

def tangent_frame(p0, p1, p2, uv0, uv1, uv2, N,
                 uv_eps=1e-12):
    e1 = np.asarray(p1, dtype=float) - np.asarray(p0, dtype=float)
    e2 = np.asarray(p2, dtype=float) - np.asarray(p0, dtype=float)
    du1, dv1 = np.asarray(uv1, dtype=float) - np.asarray(uv0, dtype=float)
    du2, dv2 = np.asarray(uv2, dtype=float) - np.asarray(uv0, dtype=float)
    D = du1 * dv2 - dv1 * du2
    if abs(D) <= uv_eps:
        raise ValueError("退化 UV 三角形")

    T_raw = (e1 * dv2 - e2 * dv1) / D
    B_raw = (e2 * du1 - e1 * du2) / D
    N = unit(N)
    T = unit(T_raw - N * np.dot(N, T_raw))
    handedness = 1.0 if np.dot(np.cross(N, T), B_raw) >= 0 else -1.0
    B = handedness * np.cross(N, T)
    return T, B, N, handedness

p0 = np.array([0.0, 0.0, 0.0])
p1 = np.array([1.0, 1.0, 0.0])
p2 = np.array([0.0, 0.0, 1.0])
uv0, uv1, uv2 = (0.0, 0.0), (1.0, 0.0), (0.0, 1.0)

e1, e2 = p1 - p0, p2 - p0
N = geometry_normal(p0, p1, p2)
T, B, _, h = tangent_frame(p0, p1, p2, uv0, uv1, uv2, N)

A = np.diag([2.0, 1.0, 1.0])
N_world = unit(normal_matrix(A) @ N)
assert abs(np.dot(N_world, A @ e1)) < 1e-12
assert abs(np.dot(N_world, A @ e2)) < 1e-12
assert abs(np.dot(T, B)) < 1e-12
assert abs(np.dot(T, N)) < 1e-12
assert abs(np.dot(B, N)) < 1e-12

try:
    normal_matrix(np.diag([1.0, 0.0, 1.0]))
except ValueError:
    pass
else:
    raise AssertionError("奇異縮放應被拒絕")

print("預期測試：垂直關係成立，奇異縮放已拒絕")

```

`area_eps` 對上述以公尺表示、約一公尺見方的例子，是外積長度的容差，量綱為平方公尺；`uv_eps` 無單位。`det_eps` 是對本例量級矩陣的簡化保護，而非通用判準。實際資產若尺度懸殊，須按輸入尺度調整容差，並留意接近奇異矩陣造成的數值放大。

若已先把頂點切線乘 A 、頂點法線乘 A^{-T} ，應在同一個目標空間重新正交化 TBN；不要先在物件空間正交化，便假設非均勻縮放後三軸仍互相垂直。

測試與預期結果

讀者可逐項檢查，不必依賴圖形畫面：

1. ****垂直性****：程式中的兩個內積斷言應通過；在例一直接以 `unit(A @ N)` 取代 `N_world`，第一個內積通常不再接近零。
2. ****正交性****：所建立的 T, B, N 三組兩兩內積應接近零，且各自長度接近一。
3. ****奇異輸入****：把 Y 縮放設為零，應進入 `ValueError` 分支，而非產生法線。
4. ****繞序****：交換 `p1`、`p2` 而不交換相應 UV，幾何法線應反向；測試時應一併檢查資料配對，避免把索引錯置誤認為法線公式錯誤。

上述是根據公式給出的**預期**，不是已執行的輸出。容差只用於數值比較，並非物理或養殖作業的安全閾值。

除錯與常見陷阱

****形狀正確、光照卻偏斜****：先檢查是否誤用 `A @ N`。對非均勻縮放，應使用 `inv(A).T @ N` 並正規化。也要檢查法線是否與頂點在同一座標空間；世界空間切線不能拿來與物件空間法線作內積測試。

****只有一側的魚鰭不亮****：檢查三角形索引繞序、鏡射變換的行列式，以及著色時如何判定正反面。鏡射後外積方向可能翻轉；單獨翻轉頂點著色法線，未必能修正背面裁切。

****貼圖細節在接縫左右顛倒****：檢查 UV 鏡射處的 D 與儲存的手性 h 。UV 接縫兩側可能共享位置，但需要分開儲存 UV、切線及其手性。不要假設每個頂點只對應唯一切線框架。

****正規化產生非數值****：依序檢查位置三角形面積、UV 行列式、線性變換可逆性，以及切線投影到著色法線的切平面後是否變成零。平滑著色法線若與原切線平行，Gram-Schmidt 也無法構成框架，需更換可靠切線或修復資料。

養殖數位分身案例

為合成池中魚隻建立低面數魚體時，可先以三角形外積檢查魚腹、魚背的外向面朝向，再在圓滑魚身計算頂點著色法線；魚鰭根部可保留銳利邊，使輪廓轉折在照明下可辨。若同一個魚模型的實例採不同長、寬、高比例，位置使用各自的 $M = TRS$ ，法線則使用其線性部分的逆轉置，避免寬體魚與窄體魚出現不一致的假光斑。

製作左右對稱魚鰭時，若以負縮放鏡射一側，應在資產輸出階段檢查該側的面繞序；若 UV 也鏡射，另檢查 TBN 手性。可用每片三角形的「變換後法線與兩條變換後邊之內積」作自動檢驗，再以外積方向檢查朝向。這些測試證明幾何資料在指定變換下自治，不證明魚體生物形態、材質或水下照明與真實養殖場一致。

習題

1. **手算。**取切向 $\mathbf{v} = (1, 2, 0)^T$ 、法線方向 $\mathbf{n} = (2, -1, 0)^T$ ，以及 $A = \text{diag}(3, 1, 2)$ 。不必先正規化，分別計算 $(A\mathbf{n}) \cdot (A\mathbf{v})$ 與 $(A^{-T}\mathbf{n}) \cdot (A\mathbf{v})$ ，說明結果。
2. **程式測試。**在本程式後加入一個測試：以 $A = \text{diag}(2, 3, 4)$ 變換 $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 、 $\mathbf{N}$ ，比較由變換後兩條邊的外積所得單位法線，與法線矩陣所得單位法線；兩者應有何關係？寫出斷言。
3. **反例與除錯。**某開發者對 $A = \text{diag}(-1, 1, 1)$ 的三角形保留原索引，以 $A^{-T}\mathbf{n}_g$ 更新法線，卻發現背面裁切與其預想相反。他主張「逆轉置公式錯了」。指出真正原因與一種修正方式。若該模型的 UV 也被鏡射，是否能只靠交換索引修復 TBN？
4. **整合應用。**一片合成魚鰭的三頂點為 $\mathbf{p}_0 = (0, 0, 0)$ 、 $\mathbf{p}_1 = (1, 0, 0)$ 、 $\mathbf{p}_2 = (0, 1, 0)$ ，對應 UV 依序為 $(0, 0)$ 、 $(0, 1)$ 、 $(1, 0)$ 。求幾何法線、 D 、 $\mathbf{T}$ 、 $\mathbf{B}$ 與手性 h 。若再施加 $A = \text{diag}(2, 1, 1)$ ，說明應如何建立世界空間 TBN。

習題解答

1. 原內積是 $(2, -1, 0) \cdot (1, 2, 0) = 0$ 。 $A\mathbf{n} = (6, -1, 0)$ 、 $A\mathbf{v} = (3, 2, 0)$ ，故錯誤變換的內積為 $18 - 2 = 16$ 。 $A^{-T}\mathbf{n} = (2/3, -1, 0)$ ，故正確變換的內積為 $(2/3)3 - 2 = 0$ 。不正規化不影響垂直性的判斷。
2. 由於 $\det A = 24 > 0$ ，兩個單位法線應同向。可加入：

```
N_from_edges = unit(np.cross(A @ e1, A @ e2))
N_from_matrix = unit(normal_matrix(A) @ N)
assert np.allclose(N_from_edges, N_from_matrix, atol=1e-12)
```

此測試同時檢查外積與逆轉置的方向，而不只檢查垂直性。

3. 逆轉置正確維持法線對切向的垂直約束；問題是 $\det A < 0$ 反轉了由原索引決定的幾何外積方向。若希望保留原設計的外向面，可交換鏡射後三角形的第二、第三個索引，並一致更新相關頂點資料。若 UV 鏡射，還須重新計算或正確儲存 TBN 手性；交換索引不能取代這項檢查。
4. $\mathbf{e}_1 = (1, 0, 0)^T$ 、 $\mathbf{e}_2 = (0, 1, 0)^T$ ，故 $\mathbf{n}_g = (0, 0, 1)^T$ 。UV 差為 $(0, 1)$ 與 $(1, 0)$ ，所以 $D = -1$ 。代入公式得 $\mathbf{T} = (0, 1, 0)^T$ 、 $\mathbf{B} = (1, 0, 0)^T$ 。因 $\mathbf{n}_g \times \mathbf{T} = (-1, 0, 0)^T$ ， $h = -1$ 。施加 A 後，先令 $\mathbf{T}_w = A\mathbf{T}$ 、 $\mathbf{N}_w = \text{unit}(A^{-T}\mathbf{n}_g)$ ；將 $\mathbf{T}_w$ 對 $\mathbf{N}_w$ 正交化並正規化，再用 $h(\mathbf{N}_w \times \widehat{\mathbf{T}}_w)$ 建立副切線。此例的 $\det A > 0$ ，沒有額外的空間鏡射；一般情況仍須另行處理負行列式與面繞序。

本章小結

位置、切向與法線承擔不同幾何角色：位置受完整仿射矩陣作用，切向受線性部分 A 作用，法線受 A^{-T} 作用並重新正規化。逆轉置來自「變換後仍垂直」的要求，而非照明公式的特例。建立 TBN 時須檢查位置與 UV 是否退化、正交化切線，並保存鏡射 UV 所需的手性；負行列式造成的面繞序變化則須另外處理。

參考來源

- G1 PBRT 4 : Transformations
- G2 PBRT 4 : Reflection Models
- G5 LearnOpenGL : Transformations
- G7 NumPy 線性代數參考

第09章 Bézier曲線與樣條

學習目標與先備知識

本章建立 Bézier 曲線與分段樣條的數學基礎，涵蓋 Bernstein 基底、de Casteljau 演算法、導數性質、拼接連續性條件及弧長重參數化。讀者需掌握 Volume I 的向量運算、內積、外積與 NumPy 基礎。

學習目標：

1. 理解 Bézier 曲線的仿射組合定義、歸一性與凸包性質。
2. 掌握 de Casteljau 演算法的遞迴結構與數值穩定性。
3. 計算一階導數以確定切線方向，並區分 C^0 、 G^1 與 C^1 連續性。
4. 實作程式驗證端點插值、凸包權重條件與切線連續性。
5. 理解等參數取樣與等弧長取樣的差異及其對網格生成的影響。

先備知識橋接：

- **微積分**：多項式求導則 $\frac{d}{dt}(t^i) = it^{i-1}$ 及鏈式法則。
- **線性代數**：仿射組合（權重和為 1 的線性組合）、向量平行判定（外積或正規化比較）。

問題與直覺

在養殖場數位分身中，魚體輪廓需平滑變化以模擬游動。多邊形網格產生折線感，而 Bézier 曲線提供由少量控制點定義平滑路徑的方法。

直覺理解：Bézier 曲線受控制點「牽引」。只有端點必經曲線，中間控制點僅影響形狀方向。**凸包性質**指出：曲線完全位於控制點所張成的凸包內。這意味著若控制點收斂，曲線亦不會超出該範圍，對確保模型不異常膨脹具有幾何保證。

數學與幾何推導

Bernstein 基底與定義

n 次 Bézier 曲線由 $n + 1$ 個控制點 $P_0, \dots, P_n$ 定義，參數 $t \in [0, 1]$ ：

$$B(t) = \sum_{i=0}^n P_i B_{i,n}(t)$$

其中 Bernstein 基底函數為：

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

關鍵性質：

1. **歸一性：** $\sum_{i=0}^n B_{i,n}(t) = 1$ 。此性質可由二項式定理 $(t + (1-t))^n = 1$ 直接推導。這確保 $B(t)$ 為控制點的仿射組合。
2. **端點插值：** $B(0) = P_0$ 且 $B(1) = P_n$ 。
3. **凸包性：**曲線位於控制點集的凸包內。這是由於對於 $t \in [0, 1]$ ，所有 $B_{i,n}(t) \geq 0$ 且和為 1，故 $B(t)$ 是控制點的凸組合。
4. **變換不變性：**若對所有控制點施加同一仿射變換 T ，則 $T(B(t)) = B_T(t)$ ，其中 B_T 是使用變換後控制點計算的曲線。注意：若不同控制點施加不同變換（如不同骨骼），此性質不成立，僅能視為以新控制點重新定義曲線。

導數與切線

對 $B(t)$ 求導，利用 $\frac{d}{dt}(t^i(1-t)^{n-i}) = t^{i-1}(1-t)^{n-i-1}[i(1-t) - (n-i)t]$ ，可推導出一階導數：

$$B'(t) = n \sum_{i=0}^{n-1} (P_{i+1} - P_i) B_{i,n-1}(t)$$

- **起點切線：** $B'(0) = n(P_1 - P_0)$ 。
- **終點切線：** $B'(1) = n(P_n - P_{n-1})$ 。

切線方向由 $B'(t)$ 決定，但真正表徵參數速度的是它的範數：曲線對弧長的變化率為

$$\frac{ds}{dt} = \|B'(t)\|.$$

若 $B'(t) = 0$ ，切線方向未定義，必須由呼叫端明確處理，不可直接把它正規化。這可能發生在所有控制點重合、內部相鄰控制點重合，或某個參數恰好使所有一階差分項互相抵消的退化情形。

要看清導數公式的來源，先把 Bernstein 基底對 t 求導。利用乘積法則：

$$\frac{d}{dt} B_{i,n}(t) = n(B_{i-1,n-1}(t) - B_{i,n-1}(t)),$$

其中下標超出 $[0, n-1]$ 的項視為零。代入 $B(t)$ 的定義並重新分組下標，即得到

$$B'(t) = n \sum_{i=0}^{n-1} (P_{i+1} - P_i) B_{i,n-1}(t).$$

這個換索引推導說明兩件事。第一，導數仍是 Bernstein 形式，只是次數低一次；這讓一階導數能直接沿用同樣的基底實作。第二，導數只依賴相鄰控制點的差分 $P_{i+1} - P_i$ ，整條曲線平移不會改變任何切線方向；控制點等距平移時，差分向量不變，切線分布也隨之保持。

在數值上， $\|B'(t)\|$ 可能隨 t 大幅變化。均勻的 Δt 因此不保證均勻的弧長增量；此現象在「弧長重參數化」一節會再完整處理。

de Casteljau 演算法

直接計算高次多項式在端點附近可能較不穩。de Casteljau 演算法透過遞迴線性插值計算曲線點，數值穩定且幾何直覺明確：

1. 設 $Q_i^{(0)} = P_i$ 。
2. 對於層級 $k = 1, \dots, n$ ：

$$Q_i^{(k)} = (1 - t)Q_i^{(k-1)} + tQ_{i+1}^{(k-1)}$$

3. 最終點 $Q_0^{(n)}$ 即為 $B(t)$ 。

此過程可視化為控制多邊形的逐步「收縮」，最終收縮至曲線上的點。第一層結果 $Q_i^{(1)}$ 是控制多邊形邊上的內分點；第二層 $Q_i^{(2)}$ 又是第一層相鄰點之間的內分點，如此遞推。每一步都只是凸組合，不會放大輸入的數值擾動，所以 de Casteljau 法比直接展開高次多項式穩定，特別是在端點附近。

de Casteljau 過程同時也提供曲線分割所需的全部資料。觀察三角形最左側的點序列 $Q_0^{(0)}, Q_0^{(1)}, \dots, Q_0^{(n)}$ 與最右側的點序列 $Q_n^{(0)}, Q_{n-1}^{(1)}, \dots, Q_0^{(n)}$ 。以 $Q_0^{(n)} = B(t)$ 為分界，左半段子曲線的控制點正好是最左側串列，右半段則是最右側串列倒序。只要把這些邊點記錄下來，就得到在參數 t 分割後、兩段各自的 Bézier 控制點，而分割後的兩段仍與原曲線完全重合，不需重新擬合。

這個「一石二鳥」的性質讓 de Casteljau 同時成為遞迴細分與曲線裁切的基礎。若某一子段相對弦的偏差已小於場景容差，即可停止細分並以線段近似；否則以中點 $t = 0.5$ 繼續分裂，直到收斂。以公尺為單位的場景，平坦度容差應以公尺設定，不能與無因次的相對容差共用同一個數字。

連續性： C^k 與 G^k

拼接兩段曲線 A （次數 m ，控制點 $P_0 \dots P_m$ ）與 B （次數 n ，控制點 $Q_0 \dots Q_n$ ）於點 J 時：

- C^0 （位置連續）： $P_m = Q_0$ 。
- G^1 （幾何連續）：接點處切線共線且同向。需滿足 P_{m-1}, P_m, Q_0, Q_1 共線，且 $\langle P_m - P_{m-1}, Q_1 - Q_0 \rangle > 0$ 。
- C^1 （參數連續）：導數向量相等。

$$m(P_m - P_{m-1}) = n(Q_1 - Q_0)$$

僅當 $m = n$ 時，可簡化為控制邊向量相等。

分段 Bézier 樣條

分段樣條由多段 Bézier 曲線拼接而成。

- **局部參數**：每段使用局部參數 $u \in [0, 1]$ 。全域時間需先映射至所屬曲段，再由該段的局部參數求值。
- **局部控制性**：移動某段內部的控制點（非接點），通常只影響該段形狀。若移動接點或為維持連續性連動相鄰控制點，則會影響相鄰段。
- **拼接條件**：位置相接為 C^0 ；切線同向且共線為 G^1 ；導數相等才是相對指定參數的 C^1 。若兩段各自用 $u \in [0, 1]$ ，局部導數相等的條件是 $m(P_m - P_{m-1}) = n(Q_1 - Q_0)$ 。若全域參數區間長度不同，還須按鏈式法則除以各段區間長度，才能保證全域速度向量連續。

全域時間到局部參數

分段樣條常以一個全域參數 $s \in [0, 1]$ （例如動畫時間的正規化值）驅動。若各段的參數範圍不同，需先把 s 映射到對應段的局部參數 u 。最簡單的做法是預先記錄每段在全域的起訖位置 $s_0^{(j)}, s_1^{(j)}$ ，再以線性內插

$$u = \frac{s - s_0^{(j)}}{s_1^{(j)} - s_0^{(j)}}$$

得到該段內的局部參數。若各段以等時間間隔播放，則 $s_0^{(j)}, s_1^{(j)}$ 也等距；若要讓各段以相同的空間速度播放，則應讓各段在全域的權重正比於其弧長。

這一步看似只是索引查找，但若忽略它，直接把 s 當作每段的局部 u ，整個樣條在接點附近的速率可能突然改變，動畫便可能出現頓挫。若兩段弧長差距很大卻以相同時間播放，兩段通過時間相同，但短段的平均速率較低、長段較高。這個平均速率差異不直接決定接點的瞬時速度；接點速度仍須由各段導數及時間尺度核對。

還要區分位置連續與速度連續。各段端點相接只保證 C^0 ；切線同向且共線可保證 G^1 ，但不必保證依全域參數移動時速度相同。令 A 段與 B 段的全域參數區間長度分別為 $\Delta s_A, \Delta s_B$ ，局部參數皆從 0 走到 1。由鏈式法則，局部導數轉成全域導數時須分別除以區間長度，因此全域 C^1 條件為

$$\frac{m(P_m - P_{m-1})}{\Delta s_A} = \frac{n(Q_1 - Q_0)}{\Delta s_B}.$$

若只比較兩段各自的局部參數導數，條件是 $m(P_m - P_{m-1}) = n(Q_1 - Q_0)$ 。兩段全域區間等長時，局部 C^1 條件才等同於全域速度連續；若區間長度不同，單靠局部條件不足以保證接點速度一致。

弧長重參數化

等 Δt 取樣的空間間距，主要由參速 $\|B'(t)\|$ 決定，其近似關係為

$$\Delta s \approx \|B'(t)\| \Delta t.$$

當 $\|B'(t)\|$ 隨 t 大幅變化時，等參數取樣在某些區段密集、在另一些區段稀疏。若要讓網格頂點或動畫取樣在空間上分布均勻，需要進行弧長重參數化：先建立弧長累積表 $L(t)$ ，再對目標弧長反查對應的 t 。

具體演算法如下。以 $M + 1$ 個取樣點 $t_k = k/M$ 計算 $B(t_k)$ ，累積弦長

$$L_k = \sum_{j=1}^k \|B(t_j) - B(t_{j-1})\|, \quad L_0 = 0.$$

L_M 近似曲線總長。給定目標弧長 $\hat{s} \in [0, L_M]$ ，以二分搜尋找 k 使得 $L_k \leq \hat{s} \leq L_{k+1}$ ，再線性內插

$$\alpha = \frac{\hat{s} - L_k}{L_{k+1} - L_k}, \quad t = (1 - \alpha)t_k + \alpha t_{k+1},$$

即得近似的 $t(\hat{s})$ 。若相鄰累積弧長相等，分母為零，不能直接套用此式；建表與反查時必須處理零長曲線和重複累積值，詳見後面的實作。將總弧長等分的 $\hat{s}$ 逐一反查，就得到近似等弧長的參數序列。

等弧長取樣改善的是間距均勻性，並不自動保證折線近似誤差足夠小。高曲率區即使等距取樣，仍可能需要依平坦度或曲率額外細分，才能把折線與曲線的偏差控制在指定容差內。間距均勻化與幾何近似誤差控制是兩個不同問題。

弧長累積表的精度取決於取樣數 M 與曲線彎曲程度。 M 太小可能低估弧長，尤其在折返或高曲率區； M 太大則增加計算量。可比較 M 與 $2M$ 的總長近似值：

$$\frac{|L_{2M} - L_M|}{\max(L_{2M}, \epsilon_L)} < \tau.$$

其中 ϵ_L 是依場景尺度設定的長度容差， τ 是無因次相對差容差。這是解析度收斂檢查，不是嚴格的真實弧長誤差上界；若需要誤差保證，須另採有誤差界的積分或細分方法。

逐步手算例題

例 1：二次 Bézier 中點與切線

控制點： $P_0(0, 0)$, $P_1(2, 2)$, $P_2(4, 0)$ 。求 $t = 0.5$ 的點與切線。

1. 點計算： $B_0(0.5) = 0.25$, $B_1(0.5) = 0.5$, $B_2(0.5) = 0.25$ 。 $B(0.5) = 0.25(0, 0) + 0.5(2, 2) + 0.25(4, 0) = (2, 1)$ 。
2. 切線計算： $B'(t) = 2[(1 - t)(P_1 - P_0) + t(P_2 - P_1)]$ 。 $B'(0.5) = 2[0.5(2, 2) + 0.5(2, -2)] = 2(2, 0) = (4, 0)$ 。切線水平向右。

例 2： G^1 連續性檢查

曲線 A： $P_1(1, 1)$, $P_2(2, 0)$ （末端）。曲線 B： $Q_0(2, 0)$, $Q_1(3, 1)$ （始端）。

1. $C^0: P_2 = Q_0$ ，成立。
2. G^1 ：向量 $v_A = P_2 - P_1 = (1, -1)$ 。向量 $v_B = Q_1 - Q_0 = (1, 1)$ 。外積（2D 行列式）： $1(1) - (-1)(1) = 2 \neq 0$ 。不共線，故不滿足 G^1 。

實作與程式

使用 Python 與 NumPy 實現 Bézier 曲線。

```
import numpy as np
from math import comb

def prepare_points(pts):
    """驗證控制點列表"""
    pts = [np.asarray(p, dtype=float) for p in pts]
    if not pts:
        raise ValueError("至少需要一個控制點")
    shape = pts[0].shape
    if len(shape) != 1 or shape[0] == 0 or any(p.shape != shape for p in pts):
        raise ValueError("所有控制點必須為相同非零維度的一維向量")
    if not all(np.all(np.isfinite(p)) for p in pts):
        raise ValueError("控制點不得含非有限值")
    return pts

def bernstein_basis(n, i, t):
    """計算第 i 個 Bernstein 基底值"""
    if i < 0 or i > n:
        raise ValueError("Index out of range")
    return comb(n, i) * (t ** i) * ((1.0 - t) ** (n - i))

def de_casteljau(pts, t):
    """使用 de Casteljau 演算法計算 Bézier 曲線點"""
    pts = prepare_points(pts)
    current = pts
    for _ in range(len(current) - 1):
        new_pts = []
        for i in range(len(current) - 1):
            interp = (1 - t) * current[i] + t * current[i + 1]
            new_pts.append(interp)
        current = new_pts
    return current[0]

def bezier_tangent(pts, t):
    """計算切線向量（未正規化）"""
    pts = prepare_points(pts)
    n = len(pts) - 1
    if n < 1:
        return np.zeros_like(pts[0])
    diffs = [pts[i + 1] - pts[i] for i in range(n)]
    tangent = np.zeros_like(pts[0])
    for i, dp in enumerate(diffs):
        tangent += dp * bernstein_basis(n - 1, i, t)
    return n * tangent

def check_g1_continuity(pts_a, pts_b,
                        position_tol=1e-9,
```

```

        tangent_tol=1e-12,
        direction_tol=1e-9):
    """檢查兩段曲線在接點處是否 G1 連續（支援 2D/3D）。

    position_tol 與 tangent_tol 是長度量，單位與控制點座標相同；
    direction_tol 是單位方向向量的無因次差容差，兩者不可共用同一個數字。
    """

    if not all(np.isfinite(x) and x >= 0 for x in (position_tol, tangent_tol, direction_tol)):
        raise ValueError("容差必須有限且非負")
    pts_a = prepare_points(pts_a)
    pts_b = prepare_points(pts_b)

    # A、B 兩段必須同維度，否則後續向量運算在此沒有意義。
    if pts_a[0].shape != pts_b[0].shape:
        raise ValueError("兩段曲線的控制點維度必須相同")

    # C0 檢查（關閉相對容差）
    if not np.allclose(pts_a[-1], pts_b[0], rtol=0.0, atol=position_tol):
        return False, "Not C0 continuous"

    if len(pts_a) < 2 or len(pts_b) < 2:
        return False, "Degree too low for G1 check"

    v_a = pts_a[-1] - pts_a[-2]
    v_b = pts_b[1] - pts_b[0]

    # 零向量檢查：長度容差，與座標同單位。
    norm_a = np.linalg.norm(v_a)
    norm_b = np.linalg.norm(v_b)
    if norm_a <= tangent_tol or norm_b <= tangent_tol:
        return False, "Zero tangent at junction"

    # 同向共線檢查：方向差容差，無因次。
    ua = v_a / norm_a
    ub = v_b / norm_b
    if np.linalg.norm(ua - ub) > direction_tol:
        return False, "Tangents not aligned or opposite"

    return True, "G1 Continuous"

def test_fish_outline():
    # 魚背脊控制點（2D）
    ctrl_pts = [
        np.array([0.0, 0.0]),
        np.array([1.0, 0.5]),
        np.array([2.0, 1.0]),
        np.array([3.0, 1.0]),
        np.array([4.0, 0.0])
    ]

    # 1. de Casteljau 一致性測試
    for t in [0.0, 0.25, 0.5, 0.75, 1.0]:
        p1 = de_casteljau(ctrl_pts, t)
        n = len(ctrl_pts) - 1
        p2 = sum(ctrl_pts[i] * bernstein_basis(n, i, t) for i in range(n + 1))
        assert np.allclose(p1, p2, rtol=1e-12, atol=1e-12), f"Mismatch at t={t}"

    # 2. 端點測試
    assert np.allclose(de_casteljau(ctrl_pts, 0.0), ctrl_pts[0], rtol=0.0, atol=1e-12)

```

```

assert np.allclose(de_casteljau(ctrl_pts, 1.0), ctrl_pts[-1], rtol=0.0, atol=1e-12)

# 3. 凸包性質的權重條件測試
for t in np.linspace(0, 1, 100):
    weights = [bernstein_basis(len(ctrl_pts)-1, i, t) for i in range(len(ctrl_pts))]
    assert all(w >= -1e-12 for w in weights), "Negative weight"
    assert abs(sum(weights) - 1.0) < 1e-12, "Weights do not sum to 1"

# 4. G1 連續性測試
pts_b = [ctrl_pts[-1], ctrl_pts[-1] + np.array([1.0, -1.0])]
is_g1, msg = check_g1_continuity(ctrl_pts, pts_b)
print(f"G1 Check: {msg}")
assert is_g1, "G1 check failed"

if __name__ == "__main__":
    test_fish_outline()

```

弧長重參數化實作

下列函式對應上一節的弧長累積表，可直接接在 `test_fish_outline` 之後使用：

```

def build_arc_length_table(pts, m=200):
    """建立離散弧長表，回傳參數表、累積弧長表與總長近似值。"""
    if isinstance(m, (bool, np.bool_)) or not isinstance(m, (int, np.integer)) or m < 1:
        raise ValueError("m 必須是正整數")
    pts = prepare_points(pts)
    ts = [k / m for k in range(m + 1)]
    samples = [de_casteljau(pts, t) for t in ts]
    arc = [0.0]
    for k in range(1, m + 1):
        arc.append(arc[-1] + float(np.linalg.norm(samples[k] - samples[k - 1])))
    return ts, arc, arc[-1]

def parameter_at_length(ts, arc, target_s, length_tol=1e-12):
    """由弧長表反查近似參數；length_tol 須按場景尺度調整。"""
    ts = np.asarray(ts, dtype=float)
    arc = np.asarray(arc, dtype=float)
    if ts.ndim != 1 or arc.ndim != 1 or len(ts) != len(arc) or len(ts) < 2:
        raise ValueError("ts 與 arc 必須是一維、等長且至少有兩項")
    if not np.isfinite(length_tol) or length_tol < 0 or not np.isfinite(target_s):
        raise ValueError("目標長度與非負長度容差必須有限")
    if not (np.all(np.isfinite(ts)) and np.all(np.isfinite(arc))):
        raise ValueError("弧長表不得含非有限值")
    if ts[0] != 0 or ts[-1] != 1 or np.any(np.diff(ts) <= 0):
        raise ValueError("參數須從0嚴格遞增至1")
    if arc[0] != 0 or np.any(np.diff(arc) < 0):
        raise ValueError("累積弧長須從0非遞減")
    total = arc[-1]
    if target_s < -length_tol or target_s > total + length_tol:
        raise ValueError("target_s 超出曲線總弧長範圍")
    target_s = min(max(target_s, 0.0), total)

    if total <= length_tol:
        if target_s <= length_tol:
            return ts[0]

```

```

        raise ValueError("零長曲線只接受零弧長")
    if target_s <= length_tol:
        return ts[0]
    if total - target_s <= length_tol:
        return ts[-1]

    # 取第一個嚴格大於 target_s 的累積值，略過重複值。
    hi = next(i for i, value in enumerate(arc) if value > target_s)
    lo = hi - 1
    denom = arc[hi] - arc[lo]
    if denom <= 0:
        raise ValueError("反查區間必須有正長度")
    alpha = (target_s - arc[lo]) / denom
    return (1.0 - alpha) * ts[lo] + alpha * ts[hi]

```

先建一次弧長表，再用回傳的總長與表反查多個目標值，避免重複求值。 `m=200` 只是示範初值，不保證特定精度；可比較 M 與 $2M$ 的總長近似值，但此收斂檢查不是嚴格誤差上界。

直線控制點 $(0, 0), (2, 0)$ 的離散總長為 2，`target_s=0.5` 預期反查為 $t = 0.25$ 。所有控制點重合時，總長為零；容差內的零弧長回傳起點參數，不會除以零；超出容差範圍的目標弧長報錯。

測試與預期結果

運行上述程式，預期輸出：

1. 無例外錯誤：所有 `assert` 通過。
2. G1 檢查：G1 Check: G1 Continuous。
3. 數值一致性：de Casteljau 與直接基底計算結果在 10^{-12} 容差內一致。

手算對照：

- $t = 0.5$ 時， $B(0.5) = (2.0, 0.75)$ 。
- 起點切線 $B'(0) = 4(1, 0.5) = (4, 2)$ 。

除錯與常見陷阱

1. NumPy API 相容性：

- 問題：NumPy 2.x 不建議使用 `np.math` 別名。
- 解決：使用標準庫 `math.comb` 計算二項式係數，避免浮點精度問題與 API 變更風險。

2. G^1 與 C^1 混淆：

- 問題：僅檢查共線性 (G^1) 而未檢查長度比例，導致 C^1 需求未滿足。
- 解決：若需 C^1 ，必須驗證 $mv_A = nv_B$ 。若 $m \neq n$ ，控制邊長度不等亦可能滿足 C^1 。

3. 凸包測試誤解：

- **問題：**僅檢查邊界盒 (AABB) 不足以驗證凸包性質。
- **解決：**驗證 Bernstein 權重非負且和為 1，這直接保證仿射組合位於凸包內。若需幾何驗證，可另加凸包演算法。

4. 參數化不均勻：

- **問題：**等 Δt 不等於等弧長。
- **解決：**生成網格頂點時，應進行弧長重參數化 (Arc-length Reparameterization)，透過累積弧長表映射 t 值，使頂點空間間距較均勻。注意這只是間距均勻化；控制折線近似誤差仍需依平坦度或曲率額外細分。

5. 零切線與尖點：

- **問題：**若某個 t 使 $B'(t) = 0$ ，方向未定義，直接正規化會得到 `nan`。重合控制點是常見成因。
- **解決：**在計算切線前先檢查範數是否低於場景長度容差；退化時應明確拒絕，或改用以相鄰取樣點差分估計方向。切線長度容差以公尺為單位設定，不可與無因次的方向差容差共用同一個數字。

6. 兩段維度不同：

- **問題：**把 2D 段與 3D 段直接拼接，`np.allclose` 會以廣播比較而靜默給出錯誤結果。
- **解決：**在拼接檢查一開始就驗證兩段維度相同，否則拋出明確的 `ValueError`。

7. 反向切線：

- **問題：**若只測試外積為零， v_A 與 v_B 反向也會被判為共線，但它其實不滿足 G^1 的同向要求。
- **解決：**除以範數比較單位方向，要求兩者距離小於方向容差；這同時排除反向與交叉的情形。

養殖數位分身案例

在養殖場數位分身中，魚體輪廓可建模為分段 Bézier 曲線：

1. **脊柱路徑：**使用多段 Bézier 曲線連接關鍵姿勢點，確保 G^1 連續以避免游動時產生尖角。
2. **輪廓生成：**沿脊柱法向偏移控制點，生成魚體上下輪廓。
3. **動畫驅動：**若整段曲線的所有控制點施加同一仿射變換，曲線求值與該變換可交換。若控制點分別受不同骨骼影響，則只是以變形後控制點重新定義曲線，需另行驗證拼接連續性與形狀品質。
4. **驗證：**利用凸包性質檢查魚體是否異常膨脹；利用切線連續性檢查鰭部連接處是否平滑。

限制：Bézier 曲線局部控制性較差，高次曲線移動一個控制點會影響整條曲線。對於複雜形態，建議使用分段低次 Bézier 樣條。

分段以後，魚體輪廓可拆成「頭部、軀幹、尾柄」三段低次曲線，接點附近的控制點可一致化以維持 G^1 ；魚鰭另建局部曲面，避免控制點互相牽動。動畫若以弧長重參數化播放，游動速度在整條魚體上會較一致，不會在接點附近突然加快。

這些結論都是針對合成幾何的形狀與運動品質。它們不能推論真實魚體生理或行為，也不能代替任何生物或水產量測。

習題

- 手算：**給定三次 Bézier 控制點 $P_0(0, 0), P_1(1, 1), P_2(2, 1), P_3(3, 0)$ 。(a) 計算 $B(0.5)$ 。(b) 計算 $B'(0)$ 。(c) 判斷曲線是否關於 $x = 1.5$ 對稱。
- 程式測試：**修改 `test_fish_outline`，增加一個三次曲線，控制點為 $Q_0(0, 0), Q_1(0, 2), Q_2(2, 2), Q_3(2, 0)$ 。驗證 $B(0.5)$ 是否為 $(1.0, 1.5)$ 。
- 反例／除錯：**考慮兩段二次 Bézier 曲線：A: $P_0(0, 0), P_1(1, 1), P_2(2, 0)$ B: $Q_0(2, 0), Q_1(3, 0), Q_2(4, 0)$ (a) 它們在 $x = 2$ 處是否 C^0 連續？(b) 它們在 $x = 2$ 處是否 G^1 連續？(c) 若保持 Q_0, Q_2 不變，如何調整 Q_1 使其達到 C^1 連續？
- 整合應用：**使用 de Casteljau 演算法，為以下 5 個控制點生成 $t = 0, 0.1, \dots, 1.0$ 共 11 個點： $P_0(0, 0), P_1(1, 1), P_2(3, 1), P_3(4, -1), P_4(5, 0)$ 。計算相鄰點間弦長，找出弦長最大的區間。討論為何等參數取樣會導致弧長不均，對網格生成的影響。

習題解答

- 手算解答：**(a) $B(0.5) = 0.125P_0 + 0.375P_1 + 0.375P_2 + 0.125P_3$ 。 $x = 0 + 0.375 + 0.75 + 0.375 = 1.5$ $y = 0 + 0.375 + 0.375 + 0 = 0.75$ 點為 $(1.5, 0.75)$ 。(b) $B'(0) = 3(P_1 - P_0) = 3(1, 1) = (3, 3)$ 。(c) 控制點 y 座標 $0, 1, 1, 0$ 關於 $x = 1.5$ 對稱， x 座標 $0, 1, 2, 3$ 均勻分佈。曲線對稱。
- 程式測試解答：** $B(0.5) = 0.125(0, 0) + 0.375(0, 2) + 0.375(2, 2) + 0.125(2, 0)$ 。 $x = 0 + 0 + 0.75 + 0.25 = 1.0$ $y = 0 + 0.75 + 0.75 + 0 = 1.5$ 點為 $(1.0, 1.5)$ 。驗證通過。
- 反例解答：**(a) $P_2(2, 0) = Q_0(2, 0)$ ，是 C^0 連續。(b) $v_A = P_2 - P_1 = (1, -1)$ $v_B = Q_1 - Q_0 = (1, 0)$ 。外積 $1(0) - (-1)(1) = 1 \neq 0$ 。不共線，非 G^1 。(c) 要滿足 C^1 ，需 $2v_A = 2v_B$ （因兩次曲線 $m = n = 2$ ）。即 $v_B = v_A = (1, -1)$ 。 $Q_1 - Q_0 = (1, -1) \implies Q_1 = (2, 0) + (1, -1) = (3, -1)$ 。故 Q_1 應調整為 $(3, -1)$ 。
- 整合應用解答：**控制點： $(0, 0), (1, 1), (3, 1), (4, -1), (5, 0)$ 。先展開為四次多項式：

$$x(t) = 4t + 6t^2 - 8t^3 + 3t^4, \quad y(t) = 4t - 6t^2 - 4t^3 + 6t^4.$$

以 $t = 0, 0.1, \dots, 1$ 取樣，依公式計算的預期結果如下：

t	$B(t)$
0.0	(0.0000, 0.0000)

t	$B(t)$
0.1	(0.4523, 0.3366)
0.2	(0.9808, 0.5376)
0.3	(1.5483, 0.6006)
0.4	(2.1248, 0.5376)
0.5	(2.6875, 0.3750)
0.6	(3.2208, 0.1536)
0.7	(3.7163, -0.0714)
0.8	(4.1728, -0.2304)
0.9	(4.5963, -0.2394)
1.0	(5.0000, 0.0000)

相鄰弦長依序約為 0.5638, 0.5654, 0.5710, 0.5799, 0.5857, 0.5775, 0.5443, 0.4834, 0.4236, 0.4693。最大弦長約 0.5857，落在參數區間 $t \in [0.4, 0.5]$ 。

討論：等 Δt 下的空間間距主要由參速 $\|B'(t)\|$ 決定， $\Delta s \approx \|B'(t)\| \Delta t$ 。當 $\|B'(t)\|$ 變化大時，弧長間距就不均勻。曲率較大不必然使弦長較大，兩者是不同的量。**解決方向：**弧長重參數化可使頂點間距較均勻；但為控制折線近似誤差，高曲率區仍可能需要依平坦度或曲率條件額外細分，這是與間距均勻化分開的問題。

本章小結

本章建立了 Bézier 曲線與分段樣條的數學基礎。關鍵要點：

1. Bernstein 基底確保仿射組合與凸包性質。
2. de Casteljau 演算法提供數值穩定的求值方法。
3. G^1 與 C^1 連續性條件需嚴格區分，特別是在不同次數曲線拼接時。
4. 參數均勻不等於弧長均勻：等參數取樣的空間間距差異來自 $\|B'(t)\|$ 的變化。弧長重參數化可以改善間距均勻性，但不單獨保證幾何近似誤差；高曲率區仍需依平坦度或曲率條件額外細分。

這些概念是後續曲面建模與物理模擬的基礎。

參考來源

1. G1 PBRT 4: Transformations. (仿射變換與曲線關係)

2. **G4** Ray Tracing in One Weekend. (幾何基礎與投影)
3. **G7** NumPy 線性代數參考. (向量運算與矩陣)

第 10 章 參數曲面與曲面離散化

學習目標與先備知識

本章從連續參數曲面推導切向量、切平面與法線，再將曲面取樣成可供渲染使用的三角網格。完成後，你應能：

- 用參數方程描述曲面，並由偏導數取得局部切向量。
- 用切向量外積計算依參數順序定向的法線，辨認退化情形。
- 選擇取樣數與索引連接方式，建立頂點、三角形及 UV 座標。
- 處理旋轉曲面的極點、週期接縫及退化三角形。
- 用 Python 標準庫產生簡化魚體 OBJ，並測試索引、面積與繞序。

先備為向量、內積、外積、三角函數及矩陣的基本概念。長度以公尺表示，角度以弧度表示；座標遵守右手系，三角形外向前面依外向法線觀看時為逆時針。網格取樣只近似幾何，不保證模型符合真實魚類形態或生理特性。

問題與直覺

魚身表面可視為連續曲面，但圖形管線通常以有限個三角形表示它。參數方程回答「如何由參數得到位置」；離散化則回答「取哪些參數位置，以及如何把它們連成面」。

令魚身長軸沿X軸。每個X位置都有橫截面，截面的半長軸、半短軸隨X改變；再以角度繞X軸掃過一圈，就能生成簡化魚身。

離散化時要處理兩個問題：

1. **取樣密度**：取樣過疏，曲面顯出折角；取樣加密則增加頂點與面數。
2. **邊界與退化**：角度繞一圈後首尾相接；若端點截面半徑縮為零，整圈頂點重合，可能形成零面積三角形。

因此，網格不能只看起來像魚；還要檢查索引、三角形面積、繞序與接縫。

數學與幾何推導

參數曲面與偏導數

令參數為 u, v ，定義曲面

$$\mathbf{S}(u, v) = \begin{pmatrix} x(u, v) \\ y(u, v) \\ z(u, v) \end{pmatrix}.$$

對兩個參數分別微分，得到

$$\mathbf{S}_u = \frac{\partial \mathbf{S}}{\partial u}, \quad \mathbf{S}_v = \frac{\partial \mathbf{S}}{\partial v}.$$

偏導向量描述位置隨單一參數微小變化的方向。若曲面可微且兩向量不平行，便張成局部切平面。以參數點 (u_0, v_0) 為基準，切平面為

$$\mathbf{X}(a, b) = \mathbf{S}(u_0, v_0) + a\mathbf{S}_u(u_0, v_0) + b\mathbf{S}_v(u_0, v_0),$$

其中 a, b 為任意實數。依參數順序定向的單位法線為

$$\mathbf{n}_{uv} = \frac{\mathbf{S}_u \times \mathbf{S}_v}{\|\mathbf{S}_u \times \mathbf{S}_v\|}.$$

交換外積順序會反轉方向。這個公式不保證所得法線朝向曲面的外側；外向方向須由參數化方向或參考方向判定。若外積長度為零，兩切向量平行或至少一者為零，這個參數化在該處不能提供唯一法線，不能直接正規化。

旋轉曲面模型

令 x 為沿魚身的參數， θ 為繞魚身的角度。以 $r_y(x)$ 、 $r_z(x)$ 表示橫截面在 Y、Z 方向的半徑，則

$$\mathbf{S}(x, \theta) = \begin{pmatrix} x \\ r_y(x) \cos \theta \\ r_z(x) \sin \theta \end{pmatrix}, \quad 0 \leq \theta \leq 2\pi.$$

這是橢圓截面；若 $r_y = r_z$ ，截面為圓。其偏導數為

$$\mathbf{S}_x = \begin{pmatrix} 1 \\ r'_y(x) \cos \theta \\ r'_z(x) \sin \theta \end{pmatrix}, \quad \mathbf{S}_\theta = \begin{pmatrix} 0 \\ -r_y(x) \sin \theta \\ r_z(x) \cos \theta \end{pmatrix}.$$

依 $\mathbf{S}_x \times \mathbf{S}_\theta$ 次序，外積為

$$\mathbf{S}_x \times \mathbf{S}_\theta = \begin{pmatrix} r'_y r_z \cos^2 \theta + r'_z r_y \sin^2 \theta \\ -r_z \cos \theta \\ -r_y \sin \theta \end{pmatrix}.$$

若半徑沿長軸變化不過陡，魚身外向法線採用相反次序：

$$\mathbf{n}_{\text{out}} \propto \mathbf{S}_\theta \times \mathbf{S}_x.$$

實際網格的法線由三角形索引繞序決定，仍須以面積向量和外向參考方向檢查。

取樣、面片與接縫

將 x 分成 N_x 段、角度分成 N_θ 段：

$$x_i = x_{\min} + \frac{i}{N_x}(x_{\max} - x_{\min}), \quad i = 0, \dots, N_x,$$

$$\theta_j = \frac{2\pi j}{N_\theta}, \quad j = 0, \dots, N_\theta.$$

保留首尾重複的角度欄時，頂點索引為

$$k(i, j) = i(N_\theta + 1) + j.$$

雖然 $\theta = 0$ 與 2π 的位置相同，仍保留兩份頂點，使 UV 的 v 可以由 0 走到 1，而不在貼圖座標上跳回 0。此處約定 $u = i/N_x$ 沿魚身， $v = j/N_\theta$ 沿角度。若不需 UV 接縫，也可只存 N_θ 個角度頂點，並以模數回接。

令四邊形角點為

$$a = k(i, j), \quad b = k(i + 1, j), \quad c = k(i + 1, j + 1), \quad d = k(i, j + 1).$$

可用 (a, b, c) 、 (a, c, d) 填滿四邊形。以 $\theta = 0$ 附近的中段為例，令 $\mathbf{S}_x \approx (1, r'_y, 0)$ 、 $\mathbf{S}_\theta \approx (0, 0, r_z)$ ，則

$$\mathbf{S}_x \times \mathbf{S}_\theta = (r'_y r_z, -r_z, 0),$$

其 Y 分量朝向 $-Y$ ，而此處魚身外側朝 $+Y$ 。因此上述面片順序朝內，需反轉為 (a, c, b) 、 (a, d, c) 。後面的程式採用此繞序，並測試中段面的面積向量是否朝外。

極點退化

若端點半徑同時為零，該端每個角度樣本都落在同一位置，形成極點。常見處理方式有：

- **共用單一極點頂點**：相鄰環帶的面都連到同一點。拓撲較精簡，但 UV 與法線需特別安排。
- **保留重複極點並略去退化面**：實作直接，但可能留下非標準拓撲；容差及極點 UV、法線也須另外處理。

本章程式採第二種方式，並在產生三角形時略過面積不大於容差的面。若需要封閉流形網格，應另建極點拓撲並檢查邊鄰接。

均勻取樣不代表固定的表面誤差。曲率較大的區域通常需要更密取樣；只增加整體取樣數可能使平坦區域過度細分。可進一步依弦高誤差或曲率自適應取樣。

逐步手算例題

例一：計算切向量與法線

考慮半徑為1公尺的圓柱：

$$\mathbf{S}(x, \theta) = \begin{pmatrix} x \\ \cos \theta \\ \sin \theta \end{pmatrix}.$$

在 $x = 0, \theta = 0$ ：

1. 對 x 微分，得 $\mathbf{S}_x = (1, 0, 0)^T$ 。
2. 對 θ 微分，得 $\mathbf{S}_\theta = (0, -\sin \theta, \cos \theta)^T$ ，因此此處 $\mathbf{S}_\theta = (0, 0, 1)^T$ 。
3. 依 $\mathbf{S}_x \times \mathbf{S}_\theta$ 次序：

$$\mathbf{S}_x \times \mathbf{S}_\theta = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \times \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix}.$$

此法線長度為1，方向朝 $-Y$ 。圓柱在該點的外側朝 $+Y$ ，所以外向法線應使用 $\mathbf{S}_\theta \times \mathbf{S}_x$ ，或反轉網格索引繞序。

例二：接縫上的離散取樣

令 $N_\theta = 4$ ，半徑為1，固定 $x = 0$ 。角度樣本為

$$0, \quad \frac{\pi}{2}, \quad \pi, \quad \frac{3\pi}{2}, \quad 2\pi.$$

其位置依序為

$$(0, 1, 0), (0, 0, 1), (0, -1, 0), (0, 0, -1), (0, 1, 0).$$

首尾位置相同，但參數相差 2π 。依本章 UV 約定，兩者的 v 分別為0與1。最後一個角度區段連接第3點與第4點，建立索引時不可漏掉。此取樣以四個區段近似圓周；增加 N_θ 會改善輪廓近似，不改變接縫處理原則。

實作與程式

以下程式只使用 Python 3.10+ 標準庫。它建立簡化魚體，寫出含頂點、UV與三角面的 OBJ，並測試接縫、索引、退化面與外向繞序。魚身長軸為X軸，兩端縮尖；這是教學用合成幾何，不是生物量測模型。

網格長度單位為公尺；OBJ 面索引從1開始。twice_area 是兩倍三角形面積，量綱為平方公尺。面法線可由三角形邊向量外積得到。

```
import math

def cross(a, b):
    return (
        a[1] * b[2] - a[2] * b[1],
        a[2] * b[0] - a[0] * b[2],
        a[0] * b[1] - a[1] * b[0],
    )

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1], a[2] - b[2])

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def length(v):
    return math.sqrt(dot(v, v))

def fish_radii(x):
    # 魚身長為 2 公尺；端點明確設為精確極點。
    t = (x + 1.0) / 2.0
    if t == 0.0 or t == 1.0:
        return 0.0, 0.0
    profile = math.sin(math.pi * t)
    return 0.34 * profile, 0.24 * profile

def make_fish(nx=20, ntheta=24):
    if nx < 2 or ntheta < 3:
        raise ValueError("nx 至少為 2，ntheta 至少為 3")

    vertices = []
    uvs = []

    # 角度接縫兩側各存一欄，角度欄數為 ntheta + 1。
    for i in range(nx + 1):
        x = -1.0 + 2.0 * i / nx
        ry, rz = fish_radii(x)
        for j in range(ntheta + 1):
            theta = 2.0 * math.pi * j / ntheta
            vertices.append((
                x,
                ry * math.cos(theta),
                rz * math.sin(theta),
            ))
            uvs.append((i / nx, j / ntheta))

    def index(i, j):
        return i * (ntheta + 1) + j
```



```

faces = []
area_epsilon = 1e-12 # 本例尺度的平方公尺容差。

for i in range(nx):
    for j in range(ntheta):
        a = index(i, j)
        b = index(i + 1, j)
        c = index(i + 1, j + 1)
        d = index(i, j + 1)

        # 反轉繞序，使中段外側面法線朝向魚身外部。
        for tri in ((a, c, b), (a, d, c)):
            p0, p1, p2 = (vertices[k] for k in tri)
            twice_area = length(cross(sub(p1, p0), sub(p2, p0)))
            if twice_area > area_epsilon:
                faces.append(tri)

return vertices, uvs, faces

def validate(vertices, uvs, faces):
    assert len(vertices) == len(uvs)
    assert all(len(p) == 3 for p in vertices)
    assert all(len(uv) == 2 for uv in uvs)

    for tri in faces:
        assert len(set(tri)) == 3
        assert all(0 <= k < len(vertices) for k in tri)
        p0, p1, p2 = (vertices[k] for k in tri)
        twice_area = length(cross(sub(p1, p0), sub(p2, p0)))
        assert twice_area > 1e-12

def write_obj(path, vertices, uvs, faces):
    with open(path, "w", encoding="utf-8") as f:
        f.write("# 合成簡化魚體；長度單位為公尺\n")
        for x, y, z in vertices:
            f.write(f"v {x:.9g} {y:.9g} {z:.9g}\n")
        for u, v in uvs:
            f.write(f"vt {u:.9g} {v:.9g}\n")
        for a, b, c in faces:
            a += 1
            b += 1
            c += 1
            f.write(f"f {a}/{a} {b}/{b} {c}/{c}\n")

def test_fish():
    nx, ntheta = 20, 24
    vertices, uvs, faces = make_fish(nx, ntheta)
    validate(vertices, uvs, faces)

    columns = ntheta + 1

    # 每個長度環的接縫位置重合，UV 的角度分量分別為 0 與 1。
    for i in range(nx + 1):
        first = vertices[i * columns]
        seam = vertices[i * columns + ntheta]

```

```

    assert length(sub(first, seam)) < 1e-12
    assert uvs[i * columns][1] == 0.0
    assert uvs[i * columns + ntheta][1] == 1.0

# 兩端各欄均重合；退化三角形不得出現在輸出面中。
for i in (0, nx):
    base = i * columns
    for j in range(1, columns):
        assert length(sub(vertices[base], vertices[base + j])) == 0.0
assert all(
    not all(i == 0 for i in tri) and
    not all(i == nx * columns for i in tri)
    for tri in (
        tuple(k // columns for k in face)
        for face in faces
    )
)

# 檢查一個非退化中段面：面積向量須朝向該處魚身外側。
i, j = nx // 2, 0
a = i * columns + j
b = (i + 1) * columns + j
c = (i + 1) * columns + j + 1
expected = (0.0, 1.0, 0.0)
p0, p1, p2 = (vertices[k] for k in (a, c, b))
area_vector = cross(sub(p1, p0), sub(p2, p0))
assert dot(area_vector, expected) > 0.0

if __name__ == "__main__":
    test_fish()
    vertices, uvs, faces = make_fish()
    write_obj("fish.obj", vertices, uvs, faces)
    print(len(vertices), len(faces))

```

測試中的極點面檢查以頂點所屬長度環編號辨認兩端；被保留的面不能整個落在同一個端點環上。接縫測試使用浮點容差比較位置，而不要求一般三角函數運算所得座標位元完全相同。

OBJ只描述位置、UV與面索引，不含頂點法線。平滑著色通常需要額外計算並寫入 `vn`；一種常見方法是累加相鄰面的面積加權法線再正規化，但接縫及極點處仍須依模型需求決定是否共用法線。

測試與預期結果

將程式存為 `surface_fish.py`，在有 Python 3.10+ 的環境執行：

```
python surface_fish.py
```

程式會呼叫內建測試並寫出 `fish.obj`。依預設參數 $N_x = 20$, $N_\theta = 24$ ，頂點數為

$$(20 + 1)(24 + 1) = 525.$$

未略去退化面時，原始四邊形網格可拆成 $2 \cdot 20 \cdot 24 = 960$ 個三角形。兩端各有24個角度區段；依此參數化，兩端各有每區段一個退化三角形。若只略去這些面，預期面數為

$$960 - 2 \cdot 24 = 912.$$

以上是依程式網格構造推算的預期，不代表已執行或檢視OBJ。因程式另有面積容差，修改尺度或取樣數後，應重新檢查被略去面的數量。

檢查OBJ時至少確認：

- **v** 行為有限數值，範圍符合公尺尺度預期。
- **f** 行索引不超出頂點數，且三角形沒有重複索引。
- 接縫環首尾位置相同，UV角度分量分別為0與1。
- 極點未留下退化面；中段面的繞序朝向外側。

除錯與常見陷阱

- **外積方向與面片繞序混淆**：交換外積順序會反轉法線；改變索引繞序則改變網格面的方向。除錯時先固定三角形索引，再計算面積向量並與外向參考方向作內積。
- **接縫重複頂點被誤認為裂縫**：接縫兩側位置相同但UV不同，是常見設計。若焊接位置相同的頂點，需另外保留UV接縫。
- **用精確相等判斷一般浮點幾何**：三角函數結果可能有浮點誤差；比較位置通常應採與模型尺度相稱的容差。程式只對明確回傳的零半徑端點使用精確位置測試。
- **極點退化被容差掩蓋**：容差過大會刪除有效小面，過小則留下近零面積面。容差量綱須與面積一致，並配合模型尺度選擇。
- **UV方向與影像方向不同**：本章沿魚身的參數是 u ，沿角度的是 v 。讀入影像時應明確說明是否翻轉影像列；世界Y向上與影像列向下是不同約定。
- **均勻參數取樣不等於均勻幾何品質**：輪廓變化較快處可能需要加密。可先從低面數檢查輪廓，再依誤差需求調整。

養殖數位分身案例

在合成養殖場場景中，可把魚體長度設定為2公尺，令X軸表示魚身長軸，並以 $r_y(x)$ 、 $r_z(x)$ 控制寬高。生成後可將模型放入池體場景圖，套用世界變換、材質與姿勢動畫。此魚體是用於測試渲染、遮擋、相機視角及幾何流程的合成資產，不是某一物種的量測或生物學驗證。

資產檢查可分三層：

1. **參數層**：半徑非負，長度與半徑使用相同單位。
2. **網格層**：索引有效、面積大於容差、繞序一致、接縫座標可對應。
3. **場景層**：網格局部到世界的變換與池體、相機座標一致；若使用非均勻縮放，法線須以線性變換的逆轉置處理。

渲染外形平滑，不能證明解剖形狀、運動或水中行為真實。若任務要求對照實際個體，需另有量測資料與明確驗證方法。

習題

習題一：手算法線

對曲面

$$\mathbf{S}(u, v) = (u, 2 \cos v, \sin v)^T$$

計算 $\mathbf{S}_u$ 、 $\mathbf{S}_v$ ，並求 $v = 0$ 時依照 $\mathbf{S}_u \times \mathbf{S}_v$ 得到的未正規化法線與單位法線。

習題二：程式測試

將程式的 `ntheta` 改為12。說明如何逐一檢查各長度環的角度接縫位置；指出應比較哪些索引，以及位置和UV應有何性質。

習題三：反例與除錯

某人以頂點 (a, b, c, d) 建立四邊形，產生三角形 (a, c, b) 、 (a, d, c) ，之後發現法線朝內。說明為何不能只靠交換 `cross` 運算順序修正網格，並給出可靠的診斷步驟。

習題四：整合應用

建立一個長度1.6公尺的局部魚身，取樣角度24段、長度16段。計算保留角度接縫頂點時的頂點數、四邊形數及未處理極點時的理論三角形數。再說明尖端極點的兩種處理方式及各自代價。

習題解答

解答一

$$\mathbf{S}_u = (1, 0, 0)^T, \quad \mathbf{S}_v = (0, -2 \sin v, \cos v)^T.$$

在 $v = 0$ ， $\mathbf{S}_v = (0, 0, 1)^T$ ，因此

$$\mathbf{S}_u \times \mathbf{S}_v = (0, -1, 0)^T.$$

其長度為1，故單位法線亦為 $(0, -1, 0)^T$ 。

解答二

保留接縫頂點時，每個長度環有 $12 + 1 = 13$ 個角度欄。對每一個環 i ，比較索引 $i * 13$ 與 $i * 13 + 12$ ：位置應在浮點容差內相同，UV角度分量 v 則分別為0與1。這是逐環檢查接縫；若還要檢查面索引，可確認每個角度區段都包含由欄11連到欄12的最後一段。

解答三

交換外積順序會反轉計算所得法線，卻不會改變三角形索引，也不會修正相鄰面間不一致的繞序。可先選一個非退化三角形，依其索引計算

$$(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0),$$

再將面中心至預期外部的參考方向與面積向量作內積。內積為負表示該面繞序與外向方向相反；交換後兩個頂點索引，再檢查相鄰面、接縫與封口是否一致。

解答四

$N_x = 16$ 、 $N_\theta = 24$ 。保留角度接縫重複頂點時：

$$(16 + 1)(24 + 1) = 425$$

個頂點。四邊形數為

$$16 \cdot 24 = 384,$$

拆成三角形後，未略去退化面的理論數為 768。

尖端可共用單一極點，讓相鄰環帶三角形扇形連接；代價是極點UV與法線要仔細處理。也可保留重複極點並略去零面積或低於容差的三角形；代價是容差須配合尺度選定，拓撲也未必是理想流形。

本章小結

參數曲面以 $\mathbf{S}(u, v)$ 將參數映射至三維位置；偏導向量張成切平面，外積得到依參數順序定向的法線。將有限參數樣本連成三角形，就得到離散網格。旋轉曲面的角度接縫須兼顧幾何閉合與UV表示；半徑縮至零的極點可能產生退化面，需用拓撲設計或面積檢查處理。取樣與容差應配合幾何尺度及用途；能渲染不等於經過生物或物理驗證。

參考來源

以下資料可供延伸閱讀；本章以自身符號與例子推導，未照錄來源段落。

- [G1] PBRT 4 : Transformations , https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] LearnOpenGL : Transformations , <https://learnopengl.com/Getting-started/Transformations>
- [G7] NumPy 線性代數參考 , <https://numpy.org/doc/stable/reference/routines.linalg.html>

第 11 章 程序化建模與幾何品質

學習目標與先備知識

本章把曲線與曲面章節留下的參數化工具，接上「如何用少量程式碼生成可用的網格」這個問題。讀完之後你應該能夠：

- 用掃掠（sweep）操作，把 2D 截面沿一條引導曲線生成封閉或開放曲面，並判斷何時需要沿曲線扭轉或調整尺度。
- 用局部框架（frame）將截面定向到引導曲線的每個位置，並說明 Frenet frame 在直線段的失敗情形與替代做法。
- 產生同一物件的多層細節（LOD），並用包圍體（bounding volume）快速篩選。
- 檢查程序化網格的三類幾何品質問題：自交、退化面、尺度失衡，並提出對應修正。
- 將魚身、魚鰭、池體等參數化模型輸出為 OBJ，並以獨立程式驗證索引、繞序與邊界。

先備：Volume I 的向量、內積、外積、矩陣、Python 基本資料結構；本卷前面章節的 Bézier 曲線、曲面離散化、繞序與法線方向。所有長度單位為公尺、角度為弧度。全書右手世界系，+X 向右、+Y 向上、+Z 由畫面向觀者；三角形前面按外向法線看為逆時針。

問題與直覺

若你只有低面數魚體與池體，最省力的生成方式不是逐頂點手打座標，而是先描述「一段輪廓」和「一條骨架」，再讓輪廓沿骨架掃過去。方法有兩個好處：

1. 語意清楚。修改骨架曲率就整體改變魚身形態，不必重做每個頂點。
2. 品質可驗。截面與骨架都有解析參數，退化與自交能被演算法檢查，不靠肉眼。

掃掠的核心問題是「截面放在哪裡、朝哪裡」。這需要對骨架每個取樣位置配置一組局部框架：一個沿骨架的切向量，加上兩個與它垂直的向量，共同定義該位置的局部 (u, v) 平面。截面若有對稱性，截面頂點就依這個平面放到世界空間。

包圍體與 LOD 是同一個問題的兩面：渲染時不想對上萬個面逐一檢查，所以先用包圍體粗篩；遠距離時不想處理全部細節，所以換低面數版本。兩者都建立在「掃掠結果的座標範圍」這個簡單量上。

數學與幾何推導

掃掠的形式定義

設骨架為參數曲線 $c: [0, 1] \rightarrow \mathbb{R}^3$ ，截面為封閉 2D 折線 $\{(u_i, v_i)\}_{i=0}^{n-1}$ ， u_i, v_i 無單位（比例座標）。在骨架參數 t 處，我們需要一組正交基底 $\{T(t), N(t), B(t)\}$ ： T 為切向量、 N 為法向、 $B = T \times N$ 為副法向。掃掠面為

$$S(t, i) = c(t) + s(t)(u_i N(t) + v_i B(t)),$$

其中 $s(t)$ 是沿骨架的尺度函數（可讓魚頭較窄、魚腹較寬）。若截面封閉， i 對 n 取模；若骨架端點要封口，需另加頂點。

對骨架與截面取樣離散化後：骨架 n_t 個樣本 $c_k = c(t_k)$ 、截面 n_u 個樣本，得到 $n_t n_u$ 個頂點，每個相鄰環之間由 n_u 個四邊形構成，共 $2 n_u (n_t - 1)$ 個三角形（封閉截面）或較少（開放截面）。

局部框架

最直接的框架是 Frenet frame：

$$T = \frac{c'}{\|c'\|}, \quad N = \frac{T'}{\|T'\|}, \quad B = T \times N.$$

它只在 $c'' \neq 0$ 且 $T' \neq 0$ 時有定義。骨架出現直線段時 $T' = 0$ ， N 無法決定；骨架出現拐點時 N 會突然翻向，掃掠面就會扭斷。程序化建模不該依賴這種脆弱性，實務上改用參考向量法：

1. 先固定一個世界參考向量 R （例如 $+Y$ ）。
2. 對每個 t ，令 $B = \frac{T \times R}{\|T \times R\|}$ 。
3. 令 $N = B \times T$ 。

這個構造有三個可直接驗證的性質：

- $B \perp T$ 因為叉積垂直兩因子； $N \perp T$ 因為 $N = B \times T$ 垂直 T 。
- $N \perp B$ 因為 $B \times T$ 與 B 垂直。
- $\|N\| = \|B\| \|T\| = 1$ ，因此 (T, N, B) 是正交單位基底。

手性： $\det(T, N, B) = T \cdot (N \times B) = T \cdot (T) = +1$ ，故 (T, N, B) 為右手系。這正是慣例要求的 $(T, N, T \times N)$ 組合；若不甚用 $T \times R$ 的符號而改用 $R \times T$ ，即得左手系，所有面繞序會反向（見習題 11.3）。

$T \parallel R$ 時 $\|T \times R\| = 0$ ，需要 fallback：改用另一組不與 T 共線的軸。程式中以 $|T \cdot R| > 1 - \epsilon$ 為切換條件；此量隨 T 連續，除了 T 恰好等於 $\pm R$ 的孤立點外都是有限值。切換時機不對會造成 $\pm B$ 的突變；程序化網格接受這種突變的前提是骨架取樣足夠密、或在切換點以 SLERP 平滑框架。

包圍體

掃掠完成後，先求軸對齊包圍盒（AABB）：

$$p_{\min} = (\min_k x_k, \min_k y_k, \min_k z_k), \quad p_{\max} = (\max_k x_k, \max_k y_k, \max_k z_k).$$

AABB 對旋轉敏感，長條魚體在斜放場景中會有大片空白；改善方式是用有向包圍盒（OBB）：取骨架平均切向 $\bar{T}$ 為主軸，將所有頂點投影到 $\bar{T}$ ，在該軸上取 min/max，副軸用 $\bar{T} \times R$ 正交化。OBB 對魚體這種近似長條形的物件可減少 30% 以上的空體積（數值依姿態而異），但計算與測試都較複雜。本章只示範 AABB；OBB 留作習題。

LOD 的離散化參數

掃掠面的面數為 $2n_u(n_t - 1)$ 。三個層級可以這樣設定：

- LOD0： $n_t = 64, n_u = 24$ ，共 3024 面，近景。
- LOD1： $n_t = 32, n_u = 12$ ，共 744 面，中景。
- LOD2： $n_t = 16, n_u = 8$ ，共 240 面，遠景與包圍體測試。

更換 LOD 時必須共用同一骨架與尺度函數，否則輪廓會在不同層級間跳動。若截面點數改變，頂點間不需一對一對應，但環與環的對應必須保持；否則同一環上不同 LOD 會取到不同角度，重疊時可見皺褶。

自交與退化

三種幾何品質問題的判準：

- **退化面**：三角形任兩頂點重合，或三頂點共線。以兩邊外積長度 $\|e_1 \times e_2\|$ 為判準，低於 ϵ_g 視為退化。 ϵ_g 依模型尺度選：魚體以公尺為尺度時取 10^{-9} 至 10^{-6} 平方公尺都是合理範圍；同一個數值不能套用到公釐級模型上。程序化掃掠最常見來源是截面在端點收成一點時產生的扇形。
- **自交**：同一面上非相鄰三角形相交。對程序化魚身，最典型來自掃掠管內側自我折疊：局部骨架曲率半徑 $1/|\kappa(t)|$ 小於截面有效半徑 $s(t)$ ，遠處環已繞到中心另一側。簡單的必要條件為 $s(t)|\kappa(t)| < 1$ ；條件不成立時幾乎一定自交，但成立時仍可能因截面形狀、局部扭轉或離心效應而自交。
- **尺度失衡**：同一網格頂點的最長邊與最短邊比值過大（例如 $> 10^4$ ），會在浮點與法線計算中產生噪聲。程序化模型應固定座標單位，避免一部份以公釐、一部份以公尺混用。

逐步手算例題

例 11.1（局部框架與無扭轉截面）

骨架為 $c(t) = (t, 0, 0)$ ， $t \in [0, 1]$ ，截面為半徑 $r_0 = 0.1$ 的圓，取樣 $n_u = 4$ ，角度為 $0, \pi/2, \pi, 3\pi/2$ ，故截面點為 $(1, 0), (0, 1), (-1, 0), (0, -1)$ （此順序自 $+N$ 軸往 $+B$ 軸）。

取參考向量 $R = (0, 1, 0)$ 。對每個 t ， $T = (1, 0, 0)$ ， $T \times R = (0, 0, 1)$ ，所以 $B = (0, 0, 1)$ ， $N = B \times T = (0, 1, 0)$ 。

第 i 個截面點在 t 處的世界座標：

$$S(t, i) = c(t) + 0.1 u_i N + 0.1 v_i B.$$

以 $t = 0.5$ 為例：

- $(u, v) = (1, 0) : S = (0.5, 0.1, 0)$ 。
- $(u, v) = (0, 1) : S = (0.5, 0, 0.1)$ 。
- $(u, v) = (-1, 0) : S = (0.5, -0.1, 0)$ 。
- $(u, v) = (0, -1) : S = (0.5, 0, -0.1)$ 。

四點在 Y - Z 平面上以 $+Y$ 起逆時針排列。因為 T 常數，這是最簡單的情形；如果 c 彎曲， B 隨 t 旋轉， t 增加時 B 若繞 T 旋轉超過 $\pi/2$ ，四邊形索引順序會使得相鄰環看起來像反折。程序化模型應檢查相鄰環 $B_k \cdot B_{k+1}$ 是否為正。

例 11.2（自交判準）

截面半徑 $r_0 = 0.2$ 公尺，骨架為半徑 $R = 0.3$ 公尺的圓弧。曲率 $\kappa = 1/R \approx 3.33$ ， $1/\kappa = 0.3$ 公尺。因為 $r_0 = 0.2 < 0.3$ ，可推測掃掠管不會自交。

若把 r_0 提高到 0.4，則 $0.4 > 0.3$ ，內側截面圓與骨架中心距離 $R - r_0 = -0.1$ ，實際已穿過骨架軸，管的內側會自我折疊。判準 $s(t) |\kappa(t)| < 1$ 在每個 t 都要成立；等號為臨界，此時管的內側剛好收縮到一點，也是退化三角形開始出現的位置。

實作與程式

下面程式只用 Python 標準庫與 NumPy，定義 `frame_from_tangent` 生成局部框架、`sweep_circle` 生成圓截面掃掠面、`aabb` 求包圍盒、`write_obj` 輸出、`check_degenerate` 篩退化面。未執行結果；讀者可自行執行驗證。

```
import numpy as np

def frame_from_tangent(T, ref=np.array([0.0, 1.0, 0.0])):
    """
    回傳 (T, N, B)，右手正交且 B = T x N。
    T 需為非零向量。若 T 幾乎平行 ref，改用 +X 作參考。
    """
    T = T / np.linalg.norm(T)
    if abs(np.dot(T, ref)) > 1.0 - 1e-8:
        ref = np.array([1.0, 0.0, 0.0])
        if abs(np.dot(T, ref)) > 1.0 - 1e-8:
            ref = np.array([0.0, 0.0, 1.0])
    B = np.cross(T, ref)
    nb = np.linalg.norm(B)
    if nb < 1e-12:
```

```

        raise ValueError("frame_from_tangent: T 與備選參考皆近平行")
    B = B / nb
    N = np.cross(B, T)
    return T, N, B

def sweep_circle(skeleton, radius_fn, n_u=16):
    """
    skeleton: (n_t, 3) 骨架取樣點
    radius_fn: t -> 半徑 (公尺)
    n_u: 截面取樣數
    回傳 (verts, faces): 頂點與三角索引
    向外法線繞序: 每個 quad (a,b,c,d) 拆成 (a,b,d) 與 (a,d,c)
    """
    n_t = skeleton.shape[0]
    verts = []
    for k in range(n_t):
        t = k / (n_t - 1)
        if k == 0:
            T_raw = skeleton[1] - skeleton[0]
        elif k == n_t - 1:
            T_raw = skeleton[-1] - skeleton[-2]
        else:
            T_raw = skeleton[k + 1] - skeleton[k - 1]
        T, N, B = frame_from_tangent(T_raw)
        r = radius_fn(t)
        for i in range(n_u):
            ang = 2.0 * np.pi * i / n_u
            p = skeleton[k] + r * (np.cos(ang) * N + np.sin(ang) * B)
            verts.append(p)
    faces = []
    for k in range(n_t - 1):
        for i in range(n_u):
            i2 = (i + 1) % n_u
            a = k * n_u + i
            b = k * n_u + i2
            c = (k + 1) * n_u + i
            d = (k + 1) * n_u + i2
            faces.append((a, b, d))
            faces.append((a, d, c))
    return np.array(verts), np.array(faces)

def aabb(verts):
    return verts.min(axis=0), verts.max(axis=0)

def write_obj(path, verts, faces, name="sweep"):
    with open(path, "w", encoding="ascii") as f:
        f.write(f"o {name}\n")
        for v in verts:
            f.write(f"v {v[0]:.6f} {v[1]:.6f} {v[2]:.6f}\n")
        for tri in faces:
            f.write(f"f {tri[0]+1} {tri[1]+1} {tri[2]+1}\n")

def check_degenerate(verts, faces, eps=1e-9):
    bad = []
    for idx, (a, b, c) in enumerate(faces):
        e1 = verts[b] - verts[a]
        e2 = verts[c] - verts[a]
        if np.linalg.norm(np.cross(e1, e2)) < eps:
            bad.append(idx)

```

```

return bad

def check_curvature_radius(skeleton, radius_fn):
    """
    以三點估曲率，回傳每點  $s(t) \cdot \kappa(t)$ 。
    值  $\geq 1$  為自交必要條件已滿足；值  $< 1$  不代表一定不自交。
    """
    ratios = []
    for k in range(1, len(skeleton) - 1):
        a, b, c = skeleton[k-1], skeleton[k], skeleton[k+1]
        e1, e2 = b - a, c - b
        L1, L2 = np.linalg.norm(e1), np.linalg.norm(e2)
        if L1 < 1e-12 or L2 < 1e-12:
            ratios.append(0.0)
            continue
        dT = e2 / L2 - e1 / L1
        dL = 0.5 * (L1 + L2)
        kappa = np.linalg.norm(dT) / dL
        ratios.append(radius_fn(k / (len(skeleton) - 1)) * kappa)
    return ratios

```

幾點說明：

- `frame_from_tangent` 使用兩階 fallback：先試 $+Y$ ，若共線再試 $+X$ ，最後試 $+Z$ 。因為三個候選互相正交，不可能同時與 T 共線；因此若前兩個候選都被判為近平行，第三個一定是有限長度，`cross` 不會得到接近零的向量。
- `sweep_circle` 的索引組合 (a, b, d) 與 (a, d, c) 不是任意選的：依慣例「外向法線看為逆時針」，需要讓四邊形的前面法線指向管外。以例 11.1 為例， $n_u = 4$ 、 $k = 0$ 、 $i = 0$ ， $a = (0, r, 0)$ 、 $b = (0, 0, r)$ 、 $c = (1, r, 0)$ 、 $d = (1, 0, r)$ 。 $e_1 = b - a = (0, -r, r)$ ， $e_2 = d - a = (1, -r, r)$ ， $e_1 \times e_2 = (0, r, r)$ ，其 $+Y$ 、 $+Z$ 分量均為正，與 a 點徑向 $+Y$ 、鄰近 b 點徑向 $+Z$ 的出向一致。若改寫成 (a, c, b) ， $e_1 = (1, 0, 0)$ 、 $e_2 = (0, -r, r)$ ，叉積 $= (0, -r, -r)$ ，向內，全部面法線反向。
- `check_degenerate` 的 ϵ_g 預設 10^{-9} ，單位為平方公尺。以公尺級魚體為例，邊長 10^{-4} 公尺的三角形面積量級 10^{-8} ，不會誤報；若模型改以公釐為單位， ϵ_g 也應同步調整。
- `check_curvature_radius` 回傳的是必要條件的比值；必須搭配實際數值檢測才能斷定自交。

測試與預期結果

以下為「預期」性質，讀者應自行執行；章節不宣稱已執行過。

1. **AABB 正確性**：骨架 $c(t) = (t, 0, 0)$ 、 $t \in [0, 1]$ 、 $r = 0.1$ 、 $n_u = 16$ ，`aabb` 預期回傳 $([0, -0.1, -0.1], [1, 0.1, 0.1])$ ，容差 10^{-6} 。
2. **向外面法線**：對上述參數，取 `faces[0]=(a,b,d)=(0,1,17)`，用手算交叉乘積驗證法線 $+Y$ 、 $+Z$ 分量皆為正。程式上可寫成：

```

verts, faces = sweep_circle(skeleton, lambda t: 0.1, n_u=16)
a, b, d = faces[0]

```

```
n = np.cross(verts[b] - verts[a], verts[d] - verts[a])
assert n[1] > 0 and n[2] > 0, "法線向內，繞序錯誤"
```

3. **退化面**：對骨架 $c(t) = (t, 0, 0)$ 、 $r = 0.1$ ，圓截面永不自交，`check_degenerate` 預期回傳空清單。
4. **自交必要條件**：對半徑 $R = 0.5$ 公尺的圓弧骨架（曲率半徑 0.5）配 $r = 0.8$ ，`check_curvature_radius` 預期在多數樣本輸出 ≥ 1 ；若把 r 調回 0.2，比值預期全為 < 1 ，此時 `check_degenerate` 仍預期回傳空清單。
5. **OBJ 往返**：以 `write_obj` 產生檔案後，讀回驗證每個索引落在 $[1, n_t n_u]$ 。索引從 1 起是 OBJ 慣例；寫出時 `tri[i] + 1`、讀回時 `i - 1`，兩處方向若顛倒，會出現索引偏移 1 的系統性錯誤。

除錯與常見陷阱

- **參考向量共線**：骨架切向幾乎與參考向量同向時，`frame_from_tangent` 回傳接近零的 B ，單位化後數值不穩。務必兩階 fallback，且第三候選必須與前兩個候選正交；若只用單一候選， $|T \cdot R|$ 接近 1 的機率不能忽略。
- **繞序翻轉**：`sweep_circle` 若把 (a, b, d) 與 (a, d, c) 寫成 (a, c, b) 與 (b, c, d) ，所有面法線會指向內側，渲染時看到「消失的背面」或明暗全黑。程序化管狀網格不會自動偵測這種錯；建議在 `write_obj` 之前執行一次法線檢查（見測試 2）。
- **尺度混用**：OBJ 沒有單位欄位。若骨架以公尺、半徑以公分輸入，模型會相差 100 倍。所有幾何量以公尺，並在檔名或 metadata 標明。
- **退化三角形群**：在骨架端點封口時，若截面收縮為單點，所有連到該點的三角形都會退化。應改用「端點扇」把該側截面頂點連到一個額外中心點，或保留開放邊界。
- **AABB 誤當碰撞體**：AABB 只在軸對齊時緊密；對斜放魚體會高估體積。欲做準確碰撞或自交篩選，需要 OBB 或 BVH（見後續章節）。
- **LOD 不一致**：不同的 n_t, n_u 會生成不同輪廓；若切換時只改變面數而未統一骨架與尺度函數，會看到輪廓抖動。
- **退化判準的單位**： ϵ_g 是平方公尺（外積的長度等於兩邊夾出的平行四邊形面積量級）。換單位時要同步調整，否則會把真實三角形誤判為退化，或讓退化面溜過檢查。

養殖數位分身案例

把魚身看成掃掠管：骨架曲線代表體軸，截面半徑函數 $r(t)$ 代表魚身厚度。典型合成魚體以 $c(t) = (t, 0.05 \sin(2\pi t), 0)$ 、 $r(t) = 0.08(0.6 + 0.4 \sin \pi t)$ 產生。魚鰭用同樣的掃掠方法，只是截面改為薄扁構形，骨架改為由體側延伸的短曲線。

池體以參數化方式產生：

- 池面矩形以 4 個角點定義，往上以直壁延伸至水深 $H = 1.5$ 公尺。
- 池底加一層薄層，避免與側壁共面。
- 池體六個面以獨立頂點，不與水面、魚體共用，以避免拓撲耦合。

整個場景匯成一個 OBJ 與一個 JSON 描述檔。JSON 記錄物件名稱、檔名、單位、LOD 層級與包圍盒，供下游渲染器或標註程式載入。此模型僅為合成場景，不表示真實魚體尺寸或生態行為；後續章節的 agent 只讀取並整理這些檔案，不控制任何設備或餵食器械。

習題

習題 11.1 (手算) 骨架 $c(t) = (t, 0, 0)$ 、 $t \in [0, 1]$ ，截面為單位正方形頂點 $(\pm 1, \pm 1)$ ，順序自訂並明寫。給定 $r(t) = 0.1 + 0.05t$ ，參考向量 $R = (0, 0, 1)$ 。寫出 $S(0, 0)$ 與 $S(1, 3)$ (索引以你的順序為準)。

習題 11.2 (程式測試) 撰寫 `check_indices(n_vertices, faces)`，驗證所有面索引介於 $[0, n_{\text{vertices}})$ ，回傳非法面的清單。用 `faces=[(0,5,2),(3,1,0)]`、`n_vertices = 4` 測試並說明輸出。

習題 11.3 (反例／除錯) 在 `frame_from_tangent` 中，若把 `B = np.cross(T, ref)` 誤寫成 `B = np.cross(ref, T)`，請指出至少一項法線方向或面繞序上的可觀察差異。

習題 11.4 (整合應用) 對半徑 $R = 0.4$ 公尺的圓弧骨架、截面半徑函數 $r(t) = 0.15 + 0.2t$ ，判定 $t \in [0, 1]$ 是否有自交。說明判準與臨界值。

習題解答

11.1 對 $t \in [0, 1]$ ， $T = (1, 0, 0)$ 。 $R = (0, 0, 1)$ ， $T \times R = (0, -1, 0)$ ，單位化後 $B = (0, -1, 0)$ 。 $N = B \times T = (0, 0, -1)$ 。截面點 $(u, v) = (1, 1) : S(t) = c(t) + r(t)(1 \cdot N + 1 \cdot B) = (t, -r(t), -r(t))$ 。在 $t = 0 : r(0) = 0.1$ ， $S = (0, -0.1, -0.1)$ 。在 $t = 1 : r(1) = 0.15$ ， $S = (1, -0.15, -0.15)$ 。順序須自訂並明寫，因為它決定後續三角形索引。

11.2 對每個面三元組檢查 `0 <= i < n_vertices`。輸入第 0 面含 5 超界，第 1 面合法，輸出 `[0]`。這函式測試的是資料結構完整性，不涉及幾何；放在 OBJ 讀寫流程之前可提早攔截錯誤。

11.3 交換後 $B = R \times T = -(T \times R)$ ，因此 $B = (0, 1, 0)$ ， $N = B \times T = (0, 0, 1)$ ；切向 T 不變。手性反轉，等同把截面繞 T 旋轉 180° 並鏡射一次。可觀察差異包括：(一) 四邊形索引 (a, b, d) 與 (a, d, c) 生成的三角形，其面法線全部反向，渲染時背面剔除下整個管會消失；(二) 在曲面彎折處，扭轉方向相反，皺褶從一側換到另一側。修正方式是還原叉積順序，或對 `faces` 整體翻轉索引 (例如把 (i, j, k) 全改為 (i, k, j))。

11.4 圓弧曲率 $\kappa = 1/R = 2.5$ 。臨界條件 $r(t) \cdot \kappa = 1$ 即 $r(t) = 0.4$ 。解 $0.15 + 0.2t = 0.4$ 得 $t = 1.25 > 1$ ，因此整個 $t \in [0, 1]$ 都不自交。若把 R 改為 0.2 ($\kappa = 5$)，臨界 $r = 0.2$ ， $0.15 + 0.2t = 0.2$ 得 $t = 0.25$ ，因此 $t > 0.25$ 起比值 ≥ 1 ，需要以數值檢測進一步確認是否自交。注意此判準為必要條件；實際自交仍應以數值檢測 (例如沿骨架比對相鄰環的頂點距離) 驗證。

本章小結

掃描把「輪廓＋骨架」壓縮為少數參數，是程序化建模的基本構件；局部框架決定如何把截面定向，參考向量法比 Frenet frame 穩定。幾何品質檢查則要落在資料層面：索引範圍、退化面、自交判準、包圍盒、尺度一致性，這些都能用獨立小程序驗證，而不必倚賴渲染結果的視覺判斷。三角形索引組合、 ϵ_g 尺度、fallback 順序是本節三處容易寫錯、且錯誤會直接反映在渲染結果與下游流程的地方。魚身／魚鰭／池體的例子示範了如何把這些規則併入資產交換流程。後續章節會把這些網格投入著色、追蹤與動畫，屆時包圍盒、LOD、繞序與單位都會直接影響結果。

參考來源

- G1 PBRT 4：Transformations（座標變換與局部框架的通用形式）
- G2 PBRT 4：Reflection Models（後續著色的法線與面向）
- G3 PBRT 4：The Light Transport Equation（後續章節使用）
- G4 Ray Tracing in One Weekend（包圍盒與求交的實作取向）
- G5 LearnOpenGL：Transformations（矩陣組合與繞序的入門說明）
- G6 Blender Manual：Skinning Introduction（骨架與網格關聯）
- G7 NumPy 線性代數參考（本章 `np.cross`、`np.linalg.norm` 之文件）
- G8 Khronos glTF 2.0 規格（後續資產交換章節使用）

上述連結僅作回查之入口，本章論述與推導為本卷自撰，未逐條複製來源文字，也未宣稱已執行任何官方範例。

第 12 章 資產交換與可重現場景

學習目標與先備知識

完成本章後，讀者應能：

1. 區分幾何資產、材質、節點與場景清單。
2. 以公尺、右手座標系與明確矩陣慣例交換資產。
3. 推導場景圖中的局部與世界變換。
4. 理解 OBJ 的頂點索引、面繞序、材質指派與限制。
5. 建立可由程式重建的 JSON 場景清單。
6. 讀寫簡化 OBJ，檢查非法索引、退化三角形與材質參照。
7. 說明 OBJ 與 glTF 在場景階層、材質及資料組織上的差異。

先備知識包括向量、矩陣乘法、三角網格、齊次座標，以及 Python 3.10+ 與 NumPy。本章內部長度單位為公尺，角度輸入可用度表示，但進入三角函數前必須轉為弧度。

問題與直覺

一個 `fish.obj` 不等於完整場景。它可能只有三角形，卻沒有回答：

- 一個座標單位是公尺、厘米，還是任意尺度？
- 魚頭與上方各朝哪一軸？
- 模型原點位於魚體中心、鼻尖或其他位置？
- 哪些面使用魚皮或魚鰭材質？
- 魚相對於池體和相機位於何處？
- 多條魚是否共用同一份幾何？
- 換一台電腦後，如何重建相同輸入？

資產交換可分為三層：

1. **幾何資產**：頂點、三角形、法線、UV。
2. **外觀資產**：材質參數與貼圖參照。
3. **場景組裝**：節點階層、變換、相機與資產指派。

OBJ 適合交換靜態網格，但不原生表達完整場景階層、現代 PBR 材質或動畫。本章以 JSON 作為教學用場景清單；glTF 則是較完整的交換格式概念橋梁。Blender 可用於選擇性的目視檢查，但不是核心實驗的必要條件。

「可重現」不代表不同渲染器一定產生逐像素相同的結果，而是輸入檔案、單位、座標約定、資產參照及變換次序均有紀錄，缺漏時也會明確失敗。

數學與幾何推導

12.1 場景圖與節點變換

場景圖可視為有根樹。節點 i 的局部變換為 M_i ，世界變換為

$$M_i^{world} = M_{parent(i)}^{world} M_i.$$

根節點則取

$$M_{root}^{world} = M_{root}.$$

本書使用 column vector，因此局部點到世界座標的轉換為

$$\mathbf{p}_{world} = M_i^{world} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}.$$

局部矩陣若由平移、旋轉與縮放組成，採

$$M_i = T_i R_i S_i.$$

矩陣由右向左作用，即先縮放、再旋轉、最後平移。JSON 欄位的文字排列不應被拿來猜測矩陣次序；格式契約必須明定 TRS 的意義。

節點與網格資產應分開。十條外形相同的魚可建立十個節點，共同參照一份 `fish.obj`。這稱為**實例化**：資產保存局部幾何，節點保存各自的位置與朝向。

12.2 單位換算

設來源資產的一個座標單位等於 s 公尺。來源頂點 $\mathbf{p}_s$ 轉成公尺為

$$\mathbf{p}_m = s\mathbf{p}_s.$$

厘米的 $s = 0.01$ ，毫米的 $s = 0.001$ 。若另有軸向轉換 C_3 ，則

$$\mathbf{p}_{target} = C_3(s\mathbf{p}_s).$$

單位換算只能有一個明確的責任位置：

- 匯入時永久縮放頂點，之後節點一律使用公尺；或
- 保留來源頂點，把單位縮放納入節點矩陣。

本章採第一種策略。若頂點與節點各縮放一次，會重複換算。

12.3 座標軸、手性與繞序

本書世界系為右手系： $+X$ 向右、 $+Y$ 向上、 $+Z$ 朝向觀者。假設來源以 $+Z$ 向上、 $+Y$ 向前，且轉換定義為

$$x_t = x_s, \quad y_t = z_s, \quad z_t = -y_s,$$

則

$$C_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix}.$$

因為

$$\det(C_3) = 1,$$

此轉換保留右手性及三角形繞序。

若線性轉換 A 滿足 $\det(A) < 0$ ，它包含鏡射，會翻轉手性。原本從外側觀看為逆時針的三角形將變為順時針。可交換索引：

$$(i_0, i_1, i_2) \rightarrow (i_0, i_2, i_1),$$

或明確改變渲染器的正面判定。法線需使用

$$\mathbf{n}' = \frac{A^{-T} \mathbf{n}}{\|A^{-T} \mathbf{n}\|}$$

變換，且要求 A 可逆。若另有切線空間資料，鏡射也會影響其手性。

12.4 包圍盒與尺度檢查

對頂點集合 $\{\mathbf{p}_k\}$ ，軸對齊包圍盒為

$$\mathbf{b}_{min} = \begin{bmatrix} \min_k x_k \\ \min_k y_k \\ \min_k z_k \end{bmatrix}, \quad \mathbf{b}_{max} = \begin{bmatrix} \max_k x_k \\ \max_k y_k \\ \max_k z_k \end{bmatrix}.$$

其尺寸與對角線長為

$$\mathbf{d} = \mathbf{b}_{max} - \mathbf{b}_{min}, \quad L = \|\mathbf{d}\|.$$

若一條預期長約 0.3 m 的魚匯入後長達 30 m，通常是厘米被誤當成公尺。這只是資料品質檢查，不能當成生物學驗證。

退化三角形可由兩倍面積判定：

$$a_2 = \|(\mathbf{p}_b - \mathbf{p}_a) \times (\mathbf{p}_c - \mathbf{p}_a)\|.$$

因為 a_2 的單位是 m^2 ，尺度相對容差可設為

$$\varepsilon_A = \varepsilon_r L^2.$$

當整個包圍盒對角線 $L = 0$ 時，所有頂點重合，網格本身已是零尺度，應直接將所有面視為退化，而不是任意以 1 m 代替其尺度。

12.5 OBJ 索引與材質指派

OBJ 常見記錄包括：

```
v x y z
vt u v
vn nx ny nz
usemtl material_name
f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3
```

OBJ 正索引從 1 開始，Python 索引從 0 開始，所以 OBJ 索引 j 要轉成 $j - 1$ 。OBJ 也允許負索引，`-1` 表示面出現位置之前最近定義的項目。

本章的最小讀取器只接受：

- 三維位置 `v`；
- 材質名稱 `usemtl`；
- 只含正頂點索引的三角形 `f`。

它不接受負索引、四邊形及 `v/vt/vn` 複合索引。正索引還必須指向該面出現前已定義的頂點；不能先寫面、後補頂點。

材質指派可視為面到材質集合的函數：

$$m : F \rightarrow \mathcal{M}.$$

`usemtl` 只提供名稱，不等於已讀入材質定義。即使 OBJ 含有 `mtllib`，本章讀取器也不解析 MTL；材質參數由 JSON 清單提供。

12.6 JSON 清單與 glTF 概念

本章 JSON 是最小教學契約，不是通用標準。它記錄：

- 格式版本與公尺單位；
- 世界座標系與正面繞序；
- 網格資產與來源單位；
- 材質參數；
- 節點父子關係與局部 TRS；
- 一個相機定義。

本程式產生的是**最小場景清單範例**：包含一條魚、作為座標父節點的 `pond`，以及相機；它沒有池壁或水面的幾何資產，也沒有隨機生成步驟，因此不記錄未使用的隨機種子。

glTF 2.0 則可描述場景、節點、網格、材質、動畫、蒙皮、緩衝區與影像。其資料可由 JSON 與外部二進位檔組成，也可封裝成 GLB。glTF 常用金屬度—粗糙度 PBR 工作流程；OBJ/MTL 材質通常不能無損轉換成相同外觀。

無論格式多完整，仍應檢查軸向、單位、相對 URI、色彩空間、矩陣慣例及鏡射。格式名稱本身不能保證場景語意正確。

逐步手算例題

例題一：單位、軸向與平移

來源魚鼻頂點以厘米記錄為

$$\mathbf{p}_s = (20, 5, 10)^T.$$

來源採 $+Z$ 向上、 $+Y$ 向前，使用前述 C_3 ；魚節點平移為

$$\mathbf{t} = (2, 0.5, -3)^T \text{ m.}$$

先換成公尺：

$$\mathbf{p}_m = 0.01\mathbf{p}_s = (0.20, 0.05, 0.10)^T.$$

再轉換軸向：

$$C_3\mathbf{p}_m = (0.20, 0.10, -0.05)^T.$$

最後加上節點平移：

$$\mathbf{p}_{world} = (0.20, 0.10, -0.05)^T + (2, 0.5, -3)^T = (2.20, 0.60, -3.05)^T \text{ m.}$$

若忘記乘 0.01，尺度會大一百倍；若頂點和節點各乘一次，尺度會縮成應有值的一百分之一。

例題二：父子節點的非交換性

父節點繞 $+Y$ 旋轉 90° ，魚節點在父座標中平移 $(1, 0, 0)^T$ 。右手正旋轉矩陣為

$$R_y(90^\circ) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

魚的局部原點為 $\mathbf{o} = (0, 0, 0, 1)^T$ ，故

$$R_y(90^\circ)T_x(1)\mathbf{o} = R_y(90^\circ) \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1 \\ 1 \end{bmatrix}.$$

若錯寫為反序：

$$T_x(1)R_y(90^\circ)\mathbf{o} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

前者表示局部位移會隨父節點旋轉；後者是在世界 $+X$ 方向平移，兩者並不等價。

實作與程式

以下完整程式會寫出低面數魚形 OBJ、讀回簡化 OBJ、驗證網格、計算世界矩陣，並建立最小 JSON 場景清單。程式提供給讀者自行執行；此處未執行。

```
from __future__ import annotations

import json
import math
from pathlib import Path
import numpy as np

def checked_faces(faces):
    """確認索引為有限整數，再轉為 NumPy 整數；不可靜默截斷 1.9。"""
    raw = np.asarray(faces)
    if raw.ndim != 2 or raw.shape[1] != 3:
        raise ValueError("faces 必須是 Mx3 三角形")
    if not np.issubdtype(raw.dtype, np.number):
        raise ValueError("面索引必須是數字")
    values = np.asarray(raw, dtype=float)
    if not np.isfinite(values).all():
        raise ValueError("面索引含 NaN 或無限值")
    if not np.equal(values, np.floor(values)).all():
```

```

        raise ValueError("面索引必須是整數值")
    return values.astype(np.int64)

def trs_matrix(translation, rotation_y_rad, scale):
    tx, ty, tz = map(float, translation)
    sx, sy, sz = map(float, scale)
    values = np.array([tx, ty, tz, sx, sy, sz, rotation_y_rad])
    if not np.isfinite(values).all():
        raise ValueError("TRS 含非有限值")
    if sx == 0.0 or sy == 0.0 or sz == 0.0:
        raise ValueError("本章拒絕奇異的零尺度")

    c, s = math.cos(rotation_y_rad), math.sin(rotation_y_rad)
    T = np.array([
        [1, 0, 0, tx],
        [0, 1, 0, ty],
        [0, 0, 1, tz],
        [0, 0, 0, 1],
    ], dtype=float)
    R = np.array([
        [c, 0, s, 0],
        [0, 1, 0, 0],
        [-s, 0, c, 0],
        [0, 0, 0, 1],
    ], dtype=float)
    S = np.diag([sx, sy, sz, 1.0])
    return T @ R @ S

def write_obj(path, vertices, faces, material="fish_skin"):
    vertices = np.asarray(vertices, dtype=float)
    faces = checked_faces(faces)

    if vertices.ndim != 2 or vertices.shape[1] != 3:
        raise ValueError("vertices 必須是 N×3")
    if not np.isfinite(vertices).all():
        raise ValueError("頂點含 NaN 或無限值")
    if len(faces) and (faces.min() < 0 or faces.max() >= len(vertices)):
        raise ValueError("面含非法頂點索引")
    if not material or any(ch.isspace() for ch in material):
        raise ValueError("此範例的材質名稱不得空白或含空格")

    with open(path, "w", encoding="utf-8", newline="\n") as f:
        f.write("# unit: meter\n")
        f.write("# axes: right-handed, +Y up\n")
        f.write(f"usemtl {material}\n")
        for x, y, z in vertices:
            f.write(f"v {x:.9g} {y:.9g} {z:.9g}\n")
        for a, b, c in faces:
            f.write(f"f {a + 1} {b + 1} {c + 1}\n")

def read_simple_obj(path):
    """只讀 v、usemtl、正索引三角形 f；mtllib 僅忽略，不載入。"""
    vertices, faces, face_materials = [], [], []
    current_material = None

    with open(path, "r", encoding="utf-8") as f:

```

```

for line_number, raw in enumerate(f, 1):
    line = raw.partition("#")[0].strip()
    if not line:
        continue
    fields = line.split()
    tag = fields[0]

    if tag == "v":
        if len(fields) != 4:
            raise ValueError(f"第 {line_number} 行: v 需三個座標")
        point = [float(x) for x in fields[1:]]
        if not np.isfinite(point).all():
            raise ValueError(f"第 {line_number} 行: 非有限頂點")
        vertices.append(point)

    elif tag == "usemtl":
        if len(fields) != 2:
            raise ValueError(f"第 {line_number} 行: 無效 usemtl")
        current_material = fields[1]

    elif tag == "f":
        if len(fields) != 4:
            raise ValueError(f"第 {line_number} 行: 只接受三角形")
        indices = []
        for token in fields[1:]:
            if "/" in token:
                raise ValueError(
                    f"第 {line_number} 行: 不接受 v/vt/vn"
                )
            obj_index = int(token)
            # 正索引必須指向此面之前已定義的頂點。
            if obj_index <= 0 or obj_index > len(vertices):
                raise ValueError(
                    f"第 {line_number} 行: 索引 {obj_index} 無效"
                )
            indices.append(obj_index - 1)
        if current_material is None:
            raise ValueError(
                f"第 {line_number} 行: 面尚未指定 usemtl"
            )
        faces.append(indices)
        face_materials.append(current_material)

    elif tag in {"o", "g", "s", "mtllib"}:
        continue
    else:
        raise ValueError(
            f"第 {line_number} 行: 不支援標記 {tag!r}"
        )

vertices = np.asarray(vertices, dtype=float).reshape((-1, 3))
faces = np.asarray(faces, dtype=np.int64).reshape((-1, 3))
return vertices, faces, face_materials

def validate_mesh(vertices, faces, relative_epsilon=1e-12):
    vertices = np.asarray(vertices, dtype=float)
    faces = checked_faces(faces)

```

```

if vertices.ndim != 2 or vertices.shape[1] != 3:
    raise ValueError("vertices 必須是 N×3")
if len(vertices) == 0:
    raise ValueError("網格沒有頂點")
if not np.isfinite(vertices).all():
    raise ValueError("頂點含 NaN 或無限值")
if relative_epsilon < 0 or not math.isfinite(relative_epsilon):
    raise ValueError("relative_epsilon 必須是有限非負數")
if len(faces) and (faces.min() < 0 or faces.max() >= len(vertices)):
    raise ValueError("非法頂點索引")

bounds_min = vertices.min(axis=0)
bounds_max = vertices.max(axis=0)
diagonal = float(np.linalg.norm(bounds_max - bounds_min))
area2_epsilon = relative_epsilon * diagonal * diagonal

degenerate = []
for i, (a, b, c) in enumerate(faces):
    area2 = float(np.linalg.norm(
        np.cross(vertices[b] - vertices[a],
                  vertices[c] - vertices[a])
    ))
    # diagonal == 0 時，所有面自然滿足 area2 <= 0。
    if area2 <= area2_epsilon:
        degenerate.append(i)

return {
    "vertex_count": len(vertices),
    "triangle_count": len(faces),
    "bounds_min": bounds_min.tolist(),
    "bounds_max": bounds_max.tolist(),
    "bounds_diagonal_m": diagonal,
    "degenerate_faces": degenerate,
}

def transform_points(matrix, points):
    points = np.asarray(points, dtype=float)
    homogeneous = np.column_stack([points, np.ones(len(points))])
    transformed = (matrix @ homogeneous.T).T
    if np.any(np.abs(transformed[:, 3]) < 1e-15):
        raise ValueError("齊次座標 w 太接近零")
    return transformed[:, :3] / transformed[:, 3:4]

def world_matrices(nodes):
    result, visiting = {}, set()

    def visit(name):
        if name in result:
            return result[name]
        if name in visiting:
            raise ValueError(f"場景圖出現循環：{name}")
        if name not in nodes:
            raise ValueError(f"不存在的節點：{name}")

        visiting.add(name)
        node = nodes[name]
        local = trs_matrix(

```



```

        node["translation"],
        math.radians(node["rotation_y_degrees"]),
        node["scale"],
    )
    parent = node.get("parent")
    world = local if parent is None else visit(parent) @ local
    visiting.remove(name)
    result[name] = world
    return world

for name in nodes:
    visit(name)
return result

def validate_scene_references(manifest):
    assets = manifest["assets"]
    materials = manifest["materials"]
    for name, node in manifest["nodes"].items():
        if "asset" in node and node["asset"] not in assets:
            raise ValueError(f"{name} 參照不存在的資產")
        if "material" in node and node["material"] not in materials:
            raise ValueError(f"{name} 參照不存在的材質")

def main():
    out_dir = Path("repro_scene")
    out_dir.mkdir(exist_ok=True)

    vertices = np.array([
        [-0.20, 0.00, 0.00],
        [0.20, 0.00, 0.00],
        [0.00, 0.08, 0.00],
        [0.00, -0.08, 0.00],
        [0.00, 0.00, 0.05],
        [0.00, 0.00, -0.05],
    ], dtype=float)

    faces = np.array([
        [0, 4, 2], [2, 4, 1],
        [0, 3, 4], [3, 1, 4],
        [0, 2, 5], [2, 1, 5],
        [0, 5, 3], [3, 5, 1],
    ], dtype=np.int64)

    obj_path = out_dir / "fish.obj"
    write_obj(obj_path, vertices, faces)
    loaded_v, loaded_f, face_materials = read_simple_obj(obj_path)
    report = validate_mesh(loaded_v, loaded_f)

    manifest = {
        "schema": "aquaculture-scene-1.0",
        "units": {"length": "meter", "time": "second"},
        "coordinates": {
            "handedness": "right",
            "up": "+Y",
            "camera_forward": "-Z",
            "front_face": "counter_clockwise"
        },
    },

```

```

"assets": {
    "fish_mesh": {
        "uri": "fish.obj",
        "format": "obj",
        "source_unit_in_meters": 1.0,
        "validation": report
    }
},
"materials": {
    "fish_skin": {
        "base_color_linear_rgb": [0.15, 0.45, 0.70],
        "roughness": 0.55,
        "metallic": 0.0
    }
},
"nodes": {
    "pond": {
        "parent": None,
        "translation": [0.0, 0.0, 0.0],
        "rotation_y_degrees": 0.0,
        "scale": [1.0, 1.0, 1.0]
    },
    "fish_01": {
        "parent": "pond",
        "asset": "fish_mesh",
        "material": "fish_skin",
        "translation": [1.2, -0.4, -2.0],
        "rotation_y_degrees": 30.0,
        "scale": [1.0, 1.0, 1.0]
    },
    "camera_main": {
        "parent": None,
        "translation": [0.0, 1.5, 4.0],
        "rotation_y_degrees": 0.0,
        "scale": [1.0, 1.0, 1.0]
    }
},
"camera": {
    "node": "camera_main",
    "projection": "perspective",
    "vertical_fov_degrees": 50.0,
    "near_m": 0.1,
    "far_m": 30.0
},
"provenance": {
    "scene_kind": "synthetic",
    "note": "最小範例；pond 只有座標節點，未含池壁與水面幾何"
}
}

validate_scene_references(manifest)
matrices = world_matrices(manifest["nodes"])
fish_world = transform_points(matrices["fish_01"], loaded_v)
manifest["nodes"]["fish_01"]["world_bounds"] = {
    "min": fish_world.min(axis=0).tolist(),
    "max": fish_world.max(axis=0).tolist()
}

if set(face_materials) - set(manifest["materials"]):

```

```

        raise ValueError("OBJ 使用了 JSON 未定義的材質")

    with open(out_dir / "scene.json", "w",
              encoding="utf-8", newline="\n") as f:
        json.dump(manifest, f, ensure_ascii=False, indent=2)
        f.write("\n")

    print(json.dumps(report, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

若未來加入程序式魚群、隨機相機或隨機材質，才應在清單記錄實際使用的亂數演算法與種子。只寫一個未使用的種子不會提高可重現性。

測試與預期結果

以下均為依程式靜態推導的**預期結果**，不是已執行結果。

1. OBJ 往返

```

assert np.allclose(loader_v, vertices)
assert np.array_equal(loader_f, faces)
assert face_materials == ["fish_skin"] * 8

```

預期成立。浮點文字往返應使用適當容差，不宜依賴字串逐位相同。

2. 網格品質

```

assert report["vertex_count"] == 6
assert report["triangle_count"] == 8
assert report["degenerate_faces"] == []
assert np.allclose(report["bounds_min"], [-0.2, -0.08, -0.05])
assert np.allclose(report["bounds_max"], [0.2, 0.08, 0.05])

```

包圍盒尺寸預期為 (0.4, 0.16, 0.1) m。

3. 浮點索引拒絕

```

bad_faces = np.array([[0.0, 1.9, 2.0]])
checked_faces(bad_faces)

```

預期拋出 `ValueError`，而不是把 1.9 靜默截斷成 1。

4. 前向引用拒絕

若 OBJ 的第一個有效記錄是 `f 1 2 3`，而頂點稍後才出現，讀取器預期立即拒絕，因為該面出現時尚無已定義頂點。

5. 退化面與尺度

加入 `[0,0,1]` 時，

$$\|(\mathbf{p}_0 - \mathbf{p}_0) \times (\mathbf{p}_1 - \mathbf{p}_0)\| = 0,$$

因此預期該面編號出現在 `degenerate_faces`。若所有頂點完全重合，包圍盒對角線為零，所有面也都應被判為退化。

6. 場景圖循環

若令 `pond.parent` 為 `"fish_01"`，而 `fish_01.parent` 仍為 `"pond"`，預期 `world_matrices` 報告循環。

7. 相對路徑

資產 URI 應相對於清單位置解析：

```
scene_path = Path("repro_scene/scene.json").resolve()
asset_path = scene_path.parent / "fish.obj"
assert asset_path.is_file()
```

不能假定程式目前工作目錄就是場景目錄。

除錯與常見陷阱

單位差百倍

OBJ 沒有可靠的內建公尺宣告。`# unit: meter` 只是註解，其他軟體未必理會。應由場景契約決定換算，再用包圍盒檢查尺度。

父子矩陣乘反

本書正確寫法為：

```
world = parent_world @ local
```

寫成 `local @ parent_world` 會改變變換語意。

NumPy 儲存方式與數學慣例混淆

`M @ p` 表示本書的 column-vector 乘法。C-order 或 Fortran-order 是記憶體排列，不會改變矩陣公式。

鏡射後模型消失

若 $\det(A) < 0$ ，三角形繞序會翻轉。需要一致處理正面判定、索引、法線及切線手性，不能只改一個座標分量。

材質名稱存在，但材質未載入

`usemtl fish_skin` 只是一個名稱。`mtllib` 也只是外部 MTL 的參照；本章讀取器允許該記錄出現，但不載入 MTL。JSON 中必須另有同名材質，否則場景參照不完整。

容差失去尺度意義

若自稱使用相對容差，就不應把所有小於 1 m 的模型強制當作尺度 1 m。面積門檻應由實際包圍盒對角線平方導出，並特別處理零尺度網格。

面法線方向不一致

三角形 (a, b, c) 的幾何法線方向為

$$\mathbf{n} \propto (\mathbf{p}_b - \mathbf{p}_a) \times (\mathbf{p}_c - \mathbf{p}_a).$$

本章驗證器只檢查索引和退化面，沒有足夠語意自動判斷哪一側必須朝外。對封閉池體可配合已知內部點檢查；對開放曲面則需要額外的方向契約。

Blender 中朝向不同

Blender 可選用來檢視 OBJ，但匯入選項可能改變軸向與尺度。應記錄設定，並以已知測試點或軸標記核對，不能只以「看起來正確」判斷。

養殖數位分身案例

考慮一座合成圓形池：中心在世界原點，水面高度 $y = 0$ ，半徑 5 m、深度 1.5 m。兩條魚可共用同一網格：

節點	世界位置 (m)	繞 +Y 旋轉	資產
fish_01	(1.2, -0.4, -2.0)	30°	fish_mesh
fish_02	(-0.8, -0.7, 1.5)	-110°	fish_mesh

完整場景應另加入池壁與水面資產；前述程式中的 `pond` 只是父座標節點，不可誤稱為已建好的池體幾何。

驗收時至少檢查：

1. 所有 URI 都能相對清單位置解析。
2. 節點資產與材質名稱都有定義。
3. 場景圖沒有循環或缺失父節點。
4. 網格索引合法，退化面已列出或拒絕。
5. 魚的世界包圍盒位於合成池尺度內。
6. 相機近平面滿足 `near_m > 0`，且 `far_m > near_m`。
7. 清單明確標示資料為合成場景。

魚模型位於池內，只能證明幾何關係符合設定；不能證明魚的姿態、群聚行為或水動力學真實。

習題

1. 手算題：單位與階層

某魚資產以毫米建模，局部點為 (300, 0, 50) mm。匯入後先換成公尺，再由魚節點平移 (1, 0, -2) m；父節點繞 +Y 旋轉 90°。求世界座標。

2. 程式測試題：OBJ 往返

使用標準庫暫存目錄，測試：

- 頂點與面往返一致；
- 每面材質皆為 `fish_skin`；
- 包圍盒尺寸為 (0.4, 0.16, 0.1) m；
- 沒有退化面。

3. 反例／除錯題：鏡射後消失

匯入器以

$$A = \text{diag}(1, 1, -1)$$

轉換頂點。模型開啟背面剔除後消失。說明原因與修正方法。

4. 整合應用題：多魚場景

建立三個共用 `fish.obj` 的魚節點，位置為

$$(0, -0.5, -1), \quad (1, -0.6, -2), \quad (-1, -0.4, -1.5)$$

公尺，繞 $+Y$ 的角度分別為 $0^\circ, 45^\circ, -30^\circ$ 。列出至少四項載入驗證，並說明為何不應複製三份 OBJ。

習題解答

1. 手算題解答

毫米轉公尺後：

$$\mathbf{p} = (0.3, 0, 0.05)^T.$$

魚節點平移後：

$$\mathbf{p}' = (1.3, 0, -1.95)^T.$$

右手系中繞 $+Y$ 旋轉 90° 有

$$(x, y, z) \rightarrow (z, y, -x),$$

所以

$$\mathbf{p}_{world} = (-1.95, 0, -1.3)^T \text{ m}.$$

2. 程式測試題解答

以下函式使用標準庫，不依賴第三方測試框架：

```
import tempfile
from pathlib import Path
import numpy as np

def test_obj_round_trip():
    vertices = np.array([
        [-0.20, 0.00, 0.00],
        [ 0.20, 0.00, 0.00],
        [ 0.00, 0.08, 0.00],
        [ 0.00,-0.08, 0.00],
```

```

        [ 0.00, 0.00, 0.05],
        [ 0.00, 0.00,-0.05],
    ])
    faces = np.array([
        [0, 4, 2], [2, 4, 1],
        [0, 3, 4], [3, 1, 4],
        [0, 2, 5], [2, 1, 5],
        [0, 5, 3], [3, 5, 1],
    ])

    with tempfile.TemporaryDirectory() as directory:
        path = Path(directory) / "fish.obj"
        write_obj(path, vertices, faces, "fish_skin")
        v2, f2, materials = read_simple_obj(path)
        report = validate_mesh(v2, f2)

        assert np.allclose(v2, vertices)
        assert np.array_equal(f2, faces)
        assert materials == ["fish_skin"] * len(faces)

        size = (np.array(report["bounds_max"])
                 - np.array(report["bounds_min"]))
        assert np.allclose(size, [0.4, 0.16, 0.1])
        assert report["degenerate_faces"] == []

```

預期所有斷言成立；仍需讀者實際執行確認環境與程式整合無誤。

3. 反例／除錯題解答

因為

$$\det(A) = 1 \cdot 1 \cdot (-1) = -1,$$

此變換包含鏡射，會翻轉手性與面繞序。若渲染器仍把逆時針視為正面，原本外向的面就會被當成背面剔除。

可採一致策略：

1. 將每個三角形 (a, b, c) 改為 (a, c, b) ；
2. 以 A^{-T} 轉換法線並重新正規化；
3. 若保留索引，則明確改變該網格的正面判定。

若含切線空間資料，也要同步修正手性。

4. 整合應用題解答

```

{
  "assets": {
    "fish_mesh": {
      "uri": "fish.obj",
      "format": "obj",

```



```

    "source_unit_in_meters": 1.0
  }
},
"nodes": {
  "pond": {
    "parent": null,
    "translation": [0, 0, 0],
    "rotation_y_degrees": 0,
    "scale": [1, 1, 1]
  },
  "fish_01": {
    "parent": "pond",
    "asset": "fish_mesh",
    "translation": [0, -0.5, -1],
    "rotation_y_degrees": 0,
    "scale": [1, 1, 1]
  },
  "fish_02": {
    "parent": "pond",
    "asset": "fish_mesh",
    "translation": [1, -0.6, -2],
    "rotation_y_degrees": 45,
    "scale": [1, 1, 1]
  },
  "fish_03": {
    "parent": "pond",
    "asset": "fish_mesh",
    "translation": [-1, -0.4, -1.5],
    "rotation_y_degrees": -30,
    "scale": [1, 1, 1]
  }
}
}

```

載入時應檢查：

1. `fish.obj` 存在且格式受支援。
2. 所有資產與父節點參照都存在。
3. 節點圖無循環。
4. TRS 數值有限且尺度非零。
5. OBJ 索引合法，且正索引沒有前向引用。
6. 單位與座標契約存在。

三條魚只有節點變換不同。共用資產可降低檔案與記憶體重複，也避免三份幾何逐漸產生版本差異。

本章小結

資產交換的重點不是「軟體能否開啟檔案」，而是幾何、材質、單位、座標與階層語意能否被明確重建。場景圖使用

$$M_{world} = M_{parent}M_{local}$$

累積變換；單位換算只能在一個明確位置完成；軸向轉換則必須檢查行列式、繞序與法線。

OBJ 適合簡單靜態網格，但場景階層與現代材質需要額外契約。本章以 JSON 串聯資產、材質、節點與相機，並以最小讀寫器示範「不支援便明確拒絕」的原則。glTF 可承載更完整的場景資料，但仍不能取代單位、版本、路徑及驗證規則。

能載入不代表尺度正確；能渲染也不代表生物或物理上真實。

參考來源

- [G1] PBRT 4，〈Transformations〉：變換、逆變換及幾何量轉換。
https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] LearnOpenGL，〈Transformations〉：齊次變換與矩陣組合。
<https://learnopengl.com/Getting-started/Transformations>
- [G7] NumPy 線性代數參考：矩陣與線性代數介面。
<https://numpy.org/doc/stable/reference/routines.linalg.html>
- [G8] Khronos，glTF 2.0 Specification：場景、節點、網格、材質與緩衝資料規格。
<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

本章教學 JSON 並不是 glTF 子集，也不宣稱與任何特定數位內容製作軟體的匯入設定完全等價。

第13章 色彩、線性光與影像取樣

學習目標與先備知識

完成本章後，你應能：

1. 分辨儲存用的 sRGB 數值與計算用的線性 RGB，實作兩者的分段轉換。
2. 在線性光下插值色彩，並以預乘 alpha 處理透明合成。
3. 依明確的 UV 與影像座標約定實作雙線性取樣。
4. 解釋縮小影像時的混疊，以及 mipmap 如何提供不同尺度的預濾波結果。
5. 判斷貼圖是色彩還是資料，避免對法線、深度與 ID 套用錯誤的色彩運算。

先備知識是 RGB、二維陣列、加權平均與基本 Python。以下影像均為合成資料；像素色彩不帶有公尺等長度單位，貼圖座標也無單位。

問題與直覺

想像在合成魚體上貼一張黑白棋盤格。魚靠近相機時，一個棋盤格可能覆蓋許多畫面像素；魚遠離時，一個畫面像素可能同時覆蓋數十格。前一種情況需要在鄰近貼圖像素之間平順取值；後一種情況則需要概括一整片區域。兩個需求不同：**雙線性取樣解決取樣位置落在像素之間的問題；預濾波解決縮小時一個樣本涵蓋太多細節的問題。**

色彩又增加一道關卡。一般影像中的 sRGB 數值經過非線性編碼，數值的一半不等於線性光量的一半。如果直接平均編碼值，漸層、透明邊緣與縮小版貼圖都可能偏暗。因此，先辨認資料意義，再決定能否解碼、平均與重新編碼，比選擇哪一種濾波函式更重要。

本書採用影像左上為像素原點：寬 W 、高 H 的影像，其欄索引 i 向右、列索引 j 向下，像素中心在影像座標 $(i + 0.5, j + 0.5)$ 。UV 則約定 s 向右、 t 向上。這裡用 s, t 避免與影像像素座標混淆；後續章節也常將它們記為 u, v 。

數學與幾何推導

線性 RGB 與 sRGB

「線性」表示通道數值與該通道所表示的光量成比例：在線性表示下，兩個樣本的等權平均可寫成 $(C_1 + C_2)/2$ 。它不表示 RGB 三通道已完整描述真實光譜，也不表示任意不同色彩空間的三個數字可以直接相加。本章假設色彩值使用相同的 sRGB 原色定義，只討論其編碼值與相應線性值之間的轉換。

令 $C_s \in [0, 1]$ 為一個 sRGB 通道， $C_\ell \in [0, 1]$ 為其線性值。解碼公式為

$$C_\ell = \begin{cases} C_s/12.92, & C_s \leq 0.04045, \\ ((C_s + 0.055)/1.055)^{2.4}, & C_s > 0.04045. \end{cases}$$

計算結束後，編碼供一般 sRGB 影像輸出的公式為

$$C_s = \begin{cases} 12.92C_\ell, & C_\ell \leq 0.0031308, \\ 1.055C_\ell^{1/2.4} - 0.055, & C_\ell > 0.0031308. \end{cases}$$

三個 RGB 通道各自套用。若原圖是 8 位元整數，先除以 255 得到 $[0, 1]$ 的值；最後輸出整數時才乘回 255 並量化。量化有資訊損失，所以不能要求整數影像反覆轉換後每次都精確不變。本章函式只接受 $[0, 1]$ ，不把高動態範圍值悄悄截斷成一般 sRGB。

資料貼圖是例外，而不是另一種色彩。法線貼圖的通道用來編碼方向；深度值代表距離或投影深度；物件 ID 代表類別標記。對這些數值套 sRGB 解碼，會改變資料意義。將貼圖接入渲染流程前，應記錄其用途、通道定義與是否採 sRGB 編碼。

Alpha 與預乘色彩

令 $\alpha \in [0, 1]$ 表示一層影像的覆蓋或不透明程度，令 $\mathbf{C} = (R, G, B)$ 是**線性 RGB**。Alpha 不套用 sRGB 轉換。對不透明背景 $\mathbf{C}_b$ ，前景 $\mathbf{C}_f$ 覆在其上的結果為

$$\mathbf{C}_o = \alpha_f \mathbf{C}_f + (1 - \alpha_f) \mathbf{C}_b.$$

這個式子隱含前景只在覆蓋部分貢獻色彩。處理多個透明圖層時，先定義預乘色彩 $\mathbf{P} = \alpha \mathbf{C}$ ，「前景覆在背景上」便是

$$\mathbf{P}_o = \mathbf{P}_f + (1 - \alpha_f) \mathbf{P}_b, \quad \alpha_o = \alpha_f + (1 - \alpha_f) \alpha_b.$$

若需要非預乘的 $\mathbf{C}_o$ ，且 $\alpha_o > 0$ ，再計算 $\mathbf{C}_o = \mathbf{P}_o / \alpha_o$ 。當 $\alpha_o = 0$ ，可見色彩無從由除法恢復，須自行約定儲存值。預乘表示也有助於透明邊緣的插值：直接插值非預乘 RGB，可能把完全透明像素中任意儲存的顏色混進可見邊緣。

雙線性取樣

以下指定一種**邊緣對齊**的貼圖座標規則，供本章實驗使用。正規化座標 $s, t \in [0, 1]$ ，影像陣列左上為 **[0]** **[0]**。將 UV 轉成連續的像素中心索引：

$$x = s(W - 1), \quad y = (1 - t)(H - 1).$$

式中的 $1 - t$ 是明確的上下翻轉。令 $i_0 = \lfloor x \rfloor$ 、 $j_0 = \lfloor y \rfloor$ ，令 $i_1 = \min(i_0 + 1, W - 1)$ 、 $j_1 = \min(j_0 + 1, H - 1)$ ；再令 $a = x - i_0$ 、 $b = y - j_0$ 。用 $T_{00}, T_{10}, T_{01}, T_{11}$ 表示左右、上下相鄰的四個樣本，則

$$T(x, y) = (1 - b) [(1 - a)T_{00} + aT_{10}] + b [(1 - a)T_{01} + aT_{11}].$$

四個權重非負且總和為一。對色彩取樣時， T 必須是解碼後的線性值；對一般資料取樣時，能否插值取決於資料語意。這套座標規則讓 $(0, 1)$ 對應左上像素中心、 $(1, 0)$ 對應右下像素中心，方便手算。它不是所有 API 的半像素約定；移植程式時應測試四角與邊界，而非只比較畫面外觀。

混疊、預濾波與 mipmap

一個輸出像素有面積。縮小貼圖時，若它在原圖覆蓋的區域包含許多黑白交替的格子，僅取中心一點可能讀到全黑，也可能讀到全白；稍微移動相機，結果就會跳變。這稱為混疊。理想上，應先按照輸出像素的覆蓋區域對原圖預濾波，再取樣；已被單點取樣丟失的細節無法事後恢復。

Mipmap 用多個解析度近似這項工作。第 0 級是原圖，下一級寬、高通常各約減半，直到 1×1 ；選擇哪一級取決於一個輸出像素在貼圖上涵蓋的足跡。足跡越大，應選越粗的級別。本章不計算畫面到 UV 的微分足跡；那需要結合三角形投影與 UV 插值，留待下一章。此處先實作可檢查的縮小級別。

建立色彩 mipmap 時，要在線性 RGB 下平均。若要濾波透明色彩，則平均預乘色彩與 alpha，避免透明像素的任意 RGB 污染邊緣。雙線性取樣只處理選定級別內的鄰近樣本；在相鄰兩個級別之間再插值，稱為三線性取樣。單一 mipmap 級別對斜視表面的細長足跡只是近似，並非一切混疊的完整解法。

逐步手算例題

例一：兩種「一半」不同

取 sRGB 灰階 $C_s = 0.5$ 。因為 $0.5 > 0.04045$ ，解碼得

$$C_\ell = \left(\frac{0.5 + 0.055}{1.055} \right)^{2.4} \approx 0.2140.$$

若把黑色與白色的線性光等權平均，應先得到 $C_\ell = (0 + 1)/2 = 0.5$ ，再編碼：

$$C_s = 1.055(0.5)^{1/2.4} - 0.055 \approx 0.7354.$$

因此，直接在 sRGB 數值上平均而得的 0.5，比正確編碼結果 0.7354 暗。兩者都可能寫作「0.5」，但所在的表示空間不同。

例二：checker 中心與透明合成

設一張 2×2 線性灰階影像，陣列由上到下是

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

在 $(s, t) = (0.5, 0.5)$ ，有 $x = y = 0.5$ ，所以 $a = b = 0.5$ 。四個樣本各佔四分之一：

$$T = 0.25(0) + 0.25(1) + 0.25(1) + 0.25(0) = 0.5.$$

若將此灰色當作 $\alpha_f = 0.25$ 的前景，覆在不透明線性白色背景上，結果是 $0.25(0.5) + 0.75(1) = 0.875$ ，alpha 為 1。0.875 是**線性**結果；要顯示在 sRGB 影像中，仍須經編碼。

例三：為何要先解碼再縮小

考慮相鄰的兩個 sRGB 灰階像素 0 與 0.5。若直接平均編碼值，縮小像素為 0.25。正確次序是先解碼：兩個線性值約為 0 與 0.2140，平均為 0.1070，再編碼約為 0.361。這一差異不是雙線性或 mipmap 所選權重造成，而是**平均發生在哪個表示空間**造成。

實作與程式

下列程式使用 Python 3.10+ 標準庫，不讀檔、不使用網路。影像是由上到下的列，每個像素為三個浮點通道。bilinear 採 clamp 邊界；downsample2 對最多 2×2 個現存像素取平均，因此奇數尺寸的最右欄、最下列不會被直接丟棄。它是一個易於驗算的示範濾波器，**不是**對任意像素足跡都精確的面積濾波器。

```
from math import floor, isclose

def srgb_to_linear(c):
    if not 0.0 <= c <= 1.0:
        raise ValueError("sRGB 通道須在 [0, 1]")
    if c <= 0.04045:
        return c / 12.92
    return ((c + 0.055) / 1.055) ** 2.4

def linear_to_srgb(c):
    if not 0.0 <= c <= 1.0:
        raise ValueError("線性通道須在 [0, 1]")
    if c <= 0.0031308:
        return 12.92 * c
    return 1.055 * c ** (1.0 / 2.4) - 0.055

def convert_rgb(rgb, fn):
    if len(rgb) != 3:
        raise ValueError("需要三個 RGB 通道")
    return tuple(fn(c) for c in rgb)

def checker(width, height):
    if width < 1 or height < 1:
        raise ValueError("寬高須為正整數")
    # 黑白端點在 sRGB 與線性表示下都是 0 與 1。
    return [[(float((i + j) % 2),) * 3 for i in range(width)]
            for j in range(height)]

def dimensions(image):
    if not image or not image[0]:
        raise ValueError("空影像")
    w = len(image[0])
    if any(len(row) != w for row in image):
        raise ValueError("各列寬度須相同")
```

```

if any(len(pixel) != 3 for row in image for pixel in row):
    raise ValueError("每個像素須有三個通道")
return w, len(image)

def bilinear(image, s, t):
    w, h = dimensions(image)
    s = max(0.0, min(1.0, s))
    t = max(0.0, min(1.0, t))
    x, y = s * (w - 1), (1.0 - t) * (h - 1)
    i0, j0 = floor(x), floor(y)
    i1, j1 = min(i0 + 1, w - 1), min(j0 + 1, h - 1)
    a, b = x - i0, y - j0

    def mix(p, q, weight):
        return tuple((1.0 - weight) * p[k] + weight * q[k]
                      for k in range(3))

    top = mix(image[j0][i0], image[j0][i1], a)
    bottom = mix(image[j1][i0], image[j1][i1], a)
    return mix(top, bottom, b)

def downsample2(linear_image):
    w, h = dimensions(linear_image)
    result = []
    for j in range(0, h, 2):
        row = []
        for i in range(0, w, 2):
            block = [linear_image[y][x]
                     for y in range(j, min(j + 2, h))
                     for x in range(i, min(i + 2, w))]
            row.append(tuple(sum(p[k] for p in block) / len(block)
                             for k in range(3)))
        result.append(row)
    return result

def over(fore_rgb, fore_alpha, back_rgb, back_alpha):
    if not 0.0 <= fore_alpha <= 1.0:
        raise ValueError("前景 alpha 須在 [0, 1]")
    if not 0.0 <= back_alpha <= 1.0:
        raise ValueError("背景 alpha 須在 [0, 1]")
    out_alpha = fore_alpha + (1.0 - fore_alpha) * back_alpha
    premult = tuple(fore_alpha * fore_rgb[k]
                    + (1.0 - fore_alpha) * back_alpha * back_rgb[k]
                    for k in range(3))
    if out_alpha == 0.0:
        return (0.0, 0.0, 0.0), 0.0
    return tuple(c / out_alpha for c in premult), out_alpha

# 讀者端測試：以下皆為按公式推得的預期值。
for c in (0.0, 0.02, 0.5, 1.0):
    assert isclose(linear_to_srgb(srgb_to_linear(c)),
                   c, abs_tol=1e-12)
assert isclose(srgb_to_linear(0.5), 0.21404, abs_tol=1e-5)

tex = checker(2, 2)
assert all(isclose(c, 0.5) for c in bilinear(tex, 0.5, 0.5))
assert bilinear(tex, 0.0, 1.0) == (0.0, 0.0, 0.0)
assert bilinear(tex, 0.0, 0.0) == (1.0, 1.0, 1.0)
assert bilinear(tex, -0.2, 1.0) == bilinear(tex, 0.0, 1.0)

```

```

small = downsample2(tex)
assert len(small) == 1 and len(small[0]) == 1
assert all(isclose(c, 0.5) for c in small[0][0])

rgb, alpha = over((0.5,) * 3, 0.25, (1.0,) * 3, 1.0)
assert all(isclose(c, 0.875) for c in rgb)
assert isclose(alpha, 1.0)

```

使用非黑白色彩影像時，應先對每個 sRGB 像素呼叫 `convert_rgb(pixel, srgb_to_linear)`，再交給 `bilinear` 或 `downsample2`；待顯示的線性 RGB 才呼叫 `convert_rgb(pixel, linear_to_srgb)`。程式刻意不在取樣函式內自動解碼，因為同一個取樣函式也可能收到已在線性表示的色彩或完全不是色彩的資料。

測試與預期結果

程式末尾斷言的**預期**是全部通過，並且不會列印內容；這些結果未宣稱已在作者端執行。它們分別核對轉換往返、0.5 sRGB 的非線性值、UV 上下方向、clamp、 2×2 平均及 alpha 合成。

往返測試本身不夠：即使兩個錯誤函式恰好互為反函式，也可能讓往返通過。因此另以手算可核對的 0.21404 作為錨點。若自行測 8 位元輸出，應容許量化誤差，且應比較相同的量化規則。另可建立 3×2 、上下兩列顏色不同的影像，檢查奇數寬度縮小後的尺寸與四個 UV 角；這比只測對稱棋盤格更容易發現索引錯誤。

除錯與常見陷阱

- ****平均值偏暗：****確認是否在 sRGB 編碼值上直接平均。色彩解碼一次、在線性空間計算、最後編碼一次；重複解碼也會出錯。
- ****貼圖上下顛倒：****陣列列索引向下、UV 的 t 向上。檢查 (0, 1) 是否讀到左上，而非依印象交換兩個角。
- ****透明邊緣出現暗邊或雜色：****檢查濾波時是否混入透明像素儲存的非預乘 RGB。預乘色彩與 alpha 應一同處理。
- ****棋盤遠處閃爍：****同級雙線性只混合附近四個樣本，無法代表一個輸出像素跨越的許多格；需要預濾波及合適的 mipmap 級別。
- ****法線圖顏色或方向異常：****確認資料沒有誤走 sRGB 色彩解碼，且方向資料插值後依其定義重新正規化。法線圖的切線空間細節留待後續章節。
- ****ID 變成不存在的類別：****物件 ID 是離散標記，不能用一般的數值平均創造「中間 ID」。

養殖數位分身案例

建立合成池體與魚模型時，可以為魚體放置 checker 色彩貼圖，以檢查取樣與 UV 方向；另用法線資料貼圖示意表面細節，用整數 ID 圖標記不同魚體。這三者都可能儲存在多通道影像裡，卻不能因此共用同一套色彩處理。

可稽核的處理順序是：記錄貼圖用途及編碼；色彩圖先解碼為線性 RGB，再取樣、合成及產生縮小級別；法線資料依其方向編碼解讀，不作 sRGB 解碼；ID 保持離散，縮小時明訂標註規則；最後只把待顯示的色彩編碼為 sRGB。若魚模型遠離相機後 checker 由閃爍變得平順，這只驗證影像取樣流程的一部分，不能據此聲稱合成魚的材質、水質或生態行為符合實測。

習題

1. **手算**：黑、白兩個 sRGB 灰階值 0、1 在線性光下等權平均，所得線性值與約略 sRGB 輸出各是多少？若直接平均編碼值得 0.5，它所代表的線性值約是多少？
2. **程式測試**：使用 `checker(3, 2)`，求左上與左下的 UV 座標和預期 RGB。再寫一個斷言，檢查 `bilinear` 對超出左邊界的 `s` 採 clamp；說明測試影像應具備甚麼條件。
3. **反例／除錯**：將線性前景紅色 $(1, 0, 0)$ 、 $\alpha_f = 0.5$ 覆在不透明線性藍色 $(0, 0, 1)$ 上。有人直接在 sRGB 數值內混合，宣稱輸出為 $(0.5, 0, 0.5)$ sRGB。指出錯誤並求正確線性結果與約略 sRGB 結果。
4. **整合應用**：魚體色彩圖、水面法線圖及物件 ID 圖都要縮小使用。逐一指出是否套 sRGB 解碼，並說明為何不能一律以「雙線性平均 RGB」建立其 mipmap。

習題解答

1. 黑白在線性表示下仍是 0、1，平均為 0.5；編碼後約為 0.7354 sRGB。直接平均得到的 0.5 sRGB 解碼後約為 0.2140 線性值，因此不是一半線性光。
2. 左上 UV 是 $(0, 1)$ ，對應欄、列 $(0, 0)$ ，預期 RGB 為 $(0, 0, 0)$ ；左下 UV 是 $(0, 0)$ ，對應 $(0, 1)$ ，預期 $(1, 1, 1)$ 。例如可寫 `assert bilinear(image, -0.2, 1.0) == bilinear(image, 0.0, 1.0)`。為使此測試足以區分 clamp 與 wrap，影像左右邊界應有不同值；只用邊界顏色相同的圖，錯誤的 wrap 實作也可能通過。
3. 必須先在线性光下合成，得 $(0.5, 0, 0.5)$ **線性 RGB**、alpha 為 1。兩個非零通道各自編碼後，約為 $(0.7354, 0, 0.7354)$ sRGB。題述數字碰巧等於正確的線性結果，錯在把它當成已編碼的輸出。
4. 魚體色彩圖先作 sRGB 解碼，在線性空間依縮小足跡濾波；若帶 alpha，宜濾波預乘色彩及 alpha。法線圖不作 sRGB 解碼；須按其格式還原方向，處理方向平均與重新正規化，不能把平均後 RGB 當成已完成的單位法線。ID 圖也不作 sRGB 解碼，且不可平均數字：ID 1 與 ID 3 的平均值 2 不代表 ID 2 出現。可依任務選最近鄰、指定優先規則，或另存各 ID 的覆蓋比例。

本章小結

色彩計算應在線性 RGB 中進行，sRGB 是輸入與輸出時需要明確處理的編碼。Alpha 有自己的合成規則；預乘表示有助於多層合成與透明邊緣取樣。雙線性取樣處理位置，mipmap 提供縮小所需的多尺度預濾波。開始運算前，先區分色彩與資料，並固定影像、UV 及邊界約定。

參考來源

- G8 Khronos glTF 2.0 規格：交換資產時核對材質與貼圖語意的參考；此處不宣稱已逐節查核。
- G2 PBRT 4：Reflection Models：線性色彩銜接表面反射模型的延伸閱讀。
- G3 PBRT 4：The Light Transport Equation：後續光傳輸計算的延伸閱讀。

第14章 UV參數化與貼圖座標

學習目標與先備知識

本章旨在建立三角形網格上的二維參數化（UV）體系，使讀者能將材質貼圖準確映射至三維表面。讀者需具備以下先備知識：

1. **三角網格拓撲**：理解頂點索引、面片與頂點的關聯，以及渲染頂點（Render Vertex）與幾何頂點的區別（第7章）。渲染頂點的關鍵由位置、法線與 UV 索引組成，即 $(i_{\text{position}}, i_{\text{normal}}, i_{\text{uv}})$ 。
2. **重心座標**：能計算點在三角形內的 $(\lambda_0, \lambda_1, \lambda_2)$ 座標，並理解其線性插值性質（第6章）。
3. **透視投影與齊次座標**：理解裁剪空間（Clip Space）的 w 分量如何影響屏幕空間（NDC）與世界空間的線性關係，特別是需要「透視正確插值（Perspective-Correct Interpolation）」的原因（第5、6章）。
4. **色彩空間**：明確區分線性RGB（Linear RGB）與sRGB。貼圖若是顏色貼圖，需進行 sRGB $\rightarrow$ Linear 轉換；若是法線貼圖或遮罩，則為線性資料，不可轉換（第13章）。

本章核心任務：

- 定義三角形UV座標系與接縫處理。
- 推導貼圖取樣座標的計算方式，包含 Wrap（重複）與 Clamp（限制）模式，並處理紋素中心（Texel Center）偏移。
- 實作一個將 Checkerboard 貼圖映射至簡化魚體模型的流程，並處理接縫、上下翻轉與透視插值誤差。

問題與直覺

在二維像素世界，我們以 (x, y) 索引像素。當模型進入三維，表面不再是平坦的網格，而是彎曲的三角形集合。若直接將三維座標映射到貼圖，會導致重疊、撕裂與比例失真。因此，我們需要一個獨立的二維參數化座標系，稱為 UV 座標。

- **U 軸**：對應貼圖水平方向（向右為正）。
- **V 軸**：對應貼圖垂直方向（向上為正）。

關鍵直覺：UV 座標是定義在**渲染頂點**上的屬性。當三角形被光柵化時，我們需要計算三角形內部任意像素的 UV 值。由於 UV 在透視投影下並非線性變化於 NDC 空間，因此必須使用**透視正確插值**來計算 UV。若直接使用 NDC 座標進行線性插值，貼圖會看起來「浮動」或錯位，特別是在模型旋轉或近大遠小時。

數學與幾何推導

1. UV 座標定義與紋素中心映射

設三角形頂點為 V_0, V_1, V_2 ，對應 UV 座標為 (u_i, v_i) 。UV 座標單位為貼圖週期，1.0 代表貼圖的整個寬度或高度。本章使用紋素中心映射 $x = uW - 0.5$ 、 $y = (1 - v)H - 0.5$ ；第13章則採邊緣對齊映射 $x = u(W - 1)$ 、 $y = (1 - v)(H - 1)$ 。兩套約定都可使用，但同一 UV 的取樣位置不同：本章在 $u = 0$ 時落於邊界位置，再依 clamp 或 repeat 規則處理；第13章則落在第0欄中心。本章貼圖原點 $(0, 0)$ 位於左下角、V 軸向上；影像陣列原點通常在左上角、列索引向下。跨章沿用取樣程式時，必須同時核對座標映射與邊界模式。

為了精確取樣，必須考慮**紋素中心 (Texel Center)**。若貼圖寬高為 W, H ，影像陣列 `tex[j, i]` 中， i 是向右增加的水平 column index， j 是向下增加的垂直 row index，且 $j = 0$ 位於頂部。該像素中心在連續 UV 空間中的座標為：

$$u_i = \frac{i + 0.5}{W}$$
$$v_j = 1 - \frac{j + 0.5}{H}$$

因此，連續 UV (u, v) 對應的像素浮點座標 (x, y) 定義為：

$$x = u \cdot W - 0.5$$
$$y = (1 - v) \cdot H - 0.5$$

其中 v 的翻轉是為了對齊「V 向上」與「影像 Y 向下」。取樣時，需計算 x, y 相鄰的整數像素索引，並進行雙線性插值。

2. 透視正確插值推導

在光柵化過程中，已知頂點在裁剪空間的 w 分量 w_i 及 UV 值 A_i (A 可為 u 或 v)。在 NDC 空間中，像素的重心座標為 λ_i ($\sum \lambda_i = 1$)。若直接計算 $A = \sum \lambda_i A_i$ ，在透視投影下是錯誤的。

推導：先說明這裡的「像素 P」對應到什麼。一個螢幕像素不直接對應世界空間中某個固定點，而是對應一條由相機位置出發、穿過該像素中心的射線。這條射線與三角形所在平面相交，得到三角形上唯一的表面點，記為 P 。 β_i 是這個表面點相對於投影前原始三角形的仿射重心座標； λ_i 則是該像素在投影後 NDC 空間中對應的重心座標。兩者的關係並非相等。

透視投影保持共線性（直線仍映到直線），但**不**保持沿線的仿射比例。這正是需要透視修正的根本原因。若把仿射比例誤以為在投影下不變，就會得到錯誤的 UV。正確的關係是把投影後的重心權重以 $1/w_i$ 加權再重新歸一化：

$$\beta_i = \frac{\lambda_i / w_i}{\sum_{k=0}^2 \lambda_k / w_k}.$$

這個式子可以從相似三角形的比值推導而得：三角形的三條邊在投影前後都是直線，但各頂點距離相機的深淺不同；越遠的頂點，其對應線段在螢幕上被壓縮得越短，因此權重需要按 $1/w$ 補償。當所有 w_i 都相等時， $1/w_i$ 是共同常數，分子分母同乘一正數， β_i 退化成 λ_i ，透視修正也就退化為一般線性插值——這正是第 6 章提到的特例。

$$\beta_i = \frac{\lambda_i/w_i}{\sum_{k=0}^2 \lambda_k/w_k}$$

屬性 A 在原三角形上是仿射（線性）插值：

$$A_p = \sum_{i=0}^2 \beta_i A_i$$

代入 β_i 的表達式：

$$A_p = \frac{\sum_{i=0}^2 (\lambda_i/w_i) A_i}{\sum_{k=0}^2 \lambda_k/w_k}$$

此公式保證了在世界空間中，屬性沿著三角形邊線性變化。

3. 接縫與 Wrap 模式

當 UV 座標超出 $[0, 1]$ 範圍時，需定義取樣行為：

1. Wrap (Repeat)：

$$u_{\text{sample}} = u \pmod{1.0}$$

Python 的 `%` 運算子對負數處理正確（結果在 $[0, 1)$ ）。此模式適用於格紋等重複紋理。

2. Clamp：

$$u_{\text{sample}} = \max(0, \min(1, u))$$

超出範圍時取邊緣像素。

接縫處理：若模型有接縫，同一 3D 幾何位置可能對應不同的 UV 值。在數據層面，必須複製**完整渲染頂點**（包含位置、法線與 UV），不能僅複製 UV。**週期接縫的連續性：**若三角形跨越 UV 0/1 邊界（例如頂點 UV 為 0.9, 0.1），直接插值會經過 0.5，即繞過整張貼圖。正確做法是：

1. 在 UV 展開資料中，將跨越邊界的頂點 UV 值調整為連續分支（例如將 0.1 表示為 1.1）。
2. 在連續值 $0.9 \rightarrow 1.1$ 間插值。
3. 最後在取樣器套用 Repeat，使 $1.0 \equiv 0.0$ 。不可逐頂點先取小數部分再插值。

4. 微分足跡與 Mipmap

微分足跡 (Differential Footprint) 描述 UV 對屏幕座標 (x_s, y_s) 的變化率。定義 Jacobian 矩陣：

$$J = \begin{bmatrix} \partial u / \partial x_s & \partial u / \partial y_s \\ \partial v / \partial x_s & \partial v / \partial y_s \end{bmatrix}$$

其量綱為 **UV 單位 / 屏幕像素**。為了估算需要的 Mipmap 等級 (LOD)，需將其轉換為 **Texel / 屏幕像素**。定義水平與垂直方向的 Texel 密度：

$$\rho_x = \sqrt{\left(W \frac{\partial u}{\partial x_s}\right)^2 + \left(H \frac{\partial v}{\partial x_s}\right)^2}$$
$$\rho_y = \sqrt{\left(W \frac{\partial u}{\partial y_s}\right)^2 + \left(H \frac{\partial v}{\partial y_s}\right)^2}$$

取最大值 $\rho = \max(\rho_x, \rho_y)$ 。若 $\rho > 1$ ，表示一個屏幕像素涵蓋超過一個 Texel，需要更高的 Mipmap 等級以避免混疊。LOD 等級可估算為 $L = \max(0, \log_2 \rho)$ 。注意：投影後三角形越小（遠離相機或細節越多）， ρ 通常越大。

逐步手算例題

例題 1：透視正確 UV 插值

設三角形頂點在裁剪空間的 w 分量為 $w_0 = 1.0, w_1 = 2.0, w_2 = 3.0$ 。對應 UV 座標： $V_0 : (0.0, 0.0)$ $V_1 : (1.0, 0.0)$ $V_2 : (0.0, 1.0)$

NDC 空間中像素 P 的重心座標為 $\lambda_0 = 0.5, \lambda_1 = 0.3, \lambda_2 = 0.2$ 。計算像素 P 的 u 與 v 值。

解：分母 $D = \sum \lambda_i / w_i = \frac{0.5}{1} + \frac{0.3}{2} + \frac{0.2}{3} = 0.5 + 0.15 + 0.0667 = 0.7167$ 。精確分數： $D = \frac{1}{2} + \frac{3}{20} + \frac{1}{15} = \frac{30+9+4}{60} = \frac{43}{60}$ 。

u 分子 $N_u = \sum \lambda_i \frac{u_i}{w_i} = 0 + 0.3 \cdot \frac{1}{2} + 0 = 0.15 = \frac{3}{20}$ 。

$$u_p = \frac{3/20}{43/60} = \frac{9}{43} \approx 0.2093$$

v 分子 $N_v = \sum \lambda_i \frac{v_i}{w_i} = 0 + 0 + 0.2 \cdot \frac{1}{3} = \frac{1}{15}$ 。

$$v_p = \frac{1/15}{43/60} = \frac{4}{43} \approx 0.0930$$

若錯誤地直接線性插值： $u_{\text{lin}} = 0.5(0) + 0.3(1) + 0.2(0) = 0.3$ ， $v_{\text{lin}} = 0.2$ 。透視插值結果 (0.2093, 0.0930) 與線性 (0.3, 0.2) 差異顯著。

例題 2：Wrap 模式下的負 UV

設 $u = -0.2, v = 1.5$ 。Wrap 模式：

$$u_{\text{sample}} = -0.2 \pmod{1.0} = 0.8$$

$$v_{\text{sample}} = 1.5 \pmod{1.0} = 0.5$$

取樣座標為 (0.8, 0.5)。

Clamp 模式：

$$u_{\text{sample}} = \max(0, \min(1, -0.2)) = 0.0$$

$$v_{\text{sample}} = \max(0, \min(1, 1.5)) = 1.0$$

取樣座標為 (0.0, 1.0)。

實作與程式

以下 Python 程式碼示範一個簡化的 CPU 光柵化實驗：

1. 定義簡化魚體（由兩個三角形組成，含接縫 UV）。
2. 實作支援 Clamp 與 Repeat 模式的雙線性取樣（處理紋素中心與邊界）。
3. 對魚體三角形進行光柵化，比較透視插值與線性插值的差異。
4. 輸出 PPM 影像，並驗證接縫處理與上下翻轉。

```
import numpy as np

def generate_checker_texture(width=8, height=8):
    """生成線性 RGB 的棋盤格貼圖，用於對稱測試。"""
    tex = np.zeros((height, width, 3), dtype=np.float32)
    for y in range(height):
        for x in range(width):
            if (x + y) % 2 == 0:
                tex[y, x] = [1.0, 1.0, 1.0]
            else:
                tex[y, x] = [0.0, 0.0, 0.0]
    return tex

def generate_stripe_texture(width=8, height=8):
    """
    生成線性 RGB 的非對稱垂直灰階條紋：每一固定 x 的直欄
```

灰階值為 $x / (W - 1)$ ， $x=0$ 為黑、 $x=W-1$ 為白。
用於辨認 Repeat 位址及連續分支插值的取樣位置；
對稱棋盤格在 $u=0.0$ 與 $u=0.5$ 可能偶然同色，無法區分接縫對錯。
"""

```
tex = np.zeros((height, width, 3), dtype=np.float32)
for y in range(height):
    for x in range(width):
        g = x / max(1, width - 1)
        tex[y, x] = [g, g, g]
return tex
```

```
def generate_orientation_texture(width=8, height=8):
    """影像上半紅、下半藍；用非對稱顏色檢查 UV 的 v 向上。"""
    tex = np.zeros((height, width, 3), dtype=np.float32)
    for y in range(height):
        tex[y, :, :] = ([1.0, 0.0, 0.0] if y < height // 2
                        else [0.0, 0.0, 1.0])
    return tex
```

```
def sample_bilinear(tex, u, v, wrap=False):
    """
    雙線性取樣。
    u, v: 連續 UV 座標。
    wrap: 若為 True，使用 Repeat 模式；否則 Clamp。
    """
    h, w, _ = tex.shape
    u = float(u)
    v = float(v)

    # 計算紋素中心座標
    x = u * w - 0.5
    y = (1.0 - v) * h - 0.5

    if wrap:
        # Repeat 模式：對整數索引取模
        x0 = int(np.floor(x))
        y0 = int(np.floor(y))
        fx = x - x0
        fy = y - y0

        x0_c = x0 % w
        x1_c = (x0 + 1) % w
        y0_c = y0 % h
        y1_c = (y0 + 1) % h
    else:
        # Clamp 模式
        x = np.clip(x, -0.5, w - 0.5) # 允許取樣到邊界外 0.5 像素以支援 clamp 行為
        y = np.clip(y, -0.5, h - 0.5)
        x0 = int(np.floor(x))
        y0 = int(np.floor(y))
        fx = x - x0
        fy = y - y0

        x0_c = min(max(x0, 0), w - 1)
        x1_c = min(max(x0 + 1, 0), w - 1)
        y0_c = min(max(y0, 0), h - 1)
        y1_c = min(max(y0 + 1, 0), h - 1)

    c00 = tex[y0_c, x0_c]
```



```

c01 = tex[y0_c, x1_c]
c10 = tex[y1_c, x0_c]
c11 = tex[y1_c, x1_c]

return c00 * (1-fx)*(1-fy) + c01 * fx*(1-fy) + c10 * (1-fx)*fy + c11 * fx*fy

def perspective_uv(uv0, uv1, uv2, w0, w1, w2, lam0, lam1, lam2):
    """計算透視正確 UV。"""
    ws = np.array([w0, w1, w2], dtype=float)
    lams = np.array([lam0, lam1, lam2], dtype=float)
    uvs = np.asarray([uv0, uv1, uv2], dtype=float)
    if uvs.shape != (3, 2) or not np.all(np.isfinite(uvs)):
        raise ValueError("UVs must be finite 2D coordinates")
    if not np.all(np.isfinite(ws)) or np.any(np.abs(ws) < 1e-12):
        raise ValueError("w must be finite and non-zero")
    if not np.all(np.isfinite(lams)):
        raise ValueError("barycentric coordinates must be finite")
    if not np.isclose(np.sum(lams), 1.0, atol=1e-8):
        raise ValueError("barycentric coordinates must sum to one")

    denom = np.sum(lams / ws)
    if not np.isfinite(denom) or abs(denom) < 1e-12:
        raise ValueError("invalid perspective denominator")

    num_u = np.sum(lams / ws * np.array([uv0[0], uv1[0], uv2[0]]))
    num_v = np.sum(lams / ws * np.array([uv0[1], uv1[1], uv2[1]]))

    return num_u / denom, num_v / denom

def edge_function(a, b, p):
    """回傳兩倍有向面積；符號可用於內外判定。"""
    return (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])

def rasterize_triangle(
    image, ndc_v0, ndc_v1, ndc_v2,
    uv0, uv1, uv2,
    w0, w1, w2,
    tex, wrap=False, perspective=True,
):
    """
    在 NDC 空間光柵化一個三角形並寫入 image。

    perspective=True 使用透視正確 UV 插值；False 則以仿射（線性）插值，
    供對照用。ndc_v* 為 (x_ndc, y_ndc)，w* 為該頂點的裁剪 $w$ 分量。
    本函式沒有深度測試或 top-left 共邊歸屬規則；後寫入者會覆蓋
    先寫入者。僅作不含透明混合及深度競爭的插值示範，不能靠呼叫
    順序代替深度測試。
    """
    H, W = image.shape[:2]
    a = np.asarray(ndc_v0, dtype=float)
    b = np.asarray(ndc_v1, dtype=float)
    c = np.asarray(ndc_v2, dtype=float)

    area2 = edge_function(a, b, c)
    if abs(area2) < 1e-12:
        return # 退化三角形，丟棄

    xmin = max(0, int(np.floor((min(a[0], b[0], c[0]) + 1.0) * W / 2.0)))

```

```

xmax = min(W - 1, int(np.ceil((max(a[0], b[0], c[0]) + 1.0) * W / 2.0)))
ymin = max(0, int(np.floor((1.0 - max(a[1], b[1], c[1])) * H / 2.0)))
ymax = min(H - 1, int(np.ceil((1.0 - min(a[1], b[1], c[1])) * H / 2.0)))

for py in range(ymin, ymax + 1):
    for px in range(xmin, xmax + 1):
        x_ndc = (px + 0.5) / (W / 2.0) - 1.0
        y_ndc = 1.0 - (py + 0.5) / (H / 2.0)
        p = np.array([x_ndc, y_ndc])

        w0e = edge_function(b, c, p)
        w1e = edge_function(c, a, p)
        w2e = edge_function(a, b, p)

        # 依三角形整體繞序一致處理，避免加入背面剔除時符號混亂。
        if area2 > 0:
            inside = (w0e >= 0.0) and (w1e >= 0.0) and (w2e >= 0.0)
        else:
            inside = (w0e <= 0.0) and (w1e <= 0.0) and (w2e <= 0.0)
        if not inside:
            continue

        lam0 = w0e / area2
        lam1 = w1e / area2
        lam2 = w2e / area2

        if perspective:
            u, v = perspective_uv(
                uv0, uv1, uv2,
                w0, w1, w2,
                lam0, lam1, lam2,
            )
        else:
            u = lam0 * uv0[0] + lam1 * uv1[0] + lam2 * uv2[0]
            v = lam0 * uv0[1] + lam1 * uv1[1] + lam2 * uv2[1]

        color = sample_bilinear(tex, u, v, wrap=wrap)
        image[py, px] = np.clip(color * 255.0 + 0.5, 0.0, 255.0).astype(np.uint8)

def write_ppm(path, image):
    """寫出 P6 PPM; image 為 uint8、shape (H, W, 3)，資料為線性 RGB 直接量化。"""
    if image.dtype != np.uint8:
        raise ValueError("PPM 影像必須使用 uint8")
    h, w, channels = image.shape
    if channels != 3:
        raise ValueError("預期 RGB 影像")
    with open(path, "wb") as f:
        f.write(f"P6\n{w} {h}\n255\n".encode("ascii"))
        f.write(image.tobytes())

def main():
    tex = generate_checker_texture(8, 8)

    # 1. 測試透視 vs 線性
    w0, w1, w2 = 1.0, 2.0, 3.0
    uv0 = np.array([0.0, 0.0])
    uv1 = np.array([1.0, 0.0])
    uv2 = np.array([0.0, 1.0])

```

```

lam0, lam1, lam2 = 0.5, 0.3, 0.2

u_pc, v_pc = perspective_uv(uv0, uv1, uv2, w0, w1, w2, lam0, lam1, lam2)
u_lin = lam0*uv0[0] + lam1*uv1[0] + lam2*uv2[0]
v_lin = lam0*uv0[1] + lam1*uv1[1] + lam2*uv2[1]

print(f"PC UV: ({u_pc:.4f}, {v_pc:.4f})")
print(f"Lin UV: ({u_lin:.4f}, {v_lin:.4f})")

c_pc = sample_bilinear(tex, u_pc, v_pc, wrap=False)
c_lin = sample_bilinear(tex, u_lin, v_lin, wrap=False)
print(f"Color PC: {c_pc}")
print(f"Color Lin: {c_lin}")

# 2. 測試接縫 Wrap
# 連續 UV: 0.9 -> 1.1
u_mid_correct = 1.0 # (0.9+1.1)/2
u_mid_wrong = 0.5 # (0.9+0.1)/2
v_mid = 0.5

c_seam_correct = sample_bilinear(tex, u_mid_correct, v_mid, wrap=True)
c_seam_wrong = sample_bilinear(tex, u_mid_wrong, v_mid, wrap=True)
print(f"Seam Correct (u=1.0): {c_seam_correct}")
print(f"Seam Wrong (u=0.5): {c_seam_wrong}")

# 非線性、非對稱欄值可區分連續分支中點 u=1.0 與錯誤中點 u=0.5。
seam_values = np.array([0.0, 0.05, 0.15, 0.30,
                        0.50, 0.65, 0.80, 1.0], dtype=np.float32)
seam_tex = np.zeros((8, 8, 3), dtype=np.float32)
for x, g in enumerate(seam_values):
    seam_tex[:, x, :] = g
seam_ok = sample_bilinear(seam_tex, 1.0, 0.5, wrap=True)
seam_bad = sample_bilinear(seam_tex, 0.5, 0.5, wrap=True)
assert np.allclose(seam_ok, sample_bilinear(seam_tex, 0.0, 0.5,
                                             wrap=True))

assert not np.allclose(seam_ok, seam_bad)

# 影像陣列上方是紅色，但 UV 的 v 向上；兩處取樣不可顛倒。
orientation = generate_orientation_texture()
assert np.allclose(sample_bilinear(orientation, 0.5, 0.75),
                    [1.0, 0.0, 0.0])
assert np.allclose(sample_bilinear(orientation, 0.5, 0.25),
                    [0.0, 0.0, 1.0])

# 3. 低面數魚身代理：兩個共享邊的三角形構成菱形。
# 四個幾何頂點 A、B、C、D；左右三角形共享對角線 B-D。
# 此處兩側 UV 相同，並未建立魚身上的週期接縫；上面的
# 0.9→1.1 與 0.9→0.1 對照才是獨立的接縫分支測試。
A_ndc = np.array([-0.8, 0.0])
B_ndc = np.array([0.0, 0.6])
C_ndc = np.array([0.8, 0.0])
D_ndc = np.array([0.0, -0.6])

# 依 NDC 的 Y 向上約定，兩個魚身三角形均為逆時針。
body_tri_1 = (A_ndc, D_ndc, B_ndc)
body_tri_2 = (B_ndc, D_ndc, C_ndc)

uv_A = np.array([0.0, 0.5])
uv_B = np.array([0.5, 1.0])

```

```

uv_C = np.array([1.0, 0.5])
uv_D = np.array([0.5, 0.0])

# 同一三角形內部三個 w 各不相同，才能觀察透視與仿射插值差異。
# 同一幾何位置的 clip w 必須相同；接縫複製屬性亦不改變它。
w_A, w_B, w_C, w_D = 1.0, 2.0, 3.5, 3.0
w1 = (w_A, w_D, w_B)
w2 = (w_B, w_D, w_C)
assert edge_function(A_ndc, D_ndc, B_ndc) > 0
assert edge_function(B_ndc, D_ndc, C_ndc) > 0

H_img, W_img = 200, 200
img_pc = np.zeros((H_img, W_img, 3), dtype=np.uint8)
img_affine = np.zeros((H_img, W_img, 3), dtype=np.uint8)

# checker 用於魚身貼圖與透視／仿射對照；非對稱條紋另作接縫定位測試。
tex_checker = generate_checker_texture(16, 16)
tex_seam = generate_stripe_texture(16, 16)
img_checker_pc = np.zeros((H_img, W_img, 3), dtype=np.uint8)
img_checker_affine = np.zeros((H_img, W_img, 3), dtype=np.uint8)

for tri, uvs, ws in [
    (body_tri_1, (uv_A, uv_D, uv_B), w1),
    (body_tri_2, (uv_B, uv_D, uv_C), w2),
]:
    rasterize_triangle(img_pc, *tri, *uvs, *ws, tex_seam,
                       wrap=True, perspective=True)
    rasterize_triangle(img_affine, *tri, *uvs, *ws, tex_seam,
                       wrap=True, perspective=False)
    rasterize_triangle(img_checker_pc, *tri, *uvs, *ws, tex_checker,
                       wrap=True, perspective=True)
    rasterize_triangle(img_checker_affine, *tri, *uvs, *ws, tex_checker,
                       wrap=True, perspective=False)

# 兩種插值應產生不同影像；背景仍為黑。這些斷言不代替人工檢視。
assert np.any(img_pc != img_affine)
assert np.array_equal(img_pc[0, 0], [0, 0, 0])
assert np.any(img_pc[H_img // 2, W_img // 2] != 0)

# 輸出兩張 PPM 以供比對透視與仿射的差異。
# 本實驗直接將線性 RGB 值乘以 255 量化寫檔，因此檔案是線性 RGB 編碼，
# 僅供資料對照；若需一般顯示，應先做線性 RGB 到 sRGB 的分段轉換。
write_ppm("fish_pc_linear.ppm", img_pc)
write_ppm("fish_affine_linear.ppm", img_affine)
write_ppm("fish_checker_pc_linear.ppm", img_checker_pc)
write_ppm("fish_checker_affine_linear.ppm", img_checker_affine)

# 驗證例題 1 手算結果。
assert np.isclose(u_pc, 9.0/43.0, atol=1e-4), f"PC U mismatch: {u_pc}"
assert np.isclose(v_pc, 4.0/43.0, atol=1e-4), f"PC V mismatch: {v_pc}"

# 驗證週期等價：Repeat 模式下 u=1.0 與 u=0.0 取樣結果相同；
# 雙線性取樣可能混合邊界兩側紋素，並非只取單一紋素。
c_seam_at_1 = sample_bilinear(tex, 1.0, 0.5, wrap=True)
c_seam_at_0 = sample_bilinear(tex, 0.0, 0.5, wrap=True)
assert np.allclose(c_seam_at_1, c_seam_at_0, atol=1e-9), (
    "Repeat 模式下 u=1.0 與 u=0.0 應等價"
)

```

```
# 邊界測試：W=1、H=1 的貼圖在 Repeat 與 Clamp 下皆不應越界。
tiny_tex = np.ones((1, 1, 3), dtype=np.float32) * 0.25
assert np.allclose(sample_bilinear(tiny_tex, 12.3, -4.5,
                                   wrap=True), [0.25, 0.25, 0.25])
assert np.allclose(sample_bilinear(tiny_tex, 12.3, -4.5,
                                   wrap=False), [0.25, 0.25, 0.25])

print("若執行且斷言通過，預期輸出：Tests Passed.")

if __name__ == "__main__":
    main()
```

測試與預期結果

1. 透視插值驗證：

- 程式應輸出 PC UV: (0.2093, 0.0930)。
- Lin UV: (0.3000, 0.2000)。
- 貼圖與取樣規則完全固定，因此 PC 與 Lin 的取樣值可以手算，不是「可能」：
 - 透視： $x = 8 \cdot \frac{9}{43} - 0.5 = \frac{101}{86}$ ； $y = 8(1 - \frac{4}{43}) - 0.5 = \frac{581}{86}$ 。故 $x_0 = 1, y_0 = 6$ ， $f_x = \frac{15}{86}, f_y = \frac{65}{86}$ 。在 8×8 棋盤貼圖中，白色 tap 為 (2, 6) 與 (1, 7)，灰階為 $f_x(1 - f_y) + (1 - f_x)f_y = \frac{15 \cdot 21 + 71 \cdot 65}{86^2} = \frac{4930}{7396} \approx 0.66658$ 。
 - 線性： $x = 1.9, y = 5.9$ ， $f_x = f_y = 0.9$ 。白色 tap 為 (1, 5) 與 (2, 6)，灰階為 $(1 - 0.9)^2 + 0.9^2 = 0.82$ 。
- 因此預期輸出 Color PC ≈ 0.66658 、Color Lin = 0.82。兩者不同，正好說明透視插值與仿射插值確實產生不同的取樣點；若使用同一貼圖卻得到相同結果，表示程式的透視修正實作有誤。
- 以上數值為依公式推得的預期結果；本卷未在特定環境實跑此程式。

2. 接縫 Wrap 測試：

- 在 8×8 對稱棋盤上， $u = 1.0$ 的取樣中心 $x = 1.0 \cdot 8 - 0.5 = 7.5$ ，Repeat 的四個 tap 為 (7, 3), (0, 3), (7, 4), (0, 4)； $u = 0.5$ 的取樣中心 $x = 3.5$ ，四個 tap 為 (3, 3), (4, 3), (3, 4), (4, 4)。兩者皆為兩白兩黑、權重相同，取值皆為 (0.5, 0.5, 0.5)，**對稱棋盤無法區分接縫對錯**。這是先前稿件的錯誤預期，已修正。
- 接縫分支測試使用欄值為 [0, 0.05, 0.15, 0.30, 0.50, 0.65, 0.80, 1] 的非對稱貼圖。寬度 $W = 8$ 時， $u = 1.0$ 的取樣位置為 $x = 7.5$ ，跨第7欄與第0欄，值為 $(1 + 0)/2 = 0.5$ ； $u = 0.5$ 的位置為 $x = 3.5$ ，跨第3欄與第4欄，值為 $(0.30 + 0.50)/2 = 0.40$ ，兩者可區分。此測試檢查**位址與插值分支**，不是外觀無縫；貼圖兩側內容不相容時，Repeat 不能保證接縫平滑。魚身兩三角形的共享邊 UV 相同，用於比較透視與仿射插值，並未刻意建立週期接縫。

3. 上下翻轉測試：

- generate_orientation_texture 建立影像上半紅、下半藍的非對稱貼圖。程式分別斷言 $v = 0.75$ 取到紅色、 $v = 0.25$ 取到藍色；若漏掉影像列號與向上 UV 之間的翻轉，兩項結果會顛倒。灰階垂直條紋不隨影像列號變化，不能單獨承擔此測試。

除錯與常見陷阱

1. V 軸翻轉錯誤：

- 陷阱：忘記翻轉 V 軸，導致貼圖上下顛倒。
- 對策：明確檢查 $y = (1 - v) \cdot H - 0.5$ 。

2. 透視插值未應用：

- 陷阱：直接對 UV 做線性插值，導致貼圖在旋轉時「滑動」。
- 對策：必須使用 w 權重的透視插值公式。

3. 接縫裂開：

- 陷阱：在 UV 0 或 1 的邊界，若兩個相鄰三角形共享 3D 頂點但 UV 不同，會顯示裂縫。
- 對策：接縫兩側建立各自的完整渲染頂點記錄，保留相同幾何位置及其 clip w ，並按需要分別指定 UV 與法線；跨週期邊界的三角形選擇連續 UV 分支（如 $0.9 \rightarrow 1.1$ ）。兩側貼圖內容若不相容，即使 Repeat 位址正確仍可能顯出接縫。

4. 小 w 值問題：

- 陷阱：若某些 w_i 接近零或異號，分母可能抵消或溢出。
- 對策：在光柵化前進行裁剪（Clipping），確保三角形頂點在視錐內且 $w > 0$ （依本書 OpenGL 式投影慣例）。

5. Texel 邊界越界：

- 陷阱： $u = 1.0$ 時 $x = W - 0.5$ ，若直接 floor 會得到 $W - 1$ ，但 x_1 會越界。
- 對策：Clamp 模式使用 Clamp 索引；Repeat 模式對索引取模。

6. PPM 輸出編碼不清：

- 陷阱：把線性 RGB 值直接乘 255 存入 PPM，卻未標註此檔案為線性編碼。一般看圖軟體會以 sRGB 顯示，顏色看起來偏暗，容易誤判為色彩錯誤。
- 對策：本節實驗把線性 RGB 值直接量化儲存，並在程式註解與正文明確標示輸出編碼為線性 RGB，僅供資料對照。若要產生一般顯示影像，應在寫檔前先做線性 RGB 到 sRGB 的分段轉換，兩者不可混用。

7. perspective_uv 檢查不足：

- 陷阱：只驗證 w_i 有限且非零，卻沒檢查 λ_i 是否有限、是否總和為 1，以及分母是否為 NaN 或 inf。若 λ_i 已非有限，或總和明顯偏離 1，即使 w_i 通過檢查，結果依然不可信。
- 對策：在任何除法之前，先檢查 w_i 、 λ_i 、重心和與分母是否為有限；容差依數值精度設定，不得把 epsilon 誤稱為物理安全閾值。

8. 三角形繞序未統一：

- 陷阱：以有向邊函數判定內外時，若同一場景同時存在順時針與逆時針三角形，某些面會被誤判為外部而消失。
- 對策：先計算三角形整體的有向面積 `area2`，再依其符號一致處理內外判定。本節的 `rasterize_triangle` 即以此方式實作；後續若加入背面剔除，應沿用同一符號約定，並與全書「從外向法線看為逆時針」的定義一致。

9. 共享邊的取樣歸屬：

- 陷阱：本節依 `area2` 的符號使用非嚴格的 ≥ 0 或 ≤ 0 邊測試，並未實作 top-left 規則；共享邊上的像素可能被兩側都涵蓋，後繪三角形會覆寫前者。即使幾何位置與 clip w 一致，刻意設置 UV 接縫時兩側取樣也未必相同，不能聲稱重複寫入必然不影響結果。

- **對策**：非接縫共享邊須保持兩側位置、 w 與 UV 一致；刻意的 UV 接縫另檢查貼圖內容是否相容。要處理透明混合、深度競爭或嚴格的共邊覆蓋，須加入深度測試及一致的 top-left 歸屬規則。本節僅作無透明混合的簡化示範。

養殖數位分身案例

在養殖數位分身中，魚體模型使用 UV 貼圖模擬魚鱗與色素。

- **UV 展開**：魚身封閉曲面需設接縫。建模時需確保接縫兩側的幾何位置一致，且 UV 展開避免過度拉伸。
- **貼圖內容**：Albedo 貼圖包含基礎顏色。
- **驗證**：若 UV 插值錯誤，魚在游動時紋路會看起來「游移」，影響視覺真實感與合成標註（如斑塊位置）的準確性。
- **接縫管理**：魚鰭 UV 展開易產生拉伸，需確保接縫處貼圖內容相容，避免過濾時滲色。

習題

1. **手算**：給定 $w_0 = w_1 = w_2 = 1$ ，UV 為 $(0, 0), (1, 0), (0, 1)$ 。NDC 重心 $(0.5, 0.5, 0.0)$ 。計算 UV 值。說明透視與線性插值在此情形下的關係。
2. **程式測試**：使用 `sample_bilinear` 的 Repeat 模式與 $W = 4$ 非對稱條紋貼圖，在 $v = 0.5$ 下比較四組： $(u = 1.0)$ 、 $(u = 0.0)$ 、 $(u = 0.9)$ 、 $(u = 1.1)$ 。(a) 驗證 $u = 1.0$ 與 $u = 0.0$ 取樣值是否相同，說明 Repeat 的週期等價。(b) 說明 $u = 0.9$ 與 $u = 1.1$ 一般取不同值（Repeat 後分別為 0.9 與 0.1），其取樣值與跨接縫連續分支插值的關係為何。(c) 寫出你預期的四組灰階值。
3. **反例/除錯**：若 $w_0 = 1, w_1 = 1, w_2 = -1$ ，且 $\lambda = (0.25, 0.25, 0.5)$ ，計算分母 D 。說明為何這導致錯誤，以及應如何修正。
4. **整合應用**：球體 UV 為 $u = \phi/2\pi, v = \theta/\pi$ 。 $P(\phi, \theta) = (\sin \theta \cos \phi, \cos \theta, \sin \theta \sin \phi)$ 。繞 Y 軸主動旋轉 90° 後，原 $(\phi = 0, \theta = \pi/2)$ 點的 UV 為何？

習題解答

1. **解答**： $w_i = 1$ 時，透視插值退化為線性插值。 $u = 0.5(0) + 0.5(1) + 0(0) = 0.5$ 。 $v = 0.5(0) + 0.5(0) + 0(1) = 0$ 。結果 $(0.5, 0)$ 。
2. **解答**：在 $W = 4$ 的非對稱灰階條紋貼圖（ $x = 0$ 為 0、 $x = 3$ 為 1，中間線性內插）、 $v = 0.5$ 下：
 - $u = 1.0$ ： $x = 1.0 \cdot 4 - 0.5 = 3.5$ 。Repeat 的 tap 為 $x_0 = 3$ 、 $x_1 = 4 \bmod 4 = 0$ ，權重皆 0.5，灰階為 $0.5 \cdot \frac{3}{3} + 0.5 \cdot \frac{0}{3} = 0.5$ 。
 - $u = 0.0$ ： $x = 0 \cdot 4 - 0.5 = -0.5$ 。Repeat 的 tap 為 $x_0 = -1 \bmod 4 = 3$ 、 $x_1 = 0$ ，權重皆 0.5，灰階為 0.5。與 $u = 1.0$ 相同，驗證 Repeat 的週期等價 $1.0 \equiv 0.0$ 。
 - $u = 0.9$ ： $x = 0.9 \cdot 4 - 0.5 = 3.1$ 。 $x_0 = 3$ 、 $x_1 = 4 \bmod 4 = 0$ ， $f_x = 0.1$ 。灰階為 $0.9 \cdot \frac{3}{3} + 0.1 \cdot \frac{0}{3} = 0.9$ 。

◦ $u = 1.1 : x = 1.1 \cdot 4 - 0.5 = 3.9$ ◦ $x_0 = 3$ 、 $x_1 = 0$ ， $f_x = 0.9$ ◦ 灰階為 $0.1 \cdot \frac{3}{3} + 0.9 \cdot \frac{0}{3} = 0.1$ ◦

結論： $u = 1.0$ 與 $u = 0.0$ 取樣相同（皆為 0.5），符合 Repeat 週期等價。 $u = 0.9$ 與 $u = 1.1$ 的取樣一般不同（0.9 對 0.1），這是因為它們在 Repeat 下分別對應 0.9 與 0.1，位於接縫兩側。若三角形跨越接縫，UV 展開必須採連續分支（如 $0.9 \rightarrow 1.1$ ），中點為 $1.0 \equiv 0.0$ ，插值才會通過接縫而非繞過整張貼圖。不可先把每個頂點 UV 個別取小數部分再插值——那樣會把 1.1 變成 0.1，中點就錯算成 0.5，取樣結果完全錯誤。

3. 解答： $D = \frac{0.25}{1} + \frac{0.25}{1} + \frac{0.5}{-1} = 0.25 + 0.25 - 0.5 = 0$ 。分母為零，插值無定義。這表示三角形跨越了 $w = 0$ 平面（視點），必須在齊次裁剪空間先進行裁切，不可直接光柵化。

4. 解答：原點 $P(0, \pi/2) = (1, 0, 0)$ 。繞 Y 軸旋轉 90° （右手系，主動）： $R_y(\pi/2) = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$ 。

$P' = (0, 0, -1)^T$ 。對應 $\sin \theta' \cos \phi' = 0, \cos \theta' = 0, \sin \theta' \sin \phi' = -1$ 。◦ $\theta' = \pi/2$ ◦ $\sin \phi' = -1 \implies \phi' = 3\pi/2$ ◦ $u' = \frac{3\pi/2}{2\pi} = 0.75$ ◦ $v' = \frac{\pi/2}{\pi} = 0.5$ ◦ 結果 (0.75, 0.5)。

本章小結

本章建立了 UV 參數化的數學基礎。重點包括：

1. 紋素中心偏移對取樣精度的影響。
2. 透視正確插值的必要性與推導。
3. 接縫處需複製完整渲染頂點並保持 UV 連續分支。
4. 微分足跡用於估算 Mipmap 等級。後續章節將結合法線貼圖與著色。

參考來源

- [G1] PBRT 4: Transformations - https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] LearnOpenGL: Transformations - <https://learnopengl.com/Getting-started/Transformations>
- [G8] Khronos glTF 2.0 Specification - <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

第15章 局部照明與BRDF基礎

學習目標與先備知識

讀完本章，你將能：

- 區分光源、表面與相機方向，計算Lambert漫反射。
- 解釋餘弦因子、立體角，以及常用輻射度量的單位。
- 以局部照明近似計算球面，並測試背面光源與輸出尺度。
- 說明Phong是經驗性著色模型，並區分它與物理BRDF。

先備知識為向量、內積、矩陣與基本Python。角度以弧度表示；長度用公尺，時間用秒。顏色運算採線性RGB；顯示前才轉為sRGB。本章的場景與光源皆為合成資料，不代表養殖池的實測光照。

問題與直覺

光柵化決定哪些表面覆蓋哪些像素，但不會自行判斷表面看起來有多亮。局部照明模型為每個可見表面點，根據材質、法線、光源方向及觀察方向估算顏色。

以魚體上一個點為例：若光由表面正前方照來，單位面積接收的光較多；若光幾乎沿表面掠過，接收量較少；若光來自表面背面，理想的不透明漫反射面不會直接受光。這個幾何效應由法線與光源方向的內積表達。

然而，「像素亮度」與「光的物理量」不是同一概念。只為了做出高光而調參，可能產生好看的圖像，卻不一定符合能量守恆。理解Lambert模型與BRDF，能幫助我們掌握近似的範圍，以及何時需要更完整的光傳輸模型。

數學與幾何推導

方向與Lambert餘弦

在表面點 p 定義：

- $\mathbf{n}$ ：朝向物體外部的單位表面法線。
- $\mathbf{l}$ ：由表面點指向光源的單位方向。
- $\mathbf{v}$ ：由表面點指向相機的單位方向。
- $\boldsymbol{\omega}_i = \mathbf{l}$ ：由表面點指向入射光來源的方向。
- $\boldsymbol{\omega}_o = \mathbf{v}$ ：離開表面、朝觀察者的方向。

這裡的 $\boldsymbol{\omega}_i$ 是BRDF慣用的「指向光源」方向，不是光線的傳播方向。若以 $\boldsymbol{\omega}_{\text{prop}}$ 表示光線傳播方向，則 $\boldsymbol{\omega}_{\text{prop}} = -\boldsymbol{\omega}_i$ 。

漫反射表面的直接受光量與

$$\max(0, \mathbf{n} \cdot \mathbf{l})$$

成正比。當光源方向與法線夾角為 θ ，且 $0 \leq \theta \leq \pi/2$ ，內積就是 $\cos \theta$ 。斜射的平行光分散到較大的表面面積，因此單位面積接收的能量下降。

令 $E_{\perp}$ 為垂直於入射光方向之平面的照度，單位為 W/m^2 ；令 ρ 為無因次漫反射反射率，對線性RGB可寫成三分量 $\boldsymbol{\rho}$ 。對一束方向固定、其法向照度為 $E_{\perp}$ 的光，表面點接收的照度為

$$E_{\text{surface}} = E_{\perp} \max(0, \mathbf{n} \cdot \mathbf{l}).$$

理想Lambert表面的出射輻亮度為

$$\mathbf{L}_o = \frac{\rho}{\pi} E_{\text{surface}} = \frac{\rho}{\pi} E_{\perp} \max(0, \mathbf{n} \cdot \mathbf{l}).$$

輻亮度的單位為 $\text{W}/(\text{m}^2 \text{ sr})$ ，其中 sr 是球面度。這裡的 $E_{\perp}$ 是已給定的照度，不是零張角方向光的輻亮度。若以入射輻亮度描述有有限立體角範圍的光源，必須對光源所占的方向範圍積分，不能單憑某一方向的輻亮度直接當作照度。

圖學程式常使用簡化的線性RGB模型：

$$\mathbf{C}_{\text{diffuse}} = \boldsymbol{\rho} \odot \mathbf{C}_L \max(0, \mathbf{n} \cdot \mathbf{l}).$$

這是便於著色的顏色公式； $\mathbf{C}_L$ 是否代表已換算的照度，或只是藝術控制值，必須由應用自行約定。除非校準了光源、材質與相機，不能把這種RGB值宣稱為實際輻射功率或相機量測值。

立體角與BRDF

平面角描述二維方向差；立體角描述從一點看出去的三維方向範圍，單位為球面度 sr 。整個球面的立體角是 $4\pi \text{ sr}$ ，朝外的半球是 $2\pi \text{ sr}$ 。對面積 dA ，若其法線與觀察方向夾角為 θ ，距離為 r ，它張開的微小立體角為

$$d\omega = \frac{\cos \theta dA}{r^2}.$$

立體角使我們能把來自不同方向的光加總。令 Ω^+ 為表面外側半球，並令 $\boldsymbol{\omega}_i$ 指向光源。反射方程為

$$\mathbf{L}_o(p, \boldsymbol{\omega}_o) = \int_{\Omega^+} f_r(p, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \mathbf{L}_i(p, \boldsymbol{\omega}_i) \max(0, \mathbf{n} \cdot \boldsymbol{\omega}_i) d\boldsymbol{\omega}_i.$$

f_r 是雙向反射分布函數（BRDF），描述入射輻亮度如何轉成指定觀察方向的出射輻亮度。由於BRDF乘上入射輻亮度與餘弦項，再對立體角積分，BRDF的單位為 sr^{-1} 。外側入射方向滿足 $\mathbf{n} \cdot \boldsymbol{\omega}_i > 0$ 。

Lambert BRDF不依賴入射或觀察方向：

$$f_r = \frac{\rho}{\pi}.$$

其能量尺度可由半球積分看出。以法線為極軸，半球方向的立體角微元為 $d\omega = \sin\theta \, d\theta \, d\phi$ ，其中 $0 \leq \theta \leq \pi/2$ 、 $0 \leq \phi < 2\pi$ 。因此

$$\int_{\Omega^+} \cos\theta \, d\omega = \int_0^{2\pi} \int_0^{\pi/2} \cos\theta \sin\theta \, d\theta \, d\phi = \pi.$$

將 $f_r = \rho/\pi$ 代入後，漫反射總反射比例為 ρ ，不會超過入射能量乘以反照率。真實魚皮可能同時呈現散射、鏡面反射、濕潤表面反光及多尺度細節；單一Lambert項通常只適合簡化的漫反射近似。

Phong與物理BRDF的差別

經典Phong模型以反射方向 $\mathbf{r}$ 和觀察方向 $\mathbf{v}$ 的對齊程度塑造高光：

對不透明單面表面，背面光源不應產生正面 Phong 高光，因此模型應明確加入入射半球條件：

$$I_{\text{Phong}} = \begin{cases} k_d(\mathbf{n} \cdot \mathbf{l}) + k_s \max(0, \mathbf{r} \cdot \mathbf{v})^s, & \mathbf{n} \cdot \mathbf{l} > 0, \\ 0, & \mathbf{n} \cdot \mathbf{l} \leq 0. \end{cases}$$

k_d, k_s 是調色用係數， s 控制高光集中程度。依本章方向慣例，入射光的傳播方向是 $-\mathbf{l}$ ，其鏡面反射方向為

$$\mathbf{r} = -\mathbf{l} - 2\mathbf{n}(\mathbf{n} \cdot (-\mathbf{l})).$$

Phong容易實作、便於藝術調整，但常見形式未必符合能量守恆，也不保證不同光源強度、材質參數與觀察角度之間一致。規範化的Phong變體可以改善尺度，但仍是經驗性模型。物理微表面模型試圖描述許多微小鏡面構成的表面，並以法線分布、遮蔽項與Fresnel效應建模；其參數較有物理意義，但仍依賴模型假設與參數映射。本章只建立BRDF的基本概念，不把任一著色公式等同於完整真實光學。

逐步手算例題

例一：傾斜表面的Lambert明暗

設表面法線與光源方向皆為單位向量：

$$\mathbf{n} = (0, 1, 0), \quad \mathbf{l} = (0, 0.6, 0.8).$$

令 $\rho = 0.7$ ，光源在垂直於入射方向的平面上產生照度 $E_{\perp} = 10 \text{ W/m}^2$ 。計算出射輻亮度：

1. 先算內積：

$$\mathbf{n} \cdot \mathbf{l} = 0.6.$$

2. 表面照度為

$$E_{\text{surface}} = E_{\perp} \times 0.6 = 6 \text{ W/m}^2.$$

3. 出射輻亮度為

$$L_o = \frac{0.7}{\pi} \times 6 = \frac{4.2}{\pi} \approx 1.337 \text{ W/(m}^2 \text{ sr)}.$$

若光源改成 $\mathbf{l} = (0, -0.6, 0.8)$ ，則 $\mathbf{n} \cdot \mathbf{l} = -0.6$ ，表面不接收這束光的直接照明，Lambert直接出射項為零。環境光或背面補光須另外建模。

例二：依球面法線計算三個點

考慮單位球面上三個外向法線：

$$\mathbf{n}_1 = (0, 1, 0), \quad \mathbf{n}_2 = \left(\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}, 0 \right), \quad \mathbf{n}_3 = (0, -1, 0).$$

令平行光方向為 $\mathbf{l} = (0, 1, 0)$ ，線性光色為 $\mathbf{C}_L = (2, 1, 0.5)$ ，漫反射反照率為 $\boldsymbol{\rho} = (0.5, 0.25, 0.8)$ 。使用簡化RGB模型：

$$\mathbf{C} = \boldsymbol{\rho} \odot \mathbf{C}_L \max(0, \mathbf{n} \cdot \mathbf{l}).$$

先計算 $\boldsymbol{\rho} \odot \mathbf{C}_L = (1, 0.25, 0.4)$ 。

- 對 $\mathbf{n}_1$ ，點積為 1，故 $\mathbf{C}_1 = (1, 0.25, 0.4)$ 。
- 對 $\mathbf{n}_2$ ，點積為 $\sqrt{2}/2 \approx 0.7071$ ，故 $\mathbf{C}_2 \approx (0.7071, 0.1768, 0.2828)$ 。
- 對 $\mathbf{n}_3$ ，點積為 -1 ，截成 0，故直接受光 $\mathbf{C}_3 = (0, 0, 0)$ 。

這些是線性RGB，不是顯示器上的sRGB碼值。若輸出時直接把大於1的分量截斷為1，會損失高光或強光資訊。本程式只為了產生示意圖而限制輸出範圍，並以數值測試檢查原始線性計算。

實作與程式

以下程式以純Python建立 128×128 PPM影像。每個像素沿相機射線與單位球求交；球面點的外法線為該點的單位向量。可見部分以正面最近交點為準。光源是遠處平行光，使用簡化Lambert線性RGB，再限制至PPM可表示的範圍。

PPM的P3格式使用文字列出整數RGB，每個通道範圍為0至255。本程式直接以線性值量化，不作sRGB轉換，因此適合檢查幾何與Lambert計算，不是色彩管理正確的顯示輸出。一般顯示器使用的sRGB編碼需由線性RGB逐通道做分段轉換；本程式沒有執行這項轉換。

```

import math

W, H = 128, 128
LIGHT = (0.35, 0.80, 0.48) # 程式稍後正規化
ALBEDO = (0.70, 0.32, 0.12)
AMBIENT = 0.08 # 無因次的簡化補光係數

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def normalize(v):
    length = math.sqrt(dot(v, v))
    if length == 0.0:
        raise ValueError("不可正規化零向量")
    return tuple(x / length for x in v)

def ray_sphere(origin, direction):
    # 單位球: |origin + t * direction|^2 = 1
    a = dot(direction, direction)
    b = 2.0 * dot(origin, direction)
    c = dot(origin, origin) - 1.0
    disc = b * b - 4.0 * a * c
    if disc < 0.0:
        return None
    root = math.sqrt(disc)
    t0 = (-b - root) / (2.0 * a)
    t1 = (-b + root) / (2.0 * a)
    positive = [t for t in (t0, t1) if t > 0.0]
    return min(positive) if positive else None

def lambert(albedo, normal, light, ambient=0.0):
    ndotl = max(0.0, dot(normal, light))
    return tuple(
        c * (ambient + ndotl) for c in albedo
    )

def to_byte(x):
    # 示意輸出: 夾至 [0, 1] 後直接量化, 不做sRGB轉換
    return max(0, min(255, round(255 * x)))

light = normalize(LIGHT)
pixels = []

for y in range(H):
    for x in range(W):
        # 像素原點左上, 中心取樣; 相機看向 -Z。
        sx = 2.0 * (x + 0.5) / W - 1.0
        sy = 1.0 - 2.0 * (y + 0.5) / H
        origin = (0.0, 0.0, 3.0)
        direction = normalize((sx, sy, -2.0))

        t = ray_sphere(origin, direction)

```

```

if t is None:
    color = (0.025, 0.045, 0.07)
else:
    point = tuple(origin[i] + t * direction[i] for i in range(3))
    normal = normalize(point)
    color = lambert(ALBEDO, normal, light, AMBIENT)

pixels.append(tuple(to_byte(c) for c in color))

with open("lambert_sphere_linear.ppm", "w", encoding="ascii") as f:
    f.write(f"P3\n{W} {H}\n255\n")
    for i in range(0, len(pixels), W):
        row = pixels[i:i + W]
        f.write(" ".join(f"{r} {g} {b}" for r, g, b in row) + "\n")

# 不依賴輸出影像的數值測試
n = (0.0, 1.0, 0.0)
assert lambert((1.0, 1.0, 1.0), n, (0.0, 1.0, 0.0)) == (1.0, 1.0, 1.0)
assert lambert((1.0, 1.0, 1.0), n, (0.0, -1.0, 0.0)) == (0.0, 0.0, 0.0)
assert ray_sphere((0.0, 0.0, 3.0), (0.0, 0.0, -1.0)) == 2.0
assert ray_sphere((0.0, 0.0, 3.0), (0.0, 1.0, 0.0)) is None

```

測試與預期結果

執行程式後，預期在目前目錄建立 `lambert_sphere_linear.ppm`。這是依程式公式推得的預期行為，並非作者已執行檢查的結果。

程式中的斷言提供四項可重現測試：

1. ****正面光源：****法線與光源方向相同時，白色反照率輸出為 (1, 1, 1)。
2. ****背面光源：****法線與光源方向相反時，直接漫反射項為零。此斷言沒有使用成圖所加的 `AMBIENT`。
3. ****球面正中射線：****從 (0, 0, 3) 沿 $-Z$ 前進，第一次碰到單位球的參數 $t = 2$ 。
4. ****不相交射線：****從同一起點沿 $+Y$ 前進，不與單位球相交。

可再加測斜射例：正規化 (0, 1, 1) 後，將它與 (0, 1, 0) 內積，結果應為 $1/\sqrt{2}$ 。光源輸入若非單位向量，內積大小也會隨其長度縮放，所以方向在進入著色計算前須正規化。

除錯與常見陷阱

- **法線或光源方向未正規化。** 若向量長度不是1，內積會混入長度因子。正規化方向，並檢查零向量。
- **混淆光線傳播方向與指向光源的方向。** 本章 `l` 由表面指向光源；光線傳播方向是 $-l$ 。採用不同慣例時必須一致。
- **沒有截除背面光。** 直接Lambert項應使用 $\max(0, \mathbf{n} \cdot \mathbf{l})$ ，不應讓負值形成負亮度。
- **把環境補光當成Lambert直接照明。** 程式的 `AMBIENT` 是避免背光側全黑的簡化項，不含方向積分，也不是量測到的環境輻亮度。

- **將線性RGB直接誤認為顯示輸出。** 線性RGB適合做光照運算；sRGB是非線性編碼。計算前不應把sRGB碼值直接當成線性值相乘。
- **把經驗模型當成能量守恆保證。** Phong高光可能超出合理能量尺度。使用物理BRDF時，需同時考慮BRDF單位、餘弦因子、光源定義與顏色空間。

養殖數位分身案例

為合成養殖池中的魚體指定線性反照率 ρ ，並讓每個可見表面點帶有外向法線 $\mathbf{n}$ 。對遠處面積光源，可以用一個或多個方向樣本近似；對單一方向光，按 $\mathbf{n} \cdot \mathbf{l}$ 計算漫反射。池底、水面與魚身應分別使用材質參數，避免以同一個反照率代表不同表面。

例如將魚身簡化為偏紅褐的漫反射材質，把光源設定為合成白光，先檢查朝光側是否隨法線角度平滑變暗，再單獨加入水面反光或環境光項。若以Phong項模擬魚身光澤，必須把它標為外觀近似；不能由高光推論魚皮的真實粗糙度、濕度或水下輻亮度。

這個模型不包含水體吸收與散射、遮蔽、折射、體積光傳輸或相機響應。若用來產生訓練影像，資料紀錄應保存合成光源、材質與模型設定，並將合成結果與實測影像分開標示。

習題

習題1：手算Lambert

表面法線為 $\mathbf{n} = (0, 1, 0)$ ，光源方向為單位向量 $\mathbf{l} = (0, 0.8, 0.6)$ ，反照率 $\rho = 0.5$ 。光源在垂直於入射方向的平面上產生照度 $E_{\perp} = 4 \text{ W/m}^2$ 。求出射輻亮度。若光源方向改成 $(0, -0.8, 0.6)$ ，直接Lambert出射輻亮度為何？

習題2：程式測試

沿用程式中的 `normalize` 和 `lambert`，寫出最小測試驗證白色反照率、垂直入射時的輸出；再測試傾斜 $\pi/3$ 的單位方向，使輸出係數為 $1/2$ 。說明為什麼不能將斜向量 $(0, 0.5, 0.5)$ 直接當作單位方向。

習題3：反例與除錯

某段著色程式直接計算 `ndotl = dot(n, l)`，不檢查正負號，且把 `l` 設為從光源指向表面的傳播方向。指出兩個問題，並用 $\mathbf{n} = (0, 1, 0)$ 、傳播方向 $(0, -1, 0)$ 算出錯誤與正確的受光係數。

習題4：整合應用

合成魚體表面點的單位法線為 $\mathbf{n} = (0, 1, 0)$ ，光源方向為 $\mathbf{l} = (0, 1, 0)$ 。線性反照率為 $(0.4, 0.2, 0.1)$ ，光源RGB為 $(1.5, 1.0, 0.5)$ 。使用本章簡化RGB模型計算該點顏色，再計算光源方向改成 $(0, -1, 0)$ 時的直接漫反射。此結果可否直接當成sRGB影像值或真實魚皮反射量？說明理由。

習題解答

解答1

正面情況的餘弦為

$$\mathbf{n} \cdot \mathbf{l} = 0.8.$$

表面照度為 $4 \times 0.8 = 3.2 \text{ W/m}^2$ ，因此

$$L_o = \frac{0.5}{\pi} \times 3.2 = \frac{1.6}{\pi} \approx 0.5093 \text{ W/(m}^2 \text{ sr)}.$$

反向情況的內積是 -0.8 ，截為零，直接出射輻亮度為 0。

解答2

可在程式末尾加入：

```
import math

white = (1.0, 1.0, 1.0)
n = (0.0, 1.0, 0.0)

assert lambert(white, n, (0.0, 1.0, 0.0)) == white
l = normalize((0.0, 0.5, math.sqrt(3.0) / 2.0))
result = lambert(white, n, l)
assert abs(result[0] - 0.5) < 1e-12
```

這個單位方向的 Y 分量為 $1/2$ ，所以與 $\mathbf{n}$ 的點積為 $1/2$ ；它與法線的夾角為 $\pi/3$ 。 $(0, 0.5, 0.5)$ 的長度是 $\sqrt{0.5}$ ，不是 1；未正規化時，點積會受方向向量長度影響，不能單獨解讀為餘弦。

解答3

第一個問題是沒有把負的餘弦截成零；第二個問題是方向慣例相反。已知傳播方向 $\boldsymbol{\omega}_{\text{prop}} = (0, -1, 0)$ ，由表面指向光源的方向應為 $\mathbf{l} = -\boldsymbol{\omega}_{\text{prop}} = (0, 1, 0)$ 。

若錯把傳播方向當作 $\mathbf{l}$ ，點積為 -1 ，未截斷會得到負受光係數。正確計算的點積為 1，受光係數為 1。即使方向慣例正確，仍應對點積取 $\max(0, \cdot)$ ，讓背面直接照明為零。

解答4

兩個方向相同時，點積為 1，因此

$$\mathbf{C} = (0.4, 0.2, 0.1) \odot (1.5, 1.0, 0.5) = (0.6, 0.2, 0.05).$$

光源反向時，點積為 -1 ，截為零，直接漫反射為 $(0, 0, 0)$ 。

計算結果是此簡化模型下的線性RGB，不是sRGB碼值；輸出前需要做適當的色彩轉換。它也不是實測魚皮反射量，因為光源量的物理定義、相機響應、表面其他反射成分及水體傳輸都未建模。

本章小結

Lambert模型以 $\max(0, \mathbf{n} \cdot \mathbf{l})$ 表示表面朝向對直接照明的影響；理想漫反射BRDF為 ρ/π 。半球上的餘弦加權立體角積分為 π ，使Lambert反射的總能量比例等於反照率。BRDF描述入射光如何反射至特定觀察方向；Phong提供容易調整的經驗性高光，但不等同於能量守恆的物理模型。實作時要一致定義方向、正規化向量、截除背面照明，並區分線性RGB、sRGB與實際物理量。

參考來源

下列為延伸閱讀；章內推導與程式以本章定義的慣例為準。引用來源不表示本章每項敘述已逐條外部查核。

- G2, *Physically Based Rendering: From Theory to Implementation, 4th Edition*, Reflection Models : https://pbr-book.org/4ed/Reflection_Models
- G3, *Physically Based Rendering: From Theory to Implementation, 4th Edition*, The Light Transport Equation : https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- G4, *Ray Tracing in One Weekend* : <https://raytracing.github.io/books/RayTracingInOneWeekend.html>

第16章 物理材質與微表面模型

學習目標與先備知識

本章把局部照明推向可比較的物理材質。讀者應先掌握第14章的 UV 與貼圖座標，以及第15章的 Lambert 模型、立體角與 BRDF 基礎；也需具備第一卷的向量內積、外積與基本微積分概念。結束時，你應該能：

- 分辨反照率 (albedo)、粗糙度 (roughness)、金屬度 (metallic) 三者 in BRDF 中扮演的角色，並說明它們不是紋理貼圖「畫得好不好看」的形容詞。
- 寫出 Cook-Torrance 形式的微表面 BRDF：由 Fresnel 項 F 、法線分佈函數 D 、幾何遮蔽項 G 組成，並知道分母 $4(N \cdot V)(N \cdot L)$ 的來源。
- 透過 Jacobian 關係把微面法線的立體角轉換到巨觀入射與出射立體角，理解乘積中各因子的來處。
- 在給定粗糙度映射下，手算簡化 BRDF 的數值，包含掠射角的 Fresnel 極限。
- 用 NumPy 實作一個完整、可獨立執行的 BRDF 計算與測試腳本。
- 判讀粗糙度貼圖、金屬度貼圖與反照率貼圖的儲存語意與 sRGB 差異。

本章使用的單位：長度公尺、角度弧度。所有 BRDF 值單位為 sr^{-1} 。輻射度量沿用第15章橋接：輻射亮度 L 、入射輻照度 E 、反射輻射亮度滿足

$$L_o(\omega_o) = \int_{\Omega^+} f_r(\omega_i, \omega_o) L_i(\omega_i) (N \cdot \omega_i) d\omega_i.$$

N 為單位化的最終著色法線；在使用法線貼圖時，它是依第17章的 TBN 將切線空間法線轉至世界空間並重新正規化後的結果。 ω_i 為由表面指向光源的單位方向， ω_o 為由表面指向觀察者的單位方向。另須保留幾何法線 N_g 作面向與可見性判定，不以貼圖擾動法線取代幾何判定；若 $N \cdot N_g \leq 0$ ，應拒絕或修正該法線貼圖結果，或採用明確的著色法線修正策略。

問題與直覺

第15章的 Lambert 材質只有一個主要參數：反照率。它的 BRDF 是常數 $f_r = \rho/\pi$ ，與觀察角度無關。真實魚鱗在背光下只閃一小圈高光，在掠射角幾乎像鏡子；同一片材質只要改變觀察幾何，反射強度、顏色與高光寬度都會變。Lambert 無法表示任何一個這類現象。

物理材質模型的核心想法是：**把表面視為大量朝向不同方向的微小鏡面拼成的統計整體**。每個微面本身遵守鏡面反射與 Fresnel，但因為朝向隨機，觀察者看到的是「有多少微面恰好把光從 ω_i 反到 ω_o 」的期望值。因此 BRDF 不是逐面計算，而是對微面朝向的統計分佈、遮蔽機率、Fresnel 加權後求總和。粗糙度控制微面朝向的分散，金屬度控制反照率是否作為鏡面反射色，反照率則在非金屬時控制漫反射基底。

直覺上：粗糙度趨近 0 時，所有微面幾乎同向， D 近似狄拉克分佈，表面變成鏡面。粗糙度變大時， D 抹平成較寬的瓣狀，高光變大變弱。金屬度從 0 到 1 時，漫反射項逐步消失，反射色由 F_0 決定。

值得先講清楚的是三個量**心理感知與物理意義的分離**。感知上的「亮/暗」「光滑/粗糙」是視覺系統把高光形狀、亮度與位置壓縮成單維判斷的結果；物理上它們分別由 F_0 、 α 、 k_d 三個獨立量決定。這意味著兩顆材質球若感知一樣「亮」，也可能有截然不同的 F_0 與粗糙度。渲染參數旋轉到 0.5 未必對應「感知中等」的粗糙表面，不同引擎的 $\alpha(r)$ 映射決定了這層差異。當我們用數位分身比對合成影像與實拍攝影時，**必須把「感知近似」與「參數等同」分開**，否則調參會變成無錨點的視覺遊戲。

數學與幾何推導

微表面 BRDF 的組成

Cook-Torrance 形式下，將表面分解為微面集合，對可見入射與出射微面求和可導出：

$$f_r(\omega_i, \omega_o) = \underbrace{\frac{k_d \rho_d}{\pi}}_{\text{漫反射}} + \underbrace{\frac{F(\omega_i, h) D(h) G(\omega_i, \omega_o)}{4 (N \cdot \omega_i)(N \cdot \omega_o)}}_{\text{鏡面}}$$

其中 $h = \text{normalize}(\omega_i + \omega_o)$ 為半角向量， h 對應「能把 ω_i 反到 ω_o 」的微面法線方向。

Jacobian 的來源

分母 $4(N \cdot \omega_i)(N \cdot \omega_o)$ 來自半向量映射 Jacobian、微面投影面積與巨觀入射／出射投影餘弦的合併，不能視為單一立體角 Jacobian。推導分三步：

1. 微面鏡面反射使 ω_i 與 ω_o 關於 h 對稱： $\omega_o = 2(\omega_i \cdot h)h - \omega_i$ 。固定入射方向時，半角向量與反射方向的立體角關係為

$$d\omega_h = \frac{d\omega_o}{4|N \cdot \omega_o|}.$$

$D(h)$ 描述微面法線相對巨觀表面的分布；微面投影、可見性與遮蔽則由推導中的其他因子處理。將微面分布、Fresnel、幾何項與反射方向的 Jacobian 合併，可得常用鏡面微表面 BRDF：

$$f_r^{\text{spec}}(\omega_i, \omega_o) = \frac{F(\omega_i, h) D(h) G(\omega_i, \omega_o)}{4|N \cdot \omega_i||N \cdot \omega_o|}.$$

這是推導綱要；完整結果取決於 D 與 G 的定義，不能由單一立體角等式直接推出。外側半球的巨觀點積為正時，分母中的絕對值可省略。

在 BRDF 推導裡，對「可見微面」求期望時， $N \cdot h$ 會被微面投影項消去一部分，最終留下的就是 $1/[4(N \cdot \omega_i)(N \cdot \omega_o)]$ 前的係數。要點在於：微面朝向的分布 $D(h)$ 映射到巨觀反射方向時，面積元素與投影因子都會改變；結合半向量映射的 Jacobian、微面投影面積與巨觀方向的投影餘弦後，才得到標準分母

$4(N \cdot \omega_i)(N \cdot \omega_o)$ 。若要具體驗證這組幾何與變數轉換因子的完整性，可以想像 $D(h)$ 收斂為狄拉克分佈 ($\alpha \rightarrow 0$) 時，鏡面項應退化為「 ω_o 是 ω_i 對 N 的正反射方向才非零」的鏡面反射機率密度；少掉 Jacobian，這個極限不會成立。

Fresnel 項

Schlick 近似：

$$F(\cos \theta) = F_0 + (1 - F_0)(1 - \cos \theta)^5, \quad \cos \theta = \max(0, \omega_i \cdot h).$$

F_0 為正向入射反射率。本章以 $F_0 \approx 0.04$ 作為常見介電質的簡化假設；它不是人眼經驗值，實際數值取決於材料的光學性質。對金屬，本章的金屬度工作流程以線性 RGB 基底色近似有色 F_0 。兩者按金屬度 m 插值：

$$F_0 = 0.04(1 - m) + \rho m.$$

掠射極限 $\cos \theta \rightarrow 0$ 時 $F \rightarrow 1$ ，這就是金屬與非金屬在邊緣都發亮的物理來源，也是掠射限制的第一個含義：**任何 Cook-Torrance 實作若在掠射角仍取 $F \approx F_0$ ，就抹掉了真實材質最明顯的邊緣特徵**。第二個含義是配合 D 的衰減：即使 F 隨掠射升高，若 D 塌陷得比 F 上升快，鏡面 BRDF 仍會下降（見習題16.2）。這兩層含義分別是「單點的 Fresnel 行為」與「整段 BRDF 的合成行為」，不要混為一談。

法線分佈函數 D

本章用 GGX (Trowbridge-Reitz)：

$$D(h) = \frac{\alpha^2}{\pi ((n \cdot h)^2(\alpha^2 - 1) + 1)^2}, \quad \alpha = \text{roughness}^2.$$

注意 α 是粗糙度的平方，不是粗糙度本身。這種 $\alpha = r^2$ 映射是實務常見約定，目的讓知覺上「線性」的滑桿對應更合理的高光寬度變化。不同引擎的映射不同，讀者若載入外部資產，必須明說映射，否則兩份「roughness=0.5」的材質不會長得一樣。GGX 在 $n \cdot h \rightarrow 0$ 的尾端比 Beckmann 更厚，這是粗糙表面「長尾高光」的來源；對掠射觀察尤其明顯：Beckmann 的高光會急遽收止，GGX 則留下明顯的拖尾。

幾何項 G

使用 Smith 形式可分解為 $G \approx G_1(\omega_i) G_1(\omega_o)$ 。以下實作用常見的 Smith-GGX 單項：

$$G_1(x) = \frac{2(n \cdot x)}{(n \cdot x) + \sqrt{\alpha^2 + (1 - \alpha^2)(n \cdot x)^2}}.$$

上式雖非唯一形式，但足以在本章演示能量趨勢，且能和 GGX 搭配保持白色爐近似。 $n \cdot x = 1$ 給 $G_1 = 1$ （垂直入射無遮蔽）， $n \cdot x \rightarrow 0$ 給 $G_1 \rightarrow 0$ （掠射方向幾乎全被相鄰微面擋住）。

漫反射權重與能量

對於非金屬， $\rho_d = \rho$ ， $k_d = (1 - F)(1 - m)$ 。加入 $(1 - F)$ 是為了不讓反射進鏡面項的能量同時算到漫反射，這個修正雖非嚴格能量守恆，但比忽略好。對於金屬 $m = 1$ ， $k_d = 0$ ，只剩鏡面項。

觀察方向 ω_o 固定時，漫反射與鏡面項的半球積分合計應符合被動材質的能量限制；該總反射率不等同於漫反射基底色 ρ 。**白色爐測試**是檢查此限制的合成實驗：假設入射輻亮度在各方向都等於 L ，被動材質的出射輻亮度不應穩定超過 L 。對設定為白色且無吸收的完整模型，結果才應接近 L ；本章的單次散射近似可能因未補償多重散射而偏低。多重散射補償（如 Kulla-Conty）可用額外項補回部分損失。本章不展開該推導，也不指定普遍適用的固定偏低比例；結果會依材質參數、觀察方向與漫反射耦合方式而變。

逐步手算例題

例題 16.1 GGX D 峰值與尾端

取 roughness $r = 0.5$ ， $\alpha = r^2 = 0.25$ ， $\alpha^2 = 0.0625$ 。

(1) 正向： $n \cdot h = 1$ 。

$$D = \frac{0.0625}{\pi (1 \cdot (-0.9375) + 1)^2} = \frac{0.0625}{\pi \cdot (0.0625)^2} = \frac{0.0625}{0.012272} \approx 5.093.$$

(2)

$$(n \cdot h)^2 = 0.64, \quad 0.64 \cdot (-0.9375) + 1 = 0.400, \quad D = \frac{0.0625}{\pi \cdot 0.16} \approx 0.124.$$

(3)

$$0.25 \cdot (-0.9375) + 1 = 0.7656, \quad D = \frac{0.0625}{\pi \cdot 0.5862} \approx 0.0339.$$

檢核： D 從 ≈ 5.09 衰減到 ≈ 0.124 到 ≈ 0.034 ，遠比高斯分布更長尾。這是 GGX 相對於 Beckmann 的關鍵差異：粗糙表面在掠射時的高光會拖長而非急速收斂。

例題 16.2 完整簡化 BRDF 一次計算

假設 $N = (0, 0, 1)$ 、 $V = N = (0, 0, 1)$ 、 $L = (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2})$ 。則 $H = \text{normalize}(V + L)$ ：

$$V + L = (0.7071, 0, 1.7071), \quad |V + L| \approx 1.8478, \quad H \approx (0.3827, 0, 0.9239).$$

內積： $N \cdot L = 0.7071$ 、 $N \cdot V = 1$ 、 $N \cdot H = 0.9239$ 、 $V \cdot H = 0.9239$ 。

取 roughness $r = 0.5$ 、 $\rho = 0.5$ 、metallic $m = 0$ 。

$$F_0 = 0.04。$$

$$F = 0.04 + 0.96(1 - 0.9239)^5 = 0.04 + 0.96 \cdot (0.0761)^5 \approx 0.04 + 0.96 \cdot 2.55 \times 10^{-6} \approx 0.0400.$$

$$D: \alpha = 0.25, \alpha^2 = 0.0625, (n \cdot h)^2 = 0.8536, 0.8536 \cdot (-0.9375) + 1 = 0.1998, \text{ 故}$$

$$D = \frac{0.0625}{\pi \cdot 0.03993} \approx 0.4983.$$

$G_1(V): n \cdot V = 1$ ，帶入 Smith-GGX 公式得 $G_1(V) = \frac{2 \cdot 1}{1 + \sqrt{0.0625 + 0.9375 \cdot 1}} = \frac{2}{2} = 1$ 。這是邊界情況： $n \cdot x = 1$ 時分子與分母都退化成 2，並非公式失效，而是正好給出無遮蔽的上限值 1。

$n \cdot L = 0.7071$ 時分子 $2 \times 0.7071 = 1.4142$ ，根號內 $= 0.0625 + 0.9375 \cdot 0.5 = 0.53125$ ，開根 0.7289，分母 1.4360， $G_1(L) \approx 0.9848$ 。故 $G \approx 0.9848$ 。

鏡面項：

$$\frac{F D G}{4(N \cdot V)(N \cdot L)} = \frac{0.0400 \cdot 0.4983 \cdot 0.9848}{4 \cdot 1 \cdot 0.7071} \approx \frac{0.01962}{2.8284} \approx 0.00694.$$

漫反射項：

$$\frac{k_d \rho}{\pi}, \quad k_d = (1 - F)(1 - m) \approx 0.96, \quad \frac{0.96 \cdot 0.5}{\pi} \approx 0.1528.$$

合計 $f_r \approx 0.1528 + 0.0069 \approx 0.1597 \text{ sr}^{-1}$ 。若入射光束沿本例方向 L ，且其垂直於光束的平面所接收的輻照度為 $E_{\perp} = 2 \text{ W/m}^2$ ，則表面接收此光束的輻照度為 $E_{\text{surf}} = E_{\perp}(N \cdot L) = 2 \times 0.7071 \approx 1.414 \text{ W/m}^2$ 。對此單一方向的光束，以本例方向組合求得的 BRDF 計算，出射輻亮度約為 $L_o = f_r(L, V)E_{\text{surf}} \approx 0.1597 \times 1.414 \approx 0.226 \text{ W/(m}^2 \text{ sr)}$ 。若 E 指多個方向合計的表面輻照度，則不能直接乘單一方向的 $f_r(L, V)$ ，須依各入射方向積分。

例題 16.3 掠射角的 Fresnel 上升與 D 塌陷

取 $V = N$ 、 L 在 xz 平面與 N 夾 85° 、 $r = 0.5$ 、 $\rho = 0.5$ 、 $m = 0$ 。

$$L = (\sin 85^\circ, 0, \cos 85^\circ) \approx (0.9962, 0, 0.0872), V + L = (0.9962, 0, 1.0872), |V + L| \approx 1.4746, \\ H \approx (0.6756, 0, 0.7373)。$$

$$N \cdot H = 0.7373, (N \cdot H)^2 = 0.5436。D \text{ 分母} = 0.5436 \cdot (-0.9375) + 1 = 0.4904, \text{ 故}$$

$$D = \frac{0.0625}{\pi \cdot 0.4904^2} \approx 0.0827.$$

$$F: V \cdot H = 0.7373,$$

$$F = 0.04 + 0.96(1 - 0.7373)^5 \approx 0.04 + 0.96 \cdot 1.252 \times 10^{-3} \approx 0.0412.$$

$G_1(V) = 1$ ， $G_1(L)$ 帶入 $n \cdot L = 0.0872$ 得 $G_1(L) \approx 0.4966$ ， $G \approx 0.4966$ 。

$$\text{spec} = \frac{0.0412 \cdot 0.0827 \cdot 0.4966}{4 \cdot 1 \cdot 0.0872} \approx \frac{0.00169}{0.3488} \approx 0.00486.$$

與例題16.2的0.00694相比，掠射角下 D 從0.498降到0.083、 G 從0.985降到0.497，而 F 只從0.0400升到0.0412。Fresnel尾端上升在此被 D 與 G 的下降壓過去，鏡面BRDF反而降低。在本例固定 $V = N$ 、只把 L 推向掠射角的路徑中，降低粗糙度會使 D 的瓣更窄，因此離開正反射方向後下降得更快。一般所謂低粗糙度表面的邊緣Fresnel高光，需要觀察方向、光源方向與正反射幾何仍互相對齊；不能由本例的固定 $V = N$ 掃描直接推出。

例題 16.4 粗糙度對高光角寬的影響

固定 $V = N$ 、 $\rho = 0.5$ 、 $m = 0$ ，令入射 L 掃過與 N 的夾角 θ_L 。 D 的峰值位置在 $n \cdot h = 1$ ，即 $\theta_L = 0$ ；當 L 偏移到 θ_L ， D 下降的角寬約與 α 成正比。三個粗糙度比較，取 $\theta_L = 15^\circ$ 作為對照點：

$L_{15^\circ} \approx (0.2588, 0, 0.9659)$ ， $V + L \approx (0.2588, 0, 1.9659)$ ， $|V + L| \approx 1.9830$ ， $H \approx (0.1305, 0, 0.9914)$ ， $N \cdot H \approx 0.9914$ ， $(N \cdot H)^2 \approx 0.9829$ 。

- $r = 0.2$ ： $\alpha = 0.04$ ， $\alpha^2 = 0.0016$ ，分母 $= 0.9829 \cdot (-0.9984) + 1 = 0.01873$ ， $D = 0.0016 / (\pi \cdot 0.01873^2) \approx 1.452$ 。
- $r = 0.5$ ： $\alpha = 0.25$ ，分母 $= 0.9829 \cdot (-0.9375) + 1 = 0.07853$ ， $D = 0.0625 / (\pi \cdot 0.07853^2) \approx 3.226$ 。
- $r = 0.8$ ： $\alpha = 0.64$ ， $\alpha^2 = 0.4096$ ，分母 $= 0.9829 \cdot (-0.5904) + 1 = 0.41965$ ， $D = 0.4096 / (\pi \cdot 0.41965^2) \approx 0.741$ 。

這個結果看似違反直覺： $r = 0.5$ 在 15° 處的 D 比 $r = 0.2$ 還大。原因是峰值高度與角寬互相耦合。由 $D(1) = 1/(\pi\alpha^2)$ 可得， $r = 0.2$ 時 $D(1) \approx 198.9$ ，但在 15° 已降到約1.45，只剩峰值的約1/137； $r = 0.5$ 時峰值約5.09，在 15° 仍有約3.23。因此，特定角度上的 D 較大不代表整體看起來更亮；低粗糙度的能量主要集中在正反射附近的窄角域。判讀材質外觀時，必須同時比較峰值高度與角寬，不能只看單一方向的BRDF數值。

實作與程式

以下腳本僅使用NumPy標準介面，供讀者自行執行；不依賴GPU、Blender或網路。

```
import numpy as np

def normalize(v):
    v = np.asarray(v, dtype=float)
    if not np.all(np.isfinite(v)):
        raise ValueError("vector must be finite")
    n = np.linalg.norm(v)
    if not np.isfinite(n) or n < 1e-12:
        raise ValueError("zero-length or non-finite vector")
    return v / n
```

```

def fresnel_schlick(cos_theta, f0):
    c = np.clip(cos_theta, 0.0, 1.0)
    return f0 + (1.0 - f0) * (1.0 - c) ** 5

def checked_roughness(roughness):
    r = float(roughness)
    if not np.isfinite(r) or not 0.0 <= r <= 1.0:
        raise ValueError("roughness 必須是 [0, 1] 內的有限純量")
    # alpha = r^2; 設正下限以避免 r=0 時 GGX 峰值形成 0/0。
    return max(r * r, 1e-4)

def ggx_d(n_dot_h, roughness):
    alpha = checked_roughness(roughness)
    a2 = alpha * alpha
    nh = np.clip(n_dot_h, 0.0, 1.0)
    denom = (nh ** 2) * (a2 - 1.0) + 1.0
    return a2 / (np.pi * denom * denom)

def smith_g1(n_dot_x, roughness):
    alpha = checked_roughness(roughness)
    a2 = alpha * alpha
    ndx = np.clip(n_dot_x, 1e-6, 1.0)
    return 2.0 * ndx / (ndx + np.sqrt(a2 + (1.0 - a2) * ndx * ndx))

def cook_torrance(N, V, L, albedo, roughness, metallic):
    N = normalize(np.asarray(N, dtype=float))
    V = normalize(np.asarray(V, dtype=float))
    L = normalize(np.asarray(L, dtype=float))
    albedo = np.asarray(albedo, dtype=float)
    metallic = float(metallic)
    if albedo.ndim != 0 and albedo.shape != (3,):
        raise ValueError("albedo 須為有限純量或 shape (3,) 的 RGB")
    if not np.all(np.isfinite(albedo)):
        raise ValueError("albedo 必須為有限值")
    if np.any((albedo < 0.0) | (albedo > 1.0)):
        raise ValueError("albedo 必須在 [0, 1]")
    if not np.isfinite(metallic) or not 0.0 <= metallic <= 1.0:
        raise ValueError("metallic 必須是 [0, 1] 內的有限值")
    checked_roughness(roughness)

    nv = float(np.dot(N, V))
    nl = float(np.dot(N, L))
    zero = np.zeros_like(albedo, dtype=float)
    if nv <= 0.0 or nl <= 0.0:
        return zero, zero, zero

    half_sum = V + L
    if np.linalg.norm(half_sum) < 1e-12:
        return zero, zero, zero
    H = normalize(half_sum)
    nh = max(float(np.dot(N, H)), 0.0)
    vh = max(float(np.dot(V, H)), 0.0)

    f0 = 0.04 * (1.0 - metallic) + albedo * metallic
    F = fresnel_schlick(vh, f0)
    D = ggx_d(nh, roughness)
    G = smith_g1(nv, roughness) * smith_g1(nl, roughness)
    spec = F * D * G / (4.0 * nv * nl)

```



```

kd = (1.0 - F) * (1.0 - metallic)
diff = kd * albedo / np.pi
return diff + spec, diff, spec

def run_tests():
    # T1 GGX 峰值
    d_peak = ggx_d(1.0, 0.5)
    assert abs(d_peak - 5.093) < 0.01, f"T1 failed: {d_peak}"
    print(f"T1 GGX peak: expected~5.093 got {d_peak:.4f}")

    # T2 Schlick 掠射極限
    f_g = fresnel_schlick(0.0, 0.04)
    assert abs(f_g - 1.0) < 1e-6, f"T2 failed: {f_g}"
    print(f"T2 Fresnel grazing: expected 1.0 got {f_g:.6f}")

    # 邊界與輸入驗證：背面方向、零粗糙度及非法參數。
    total, diff, spec = cook_torrance(
        [0, 0, 1], [0, 0, 1], [0, 0, -1], 0.5, 0.5, 0.0)
    assert np.allclose(total, 0.0)
    assert np.allclose(diff, 0.0) and np.allclose(spec, 0.0)
    assert np.isfinite(ggx_d(1.0, 0.0))
    for albedo_bad, roughness_bad, metallic_bad in (
        (0.5, np.nan, 0.0),
        (np.array([0.2, 0.3]), 0.5, 0.0),
        (0.5, 0.5, -0.1),
    ):
        try:
            cook_torrance([0, 0, 1], [0, 0, 1], [0, 0, 1],
                          albedo_bad, roughness_bad, metallic_bad)
        except ValueError:
            pass
        else:
            raise AssertionError("非法材質參數應被拒絕")

    # T3 金屬無漫反射
    total, diff, spec = cook_torrance([0,0,1], [0,0,1], [1,0,1], 0.5, 0.5, 1.0)
    assert diff < 1e-6, f"T3 failed: {diff}"
    print(f"T3 metallic diffuse: expected 0 got {diff:.8f}")

    # T4 例題16.2 合計
    total, diff, spec = cook_torrance(
        [0,0,1], [0,0,1], [np.sqrt(2)/2, 0, np.sqrt(2)/2], 0.5, 0.5, 0.0)
    assert 0.155 < total < 0.165, f"T4 failed: {total}"
    print(f"T4 Ex16.2: expected~0.1597 got {total:.4f} (diff {diff:.4f}, spec {spec:.4f})")

    # T5 例題16.3 掠射鏡面
    L85 = [np.sin(np.deg2rad(85)), 0, np.cos(np.deg2rad(85))]
    total, diff, spec = cook_torrance([0,0,1], [0,0,1], L85, 0.5, 0.5, 0.0)
    assert abs(spec - 0.00486) < 1e-3, f"T5 failed: {spec}"
    print(f"T5 Ex16.3 85deg spec: expected~0.00486 got {spec:.5f}")

    # T6 白色爐粗略診斷 (Monte Carlo 估計)。
    # 對每個均勻抽樣的 V，估計方向半球反射率
    #  $\int f_r(V, L)(N \cdot L) d\omega_L$ ，再對 V 做均勻平均。
    rng = np.random.default_rng(7)
    N = np.array([0.0, 0.0, 1.0])
    acc = 0.0
    K = 4000
    for _ in range(K):

```

```

# V 半球均勻取樣
u, v = rng.random(2)
z = u
phi = 2 * np.pi * v
r = np.sqrt(max(0.0, 1 - z * z))
V = np.array([r * np.cos(phi), r * np.sin(phi), z])
# L 半球均勻取樣，p(L)=1/(2π)
u, v = rng.random(2)
z = u
phi = 2 * np.pi * v
r = np.sqrt(max(0.0, 1 - z * z))
L = np.array([r * np.cos(phi), r * np.sin(phi), z])
tot, _, _ = cook_torrance(N, V, L, 1.0, 0.5, 0.0)
# Monte Carlo 項：f_r(V,L)(N·L)/p(L)
acc += tot * L[2] * (2 * np.pi)

rho_avg_est = acc / K
print(f"T6 white furnace diagnostic: expected near or below 1.0 got {rho_avg_est:.4f}")

print("All tests passed (assertions only verify the formulas above).")

if __name__ == "__main__":
    run_tests()

```

輸出訊息標為「預期」對應值；實際執行時 assert 不應觸發，觸發即表示實作與本章公式偏離。T6 沒有設定 assert，而是以固定亂數種子進行粗略診斷：它先估計各隨機觀察方向的方向半球反射率，再對觀察方向平均。單次散射模型通常使結果接近或低於 1；有限樣本仍有 Monte Carlo 誤差，因此不能把此輸出當成嚴格的能量守恆證明。

測試與預期結果

下表列出每項測試的輸入、期望輸出與容差。這些數字由公式手算，讀者在自己環境執行後再比對，不得視為外部已驗證的結果。

編號	測試	輸入	預期輸出	容差
T1	GGX 正向峰值	$n \cdot h = 1.0$, $r = 0.5$	5.093	10^{-2}
T2	Schlick 掠射	$\cos = 0.0$, $f_0 = 0.04$	1.0	10^{-6}
T3	金屬無漫反射	$m = 1.0$	diff \$=0\$	10^{-6}
T4	例題16.2 合計	見上	0.155 ~ 0.165	區間
T5	例題16.3 鏡面	85°	0.00486	10^{-3}
T6	白色爐平均反射率診斷	$\rho = 1, r = 0.5$	接近或低於 1.0	無 assert；觀察趨勢

T4 使用區間而非單點，是因為手算僅保留四位小數；若你把自己的中間量算到更多位，把區間改成單點也可。T5 的容差設 10^{-3} 而非 5×10^{-4} ，是為了容忍手算與程式浮點在中間步驟（ $\sin 85^\circ$ 、 $\cos 85^\circ$ ）的差異；這是**工程容差**，不是物理容差。T6 以半球均勻取樣估計反射率，並額外對觀察方向取平均；它只適合

作為粗略趨勢診斷。單次散射模型在高粗糙度下可能損失能量，而有限樣本也會造成隨機誤差；若估計值穩定且顯著超過 1.0，才應優先檢查 Jacobian、 k_d 或 PDF 補償。

預期結果以公式計算為準。若你得到明顯不同，先檢查 H 是否正規化、 $n \cdot x$ 零截斷、 α 是否誤用成 roughness 而非其平方。

除錯與常見陷阱

- 粗糙度映射混淆：正確版本 $\alpha = r^2$ ，錯誤版本把 r 直接當 α 。下表以 $r = 0.5$ 對比：

量	正確 ($\alpha = 0.25$)	錯誤 ($\alpha = 0.5$)
α^2	0.0625	0.25
$D(n \cdot h = 1)$	$0.0625 / (\pi \cdot 0.0625^2) \approx 5.09$	$0.25 / (\pi \cdot 0.25^2) \approx 1.27$
$D(n \cdot h = 0.5)$	≈ 0.0339	≈ 0.0612

峰值差約四倍，尾端也一併過度平滑。特別注意峰值**變低**不代表整體變暗：尾端若同步變大，視覺上高光會變寬、變鈍，但總能量未必下降，這是最容易在比對合成圖時誤判的地方。

- 零長向量**： $V + L = 0$ 時半角未定義（背向觀察）。必須在 V 與 L 反向時回傳 0 反射，不要嘗試 normalize 零向量。
- 忽略標準分母** $4(N \cdot V)(N \cdot L)$ ：此分母結合半向量映射 Jacobian、微面投影與巨觀方向投影因子；直接把 $F D G$ 當 BRDF，數值會大一至二個數量級，低粗糙度下直接爆白。
- Fresnel 用錯角度**： F 應使用 $\omega_o \cdot h$ 或 $\omega_i \cdot h$ （互易等價），不能使用 $N \cdot V$ ，否則掠射行為整段錯位。
- 能量未守恆**：粗暴地令 $k_d = 1 - m$ 忽略 $(1 - F)$ ，在掠射角會多出接近 $F \approx 1$ 的能量；短期看起來更「亮」，白色爐測試會暴露爆增。
- 把 D 峰值當視覺外觀指標**：例題16.4 顯示 $r = 0.5$ 在 15° 處的 D 反而大於 $r = 0.2$ 。只比較峰值會得到與視覺相反的結論；討論材質外觀時應同時報峰值與角寬。
- 資料貼圖色彩空間**：粗糙度圖、金屬圖、法線圖皆為**資料**，絕不能做 sRGB 解碼。只有 albedo 貼圖才做 sRGB。
- 數值穩健**： α 為 0 時 D 在 $n \cdot h \neq 1$ 為 0、在 $n \cdot h = 1$ 為無定義。實作應以 $\alpha_{\min}$ （如 10^{-4} ）下限替代 0。

養殖數位分身案例

我們不追求商業即時渲染的視覺說服力，而在意**參數可追溯**。建議每條魚的材質記錄一份 JSON，欄位至少包括：

```
{
  "material_name": "fish_skin_synthetic",
```

```

"version": "1.0",
"source": "synthetic",
"albedo": {
  "values": [0.42, 0.38, 0.31],
  "color_space": "linear_rgb",
  "decoded_from_srgb": true
},
"roughness": {
  "map_path": "assets/fish_roughness.ppm",
  "mapping": "perceptual_sqrt",
  "color_space": "linear_data",
  "decoded_from_srgb": false
},
"metallic": {
  "constant": 0.0,
  "color_space": "linear_data"
},
"fresnel_f0": 0.04,
"units": {
  "roughness_range": [0.0, 1.0],
  "length_unit": "meters"
},
"notes": "Synthetic parameters for visual sensitivity testing only. Not derived from biological measurement."
}

```

材質敏感度對照實驗（合成）：固定點光源位置、相機位置與場景幾何，僅改變 `roughness` 為 0.2、0.5、0.8。以下數值為本章公式在例題16.4 給定幾何下算出的**相對關係**，讀者在自己的場景中會得到不同的絕對數字，但排序應一致：

roughness	D 峰值 ($n \cdot h = 1$)	D 在 15°	峰值角寬 (D 掉到峰值 1/10 的角度)
0.2	≈ 198.9	≈ 1.45	半角 h 約 3.37° (固定 $V = N$ 時光源角約 6.74°)
0.5	≈ 5.09	≈ 3.23	半角 h 約 22.3° (固定 $V = N$ 時光源角約 44.6°)
0.8	≈ 0.78	≈ 0.74	半球內無法降至峰值的 1/10

這張表把「峰值下降」與「角寬擴大」分開呈現： $r = 0.2$ 峰值極高但角寬極窄； $r = 0.8$ 峰值極低但角寬極大。它只是渲染模型的數值行為，**不用於推斷真實魚鱗的粗糙度**。真實魚鱗具有微結構多層散射、角度依賴色偏與次表面成分，無法以單一 BRDF 參數完整描述。

在稽核層面，建議每個合成幀的 metadata 記錄：使用的 JSON 檔名與雜湊、相機外參、光源設定、粗糙度參數。當 agent 之後要回答「這一幀的魚為何看起來比上一幀亮」時，能指到具體參數差異，而非靠人眼猜測。所有這些操作都在合成資料上，不接真實感測器、不控制水質或投餵。

習題

習題 16.1（手算）

取 $r = 0.3$ ， $\alpha = r^2$ ，求 $n \cdot h = 1$ 、 0.7 、 0.3 三點的 GGX D 值。指出 $n \cdot h = 1$ 時的 D 是否隨 α 增大而變小。

習題 16.2（程式測試）

用本章 `cook_torrance` 函式，固定 $N = (0, 0, 1)$ 、 $V = (0, 0, 1)$ 、 $\rho = 0.6$ 、 $m = 0$ 、 $r = 0.5$ ，令 L 在 xz 平面與 N 夾 5° 、 45° 、 85° ，計算鏡面項。列出三個角度，並說明「掠射 Fresnel 上升」在此粗糙度下是否能帶動鏡面 BRDF 上升。

習題 16.3（反例與除錯）

讀者 A 將 `ggx_d` 中的 `alpha = roughness * roughness` 改為 `alpha = roughness`，再跑例題 16.2。請指出峰值、尾端行為、以及與能量測試的差異各發生什麼改變。並給出一個能區分正確與錯誤版本的單點測試。

習題 16.4（整合應用）

設計一份可稽核的材質清單：魚皮、魚鰭、池壁、水位線附近的金屬感測器外殼。對每一項寫出 albedo（線性）、roughness、metallic、 F_0 假設與色彩空間處理方式。列出至少兩條「合成 vs 實測」界線聲明。

習題解答

16.1

$\alpha = 0.09$ ， $\alpha^2 = 0.0081$ ， $\alpha^2 - 1 = -0.9919$ 。

$n \cdot h = 1$ ：

$$D = \frac{0.0081}{\pi((1)(-0.9919) + 1)^2} = \frac{0.0081}{\pi(0.0081)^2} = \frac{0.0081}{\pi \cdot 6.561 \times 10^{-5}} \approx 39.28.$$

$n \cdot h = 0.7$ ： $(0.7)^2 = 0.49$ ， $0.49 \cdot (-0.9919) + 1 = 0.5139$ ， $D = 0.0081/(\pi \cdot 0.2641) \approx 0.00977$ 。

$n \cdot h = 0.3$ ： $0.09 \cdot (-0.9919) + 1 = 0.9107$ ， $D = 0.0081/(\pi \cdot 0.8294) \approx 0.00311$ 。

對比例題 16.1 的 $r = 0.5$ （ $n \cdot h = 1$ 時 $D \approx 5.09$ ），可見 roughness 減小使峰值 D 變大： $r = 0.3$ 的峰值約為 $r = 0.5$ 的 7.7 倍。物理直覺：粗糙度越小，微面越集中，峰值越尖，同時高光瓣越窄。

16.2

$L_{5^\circ} = (\sin 5^\circ, 0, \cos 5^\circ) \approx (0.0872, 0, 0.9962)$; $L_{45^\circ} = (0.7071, 0, 0.7071)$; $L_{85^\circ} = (\sin 85^\circ, 0, \cos 85^\circ) \approx (0.9962, 0, 0.0872)$ 。

對每個 L 計算 $H = \text{normalize}(V + L)$ 、 $N \cdot H$ 、 $V \cdot H$ ，再套入 F, D, G 。所有數值以公式計算，讀者應以程式核對。

L 角度	$N \cdot L$	$N \cdot H$	$V \cdot H$	F	D	G	鏡面 BRDF
5°	0.9962	0.9990	0.9990	0.0400	4.8144	0.9999	0.0483
45°	0.7071	0.9239	0.9239	0.0400	0.4984	0.9848	0.00694
85°	0.0872	0.7373	0.7373	0.0412	0.0827	0.4966	0.00486

舉 85° 為驗算示例： $|V + L| = \sqrt{0.9962^2 + 1.0872^2} \approx 1.4746$ ， $H \approx (0.6756, 0, 0.7373)$ ， $N \cdot H = 0.7373$ 。 $D = 0.0625 / [\pi(0.7373^2 \cdot (-0.9375) + 1)^2] = 0.0625 / (\pi \cdot 0.4904^2) \approx 0.0827$ 。 $G_1(V) = 1$ ， $G_1(L)$ 帶入 $n \cdot L = 0.0872$ 得 ≈ 0.4966 。 $F = 0.04 + 0.96(1 - 0.7373)^5 \approx 0.0412$ 。 $F D G / (4 \cdot 1 \cdot 0.0872) \approx 0.00486$ 。

觀察：在這組固定 $V = N$ 的測試中，鏡面 BRDF 隨光源角度增大而**下降**。 F 僅從 0.0400 升至 0.0412；而 D 從 4.81 降到 0.083、 G 從 1.0 降到 0.50，分母中的 $1/(N \cdot L)$ 放大不足以補償。降低粗糙度不保證此掃描路徑的掠射 BRDF 上升，因為更窄的 D 會更快偏離正反射方向。若要研究視覺上的邊緣高光，應另設 V 掠射且 L 接近其鏡射對應方向的案例。

16.3

錯誤版本以 `alpha = roughness = 0.5`， $n \cdot h = 1$ 時

$$D_{\text{wrong}} = \frac{\alpha^2}{\pi \alpha^4} = \frac{1}{\pi \alpha^2} = \frac{1}{\pi \cdot 0.25} \approx 1.273,$$

而正確實為 5.09。峰值被壓縮約四倍，尾端因為分母 $(\alpha^2 - 1)$ 只變為 -0.75 而過度平滑。整體外觀：高光變寬、峰值變鈍，掠射高光在數值上偏低。

能區分的單點測試：呼叫 `ggx_d(1.0, 0.5)`。以本章約定應回傳 5.093，若得到約 1.273 即為誤用。此測試不依賴幾何，只依賴 r ，是乾淨的分辨器。若要更強的除錯訊號，再加一個尾端檢查 `ggx_d(0.5, 0.5)`：正確應約 0.0339，錯誤約 0.0612；兩點同時落入錯誤值，幾乎可斷定是映射問題。

16.4

範例清單（每項數值僅為合成假設，非實測）：

物件	albedo (線性)	roughness	metallc	F_0	色彩空間
魚皮	(0.42, 0.38, 0.31)	0.35	0.0	0.04	albedo貼圖 sRGB 解碼；roughness 圖線性
魚鱗	(0.72, 0.55, 0.48)	0.55	0.0	0.04	同上；roughness 需與透明度分開通道
池壁（混凝土）	(0.48, 0.46, 0.42)	0.85	0.0	0.04	albedo 貼圖線性；不做 sRGB 解碼
感測器外殼（金屬）	(0.85, 0.86, 0.88)	0.18	1.0	albedo 充當	albedo 若為圖片須 sRGB；roughness 線性

合成 vs 實測界線聲明：

1. 上述 albedo 數值為目視調校的合成值，未經分光量測；不得宣稱其對應真實魚鱗或混凝土反射率。
2. Roughness 的 $[0, 1]$ 為渲染參數，不具物理單位；不能引用為量測粗糙度或表面 RMS 高度。
3. 金屬度為二值化的渲染近似；實際魚鱗為介電質多層結構，在沒有實測資料前不得作為光學分類依據。

本章小結

物理材質 BRDF 的核心是 Fresnel、法線分佈、幾何遮蔽與投影因子的組合。結合半向量映射的 Jacobian、微面投影面積及巨觀方向的投影餘弦後，得到分母 $4(N \cdot V)(N \cdot L)$ ；它不是可隨意省略的係數。在 $\alpha \rightarrow 0$ 的極限下，缺少這些幾何因子會使 BRDF 無法退化為鏡面反射分布。粗糙度控制 D 的寬窄，金屬度控制 F_0 與漫反射權重，反照率決定漫反射基底與金屬反射色。 $\alpha = r^2$ 的映射必須明示，否則兩份相同粗糙度的資產外觀不一致。

掠射角下 Fresnel 固然上升，但是否帶動鏡面 BRDF 上升，還取決於 D 、 G 及入射與觀察方向的相對幾何。本章固定 $V = N$ 、只增加光源角度的例子在 $r = 0.5$ 時呈下降趨勢；降低粗糙度並不保證同一路徑出現邊緣高光，仍須檢查光源與觀察方向是否接近正反射配置。討論材質外觀時，必須同時報峰值與角寬，只看峰值容易得到與視覺相反的結論。單次散射的 Smith-GGX 在高粗糙度下會整體偏暗，這是模型已知的能量缺口，不是參數設錯。零長向量、能量守恆與資料貼圖色彩空間是本章反覆指出的工程細節；任一項漏掉，都可能讓渲染結果悄悄偏離物理。

本章內容屬合成模型；用它生成的圖像與參數表僅作為數位分身場景的輸入，不能替代真實感測，也不能推得生物或生態結論。

參考來源

- [G1] PBRT 4：Transformations，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G2] PBRT 4：Reflection Models，https://pbr-book.org/4ed/Reflection_Models

- [G3] PBRT 4 : The Light Transport Equation , https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- [G4] Ray Tracing in One Weekend , <https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- [G5] LearnOpenGL : Transformations , <https://learnopengl.com/Getting-started/Transformations>
- [G6] Blender Manual : Skinning Introduction , <https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>
- [G7] NumPy 線性代數參考 , <https://numpy.org/doc/stable/reference/routines.linalg.html>
- [G8] Khronos glTF 2.0 規格 , <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

以上來源僅作為回查入口，本章所有公式皆為自行推導與重新表達；未複製來源文字，也未宣稱任何外部頁面已逐條驗證。

第 17 章 法線貼圖與表面細節

學習目標與先備知識

完成本章後，讀者應能：

1. 區分幾何法線、著色法線與法線貼圖中的局部法線。
2. 由三角形位置與 UV 建立切線空間基底 TBN。
3. 解碼 RGB 法線貼圖，並將切線空間法線轉至世界空間。
4. 以 Gram-Schmidt 方法正交化切線，正確處理鏡射 UV 的手性。
5. 在非均勻縮放與鏡射變換下分別處理切線、法線與手性。
6. 說明法線貼圖、凹凸貼圖與位移貼圖的差異。
7. 測試中性法線、病態 UV、小尺度變換與奇異縮放。

先備知識為向量、內積、外積、矩陣逆，以及第 13~16 章的線性色彩、UV、局部照明與材質概念。本章長度單位為公尺；UV 與方向向量無單位。

問題與直覺

若要表現魚鱗、混凝土池壁或水槽刮痕，直接把每條紋路建成三角形通常代價很高。法線貼圖不改變網格位置，而是在每個片段提供不同的表面方向，使照明產生細小明暗變化。

貼圖中的 RGB 通常不是世界空間方向，而是**切線空間法線**。對表面每一點建立三個局部軸：

- **T**：UV 的 u 增加方向，稱切線；
- **B**：UV 的 v 增加方向，稱副切線；
- **N**：表面法線。

三者組成 TBN 基底。中性法線貼圖表示 $(0, 0, 1)^T$ ，經 TBN 轉換後恰好成為原本的 **N**，所以不應改變照明。

法線貼圖只改變著色所使用的方向，不會：

- 改變物體輪廓；
- 真正遮擋其他幾何；
- 改變射線與表面的交點；
- 自動產生正確的自陰影。

若起伏必須改變輪廓、深度或交點，就需要位移或實際幾何。

數學與幾何推導

1. 幾何法線、著色法線與貼圖法線

三角形頂點為 $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2 \in \mathbb{R}^3$ ，令

$$\mathbf{e}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{e}_2 = \mathbf{p}_2 - \mathbf{p}_0.$$

其單位幾何法線為

$$\mathbf{N}_g = \frac{\mathbf{e}_1 \times \mathbf{e}_2}{\|\mathbf{e}_1 \times \mathbf{e}_2\|}.$$

若外積長度接近零，三角形退化，法線沒有穩定定義。

著色法線 $\mathbf{N}_s$ 可以由頂點法線插值得到，通常比逐面幾何法線平滑。法線貼圖解碼出的 $\mathbf{n}_t$ 位於切線空間。最終世界空間著色法線為

$$\mathbf{n}_w = \text{normalize}(\mathbf{T}_w n_x + \mathbf{B}_w n_y + \mathbf{N}_w n_z).$$

下標 t 表示 tangent space，下標 w 表示 world space。

2. 由 UV 導出切線與副切線

三個頂點的 UV 為

$$\mathbf{q}_i = (u_i, v_i)^T.$$

定義

$$\Delta u_1 = u_1 - u_0, \quad \Delta v_1 = v_1 - v_0,$$

$$\Delta u_2 = u_2 - u_0, \quad \Delta v_2 = v_2 - v_0.$$

假設三角形內位置對 UV 局部近似線性：

$$\mathbf{e}_1 = \mathbf{T}_{\text{raw}} \Delta u_1 + \mathbf{B}_{\text{raw}} \Delta v_1,$$

$$\mathbf{e}_2 = \mathbf{T}_{\text{raw}} \Delta u_2 + \mathbf{B}_{\text{raw}} \Delta v_2.$$

寫成矩陣形式：

$$\begin{bmatrix} \Delta u_1 & \Delta v_1 \\ \Delta u_2 & \Delta v_2 \end{bmatrix} \begin{bmatrix} \mathbf{T}_{\text{raw}}^T \\ \mathbf{B}_{\text{raw}}^T \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1^T \\ \mathbf{e}_2^T \end{bmatrix}.$$

令 UV 差分矩陣為 $\mathbf{U} \in \mathbb{R}^{2 \times 2}$ ，且

$$D = \det(\mathbf{U}) = \Delta u_1 \Delta v_2 - \Delta u_2 \Delta v_1.$$

若 $D \neq 0$ ，解析解為

$$\begin{aligned}\mathbf{T}_{\text{raw}} &= \frac{\Delta v_2 \mathbf{e}_1 - \Delta v_1 \mathbf{e}_2}{D}, \\ \mathbf{B}_{\text{raw}} &= \frac{-\Delta u_2 \mathbf{e}_1 + \Delta u_1 \mathbf{e}_2}{D}.\end{aligned}$$

只測試 $|D|$ 是否小於固定常數並不理想。若整個 UV 島等比例縮小， D 會按尺度平方縮小，但其形狀可能仍良好；另一方面，極狹長的 UV 三角形即使面積不算極小，也可能病態。

因此實作以 $\mathbf{U}$ 的奇異值判斷。設

$$\sigma_{\max} \geq \sigma_{\min} \geq 0.$$

若

$$\sigma_{\min} \leq r_{\text{uv}} \sigma_{\max},$$

便將 UV 映射視為奇異或過度病態。比例 $\sigma_{\min}/\sigma_{\max}$ 衡量形狀品質，不受 UV 整體非零縮放影響。

3. TBN 正交化與手性

原始切線未必與著色法線垂直。先移除其法線分量：

$$\mathbf{T} = \text{normalize}(\mathbf{T}_{\text{raw}} - \mathbf{N}(\mathbf{N} \cdot \mathbf{T}_{\text{raw}})).$$

再由原始副切線判斷手性：

$$h = \text{sign}[(\mathbf{N} \times \mathbf{T}) \cdot \mathbf{B}_{\text{raw}}], \quad h \in \{-1, +1\}.$$

最後重建

$$\mathbf{B} = h(\mathbf{N} \times \mathbf{T}).$$

TBN 矩陣的三個**縱列 (column) **依序為 $\mathbf{T}, \mathbf{B}, \mathbf{N}$ ：

$$\mathbf{M}_{\text{TBN}} = \begin{bmatrix} | & | & | \\ \mathbf{T} & \mathbf{B} & \mathbf{N} \\ | & | & | \end{bmatrix}.$$

依本書 column-vector 慣例，切線空間到世界空間的主動轉換為

$$\mathbf{n}_w = \text{normalize}(\mathbf{M}_{\text{TBN}}\mathbf{n}_t).$$

鏡射 UV 通常會改變 D 的符號，進而改變 h 。若永遠使用 $\mathbf{B} = \mathbf{N} \times \mathbf{T}$ ，鏡射區域的凹凸方向可能翻轉。

網格格式常儲存四分量切線 (T_x, T_y, T_z, h) ，副切線在著色時重建。不同格式也可能使用不同 UV 軸方向；匯入時必須確認，不能只靠外觀猜測。

4. 法線貼圖解碼

貼圖通道通常儲存在 $[0, 1]$ 。完整 RGB 解碼為

$$\tilde{\mathbf{n}}_t = 2\mathbf{c} - \mathbf{1}, \quad \mathbf{n}_t = \frac{\tilde{\mathbf{n}}_t}{\|\tilde{\mathbf{n}}_t\|}.$$

中性值為

$$\mathbf{c} = (0.5, 0.5, 1), \quad \mathbf{n}_t = (0, 0, 1).$$

法線貼圖是方向資料，不是色彩，不可套用 sRGB 到線性 RGB 的轉換。若錯把 0.5 當作 sRGB 解碼，線性值約為 0.214；再映射到 $[-1, 1]$ 後約為 -0.572 ，中性法線會嚴重偏斜。

有些格式只儲存 x, y ，並假設法線位於正 z 半球：

$$z = \sqrt{\max(0, 1 - x^2 - y^2)}.$$

若 $x^2 + y^2 > 1$ ，可能源於量化、壓縮或資料錯誤。截斷至零可以避免平方根失效，但不能恢復遺失的資訊。

5. 非均勻縮放、鏡射與可逆性

物件到世界空間仿射變換的線性部分為可逆矩陣 $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ 。切向量依一般方向變換：

$$\mathbf{T}'_{\text{raw}} = \mathbf{A}\mathbf{T}.$$

法線必須保持與變換後切平面垂直，因此使用逆轉置：

$$\mathbf{N}' = \text{normalize}(\mathbf{A}^{-T}\mathbf{N}).$$

接著再相對 $\mathbf{N}'$ 正交化 $\mathbf{T}'_{\text{raw}}$ 。若直接以 $\mathbf{A}\mathbf{N}$ 變換法線，非均勻縮放下通常不再垂直切面。

若 $\det(\mathbf{A}) < 0$ ，變換包含鏡射，空間手性翻轉：

$$h' = h \operatorname{sign}(\det \mathbf{A}).$$

再以

$$\mathbf{B}' = h'(\mathbf{N}' \times \mathbf{T}')$$

重建副切線。渲染器也要一致處理三角形繞序與背面剔除。

可逆性不能只看行列式絕對值

直接使用固定容差判斷 $|\det(\mathbf{A})|$ 是否接近零並不穩健。例如

$$\mathbf{A} = \text{diag}(10^{-4}, 10^{-4}, 10^{-4})$$

完全可逆且條件良好，但行列式為 10^{-12} 。

設 $\mathbf{A}$ 的最大、最小奇異值為 $\sigma_{\max}$ 、 $\sigma_{\min}$ 。若

$$\sigma_{\min} \leq r_A \sigma_{\max},$$

便將矩陣視為數值上奇異或過度病態。其二範數條件數為

$$\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

即使矩陣在精確數學上可逆，條件數過大仍可能放大浮點誤差。

手性符號也不能直接依賴普通行列式值

若 $\mathbf{A}$ 是極小的正均勻縮放，普通浮點行列式可能下溢為 0.0。若程式寫成：

```
mirror_sign = 1.0 if np.linalg.det(A) > 0.0 else -1.0
```

便可能把正尺度誤判成鏡射。

較穩健的方法是先除以正數 $\sigma_{\max}$ ：

$$\widehat{\mathbf{A}} = \frac{\mathbf{A}}{\sigma_{\max}}.$$

此正尺度正規化不改變方向、條件數或行列式符號。再以具符號對數行列式取得符號：

$$(s, \ell) = \text{slogdet}(\widehat{\mathbf{A}}),$$

其中 $s \in \{-1, 0, +1\}$ 是行列式符號， ℓ 是絕對行列式的自然對數。使用正規化矩陣也可避免直接計算極小逆尺度時溢位。

6. 凹凸、法線與位移

高度函數 $H(u, v)$ 可描述沿基準法線的小位移。對局部平面近似：

$$\mathbf{P}(u, v) = u\mathbf{T} + v\mathbf{B} + H(u, v)\mathbf{N}.$$

偏導為

$$\mathbf{P}_u = \mathbf{T} + H_u\mathbf{N}, \quad \mathbf{P}_v = \mathbf{B} + H_v\mathbf{N}.$$

在正交右手基底中，其法線方向近似

$$\mathbf{n}_t \propto (-H_u, -H_v, 1).$$

這就是凹凸貼圖：由高度差分估計斜率，再修改法線。法線貼圖直接儲存方向，能表示更自由的細節，但不一定能積分回單一一致的高度場。位移貼圖直接改變 $\mathbf{P}$ ，所以可能改變輪廓、遮擋與交點。

逐步手算例題

例 1：標準 UV 三角形與中性法線

令

$$\mathbf{p}_0 = (0, 0, 0), \quad \mathbf{p}_1 = (2, 0, 0), \quad \mathbf{p}_2 = (0, 1, 0),$$

$$\mathbf{q}_0 = (0, 0), \quad \mathbf{q}_1 = (1, 0), \quad \mathbf{q}_2 = (0, 1).$$

因此

$$\mathbf{e}_1 = (2, 0, 0), \quad \mathbf{e}_2 = (0, 1, 0),$$

且 $D = 1$ 。代入公式：

$$\mathbf{T}_{\text{raw}} = (2, 0, 0), \quad \mathbf{B}_{\text{raw}} = (0, 1, 0).$$

幾何法線為

$$\mathbf{N} = (0, 0, 1).$$

正規化後

$$\mathbf{T} = (1, 0, 0), \quad h = \text{sign}[(0, 1, 0) \cdot (0, 1, 0)] = +1,$$

$$\mathbf{B} = (0, 1, 0).$$

中性法線 $\mathbf{c} = (0.5, 0.5, 1)$ 解碼為 $\mathbf{n}_t = (0, 0, 1)$ ，所以

$$\mathbf{n}_w = \mathbf{T}(0) + \mathbf{B}(0) + \mathbf{N}(1) = (0, 0, 1).$$

例 2：鏡射 UV

位置不變，但令

$$\mathbf{q}_1 = (0, 1), \quad \mathbf{q}_2 = (1, 0).$$

此時

$$D = 0 \cdot 0 - 1 \cdot 1 = -1.$$

計算得到

$$\mathbf{T}_{\text{raw}} = (0, 1, 0), \quad \mathbf{B}_{\text{raw}} = (2, 0, 0).$$

正規化切線為 $\mathbf{T} = (0, 1, 0)$ ，而

$$(\mathbf{N} \times \mathbf{T}) \cdot \mathbf{B}_{\text{raw}} = (-1, 0, 0) \cdot (2, 0, 0) = -2.$$

故 $h = -1$ ，且

$$\mathbf{B} = -[\mathbf{N} \times \mathbf{T}] = (1, 0, 0).$$

若遺失 $h = -1$ ，副切線會錯成 $(-1, 0, 0)$ 。

例 3：非均勻縮放

取

$$\mathbf{T} = \frac{(1, 1, 0)}{\sqrt{2}}, \quad \mathbf{N} = \frac{(-1, 1, 0)}{\sqrt{2}},$$

以及

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

切線直接變換：

$$\mathbf{AT} = \frac{(2, 1, 0)}{\sqrt{2}}.$$

若錯誤地直接變換法線，得到 $(-2, 1, 0)/\sqrt{2}$ 。忽略共同尺度，其內積為

$$(2, 1, 0) \cdot (-2, 1, 0) = -3 \neq 0.$$

正確法線為

$$\mathbf{A}^{-T} \mathbf{N} = \frac{(-1/2, 1, 0)}{\sqrt{2}}.$$

因此

$$(2, 1, 0) \cdot (-1/2, 1, 0) = 0,$$

仍與切線垂直。

實作與程式

以下程式只需 Python 3.10+ 與 NumPy。它不讀檔、不連網，也不執行 GPU 程式。

程式分別使用：

- `VECTOR_EPS`：向量長度的絕對判定；
- `UV_RTOL`：UV 差分矩陣的奇異值相對判定；
- `MATRIX_RTOL`：物件變換的奇異值相對判定；
- `FRAME_RTOL`：副切線與重建方向的相對一致性判定。

```
import numpy as np

VECTOR_EPS = 1e-12
UV_RTOL = 1e-12
MATRIX_RTOL = 1e-12
FRAME_RTOL = 1e-12

def normalize(v, eps=VECTOR_EPS):
    v = np.asarray(v, dtype=float)
    length = np.linalg.norm(v)
    if not np.isfinite(length) or length <= eps:
        raise ValueError("無法正規化零長、近零或非有限向量")
    return v / length

def relative_singular_values(M):
    """回傳最大與最小奇異值。M 必須含有限數值。"""
    M = np.asarray(M, dtype=float)
    if not np.all(np.isfinite(M)):
        raise ValueError("矩陣含有非有限數值")
    s = np.linalg.svd(M, compute_uv=False)
    return float(s[0]), float(s[-1])

def triangle_tangent_frame(
    p0, p1, p2, uv0, uv1, uv2,
    uv_rtol=UV_RTOL,
```



```

frame_rtol=FRAME_RTOL
):
    p0, p1, p2 = map(
        lambda x: np.asarray(x, dtype=float), (p0, p1, p2)
    )
    uv0, uv1, uv2 = map(
        lambda x: np.asarray(x, dtype=float), (uv0, uv1, uv2)
    )

    e1 = p1 - p0
    e2 = p2 - p0
    ng_raw = np.cross(e1, e2)
    n = normalize(ng_raw)

    d1 = uv1 - uv0
    d2 = uv2 - uv0

    # U 的每個橫列分別是兩條 UV 邊。
    U = np.array([
        [d1[0], d1[1]],
        [d2[0], d2[1]]
    ], dtype=float)

    sigma_max, sigma_min = relative_singular_values(U)
    if sigma_max == 0.0 or sigma_min <= uv_rtol * sigma_max:
        raise ValueError("UV 映射奇異或過度病態")

    # 正數縮放不改變方程的解；先將 U 以 sigma_max 正規化。
    # 原方程  $U X = E$ ，其中  $U = \text{sigma\_max} * U\_scaled$ 。
    # 因此  $X = U^{-1} E = (1 / \text{sigma\_max}) * U\_scaled^{-1} E$ ；
    # 右側先除以 sigma_max，避免解出後再額外縮放一次。
    U_scaled = U / sigma_max

    #  $U @ [T^T; B^T] = [e1^T; e2^T]$ 
    edges = np.vstack([e1, e2])
    tb = np.linalg.solve(U_scaled, edges / sigma_max)
    t_raw = tb[0]
    b_raw = tb[1]

    if not np.all(np.isfinite(tb)):
        raise ValueError("切線求解超出可表示範圍")

    # 殘差核對：解回  $U @ tb$  應還原 edges。
    residual = U @ tb - edges
    residual_scale = max(1.0, float(np.linalg.norm(edges)))
    if float(np.linalg.norm(residual)) > 1e-9 * residual_scale:
        raise ValueError("切線求解殘差過大，UV 或位置資料可能不一致")

    t_ortho = t_raw - n * np.dot(n, t_raw)
    t = normalize(t_ortho)

    b_length = np.linalg.norm(b_raw)
    if not np.isfinite(b_length) or b_length <= VECTOR_EPS:
        raise ValueError("副切線為零長、近零或非有限")

    triple = np.dot(np.cross(n, t), b_raw)
    if abs(triple) <= frame_rtol * b_length:
        raise ValueError("無法穩定判定切線空間手性")

```

```

    handedness = 1.0 if triple > 0.0 else -1.0
    b = handedness * np.cross(n, t)
    return t, b, n, handedness

def decode_normal(rgb):
    rgb = np.asarray(rgb, dtype=float)
    if rgb.shape != (3,) or not np.all(np.isfinite(rgb)):
        raise ValueError("RGB 必須是三個有限分量")
    return normalize(2.0 * rgb - 1.0)

def tangent_to_world(n_tangent, t, n, handedness):
    n_tangent = normalize(n_tangent)
    n = normalize(n)
    t = normalize(t - n * np.dot(n, t))
    b = handedness * np.cross(n, t)

    return normalize(
        t * n_tangent[0]
        + b * n_tangent[1]
        + n * n_tangent[2]
    )

def transform_frame(
    t, n, handedness, A, matrix_rtol=MATRIX_RTOL
):
    A = np.asarray(A, dtype=float)
    if A.shape != (3, 3):
        raise ValueError("A 必須是 3x3 矩陣")

    sigma_max, sigma_min = relative_singular_values(A)
    if sigma_max == 0.0 or sigma_min <= matrix_rtol * sigma_max:
        raise ValueError("線性變換奇異或過度病態")

    # 除以正數不改變方向、條件數或行列式符號。
    A_scaled = A / sigma_max

    sign, logabsdet = np.linalg.slogdet(A_scaled)
    if sign == 0.0 or not np.isfinite(logabsdet):
        raise ValueError("無法穩定判定變換手性")

    n = np.asarray(n, dtype=float)
    t = np.asarray(t, dtype=float)

    # solve(A_scaled.T, n) 與  $A^{-T}n$  方向相同。
    n_raw = np.linalg.solve(A_scaled.T, n)
    if not np.all(np.isfinite(n_raw)):
        raise ValueError("法線逆轉置結果非有限")
    n_world = normalize(n_raw)

    # 使用正規化矩陣可避免極小尺度使方向下溢。
    t_raw = A_scaled @ t
    if not np.all(np.isfinite(t_raw)):
        raise ValueError("切線變換結果非有限")

    t_world = normalize(
        t_raw - n_world * np.dot(n_world, t_raw)
    )

    mirror_sign = float(sign)

```

```

h_world = handedness * mirror_sign
b_world = h_world * np.cross(n_world, t_world)

return t_world, b_world, n_world, h_world

if __name__ == "__main__":
    p0 = [0, 0, 0]
    p1 = [2, 0, 0]
    p2 = [0, 1, 0]

    # 標準 UV 與中性法線。
    t, b, n, h = triangle_tangent_frame(
        p0, p1, p2, [0, 0], [1, 0], [0, 1]
    )
    neutral = decode_normal([0.5, 0.5, 1.0])
    n_shaded = tangent_to_world(neutral, t, n, h)

    assert np.allclose(n_shaded, n, atol=1e-9)
    assert np.isclose(np.dot(t, n), 0.0, atol=1e-9)
    assert np.isclose(np.dot(b, n), 0.0, atol=1e-9)

    # 鏡射 UV。
    tm, bm, nm, hm = triangle_tangent_frame(
        p0, p1, p2, [0, 0], [0, 1], [1, 0]
    )
    assert hm == -1.0

    # 小型但形狀良好的 UV 島應被接受，且方向與標準 UV 案例一致。
    tu, bu, nu, hu = triangle_tangent_frame(
        p0, p1, p2,
        [0, 0], [1e-6, 0], [0, 1e-6]
    )
    assert np.allclose(tu, t, atol=1e-9)
    assert np.allclose(bu, b, atol=1e-9)
    assert hu == h

    # 極狹長、病態的 UV 映射應被拒絕。
    try:
        triangle_tangent_frame(
            p0, p1, p2,
            [0, 0], [1, 0], [1, 1e-14]
        )
    except ValueError:
        pass
    else:
        raise AssertionError("病態 UV 應被拒絕")

    # 非均勻縮放。
    A = np.diag([2.0, 1.0, 0.5])
    tw, bw, nw, hw = transform_frame(t, n, h, A)
    assert np.isclose(np.dot(tw, nw), 0.0, atol=1e-9)
    assert np.isclose(np.dot(bw, nw), 0.0, atol=1e-9)

    # 極小但良態的正均勻縮放不應翻轉手性。
    A_tiny = np.diag([1e-200, 1e-200, 1e-200])
    ts, bs, ns, hs = transform_frame(t, n, h, A_tiny)
    assert np.allclose(ts, t, atol=1e-9)
    assert np.allclose(ns, n, atol=1e-9)
    assert hs == h

```

```

# 鏡射模型變換會翻轉切線框架手性。
A_mirror = np.diag([-1.0, 1.0, 1.0])
tx, bx, nx, hx = transform_frame(t, n, h, A_mirror)
assert hx == -1.0
assert np.isclose(np.dot(tx, nx), 0.0, atol=1e-9)
assert np.isclose(np.dot(bx, nx), 0.0, atol=1e-9)

# 精確奇異縮放必須拒絕。
try:
    transform_frame(t, n, h, np.diag([1.0, 0.0, 1.0]))
except ValueError:
    pass
else:
    raise AssertionError("奇異縮放應被拒絕")

print("所有測試完成")

```

實際資產通常會對共享頂點所累積的切線加權平均，再正交化。若同一位置跨越 UV 接縫、硬法線邊界或手性相反的鏡射區域，必須拆成不同頂點屬性，否則平均後可能互相抵消。

測試與預期結果

上述程式未在此執行；依公式，其預期結果如下：

1. 中性 RGB (0.5, 0.5, 1) 解碼為 (0, 0, 1)。
2. 標準 UV 三角形得到 $h = +1$ 。
3. 鏡射 UV 三角形得到 $h = -1$ 。
4. 等比例縮小至 10^{-6} 的良態 UV 島仍應被接受，且其單位切線、副切線方向與手性應與標準 UV 案例一致。求解後亦應通過 `U tb = edges` 的殘差核對。
5. 奇異值比例約為 10^{-14} 的極狹長 UV 映射應被拒絕。
6. 非均勻縮放後， $\mathbf{T}_w \cdot \mathbf{N}_w$ 與 $\mathbf{B}_w \cdot \mathbf{N}_w$ 應在容差內為零。
7. `diag( $10^{-200}$ ,  $10^{-200}$ ,  $10^{-200}$ )` 應被接受，且不翻轉手性。
8. `diag(-1, 1, 1)` 應使手性由 +1 變成 -1。
9. `diag(1, 0, 1)` 必須拋出 `ValueError`。
10. 最後一行預期印出：

所有測試完成

`UV_RTOL`、`MATRIX_RTOL` 與 `FRAME_RTOL` 都是無量綱相對容差。`VECTOR_EPS` 則是向量長度的絕對容差，必須依場景尺度與資料正規化方式設定。這些數值都不是物理安全閾值。

除錯與常見陷阱

法線貼圖被當成 sRGB

平坦區域可能仍出現明顯斜向光照。法線貼圖應作為線性資料取樣，不進行 sRGB 解碼。預覽影像時可以色彩方式顯示，但不能改變著色計算讀取的數值。

綠色通道方向相反

不同工具可能把貼圖的 y 軸定義為朝上或朝下。若凹槽看起來像凸起，可檢查是否需要 $n_y \leftarrow -n_y$ 。這是格式轉換問題，不應以任意翻轉掩蓋錯誤的 TBN。

忽略鏡射 UV 手性

若接縫一側正常、另一側光影翻轉，先檢查 h 。鏡射區與非鏡射區即使共享幾何位置，通常仍需拆開切線屬性。

直接以模型矩陣變換法線

均勻縮放或純旋轉時可能看不出錯誤，但非均勻縮放會暴露問題。法線使用 $\mathbf{A}^{-T}$ ，切線使用 $\mathbf{A}$ ，之後再正交化。

用行列式絕對值判斷奇異

固定測試 `abs(det(A)) < epsilon` 會誤拒小尺度但良態的矩陣。應使用奇異值比例或條件數判斷可逆性的數值品質。

用普通行列式值判斷手性

即使只需要正負號，極端尺度仍可能使普通行列式下溢成零。應先作正尺度正規化，再使用 `np.linalg.slogdet` 的符號；若符號為零或結果非有限，應拒絕變換。

只用 UV 面積絕對值判斷退化

小型但形狀良好的 UV 島可能被誤拒，極狹長映射也可能漏過。應檢查 UV 差分矩陣的奇異值比例，判斷的是形狀病態程度，而不只是絕對面積。

逐頂點正交但不在片段重新正規化

插值不保持單位長度，也不保證 TBN 完全正交。片段階段至少應重新正規化最終法線；需要更高品質時，應重新正交化插值後的基底。

法線翻過幾何背面

過強法線可能使 $\mathbf{n}_w \cdot \mathbf{N}_g < 0$ ，造成漏光或不穩定反射。可限制擾動、修正法線資產或採用著色法線修正策略；直接取絕對值通常會破壞方向一致性。

養殖數位分身案例

考慮合成養殖場景中的魚體與混凝土池壁：

- 魚體網格描述主要輪廓與魚鰭。
- 魚鱗使用切線空間法線貼圖，只增加高頻照明細節。
- 池壁細刮痕可由高度圖差分形成凹凸法線。
- 破損邊角或突出管線會改變輪廓與遮擋，必須建成幾何或使用實際位移。

若魚模型沿身體方向縮放以產生不同尺寸，應把切線以 $\mathbf{A}$ 變換、法線以 $\mathbf{A}^{-T}$ 變換，再重建 TBN。若左右魚身使用鏡射 UV，兩側的切線手性通常不同。

場景單位為公尺，但資產可能以不同 UV 尺度展開。小 UV 島不一定退化；應檢查其形狀條件，而不是只看 UV 面積。類似地，小型魚模型可能套用極小均勻尺度，其行列式雖小，卻不代表矩陣病態或含有鏡射。

在合成標註中也要區分：

- 深度與實例 ID 由實際幾何和可見性決定；
- 法線貼圖只影響著色法線，不應假造幾何深度；
- 若輸出「表面法線」，需註明是幾何法線、插值法線或貼圖擾動後法線。

這些影像是圖學模型的結果，不足以證明真實魚鱗、池壁磨損或水下光學已被準確重現。

習題

1. 手算題

已知

$$\mathbf{T} = (1, 0, 0), \quad \mathbf{B} = (0, 1, 0), \quad \mathbf{N} = (0, 0, 1),$$

法線貼圖值為 $\mathbf{c} = (0.75, 0.5, 1)$ 。求解碼並正規化後的世界空間法線。

2. 程式測試題

為程式加入測試，證明

$$\mathbf{A} = \text{diag}(-1, 1, 1)$$

會使 $h = +1$ 的切線框架變為 $h' = -1$ ，同時維持 TBN 互相垂直。

3. 反例與除錯題

某程式使用

```
n_world = normalize(A @ n)
```

處理法線。給出一組非均勻縮放、切線與法線，使變換後兩者不垂直，並寫出修正式。

4. 整合應用題

魚身左右兩側使用鏡射 UV，共享相同位置與法線。工程師把兩側切線直接平均，結果接縫附近切線長度接近零。說明原因，並提出資料與著色流程上的修正方案。

5. 數值穩健性題

比較下列矩陣：

$$\mathbf{A}_1 = \text{diag}(10^{-200}, 10^{-200}, 10^{-200}),$$

$$\mathbf{A}_2 = \text{diag}(1, 1, 10^{-14}).$$

說明普通行列式可能造成的問題，並以奇異值比例 $r_A = 10^{-12}$ 判斷兩者是否接受。

6. UV 判定題

比較

$$\mathbf{U}_1 = \begin{bmatrix} 10^{-6} & 0 \\ 0 & 10^{-6} \end{bmatrix}, \quad \mathbf{U}_2 = \begin{bmatrix} 1 & 0 \\ 1 & 10^{-14} \end{bmatrix}.$$

以 $r_{uv} = 10^{-12}$ 判斷兩者是否適合作為切線求解的 UV 差分矩陣。

7. 概念比較題

分別判斷下列需求應優先使用法線貼圖、凹凸貼圖或位移／幾何：

1. 大量細小魚鱗，只需影響高光。
2. 由單通道程序高度函數生成池壁細紋。
3. 池壁破口必須在剪影與深度圖中可見。

4. 表面方向由雕刻軟體直接輸出，不要求可還原為高度。

習題解答

1. 手算題解答

先映射到 $[-1, 1]$ ：

$$\tilde{\mathbf{n}}_t = 2(0.75, 0.5, 1) - (1, 1, 1) = (0.5, 0, 1).$$

其長度為

$$\sqrt{0.5^2 + 1^2} = \frac{\sqrt{5}}{2}.$$

故

$$\mathbf{n}_t = \left(\frac{1}{\sqrt{5}}, 0, \frac{2}{\sqrt{5}} \right).$$

TBN 為單位矩陣，所以

$$\mathbf{n}_w \approx (0.4472, 0, 0.8944).$$

2. 程式測試題解答

```
A_mirror = np.diag([-1.0, 1.0, 1.0])
tw, bw, nw = transform_frame(t, n, 1.0, A_mirror)

assert hw == -1.0
assert np.isclose(np.dot(tw, nw), 0.0, atol=1e-9)
assert np.isclose(np.dot(bw, nw), 0.0, atol=1e-9)
assert np.isclose(np.dot(tw, bw), 0.0, atol=1e-9)
```

因 $\det(\mathbf{A}) < 0$ ，所以

$$h' = h \operatorname{sign}(\det \mathbf{A}) = 1(-1) = -1.$$

3. 反例與除錯題解答

取

$$\mathbf{T} = (1, 1, 0), \quad \mathbf{N} = (-1, 1, 0),$$

兩者原先內積為零。令

$$\mathbf{A} = \text{diag}(2, 1, 1).$$

錯誤變換得到

$$\mathbf{T}' = (2, 1, 0), \quad \mathbf{N}'_{\text{wrong}} = (-2, 1, 0),$$

其內積為 -3 。正確方法為

$$\mathbf{N}' = \text{normalize}(\mathbf{A}^{-T} \mathbf{N}),$$

並將 $\mathbf{A}\mathbf{T}$ 相對 $\mathbf{N}'$ 重新正交化。

4. 整合應用題解答

鏡射 UV 使兩側的參數方向或手性相反。方向相反的切線直接平均後可能接近零，因而無法正規化。

修正流程為：

1. 在 UV 接縫與手性改變處拆分頂點屬性。
2. 每一側分別累積切線。
3. 相對各自的著色法線正交化。
4. 儲存 $\mathbf{T}$ 與手性 h 。
5. 片段階段以 $\mathbf{B} = h(\mathbf{N} \times \mathbf{T})$ 重建副切線。
6. 確認法線貼圖綠色通道與 UV 的 v 方向一致。

5. 數值穩健性題解答

對 $\mathbf{A}_1$ ，三個奇異值皆為 10^{-200} ，所以

$$\frac{\sigma_{\min}}{\sigma_{\max}} = 1.$$

它是良態正均勻縮放，應接受。其數學行列式為 10^{-600} ，超出一般雙精度正常可表示範圍，普通行列式可能下溢成零；因此不能用其數值正負直接判斷手性。正規化後矩陣為單位矩陣，`slogdet` 符號為 $+1$ 。

對 $\mathbf{A}_2$ ：

$$\frac{\sigma_{\min}}{\sigma_{\max}} = 10^{-14} \leq 10^{-12}.$$

它雖在精確數學上可逆，但數值上過度病態，本章實作會拒絕。

6. UV 判定題解答

U_1 的兩個奇異值皆為 10^{-6} ，所以比例為 1。它雖然面積只有 10^{-12} ，形狀仍良好，應接受。

U_2 的最大奇異值約為 $\sqrt{2}$ ，最小奇異值約為 $10^{-14}/\sqrt{2}$ ，比例約為 5×10^{-15} ，小於 10^{-12} ，應拒絕。它的問題是形狀極狹長，而不只是面積大小。

7. 概念比較題解答

1. 魚鱗高光：法線貼圖。
2. 單通道程序高度：凹凸貼圖，由高度梯度求法線。
3. 可見破口：位移或直接幾何。
4. 雕刻輸出方向：法線貼圖，因其不必對應可積分高度場。

本章小結

法線貼圖的核心不是把 RGB 當成世界方向，而是建立可靠的切線空間。由位置與 UV 可解出原始切線及副切線，再以著色法線正交化，並保存鏡射 UV 的手性。

中性法線 (0.5, 0.5, 1) 應還原原本的表面法線；法線貼圖屬於線性資料，不可套用 sRGB 解碼。非均勻縮放下，切線使用 $\mathbf{A}$ ，法線使用 $\mathbf{A}^{-T}$ 。

矩陣與 UV 映射的數值品質應以奇異值比例或條件數判定，不應只看行列式或面積的固定絕對容差。極端尺度下，手性符號應由正規化矩陣的 `slogdet` 取得，以避免普通行列式下溢。

法線與凹凸技術只修改著色方向；位移才會改變實際表面。選擇技術時，應先判斷細節是否需要影響輪廓、可見性、深度與交點。

參考來源

- PBRT 4：Transformations——法線與一般方向在變換下的差異。
- PBRT 4：Reflection Models——表面方向與反射模型的關係。
- LearnOpenGL：Transformations——圖學變換與矩陣慣例的入門說明。
- NumPy 線性代數參考——奇異值分解、具符號對數行列式與線性系統介面。
- Khronos glTF 2.0 規格——資產交換中的頂點切線與材質資料規範；使用前應依實際版本回查相關條文。

第18章 著色器與GPU管線橋接

學習目標與先備知識

完成本章後，你應能區分頂點與片段階段的工作、說明頂點屬性如何插值到像素、設計一份不含歧義的 uniform 與 buffer 契約，並用 CPU 參考程式檢查最小著色流程。先備知識是三角形重心座標、齊次裁切座標、透視除法、UV，以及線性 RGB 與 sRGB；本章會在需要處重建關鍵公式。核心實驗只需 Python 3.10+ 標準庫，不需要 GPU。

問題與直覺

前幾章已能在 CPU 上計算幾何、UV 和材質。移到 GPU 時，困難不只是把公式改寫成 GLSL：資料何時仍是逐頂點的？哪些量由光柵器插值？矩陣與頂點在記憶體中如何排列？輸出的是線性光還是可顯示的 sRGB？

把流程視為一份**階段間契約**較可靠：

1. CPU 準備頂點 buffer、索引 buffer 與每次繪製共用的 uniform。
2. 頂點著色器為每個輸入頂點產生裁切座標，並傳出 UV 等屬性。
3. 固定功能管線裁切三角形、做透視除法與光柵化，決定哪些像素中心被覆蓋。
4. 片段著色器接收插值後的屬性，算出顏色；深度測試等操作決定結果是否寫入。

頂點著色器**不會逐像素執行**；片段著色器也不能假定它收到的是某個原始頂點的 UV。本章以一塊合成池壁上的棋盤格三角形，建立兩階段的可核對版本。

數學與幾何推導

沿用本卷右手世界座標： $+X$ 向右、 $+Y$ 向上、 $+Z$ 朝觀者；相機朝局部 $-Z$ 看。位置是 column vector， $p_h = (x, y, z, 1)^T$ 。頂點階段的核心運算為

$$p_{\text{clip}} = PVMp_h,$$

其中 M, V, P 均為 4×4 矩陣，乘法由右先作用。裁切座標 $p_{\text{clip}} = (x_c, y_c, z_c, w_c)^T$ **還不是螢幕像素**。對本章採用的 OpenGL 式裁切範圍，可見頂點須位於 $-w_c \leq x_c, y_c, z_c \leq w_c$ 的裁切體內；跨越邊界的三角形須先裁切，不能只丟掉越界頂點。這裡要求 $w_c > 0$ 才進行後述透視除法。

除以 w_c 後得到 NDC： $(x_n, y_n, z_n) = (x_c, y_c, z_c)/w_c$ 。若影像寬 W 、高 H ，左上為像素原點，則連續影像座標是

$$x_s = \frac{W}{2}(x_n + 1), \quad y_s = \frac{H}{2}(1 - y_n).$$

整數像素 (u, v) 的取樣中心為 $(u + 0.5, v + 0.5)$ 。式中的 y_s 向下增加；它不是世界 Y ，也不是約定向上的 UV v 。

對螢幕三角形頂點 $s_0, s_1, s_2 \in \mathbb{R}^2$ ，像素中心的螢幕重心權重為 λ_i ，且 $\sum_i \lambda_i = 1$ 。若頂點屬性為 a_i ，直接算 $\sum_i \lambda_i a_i$ 是**螢幕仿射插值**，通常不是透視下正確的表面屬性。原因是投影中的除法依頂點 w_i 而異；要先插值除過 w_i 的量，再還原比例：

$$a = \frac{\sum_{i=0}^2 \lambda_i a_i / w_i}{\sum_{i=0}^2 \lambda_i / w_i}. \quad (18.1)$$

可以從投影前的三角形驗證此式。設其重心權重為 β_i ，則齊次位置與附著在表面的屬性分別為 $\sum_i \beta_i p_{c,i}$ 、 $\sum_i \beta_i a_i$ ，且 $\sum_i \beta_i = 1$ 。投影後，螢幕權重與 $\beta_i w_i$ 成正比；換言之， β_i 與 λ_i / w_i 成正比。用 $\sum_i \beta_i = 1$ 求比例常數，便得到 $\beta_i = (\lambda_i / w_i) / \sum_j (\lambda_j / w_j)$ 。將它代回 $\sum_i \beta_i a_i$ ，即得式 (18.1)。這也解釋了為何三頂點的 w 相等時，透視公式才退化為普通仿射插值。

UV 與世界位置等供片段使用的屬性採此規則。相對地，供深度測試的 OpenGL 式 NDC 深度在螢幕上是

$$z_n = \sum_i \lambda_i (z_{c,i} / w_i), \quad d = \frac{z_n + 1}{2}. \quad (18.2)$$

此處 $d \in [0, 1]$ ，近處較小。不要把式 (18.1) 再套到 $z_{c,i} / w_i$ 上；也不要將這套 $z_n \in [-1, 1]$ 的約定直接當作 WebGPU 的深度約定。本章的 d 對應預設深度範圍 $[0, 1]$ ；實際 API 若改設 viewport 深度範圍，NDC 到儲存深度的映射也須跟著改。裁切體外的點應先按裁切規則處理，而不是依賴將越界深度硬截到 $[0, 1]$ 。

頂點屬性隨每個頂點儲存，例如位置、UV；**uniform** 在一次繪製期間對相關著色器呼叫維持相同，例如 *PVM*、材質色。屬性插值不會替代 uniform。實務資料契約至少應列出：每個欄位的型別、單位、座標空間、位置或方向的 w 、色彩空間，以及 buffer 的位移與步長。CPU 中矩陣的列印形狀、NumPy 陣列的實際儲存順序、GPU buffer 的位元組布局，是三件不同的事；不可憑 4×4 便推定可直接複製位元組。

最後，片段顏色在**線性 RGB** 中相乘或照明。若要直接寫出供 PPM 顯示的 8-bit sRGB，每個已截到 $[0, 1]$ 的線性分量 c 轉為

$$S(c) = \begin{cases} 12.92c, & c \leq 0.0031308, \\ 1.055c^{1/2.4} - 0.055, & c > 0.0031308. \end{cases}$$

法線、深度及 ID 是資料，不做這項色彩轉換。

逐步手算例題

例一：像素位置與透視 UV。 設 $W = H = 16$ ，三個頂點經頂點階段與透視除法後，螢幕座標依序是 $s_0 = (2, 2)$ 、 $s_1 = (13, 2)$ 、 $s_2 = (2, 13)$ ；其 w 分別是 1, 1, 2，UV 分別是 $(0, 0)$, $(1, 0)$, $(0, 1)$ 。這些點也可反推其 NDC，例如 s_0 對應 $(x_n, y_n) = (-3/4, 3/4)$ ；第三點的 $w = 2$ ，故其 $x_c = wx_n$ ，不能直接把 x_n 當裁切座標。

在像素 $(5, 5)$ 的中心 $(5.5, 5.5)$ ，沿兩條直角邊各前進 $3.5/11 = 7/22$ ，因此

$$(\lambda_0, \lambda_1, \lambda_2) = (8/22, 7/22, 7/22).$$

仿射 UV 會是 $(7/22, 7/22)$ 。但式 (18.1) 的分母為

$$\frac{8}{22} + \frac{7}{22} + \frac{7/22}{2} = \frac{37}{44},$$

所以正確結果是 $U = 14/37$ 、 $V = 7/37$ 。兩者的 V 分別約為 0.318 與 0.189；在高頻貼圖上可能落入不同格子。

例二：片段色與深度。 將例一的 UV 取樣於每軸四格的棋盤：格號為 $\lfloor 4U \rfloor, \lfloor 4V \rfloor$ 。正確 UV 的格號是 $(1, 0)$ ，指定該格的線性 RGB 為 $(0.25, 0.5, 1)$ ，材質乘色為 $(1, 1, 1)$ 。逐通道套用 S 、乘 255 並四捨五入，8-bit 值約為 $(137, 188, 255)$ ，標為**預期**而非實測。若三個頂點的 z_c 都是 0，式 (18.2) 給 $z_n = 0$ 、深度 $d = 0.5$ 。色彩編碼與深度映射是不同運算。

實作與程式

下例以標準庫產生 `bridge.ppm`，並在寫檔前做數值斷言。為聚焦階段契約，它只畫一個完全位於裁切體內的三角形：沒有實作近平面裁切、多三角形共邊的 top-left 歸屬規則或完整 GPU 驅動。`vertex` 對應頂點階段；迴圈中的覆蓋與插值對應光柵化；`fragment` 對應片段階段。矩陣先用單位矩陣，但仍保留明確的 column-vector 乘法位置。`make_clip` 是為隔離投影與插值測試而由螢幕位置反構裁切座標；一般場景應由 PVM_{p_h} 產生裁切座標，不應把反構函式當作相機管線。

```
# Python 3.10+；標準庫；執行後寫出 bridge.ppm
import math

W = H = 16
MVP = tuple(
    tuple(1.0 if r == c else 0.0 for c in range(4))
    for r in range(4)
)
TINT = (1.0, 1.0, 1.0) # 線性 RGB，繪製期間固定

def matvec(m, v):
    return tuple(sum(m[r][c] * v[c] for c in range(4))
                  for r in range(4))

def make_clip(sx, sy, w):
```

```

# 由指定螢幕位置構造完全可見的裁切座標；z_c = 0
xn = 2.0 * sx / W - 1.0
yn = 1.0 - 2.0 * sy / H
return (xn * w, yn * w, 0.0, w)

# 每筆頂點：[位置 vec4, UV vec2]；UV 的 v 向上
vertices = (
    (make_clip(2, 2, 1), (0.0, 0.0)),
    (make_clip(13, 2, 1), (1.0, 0.0)),
    (make_clip(2, 13, 2), (0.0, 1.0)),
)
indices = (0, 1, 2)

def vertex(position, uv):
    return matvec(MVP, position), uv

def screen(clip):
    x, y, z, w = clip
    if w <= 0:
        raise ValueError("本例要求正的 clip w")
    return (W * (x / w + 1) / 2, H * (1 - y / w) / 2)

def edge(a, b, p):
    return ((b[0] - a[0]) * (p[1] - a[1])
            - (b[1] - a[1]) * (p[0] - a[0]))

def fragment(uv):
    u, v = uv
    cell = (math.floor(4 * u) + math.floor(4 * v)) % 2
    base = (0.25, 0.5, 1.0) if cell else (1.0, 0.25, 0.25)
    return tuple(base[k] * TINT[k] for k in range(3))

def to_byte(linear):
    c = max(0.0, min(1.0, linear))
    srgb = 12.92 * c if c <= 0.0031308 else 1.055 * c**(1/2.4) - 0.055
    return round(255 * srgb)

shaded = [vertex(*vertices[i]) for i in indices]
clips = [item[0] for item in shaded]
uvs = [item[1] for item in shaded]
points = [screen(c) for c in clips]
area = edge(points[0], points[1], points[2])
assert abs(area) > 1e-12

pixels = bytearray(W * H * 3) # 背景為黑
depth = [1.0] * (W * H)
center_uv = None

for py in range(H):
    for px in range(W):
        p = (px + 0.5, py + 0.5)
        lam = (
            edge(points[1], points[2], p) / area,
            edge(points[2], points[0], p) / area,
            edge(points[0], points[1], p) / area,
        )
        # 單一三角形：不處理共邊像素的 top-left 所有權
        if min(lam) < -1e-12:
            continue

```

```

    invw = [1.0 / c[3] for c in clips]
    denom = sum(lam[i] * invw[i] for i in range(3))
    uv = tuple(
        sum(lam[i] * uvs[i][k] * invw[i] for i in range(3)) / denom
        for k in range(2)
    )
    zn = sum(lam[i] * clips[i][2] * invw[i] for i in range(3))
    d = (zn + 1.0) / 2.0
    j = py * W + px
    if not (0.0 <= d <= 1.0) or d >= depth[j]:
        continue
    depth[j] = d
    rgb = fragment(uv)
    pixels[3*j:3*j+3] = bytes(to_byte(c) for c in rgb)
    if (px, py) == (5, 5):
        center_uv = uv

assert center_uv is not None
assert all(abs(a - b) < 1e-12 for a, b in
           zip(center_uv, (14/37, 7/37)))
assert tuple(pixels[3*(5*W+5):3*(5*W+5)+3]) == (137, 188, 255)
assert tuple(pixels[:3]) == (0, 0, 0)
with open("bridge.ppm", "wb") as f:
    f.write(f"P6\n{W} {H}\n255\n".encode("ascii"))
    f.write(pixels)

```

這份例子的階段契約可以濃縮成下表；「每頂點」與「每次繪製」不能互換：

資料	型別與數量	所屬空間或意義	更新頻率
position	每頂點四個浮點數	本測試為預造的裁切座標	每頂點
uv	每頂點兩個浮點數	無單位， v 約定向上	每頂點
MVP	4×4 浮點矩陣	本測試為單位矩陣；一般為 PVM	每次繪製
TINT	三個浮點數	線性 RGB 乘色	每次繪製
depth	每像素一個浮點數	OpenGL 式映射後深度	每片段測試

程式讓螢幕下方的 s_2 配到 UV $v = 1$ ，是刻意指定的棋盤座標，並不表示螢幕向下等於 UV 向上。若改接按影像上列起始儲存的貼圖，須依貼圖與 UV 的定義明確翻轉列號。本例的 `edge` 在向下的螢幕座標算得正面積；判斷世界空間三角形前面是否按外向法線為逆時針，不能只看這個螢幕符號。

下面是與參考程式對應的 **GLSL 式偽最小流程**，供閱讀資料流，不聲稱可在任一特定 GPU／驅動上直接編譯或已實測。CPU 端的 `make_clip` 已預先造出位置，故此處的 `uMVP` 為單位矩陣；一般場景才把 PVM 放入其中。

```

// 頂點階段：概念性 GLSL
layout(location = 0) in vec4 aPosition;
layout(location = 1) in vec2 aUV;
uniform mat4 uMVP;

```

```

out vec2 vUV;
void main() {
    gl_Position = uMVP * aPosition;
    vUV = aUV;          // 預設平滑、透視正確插值
}

// 片段階段：概念性 GLSL，與上段分屬不同著色器
in vec2 vUV;
uniform vec3 uTint;      // 線性 RGB
out vec4 outColor;
void main() {
    float cell = mod(floor(4.0*vUV.x) + floor(4.0*vUV.y), 2.0);
    vec3 base = cell > 0.5
        ? vec3(0.25, 0.5, 1.0) : vec3(1.0, 0.25, 0.25);
    outColor = vec4(base * uTint, 1.0); // 線性輸出
}

```

此處 `vUV` 採預設平滑插值，對應式 (18.1)。若把它改為 `flat`，片段會取得某個指定頂點的值而非連續插值；這適合不應產生中間值的面 ID，卻不適合此處的棋盤 UV。離散 ID 還須選擇能保留整數標籤的輸出與讀回方式，不能只靠改插值修飾詞。

兩者的**色彩出口刻意不同**：CPU 在寫 PPM 前顯式編成 sRGB；偽 GLSL 輸出線性值，需由實際渲染目標的設定負責恰好一次線性至 sRGB 轉換。不能同時在片段中手動編碼，又啟用另一道自動編碼。

若改用實際 GPU buffer，建議先寫規格表而不是猜位址。例如交錯頂點可定為每頂點六個連續 32-bit float：位置四個、UV 兩個；步長 $6 \times 4 = 24$ bytes，位置起點偏移 0，UV 偏移 16 bytes，索引另存。若 uniform 改用 `std140` 區塊，單一 `mat4` 可列為偏移 0、占 64 bytes，隨後的 `vec4 tint` 偏移 64、區塊至少 80 bytes；這與上面使用獨立 `mat4`、`vec3` uniform 的偽碼是**兩種不同介面設計**，不可混接。上傳矩陣時還必須核對 API 的矩陣記憶體布局及轉置設定，維持數學上的 $uMVPp$ 。

測試與預期結果

執行前可先以例一獨立算出 (14/37, 7/37)。程式的斷言分別檢查透視 UV、中心像素顏色，以及未覆蓋的左上像素；若都成立，應寫出標頭為 P6、尺寸 16 16、最大值 255 的 PPM 檔。這些是**預期結果**，不是本章曾執行的紀錄。可用支援 PPM 的本機檢視器選擇性觀看；影像方向不應藉檢視器是否自動翻轉來猜。

再做兩個有針對性的修改測試：

- 把第三個頂點的 w 改成 1，同時仍用 `make_clip` 保持三個螢幕位置不變；此時中心 UV 應改為 (7/22, 7/22)，原本檢查 14/37, 7/37 的斷言應失敗。
- 保持幾何不變，暫時把 UV 計算整段換成 `tuple(sum(lam[i]*uvs[i][k] for i in range(3)) for k in range(2))`。這才是刻意使用螢幕仿射插值的對照；其中心結果應為 (7/22, 7/22)，有助於定位「忘記透視校正」的錯誤。

精度測試宜分層進行。Python 一般浮點數通常採 64-bit 二進位浮點；GPU 路徑可能以 32-bit 等精度計算。就 1 附近的表示間距而言，binary32 約為 2^{-23} ，binary64 約為 2^{-52} ；這只說明表示尺度，**不能**據此保證完整光柵與貼圖流程的最終誤差。讀者可先比較連續的 UV，再比較棋盤格號，最後才比較 sRGB 量化

位元組；若 UV 恰在格界附近，微小誤差也可能令格號突變。本例中心 UV 不在格界上，較適合作為入門對照。

除錯與常見陷阱

- ****整張圖倒置：****先查 NDC 至像素的 y_s 公式，再查讀入影像時是否將向下的影像列號轉成向上的 UV v ；兩次翻轉也會造成錯誤。
- ****貼圖沿斜面漂移：****核對是否把 UV 當 `flat` 屬性、採仿射插值，或錯把 w 寫成投影後的 z 。
- ****前後遮擋錯亂：****核對比較的是 $d = (z_n + 1)/2$ ，不是相機 z ；深度清除值在本例是 1。近平面外的頂點須裁切，本程式沒有提供替代辦法。
- ****顏色偏亮或偏暗：****核對材質乘法是否在線性 RGB，及輸出是否被轉成 sRGB **恰好一次**。normal/depth buffer 不做此轉換。
- ****GPU 讀到扭曲數值：****檢查 stride、欄位 byte offset、float 型別、矩陣轉置及 uniform 布局。不同著色器宣告看似同名，不保證與 CPU 位元組契約相符。
- ****邊緣裂縫或重畫：****本例只測一個三角形，使用容差判覆蓋；多三角形共邊時須採一致的 top-left 規則。浮點 epsilon 是依尺度選的數值容差，不是幾何或物理安全界線。
- ****精度不穩：****CPU 的 Python 浮點計算與常見 GPU 著色器精度不必逐 bit 相同。比較時先用容差檢查連續量，再單獨檢查量化後的色階；不要把一致的公式誤寫成保證相同的每像素整數。

養殖數位分身案例

假設合成池壁網格以公尺建模，局部頂點經 M 放入世界，再由 V 、 P 投影。池壁每頂點存 UV；「材質色」作線性 uniform；水位標註、魚隻 ID 則另走資料輸出，不應沿用 sRGB 色彩處理。可先挑一個完全可見的池壁三角形，用本章 CPU 流程製作低解析度參考圖，再以相同相機矩陣、頂點欄位和材質規則設計可選的 GPU 版本。

比較時記錄每一層契約：網格位置單位是公尺；UV 的 v 向上；矩陣按 PVM_p 作用；裁切與深度採 OpenGL 式範圍；參考 PPM 已編成 sRGB。這樣即使兩張圖不同，也能先判別差異來自幾何、插值、buffer 或色彩，而非把渲染外觀誤稱為養殖環境的實測驗證。

習題

1. **手算。** 沿用例一的三角形與像素中心。求透視正確的 V 、仿射 V ；若所有頂點 $z_c = 0$ ，求深度 buffer 值。說明哪個值用來選棋盤格。
2. **程式測試。** 不改螢幕三角形及 UV，將第三頂點的 w 設為 1。應修改哪一行斷言？中心 UV 的預期值為何？如何確認背景仍為黑？
3. **反例／除錯。** 某實作把像素中心 UV 算成 $(7/22, 7/22)$ ，而 GPU 偽流程預期透視插值；另一實作把線性值 0.25 直接乘 255 寫入 PPM。分別指出錯誤及修法。
4. **整合應用。** 規劃一個池壁三角形的 GPU 輸入契約：每頂點有 `vec4 position`、`vec2 uv`，另有 PVM 與線性材質乘色。給出交錯頂點的步長與 UV 偏移，列出從局部位置到 sRGB 輸出的順序，並

指出深度圖及魚隻 ID 圖與色彩圖的一項處理差異。

習題解答

1. 分母是 $37/44$ ； $V = (\lambda_2/2)/(37/44) = 7/37$ 。仿射 $V = \lambda_2 = 7/22$ 。由 $z_c = 0$ 得 $z_n = 0$ 、 $d = 0.5$ 。棋盤格須使用透視正確的 $(U, V) = (14/37, 7/37)$ ，而非仿射 UV 或深度。
2. `make_clip(2, 13, 2)` 改成 `make_clip(2, 13, 1)`；把 UV 斷言的目標改為 $(7/22, 7/22)$ 。
`assert tuple(pixels[:3]) == (0, 0, 0)` 仍檢查左上背景。原中心顏色斷言也會因棋盤格變動而失敗：此時兩個格號都是 1，其和為偶數，線性基色改為 $(1, 0.25, 0.25)$ ，故應同步更新顏色測試，而不是只改 UV 測試。
3. 第一個結果直接使用螢幕重心座標做仿射 UV；依式 (18.1) 插值 UV/w 與 $1/w$ ，相除可修正。第二個結果把線性光當作 sRGB；先套用分段函數 $S(0.25)$ ，再乘 255 並量化，預期約為 137，不是直接量化所得約 64。
4. 若每欄位均為 32-bit float，交錯步長為 24 bytes，UV 起點偏移為 16 bytes。順序為：局部位置 → PVM_p → 裁切 → 透視除法與光柵化 → 透視正確 UV → 在線性 RGB 中計算材質色 → 恰好一次 sRGB 編碼供顯示。深度依約定映射後供深度測試／儲存，魚隻 ID 保持離散標籤；兩者都不當作線性顏色去做 sRGB 編碼。

本章小結

著色器橋接的核心是明訂資料與階段契約。頂點階段輸出裁切座標及屬性；光柵器負責覆蓋、深度相關量與透視正確插值；片段階段在指定色彩空間內計算結果。CPU 參考圖能檢查公式與布局設計，但不等於已完成 GPU 實測；進入真實 API 時，裁切規則、buffer 位元組布局和色彩出口仍須逐項核對。

參考來源

- [G1] *PBRT 4 : Transformations*, https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] *LearnOpenGL : Transformations*, <https://learnopengl.com/Getting-started/Transformations>

以上供讀者回查變換背景；本章的 CPU／GLSL 對照、數值與測試契約以本文定義為準。

第19章 射線、交點與數值穩健性

學習目標與先備知識

本章建立 CPU 端的最小射線光追基礎，核心在於**正確判定幾何相交與處理數值邊界**。讀者應已具備 Volume I 的向量運算與第 3 章的齊次座標基礎。

核心技能目標：

1. 定義射線參數 $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ ，理解 t 的物理意義（距離）與數學意義（參數）。
2. 推導並實作射線與球面、平面、三角形的解析解。
3. 掌握穩定二次求根算法，避免浮點消去誤差（Catastrophic Cancellation）。
4. 理解並實作「起點法線偏移」（Origin Epsilon）與「射線下界」（ t_{min} ）在避免自相交中的作用。
5. 建構一個可生成 16×16 PPM 影像的最小 Ray Caster，並包含單元測試。

單位與約定：

- 長度：公尺 (m)。
- 座標系：右手系，+Z 由畫面向觀者。相機局部看向 -Z。
- 像素：原點左上，中心 $(u + 0.5, v + 0.5)$ 。
- 色彩：本例直接輸出顯示用 RGB 端點值，不進行光學或線性光色彩計算；這些數值僅供示意，不應當作線性輻射量。

問題與直覺

在光柵化中，我們掃描像素並檢查哪些三角形覆蓋該像素；在光追中，我們從像素中心發射射線，尋找第一個命中的幾何體。

為什麼需要數值穩健性？

1. **自相交（Self-Intersection）**：射線從表面點 $\mathbf{p}$ 出發時，浮點表示與求交計算可能使原表面再次被判為命中。起點偏移與 t_{min} 是兩種不同的處理方式：前者改變起點，後者限制可接受的射線參數。
2. **球面相切與判別式**：球面求交會得到二次方程。相切時判別式 Δ 為零；有限精度下，接近零的值可能因誤差變號，造成相切被誤判為相交或未相交。
3. **二次方程消去誤差**：當 $|b|$ 與 $\sqrt{\Delta}$ 接近時，直接套用 $\frac{-b \pm \sqrt{\Delta}}{2a}$ 的其中一個分子可能是兩個近似數相減，損失有效位元。這與判別式接近零是不同問題，穩定求根可減少此類消去。
4. **平面平行**：平面求交不是二次方程。當射線方向與平面法線內積接近零，交點公式分母接近零， t 可能不穩定或不存在。

直覺模型：想像雷射筆從鏡面上的點出發。數值計算可能讓它立即再次命中同一表面，形成自相交。可用起點偏移將射線原點沿適當法線方向移開，或設定 t_{min} 排除起點附近的命中參數；兩者可併用，但都只是幾何計算的數值處理，不是物理安全閾值。偏移量與參數下界需依場景尺度、座標精度及物件尺寸選擇，不能將固定的 10^{-6} 公尺視為通用常數。

數學與幾何推導

射線參數化

射線是起點與方向定義的一維集合。本文令方向正規化，使參數 t 的量綱為公尺；若輸入未正規化方向，程式會先正規化，因此同一條幾何射線上的參數值會依方向縮放而改變。這也是求交函式內部採用一致參數尺度的原因。

射線區間可採閉區間或半開區間，必須由API明確決定。本章程式的球、平面與三角形交點均以 $t_{min} \leq t \leq t_{max}$ 接受端點；`shadow_test` 為了排除光源端點，額外要求命中滿足 $t_{min} < t < t_{max}$ 。在場景中尋找最近物體時，應比較所有有效命中的 t ，選取最小值，而不能以物件輸入順序決定可見表面。

定義射線：

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \in \mathbb{R}$$

其中 $\mathbf{o}$ 為起點， $\mathbf{d}$ 為單位方向向量 ($\|\mathbf{d}\| = 1$)。

- 若 $\mathbf{d}$ 未正規化， t 的單位不再是公尺，所有容差判斷需調整。本節假設 $\mathbf{d}$ 已正規化。
- 有效射線區間定義為 $[t_{min}, t_{max}]$ 。通常 $t_{min} > 0$ 用於避免自相交， $t_{max} = \infty$ 或光源距離。

射線與球面求交

球心 $\mathbf{c}$ ，半徑 R 。交點滿足 $\|\mathbf{r}(t) - \mathbf{c}\|^2 = R^2$ 。令 $\mathbf{m} = \mathbf{o} - \mathbf{c}$ ，展開得：

$$(\mathbf{m} + t\mathbf{d}) \cdot (\mathbf{m} + t\mathbf{d}) = R^2$$

$$\mathbf{m} \cdot \mathbf{m} + 2t(\mathbf{m} \cdot \mathbf{d}) + t^2(\mathbf{d} \cdot \mathbf{d}) = R^2$$

因 $\|\mathbf{d}\| = 1$ ，整理為 $at^2 + bt + c = 0$ ：

$$a = 1, \quad b = 2\mathbf{m} \cdot \mathbf{d}, \quad c = \|\mathbf{m}\|^2 - R^2$$

數值穩定求根：判別式 $\Delta = b^2 - 4ac$ 。若 $\Delta < 0$ ，無交點。若 $\Delta \approx 0$ ，需容差處理。直接計算 $t = \frac{-b \pm \sqrt{\Delta}}{2a}$ 時，若 b 與 $\sqrt{\Delta}$ 符號相反且大小接近，會發生消去誤差。

- 若 $b > 0$ ， $-b < 0$ ， $\sqrt{\Delta} > 0$ 。 $-b + \sqrt{\Delta}$ 是兩個負/正小數相加（若 $\sqrt{\Delta} \approx b$ ），可能抵消。應使用 $-b - \sqrt{\Delta}$ （兩負數相加，結果大且穩定）。

- 若 $b < 0$ ， $-b > 0$ ， $\sqrt{\Delta} > 0$ 。 $-b - \sqrt{\Delta}$ 是兩個正數相減，可能抵消。應使用 $-b + \sqrt{\Delta}$ （兩正數相加，結果大且穩定）。

穩定算法：

1. 計算 $q = -0.5 \cdot (b + \text{copysign}(\sqrt{\Delta}, b))$ 。
 - 若 $b > 0$ ， $\text{copysign}(\sqrt{\Delta}, b) = \sqrt{\Delta}$ ， $q = -0.5(b + \sqrt{\Delta})$ 。
 - 若 $b < 0$ ， $\text{copysign}(\sqrt{\Delta}, b) = -\sqrt{\Delta}$ ， $q = -0.5(b - \sqrt{\Delta})$ 。
2. 若 $q \neq 0$ ，兩根為 $t_0 = q/a$ 和 $t_1 = c/q$ 。
3. 若 $q = 0$ ，不可計算 c/q ；直接使用雙根 $-b/(2a)$ 。一般相切由經容差處理後的 $\Delta = 0$ 判定，並不必然使 $q = 0$ 。
4. 對可計算的根排序，並檢查是否落在射線區間 $[t_{\min}, t_{\max}]$ 。若區間端點要排除，必須在實作中改用嚴格不等式；本程式採含端點的閉區間判斷。

容差尺度： Δ 的單位是 m^2 （因為 b 是長度、 c 是長度平方）。容差須與這些項同量綱；本章以球半徑 $R > 0$ 作特徵長度，取 $\tau_{\Delta} = 10^{-12} \max(b^2, 4|ac|, R^2)$ 。括號內每項都是長度平方；若整個場景及球半徑同時改用另一種長度單位表示，此容差也隨長度平方縮放。若 $\Delta < -\tau_{\Delta}$ ，判無交點；若 $-\tau_{\Delta} \leq \Delta < 0$ ，將 Δ 視為零處理近相切。係數 10^{-12} 仍只是本例的數值選擇，不能保證所有浮點尺度下的判定。

射線與平面求交

平面定義： $\mathbf{n} \cdot \mathbf{p} + d = 0$ ，其中 $\mathbf{n}$ 為單位法線。

$$\mathbf{n} \cdot (\mathbf{o} + t\mathbf{d}) + d = 0 \implies t = -\frac{\mathbf{n} \cdot \mathbf{o} + d}{\mathbf{n} \cdot \mathbf{d}}$$

邊界檢查：

- 若 $|\mathbf{n} \cdot \mathbf{d}| < \tau_{\text{par}}$ （例如 10^{-8} ），視為平行，返回無交點。
- 注意：此處 $\mathbf{n}$ 必須正規化，否則 τ_{par} 的意義隨法線長度改變。

射線與三角形求交（Möller-Trumbore）

頂點 $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$ 。

1. $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0, \mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$ 。
2. $\mathbf{p} = \mathbf{d} \times \mathbf{e}_2$ 。
3. $\text{det} = \mathbf{e}_1 \cdot \mathbf{p}$ 。
 - 若 $|\text{det}| < \tau_{\text{det}}$ ，平行或退化。 τ_{det} 應與三角形面積尺度相關，例如 $10^{-8} \cdot \|\mathbf{e}_1\| \|\mathbf{e}_2\|$ 。
4. $\text{invDet} = 1.0/\text{det}$ 。
5. $\mathbf{tvec} = \mathbf{o} - \mathbf{v}_0$ 。
6. $u = (\mathbf{tvec} \cdot \mathbf{p}) \cdot \text{invDet}$ 。
7. $\mathbf{q} = \mathbf{tvec} \times \mathbf{e}_1$ 。
8. $v = (\mathbf{d} \cdot \mathbf{q}) \cdot \text{invDet}$ 。

9. $t = (\mathbf{e}_2 \cdot \mathbf{q}) \cdot \text{invDet}$ 。
10. 檢查 $u \geq -\tau_{bary}, v \geq -\tau_{bary}, u + v \leq 1 + \tau_{bary}$ 且 $t \in [t_{min}, t_{max}]$ 。
 - τ_{bary} 為無因次容差（例如 10^{-4} ）。
 - 回傳重心座標 $(w_0, w_1, w_2) = (1 - u - v, u, v)$ 與交點。

自相交處理：起點偏移與 t_{min}

起點法線偏移：若新射線從交點 $\mathbf{p}$ 沿方向 $\mathbf{d}_{new}$ 發射，幾何法線為 $\mathbf{n}_g$ 。

$$\mathbf{o}_{new} = \mathbf{p} + \epsilon_{orig} \cdot \text{sign}(\mathbf{d}_{new} \cdot \mathbf{n}_g) \cdot \mathbf{n}_g$$

若內積為 0，通常選 +1 或依上下文決定。 ϵ_{orig} 為小距離（如 10^{-4} m）。

射線下界 t_{min} ：即使起點偏移，數值誤差仍可能存在。因此求交函式應接受 t_{min} ，僅接受 $t > t_{min}$ 的解。

- 對於陰影射線， t_{max} 設為光源距離減去端點容差。

逐步手算例題

例題 1：射線與球面（穿越與內部出射）

場景 A：外部穿越 球心 $\mathbf{c} = (0, 0, 0)$ 、半徑 $R = 1$ ，射線起點 $\mathbf{o} = (0, 0, -5)$ ，單位方向 $\mathbf{d} = (0, 0, 1)$ ，區間為 $[10^{-4}, \infty)$ 。

令 $\mathbf{m} = \mathbf{o} - \mathbf{c} = (0, 0, -5)$ ，得到 $a = 1$ 、 $b = -10$ 、 $c = 24$ ，故

$$\Delta = b^2 - 4ac = 100 - 96 = 4, \quad \sqrt{\Delta} = 2.$$

$\text{copysign}(2, b) = -2$ ，所以 $q = -0.5(-10 - 2) = 6$ 。兩根為 $q/a = 6$ 與 $c/q = 4$ ；排序後是 4、6。近根 4 落在射線區間內，交點為 $(0, 0, -1)$ 。

場景 B：內部出射

- 球心 $\mathbf{c} = (0, 0, 0)$ ， $R = 1$ 。
- 射線 $\mathbf{o} = (0, 0, 0)$ ， $\mathbf{d} = (1, 0, 0)$ ， $t_{min} = 10^{-4}$ 。

計算： $\mathbf{m} = (0, 0, 0)$ ，因此 $a = 1$ 、 $b = 0$ 、 $c = -1$ ，判別式 $\Delta = 4$ 。依穩定公式， $q = -0.5(0 + 2) = -1$ ，兩根為 $q/a = -1$ 與 $c/q = 1$ 。在閉區間 $[10^{-4}, \infty)$ 中，負根被排除，正根命中，交點為 $(1, 0, 0)$ 。

例題 2：射線與三角形（Möller-Trumbore）

場景：

- $\mathbf{v}_0 = (0, 0, 0)$ ， $\mathbf{v}_1 = (1, 0, 0)$ ， $\mathbf{v}_2 = (0, 1, 0)$ 。

- $\mathbf{o} = (0.2, 0.2, -1)$, $\mathbf{d} = (0, 0, 1)$ 。

計算： $\mathbf{e}_1 = (1, 0, 0)$, $\mathbf{e}_2 = (0, 1, 0)$ 。

$\mathbf{p} = \mathbf{d} \times \mathbf{e}_2 = (0, 0, 1) \times (0, 1, 0) = (-1, 0, 0)$ 。

$\det = \mathbf{e}_1 \cdot \mathbf{p} = -1$ 。

$|det| = 1 > \tau_{det}$ 。

$invDet = -1$ 。

$\mathbf{tvec} = (0.2, 0.2, -1)$ 。

$u = (\mathbf{tvec} \cdot \mathbf{p}) \cdot invDet = (-0.2) \cdot (-1) = 0.2$ 。

$\mathbf{q} = \mathbf{tvec} \times \mathbf{e}_1 = (0.2, 0.2, -1) \times (1, 0, 0) = (0, -1, -0.2)$ 。

$v = (\mathbf{d} \cdot \mathbf{q}) \cdot invDet = ((0, 0, 1) \cdot (0, -1, -0.2)) \cdot (-1) = (-0.2) \cdot (-1) = 0.2$ 。

現在 $u = 0.2, v = 0.2$ 。

$u \geq 0, v \geq 0, u + v = 0.4 \leq 1$ 。

OK 。

$t = (\mathbf{e}_2 \cdot \mathbf{q}) \cdot invDet = ((0, 1, 0) \cdot (0, -1, -0.2)) \cdot (-1) = (-1) \cdot (-1) = 1$ 。

$t > t_{min}$ 。

結果： $t = 1$, 交點 $(0.2, 0.2, 0)$ 。

重心 $(0.6, 0.2, 0.2)$ 。

實作與程式

以下程式包含完整的 Ray Caster、PPM 輸出與測試，使用 NumPy 陣列儲存三維向量。球面二次式採 $a = 1$ ，因 Ray 建構時會將方向正規化。對球面判別式使用相對容差：尺度項至少包含 b^2 與 $|4ac|$ ，避免僅以固定絕對 epsilon 套用於不同尺寸場景。平面法線在建構時正規化並同步縮放平面常數 d ；三角形 determinant 的容差依兩條邊向量長度縮放，退化三角形因此不會除以零。

命中函式回傳的 t 在正規化方向下以公尺計。若場景物件有重疊，渲染迴圈以最小有效 t 決定可見物件。球與平面只回傳最近的有效根；三角形則回傳命中距離、交點、重心座標與 face-forward 法線。

```
import numpy as np
from typing import Optional, Tuple

def vector3(value):
    result = np.asarray(value, dtype=float)
    if result.shape != (3,) or not np.all(np.isfinite(result)):
        raise ValueError("Expected a finite three-component vector")
    return result

class Ray:
    def __init__(self, origin: np.ndarray, direction: np.ndarray, t_min: float = 1e-4, t_max: float = float('inf')):
        origin = vector3(origin)
        direction = vector3(direction)
        if not np.isfinite(t_min) or t_min < 0.0:
            raise ValueError("t_min must be finite and non-negative")
        if np.isnan(t_max) or t_max < t_min:
            raise ValueError("t_max must not be NaN or less than t_min")
        self.origin = origin
        length = np.linalg.norm(direction)
        if not np.isfinite(length) or length <= 0.0:
            raise ValueError("Ray direction must be finite and non-zero")
        self.direction = direction / length
        self.t_min = t_min
        self.t_max = t_max

class Sphere:
    def __init__(self, center: np.ndarray, radius: float, color: np.ndarray = None):
        center = vector3(center)
        if not np.isfinite(radius) or radius <= 0:
```

```

        raise ValueError("Invalid sphere parameters")
    self.center = center
    self.radius = radius
    self.color = color if color is not None else np.array([1.0, 0.0, 0.0])

class Plane:
    def __init__(self, normal: np.ndarray, d: float, color: np.ndarray = None):
        normal = vector3(normal)
        if not np.isfinite(d):
            raise ValueError("Plane offset must be finite")
        length = np.linalg.norm(normal)
        if not np.isfinite(length) or length <= 0.0:
            raise ValueError("Plane normal must be finite and non-zero")
        self.normal = normal / length
        self.d = d / length # 必須同步縮放 d
        self.color = color if color is not None else np.array([0.0, 1.0, 0.0])

class Triangle:
    def __init__(self, v0: np.ndarray, v1: np.ndarray, v2: np.ndarray, color: np.ndarray = None):
        v0, v1, v2 = vector3(v0), vector3(v1), vector3(v2)
        self.v0 = v0
        self.v1 = v1
        self.v2 = v2
        self.color = color if color is not None else np.array([0.0, 0.0, 1.0])

def intersect_sphere(ray: Ray, sphere: Sphere) -> Optional[float]:
    m = ray.origin - sphere.center
    a = 1.0
    b = 2.0 * np.dot(m, ray.direction)
    c = np.dot(m, m) - sphere.radius**2

    delta = b*b - 4*a*c
    # 相對容差
    scale = max(b*b, abs(4*a*c), sphere.radius**2)
    tol = 1e-12 * scale

    if delta < -tol:
        return None

    if delta < 0:
        delta = 0.0

    sqrt_delta = np.sqrt(delta)

    # 穩定求根
    if b > 0:
        t0 = (-b - sqrt_delta) / (2*a)
    else:
        t0 = (-b + sqrt_delta) / (2*a)

    roots = []
    if t0 >= ray.t_min and t0 <= ray.t_max:
        roots.append(t0)

    if t0 != 0:
        t1 = c / t0
    else:
        t1 = -b / (2*a)

```



```

    if t1 >= ray.t_min and t1 <= ray.t_max:
        roots.append(t1)

    if not roots:
        return None
    return min(roots)

def intersect_plane(ray: Ray, plane: Plane) -> Optional[float]:
    denom = np.dot(ray.direction, plane.normal)
    if abs(denom) < 1e-8:
        return None
    t = -np.dot(ray.origin, plane.normal) - plane.d
    t /= denom
    if t >= ray.t_min and t <= ray.t_max:
        return t
    return None

def intersect_triangle(ray: Ray, tri: Triangle) -> Optional[Tuple[float, np.ndarray, Tuple[float, float, float], np.ndarray]]:
    e1 = tri.v1 - tri.v0
    e2 = tri.v2 - tri.v0
    p = np.cross(ray.direction, e2)
    det = np.dot(e1, p)

    scale = np.linalg.norm(e1) * np.linalg.norm(e2)
    if scale == 0:
        return None
    det_tol = 1e-8 * scale

    if abs(det) < det_tol:
        return None

    inv_det = 1.0 / det
    tvec = ray.origin - tri.v0

    u = np.dot(tvec, p) * inv_det
    if u < -1e-4 or u > 1.0 + 1e-4:
        return None

    q = np.cross(tvec, e1)
    v = np.dot(ray.direction, q) * inv_det
    if v < -1e-4 or u + v > 1.0 + 1e-4:
        return None

    t = np.dot(e2, q) * inv_det
    if t >= ray.t_min and t <= ray.t_max:
        w0 = 1.0 - u - v
        w1 = u
        w2 = v
        point = w0 * tri.v0 + w1 * tri.v1 + w2 * tri.v2
        n = np.cross(e1, e2)
        n = n / np.linalg.norm(n)
        if np.dot(n, ray.direction) > 0:
            n = -n
        return t, point, (w0, w1, w2), n
    return None

def render(width: int, height: int, objects: list, cam_pos: np.ndarray, cam_dir: np.ndarray, fov_v: float = 90.0) -> np.ndarray:

```

```

if not isinstance(width, int) or not isinstance(height, int) or width <= 0 or height <= 0:
    raise ValueError("Image dimensions must be positive integers")
cam_pos, cam_dir = vector3(cam_pos), vector3(cam_dir)
img = np.zeros((height, width, 3), dtype=np.uint8)

# 驗證相機
d_norm = np.linalg.norm(cam_dir)
if not np.isfinite(d_norm) or d_norm < 1e-8:
    raise ValueError("Camera direction invalid")
cam_dir = cam_dir / d_norm

if not (0 < fov_v < 180):
    raise ValueError("FOV must be between 0 and 180")
focal_length = 1.0 / np.tan(np.radians(fov_v) / 2.0)
aspect = width / height

up = np.array([0.0, 1.0, 0.0])
right = np.cross(cam_dir, up)
right_len = np.linalg.norm(right)
if right_len < 1e-8:
    raise ValueError("Camera up vector parallel to look direction")
right = right / right_len
up = np.cross(right, cam_dir)

for j in range(height):
    for i in range(width):
        # 像素中心
        x = (2.0 * (i + 0.5) / width - 1.0) * aspect / focal_length
        y = (1.0 - 2.0 * (j + 0.5) / height) / focal_length
        ray_dir = cam_dir + x * right + y * up
        ray = Ray(cam_pos, ray_dir)

        hit_t = float('inf')
        hit_obj = None

        for obj in objects:
            t = None
            if isinstance(obj, Sphere):
                t = intersect_sphere(ray, obj)
            elif isinstance(obj, Plane):
                t = intersect_plane(ray, obj)
            elif isinstance(obj, Triangle):
                res = intersect_triangle(ray, obj)
                if res:
                    t = res[0]

            if t is not None and t < hit_t:
                hit_t = t
                hit_obj = obj

        if hit_obj is not None:
            color = hit_obj.color
            # 簡單量化到 8-bit
            pixel = np.clip(color, 0.0, 1.0) * 255.0
            img[j, i] = pixel.astype(np.uint8)
        else:
            img[j, i] = np.array([255, 255, 255], dtype=np.uint8)

return img

```

```

def write_ppm(filename: str, img: np.ndarray):
    if not isinstance(img, np.ndarray) or img.ndim != 3 or img.shape[2] != 3:
        raise ValueError("PPM image must have shape (height, width, 3)")
    if img.dtype != np.uint8:
        raise ValueError("PPM image must use uint8 channels")
    h, w, _ = img.shape
    if h <= 0 or w <= 0:
        raise ValueError("PPM dimensions must be positive")
    with open(filename, 'wb') as f:
        f.write(b"P6\n")
        f.write(f"{w} {h}\n".encode())
        f.write(b"255\n")
        f.write(img.tobytes())

def shadow_test(point: np.ndarray, geometric_normal: np.ndarray,
               light_position: np.ndarray, objects: list,
               epsilon: float = 1e-4) -> bool:
    """若點至點光源之間有遮擋物回傳 True；僅作合成幾何測試。"""
    point, geometric_normal, light_position = map(vector3, (point, geometric_normal, light_position))
    n_len = np.linalg.norm(geometric_normal)
    if not np.isfinite(n_len) or n_len <= 0.0:
        raise ValueError("Geometric normal must be finite and non-zero")
    if not (np.all(np.isfinite(point)) and
            np.all(np.isfinite(light_position))):
        raise ValueError("Point and light position must be finite")
    if not np.isfinite(epsilon) or epsilon < 0.0:
        raise ValueError("epsilon must be finite and non-negative")

    n = geometric_normal / n_len
    to_light = light_position - point
    distance = np.linalg.norm(to_light)
    if not np.isfinite(distance) or distance <= epsilon:
        return False
    direction = to_light / distance
    side = 1.0 if np.dot(direction, n) >= 0.0 else -1.0
    origin = point + side * epsilon * n
    shifted_to_light = light_position - origin
    t_max = np.linalg.norm(shifted_to_light)
    if t_max <= epsilon:
        return False
    # 基本求交使用閉區間；先把開區間界移到內側可表示浮點數，
    # 避免最近根恰在被排除的端點時，漏掉同一球的另一個內部根。
    lower = np.nextafter(epsilon, np.inf)
    upper = np.nextafter(t_max, -np.inf)
    if lower > upper:
        return False
    ray = Ray(origin, shifted_to_light, t_min=lower, t_max=upper)

    for obj in objects:
        if isinstance(obj, Sphere):
            hit = intersect_sphere(ray, obj)
        elif isinstance(obj, Plane):
            hit = intersect_plane(ray, obj)
        elif isinstance(obj, Triangle):
            result = intersect_triangle(ray, obj)
            hit = result[0] if result is not None else None
        else:
            raise TypeError("Unsupported object type")

```

```

        if hit is not None:
            return True
    return False

def run_tests():
    print("Running tests...")
    # 1. Sphere Tangent
    s = Sphere(np.array([0.0, 0.0, 0.0]), 1.0)
    r = Ray(np.array([1.0, 0.0, -5.0]), np.array([0.0, 0.0, 1.0]), t_min=1e-4)
    t = intersect_sphere(r, s)
    assert t is not None and abs(t - 5.0) < 1e-5, f"Tangent sphere failed: {t}"

    # 最近根回歸測試：由球外的負Z位置沿 +Z 入射，須取近側根 t=4。
    r_near = Ray(np.array([0.0, 0.0, -5.0]),
                  np.array([0.0, 0.0, 1.0]), t_min=1e-4)
    t_near = intersect_sphere(r_near, s)
    assert t_near is not None and abs(t_near - 4.0) < 1e-5

    # 2. Sphere Inside
    r_in = Ray(np.array([0.0, 0.0, 0.0]), np.array([1.0, 0.0, 0.0]), t_min=1e-4)
    t_in = intersect_sphere(r_in, s)
    assert t_in is not None and abs(t_in - 1.0) < 1e-5, f"Inside sphere failed: {t_in}"

    # 3. Plane Parallel
    p = Plane(np.array([0.0, 1.0, 0.0]), 0.0)
    r_par = Ray(np.array([0.0, 0.0, 0.0]), np.array([1.0, 0.0, 0.0]))
    t_par = intersect_plane(r_par, p)
    assert t_par is None, "Parallel plane should be None"

    # 4. Triangle Hit
    tri = Triangle(np.array([0.0, 0.0, 0.0]), np.array([1.0, 0.0, 0.0]), np.array([0.0, 1.0, 0.0]))
    r_tri = Ray(np.array([0.2, 0.2, -1.0]), np.array([0.0, 0.0, 1.0]), t_min=1e-4)
    res = intersect_triangle(r_tri, tri)
    assert res is not None, "Triangle hit failed"
    t_tri = res[0]
    assert abs(t_tri - 1.0) < 1e-5, f"Triangle t failed: {t_tri}"
    assert np.allclose(res[2], (0.6, 0.2, 0.2))
    r_outside = Ray(np.array([1.2, 1.2, -1.0]),
                    np.array([0.0, 0.0, 1.0]), t_min=1e-4)
    assert intersect_triangle(r_outside, tri) is None

    # 5. Epsilon Test (Self-intersection avoidance)
    # Ray starts on surface, shoots out
    r_surf = Ray(np.array([0.0, 0.0, 1.0]), np.array([0.0, 0.0, 1.0]), t_min=1e-4)
    s_small = Sphere(np.array([0.0, 0.0, 0.0]), 1.0)
    t_surf = intersect_sphere(r_surf, s_small)
    # Should be None because t=0 is < t_min, and other root is negative
    assert t_surf is None, f"Surface self-intersection failed: {t_surf}"

    # 獨立展示場景：相機在 y=1、朝 -Z；球與三角形均位於相機前方。
    scene_sphere = Sphere(np.array([0.0, 0.0, -2.0]), 0.5,
                           np.array([1.0, 0.0, 0.0]))
    scene_plane = Plane(np.array([0.0, 1.0, 0.0]), 0.0,
                        np.array([0.0, 1.0, 0.0]))
    scene_tri = Triangle(np.array([-1.0, 0.1, -3.0]),
                         np.array([1.0, 0.1, -3.0]),
                         np.array([0.0, 1.5, -3.0]),
                         np.array([0.0, 0.0, 1.0]))

```

```

img = render(16, 16, [scene_sphere, scene_plane, scene_tri],
              np.array([0.0, 1.0, 0.0]),
              np.array([0.0, 0.0, -1.0]))
assert img.shape == (16, 16, 3)
assert img.dtype == np.uint8
assert np.all((img >= 0) & (img <= 255))
write_ppm("output_19.ppm", img)
print("Tests passed. PPM contents are subject to the scene and camera setup.")

if __name__ == "__main__":
    run_tests()

```

測試與預期結果

程式中的 `run_tests()` 測試幾何函式，最後另以獨立場景呼叫 `render()` 及 `write_ppm()`。以下均為依幾何配置推得的預期，未宣稱已執行或實際檢視影像。

本程式的測試集中檢查幾何條件：相切、最近根、內部出射、平行平面、三角形命中與未命中，以及影像形狀和資料型別。若要再驗證 `t_min` 的數值效果，可在球面外側沿法線發射，分別設定較小和較大的下界：較大的下界可能排除非常近的另一個有效表面；下界為零則可能接受起點處的根。這種比較應同時檢查交點是否真的位於場景尺度所需的解析範圍內，而不能只為讓測試通過任意調整 `epsilon`。

為測試 `shadow_test`，可令接收點位於平面上、點光源在其上方，再放置一個位於兩者之間的三角形；預期回傳遮擋。將三角形移至光源後方，射線仍可能與三角形相交，但命中參數應大於光源距離，預期不遮擋。還可測法線反向時偏移側是否隨新射線方向改變，以及零長度法線是否引發 `ValueError`。這些測試檢查的是線段遮擋判定，不代表光照強度或水下成像。

渲染場景與單元測試場景分開：單元測試使用位於原點的單位球、平面與三角形；展示場景則將球放在相機前方、三角形放在較遠的負Z位置。逐像素畫面受物件遮擋、投影及取樣影響，因此本章只承諾程式推得的尺寸、型別與值域，不宣稱特定未檢視像素顏色或視覺品質。

1. **相切測試**：射線從 $(1, 0, -5)$ 沿 $+Z$ 射向單位球，預期 $t = 5$ 。
2. **最近根回歸測試**：射線從 $(0, 0, -5)$ 沿 $+Z$ 射入單位球，預期回傳近根 $t = 4$ ，不是遠根6。
3. **內部出射**：射線從球心沿 $+X$ 射出，預期 $t = 1$ 。
4. **平行平面**：測試平面法線為 $(0, 1, 0)$ ，故平面是XZ平面；沿 $+X$ 的射線與之平行，預期 `None`。
5. **三角形命中**：射線打向三角形內部，預期 $t = 1$ ，重心座標為 $(0.6, 0.2, 0.2)$ ；另有三角形外射線，預期 `None`。
6. **自相交下界**：射線由球面向外發射， $t = 0$ 被正的 t_{min} 排除，且沒有其他有效正根，預期 `None`。可另比較略低於表面的起點在不同 t_{min} 下的結果，理解下界也可能排除真實近交點。
7. **PPM輸出**：預期建立 16×16 、三通道 `uint8` 影像；測試檢查尺寸、型別與通道值域。展示場景把球置於相機前方、三角形置於較遠的負Z位置，平面位於 $y = 0$ 。顏色是顯示RGB，沒有光照計算；不以未檢視的畫面宣稱特定像素必然呈現某顏色。

t_{min} 過大可能漏掉靠近表面的有效交點；移除下界則可能讓起點附近的根被接受。應依場景尺度選值，並用近表面測試檢查兩種失效方向。

除錯與常見陷阱

1. 浮點精度丟失：

- 當 $|b| \approx \sqrt{\Delta}$ 時，其中一個分子可能發生近似數相減的消去誤差。這與 $\Delta \approx 0$ 時判別式正負可被誤差改變，是不同問題。
- **解決：** 使用穩定求根算法。

2. 法線未正規化：

- 平面求交中，若 $\mathbf{n}$ 未正規化， τ_{par} 失效。
- **解決：** 建構時強制正規化並同步縮放 d 。

3. t_{min} 與起點偏移混淆：

- 起點偏移改變射線原點； t_{min} 限制接受的參數區間。兩者功能不同，且容差都必須按場景尺度選擇。
- **解決：** 對表面發射的次級射線，先依幾何法線及新方向選擇偏移側，再設定適當的 t_{min} ；不應把固定 epsilon 當成通用物理閾值。陰影射線另將 t_{max} 設為光源距離，避免把光源後方物體誤判為遮擋。

4. 三角形繞序與法線：

- $\mathbf{e}_1 \times \mathbf{e}_2$ 的方向由頂點繞序決定；本程式求交後會把法線翻至與射線方向相反，這是 face-forward 法線，不保留原始外向方向。
- **解決：** 若後續需要判定幾何外側或處理折射，應另外保存原始幾何法線與繞序，不要把 face-forward 法線誤當成外向法線。

養殖數位分身案例

- 遮蔽分析：** 對池底合成點 $\mathbf{p}$ ，朝點光源位置 $\mathbf{l}$ 發射有限長射線。使用幾何法線決定起點偏移側，並令 t_{max} 不超過光源距離；只有開區間內命中棚架三角形才判為遮擋。平行、共面或退化三角形應依求交API的容差與回傳慣例處理，不能把「無交點」一概解讀成確定無遮蔽。
- 感測器幾何示意：** 可為合成相機的像素建立射線，檢查是否命中以球體或三角網格表示的合成魚。這只提供幾何可見性，不會自動生成真實攝影機的曝光、散射、折射或噪聲。
- 介質限制：** 本章只處理幾何相交，未包含水體吸收與散射。若加入Beer-Lambert吸收，須另定義介質係數、路徑長度及色彩單位；單靠幾何命中不能推出水下影像的物理準確度。
- 稽核資料：** 每條測試射線可記錄起點、單位方向、 t_{min} 、 t_{max} 、命中物件ID及資料來源。這些欄位有助於重現幾何判定，但合成結果不是現場感測值。

習題

- 手算：** 射線 $\mathbf{o} = (1, -1, -1)$, $\mathbf{d} = (0, 0, 1)$ ，球心 $(0, 0, 0)$, $R = 1$ 。求 t 。

2. **程式測試**：修改 `intersect_sphere` 返回所有有效根。測試外部穿越與內部出射。
3. **除錯**：若 $b > 0$ 且 $\sqrt{\Delta} \approx b$ ，直接計算 $-b + \sqrt{\Delta}$ 有何問題？
4. **整合**：使用本章完整的 `shadow_test`，以起點法線偏移及 t_{min} 排除自相交，並用 t_{max} 限制至點光源。測試一個光源與遮擋三角形之間的命中，並測試光源後方的物體不得算作遮擋。

若將其改寫成獨立練習，可用下列流程核對，而非引用未提供的外部函式：先正規化幾何法線；若新方向與法線內積非負，沿法線偏移，否則反向偏移；從偏移後位置計算至光源的單位方向和距離；建構區間為 $[t_{min}, t_{max}]$ 的射線；逐一呼叫本章三種求交函式；僅當命中嚴格滿足 $t_{min} < t < t_{max}$ 時回報遮擋。若光源距離不大於偏移量，沒有可測的線段，應回報未遮擋或依應用明確定義該退化情形。

習題解答

1. $\mathbf{m} = (1, -1, -1)$ 。 $b = 2(0 + 0 - 1) = -2$ 。 $c = 1 + 1 + 1 - 1 = 2$ 。 $\Delta = 4 - 8 = -4 < 0$ 。無交點。
2. 修改程式收集所有 $t \in [t_{min}, t_{max}]$ 的根。例題中由 $(0, 0, -5)$ 沿 $+Z$ 穿越單位球，預期兩根為 $[4, 6]$ 。內部出射預期 $[1]$ （若 t_{min} 小於 1）。
3. 若 $b > 0$ ， $-b < 0$ ， $\sqrt{\Delta} > 0$ 。若 $\sqrt{\Delta} \approx b$ ，則 $-b + \sqrt{\Delta}$ 為兩接近數相減，有效位元喪失。應使用 $-b - \sqrt{\Delta}$ 。
4. `shadow_test` 先將幾何法線正規化，依新射線與法線內積選擇偏移側，再由偏移後起點重新計算至光源的方向和距離。它將 `t_min` 設為正值，`t_max` 設為光源距離，並分派球、平面或三角形求交；若存在嚴格落在兩界之間的命中，回傳遮擋。習題要求的是起點偏移與射線下界兩種處理，而不是把 t_{min} 稱為偏移。

本章小結

射線以起點、正規化方向及參數區間描述。球、平面與三角形求交各有不同的退化與平行情形；球面求根須分別處理判別式符號誤差與相近數相減造成的消去。起點偏移改變射線原點， t_{min} 限制接受的參數範圍，兩者用途不同。最近命中由有效交點中最小的 t 決定；有限光源陰影測試還要限制 t_{max} 。測試應涵蓋相切、最近根、內部出射、平行、三角形重心、未命中及影像輸出契約。容差須按量綱與場景尺度選擇，不能當作通用物理安全閾值。

參考來源

- [G4] *Ray Tracing in One Weekend*，<https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- [G1] PBRT 4：Transformations，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G7] NumPy線性代數參考，<https://numpy.org/doc/stable/reference/routines.linalg.html>

列出來源僅供延伸閱讀，不表示本章已逐項獨立查證或執行來源範例。

第20章 包圍盒與BVH加速

學習目標與先備知識

讀完本章後，你應能：

- 以射線一軸對齊包圍盒（AABB）的 slab 方法判斷是否相交，並處理射線方向分量為零的情況。
- 說明包圍盒如何保守地包住三角形，以及為何「沒有打到包圍盒」即可略過盒內幾何。
- 建立二元包圍體積階層（BVH），遍歷節點並找到最近交點。
- 比較暴力求交與BVH的結果及幾何測試次數；未實際執行的效能不得寫成測量值。

先備知識是向量、內積、射線方程、三角形求交及Python基本語法。本章採右手座標系，長度以公尺計；射線為

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \in [t_{\min}, t_{\max}],$$

其中原點 $\mathbf{o} \in \mathbb{R}^3$ 、方向 $\mathbf{d} \in \mathbb{R}^3$ ， t 是參數，不一定是公尺。若 $\mathbf{d}$ 為單位向量， t 才等於沿射線的距離。所有端點是否包含在測試範圍內，必須明確約定；本章使用閉區間。

問題與直覺

場景有很多三角形時，逐一對每個三角形求交，對每條射線都要付出線性成本。包圍盒提供保守篩選：先測射線是否碰到物件的外盒，沒碰到就能安全略過其內所有三角形；碰到才進一步檢查。

BVH把多個物件的包圍盒再包入較大的盒中，形成樹。根節點包住整個場景；內部節點包住子節點；葉節點存放少量三角形。一次射線查詢由根往下測試：若節點盒不相交，整個子樹都可略過；若相交，才繼續檢查孩子或葉中的三角形。

BVH不會改變幾何答案，但會改變候選物件的搜尋順序。要比較命中結果，還須訂明距離相同時如何選擇三角形。本章統一選擇字典序最小的 $(t, \text{索引})$ ：先選較小的 t ，若距離相同則選較小索引。盒必須確實包住所屬幾何；否則加速結構可能漏掉交點。

數學與幾何推導

AABB由各軸最小與最大座標定義：

$$B = [b_x^-, b_x^+] \times [b_y^-, b_y^+] \times [b_z^-, b_z^+].$$

對單一軸 i ，若 $d_i \neq 0$ ，射線進入與離開此軸向區間的參數為

$$t_{i,0} = \frac{b_i^- - o_i}{d_i}, \quad t_{i,1} = \frac{b_i^+ - o_i}{d_i}.$$

方向為負時兩者順序會反轉，所以定義

$$t_i^{\text{near}} = \min(t_{i,0}, t_{i,1}), \quad t_i^{\text{far}} = \max(t_{i,0}, t_{i,1}).$$

射線必須同時落在三個軸的範圍內。令

$$t_{\text{enter}} = \max_i t_i^{\text{near}}, \quad t_{\text{exit}} = \min_i t_i^{\text{far}}.$$

再與射線有效範圍相交：

$$t'_{\text{enter}} = \max(t_{\text{enter}}, t_{\min}), \quad t'_{\text{exit}} = \min(t_{\text{exit}}, t_{\max}).$$

存在交點的條件為 $t'_{\text{enter}} \leq t'_{\text{exit}}$ 。若要求離開射線原點之後的交點，常取 $t_{\min} > 0$ ；陰影射線則常以光源距離作為 $t_{\max}$ ，避免把光源後方的物件算進去。

當 $d_i = 0$ ，射線在該軸座標固定為 o_i 。若 $o_i < b_i^-$ 或 $o_i > b_i^+$ ，射線不可能碰盒；否則該軸不限制 t ，相當於給它區間 $(-\infty, +\infty)$ 。程式中應直接分支處理，不要除以零，也不要「很小的方向」一律當作零而改變幾何問題。

三角形 $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$ 的緊密AABB逐軸取三個頂點座標的最小值與最大值。浮點誤差或幾何膨脹需求可使盒稍微擴大；膨脹量應與場景尺度相稱，不能任意大到使篩選失效。

BVH分割與成本

常見的二元BVH以物件中心作分割：計算當前物件中心在各軸的範圍，選範圍最大的軸，依該軸排序後分成兩半，遞迴建立子樹。分割不必讓兩邊盒子不重疊；只要每個子樹仍保守包住其物件即可。

若每個葉節點最多存 L 個三角形，並以 N 個三角形建樹，平衡分割通常能將樹高控制在約 $\log_2(N/L)$ 的量級。不過，樹高不是唯一成本：盒子彼此重疊時，同一條射線可能走訪許多分支；葉子太大則會做較多三角形測試；葉子太小則增加節點與盒子測試。常用的表面積啟發式（SAH）會近似比較分割後的成本：

$$C_{\text{split}} \approx C_{\text{box}} + \frac{A_L}{A_P} N_L C_{\text{tri}} + \frac{A_R}{A_P} N_R C_{\text{tri}},$$

其中 A_P, A_L, A_R 分別是父盒及左右子盒的表面積， N_L, N_R 是其物件數； C_{box} 與 C_{tri} 是盒測試與三角形測試的估計成本。這是建樹時的近似準則，不是實際執行時間的保證。

逐步手算例題

例一：斜向射線穿過盒子

令盒子為 $[1, 3] \times [-1, 1] \times [2, 4]$ ，射線原點為 $(0, 0, 0)$ ，方向為 $(1, 0, 1)$ ，有效參數範圍為 $[0, +\infty)$ 。

- x 軸的端點參數是 1 與 3。
- y 軸方向為零，且原點的 $y = 0$ 在 $[-1, 1]$ 內；因此此軸不限制參數。
- z 軸的端點參數是 2 與 4。
- 整體進入參數為 $\max(1, 2) = 2$ ，離開參數為 $\min(3, 4) = 3$ 。

因此射線於 $t \in [2, 3]$ 穿過盒子。若把 $t_{\max}$ 改成 1.5，有效範圍與盒子區間沒有重疊，結果為不相交。

例二：零方向分量與平行盒面

仍用同一盒子，令射線原點為 $(4, 0, 3)$ ，方向為 $(0, 1, 0)$ 。在 x 軸方向為零，但 $o_x = 4$ 位於盒外的 $[1, 3]$ ，故立即判定不相交。

若原點改為 $(2, -2, 3)$ ，方向仍不變， x, z 軸都在盒內。 y 軸的端點參數為 $(-1 - (-2))/1 = 1$ 與 $(1 - (-2))/1 = 3$ ，因此射線於 $t \in [1, 3]$ 穿過盒子。這說明「方向分量為零」不等於必定不相交，必須再檢查平行軸上的座標。

實作與程式

以下程式只使用Python標準庫。它建立軸對齊盒、測試盒與射線的交集、使用Möller-Trumbore方法測三角形，並以中位數切分建立二元BVH。所有射線、三角形與AABB使用三維浮點tuple；程式把退化三角形視為無交點。`build` 的前置條件是三角形清單非空、索引有效，且 `leaf_size` 為正整數。

遍歷時，程式會先算出左右子盒的進入參數，優先探索較近的孩子。若兩個孩子都相交，先壓入較遠孩子，再壓入較近孩子；堆疊後進先出，因此近者先被取出。這有助於較早找到近交點，進而縮短後續節點與三角形測試的上界。

```
from dataclasses import dataclass
from math import inf

Vec = tuple[float, float, float]
Tri = tuple[Vec, Vec, Vec]
Ray = tuple[Vec, Vec]

@dataclass
class Box:
    lo: Vec
    hi: Vec

def union(a: Box, b: Box) -> Box:
```

```

    return Box(
        tuple(min(a.lo[i], b.lo[i]) for i in range(3)),
        tuple(max(a.hi[i], b.hi[i]) for i in range(3)),
    )

def tri_box(t: Tri) -> Box:
    return Box(
        tuple(min(v[i] for v in t) for i in range(3)),
        tuple(max(v[i] for v in t) for i in range(3)),
    )

def box_enter(box: Box, ray: Ray, tmin: float, tmax: float):
    """回傳射線有效區間內的進入參數；不相交則回傳 None。"""
    origin, direction = ray
    enter, exit = tmin, tmax
    for i in range(3):
        if direction[i] == 0.0:
            if origin[i] < box.lo[i] or origin[i] > box.hi[i]:
                return None
            continue
        a = (box.lo[i] - origin[i]) / direction[i]
        b = (box.hi[i] - origin[i]) / direction[i]
        near, far = min(a, b), max(a, b)
        enter = max(enter, near)
        exit = min(exit, far)
        if enter > exit:
            return None
    return enter

def triangle_hit(tri: Tri, ray: Ray, tmin: float, tmax: float):
    o, d = ray
    a, b, c = tri

    def sub(x, y):
        return tuple(x[i] - y[i] for i in range(3))

    def cross(x, y):
        return (
            x[1] * y[2] - x[2] * y[1],
            x[2] * y[0] - x[0] * y[2],
            x[0] * y[1] - x[1] * y[0],
        )

    def dot(x, y):
        return sum(x[i] * y[i] for i in range(3))

    e1, e2 = sub(b, a), sub(c, a)
    d_len = dot(d, d) ** 0.5
    e1_len = dot(e1, e1) ** 0.5
    e2_len = dot(e2, e2) ** 0.5
    if d_len == 0.0 or e1_len == 0.0 or e2_len == 0.0:
        return None
    p = cross(d, e2)
    det = dot(e1, p)
    # 無因次角度容差；門檻隨方向及兩條邊的長度縮放。
    if abs(det) < 1e-8 * d_len * e1_len * e2_len:

```

```

        return None
    inv_det = 1.0 / det
    s = sub(o, a)
    u = dot(s, p) * inv_det
    if u < 0.0 or u > 1.0:
        return None
    q = cross(s, e1)
    v = dot(d, q) * inv_det
    if v < 0.0 or u + v > 1.0:
        return None
    t = dot(e2, q) * inv_det
    return t if tmin <= t <= tmax else None

@dataclass
class Node:
    box: Box
    left: object = None
    right: object = None
    ids: list[int] | None = None

def build(tris: list[Tri], ids: list[int], leaf_size: int = 2) -> Node:
    if not ids:
        raise ValueError("ids 不可為空")
    if leaf_size < 1:
        raise ValueError("leaf_size 必須為正整數")

    box = tri_box(tris[ids[0]])
    for idx in ids[1:]:
        box = union(box, tri_box(tris[idx]))
    if len(ids) <= leaf_size:
        return Node(box, ids=ids)

    centers = [
        tuple((tris[k][0][i] + tris[k][1][i] + tris[k][2][i]) / 3.0
              for i in range(3))
        for k in ids
    ]
    axis = max(
        range(3),
        key=lambda i: max(c[i] for c in centers) - min(c[i] for c in centers)
    )
    ordered = sorted(ids, key=lambda k: sum(v[axis] for v in tris[k]) / 3.0)
    mid = len(ordered) // 2
    return Node(
        box,
        build(tris, ordered[:mid], leaf_size),
        build(tris, ordered[mid:], leaf_size),
    )

def brute_force(tris: list[Tri], ray: Ray, tmin=1e-6, tmax=inf):
    best = None
    tests = 0
    for idx, tri in enumerate(tris):
        tests += 1
        limit = tmax if best is None else min(tmax, best[0])
        t = triangle_hit(tri, ray, tmin, limit)

```

```

        if t is not None:
            candidate = (t, idx)
            if best is None or candidate < best:
                best = candidate
    return best, tests

def traverse(root: Node, tris: list[Tri], ray: Ray,
            tmin=1e-6, tmax=inf):
    best = None
    box_tests = 0
    tri_tests = 0
    stack = [root]

    while stack:
        node = stack.pop()
        box_tests += 1
        limit = tmax if best is None else min(tmax, best[0])
        if box_enter(node.box, ray, tmin, limit) is None:
            continue

        if node.ids is not None:
            for idx in node.ids:
                tri_tests += 1
                limit = tmax if best is None else min(tmax, best[0])
                t = triangle_hit(tris[idx], ray, tmin, limit)
                if t is not None:
                    candidate = (t, idx)
                    if best is None or candidate < best:
                        best = candidate
        else:
            # 先測孩子盒，將較遠者先壓入堆疊，讓較近者先取出。
            children = []
            limit = tmax if best is None else min(tmax, best[0])
            for child in (node.left, node.right):
                box_tests += 1
                entry = box_enter(child.box, ray, tmin, limit)
                if entry is not None:
                    children.append((entry, child))
            children.sort(key=lambda pair: pair[0], reverse=True)
            for _, child in children:
                stack.append(child)

    return best, box_tests, tri_tests

def run_tests():
    box = Box((1.0, -1.0, 2.0), (3.0, 1.0, 4.0))
    assert box_enter(box, ((0., 0., 0.), (1., 0., 1.)), 0., inf) == 2.0
    assert box_enter(box, ((4., 0., 3.), (0., 1., 0.)), 0., inf) is None
    assert box_enter(
        box, ((2., -2., 3.), (0., 1., 0.)), 0., inf
    ) == 1.0

    tris = [
        ((-1., -1., 3.), (1., -1., 3.), (0., 1., 3.)),
        ((-1., -1., 5.), (1., -1., 5.), (0., 1., 5.)),
        ((3., -1., 4.), (5., -1., 4.), (4., 1., 4.)),
        ((3., -1., 6.), (5., -1., 6.), (4., 1., 6.)),
    ]

```

```

        ((-1., -1., 3.), (1., -1., 3.), (0., 1., 3.)), # 與第0面重合
    ]
    rays = [
        ((0., 0., 0.), (0., 0., 1.)),
        ((4., 0., 0.), (0., 0., 1.)),
        ((0., 4., 0.), (0., 0., 1.)),
    ]
    # 同一幾何射線：正比例縮放方向不應改變命中索引。
    base, _ = brute_force(tris, ((0., 0., 0.), (0., 0., 1.)))
    scaled, _ = brute_force(tris, ((0., 0., 0.), (0., 0., 1e-8)))
    assert base is not None and scaled is not None
    assert base[1] == scaled[1] == 0
    assert abs(base[0] - 3.0) < 1e-12
    assert abs(scaled[0] * 1e-8 - 3.0) < 1e-12

    root = build(tris, list(range(len(tris))))
    for ray in rays:
        direct, _ = brute_force(tris, ray)
        accelerated, _, _ = traverse(root, tris, ray)
        assert direct == accelerated

if __name__ == "__main__":
    run_tests()

```

三角形行列式測試使用無因次係數 `1e-8` 乘以方向及兩條邊的長度，避免單純縮放射線方向或網格尺寸就改變近平行判定；零長方向與零長邊回傳無交點。這仍只是範例容差，真實專案須依座標精度及求交方法檢查。方向未正規化時，同一幾何交點的參數 t 會反比於方向長度改變，距離仍為 $t\|\mathbf{d}\|$ 。`tmin=1e-6` 是參數下界：改變方向長度而不相應調整它，可能使極近交點的接受結果不同；它不是通用自相交解法或物理安全距離。

測試與預期結果

將程式存成Python檔案後執行。依照程式與測試條件，預期三個直接的盒測試，以及三條射線的暴力／BVH最近交點比對都通過。這是預期結果，不是作者實際執行結果。

新增的重合三角形與第0個三角形距離相同；兩種查詢都採 $(t, \text{索引})$ 字典序，因此預期選擇較小索引。建議另做以下測試：

1. **盒面邊界**：****射線平行於某軸，且原點剛好在盒子的最小或最大邊界上，應算相交。**
2. **切觸角點**：****射線只碰到盒子的角，閉區間 slab 測試應算相交。**
3. **反向射線**：****令方向某分量為負，確認 near/far 交換後仍正確。**
4. **最近交點**：****使用同一方向上距離不同的兩個三角形，確認兩種查詢回傳相同距離和索引。**
5. **測試次數**：****對每條射線記錄暴力三角形測試數及BVH的盒、三角形測試數。若比較執行時間，須實際計時並報告硬體、資料量、計時方式及重複次數。**

除錯與常見陷阱

- **零方向仍做除法**：方向為零時應先判斷原點是否在該軸 slab 內。
- **只取正的交點卻忘記範圍**：射線一盒求交必須與 $[t_{\min}, t_{\max}]$ 相交。陰影射線若缺少有限 $t_{\max}$ ，可能把光源後方幾何也當遮擋物。
- **包圍盒漏包幾何**：建立盒時漏掉頂點，或更新模型後沒有更新盒，會讓BVH漏掉交點。保守盒可略寬，不能太窄。
- **命中距離相同但索引不同**：若暴力法與BVH使用不同的平手規則，回傳索引便可能不同。本章兩者均以 $(t, \text{索引})$ 字典序選擇。
- **盒測試通過就當成表面命中**：AABB只表示可能相交，必須繼續測葉子中的三角形。
- **建樹時分割失敗**：中心座標完全相同時仍以中位數分割可終止遞迴；若以分割平面分類而未處理全部落在同側的情況，可能產生空子樹或無限遞迴。
- **把比較次數等同時間**：盒測試與三角形測試成本不同，記憶體配置、快取及程式語言也會影響時間。測試數可用來理解演算法，不足以單獨宣稱某方法快多少。

養殖數位分身案例

考慮合成的長方形養殖池，池內有魚體三角網格與幾個水下構件。可為每個魚體建立局部三角形索引，再建立該魚的BVH；場景層另以魚體AABB建立上層BVH。射線相機渲染時，先查場景層，再查候選魚體內的三角形；陰影射線則使用從表面點指向光源的有限射程。

若魚體會動，不能把上一影格的AABB直接視為目前姿態的有效盒。可以每影格重算包圍盒，或採用適合動態場景的更新策略；選擇取決於物件數、變形範圍及更新頻率。靜態水池牆面則可保留固定BVH。

這些模型與影像是合成資料。BVH只能改善幾何查詢，不會使魚體形狀、光學模型、行為或影像變得符合真實養殖場測量。

習題

1. **手算**：盒子為 $[0, 2] \times [1, 3] \times [-1, 1]$ 。射線原點為 $(-1, 2, 0)$ 、方向為 $(1, 0, 0)$ ，範圍為 $[0, +\infty)$ 。求進入、離開參數並判定是否相交。
2. **程式測試**：在範例程式中新增一條射線，令它從三角形背後沿 $-Z$ 方向射向三角形；用暴力法和BVH比較最近交點。說明方向長度改變對距離參數的影響。
3. **反例／除錯**：有人將每軸 slab 測試寫成 $a = (\text{lo-origin})/\text{direction}$ 、 $b = (\text{hi-origin})/\text{direction}$ ，再直接令 $\text{enter} = \max(\text{enter}, a)$ 、 $\text{exit} = \min(\text{exit}, b)$ ，沒有排序 a, b 。給一個方向分量為負的反例，指出錯誤並修正。
4. **整合應用**：某合成池場景有固定池壁及逐影格變形的魚網格。設計兩層BVH的組織方式，指出哪些資料需更新，並提出一組不依賴未測效能數字的正確性與成本評估流程。

習題解答

1. x 軸端點參數為 1 和 3； y, z 軸方向為零，原點座標 2, 0 都在各自區間內，不限制參數。因此進入為 1、離開為 3，相交區間為 $[1, 3]$ 。
2. 可新增射線 $((0., 0., 4.), (0., 0., -1.))$ ；對位於 $z = 3$ 且包含原點投影位置的三角形，它將沿 $-Z$ 命中。用 `brute_force` 及 `traverse` 比對回傳結果。若方向從 $(0, 0, -1)$ 改為 $(0, 0, -2)$ ，同一幾何點的 t 會減半；實際空間距離仍是 $t\|\mathbf{d}\|$ ，不能直接把 t 當公尺。
3. 例如 x 軸盒區間為 $[1, 3]$ ，原點座標 4、方向為 -1 ：直接計算得 $a = 3$ 、 $b = 1$ 。未排序會把近端錯當為 3、遠端錯當為 1，造成 `enter > exit` 而錯誤拒絕。應使用 `near=min(a,b)`、`far=max(a,b)`，再更新進入與離開界。
4. 上層BVH可含池壁物件與魚體物件盒；固定池壁的盒與下層結構可重用。魚網格變形後，須更新受影響三角形盒、魚體下層BVH節點盒，以及上層魚體盒；若拓撲不變但頂點改變，也不能沿用未更新的頂點包圍盒。先以固定射線集合比較暴力與BVH的最近交點及索引；再統計盒測試及三角形測試數。若要報時間，須實際計時並記錄測試平台、網格規模與重複方式；沒有計時時只報計數及方法，不宣稱速度提升。

本章小結

AABB以每個座標軸的一維區間交集構成 slab 測試。零方向分量須分開處理；非零分量須排序端點參數，再與射線的有效範圍求交。BVH用保守包圍盒組織物件，使不相交的子樹能整批略過。近優先遍歷可望提早找到最近交點，縮短後續測試範圍，但不保證實際效能。比較暴力法與BVH時，先驗證最近交點及平手規則一致，再以測試數或有紀錄的計時討論成本。

參考來源

- [G1] *Physically Based Rendering: From Theory to Implementation*, 4th ed., “Transformations.” https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G4] Peter Shirley 等，*Ray Tracing in One Weekend*. <https://raytracing.github.io/books/RayTracingInOneWeekend.html>

以上來源供延伸閱讀；本章推導與範例以可重現的幾何條件說明，未宣稱執行來源範例或實測本章程式效能。

第 21 章 光傳輸積分與 Monte Carlo

學習目標與先備知識

本章處理渲染方程裡最核心的一件事：把連續的積分轉成可以用有限次隨機取樣估計的量。讀完本章，你應該能夠：

1. 寫出反射光 L_o 的半球積分式，並指出 $\cos \theta$ 項、機率密度函數 (PDF) 與估計式各自的角色。
2. 手算 Lambert 半球積分的真值，以及均勻半球取樣、餘弦加權取樣在單一樣本下的估計值。
3. 推導並比較兩種取樣的變異數，說明為什麼餘弦加權在常數光源下可以得到零變異數。
4. 寫出一支用標準庫實作的 Monte Carlo 估計程式，處理 PDF 為零、樣本數收斂與多個 seed 重複實驗。
5. 對「合成資料可以驗證管線」與「合成資料不能取代實際量測」之間的邊界保持一致。

先備知識：本章假設你已讀過第 15 章的 Lambert 反射與立體角、第 19 章的射線交點與數值穩健性，以及 Volume I 的機率密度函數與期望值。如果你對 $E[g(X)] = \int g(x)p(x) dx$ 這個式子的意義不熟，先在紙上寫一次一維的例子再往下讀。本章的所有數值皆標為合成，不含任何真實輻射量測。

全書慣例：右手世界系 +X 右、+Y 上、+Z 面向觀者；半球方向 ω 為單位向量；角度用弧度； $\cos \theta$ 為 ω 與表面法線 n 的夾角餘弦。輻射亮度 L 的單位為 $\text{W m}^{-2} \text{sr}^{-1}$ ；輻照度 E 的單位才是 W m^{-2} ，兩者不可混用。輻射量計算在線性 RGB 三通道獨立完成。

問題與直覺

渲染方程告訴我們，一個表面點 p 往某方向 ω_o 反射的輻射亮度，等於自發光加上由所有入射方向進來的能量的某種加權積分。對不透明表面可以寫成

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega^+} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) |\cos \theta_i| d\omega_i.$$

這裡 f_r 是 BRDF（單位 sr^{-1} ）， L_i 是從方向 ω_i 進來的入射輻射亮度， Ω^+ 是以上半球 n 為軸的上半球， $d\omega_i$ 是立體角測度。

積分有兩個麻煩。第一， Ω^+ 是連續集合，不可能逐方向檢查。第二， L_i 通常來自場景本身，沒有一個簡單的封閉式。處理這種問題的標準做法是 **Monte Carlo 積分**：用隨機方向的加權平均去估計這個積分。

Monte Carlo 的核心是把「對方向的積分」重寫成「對某個機率分佈的期望值」。一旦寫成期望值，就可以用樣本平均逼近。兩個問題就冒出來了：

- 要選哪個機率分佈？**任意分佈理論上都無偏**，但不同分佈的變異數差很多。
- PDF 在某些方向上是零怎麼辦？因為我們要除以 $p(\omega)$ ，如果 $f(\omega) \neq 0$ 但 $p(\omega) = 0$ ，估計式就壞掉。

這兩個問題貫穿整章：**選擇好的分佈和處理退化方向**。它們不是隨機性本身的問題，而是「估計式設計」的問題。

數學與幾何推導

從積分到期望

給定一個方向定義域 D ，機率密度函數 p 滿足 $p(\omega) \geq 0$ 與 $\int_D p(\omega) d\omega = 1$ 。對任意函數 g ，期望值定義為

$$E_p[g] = \int_D g(\omega) p(\omega) d\omega.$$

把 $g(\omega) = f(\omega)/p(\omega)$ 代入，得到

$$E_p\left[\frac{f}{p}\right] = \int_D \frac{f(\omega)}{p(\omega)} p(\omega) d\omega = \int_D f(\omega) d\omega.$$

所以只要 $p(\omega) > 0$ 在所有 $f(\omega) \neq 0$ 的地方成立，估計式

$$\hat{F}_N = \frac{1}{N} \sum_{i=1}^N \frac{f(\omega_i)}{p(\omega_i)}, \quad \omega_i \sim p$$

就是無偏的： $E[\hat{F}_N] = I$ 。其中 $I = \int_D f d\omega$ 。

變異數與誤差

單一樣本的變異數為

$$\sigma^2 = E_p\left[\left(\frac{f}{p}\right)^2\right] - I^2.$$

獨立同分佈的 N 個樣本平均，變異數是

$$\text{Var}[\hat{F}_N] = \frac{\sigma^2}{N}, \quad \text{標準誤差} = \frac{\sigma}{\sqrt{N}}.$$

變異數決定了「需要的樣本數」。若把 p 選得和 f 成正比，單樣本變異數可能為零——也就是常數估計。

兩種半球取樣

半球上有兩種最常見的取樣分佈：

- **均勻半球**： $p_u(\omega) = \frac{1}{2\pi}$ 。

- 餘弦加權半球： $p_c(\omega) = \frac{\cos \theta}{\pi}$ ，其中 θ 是與法線的夾角。

兩者都滿足機率歸一化。注意角度與立體角的換算：以球面座標 (θ, ϕ) 表示時， $d\omega = \sin \theta d\theta d\phi$ 。這是本章最常在實作裡被忘記的一項。

由反函數取樣法得到兩種分佈的方向：

- 均勻： $\cos \theta = 1 - r_1$ ， $\phi = 2\pi r_2$ 。用 $r_1 \sim U[0, 1]$ 亦可寫成 $\cos \theta = r_1$ 。
- 餘弦加權： $\cos \theta = \sqrt{1 - r_1}$ ， $\sin \theta = \sqrt{r_1}$ ， $\phi = 2\pi r_2$ 。

方向在局部座標系（+Y 為法線）下為

$$\omega = (\sin \theta \cos \phi, \cos \theta, \sin \theta \sin \phi).$$

帶入 Lambert 反射

Lambert BRDF 為 $f_r = \rho/\pi$ （ ρ 為反照率， $\in [0, 1]$ ）。設入射光為 $L_i(\omega)$ ，估計式為

$$\hat{L}_o = \frac{1}{N} \sum_{i=1}^N \frac{(\rho/\pi) L_i(\omega_i) \cos \theta_i}{p(\omega_i)}.$$

代入兩種取樣分佈：

- 均勻： $\hat{L}_o^{(u)} = \frac{1}{N} \sum 2\rho L_i \cos \theta_i$ 。
- 餘弦加權： $\hat{L}_o^{(c)} = \frac{1}{N} \sum \rho L_i$ 。因為 $\cos \theta_i$ 與 p 的 $\cos \theta$ 因子完全約掉。

若 $L_i \equiv 1$ （上半球所有方向入射光相同），餘弦加權估計式的每一項都是 ρ ，所以單樣本估計恆等於真值 ρ ，**變異數為零**。這正是「把 p 選得和 f 成正比」的具體例子。後續章節的計算結果皆為線性 RGB 下的量，輸出成影像前才做 sRGB 編碼。

逐步手算例題

例題一：常數光下的真值與兩種估計

設 $\rho = 0.6$ ， $L_i(\omega) = 1$ 對所有 ω 。

真值：

$$L_o = \frac{\rho}{\pi} \int_{\Omega^+} \cos \theta d\omega = \frac{\rho}{\pi} \cdot \pi = \rho = 0.6.$$

均勻半球估計，單樣本。取樣 $r_1 = 0.5$ ，則 $\cos \theta = 0.5$ （ $\theta = 60^\circ$ ）， ϕ 任意：

$$\hat{L}_o^{(u)} = 2\rho \cos \theta = 2 \cdot 0.6 \cdot 0.5 = 0.6.$$

這個樣本恰好命中真值。均勻取樣的期望值為 $\rho \cdot 2E[\cos \theta] = 0.6 \cdot 2 \cdot (1/2) = 0.6$ ，正確；但單樣本變異數不是零。

餘弦加權估計，任一樣本：

$$\hat{L}_o^{(c)} = \rho L_i = 0.6.$$

每一樣本都給 0.6。變異數為零。

例題二：方向性入射光

現在改取 $L_i(\omega) = \cos \theta$ （本題僅作無量綱的數學測試函數使用，以便專注於積分結構）， $\rho = 0.6$ 。真值：

$$L_o = \frac{\rho}{\pi} \int_{\Omega^+} \cos^2 \theta d\omega.$$

由 $\int_{\Omega^+} \cos^2 \theta d\omega = 2\pi/3$ ，得

$$L_o = \frac{0.6}{\pi} \cdot \frac{2\pi}{3} = 0.4.$$

均勻半球單樣本，仍取 $\theta = 60^\circ$ ($\cos \theta = 0.5$)：

$$g_u = 2\rho \cos^2 \theta = 2 \cdot 0.6 \cdot 0.25 = 0.3.$$

餘弦加權單樣本，仍取 $\theta = 60^\circ$ ：

$$g_c = \rho \cos \theta = 0.6 \cdot 0.5 = 0.3.$$

巧合下兩者同為 0.3。這是偶然而非通則。若取 $\theta = 0$ ($r_1 = 0$ 或 $r_1 = 1$ 的極端)：

- 均勻： $g_u = 2 \cdot 0.6 \cdot 1 = 1.2$ 。
- 餘弦： $g_c = 0.6 \cdot 1 = 0.6$ 。

此時餘弦加權的樣本值較接近真值 0.4，但單一樣本仍可能偏離真值很遠。要判斷哪個估計式較好，仍須看變異數，不能只看單一樣本。

例題三：兩種取樣的理論標準誤差

沿用例題二，計算 $N = 500$ 時的標準誤差，用來設定下一節的預期輸出。

均勻取樣，樣本值 $g_u = 1.2 \cos^2 \theta$ ：

$$E[g_u] = 1.2 \cdot \frac{1}{2\pi} \cdot \frac{2\pi}{3} = 0.4, \quad E[g_u^2] = 1.44 \cdot \frac{1}{2\pi} \cdot \frac{2\pi}{5} = 0.288,$$

$$\sigma_u^2 = 0.288 - 0.16 = 0.128, \quad \sigma_u \approx 0.3578, \quad \frac{\sigma_u}{\sqrt{500}} \approx 0.0160.$$

餘弦加權取樣，樣本值 $g_c = 0.6 \cos \theta$ ：

$$E[g_c] = 0.6 \cdot \frac{2}{3} = 0.4, \quad E[g_c^2] = 0.36 \cdot \frac{1}{2} = 0.18,$$

$$\sigma_c^2 = 0.18 - 0.16 = 0.02, \quad \sigma_c \approx 0.1414, \quad \frac{\sigma_c}{\sqrt{500}} \approx 0.0063.$$

餘弦加權在這裡的單樣本變異數比均勻取樣低 ~ 6.4 倍，標準誤差低約 2.5 倍。原因不是隨機性本身，而是 p 與 f 的形狀更接近。若光源是常數， σ_c 進一步降到 0。

實作與程式

下面是一支只依賴 Python 標準庫的 Monte Carlo 估計器。它內建四項自檢：常數光真值、方向光收斂、正常取樣結果為有限值，以及餘弦加權在常數光下標準差為 0。現有自檢未注入零 PDF，不能據此驗證零 PDF 防護分支。

```
"""mc_lambert.py — Monte Carlo 半球積分；只用標準庫。"""
import math, random

def sample_uniform_hemisphere(r1, r2):
    """上半球均勻取樣，+Y 為法線。回傳單位方向。"""
    cos_t = r1
    sin_t = math.sqrt(max(0.0, 1.0 - cos_t * cos_t))
    phi = 2.0 * math.pi * r2
    return (sin_t * math.cos(phi), cos_t, sin_t * math.sin(phi))

def sample_cosine_hemisphere(r1, r2):
    """餘弦加權取樣  $p(\omega)=\cos\theta/\pi$ ，+Y 為法線。"""
    cos_t = math.sqrt(max(0.0, 1.0 - r1))
    sin_t = math.sqrt(r1)
    phi = 2.0 * math.pi * r2
    return (sin_t * math.cos(phi), cos_t, sin_t * math.sin(phi))

def estimate_lambert(rho, L_i_fn, N, mode, seed):
    """估計  $L_o = (\rho/\pi) \int L_i(\omega) \cos\theta \, d\omega$ 。
    mode ∈ {'uniform', 'cosine'}。"""
    rng = random.Random(seed)
    total = 0.0
    used = 0
    for _ in range(N):
        r1, r2 = rng.random(), rng.random()
```

```

    if mode == "uniform":
        w = sample_uniform_hemisphere(r1, r2)
        cos_t = w[1]
        p = 1.0 / (2.0 * math.pi)
    else:
        w = sample_cosine_hemisphere(r1, r2)
        cos_t = w[1]
        p = cos_t / math.pi
    if p <= 0.0:
        # PDF=0 且 f=0 的方向，貢獻定義為 0，直接跳過。
        # 若這裡誤加 f/p，就會得到 inf 或 NaN。
        continue
    total += (rho / math.pi) * L_i_fn(w) * max(0.0, cos_t) / p
    used += 1
return total / max(1, used)

def mean_std(xs):
    n = len(xs)
    m = sum(xs) / n
    v = sum((x - m) ** 2 for x in xs) / n
    return m, math.sqrt(v)

def self_check():
    L_const = lambda w: 1.0
    L_dir = lambda w: max(0.0, w[1]) # 非負的 cosθ 測試光
    rho = 0.6
    out = {}

    # 檢查 1：常數光真值 0.6，多 seed 平均要靠近。
    for mode in ("uniform", "cosine"):
        ests = [estimate_lambert(rho, L_const, 200, mode, s) for s in range(50)]
        m, sd = mean_std(ests)
        out[f"const_{mode}"] = (m, sd)
        assert abs(m - rho) < 0.05, (mode, m, sd)

    # 檢查 2：方向光真值 0.4。
    for mode in ("uniform", "cosine"):
        ests = [estimate_lambert(rho, L_dir, 500, mode, s) for s in range(50)]
        m, sd = mean_std(ests)
        out[f"dir_{mode}"] = (m, sd)
        assert abs(m - 0.4) < 0.05, (mode, m, sd)

    # 檢查 3：正常取樣的結果必須是有限實數；此處未測到 PDF=0。
    for mode in ("uniform", "cosine"):
        v = estimate_lambert(rho, L_const, 100, mode, 12345)
        assert math.isfinite(v), (mode, v)

    # 檢查 4：餘弦加權在常數光下變異數應接近零。
    _, sd_cos = out["const_cosine"]
    assert sd_cos < 1e-8, sd_cos
    return out

if __name__ == "__main__":
    r = self_check()

```

```
for k, v in r.items():
    print(f"{k}: mean={v[0]:.5f} sd={v[1]:.5f}")
```

幾個使用上的要點：

- 取樣函式只接收兩個純量 r_1, r_2 ，方便做固定序列測試與跨 seed 對照。
- `p <= 0.0` 是防止除以零的分支：本章使用 Python 標準庫浮點數，直接除以零會拋出 `ZeroDivisionError`。程式跳過該樣本，並以 `max(1, used)` 避免最後除以零；但目前的正常取樣自檢不會觸發此分支。若故障分佈在被積函數非零處給出零 PDF，跳過樣本也無法使估計有效；習題 2 才要求刻意注入零 PDF 以檢查防護行為。
- 餘弦加權模式下每一樣本值都等於 ρL_i ，因此常數光檢驗的標準差數值上應為 0（雙精度浮點運算下約 10^{-16} 量級）。

測試與預期結果

執行 `python mc_lambert.py`，預期輸出（每個 seed 為 0 到 49，樣本數如上）：

```
const_uniform: mean=0.600 sd=0.025
const_cosine:  mean=0.600 sd=0.00000
dir_uniform:   mean=0.400 sd=0.016
dir_cosine:    mean=0.400 sd=0.006
```

這些 `sd` 值的量級可由例題三直接核對：

- `const_cosine` 恆為 0，因為單樣本估計本身就是常數 ρL_i 。
- `dir_uniform`、`dir_cosine` 分別對應 $\sigma_u/\sqrt{500} \approx 0.016$ 與 $\sigma_c/\sqrt{500} \approx 0.006$ 。
- `const_uniform` 的單樣本值為 $g_u = 1.2 \cos \theta$ 。由 $E[g_u] = 0.6$ 、 $E[g_u^2] = 1.44 \cdot (1/2\pi) \cdot (2\pi/3) = 0.48$ ，得 $\sigma_u^2 = 0.12$ 、 $\sigma_u \approx 0.3464$ 、 $\sigma_u/\sqrt{200} \approx 0.0245$ ，即預期輸出的 0.025 量級。

收斂速率皆為 $1/\sqrt{N}$ 。若把 N 從 500 改為 50000，預期 `dir_*` 的 `sd` 下降約 10 倍。不要把標準差當作「準確度」；它只是估計值本身的不確定度。真值檢驗只能靠已知解析解。

以上皆為依據程式碼直讀與理論推導得到的預期結果；本卷未在特定硬體或環境中執行過。

除錯與常見陷阱

1. **立體角測度混用**：如果寫 $\int \cos \theta d\theta d\phi$ 而不是 $\int \cos \theta \sin \theta d\theta d\phi$ ，結果會少一個 $\sin \theta$ 且積分值不正確。任何時候從平面角度換算到立體角，都要檢查 $d\omega = \sin \theta d\theta d\phi$ 。
2. **PDF 為零除以零**：餘弦加權分佈在 θ 接近 $\pi/2$ 時 $p \rightarrow 0$ 。若你的取樣函式允許 $\cos \theta$ 恰好為零而分子 f 也為零，就必須事先判定：貢獻為 0，跳過，不要讓浮點產生 NaN。
3. **兩種取樣的形式寫反**：均勻半球是 `cosθ = r1`（或 `1-r1`），餘弦加權是 `cosθ = sqrt(1-r1)`。互換會同時錯分佈與錯估計。

4. 把 L_i 當作照明功率的平方： L_i 是輻射亮度；在多光源或紋理環境裡，它是「往方向 ω 看過去的值」，不隨立體角作進一步加權。
5. 缺少多 seed 重複：單次執行的估計值不可能代表收斂，一定要看多 seed 下的分布。
6. 變異數為零時誤以為有 bug：如果 p 與 f 恰好成正比、樣本值 f/p 就是常數，變異數自然為 0。這是最理想的狀況，不是錯誤。

養殖數位分身案例

設想在池體底部的平坦水泥面上，反照率 $\rho = 0.6$ ，入射光 L_i 來自四面八方的合成環境（例如一個均勻灰色天空模型加一個方向主光）。要不要用 Monte Carlo？只在連環境都無法解析求解時才需要。以下三種狀況依序複雜化：

1. 純 $L_i = 1$ ：本章例題一，可直接算真值 0.6；Monte Carlo 只是驗證工具。
2. 單一平行主光：解析可解，不需要 Monte Carlo 的隨機性。
3. 池體上方懸掛多盞燈、加上水波反射的環境光： L_i 隨方向變化且難以封閉式書寫，才進入 Monte Carlo 的適用範圍。

必須標註：以上都是合成光分布，與真實養殖池水面反射、藻類遮光、浮游顆粒散射完全不同。合成資料可以用來驗證估計式是否收斂、是否正確地積分 $\cos \theta$ ，但不能用來推論池底真實照度或魚類行為。

習題

習題 1（手算） 設 $\rho = 0.5$ ，入射光 $L_i(\omega) = 1 + \cos \theta$ 。請：(a) 計算真值 $L_o = \frac{\rho}{\pi} \int_{\Omega^+} (1 + \cos \theta) \cos \theta d\omega$ ；(b) 用均勻半球取樣寫出單樣本估計式的顯式形式；(c) 若某樣本為 $\theta = 30^\circ$ ，求出該估計值，並說明這個值與 (a) 的真值差多少。

習題 2（程式測試） 修改 `estimate_lambert`，讓它可選擇「刻意製造 PDF=0」的模式：在 `mode='broken'` 下對所有樣本使用 $p = 0$ 。先保留 `if p <= 0.0: continue`，確認 10 個樣本都被跳過；再於測試副本中移除此保護，直接計算 f/p ，觀察 Python 的例外。說明兩次結果的差異，以及為何合法取樣分佈必須在被積函數非零處具有正 PDF。不要在正式估計器中保留故障模式。

習題 3（反例／除錯） 有人把 `sample_cosine_hemisphere` 改成 `cos_t = math.sqrt(1 - r1); sin_t = 1 - cos_t`。請用至少兩個具體 r_1 值（例如 $r_1 = 0.25$ 與 $r_1 = 0.5$ ）檢查 `sin2+cos2` 是否為 1。指出這種錯誤會如何在後續估計式中污染結果，並說明哪些量仍然正確、哪些量會失準。

習題 4（整合應用） 你拿到一個合成環境光 $L_i(\omega) = \max(0, w_y)$ ，即入射亮度等於方向與世界 Y 軸夾角的餘弦。對一個反照率 $\rho = 0.5$ 、法線 $+Y$ 的水平表面，請：(a) 用文字寫出真值積分與其解析解；(b) 設計一份實驗計畫，用均勻半球與餘弦加權兩種模式各 50 個 seed、每個 $N = 1000$ ，比較標準誤差；(c) 說明這份計畫「能證明什麼」、「不能證明什麼」。

習題解答

習題 1 解答 (a) 展開：

$$\int_{\Omega^+} (1 + \cos \theta) \cos \theta d\omega = \int \cos \theta d\omega + \int \cos^2 \theta d\omega = \pi + \frac{2\pi}{3} = \frac{5\pi}{3}.$$

真值 $L_o = (0.5/\pi)(5\pi/3) = 5/6 \approx 0.8333$ 。

(b) 均勻半球 $p = 1/(2\pi)$ ，估計式：

$$\hat{L}_o^{(u)} = \frac{\rho}{\pi} (1 + \cos \theta) \cos \theta \cdot 2\pi = 1.0 \cdot (1 + \cos \theta) \cos \theta.$$

展開： $\hat{L}_o^{(u)} = \cos \theta + \cos^2 \theta$ 。

(c) $\theta = 30^\circ$ ： $\cos \theta = \sqrt{3}/2 \approx 0.8660$ ， $\cos^2 \theta = 0.75$ 。 $\hat{L}_o = 0.8660 + 0.75 = 1.6160$ 。

請注意：此值遠大於真值 $5/6 \approx 0.8333$ ，這是因為 Monte Carlo 估計在樣本數為 1 時具有高度隨機性。單一隨機樣本可能出現極端值，須透過大量樣本平均才能逼近真值。這不代表公式錯誤，而是估計式設計上的必然現象；下一節的程式正是用來檢驗這個收斂性。

習題 2 解答 保留原有保護時，所有 $p = 0$ 的樣本都會被 `continue` 跳過，因此 `used=0`，函式最後回傳 `total / max(1, used)=0.0`；它不會產生 `inf` 或 `nan`。這個結果只表示防護分支被觸發，並不是原積分的有效估計值。

若在測試副本中移除保護，Python 內建 `float` 直接執行非零有限數除以 `0.0` 時會拋出 `ZeroDivisionError`，而不是自動回傳 `inf`；`0.0/0.0` 同樣會拋出例外。某些採 IEEE 754 陣列運算的數值函式庫可能改以警告搭配 `inf` 或 `nan`，但本章標準庫程式不可假定該行為。

更根本的問題是支撐集條件：若被積函數在某方向非零而 PDF 為零， f/p 估計式在該處沒有定義，不能靠跳過樣本修復。正式程式應拒絕這種取樣分佈；只有在 PDF 與被積函數同時為零的零測度邊界上，才可依估計式的極限或已約分形式安全處理。

習題 3 解答 檢驗 $r_1 = 0.25$ ： $\cos \theta = \sqrt{0.75} \approx 0.8660$ ，錯誤版 $\sin \theta = 1 - 0.8660 = 0.1340$ 。則 $\sin^2 + \cos^2 = 0.017956 + 0.75 = 0.767956 \neq 1$ 。正確的 $\sin \theta$ 應為 $\sqrt{r_1} = 0.5$ ， $\sin^2 = 0.25$ ，總和 1。

$r_1 = 0.5$ ： $\cos \theta \approx 0.7071$ ，錯誤版 $\sin \theta = 0.2929$ ， $\sin^2 + \cos^2 \approx 0.0858 + 0.5 = 0.5858 \neq 1$ 。

方向向量因此不是單位向量， w_y 這個分量本身仍等於 $\cos \theta$ （錯誤只發生在 $\sin \theta$ ），所以在只使用 $\cos \theta$ 的純量估計式裡，期望值的方向可能沒壞；但向量不再能用來計算反射方向、夾角、著色或任何依賴 $|\omega| = 1$ 的後續運算。此外，若 PDF 寫成 $p = \cos \theta / \pi$ （與 r_1 相容）而樣本值 $f/p = \rho L_i$ ，簡潔性質依然成立；但若任何一步驟查到 $\sin \theta$ 或把 ω 當成單位向量處理，錯誤就會系統性累積。這也是為什麼取樣函式一定要用「 $\sin^2 + \cos^2 = 1$ 」這種獨立恆等式做單元測試。

習題 4 解答 (a) 表面法線 $+Y$ ，入射方向 ω 與 $+Y$ 夾角即 θ ， $w_y = \cos \theta$ 。故 $L_i(\omega) = \cos \theta$ ($\theta \in [0, \pi/2]$ ；下半球 $w_y < 0$ 時 $L_i = 0$ ，不屬於 Ω^+ 故無貢獻)。

$$L_o = \frac{\rho}{\pi} \int_{\Omega^+} \cos \theta \cdot \cos \theta d\omega = \frac{\rho}{\pi} \cdot \frac{2\pi}{3} = \frac{2\rho}{3}.$$

代入 $\rho = 0.5$ 得真值 $1/3 \approx 0.3333$ 。

(b) 實驗計畫：

- 每個模式跑 50 個 seed (seed = 0..49)，每個 seed 內 $N = 1000$ 樣本。
- 計算每個 seed 的單次估計值，再對 50 個估計值求平均與樣本標準差。
- 若平均與 $1/3$ 的距離在閾值 (例如 0.01) 內，且兩模式結果都符合，即接受估計式實作無誤。
- 記錄每個模式的標準差，用以比較兩者的效率。均勻取樣樣本值為 $2\rho \cos^2 \theta$ ，餘弦取樣樣本值為 $\rho \cos \theta$ ；兩者期望相同，但第二動差不同，因此期望變異數也不同。

(c) 能證明的：程式實作了兩種取樣、估計式在合成環境下無偏、以及二者在有限樣本下的效率差異。不能證明的：這份實驗假設的環境光 $L_i = \max(0, w_y)$ 只是一個數學合成模型，它既不描述任何真實天空，也不描述實際養殖池光照環境；實驗結果不能推論真實場景的照度或生態效應。

本章小結

本章的核心可以壓縮成三句話：

1. **積分即期望**：只要找到一個在 $f \neq 0$ 處為正的 PDF， f/p 的樣本平均就是無偏估計。
2. **PDF 可以設計**：要降低變異數，就讓 p 盡量與 f 在形狀上匹配；餘弦加權半球在 Lambert 反射下把變異數壓到零，就是這個原則的極致例子。
3. **退化要事先處理**：PDF 為零的方向必須跳過，立體角測度必須用 $\sin \theta d\theta d\phi$ ；這兩件事做錯，程式不會報錯，只會靜靜地輸出錯的數。

後面第 22 章會把這套估計式放進完整的路徑追蹤迴圈。本章的單點半球積分是那條迴圈的內核；一旦你確定了 PDF、估計式與零 PDF 的處理原則，路徑追蹤只是把它串起來的工程問題。

參考來源

- [G1] PBRT 4：Transformations，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G2] PBRT 4：Reflection Models，https://pbr-book.org/4ed/Reflection_Models
- [G3] PBRT 4：The Light Transport Equation，https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- [G4] Ray Tracing in One Weekend，<https://raytracing.github.io/books/RayTracingInOneWeekend.html>

- [G5] LearnOpenGL：Transformations，<https://learnopengl.com/Getting-started/Transformations>
- [G6] Blender Manual：Skinning Introduction，<https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>
- [G7] NumPy 線性代數參考，<https://numpy.org/doc/stable/reference/routines.linalg.html>
- [G8] Khronos glTF 2.0 規格，<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

以上連結僅為延伸查閱入口；本章推導與數值皆獨立撰寫，未逐條對應來源論點，也未聲稱已跑過官方範例或在其環境中驗證。

第 22 章 路徑追蹤與重要性取樣

第四部 | 光線追蹤與光傳輸

學習目標與先備知識

本章把上一章的 Monte Carlo 積分推進到完整影像生成。完成後，讀者應能：

1. 從渲染方程理解路徑追蹤的遞迴結構。
2. 定義並更新路徑權重 `throughput`。
3. 區分 BSDF 取樣與顯式光源取樣。
4. 用機率密度函數 PDF 正確加權樣本。
5. 以俄羅斯輪盤終止長路徑而不系統性壓低期望值。
6. 理解多重重要性取樣 MIS 如何結合兩種取樣策略。
7. 分辨「表面點事件」與「方向事件」，避免混用 PDF。
8. 建立固定種子、低解析度、固定樣本預算的 CPU 路徑追蹤器。

先備知識包括射線求交、Lambert BRDF、半球立體角、隨機變數、PDF 與 Monte Carlo 估計。全章顏色皆為線性 RGB； L 、 f_r 等 RGB 值是三個色彩通道的數值近似，不代表完整光譜模型。

問題與直覺

光線投射器只找出相機最先看見的表面；局部照明通常只計算表面直接接收的光。實際場景中，池壁會把光反射到魚體，魚腹也可能被池底的間接反射照亮。這類多次反射形成全域光傳輸。

路徑追蹤從相機發出射線。射線碰到表面後，依材質抽樣新方向，再繼續追蹤。每條路徑可能：

- 直接碰到發光表面；
- 經一次或多次反射後碰到光源；
- 離開場景而取得環境光；
- 被俄羅斯輪盤提前終止；
- 到達工程設定的最大深度。

若只靠隨機反射方向偶然撞到小光源，大多數樣本不會得到直接光，影像便有強烈雜訊。重要性取樣不改變原積分，而是把更多樣本放在貢獻可能較大的方向，再以 PDF 補償抽樣不均。

光源取樣擅長尋找小面積光源；BSDF 取樣擅長尋找材質偏好的方向。MIS 讓兩者共同工作。不過，兩種策略的 PDF 必須描述同一種隨機事件。若一邊描述「抽到光源表面點」，另一邊描述「抽到方向」，便不能直接比較。

數學與幾何推導

1. 渲染方程

表面點 $\mathbf{x}$ 沿出射方向 ω_o 的輻射亮度為

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\mathcal{H}^2} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) |\mathbf{n} \cdot \omega_i| d\omega_i.$$

其中：

- L_o ：出射輻射亮度，單位可寫為 $\text{W m}^{-2} \text{sr}^{-1}$ ；
- L_e ：表面自身發光；
- L_i ：沿 ω_i 入射的輻射亮度；
- f_r ：BRDF，單位為 sr^{-1} ；
- $\mathbf{n}$ ：表面單位法線；
- $\mathcal{H}^2$ ：法線上方半球；
- $d\omega_i$ ：微小立體角，單位為 sr 。

若射線從 $\mathbf{x}$ 沿 ω_i 首次碰到另一表面 $\mathbf{x}'$ ，則

$$L_i(\mathbf{x}, \omega_i) = L_o(\mathbf{x}', -\omega_i).$$

代回後，右側又出現另一個出射輻射亮度，形成遞迴結構。路徑追蹤以隨機樣本估計這個遞迴積分。

2. 單次 Monte Carlo 估計

若方向 ω 由 PDF $p(\omega)$ 抽樣，且在非零被積函數處滿足 $p(\omega) > 0$ ，則

$$\int_{\mathcal{H}^2} g(\omega) d\omega = \mathbb{E} \left[\frac{g(\omega)}{p(\omega)} \right].$$

對渲染方程，

$$g(\omega_i) = f_r L_i |\mathbf{n} \cdot \omega_i|.$$

單樣本估計量為

$$\hat{L}_o = L_e + \frac{f_r(\omega_i, \omega_o) L_i(\omega_i) |\mathbf{n} \cdot \omega_i|}{p(\omega_i)}.$$

除以 PDF 是補償抽樣分布不均。若忘記除以 PDF，結果通常有偏；若 PDF 為零但貢獻非零，估計器則無法涵蓋該部分積分。

3. Lambert 材質與餘弦加權取樣

Lambert BRDF 為

$$f_r = \frac{\rho}{\pi},$$

其中 $\rho = (\rho_r, \rho_g, \rho_b)$ 是線性 RGB 反照率。

餘弦加權半球取樣的 PDF 是

$$p_{\text{bsdf}}(\omega_i) = \frac{\cos \theta}{\pi}, \quad \cos \theta = \max(0, \mathbf{n} \cdot \omega_i).$$

因此

$$\frac{f_r \cos \theta}{p_{\text{bsdf}}} = \frac{(\rho/\pi) \cos \theta}{\cos \theta / \pi} = \rho.$$

純 Lambert 表面使用餘弦取樣時，每次反射只需把 throughput 乘上反照率。這個化簡不適用於任意 BRDF。

4. 路徑 throughput

令 β 表示從相機到目前頂點累積的路徑權重。起始時

$$\beta_0 = (1, 1, 1).$$

每次抽樣方向後更新

$$\beta_{k+1} = \beta_k \odot \frac{f_r(\omega_{k+1}, \omega_k) |\mathbf{n} \cdot \omega_{k+1}|}{p(\omega_{k+1})}.$$

若目前頂點有發光 $\mathbf{L}_e$ ，像素貢獻為

$$\mathbf{L} \leftarrow \mathbf{L} + \beta \odot \mathbf{L}_e.$$

throughput 是路徑樣本在估計器中的乘法權重，不是「剩餘光能」。其值可能大於 1；任意夾住它會引入偏差。

5. 面積取樣與立體角 PDF

在表面點 $\mathbf{x}$ 抽樣光源點 $\mathbf{y}$ ，稱為 next-event estimation，簡稱 NEE。若以面積 PDF $p_A(\mathbf{y})$ 抽樣，令

$$\mathbf{d} = \mathbf{y} - \mathbf{x}, \quad r^2 = \|\mathbf{d}\|^2, \quad \omega_i = \frac{\mathbf{d}}{\|\mathbf{d}\|}.$$

光源法線為 $\mathbf{n}_L$ ，定義幾何 Jacobian 中的絕對餘弦

$$|\cos \theta_L| = |\mathbf{n}_L \cdot (-\omega_i)|.$$

單一表面分支轉成立體角 PDF：

$$p_{\omega,j}(\omega_i) = p_A(\mathbf{y}_j) \frac{r_j^2}{|\cos \theta_{L,j}|}.$$

這裡的絕對值來自面積與立體角的幾何轉換，不等於發光模型。單面光源是否沿該方向發光，仍應另以

$$\max(0, \mathbf{n}_L \cdot (-\omega_i))$$

判定。某表面點背向著色點時，貢獻可為零，但只要取樣器可能抽到該點，其面積 PDF 就不能被說成零。

6. 多對一映射與方向事件

面積點映射到方向不一定一對一。若同一方向 ω 可由光源上的多個表面點 $\mathbf{y}_j$ 生成，方向總 PDF 應加總所有分支：

$$p_{\omega}(\omega) = \sum_j p_A(\mathbf{y}_j) \frac{r_j^2}{|\cos \theta_{L,j}|}.$$

例如從球外觀察球面，一條穿過球的射線通常與球面有前、後兩個交點。若取樣器均勻抽整個球面，兩個點都可能映射到同一方向，因此不能只把前交點的 Jacobian 當成完整方向 PDF。

此外，後表面對單面向外發光球的直接貢獻為零，而且通常被前表面遮擋；但「貢獻為零」與「取樣機率為零」是兩件不同的事。

MIS 最簡單的做法，是讓所有策略都在方向事件空間工作：

- BSDF 策略直接抽樣方向；
- 光源策略也直接抽樣方向；
- 兩者 PDF 都以 sr^{-1} 表示；
- 射線的首次交點決定實際光源事件。

本章程式採此方法：從著色點直接均勻抽樣球形光源所張成的可見方向圓錐，而不是均勻抽整個球面。圓錐內每個方向對應首次可見球面交點，避免前、後表面多分支歧義。

7. 球形光源的方向圓錐取樣

設著色點為 $\mathbf{x}$ ，球心為 $\mathbf{c}$ ，半徑為 R ，且著色點位於球外。令

$$d = \|\mathbf{c} - \mathbf{x}\|, \quad d > R.$$

球在著色點所張成圓錐的半角 $\theta_{\max}$ 滿足

$$\sin \theta_{\max} = \frac{R}{d},$$

因此

$$\cos \theta_{\max} = \sqrt{1 - \frac{R^2}{d^2}}.$$

圓錐立體角為

$$\Omega = 2\pi(1 - \cos \theta_{\max}).$$

若在此圓錐內均勻抽樣方向，PDF 為

$$p_L(\omega) = \frac{1}{\Omega} = \frac{1}{2\pi(1 - \cos \theta_{\max})}.$$

這個 PDF 直接定義於方向事件，不需再做面積 Jacobian 轉換。抽得方向後，射線與球的第一交點就是 NEE 使用的光源點。若光源只向外發光，該可見前表面的出射餘弦應為正；程式仍會明確檢查朝向。

8. 多光源的完整 PDF

若場景有 M 個光源，通常先抽樣離散光源索引 J ，再由該光源抽樣方向。完整 PDF 為

$$p_L(\omega, j) = P(J = j) p(\omega \mid J = j).$$

直接光估計器與 MIS 權重都必須使用完整生成機率。最簡單是均勻選燈：

$$P(J = j) = \frac{1}{M}.$$

也可依近似功率 q_j 選燈：

$$P(J = j) = \frac{q_j}{\sum_{k=1}^M q_k}.$$

若多個光源分布都能生成同一方向，邊際方向 PDF 是混合密度：

$$p_L(\omega) = \sum_{j=1}^M P(J = j) p(\omega \mid J = j).$$

本章程式只有一個光源，所以選燈機率為 1。

9. 顯式光源估計

若光源策略直接抽樣方向 ω_i ，直接光單樣本估計為

$$\hat{\mathbf{L}}_{\text{direct}} = \beta \odot \frac{f_r \mathbf{L}_e \cos \theta}{p_L(\omega_i)} V(\mathbf{x}, \omega_i),$$

其中

$$\cos \theta = \max(0, \mathbf{n} \cdot \omega_i).$$

V 表示沿該方向的首次交點是否為所選光源。光源背面、被遮擋或材質半球外的樣本，其貢獻為零；這不表示原取樣 PDF 必須為零。

10. 俄羅斯輪盤與最大深度

令存活機率為 $p_s \in (0, 1]$ 。路徑以機率 $1 - p_s$ 終止；若存活，則

$$\beta \leftarrow \frac{\beta}{p_s}.$$

因為

$$p_s \frac{\beta}{p_s} + (1 - p_s) \mathbf{0} = \beta,$$

所以期望值不變。實作可用

$$p_s = \min(0.95, \max(\beta_r, \beta_g, \beta_b)).$$

最大深度與輪盤不同。若最大深度為 D ，所有更長路徑都被捨棄；只要其真實貢獻不為零，就會留下截斷偏差。輪盤則讓長路徑仍有存活機率並補償權重。本章同時使用兩者：輪盤控制平均成本，最大深度提供固定預算與防呆上限。

11. MIS 與成立條件

光源取樣與 BSDF 取樣可生成相同直接光方向。若各取一個樣本，power heuristic 為

$$w_L = \frac{p_L^2}{p_L^2 + p_B^2}, \quad w_B = \frac{p_B^2}{p_L^2 + p_B^2}.$$

兩個 PDF 必須描述相同方向事件，且使用相同測度。若某策略在該方向不可能生成樣本，其 PDF 與權重才應為零。

一般而言，MIS 權重需在有貢獻區域滿足

$$\sum_i w_i(x) = 1,$$

且所有非零貢獻都至少被一種策略覆蓋。

若策略 i 取 n_i 個樣本，power heuristic 應使用 $n_i p_i$ ：

$$w_i(x) = \frac{(n_i p_i(x))^2}{\sum_j (n_j p_j(x))^2}.$$

本章每個頂點各取一個光源方向樣本與一個 BSDF 樣本，所以 $n_L = n_B = 1$ 。

逐步手算例題

例題一：Lambert throughput 更新

某表面反照率為

$$\boldsymbol{\rho} = (0.8, 0.6, 0.4),$$

入射方向餘弦為 0.5，進入表面前

$$\boldsymbol{\beta} = (0.5, 0.5, 0.5).$$

由

$$f_r = \frac{\boldsymbol{\rho}}{\pi}, \quad p_B = \frac{0.5}{\pi},$$

可得

$$\boldsymbol{\beta}' = \boldsymbol{\beta} \odot \frac{(\boldsymbol{\rho}/\pi)0.5}{0.5/\pi} = (0.4, 0.3, 0.2).$$

例題二：面積 PDF 轉成立體角 PDF

面積為 $A_L = 2 \text{ m}^2$ 的平面光源被均勻取樣。若

$$r^2 = 9 \text{ m}^2, \quad |\cos \theta_L| = 0.5,$$

則

$$p_A = \frac{1}{2} = 0.5 \text{ m}^{-2},$$

以及

$$p_\omega = p_A \frac{r^2}{|\cos \theta_L|} = 0.5 \frac{9}{0.5} = 9 \text{ sr}^{-1}.$$

PDF 大於 1 並不違法；密度不是機率本身。

例題三：球形光源的圓錐 PDF

著色點到球心距離為 $d = 5$ 公尺，球半徑為 $R = 1$ 公尺。則

$$\cos \theta_{\max} = \sqrt{1 - \frac{1^2}{5^2}} = \sqrt{\frac{24}{25}} \approx 0.979796.$$

圓錐立體角為

$$\Omega = 2\pi(1 - 0.979796) \approx 0.126946 \text{ sr}.$$

均勻方向 PDF 為

$$p_L = \frac{1}{\Omega} \approx 7.877 \text{ sr}^{-1}.$$

圓錐外方向的 PDF 為零；圓錐內方向皆為此常數。這與均勻抽整個球面不是同一個取樣分布。

例題四：球面前後分支

若改成均勻抽整個球面，一條穿過球心附近的方向通常對應前、後兩個球面點。假設兩分支轉換後密度分別為

$$p_{\omega,1} = 3 \text{ sr}^{-1}, \quad p_{\omega,2} = 5 \text{ sr}^{-1},$$

則方向總 PDF 為

$$p_\omega = 3 + 5 = 8 \text{ sr}^{-1}.$$

即使後表面因單面發光或遮擋而貢獻為零，只要取樣器可能抽到它，它仍屬於生成該方向的分支。不能因貢獻為零就把其取樣密度改成零。

例題五：MIS 與俄羅斯輪盤

若

$$p_L = 0.8, \quad p_B = 0.2,$$

則

$$w_L = \frac{0.8^2}{0.8^2 + 0.2^2} \approx 0.941176,$$

$$w_B \approx 0.058824.$$

另設 throughput 為

$$\beta = (0.4, 0.2, 0.1),$$

存活機率 $p_s = 0.4$ 。存活後

$$\beta' = (1, 0.5, 0.25).$$

其期望為

$$0.4(1, 0.5, 0.25) + 0.6(0, 0, 0) = (0.4, 0.2, 0.1).$$

實作與程式

以下是可獨立閱讀的小型 CPU 路徑追蹤器。場景只包含 Lambert 球、單一球形面積光源與常數環境光；輸出 ASCII PPM。解析度為 32×24 、每像素 16 樣本、最多 8 次交點，使用固定亂數種子。

程式直接抽樣球形光源的可見方向圓錐，因此 NEE 與 BSDF 都使用方向事件及 sr^{-1} PDF。

```
import math
from dataclasses import dataclass
import numpy as np

PI = math.pi
EPS = 1e-5

def normalize(v):
    v = np.asarray(v, dtype=np.float64)
    n = float(np.linalg.norm(v))
    if not np.isfinite(n) or n <= 0.0:
        raise ValueError("不能正規化零長或非有限向量")
    return v / n

def power_heuristic(pdf_a, pdf_b):
    if not np.isfinite(pdf_a) or not np.isfinite(pdf_b):
```

```

        raise ValueError("PDF 必須是有限值")
    if pdf_a < 0.0 or pdf_b < 0.0:
        raise ValueError("PDF 不可為負")
    scale = max(pdf_a, pdf_b)
    if scale == 0.0:
        return 0.0
    a = pdf_a / scale
    b = pdf_b / scale
    return (a * a) / (a * a + b * b)

@dataclass
class Sphere:
    center: np.ndarray
    radius: float
    albedo: np.ndarray
    emission: np.ndarray

@dataclass
class Hit:
    t: float
    point: np.ndarray
    normal: np.ndarray
    sphere_index: int

def intersect_sphere(origin, direction, sphere, t_min=EPS, t_max=math.inf):
    oc = origin - sphere.center
    half_b = float(np.dot(oc, direction))
    c = float(np.dot(oc, oc) - sphere.radius * sphere.radius)
    discriminant = half_b * half_b - c
    if discriminant < 0.0:
        return None

    root = math.sqrt(discriminant)
    for t in (-half_b - root, -half_b + root):
        if t_min < t < t_max:
            point = origin + t * direction
            normal = normalize(point - sphere.center)
            return t, point, normal
    return None

def intersect_scene(origin, direction, spheres, t_max=math.inf):
    closest = t_max
    best = None
    for index, sphere in enumerate(spheres):
        result = intersect_sphere(
            origin, direction, sphere, EPS, closest
        )
        if result is not None:
            t, point, normal = result
            closest = t
            best = Hit(t, point, normal, index)
    return best

def make_basis(normal):

```

```

normal = normalize(normal)
if abs(normal[1]) < 0.999:
    tangent = normalize(np.cross([0.0, 1.0, 0.0], normal))
else:
    tangent = normalize(np.cross([1.0, 0.0, 0.0], normal))
bitangent = np.cross(normal, tangent)
return tangent, bitangent, normal

def sample_cosine_hemisphere(normal, rng):
    u1 = rng.random()
    u2 = rng.random()
    r = math.sqrt(u1)
    phi = 2.0 * PI * u2

    local = np.array([
        r * math.cos(phi),
        r * math.sin(phi),
        math.sqrt(max(0.0, 1.0 - u1)),
    ])

    tangent, bitangent, n = make_basis(normal)
    direction = normalize(
        local[0] * tangent
        + local[1] * bitangent
        + local[2] * n
    )
    cosine = max(0.0, float(np.dot(n, direction)))
    return direction, cosine / PI

def sphere_light_cone(point, light):
    to_center = light.center - point
    distance2 = float(np.dot(to_center, to_center))
    radius2 = light.radius * light.radius
    if distance2 <= radius2:
        raise ValueError("本章球光源取樣要求著色點位於光源球外")

    axis = to_center / math.sqrt(distance2)
    cos_theta_max = math.sqrt(max(0.0, 1.0 - radius2 / distance2))
    solid_angle = 2.0 * PI * (1.0 - cos_theta_max)
    if solid_angle <= 0.0:
        raise ValueError("球形光源立體角無法可靠表示")
    return axis, cos_theta_max, solid_angle

def sample_sphere_light_direction(point, light, rng):
    axis, cos_theta_max, solid_angle = sphere_light_cone(
        point, light
    )

    u1 = rng.random()
    u2 = rng.random()
    cos_theta = 1.0 - u1 * (1.0 - cos_theta_max)
    sin_theta = math.sqrt(max(0.0, 1.0 - cos_theta * cos_theta))
    phi = 2.0 * PI * u2

    tangent, bitangent, axis = make_basis(axis)
    direction = normalize(

```

```

        sin_theta * math.cos(phi) * tangent
        + sin_theta * math.sin(phi) * bitangent
        + cos_theta * axis
    )
pdf = 1.0 / solid_angle

result = intersect_sphere(point, direction, light)
if result is None:
    raise RuntimeError("圓錐方向理應與球形光源相交")
_, light_point, light_normal = result
return direction, light_point, light_normal, pdf

def sphere_light_direction_pdf(point, direction, light):
    """
    回傳球形光源的方向圓錐 PDF。
    PDF 描述方向能否由取樣器生成，不以發光朝向歸零。
    """
    _, _, solid_angle = sphere_light_cone(point, light)
    result = intersect_sphere(point, direction, light)
    if result is None:
        return 0.0
    return 1.0 / solid_angle

def visible_to_light(point, direction, distance, spheres):
    blocker = intersect_scene(
        point + EPS * direction,
        direction,
        spheres,
        t_max=distance - EPS,
    )
    return blocker is None

def trace_path(origin, direction, spheres, light_index, rng, max_depth=8):
    radiance = np.zeros(3, dtype=np.float64)
    beta = np.ones(3, dtype=np.float64)
    environment = np.array([0.02, 0.03, 0.05])

    previous_point = None
    previous_bsdf_pdf = 0.0

    for depth in range(max_depth):
        hit = intersect_scene(origin, direction, spheres)

        if hit is None:
            radiance += beta * environment
            break

        obj = spheres[hit.sphere_index]

        if np.any(obj.emission > 0.0):
            # 單面向外發光：-direction 是從光源指向前一頂點。
            front_emission = float(np.dot(hit.normal, -direction)) > 0.0
            if front_emission:
                if depth == 0 or previous_point is None:
                    weight = 1.0
                else:

```

```

        light_pdf = sphere_light_direction_pdf(
            previous_point, direction, obj
        )
        weight = power_heuristic(
            previous_bsdf_pdf, light_pdf
        )
        radiance += beta * obj.emission * weight
    break

normal = hit.normal
outgoing = -direction
if np.dot(normal, outgoing) < 0.0:
    normal = -normal

# 顯式抽樣唯一球形光源的可見方向圓錐。
light = spheres[light_index]
(
    wi,
    light_point,
    light_normal,
    light_pdf,
) = sample_sphere_light_direction(hit.point, light, rng)

cosine_surface = max(0.0, float(np.dot(normal, wi)))
cosine_emission = max(
    0.0, float(np.dot(light_normal, -wi))
)
distance = float(np.linalg.norm(light_point - hit.point))

if (
    cosine_surface > 0.0
    and cosine_emission > 0.0
    and visible_to_light(
        hit.point, wi, distance, spheres
    )
):
    bsdf_pdf = cosine_surface / PI
    mis_weight = power_heuristic(
        light_pdf, bsdf_pdf
    )
    brdf = obj.albedo / PI
    radiance += (
        beta
        * brdf
        * light.emission
        * cosine_surface
        * mis_weight
        / light_pdf
    )

new_direction, bsdf_pdf = sample_cosine_hemisphere(
    normal, rng
)
if bsdf_pdf <= 0.0:
    break

# Lambert 配合餘弦取樣： $f \cdot \cos / \text{pdf} = \text{albedo}$ 。
beta *= obj.albedo

```



```

        previous_point = hit.point.copy()
        previous_bsdf_pdf = bsdf_pdf

    if depth >= 2:
        survival = min(0.95, float(np.max(beta)))
        if survival <= 0.0 or rng.random() > survival:
            break
        beta /= survival

    origin = hit.point + EPS * new_direction
    direction = new_direction

# max_depth 耗盡時直接回傳，可能留下截斷偏差。
return radiance

def make_camera_ray(x, y, width, height, rng):
    origin = np.array([0.0, 1.0, 4.5])
    target = np.array([0.0, 0.0, -2.0])
    world_up = np.array([0.0, 1.0, 0.0])

    forward = normalize(target - origin)
    right = normalize(np.cross(forward, world_up))
    up = np.cross(right, forward)

    aspect = width / height
    scale = math.tan(math.radians(45.0) * 0.5)

    pixel_x = (x + rng.random()) / width
    pixel_y = (y + rng.random()) / height
    screen_x = (2.0 * pixel_x - 1.0) * aspect * scale
    screen_y = (1.0 - 2.0 * pixel_y) * scale

    return origin, normalize(
        forward + screen_x * right + screen_y * up
    )

def linear_to_srgb(value):
    value = max(0.0, value)
    if value <= 0.0031308:
        return 12.92 * value
    return 1.055 * value ** (1.0 / 2.4) - 0.055

def write_ppm(filename, image):
    height, width, _ = image.shape
    with open(filename, "w", encoding="ascii") as file:
        file.write(f"P3\n{width} {height}\n255\n")
        for y in range(height):
            values = []
            for x in range(width):
                rgb = [
                    linear_to_srgb(float(c))
                    for c in image[y, x]
                ]
                rgb8 = [
                    max(0, min(255, int(round(c * 255.0))))
                    for c in rgb
                ]

```

```

        ]
        values.extend(str(v) for v in rgb8)
        file.write(" ".join(values) + "\n")

def self_test():
    sphere = Sphere(
        np.array([0.0, 0.0, -3.0]),
        1.0,
        np.array([0.8, 0.8, 0.8]),
        np.zeros(3),
    )
    result = intersect_sphere(
        np.array([0.0, 0.0, 0.0]),
        np.array([0.0, 0.0, -1.0]),
        sphere,
    )
    assert result is not None
    assert np.isclose(result[0], 2.0)

    assert np.isclose(power_heuristic(1.0, 1.0), 0.5)
    assert np.isclose(
        power_heuristic(0.8, 0.2), 0.64 / 0.68
    )

    rng = np.random.default_rng(7)
    direction, pdf = sample_cosine_hemisphere(
        np.array([0.0, 1.0, 0.0]), rng
    )
    assert np.isclose(np.linalg.norm(direction), 1.0)
    assert direction[1] >= 0.0
    assert pdf >= 0.0

    try:
        normalize([0.0, 0.0, 0.0])
    except ValueError:
        pass
    else:
        raise AssertionError("零長向量應拋出 ValueError")

    light = Sphere(
        np.array([0.0, 0.0, -5.0]),
        1.0,
        np.zeros(3),
        np.ones(3),
    )
    point = np.array([0.0, 0.0, 0.0])
    center_direction = np.array([0.0, 0.0, -1.0])
    cone_pdf = sphere_light_direction_pdf(
        point, center_direction, light
    )
    expected_cos = math.sqrt(24.0 / 25.0)
    expected_pdf = 1.0 / (
        2.0 * PI * (1.0 - expected_cos)
    )
    assert np.isclose(cone_pdf, expected_pdf)

    miss_pdf = sphere_light_direction_pdf(
        point, np.array([1.0, 0.0, 0.0]), light

```

```

)
assert miss_pdf == 0.0

def main():
    self_test()

    width = 32
    height = 24
    samples_per_pixel = 16
    max_depth = 8
    rng = np.random.default_rng(20250308)

    spheres = [
        Sphere(
            np.array([0.0, -1001.0, -2.0]),
            1000.0,
            np.array([0.65, 0.72, 0.75]),
            np.zeros(3),
        ),
        Sphere(
            np.array([-0.9, -0.2, -2.7]),
            0.8,
            np.array([0.15, 0.55, 0.75]),
            np.zeros(3),
        ),
        Sphere(
            np.array([0.9, -0.4, -2.0]),
            0.6,
            np.array([0.85, 0.35, 0.12]),
            np.zeros(3),
        ),
        Sphere(
            np.array([0.0, 3.0, -2.5]),
            0.6,
            np.zeros(3),
            np.array([12.0, 11.0, 9.0]),
        ),
    ]
    light_index = 3
    image = np.zeros((height, width, 3), dtype=np.float64)

    for y in range(height):
        for x in range(width):
            total = np.zeros(3)
            for _ in range(samples_per_pixel):
                origin, direction = make_camera_ray(
                    x, y, width, height, rng
                )
                total += trace_path(
                    origin,
                    direction,
                    spheres,
                    light_index,
                    rng,
                    max_depth,
                )
            image[y, x] = total / samples_per_pixel

```

```
write_ppm("path_trace.ppm", image)
print("預期：寫出 path_trace.ppm")

if __name__ == "__main__":
    main()
```

程式只寫出 PPM，不讀取檔案。固定種子使相同程式、參數與隨機數實作可重現；修改取樣呼叫順序或函式庫版本後，不保證逐位元相同。

測試與預期結果

`self_test()` 檢查：

1. 球面交點預期為 $t = 2$ 。
2. 相同 PDF 的 MIS 權重預期為 0.5。
3. `power_heuristic(0.8, 0.2)` 預期約為 0.94117647。
4. 餘弦半球樣本為單位向量且位於法線上方。
5. 零長向量預期拋出 `ValueError`。
6. 距離 5、公尺半徑 1 的球光源，其中心方向 PDF 預期約為 7.877 sr^{-1} 。
7. 完全錯過球光源的方向 PDF 預期為 0。

成功完成後，終端文字預期為：

```
預期：寫出 path_trace.ppm
```

影像預期包含灰藍地面、左側藍球、右側橙球與上方亮球。陰影與間接光仍帶有 Monte Carlo 雜訊；這是結構預期，不是已執行的參考圖比對。

可重現實驗包括：

- 將每像素樣本數由 16 改為 1，預期雜訊增加。
- 關閉 NEE，預期小光源更難由 BSDF 路徑命中。
- 刪除 `beta /= survival`，預期結果系統性偏暗。
- 把兩個 MIS 權重都設為 1，可能重複計算直接光。
- 將 `max_depth` 改為 1，間接反射預期顯著減少。

效能應由讀者在自己的環境計時；本章不提供未執行的 FPS。

除錯與常見陷阱

1. 忘記除以 PDF

非均勻取樣後必須以 $1/p$ 補償，否則亮度會隨取樣策略改變。

2. 混用事件與測度

面積 PDF 單位為 m^{-2} ，方向 PDF 單位為 sr^{-1} 。MIS 比較前必須統一事件空間及測度。

3. 把零貢獻當成零 PDF

背向光源點可能不發光，被遮擋點也可能沒有貢獻；只要取樣器可能生成該樣本，其 PDF 就不能因此改成零。

4. 忽略多對一映射

均勻抽整個球面時，同一方向可能對應前後兩個表面點。方向 PDF 必須加總分支。若不想處理，可像本章一樣直接取樣可見方向圓錐。

5. 漏掉選燈機率

多光源完整 PDF 是選燈機率乘上條件 PDF。漏掉離散機率會使估計量與 MIS 權重錯誤。

6. 多樣本 MIS 沿用單樣本公式

不同策略樣本數不同時，權重應使用 $n_i p_i$ 。

7. 俄羅斯輪盤未補償

存活後必須除以 p_s 。只刪除路徑而不補償會偏暗。

8. 把最大深度當成無偏終止

固定深度會捨棄更長路徑；它是預算上限，不是輪盤補償。

9. 自相交與 epsilon

次級射線需避開原表面，但 EPS 應依場景尺度設定，不能視為物理厚度。

10. 在 sRGB 中累積

所有樣本必須在線性 RGB 累積與平均，輸出時才轉為 sRGB。

養殖數位分身案例

可把程式中的大球頂部視為簡化池底，兩個小球視為魚體占位幾何，球形光源視為合成照明設備。不同反射率可展示直接光、陰影與間接反射如何改變外觀。

可稽核合成資料至少應記錄：

- 影像寬高、每像素樣本數與亂數種子；
- 最大路徑深度與輪盤起始深度；
- 幾何位置、半徑、反射率與發光值；
- 光源取樣事件空間、選燈分布及方向 PDF；
- EPS 與長度單位；
- 是否啟用 NEE 與 MIS；
- 線性 RGB 至 sRGB 的輸出轉換。

若擴充多盞燈，可用光源面積 A_j 與平均發光亮度 $\bar{L}_j$ 建立近似權重

$$q_j = A_j \bar{L}_j,$$

再正規化成選燈機率。這只改變取樣效率，不改變光源物理參數。

增加樣本數只會降低此簡化模擬器的取樣雜訊，不會使 Lambert 球、常數環境光或空氣中的光傳輸自動成為真實水下模型。水面折射、吸收與體散射需另行建模。

習題

習題 1：手算

某 Lambert 表面的反射率為 (0.6, 0.3, 0.2)，目前 throughput 為 (0.5, 0.8, 1.0)。使用餘弦加權半球取樣後，求新 throughput。

習題 2：程式測試

為 `power_heuristic` 加入測試：

1. $p_a = p_b = 0.4$ 時權重為 0.5；
2. $p_a = 0, p_b = 1$ 時權重為 0；

3. 兩個 PDF 都為零時回傳 0；
4. 負 PDF 會拋出 `ValueError`。

習題 3：反例／除錯

某程式均勻抽樣整個球面，卻在抽到背向著色點的球面位置時把 PDF 設為零。說明錯誤。若同一方向的前、後分支密度分別為 2 與 3 sr^{-1} ，求方向總 PDF。

習題 4：俄羅斯輪盤

若 throughput 為 $(0.3, 0.2, 0.1)$ ，存活機率為 0.25，求存活後的正確 throughput，並說明未補償時的偏差方向。

習題 5：整合應用

Lambert 反照率 $\rho = 0.5$ ，且

$$L_e = 8, \quad \cos \theta = 0.6, \quad p_L = 0.9, \quad p_B = 0.3, \quad \beta = 0.75.$$

可見性為 1。求使用 power heuristic 的直接光估計

$$\beta \frac{(\rho/\pi) L_e \cos \theta}{p_L} w_L.$$

習題 6：多光源與多樣本 MIS

四個光源的選取機率為 $(0.1, 0.2, 0.3, 0.4)$ 。第三個光源的條件方向 PDF 為 5 sr^{-1} 。另有 $n_L = 4$ 、 $n_B = 1$ ，且某方向上 $p_L = 0.2, p_B = 0.4$ 。

1. 求第三個光源的完整 PDF。
2. 求含樣本數的 w_L, w_B 。

習題解答

習題 1 解答

Lambert BRDF 配合餘弦取樣時，權重因子等於反照率：

$$\beta' = (0.5, 0.8, 1.0) \odot (0.6, 0.3, 0.2) = (0.3, 0.24, 0.2).$$

習題 2 解答

```
def test_power_heuristic():
    assert np.isclose(power_heuristic(0.4, 0.4), 0.5)
    assert np.isclose(power_heuristic(0.0, 1.0), 0.0)
    assert np.isclose(power_heuristic(0.0, 0.0), 0.0)

    try:
        power_heuristic(-0.1, 1.0)
    except ValueError:
        pass
    else:
        raise AssertionError("負 PDF 應被拒絕")
```

習題 3 解答

背向表面點可能不發光，但均勻球面取樣器仍可能抽到它，所以其面積 PDF 不是零。若映射到同一方向的兩個分支密度為 2 與 3，方向總 PDF 是

$$p_{\omega} = 2 + 3 = 5 \text{ sr}^{-1}.$$

貢獻函數可因朝向或遮擋而為零；PDF 必須忠實描述取樣器實際生成樣本的機率。

習題 4 解答

存活後應除以 0.25：

$$\beta' = \frac{(0.3, 0.2, 0.1)}{0.25} = (1.2, 0.8, 0.4).$$

若不補償，期望值只剩原值的四分之一，結果系統性偏暗。

習題 5 解答

光源策略權重為

$$w_L = \frac{0.9^2}{0.9^2 + 0.3^2} = 0.9.$$

因此

$$\hat{L}_{\text{direct}} = 0.75 \frac{(0.5/\pi)(8)(0.6)}{0.9} (0.9) = \frac{1.8}{\pi} \approx 0.573.$$

習題 6 解答

第三個光源的完整 PDF 為

$$0.3 \times 5 = 1.5 \text{ sr}^{-1}.$$

多樣本 power heuristic 使用

$$n_L p_L = 4(0.2) = 0.8, \quad n_B p_B = 1(0.4) = 0.4.$$

所以

$$w_L = \frac{0.8^2}{0.8^2 + 0.4^2} = 0.8,$$

$$w_B = 0.2.$$

本章小結

路徑追蹤把渲染方程的遞迴積分轉成隨機光路徑。throughput 累積每次反射的

$$\frac{f_r \cos \theta}{p}$$

因子；對 Lambert BRDF 與餘弦取樣，此因子簡化為反照率。

顯式光源取樣與 BSDF 取樣進行 MIS 時，必須描述同一事件並使用相同測度。面積取樣映射到方向若不是一對一，方向 PDF 必須加總所有分支。背向、遮擋或零發光只會令貢獻為零，不能任意改寫取樣 PDF。本章直接抽樣球形光源的可見方向圓錐，以首次交點定義光源事件。

俄羅斯輪盤透過存活後除以存活機率維持期望值；固定最大深度則可能留下截斷偏差。固定種子、解析度、樣本數、事件定義、PDF 與場景參數，是建立可重現路徑追蹤實驗的最低要求。

參考來源

- [G2] *Physically Based Rendering, 4th ed.* : Reflection Models
https://pbr-book.org/4ed/Reflection_Models
- [G3] *Physically Based Rendering, 4th ed.* : The Light Transport Equation
https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation

- [G4] *Ray Tracing in One Weekend*
<https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- [G7] NumPy 線性代數參考
<https://numpy.org/doc/stable/reference/routines.linalg.html>

以上來源供讀者回查 BRDF、光傳輸、路徑追蹤與 NumPy API 背景；本章推導、案例與程式依本書符號、單位及可重現性約定整理。

第23章 水面反射、折射與介質

學習目標與先備知識

本章要把射線追蹤中的「碰到水面」拆成三件事：決定反射與折射的方向、分配兩條路徑的光量，以及計算光在水中行進時的衰減。完成後，你應能：

- 依射線所在介質選擇折射率及法線方向，用 Snell 定律求折射方向；
- 判斷全反射，並區分 Fresnel 反射率與固定的「透明度」；
- 用 Beer-Lambert 定律計算指定距離的吸收；
- 以可重現的小程式測試入水、出水、全反射及介質路徑長度。

先備知識是單位向量、內積、射線交點，以及前章的線性 RGB 與路徑追蹤概念。世界座標仍採右手系，+Y 向上；下例以水平面 $y = 0$ 為合成水面，空氣在 $y > 0$ ，水在 $y < 0$ 。長度以公尺、角度以弧度計。折射率 n 是無單位數；所有色彩光量先在線性 RGB 中計算。

問題與直覺

從池上方向下看魚，水面同時顯示上方環境的倒影，以及穿過水面後的魚。觀看方向變得貼近水面時，倒影通常更顯著。若光從水中斜向上走，角度足夠大時，還可能完全無法形成向空氣傳播的折射射線。即使成功入水，光在水中走得越遠，某些波段也會衰減得越多。

因此，「畫一片半透明藍色平面」不能替代水面模型。它可能產生可用的示意圖，卻沒有依入射角改變反射、沒有全反射，也沒有把水中行程與吸收分開。以下建立的是一個**清澈、平坦、均勻介質**的合成參考模型；它不包含波浪微表面、散射、懸浮粒子、焦散或魚體表面的完整光傳輸。

數學與幾何推導

介質、入射方向與法線

令 $\mathbf{d}$ 為指向交點的**單位射線方向**， $\mathbf{N}$ 為由第一介質指向第二介質反方向、也就是**朝向入射射線所在一側的單位法線**。本章在每次交界時都調整 $\mathbf{N}$ ，使

$$c_i = \cos \theta_i = -\mathbf{d} \cdot \mathbf{N} \in [0, 1].$$

θ_i 是射線與法線的夾角。令 n_1 為射線目前所在介質的折射率、 n_2 為將進入介質的折射率。對水平水面，從空氣往下的 $\mathbf{d}_y < 0$ 應使用 $\mathbf{N} = (0, 1, 0)$ 、 $n_1 = n_{\text{air}}$ 、 $n_2 = n_{\text{water}}$ ；從水往上的 $\mathbf{d}_y > 0$ 則使用 $\mathbf{N} = (0, -1, 0)$ ，並交換兩個折射率。這裡的 $\mathbf{N}$ 是**交界計算用的朝向法線**，不必等於網格儲存的固定外向法線。

反射方向是入射方向沿法線分量翻轉：

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{N})\mathbf{N}.$$

由於 $\mathbf{d}$ 、 $\mathbf{N}$ 都是單位向量， $\mathbf{r}$ 也是單位向量。反射留在原介質，折射才跨越界面。

Snell 定律與全反射

在平坦界面上，折射方向的切向分量必須符合 Snell 定律：

$$n_1 \sin \theta_i = n_2 \sin \theta_t.$$

設 $\eta = n_1/n_2$ ，把入射方向分解為切向部分 $\mathbf{d} + c_i \mathbf{N}$ 與法向部分 $-c_i \mathbf{N}$ 。折射方向的切向部分因而是 $\eta(\mathbf{d} + c_i \mathbf{N})$ 。若

$$k = 1 - \eta^2(1 - c_i^2) \geq 0,$$

折射角餘弦為 $c_t = \sqrt{k}$ ，折射方向可寫成

$$\mathbf{t} = \eta \mathbf{d} + (\eta c_i - c_t) \mathbf{N}.$$

這個寫法便於程式實作，也提供直接測試： $\mathbf{t}$ 應是單位向量，且指向第二介質，即 $\mathbf{t} \cdot \mathbf{N} \leq 0$ 。

當 $k < 0$ ， $\sin \theta_t$ 會被要求大於 1，沒有向另一介質傳播的折射射線，稱為**全反射**。它只可能在 $n_1 > n_2$ 時發生；臨界角為

$$\theta_c = \arcsin\left(\frac{n_2}{n_1}\right), \quad n_1 > n_2.$$

在理想幾何光學模型中， $\theta_i > \theta_c$ 全反射；恰在臨界角時，折射方向沿著界面。數值計算接近 $k = 0$ 時應容許極小的浮點誤差，卻不能把明顯的負值當成有效折射。

Fresnel：兩條路徑如何分光

方向確定後，還須決定反射與透射的光量。對無吸收、光滑的介電界面，無偏振入射光的 Fresnel **功率反射率**可由兩個偏振分量平均：

$$R_s = \left(\frac{n_1 c_i - n_2 c_t}{n_1 c_i + n_2 c_t}\right)^2, \quad R_p = \left(\frac{n_1 c_t - n_2 c_i}{n_1 c_t + n_2 c_i}\right)^2, \quad F = \frac{R_s + R_p}{2}.$$

此式在有效折射方向存在時使用；全反射時令 $F = 1$ 。這裡的 F 是介面處的反射光通量比例，不是某個顏色通道的藍色係數。在此理想、無界面吸收的設定下，透射光通量比例是 $1 - F$ 。若把它們接進完整路徑追蹤器，方向取樣機率、輻射亮度跨介質的折射率因子及 PDF 必須一致處理；不可僅因兩個比例相加為一，就宣稱任意實作皆能量正確。

正入射時 $c_i = c_t = 1$ ，上式化成

$$F_0 = \left(\frac{n_1 - n_2}{n_1 + n_2} \right)^2.$$

常見的 Schlick 近似 $F_{\text{Schlick}} = F_0 + (1 - F_0)(1 - c_i)^5$ 計算方便，但只是角度曲線的近似，不應拿它在全反射條件下取代 $F = 1$ 。本章程式用上面的無偏振 Fresnel 公式，方便檢查臨界角附近的行為。

Beer-Lambert：跨過界面之後

水中行進時的吸收與界面 Fresnel 是不同事件。對均勻、**只吸收、不散射且不發光**的介質，令每個線性 RGB 通道的吸收係數為 σ_a ，單位 m^{-1} ；水中行程為 ℓ 公尺。各通道透射率為

$$\tau(\ell) = \exp(-\sigma_a \ell), \quad \mathbf{L}_{\text{after}} = \tau(\ell) \odot \mathbf{L}_{\text{before}},$$

其中 $\odot$ 表示逐通道相乘。指數的乘積 $\sigma_a \ell$ 沒有單位。舉例說，兩公尺路程若 $\sigma_a = 0.4 \text{ m}^{-1}$ ，該通道透射率是 $e^{-0.8}$ ，約 0.449；這不是「水面有百分之四十四點九透明」，而是指定**介質路程**造成的衰減。

實際水下影像還有向視線散入的光。只乘上 τ 不會產生朦朧的水色，也不能代表濁水的散射；用 RGB 三係數亦只是合成影像近似，並非完整波長光譜模型。

逐步手算例題

例一：由空氣入水。為使算式易驗算，設定合成折射率 $n_{\text{air}} = 1$ 、 $n_{\text{water}} = 4/3$ 。取入射角 $\theta_i = 30^\circ = \pi/6$ ，則 $\sin \theta_i = 1/2$ 。Snell 定律給出

$$\sin \theta_t = \frac{1}{4/3} \frac{1}{2} = \frac{3}{8} = 0.375, \quad \theta_t = \arcsin(0.375) \approx 22.02^\circ.$$

折射角較小，光線朝法線彎折。 $c_i = \sqrt{3}/2 \approx 0.8660$ 、 $c_t = \sqrt{1 - (3/8)^2} \approx 0.9270$ 。此時 $k = c_t^2 = 55/64 > 0$ ，故不是全反射。正入射的基準反射率另可手算：

$$F_0 = \left(\frac{1 - 4/3}{1 + 4/3} \right)^2 = \left(-\frac{1}{7} \right)^2 = \frac{1}{49} \approx 0.02041.$$

注意 F_0 是**正入射**值；不能未計算便把它宣稱為本題 30° 的精確反射率。

例二：由水出空氣與臨界角。交換折射率後， $n_1 = 4/3$ 、 $n_2 = 1$ 。臨界角為

$$\theta_c = \arcsin(3/4) \approx 48.59^\circ.$$

若水中入射角是 30° ，則 $\sin \theta_t = (4/3)(1/2) = 2/3$ ，空氣中的折射角約 41.81° ，有折射射線。若改為 60° ，則 $\sin \theta_t = (4/3)(\sqrt{3}/2) = 2\sqrt{3}/3 > 1$ ，所以全反射。把這條射線標成「完全不透明水面」並不精確：判斷取決於**入射側是水**及入射角，而非整片水面具有固定透明度。

實作與程式

以下完整程式僅用 Python 3.10+ 標準庫，計算單條射線在理想平面交界的反射、折射、Fresnel 反射率及水中吸收。`interface` 明確接收入射側與透射側的折射率；`face_normal` 檢查傳入的法線是否朝向入射側，避免在呼叫端暗中交換介質。此小程式不追蹤後續物件交點，也不讀寫檔案。

```
import math

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def scale(k, v):
    return tuple(k * x for x in v)

def add(a, b):
    return tuple(x + y for x, y in zip(a, b))

def unit(v):
    length = math.sqrt(dot(v, v))
    if not math.isfinite(length) or length == 0:
        raise ValueError("方向必須是非零有限向量")
    return scale(1 / length, v)

def face_normal(direction, normal):
    d, n = unit(direction), unit(normal)
    if dot(d, n) >= 0:
        raise ValueError("法線須指向入射側，且非切向入射")
    return d, n

def interface(direction, normal, n1, n2):
    if not (math.isfinite(n1) and math.isfinite(n2)
            and n1 > 0 and n2 > 0):
        raise ValueError("折射率必須是正的有限數")
    d, n = face_normal(direction, normal)
    ci = -dot(d, n)
    eta = n1 / n2
    reflected = unit(add(d, scale(2 * ci, n)))
    k = 1 - eta * eta * (1 - ci * ci)

    if k < -1e-14:
        return reflected, None, 1.0 # 全反射
    ct = math.sqrt(max(0.0, k)) # 僅修正邊界浮點誤差
    transmitted = unit(add(scale(eta, d),
                             scale(eta * ci - ct, n)))
    rs = ((n1 * ci - n2 * ct) /
          (n1 * ci + n2 * ct)) ** 2
    rp = ((n1 * ct - n2 * ci) /
          (n1 * ct + n2 * ci)) ** 2
    return reflected, transmitted, (rs + rp) / 2

def beer_lambert(rgb, absorption, distance):
    if len(rgb) != 3 or len(absorption) != 3:
        raise ValueError("需要三個線性 RGB 通道")
    if not math.isfinite(distance) or distance < 0:
        raise ValueError("路程必須是非負有限公尺數")
    if any(not math.isfinite(x) or x < 0 for x in absorption):
```

```

        raise ValueError("吸收係數須為非負有限數")
    if any(not math.isfinite(x) or x < 0 for x in rgb):
        raise ValueError("輸入光量須為非負有限數")
    return tuple(value * math.exp(-sigma * distance)
                 for value, sigma in zip(rgb, absorption))

air, water = 1.0, 4.0 / 3.0
up = (0.0, 1.0, 0.0)
down = (0.0, -1.0, 0.0)

# 由空氣往下：法線朝上；入射角 30 度。
incident = (0.5, -math.sqrt(3) / 2, 0.0)
r, t, f = interface(incident, up, air, water)
assert t is not None
assert math.isclose(-dot(t, up), math.sqrt(55) / 8,
                    abs_tol=1e-12)
assert 0 <= f <= 1
assert math.isclose(dot(r, r), 1.0, abs_tol=1e-12)

# 正入射，以及由水往上、超過臨界角的全反射。
_, straight, f0 = interface(down, up, air, water)
assert straight is not None
assert math.isclose(f0, 1 / 49, abs_tol=1e-12)
from_water = (math.sqrt(3) / 2, 0.5, 0.0) # 與向下法線夾 60 度
_, no_transmission, ftir = interface(
    from_water, down, water, air)
assert no_transmission is None and ftir == 1.0

# 臨界角及其兩側：k 是無量綱量；極小負值按臨界處理。
critical_sin = air / water
critical_cos = math.sqrt(1.0 - critical_sin ** 2)
_, critical_ray, _ = interface(
    (critical_sin, critical_cos, 0.0), down, water, air)
assert critical_ray is not None
_, below_ray, _ = interface(
    (math.sqrt(1.0 - (critical_cos + 1e-5) ** 2),
     critical_cos + 1e-5, 0.0), down, water, air)
assert below_ray is not None
_, above_ray, above_f = interface(
    (math.sqrt(1.0 - (critical_cos - 1e-5) ** 2),
     critical_cos - 1e-5, 0.0), down, water, air)
assert above_ray is None and above_f == 1.0

out = beer_lambert((1.0, 1.0, 1.0),
                  (0.4, 0.2, 0.1), 2.0)
assert math.isclose(out[0], math.exp(-0.8), abs_tol=1e-12)
assert out[0] < out[1] < out[2]

```

unit 保證本例方向長度為一，因此介質行程以射線參數差計算時才可直接視為公尺。其他射線追蹤器若使用未正規化方向，必須把參數區間乘以方向長度，才得到 Beer-Lambert 所需的實際行程。例中沒有設定後續交點偏移量；若擴充成多次反彈的追蹤器，應按場景尺度處理射線自相交，而不是把 $1e-14$ 當成交點的通用 epsilon。

測試與預期結果

讀者執行上述程式時，**預期**各斷言通過且無文字輸出；這不是已執行的報告。測試分別覆蓋折射方向的法向分量、反射方向的單位長度、正入射的 F_0 、出水時的全反射，以及水中走兩公尺的逐通道吸收。

可再自行增添兩種邊界測試。其一，取水中入射角 30° ，應得到有效透射方向，其折射角約 41.81° ；其二，取零公尺水中路程，`beer_lambert(rgb, absorption, 0)` 應等於原輸入光量。它們分別檢查「由水出空氣不一定全反射」及「沒有行程就沒有體吸收」。

除錯與常見陷阱

- **只翻轉法線，未交換折射率：**射線入水和出水的 n_1, n_2 不同。用同一組折射率處理兩側，會錯過出水時的全反射。
- **用法線直接代入公式：**網格面法線可能指向與入射射線相同的一側。先確定射線位於哪種介質，再讓計算法線朝向入射側，使 $-\mathbf{d} \cdot \mathbf{N} \geq 0$ 。
- **平方根得到負數：** $k < 0$ 是全反射判定，並非應以絕對值開根號的普通數值錯誤；只有非常接近零的負值才可能按浮點容差修正。
- **以 $1 - F$ 取代水中吸收：**前者是界面分光，後者是以公尺計算的介質行程效應。光可先透射、再於水中吸收，兩步不可互換成一個固定 α 。
- **在 sRGB 數值上乘指數：**Beer-Lambert 應作用於線性光量。顯示用 sRGB 編碼及資料貼圖不可混作介質光量。
- **以藍色濾鏡聲稱模擬真實濁水：**濁度、散射與向視線補入的光均不在本章模型內；調整三個吸收係數只能建立特定的合成示意效果。

養殖數位分身案例

建立一個可檢查的簡化場景：水面是 $y = 0$ 的平面，水中一尾合成魚位於 $y = -2$ 公尺附近，相機自水面上方看向魚。為展示機制，暫取 $n_{\text{air}} = 1$ 、 $n_{\text{water}} = 4/3$ ，以及合成吸收係數 $(0.4, 0.2, 0.1) \text{ m}^{-1}$ ；這些數字只為測試與示意，不是對池水採樣的測量值。

一條相機射線先與平面求交。交界處依入射角計算反射射線和折射射線；向水下的路徑再與魚求交，兩點之間的**實際水中距離**才是吸收公式的 ℓ 。若某通道進入水中的線性光量是 1，沿水中行進兩公尺，例中的三通道分別乘上 $e^{-0.8}$ 、 $e^{-0.4}$ 、 $e^{-0.2}$ 。斜看同一深度的魚時，水中射線通常走得更遠，因此不能一律用魚的垂直深度代替行程。

這個場景可以產生「反射比例隨角度變動、魚的水中路径隨距離衰減」的合成對照資料；它沒有波浪、懸浮物散射或真實水質校準。即使畫面看起來像養殖池，也不能據此推斷真實池水能見度或魚的狀態。

習題

1. **手算**：使用 $n_{\text{air}} = 1$ 、 $n_{\text{water}} = 4/3$ 。空氣中一條射線以 30° 入水，求 $\sin \theta_t$ 、折射角約值及正入射基準 F_0 。再判斷水中以 60° 向上入射是否會全反射。
2. **程式測試**：在本程式後增加兩個斷言：水中以 30° 向上入射能夠折射；對線性 RGB (0.8, 0.6, 0.4)，在任意合法吸收係數下走零公尺，輸出維持不變。給出程式片段。
3. **反例／除錯**：某實作把從水往上走的射線仍以 `interface(d, up, air, water)` 計算，並在出現負的平方根輸入時改用 `sqrt(abs(k))`。指出至少兩個獨立錯誤，說明水中 60° 入射時應採取的行為。
4. **整合應用**：一條已折射進水的射線，方向為單位向量，從水面交點走到合成魚表面交點共 3 公尺。水中吸收係數為 $(0, \ln 2/3, \ln 4/3) \text{ m}^{-1}$ ，入水後的線性 RGB 光量為 (0.6, 0.6, 0.6)。求到達魚表面前的三通道光量。若魚的垂直深度只有 1.5 公尺，為何不能用 1.5 取代 3？

習題解答

1. $\sin \theta_t = (1/(4/3)) \sin 30^\circ = 3/8$ ，故 $\theta_t \approx 22.02^\circ$ ； $F_0 = ((1 - 4/3)/(1 + 4/3))^2 = 1/49 \approx 0.02041$ 。出水臨界角 $\arcsin(3/4) \approx 48.59^\circ$ ， 60° 超過臨界角，因此全反射，沒有向空氣傳播的折射射線。
2. 可加入以下片段；由水向上的入射方向與朝下法線夾 30° ：

```
upward_30 = (0.5, math.sqrt(3) / 2, 0.0)
_, transmitted_30, fresnel_30 = interface(
    upward_30, down, water, air)
assert transmitted_30 is not None
assert 0 <= fresnel_30 < 1

original = (0.8, 0.6, 0.4)
assert beer_lambert(
    original, (0.3, 0.2, 0.1), 0.0) == original
```

零距離時每個指數因子都是 $e^0 = 1$ ；**預期**這些斷言通過。

3. 第一，射線在水中，應設 $n_1 = n_{\text{water}}$ 、 $n_2 = n_{\text{air}}$ ；第二，計算法線應朝水中，即 `down`，不能直接使用 `up`。第三，明顯負的 k 代表不存在傳播中的折射方向，`sqrt(abs(k))` 會捏造一條射線。正確呼叫是 `interface(d, down, water, air)`；水中 60° 時應回傳反射方向、無折射方向，且 $F = 1$ 。
4. 三公尺行程的透射因子依次是 $e^0 = 1$ 、 $e^{-\ln 2} = 1/2$ 、 $e^{-\ln 4} = 1/4$ ，因此到達前的光量為 (0.6, 0.3, 0.15)。Beer-Lambert 使用光線沿路徑行進的距離；斜射線可在下降 1.5 公尺時走過 3 公尺，垂直深度不等於行程。

本章小結

水面交界先由介質順序、入射方向與朝向法線決定幾何：Snell 定律給折射方向，負的 k 判定全反射。Fresnel 給理想界面的反射光量比例；光成功入水後，Beer-Lambert 才按水中**實際路程**計算吸收。將這三步分開實作與測試，才能清楚指出合成水面呈現了哪些機制，以及哪些水下光學現象仍未建模。

參考來源

- [G2] *PBRT 4 : Reflection Models* , https://pbr-book.org/4ed/Reflection_Models
- [G3] *PBRT 4 : The Light Transport Equation* , https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- [G4] *Ray Tracing in One Weekend* , <https://raytracing.github.io/books/RayTracingInOneWeekend.html>

以上供延伸閱讀理想介電界面與光傳輸概念；本章的合成折射率、吸收係數及測試設定均已在文中明示，不作真實池水量測或完整水下光學模型的聲稱。

第24章 渲染誤差、降噪與效能驗證

學習目標與先備知識

本章建立量化評估渲染品質與效能的嚴格框架。讀者需具備第21章蒙特卡洛積分、第22章路徑追蹤基礎及第13章線性色彩處理。重點在於：

1. 定義「正確性」：以解析解或高樣本參考圖為基準。
2. 定義「誤差」：區分偏誤（Bias）、變異數（Variance）與均方誤差（MSE），並理解單張影像與統計期望的差異。
3. 定義「效能」：以時間為單位，區分單次渲染時間與統計穩定性，並正確計算每秒樣本數。

本節使用線性 RGB 空間進行物理誤差計算。最終輸出前的 sRGB 轉換屬於顯示編碼，物理數值誤差應在線性 RGB 計算；顯示域指標可能因非線性映射而得到不同排序。所有測試皆基於合成場景，確保可重現性。

問題與直覺

在 CPU 路徑追蹤中，我們無法獲得「真實」物理光場。我們能獲得的是蒙特卡洛估計 $\hat{I}$ 。其品質由兩個維度決定：

- **準確度**：估計器的期望值 $\mathbb{E}[\hat{I}]$ 是否接近真值 I_{true} ？
- **精確度**：多次獨立運行結果的一致性（噪聲大小，即變異數）。

直覺上，增加每像素樣本數（Samples per Pixel, spp）會降低噪聲。但效能與品質存在權衡。我們需要一套指標來客觀判斷：

1. **MSE**：衡量整體平方差異，包含偏誤與變異數貢獻。
2. **偏誤與變異數分解**：需多組獨立 seed 才能估計偏誤與變異數。
3. **噪聲指標**：檢測隨機波動。
4. **時間記錄**：評估計算成本。

關鍵原則：本書報告的實測結論基於實際代碼執行結果；理論推導結論基於數學模型。未執行的程式不得假設其輸出或效能。

數學與幾何推導

設參考圖為 R ，測試圖為 T 。令影像尺寸為 $H \times W$ ，通道數為 C （RGB 為 3）。總純量通道值數 $N = HWC$ 。

1. **均方誤差 (MSE)**：

$$\text{MSE}(T, R) = \frac{1}{HWC} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} \sum_{c=0}^{C-1} (T_{yxc} - R_{yxc})^2$$

MSE 是單張影像相對參考圖的經驗誤差。

2. 平均絕對偏差 (MAE)：

$$\text{MAE}(T, R) = \frac{1}{HWC} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} \sum_{c=0}^{C-1} |T_{yxc} - R_{yxc}|$$

全域平均 MAE 可能稀釋局部錯誤，檢測黑斑或漏光需搭配最大絕對誤差或區域遮罩分析。

3. 偏誤—變異數分解：此分解是對隨機估計器重複實驗取期望後成立：

$$\mathbb{E}[(\hat{I} - I)^2] = (\mathbb{E}[\hat{I}] - I)^2 + \text{Var}(\hat{I})$$

單張影像不能直接唯一分解。偏誤估計需 K 個獨立 seed 的渲染結果 $\hat{I}_k$ ：

$$\bar{I} = \frac{1}{K} \sum_{k=1}^K \hat{I}_k, \quad \widehat{\text{Bias}} = \bar{I} - R$$

變異數可從跨 seed 波動估計。有限 K 下仍有估計誤差。

有限 K 的精確經驗分解。 直接把「跨 seed 平均影像相對參考圖的平方誤差」加上「跨 seed 樣本變異數」當作總 MSE 是錯的。跨 seed 平均影像本身仍有方差 $\text{Var}(\hat{I})/K$ ，若兩者直接相加，會把這一部分方差重複計入。正確做法是利用樣本變異數的精確恆等式。對每個像素（及通道）位置，令 $\hat{I}_1, \dots, \hat{I}_K$ 為 K 個獨立同分佈樣本， $\bar{I}$ 為樣本平均， $S^2 = \frac{1}{K-1} \sum (\hat{I}_k - \bar{I})^2$ 為樣本變異數。則對任意固定常數 R 有

$$\frac{1}{K} \sum_{k=1}^K (\hat{I}_k - R)^2 = (\bar{I} - R)^2 + \frac{K-1}{K} S^2.$$

左側就是**經驗 MSE**（單張測試影像相對參考圖的 MSE 的跨 seed 平均）；右側第一項的期望是 $\text{Bias}^2 + \text{Var}(\hat{I})/K$ ，第二項則精確對應到 $(K-1)/K$ 倍的真實方差之無偏估計。此恆等式對任何有限 K 都精確成立，不需要任何漸近假設。程式實作應以左側作為 empirically observed 的 MSE，再把右側兩項一起報告；只要輸出同時列出三者，就不會誤導讀者以為右側任一項就是真實偏誤或真實變異數。

若要進一步估計**真實偏誤平方**，可用 $\widehat{\text{Bias}}^2 = \|\bar{I} - R\|^2 - S^2/K$ 。這是無偏估計，但因為帶有抽樣噪聲，有限 K 下可能出現負值。負值不代表「負偏誤」，只能說明估計噪聲大於估計到的偏誤。 K 越大估計越穩定，但成本成比例增加；實驗開始前就應決定 K （例如 30 至 100），事後不可為了挑選「漂亮」的數值而重選 K 或 seed。

4. 收斂速率：若噪聲主導且無偏， $MSE \propto 1/n$ 。若參考圖 R_{ref} 本身含噪聲方差 σ_{ref}^2 ，測試圖 T 方差 σ_{test}^2 ，且兩者誤差獨立，則 $\mathbb{E}[\text{MSE}(T, R_{ref})] = \sigma_{test}^2 + \sigma_{ref}^2$ 。若使用共同隨機數，差值的方差須加

入 $-2 \text{Cov}(T, R_{ref})$ ，因此單純相加 $\sigma_{test}^2 + \sigma_{ref}^2$ 不再成立。使用解析真值可完全避免參考圖噪聲底限，這也是本章所有統計檢驗優先採用解析解的原因。

配對 seed 的作用。若 T 與 R_{ref} 使用同一組隨機 seed（或共同隨機數），兩者的誤差會相關，差值 $T - R_{ref}$ 的變異數變成 $\sigma_{test}^2 + \sigma_{ref}^2 - 2 \text{Cov}$ 。當兩者高度正相關時，這個差值方差可以遠小於獨立情形，讓「相對誤差」檢驗更敏感；但若兩者負相關（例如某處多算、另一處少算），差值方差反而變大。配對 seed 是實驗設計選擇，報告時必須明示是否配對；不配對則採用獨立情形的底限公式。兩種設計都可使用，但不能把兩者混為一談。

MSE 對極端值的敏感性。MSE 對極端 firefly（單一起亮像素）特別敏感，因為誤差是平方的。一個只在少數像素上超亮的亮點，就可以讓 MSE 遠大於視覺觀感所暗示的量。MAE 對這種局部錯誤相對穩健，但也因此對單一嚴重錯誤不夠敏感。診斷 firefly 需要同時看 MSE、MAE 與最大絕對誤差，或使用穩健型指標（例如修剪均值）。本章示範的都是合成場景，不含真路徑追蹤的 firefly，但相同原則依然適用。

5. 效能指標：

$$\text{Samples/sec} = \frac{WH \cdot n}{\text{Time}}$$

注意： n 是每像素樣本數，不乘以通道數 C 。計算速率的前提是：分子所宣稱處理的樣本數，確實對應到計時區塊內真正執行的工作量。若計時函式只生成固定大小的陣列、或只執行部分工作，任何推導出的速率都是無意義的。

逐步手算例題

案例1：偏誤與噪聲分離（統計期望） 假設真值 $I = 1.0$ 。

- **實驗 A：**估計器 $\hat{I}_A = 1.1$ （常數）。 $\mathbb{E}[\hat{I}_A] = 1.1$ 。偏誤 = 0.1。變異數 = 0。 $\mathbb{E}[(\hat{I}_A - 1)^2] = 0.01$ 。
- **實驗 B：**估計器 $\hat{I}_B$ 為單次均勻隨機變數 $U[0.5, 1.5]$ 。 $\mathbb{E}[\hat{I}_B] = 1.0$ 。偏誤 = 0。變異數 $\text{Var}(U[a, b]) = \frac{(b-a)^2}{12} = \frac{1}{12} \approx 0.0833$ 。 $\mathbb{E}[(\hat{I}_B - 1)^2] = 0.0833$ 。

結論：實驗 B 無偏但方差較高；實驗 A 有偏但方差為零。A 的平方風險（0.01）低於 B（0.0833）。這說明無偏性不保證整體品質較佳；降噪器常以引入偏誤換取方差降低，需視應用需求權衡。

案例2：收斂速率與參考圖影響 假設測試估計器無偏，單樣本方差 $\sigma^2 = 0.04$ 。

- $n = 16$ 時， $\text{MSE}_{16} \approx \frac{0.04}{16} = 0.0025$ 。
- $n = 64$ 時， $\text{MSE}_{64} \approx \frac{0.04}{64} = 0.000625$ 。若使用含噪聲參考圖，設其方差為 $\sigma_{ref}^2 = 0.001$ ，且測試圖與參考圖的誤差獨立，則 $\mathbb{E}[\text{MSE}(T, R_{ref})] = \sigma_{test}^2/n + \sigma_{ref}^2$ 。代入 $n = 16$ 得 $\mathbb{E}[\text{MSE}_{16}] \approx 0.0025 + 0.001 = 0.0035$ ； $n = 64$ 得 $\mathbb{E}[\text{MSE}_{64}] \approx 0.000625 + 0.001 = 0.001625$ 。只有當 $n \rightarrow \infty$ （測試圖方差趨近零）時，期望 MSE 才趨近 0.001；有限 n 下整個值是兩者之和，不宜整體稱為「底限」，只有其漸近值才是參考圖方差。

單點觀測值偏低或偏高可能由隨機波動、有限 seed、或取樣策略造成，不能直接推論存在系統性錯誤。應以多 seed 平均值與樣本標準差判斷。若使用共同隨機數或配對 seed，會出現協方差項，上述獨立情形的底限公式不再成立。

實作與程式

Python 3.10+，使用 NumPy。此程式驗證 MSE 計算邏輯、偏誤—變異數估計，並示範簡單 Box 模糊濾波器對合成圖的影響。註：此處使用解析真值與模擬噪聲，不執行完整路徑追蹤。`render_simulated` 只生成固定大小的高斯噪聲陣列，運算量幾乎不隨 *spp* 改變，因此計時結果不能驗證真實渲染時間與 *spp* 的關係。

```
import numpy as np
import time

def calculate_metrics(test_img, ref_img):
    """計算 MSE、MAE 與最大絕對誤差；要求非空、同形且有限。"""
    test_img = np.asarray(test_img, dtype=np.float64)
    ref_img = np.asarray(ref_img, dtype=np.float64)
    if (test_img.ndim != 3 or ref_img.ndim != 3
        or test_img.shape[2] != 3 or ref_img.shape[2] != 3):
        raise ValueError("images must have shape (H, W, 3)")
    if test_img.shape != ref_img.shape:
        raise ValueError("Shapes must match")
    if test_img.shape[0] == 0 or test_img.shape[1] == 0:
        raise ValueError("image dimensions must be non-zero")
    if not (np.all(np.isfinite(test_img)) and np.all(np.isfinite(ref_img))):
        raise ValueError("Images must contain finite values")
    diff = test_img - ref_img
    mse = np.mean(diff ** 2)
    mae = np.mean(np.abs(diff))
    max_err = np.max(np.abs(diff))
    return mse, mae, max_err

def render_simulated(spp, seed, bias=0.0, sigma=1.0):
    """
    模擬渲染：真值 0.5 + 偏誤 + 高斯噪聲。
    噪聲標準差 sigma / sqrt(spp)。
    注意：此為統計玩具模型，可能產生負值，不代表物理輻射。
    """
    if not isinstance(spp, (int, np.integer)) or spp <= 0:
        raise ValueError("spp must be a positive integer")
    if not np.isfinite(sigma) or sigma < 0:
        raise ValueError("sigma must be non-negative finite")
    if not np.isfinite(bias):
        raise ValueError("bias must be finite")

    H, W, C = 16, 16, 3
    rng = np.random.default_rng(seed)
    noise = rng.normal(0.0, sigma / np.sqrt(spp), size=(H, W, C))
    img = np.full((H, W, C), 0.5, dtype=np.float64)
    img += noise + bias
    return img
```

```

def box_blur(img, radius=1):
    """
    簡單 Box 模糊濾波器（教學用）。
    使用 edge padding 避免週期邊界問題。只支援 (H, W, 3) 的浮點影像。
    """
    img = np.asarray(img, dtype=np.float64)
    if img.ndim != 3 or img.shape[2] != 3:
        raise ValueError("img must have shape (H, W, 3)")
    if img.shape[0] == 0 or img.shape[1] == 0:
        raise ValueError("image dimensions must be non-zero")
    if not np.all(np.isfinite(img)):
        raise ValueError("img must contain finite values")
    if not isinstance(radius, (int, np.integer)) or radius < 0:
        raise ValueError("radius must be a non-negative integer")
    if radius == 0:
        return img.copy()

    padded = np.pad(img, ((radius, radius), (radius, radius), (0, 0)), mode="edge")
    blurred = np.zeros_like(img, dtype=np.float64)
    H, W, C = img.shape
    for dy in range(-radius, radius + 1):
        for dx in range(-radius, radius + 1):
            row_start = dy + radius
            row_end = row_start + H
            col_start = dx + radius
            col_end = col_start + W
            blurred += padded[row_start:row_end, col_start:col_end, :]
    blurred /= (2 * radius + 1) ** 2
    return blurred

def run_experiment():
    print("=== 1. 收斂速率與偏誤估計 ===")
    ref_true = np.full((16, 16, 3), 0.5, dtype=np.float64)
    K = 10 # 使用較多 seed 以提高估計穩定性

    for bias_val in [0.0, 0.1]:
        for spp in [1, 16, 64]:
            images = []
            times = []
            for seed in range(100, 100 + K):
                start = time.perf_counter()
                test_img = render_simulated(spp=spp, seed=seed, bias=bias_val)
                elapsed = time.perf_counter() - start
                images.append(test_img)
                times.append(elapsed)

            stack = np.stack(images, axis=0)
            mean_img = np.mean(stack, axis=0)

            # 有限 K 的精確經驗分解（對任何有限 K 都精確成立）：
            # 
$$(1/K) \sum ||\hat{I}_k - R||^2 = ||\text{mean}(\hat{I}) - R||^2 + ((K-1)/K) S^2$$

            empirical_mse = np.mean((stack - ref_true[None, ...]) ** 2)
            mean_error2 = np.mean((mean_img - ref_true) ** 2)
            sample_variance = np.mean(np.var(stack, axis=0, ddof=1))
            decomposed_mse = mean_error2 + ((K - 1) / K) * sample_variance
            assert np.allclose(empirical_mse, decomposed_mse)
            # 真實偏誤平方的無偏估計；有限樣本下可能出現負值，
            # 這是抽樣噪聲，不可強行解讀為「負偏誤平方」。
            estimated_bias2 = mean_error2 - sample_variance / K

```

```

    avg_time = np.mean(times)

    # 注意：render_simulated 只生成固定大小的 NumPy 亂數陣列，
    # 並不真的計算 WH*spp 個樣本。這裡刻意不輸出 Samples/s，
    # 以避免誤導為路徑追蹤效能。
    print(f"Bias={bias_val:.1f}, spp={spp:3d} | "
          f"empirical_MSE={empirical_mse:.6f} "
          f"(decomposed={decomposed_mse:.6f}) | "
          f"mean_image_err²={mean_error2:.6f} "
          f"(bias² estimate={estimated_bias2:.6f}) | "
          f"Var={sample_variance:.6f} | toy_call_time={avg_time:.6f}s")

print("\n=== 2. 降噪濾波器測試 ===")
mse_diffs = []
for seed in range(100, 110):
    noisy_img = render_simulated(spp=4, seed=seed, bias=0.0)
    blurred_img = box_blur(noisy_img, radius=1)

    mse_noisy, _, _ = calculate_metrics(noisy_img, ref_true)
    mse_blur, _, _ = calculate_metrics(blurred_img, ref_true)

    mse_diffs.append(mse_blur - mse_noisy)

avg_diff = np.mean(mse_diffs)
std_diff = np.std(mse_diffs, ddof=1)
print(f"[Constant scene] Blur - Noisy MSE: Avg={avg_diff:.6f}, Std={std_diff:.6f}")
if avg_diff < 0:
    print("[Constant scene] On average, blur reduces MSE.")
else:
    print("[Constant scene] On average, blur did not reduce MSE.")

print("\n=== 3. 邊緣與結構測試 ===")
# 3a. 無噪聲階梯：隔離 Box filter 的確定性邊緣效應。
H, W, C = 16, 16, 3
step_ref = np.zeros((H, W, C), dtype=np.float64)
step_ref[:, W // 2:, :] = 1.0
step_blur_clean = box_blur(step_ref, radius=1)
rng_edge = np.random.default_rng(20250101)

left_col = W // 2 - 1
right_col = W // 2
left_value = float(step_blur_clean[H // 2, left_col, 0])
right_value = float(step_blur_clean[H // 2, right_col, 0])
left_bias = left_value - float(step_ref[H // 2, left_col, 0])
right_bias = right_value - float(step_ref[H // 2, right_col, 0])
print(f"[Clean step] left={left_value:.4f} right={right_value:.4f} "
      f"left_bias={left_bias:.4f} right_bias={right_bias:.4f}")
# 中央橫列的局部剖面直接顯示原本的跳變如何被展寬；
# 不以含噪全圖的平均梯度推斷真實邊緣寬度。
profile = step_blur_clean[H // 2, left_col - 1:right_col + 2, 0]
print(f"[Clean step] local_profile={profile}")

# 3b. 單一亮像素：能量擴散（質量守恆檢查）。
spike_ref = np.zeros((H, W, C), dtype=np.float64)
spike_ref[H // 2, W // 2, :] = 1.0
spike_blur = box_blur(spike_ref, radius=1)
center = float(spike_blur[H // 2, W // 2, 0])
spread = float(np.sum(spike_blur[:, :, 0]) - center)

```



```

print(f"[Spike] center={center:.4f} spread_to_neighbors={spread:.4f} "
      f"(center + spread = {center + spread:.4f})")

# 3c. 純噪聲：確認模糊確實降低噪聲方差。
noise_ref = np.full((H, W, C), 0.5, dtype=np.float64)
noise_img = noise_ref + rng_edge.normal(0.0, 0.5, size=noise_ref.shape)
noise_blur = box_blur(noise_img, radius=1)
var_before = float(np.var(noise_img))
var_after = float(np.var(noise_blur))
print(f"[Pure noise] var(noisy)={var_before:.4f} var(blurred)={var_after:.4f}")

# 3d. 細棋盤格：高頻結構是否被移除。
checker = np.zeros((H, W, C), dtype=np.float64)
for yy in range(H):
    for xx in range(W):
        if (xx + yy) % 2 == 0:
            checker[yy, xx, :] = 1.0
checker_blur = box_blur(checker, radius=1)
amp_before = float(np.max(checker) - np.min(checker))
amp_after = float(np.max(checker_blur) - np.min(checker_blur))
print(f"[Checker] amp_before={amp_before:.4f} amp_after={amp_after:.4f} "
      f"(高頻衰减比例≈{amp_after / amp_before:.4f})")

print("\n註：Box filter 通常不會幻覺出新的高頻或語意細節，"
      "但會產生中間色帶、邊緣拓寬與滲色等濾波偽影。"
      "本程式不測試資料驅動的細節重建。")

if __name__ == "__main__":
    run_experiment()

```

測試與預期結果

1. 收斂速率與偏誤：

- 無偏時 (Bias=0.0)， spp 從 1 到 64，`empirical_MSE` 應約減小 64 倍。
- `mean_image_err2` 是跨 K 個 seed 平均影像對真值的平方誤差，期望為 $\sigma^2 / (spp \cdot K)$ ，因此不會接近零。以程式的預設 $\sigma = 1$ 、 $spp = 1$ 、 $K = 10$ 為例，期望約為 0.1； $spp = 64$ 時期望約為 $1/640 \approx 0.0015625$ 。
- `bias2 estimate = mean_error2 - sample_variance/K` 在無偏情形下期望為 0，但有限樣本下可能出現負值，這是抽樣噪聲，不是物理負偏誤平方。
- 有偏時 (Bias=0.1)，真實 $\text{Bias}^2 = 0.01$ 。理論期望的 `empirical_MSE` 為 $0.01 + 1/spp$ ，`mean_image_err2` 為 $0.01 + 1/(spp \cdot K)$ ；兩者在 $spp \rightarrow \infty$ 時都趨近 0.01。有限 seed 的經驗值可能暫時低於 0.01，並不矛盾；0.01 是期望 MSE 的偏誤底限與漸近值，而非每次觀測的下界。
- `toy_call_time`：`render_simulated` 只生成固定大小的陣列並縮放噪聲標準差，工作量不隨 spp 成比例增加。列印的十次單獨呼叫平均值只是玩具程式的計時示意；呼叫過短，差異可能由計時及排程雜訊主導，不得據此判定效能。程式刻意**不輸出** `Samples/s`，因為模擬器並未真正執行 $WH \cdot spp$ 個樣本。

真正計時實驗的欄位。 報告路徑追蹤效能時應明確列出：解析度、每像素樣本數、最大路徑深度、seed 集合、是否使用共同隨機數、執行環境（Python、NumPy、CPU 與執行緒設定）、暖機政策、檔案 I/O 是否計入。並對每個 seed 個別記錄時間，再報告平均值與樣本標準差。未實跑時全部標為待辦，不得輸出虛構 FPS。

2. 降噪測試（常數場景）：

- 在常數真值場景中，Box blur 應在期望上降低噪聲方差。
- **預期：** [Constant scene] Blur - Noisy MSE 的平均值應為負值。單一 seed 可能因隨機波動不符合此趨勢。
- **注意：** 此結果僅適用於無邊緣的常數區域。

3. 邊緣與結構測試：

- **無噪聲階梯邊緣：** 左半為 0、右半為 1。半徑 1 的 3×3 Box blur 使邊界相鄰左欄成為 $1/3$ 、右欄成為 $2/3$ ；相對真值的偏誤分別為 $+1/3$ 、 $-1/3$ 。中央橫列的局部剖面顯示邊緣展寬，不以含噪全圖的平均梯度作此判斷。
- **單一亮像素：** 中心像素的能量會被分攤到鄰居；中心值 + 鄰居值之和應仍接近原本的 1.0（質量守恒）。
- **純噪聲：** 模糊後 `var(blurred)` 應明顯小於 `var(noisy)`，因為 Box 平均降低了獨立噪聲的方差。
- **細棋盤格：** 經過 blur 後振幅應大幅下降，說明高頻結構被移除。
- **注意：** Box filter 通常不會幻覺出新的高頻或語意細節，但可能產生參考圖沒有的中間色帶、邊緣拓寬及滲色等偽影。因此本測試不能用來展示生成式假細節；資料驅動的細節重建也不在本程式範圍內。

除錯與常見陷阱

1. **參考圖偏差：** 若參考圖本身樣本數不足，其誤差會計入測試圖誤差。解決方案：使用解析解或極高樣本數參考圖，並說明誤差底限。
2. **色彩空間混淆：** 物理誤差必須在線性 RGB 計算。sRGB 空間的 MSE 不反映物理光量差異。
3. **降噪偽影：**
 - **過度平滑：** 真實高頻細節被移除。
 - **邊緣滲色/光暈：** 跨物體邊界混合顏色。
 - **假細節與濾波偽影：** 資料驅動或非線性重建可能幻覺出參考圖不存在的高頻或語意細節；簡單 Box filter 通常不會如此，但仍會新增中間色帶並造成邊緣拓寬與滲色。
4. **時間計量：** 使用 `time.perf_counter()` 計時核心計算區塊，並明確排除 I/O 與資料轉換。對於 16×16 這類極短的計算，單次 `perf_counter` 容易受排程、配置與計時解析度影響。若保留微型基準測試，應重複多次並以批次總時間除以次數，第一批視需要列為暖機（warm-up）。不可把十次單獨呼叫的平均直接當成可靠的效能結論。更完整的計時協議見上節「真正計時實驗的欄位」。
5. **隨機種子：** 使用 `np.random.default_rng(seed)` 避免修改全域狀態。跨 seed 比較時，必須固定所有其他參數。

6. **局部錯誤檢測**：全域 MSE/MAE 可能稀釋小面積嚴重錯誤。應檢查最大絕對誤差或區域統計。
7. **邊界處理**：使用 `np.roll` 會造成週期邊界，不相鄰像素互相污染。應使用 `np.pad` 配合 `mode="edge"` 或其他合理邊界策略。

養殖數位分身案例

在養殖場合成場景中，驗證水體消光模型的實現。

1. **基準測試**：設定均勻消光係數 σ ，無散射、無水面介面、直線路徑。理論透射強度 $I = I_0 e^{-\sigma d}$ 。
2. **誤差分析**：比較渲染結果與解析解的 MSE。若 MSE 隨水深增加呈現非預期模式，可能暗示消光係數 σ 錯誤或路徑長度計算錯誤。
3. **降噪驗證**：比較不同 spp 的 MSE。若 spp 加倍後 MSE 未按預期下降，可能存在系統性偏誤（如幾何漏光）。
4. **效能記錄**：記錄不同 σ 與 spp 下的渲染時間、總路徑數、總射線數。高消光係數可能縮短平均路徑長度，降低每樣本計算成本，但啟動樣本數 $WH \cdot spp$ 不變。需記錄這些統計量以分析每樣本成本。

習題

1. **手算**：給定參考像素 $R = [1.0, 2.0]$ 和測試像素 $T = [1.1, 1.8]$ ，計算 MSE 和 MAE。
2. **程式測試**：`render_simulated` 已經支援 `bias` 參數，請直接使用 `bias=0.05`，設計跨 spp（例如 1、4、16、64）與跨 seed（例如 30 個獨立 seed，seed 從 0 到 29）的實驗。利用有限 K 修正分解，觀察 `mean_image_err2` 與 `empirical_MSE` 在 spp 增大時的極限值。說明此極限與偏誤、變異數各有什麼關係，並解釋為何 `bias2 estimate` 在有限樣本下可能出現負值。
3. **反例/除錯**：假設路徑追蹤器在處理金屬表面時出現黑色斑塊。設計一個測試案例，通過跨 spp（至少 4、16、64、256 四檔）與跨 seed（至少 30 個獨立 seed）的 MSE 分析，判斷這是偏誤（能量損失）還是噪聲（隨機取樣失敗）。注意：即使平均誤差確實不隨 spp 下降，也僅是偏誤證據增強，不能單獨證明根因；還須檢查 NaN、幾何法線、PDF、BRDF/PDF 比值與路徑終止條件。
4. **整合應用**：寫一個函式 `compare_renders`，輸入參考圖、兩張測試圖及其渲染時間和 spp，輸出比較報告（包含 MSE、MAE、時間、Samples/sec）。用於比較兩個不同配置的效能與品質。

習題解答

1. $diff = [0.1, -0.2]$ $diff^2 = [0.01, 0.04]$ $MSE = (0.01 + 0.04)/2 = 0.025$ $MAE = (|0.1| + |-0.2|)/2 = 0.15$
2. 設程式中的 $K = 30$ 、 $\sigma = 1$ ，偏誤 $b = 0.05$ ，每張影像各通道的噪聲方差為 $1/spp$ 。跨 seed 平均後，噪聲方差為 $1/(spp K)$ ，因此 $\mathbb{E}[\text{mean_image_err}^2] = 0.0025 + 1/(spp K)$ ；逐張計算再平均則有 $\mathbb{E}[\text{empirical_MSE}] = 0.0025 + 1/spp$ 。兩者在 $spp \rightarrow \infty$ 時都趨近 0.0025，這是期望 MSE 的偏誤底限，不是有限 seed 觀測值的硬性下界。有限 K 時，經驗分解仍精確成立：`empirical_MSE =`

$\text{mean_image_err}^2 + (K-1)/K \times \text{sample_variance}$ 。 $\text{bias}^2 \text{ estimate} = \text{mean_image_err}^2 - \text{sample_variance}/K$ 的期望為 0.0025；它是兩個隨機估計量之差，有限樣本下可能為負，並不表示偏誤平方真的為負。增加樣本數只會降低這個模型的噪聲項，不會消除 0.05 的系統性偏誤。

3.
 - **設計**：渲染一個具有已知導體 Fresnel 參數的金屬球，背景黑色。
 - **測試步驟**：
 1. 建立高可信參考圖或可解析案例。
 2. 對每個 spp 使用多個 seed。
 3. 計算黑斑遮罩區域的跨 seed 平均與變異數。
 4. 檢查平均誤差是否隨 spp 仍穩定非零。
 5. 分別檢查 NaN、法線、PDF、BRDF/PDF 比值及路徑終止。
 - **判斷**：
 - 若多 seed 平均後黑斑仍存在且平均誤差不隨 spp 下降，傾向為**偏誤**。
 - 若多 seed 平均後黑斑消失或誤差按 $1/\text{spp}$ 下降，傾向為**噪聲**。
 - MSE 趨勢只能提供證據，不能單獨證明根因。

4.

```
def compare_renders(ref, img1, time1, spp1, img2, time2, spp2):
    """
    比較兩個渲染結果。
    ref: 參考圖
    img1, img2: 測試圖
    time1, time2: 渲染時間 (秒)
    spp1, spp2: 每像素樣本數
    """

    # 檢查輸入 (含維度與通道數)
    if (ref.ndim != 3 or ref.shape[2] != 3
        or img1.ndim != 3 or img1.shape[2] != 3
        or img2.ndim != 3 or img2.shape[2] != 3):
        raise ValueError("images must have shape (H, W, 3)")
    if img1.shape != ref.shape or img2.shape != ref.shape:
        raise ValueError("Shapes must match")
    if not (np.all(np.isfinite(img1)) and np.all(np.isfinite(img2)) and np.all(np.isfinite(ref))):
        raise ValueError("Images must contain finite values")
    if not (np.isfinite(time1) and time1 > 0) or not (np.isfinite(time2) and time2 > 0):
        raise ValueError("Times must be positive finite")
    if not isinstance(spp1, (int, np.integer)) or spp1 <= 0 or not isinstance(spp2, (int, np.integer))
    or spp2 <= 0:
        raise ValueError("spp must be positive integers")

    mse1, mae1, max1 = calculate_metrics(img1, ref)
    mse2, mae2, max2 = calculate_metrics(img2, ref)

    H, W, C = img1.shape
    rate1 = (H * W * spp1) / time1
    rate2 = (H * W * spp2) / time2

    report = {
        "Config1": {"MSE": mse1, "MAE": mae1, "MaxErr": max1, "Time": time1, "Samples/s": rate1},
        "Config2": {"MSE": mse2, "MAE": mae2, "MaxErr": max2, "Time": time2, "Samples/s": rate2}
    }
    return report
```

注意：若無參考圖，可計算兩圖之間的差異以評估一致性，但無法評估絕對準確度。

本章小結

本章強調了渲染驗證的科學方法。通過 MSE、MAE 和效能指標，我們可以客觀評估算法的準確度與效率。關鍵在於區分偏誤與噪聲：有限 K 下要使用 $\|\bar{I} - R\|^2 + \frac{K-1}{K} S^2$ 的精確分解，而不是把 `mean_error2` 與 `sample_variance` 直接相加；估計真實偏誤平方時要接受可能出現的負值。使用含噪參考圖時，需明示測試圖與參考圖的方差關係；若使用配對 seed，須計入協方差項。效能方面，只有真實在渲染核心上計時、且報告中完整列出解析度、spp、seed、環境與暖機政策，才構成可稽核的效能結論。

在養殖場數位分身中，這些方法提供**可稽核的數值證據**，但不能取代模型假設、單位、材質、介質及實作正確性的獨立驗證。MSE、MAE 與計時指標只是驗證流程的一部分；任何關於物理可信度的聲明，都必須建立在合理的模型假設、正確的單位，以及獨立可重現的實測之上。簡單降噪濾波器可作為教學工具，但需理解它在邊緣處理上的局限：Box filter 通常不會幻覺出高頻或語意細節，但會產生中間色帶、邊緣拓寬與滲色；資料驅動或非線性重建則可能另外生成假細節。

未執行的程式一律標為預期或待辦，不得虛構驗證結果、效能數字或物理準確度。

參考來源

- G2: PBRT 4 Reflection Models — 提供 BRDF 與取樣機率密度的背景；不直接支持本章的降噪、MSE 或計時方法。
- G3: PBRT 4 The Light Transport Equation — 提供光傳輸方程與蒙特卡洛估計的一般背景；本章的有限樣本偏誤—變異數分解為本章自行推導，未逐條依賴此來源。
- G4: Ray Tracing in One Weekend — 提供入門光線追蹤背景；不能支持本章的統計公式或效能協議。

以上來源僅為延伸閱讀入口，不等於本章所有數值常數、統計公式或實驗協議已由其逐條驗證。本章自行推導的部分會明確標示；引用頁面的具體條文需由讀者回查確認。本審查未執行程式，也未獨立查核來源。

第25章 場景階層與關鍵影格

學習目標與先備知識

本章把靜態場景變成可按時間取樣的動畫場景。讀完後，你應能：

- 以局部變換描述物件相對父節點的位置，並組合成世界變換。
- 區分局部座標、父座標與世界座標。
- 以關鍵影格插值位置，並說明旋轉與縮放插值的限制。
- 區分固定時間步更新與關鍵影格取樣，並測試座標往返。
- 辨識矩陣次序錯誤、影格時間無效及奇異縮放等常見問題。

先備知識是向量、矩陣、齊次座標與基本 Python。沿用全書慣例：向量為直向量，使用右手座標系；變換矩陣由右向左作用。局部矩陣採

$$M_{\text{local}} = TRS,$$

所以點先縮放、再旋轉、最後平移。父子關係採

$$M_{\text{world}} = M_{\text{parent}} M_{\text{local}}.$$

長度以公尺、時間以秒表示。動畫案例使用合成資料，不代表真實魚類運動的測量或生物學模型。

問題與直覺

若魚鰭的位置直接記錄在世界座標中，魚身移動時就得同步更新魚鰭的位置、方向與尺度。場景階層讓魚鰭以魚身為父節點，只需描述魚鰭相對魚身的局部變換；魚身移動時，魚鰭便跟著移動。

關鍵影格是在指定時間記錄物件狀態。例如魚身在 $t = 0$ 秒位於池中央，在 $t = 2$ 秒移至右側。中間時刻可以用兩個影格插值取得。本章實作位置的線性插值；旋轉和縮放雖也能建立關鍵影格，但須採用符合其幾何性質的方法。任意矩陣逐元素插值可能破壞旋轉，縮放跨過零則會令變換奇異。

數學與幾何推導

局部變換與世界變換

齊次點寫作

$$p_h = (x, y, z, 1)^T.$$

若節點有父節點，局部點轉成世界點的關係為

$$p_{\text{world}} = M_{\text{world}} p_{\text{local}} = M_{\text{parent}} M_{\text{local}} p_{\text{local}}.$$

沒有父節點時，令 $M_{\text{parent}} = I$ 。多層階層沿根節點到目標節點逐層相乘：

$$M_{\text{world},k} = M_{\text{world,parent}(k)} M_{\text{local},k}.$$

父矩陣必須左乘局部矩陣，因為矩陣乘法通常不可交換。對 $M_{\text{local}} = TRS$ ，右側先作用，點先縮放、再旋轉、最後平移。

關鍵影格與時間參數

對時間 $t_0 < t_1$ 的兩個位置 $p_0, p_1 \in \mathbb{R}^3$ ，取樣時間 $t \in [t_0, t_1]$ 對應無單位參數

$$\alpha = \frac{t - t_0}{t_1 - t_0}, \quad 0 \leq \alpha \leq 1.$$

位置線性插值為

$$p(t) = (1 - \alpha)p_0 + \alpha p_1.$$

$t = t_0$ 時結果為 p_0 ； $t = t_1$ 時結果為 p_1 。影格資料必須依時間嚴格遞增；若兩影格時間相等，分母為零。超出影格範圍時，須明確選擇外插、夾取端點或循環播放；本章程式採夾取。

旋轉的插值限制：角度只是特定軸上的簡化旋轉表示。若以角度作關鍵影格，必須先決定如何處理角度跨越，例如 179° 到 -179° 究竟沿短路徑轉 2° ，還是沿長路徑轉 358° 。一般三維旋轉應使用單位四元數及 SLERP，並處理 q 與 $-q$ 代表同一旋轉的雙覆蓋性質；本章只固定 Z 軸旋轉，不實作其插值。

縮放的插值限制：對每軸縮放值做線性插值前，應確認區間內不會跨過零。若端點異號，尺度會在某時刻為零，使矩陣奇異；即使端點同號，極小的縮放值也會令反變換數值不穩定。實務上可在資料驗證時拒絕不允許的尺度，或明確設計鏡射與翻面處理；本章程式固定使用單位縮放。

固定時間步與座標往返

固定時間步以常數 Δt 更新狀態：

$$t_{k+1} = t_k + \Delta t.$$

固定步積分會推進模擬狀態；關鍵影格取樣則是給定時間，直接計算該時刻狀態。兩者可以併用，但不能混為一談。本章程式按固定間隔取樣位置影格，不進行物理積分。

固定間隔取樣能驗證的事情包括：端點取樣是否與關鍵影格一致、相鄰區間內插參數 α 是否落在 $[0, 1]$ 、取樣時間是否單調遞增、以及同一時間重複取樣的結果是否可重現。這些性質只依賴插值公式與影格資料，不需任何物理模型即可獨立檢查，因此適合當成動畫資料載入後的第一層防護。

反過來，單憑固定間隔的有限位置樣本，不能嚴格證明速度連續性，也不能驗證動量守恆、碰撞回應或完整的狀態更新規則。若某個動畫在某個時間點出現速度突變，取樣可能只看到前後兩點的座標都「接近」，忽略中間的異常；加速度在取樣點之間劇烈變化，也未必在稀疏座標序列中留下明顯痕跡。增加取樣密度並配合差分，有助於偵測或估計速度與加速度異常，但不能取代對插值函數、區段接點及狀態更新法則的分析，例如核對接點兩側的一階導數是否相同。固定間隔取樣是輕量的資料一致性檢查工具，可作為除錯證據，不能代替對狀態更新演算法與物理模型的驗證。

若世界矩陣可逆，世界點可反變換回局部座標：

$$p_{\text{local}} = M_{\text{world}}^{-1} p_{\text{world}}.$$

零尺度會使矩陣不可逆，因此不能做一般座標往返。測試時可選一個局部點，先乘世界矩陣，再乘其反矩陣，確認還原結果與原點在容差內一致。

本節範例刻意選擇**魚鰭的世界矩陣**之逆，而不是魚身矩陣之逆。原因在於往返測試所使用的局部點，是在魚鰭的局部座標系中定義的。若誤用魚身矩陣的逆來反算，就等於把魚鰭局部座標當成魚身局部座標來解讀，得到的結果只是「該點在魚身座標系中的位置」，一般不會等於「該點在魚鰭座標系中的位置」。若要求對所有局部點皆相同，魚鰭局部變換須為單位矩陣；對個別點則仍可能相同，例如非零旋轉也會固定局部原點。這是階層場景常被忽略的錯誤：即使世界矩陣本身可逆、數值上也看似收斂，選錯「對應哪一層的反矩陣」依然會得到看似合理、實則錯位的結果。一般階層場景應以「該點在哪個節點的局部座標系中定義」決定使用哪一層的反矩陣，不能一律使用最底層或頂層的逆。

逐步手算例題

例一：魚身與魚鰭的父子平移

魚身平移為 $(2, 0, 0)$ 公尺，魚鰭相對魚身的局部平移為 $(0, 1, 0)$ 公尺。兩者沒有旋轉或縮放，因此魚鰭世界位置為

$$(2, 0, 0) + (0, 1, 0) = (2, 1, 0).$$

若魚身移到 $(3, 0, 0)$ ，魚鰭世界位置變為 $(3, 1, 0)$ ；魚鰭局部位置仍是 $(0, 1, 0)$ 。

例二：父節點旋轉改變子節點偏移

魚身繞 Z 軸逆時針旋轉 90° ，魚鰭局部位置為 $(0, 1, 0)$ 。旋轉矩陣是

$$R_z(90^\circ) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

因此

$$R_z(90^\circ)(0, 1, 0, 1)^T = (-1, 0, 0, 1)^T.$$

若魚身另平移 $(2, 0, 0)$ ，魚鰭世界位置為 $(1, 0, 0)$ 。

例三：位置關鍵影格

魚身在 $t_0 = 0$ 秒時位於 $p_0 = (0, 1, 0)$ 公尺，在 $t_1 = 2$ 秒時位於 $p_1 = (4, 3, 0)$ 公尺。取樣 $t = 0.5$ 秒：

$$\alpha = \frac{0.5 - 0}{2 - 0} = 0.25.$$

所以

$$p(0.5) = 0.75(0, 1, 0) + 0.25(4, 3, 0) = (1, 1.5, 0).$$

魚鰭是魚身子節點時，先求魚身世界矩陣，再乘魚鰭局部矩陣；不可把魚鰭局部座標誤當成世界座標。

實作與程式

以下完整程式只用 Python 標準庫。它實作 4×4 矩陣運算、TRS、位置關鍵影格、父子世界矩陣、固定間隔取樣，以及可逆變換下的局部—世界—局部往返。旋轉固定為 Z 軸 90° ，縮放固定為單位尺度；程式不宣稱插值旋轉或縮放。

```
import math

def identity():
    return [
        [1.0, 0.0, 0.0, 0.0],
        [0.0, 1.0, 0.0, 0.0],
        [0.0, 0.0, 1.0, 0.0],
        [0.0, 0.0, 0.0, 1.0],
    ]

def mat_mul(a, b):
    return [
        [sum(a[r][k] * b[k][c] for k in range(4))
         for c in range(4)]
        for r in range(4)
    ]
```

```

def mat_vec(m, v):
    return [
        sum(m[r][c] * v[c] for c in range(4))
        for r in range(4)
    ]

def translation(x, y, z):
    m = identity()
    m[0][3], m[1][3], m[2][3] = x, y, z
    return m

def scaling(x, y, z):
    return [
        [x, 0.0, 0.0, 0.0],
        [0.0, y, 0.0, 0.0],
        [0.0, 0.0, z, 0.0],
        [0.0, 0.0, 0.0, 1.0],
    ]

def rotation_z(angle):
    c, s = math.cos(angle), math.sin(angle)
    return [
        [c, -s, 0.0, 0.0],
        [s, c, 0.0, 0.0],
        [0.0, 0.0, 1.0, 0.0],
        [0.0, 0.0, 0.0, 1.0],
    ]

def local_trs(position, angle_z, scale):
    # TRS 由右先作用：先縮放、再旋轉、最後平移。
    return mat_mul(
        translation(*position),
        mat_mul(rotation_z(angle_z), scaling(*scale))
    )

def lerp3(a, b, alpha):
    return tuple((1.0 - alpha) * a[i] + alpha * b[i]
        for i in range(3))

def validate_keys(keys):
    if not keys:
        raise ValueError("至少需要一個關鍵影格")
    for i in range(1, len(keys)):
        if keys[i][0] <= keys[i - 1][0]:
            raise ValueError("關鍵影格時間必須嚴格遞增")

def sample_position(keys, time):
    """影格時間須嚴格遞增；範圍外夾取端點。"""
    validate_keys(keys)

    if time <= keys[0][0]:

```

```

        return keys[0][1]
    if time >= keys[-1][0]:
        return keys[-1][1]

    for (t0, p0), (t1, p1) in zip(keys, keys[1:]):
        if t0 <= time <= t1:
            alpha = (time - t0) / (t1 - t0)
            return lerp3(p0, p1, alpha)

    raise AssertionError("時間應落在某個關鍵影格區間")

def inverse_affine(m):
    """反轉仿射矩陣；行列式為零或接近零時拒絕。"""
    a = [[m[r][c] for c in range(3)] for r in range(3)]
    t = [m[r][3] for r in range(3)]

    det = (
        a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1])
        - a[0][1] * (a[1][0] * a[2][2] - a[1][2] * a[2][0])
        + a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0])
    )
    if abs(det) < 1e-14:
        raise ValueError("矩陣奇異或接近奇異，不能反轉")

    inv_det = 1.0 / det
    inv_a = [
        [
            (a[1][1] * a[2][2] - a[1][2] * a[2][1]) * inv_det,
            (a[0][2] * a[2][1] - a[0][1] * a[2][2]) * inv_det,
            (a[0][1] * a[1][2] - a[0][2] * a[1][1]) * inv_det,
        ],
        [
            (a[1][2] * a[2][0] - a[1][0] * a[2][2]) * inv_det,
            (a[0][0] * a[2][2] - a[0][2] * a[2][0]) * inv_det,
            (a[0][2] * a[1][0] - a[0][0] * a[1][2]) * inv_det,
        ],
        [
            (a[1][0] * a[2][1] - a[1][1] * a[2][0]) * inv_det,
            (a[0][1] * a[2][0] - a[0][0] * a[2][1]) * inv_det,
            (a[0][0] * a[1][1] - a[0][1] * a[1][0]) * inv_det,
        ],
    ],
    ]

    inv_t = [
        -sum(inv_a[r][c] * t[c] for c in range(3))
        for r in range(3)
    ]
    return [
        [inv_a[0][0], inv_a[0][1], inv_a[0][2], inv_t[0]],
        [inv_a[1][0], inv_a[1][1], inv_a[1][2], inv_t[1]],
        [inv_a[2][0], inv_a[2][1], inv_a[2][2], inv_t[2]],
        [0.0, 0.0, 0.0, 1.0],
    ]

class Node:
    def __init__(self, local_matrix, parent=None):
        self.local_matrix = local_matrix

```

```

        self.parent = parent

def world_matrix(self):
    if self.parent is None:
        return self.local_matrix
    return mat_mul(self.parent.world_matrix(), self.local_matrix)

def close_vec(a, b, eps=1e-9):
    return all(abs(x - y) <= eps for x, y in zip(a, b))

def main():
    fish_keys = [
        (0.0, (0.0, 1.0, 0.0)),
        (2.0, (4.0, 3.0, 0.0)),
    ]

    # 關鍵影格中點取樣，魚身位置預期為 (1, 1.5, 0)。
    time = 0.5
    fish_position = sample_position(fish_keys, time)
    assert close_vec(fish_position, (1.0, 1.5, 0.0))

    fish = Node(local_trs(
        fish_position, math.pi / 2.0, (1.0, 1.0, 1.0)
    ))
    fin = Node(local_trs(
        (0.0, 1.0, 0.0), 0.0, (1.0, 1.0, 1.0)
    ), parent=fish)

    fish_world = fish.world_matrix()
    fin_world = fin.world_matrix()

    fish_origin = mat_vec(fish_world, [0.0, 0.0, 0.0, 1.0])
    fin_origin = mat_vec(fin_world, [0.0, 0.0, 0.0, 1.0])

    assert close_vec(fish_origin, (1.0, 1.5, 0.0, 1.0))

    # 局部偏移 (0,1,0) 旋轉成 (-1,0,0)，再加 (1,1.5,0)。
    assert close_vec(fin_origin, (0.0, 1.5, 0.0, 1.0))

    # 固定間隔取樣關鍵影格：0 到 2 秒，每隔 0.5 秒取樣。
    dt = 0.5
    times = [i * dt for i in range(5)]
    positions = [sample_position(fish_keys, t) for t in times]
    assert times == [0.0, 0.5, 1.0, 1.5, 2.0]
    assert close_vec(positions[0], (0.0, 1.0, 0.0))
    assert close_vec(positions[-1], (4.0, 3.0, 0.0))

    # 局部點轉到世界，再反轉回局部。
    local_point = [0.25, 0.5, 0.0, 1.0]
    world_point = mat_vec(fin_world, local_point)
    recovered = mat_vec(inverse_affine(fin_world), world_point)
    assert close_vec(recovered, local_point)

    # 同一時間重複取樣須可重現。
    assert sample_position(fish_keys, time) == fish_position

    print("hierarchy, fixed-interval sampling, and round-trip checks passed")

```

```
if __name__ == "__main__":  
    main()
```

`inverse_affine` 對線性部分行列式接近零的矩陣會拒絕反轉。程式中的 10^{-14} 門檻只適用於此範例的尺度，不是通用容差。大型階層可由根節點逐層計算並快取世界矩陣，避免對每個節點重複遞迴。

測試與預期結果

程式斷言檢查：

1. $t = 0.5$ 秒時，魚身位置為 $(1, 1.5, 0)$ 公尺。
2. 魚鰭經父節點旋轉和平移後，世界位置為 $(0, 1.5, 0)$ 公尺。
3. 固定間隔取樣從 0 到 2 秒，每隔 $\Delta t = 0.5$ 秒取樣；首尾位置與關鍵影格一致。
4. 魚鰭局部點經世界變換與逆變換後返回原值。
5. 同一時間重複取樣會得到相同位置。

若讀者執行程式，預期印出：

```
hierarchy, fixed-interval sampling, and round-trip checks passed
```

這是依程式與公式可推得的預期結果，不表示作者已執行程式或驗證特定環境。

除錯與常見陷阱

- **父子矩陣順序顛倒**：使用 $M_{\text{world}} = M_{\text{parent}}M_{\text{local}}$ 。顛倒後，子節點局部變換不再依父節點座標系組合。
- **局部位置誤當世界位置**：子節點局部座標須乘父節點世界矩陣。
- **點與方向混淆**：位置的齊次座標 $w = 1$ ，方向的 $w = 0$ ；方向不受平移影響。
- **關鍵影格重複或未排序**：插值要求相鄰影格時間嚴格遞增。載入時應檢查，取樣函式也應再次防護。
- **未定義區間外行為**：外插、夾取與循環是不同策略。本程式採端點夾取。
- **逐元素插值旋轉矩陣**：可能破壞旋轉矩陣的正交性。一般三維旋轉應使用合適的旋轉表示法插值。
- **縮放跨過零**：會使變換不可逆，也可能令幾何退化。若要變號，須明確處理翻面與法線方向。
- **固定步與影格取樣混淆**：本程式以固定間隔取樣關鍵影格，不是固定步物理積分。
- **奇異矩陣仍做往返**：零尺度矩陣不可逆；座標往返測試應使用非奇異變換，並選擇合宜容差。

養殖數位分身案例

建立一個合成池景階層：

```
scene
├─ tank
│   └─ fish_body
│       ├── left_fin
│       ├── right_fin
│       └─ tail
```

池體是場景中的固定節點。魚身局部變換描述它相對池體的位置、方向與尺度；魚鰭及尾部以魚身為父節點，保存相對魚身的局部變換。

輸出影格時，先選取時間 t ，由位置影格求魚身位置，再組合魚身局部 TRS，最後由場景根節點向下計算魚身、魚鰭與尾部的世界矩陣。若使用固定時間步更新，應記錄步長 Δt 與初始狀態；若直接取樣關鍵影格，則記錄取樣時間與影格資料版本。

這些資訊有助於重現合成影格，但不代表動畫符合真實魚群行為，也不代表場景經過實地量測。

習題

1. 手算：父子旋轉與平移

魚身平移為 $(3, 0, 0)$ 公尺，並繞 Z 軸逆時針旋轉 90° 。魚鰭局部位置為 $(0, 2, 0)$ 公尺。求魚鰭世界位置。

2. 手算：關鍵影格插值

位置影格為 $t_0 = 2$ 秒、 $(1, 0, 0)$ 公尺； $t_1 = 6$ 秒、 $(5, 4, 0)$ 公尺。求 $t = 3$ 秒的位置。若採端點夾取， $t = 8$ 秒的位置為何？

3. 程式測試與除錯：影格時間

若兩個相鄰影格時間都等於 1 秒，插值參數的分母是多少？請提出一項載入時檢查與一項取樣時保護，並說明用途。

4. 整合應用：魚身與魚鰭

魚身在 $t = 0$ 秒的位置為 $(0, 1, 0)$ ，在 $t = 2$ 秒的位置為 $(4, 1, 0)$ 。魚身固定旋轉 90° ，魚鰭局部位置為 $(0, 1, 0)$ 。求 $t = 1$ 秒時魚身及魚鰭的世界位置。若更換父節點，局部位置是否仍代表相同世界位置？

5. 反例：逐元素插值旋轉矩陣

將 0° 與 180° 的 Z 軸旋轉矩陣逐元素取平均，會得到什麼結果？為什麼不能當作一般旋轉矩陣？

習題解答

1. 手算：父子旋轉與平移

魚鰭偏移先旋轉：

$$R_z(90^\circ)(0, 2, 0) = (-2, 0, 0).$$

再加魚身平移：

$$p_{\text{world}} = (3, 0, 0) + (-2, 0, 0) = (1, 0, 0).$$

2. 手算：關鍵影格插值

$t = 3$ 時

$$\alpha = \frac{3 - 2}{6 - 2} = \frac{1}{4}.$$

因此

$$p(3) = \frac{3}{4}(1, 0, 0) + \frac{1}{4}(5, 4, 0) = (2, 1, 0).$$

$t = 8$ 超出最後一個影格，夾取結果為 $(5, 4, 0)$ 。

3. 程式測試與除錯：影格時間

$t_1 - t_0 = 0$ ，插值參數的分母為零。載入時應檢查影格時間是否嚴格遞增，以拒絕重複或倒序資料；取樣函式也應檢查相鄰時間，遇到 $t_1 \leq t_0$ 就報錯。前者保護資料品質，後者避免錯誤資料繞過載入檢查後產生無效計算。

4. 整合應用：魚身與魚鰭

$t = 1$ 秒位於兩影格中點，魚身位置為

$$p_{\text{body}}(1) = \frac{1}{2}(0, 1, 0) + \frac{1}{2}(4, 1, 0) = (2, 1, 0).$$

魚鰭局部偏移經 90° 旋轉後為 $(-1, 0, 0)$ ，世界位置為

$$p_{\text{fin}} = (2, 1, 0) + (-1, 0, 0) = (1, 1, 0).$$

更換父節點後，局部位置仍表示相對新父節點的座標，一般不會保持原世界位置。若要求保持世界位置，須以新父節點世界矩陣的逆矩陣重新計算局部變換。

5. 反例：逐元素插值旋轉矩陣

0° 與 180° 的 XY 平面矩陣區塊為

$$R_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_{180} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}.$$

逐元素取平均得到零矩陣。它把所有平面向量映到零向量，既不保長度也不可逆，並非旋轉矩陣。

本章小結

場景階層以 $M_{\text{world}} = M_{\text{parent}} M_{\text{local}}$ 累積局部變換。位置關鍵影格可在相鄰時間點間線性插值；影格時間必須嚴格遞增，超出範圍的策略也須明訂。固定時間步是狀態更新方法，關鍵影格取樣則依時間直接求值。本章實作位置插值；一般三維旋轉需要適當的旋轉插值方法，縮放則應避免穿越零。座標往返測試須使用可逆變換，並採用符合場景尺度的容差。

參考來源

以下來源供延伸閱讀；列出不表示本章已逐條外部驗證所有論點。

- [G1] *Physically Based Rendering, Fourth Edition*，Transformations：https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G5] LearnOpenGL，Transformations：<https://learnopengl.com/Getting-started/Transformations>
- [G6] Blender Manual，Skinning Introduction：
<https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>

第 26 章 四元數、旋轉與SLERP

學習目標與先備知識

讀完本章，你應該能：

1. 說明 Euler 角的三個具體局限：次序依賴、萬向鎖（gimbal lock）、插值不沿最短路徑；並指出什麼情況下仍可用 Euler 角。
2. 用 (w, x, y, z) 固定順序表示單位四元數 q ，並以半角公式 $q = (\cos \frac{\theta}{2}, \sin \frac{\theta}{2} \hat{n})$ 構造繞軸 $\hat{n}$ 旋轉 θ 的旋轉。
3. 以 Hamilton 乘法計算四元數乘積，並以 $v' = q v q^*$ 對三維向量做主動旋轉， v 以純虛四元數 $(0, v_x, v_y, v_z)$ 表示。
4. 解釋**雙覆蓋**： q 與 $-q$ 表示同一旋轉，並在插值前檢查內積符號取**最短弧**。
5. 推導並實作 **SLERP** $\text{slerp}(q_0, q_1, t)$ ，在 $q_0 \cdot q_1 \rightarrow 1$ （即 q_0 與 $\pm q_1$ 近共線）時改用線性插值+正規化以避免除以零。

先備（Volume I／本卷）：向量內積、外積、矩陣乘法次序、三角函數、弧度制。本章沿用共同約定：

- **四元數固定** (w, x, y, z) **順序**，Hamilton 乘法，單位四元數主動旋轉， $q v q^*$ 。
- 角度運算用弧度；右手正角； $+X$ 右、 $+Y$ 上、 $+Z$ 朝觀者。
- 旋轉的正向為從該軸正向看向原點時的逆時針。
- 程式碼 Python 3.10+ 標準庫即可，可選 NumPy 2.2.6 相容寫法，但不依賴安裝。

本章不重複矩陣旋轉的細節（第 3 章已介紹）；重點是四元數與其插值如何解決 Euler 角在動畫路徑上的問題。

問題與直覺

動畫需要**連續旋轉**：一條魚在池中轉向、翻滾、上下擺動，每一幀給一個姿態。表示姿態有三種常見方式：

- **Euler 角**（yaw-pitch-roll）：直觀、可讀，但依賴應用次序，且當 pitch 趨近 $\pm 90^\circ$ 時 yaw 與 roll 耦合、失去一個自由度，這叫**萬向鎖**。
- **旋轉矩陣**：可做複合，但插值不自然；對兩矩陣做線性插值後一般不再是旋轉矩陣（正交性被破壞）。
- **單位四元數**：四維單位球面 S^3 上的點，乘法對應旋轉複合，插值沿球面大圓走，天然解決最短路徑與萬向鎖。

核心問題：

1. 如何用四個數表示旋轉？半角與軸角。

2. 如何複合兩個旋轉？Hamilton 乘法。

3. 如何對兩個姿態插值？球面線性插值（SLERP），沿單位球面大圓弧走，且要先處理雙覆蓋。

雙覆蓋之所以是問題： q 與 $-q$ 在 S^3 上是對徑點，對應同一旋轉。若直接對 q_0 與 q_1 做 SLERP，當 $q_0 \cdot q_1 < 0$ 時會走長弧（旋轉超過 180° ），違背動畫中「走最短路徑」的直覺。

數學與幾何推導

Euler 角的三個局限

設 $R = R_z(\psi)R_y(\theta)R_x(\phi)$ （固定次序 yaw-pitch-roll，最右邊先作用於向量）。局限如下：

- **次序依賴**： $R_x(\phi)R_z(\psi) \neq R_z(\psi)R_x(\phi)$ 。寫下「某物體 yaw 30° 、pitch 20° 」時必須同時指定次序，否則數值不同。
- **萬向鎖**：當 pitch $\theta = \pm \frac{\pi}{2}$ ， $R_y(\theta)$ 把 X 軸映到 $\mp Z$ ，yaw 與 roll 的旋轉軸重合， $R_z(\psi)R_x(\phi)$ 兩個角度只貢獻一個自由度。此時對兩姿態插值會出現抖動或突然反轉。
- **插值非最短路徑**：對 Euler 角三元組做線性插值，每個分量的變化是獨立的，結合後在旋轉流形上一般不是測地線。

Euler 角仍適用於**單獨的姿態輸入**（例如人工指定魚頭朝向）或**單軸連續旋轉**；不適合插值路徑與複合關鍵影格。

單位四元數與旋轉公式

四元數 $q = (w, x, y, z) \in \mathbb{R}^4$ 可寫成 $q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$ ，其中 $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$ 。Hamilton 乘法為

$$(w_1, \mathbf{v}_1)(w_2, \mathbf{v}_2) = (w_1w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, w_1\mathbf{v}_2 + w_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2),$$

其中 $\mathbf{v}_i = (x_i, y_i, z_i) \in \mathbb{R}^3$ 。共軛 $q^* = (w, -x, -y, -z)$ ；對單位四元數（ $w^2 + x^2 + y^2 + z^2 = 1$ ）而言 $q^{-1} = q^*$ 。

繞軸單位向量 $\hat{n}$ 旋轉角度 θ 的四元數為

$$q = \left(\cos \frac{\theta}{2}, \sin \frac{\theta}{2} \hat{n} \right).$$

半角來自「旋轉需要 4π 才回到原點」： q 作為 S^3 的點，走一圈 2π 得到 $-q$ 而非 q ；這正是雙覆蓋。

對三維向量 v 做主動旋轉：

$$v' = q(0, v_x, v_y, v_z)q^*,$$

展開後為純虛四元數；取其 (x, y, z) 即為旋轉後的向量。等價的矩陣形式為 $R(q)$ ，其元素由 q 的分量構成（此處不展開，讀者可自 PBRT 或標準線代教材核對）。

複合、逆旋轉與雙覆蓋

- **複合**： $q_{1 \rightarrow 3} = q_{2 \rightarrow 3} q_{1 \rightarrow 2}$ （先 $q_{1 \rightarrow 2}$ 再 $q_{2 \rightarrow 3}$ ）。注意次序；Hamilton 乘法非交換。
- **逆旋轉**： $q^{-1} = q^*$ 。
- **雙覆蓋**： q 與 $-q$ 表示同一旋轉。物理上無差異，但插值時有差異：見 SLERP。

SLERP

給單位四元數 q_0, q_1 與參數 $t \in [0, 1]$ 。令

$$d = q_0 \cdot q_1 = w_0 w_1 + x_0 x_1 + y_0 y_1 + z_0 z_1.$$

第一步（短弧調整）：若 $d < 0$ ，以 $q_1 \leftarrow -q_1$ 、 $d \leftarrow -d$ 取代，使兩者落在 S^3 同一半球上，對應最短路徑。

第二步：令 $\Omega = \arccos d \in [0, \pi/2]$ （因已取 $d \geq 0$ ）。當 $\sin \Omega$ 明顯不為零（例如 $\sin \Omega > 10^{-6}$ ）：

$$\text{slerp}(q_0, q_1, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1.$$

第三步（近共線退化）： $d \rightarrow 1$ 時 $\Omega \rightarrow 0$ 、 $\sin \Omega \rightarrow 0$ ，數值不穩。此時改用「線性插值後正規化」（nlerp）：

$$\tilde{q} = (1-t)q_0 + tq_1, \quad q = \frac{\tilde{q}}{\|\tilde{q}\|}.$$

需要注意的邊界情況： $d = -1$ （ q_1 為 q_0 的對徑點）在短弧調整後同樣變成 $d = 1$ ，也走 nlerp 分支。**這裡必須分清兩個不同的「路徑」概念**：

- **球面原始端點之間的弧**：在 S^3 上， q_0 與 $-q_0$ 是對徑點，兩者之間有無窮多條等長大圓弧（長度皆為 π ）。單看 S^3 的幾何，哪一條「最短」並不唯一。
- **雙覆蓋後的旋轉路徑**： q_0 與 $-q_0$ 表示同一個旋轉。短弧調整把 q_1 翻轉成與 q_0 同向後，兩端在 S^3 上變成同一個點，nlerp 給出的就是**常值姿態**：從起點到終點的旋轉距離為零，路徑是一個點，不是某條半圓弧。

換句話說，此情形下不存在「選哪條半圓弧」的問題，因為在旋轉空間裡起點與終點是同一姿態；本章的處理只是給出一個可重現的結果，不做進一步的球面 tie-breaking。 $d = 0$ **不是奇異點**：此時 $\Omega = \pi/2$ 、 $\sin \Omega = 1$ ，SLERP 公式直接可用。

nlerp 與 SLERP 的差異在 $t \in (0, 1)$ 中間：nlerp 沿弦走，速度略非等角，但在動畫中通常可接受；若需要精確等角速度（例如軌道運動、機械關節）用 SLERP。

性質檢核： $\text{slerp}(q_0, q_1, 0) = q_0$ ， $\text{slerp}(q_0, q_1, 1) = q_1$ （在短弧調整後），且對任意 t ，結果為單位四元數。若跳過短弧調整， $d < 0$ 時 $t = 1$ 會得到原始的 q_1 （因為 $t = 1$ 時只有 q_1 前的係數非零）；做了短弧調整之後， q_1 已被翻轉為 $-q_1$ ， $t = 1$ 得到的是 $-q_1$ 。兩者表示同一旋轉、但對應的插值路徑不同：前者走長弧，後者走短弧。

逐步手算例題

例 1（繞 Z 軸旋轉 90° ，作用於 $+X$ ）。取 $\hat{n} = (0, 0, 1)$ 、 $\theta = \pi/2$ ，則 $\frac{\theta}{2} = \pi/4$ ，令 $c = \frac{\sqrt{2}}{2}$ 。

$$q = (c, 0, 0, c), \quad q^* = (c, 0, 0, -c).$$

$v = (1, 0, 0)$ 對應純虛四元數 $p = (0, 1, 0, 0)$ 。以下分兩步計算 qpq^* 。

第一步： $r = qp$ 。將 $q = (w_1, \mathbf{v}_1)$ 拆為 $w_1 = c$ 、 $\mathbf{v}_1 = (0, 0, c)$ ； $p = (w_2, \mathbf{v}_2)$ 拆為 $w_2 = 0$ 、 $\mathbf{v}_2 = (1, 0, 0)$ 。

- 純量部分： $w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 = c \cdot 0 - (0, 0, c) \cdot (1, 0, 0) = 0$ 。
- 向量部分： $w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 = c(1, 0, 0) + 0 + (0, 0, c) \times (1, 0, 0)$ 。
 - $c(1, 0, 0) = (c, 0, 0)$
 - 叉積 $(0, 0, c) \times (1, 0, 0) = (0 \cdot 0 - c \cdot 0, c \cdot 1 - 0 \cdot 0, 0 \cdot 0 - 0 \cdot 1) = (0, c, 0)$
 - 合計 $(c, c, 0)$

所以 $r = (0, c, c, 0)$ 。

第二步： $v' = r q^*$ 。將 $r = (w_1, \mathbf{v}_1) = (0, (c, c, 0))$ 、 $q^* = (w_2, \mathbf{v}_2) = (c, (0, 0, -c))$ 。

- 純量部分： $w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 = 0 - (c \cdot 0 + c \cdot 0 + 0 \cdot (-c)) = 0$ 。
- 向量部分： $w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2$ 。
 - $w_1 \mathbf{v}_2 = 0$
 - $w_2 \mathbf{v}_1 = c(c, c, 0) = (c^2, c^2, 0)$
 - $(c, c, 0) \times (0, 0, -c) = (c \cdot (-c) - 0 \cdot 0, 0 \cdot 0 - c \cdot (-c), c \cdot 0 - c \cdot 0) = (-c^2, c^2, 0)$
 - 合計 $(c^2 - c^2, c^2 + c^2, 0) = (0, 2c^2, 0) = (0, 1, 0)$

結果為純虛四元數 $(0, 0, 1, 0)$ ，對應向量 $(0, 1, 0)$ 。即繞 $+Z$ 旋轉 90° 把 $+X$ 映到 $+Y$ ，與右手系約定一致。

例 2（SLERP）。 $q_0 = (1, 0, 0, 0)$ （恆等旋轉）， $q_1 = (\cos \frac{\pi}{4}, 0, 0, \sin \frac{\pi}{4}) = (\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2})$ （繞 Z 軸 90° ）。取 $t = 0.5$ 。

先算 $d = q_0 \cdot q_1 = \frac{\sqrt{2}}{2} > 0$ ，不需翻轉。 $\Omega = \arccos \frac{\sqrt{2}}{2} = \pi/4$ 。

$$\text{slerp} = \frac{\sin(\pi/8)}{\sin(\pi/4)} q_0 + \frac{\sin(\pi/8)}{\sin(\pi/4)} q_1.$$

$\sin(\pi/8) \approx 0.38268$ ， $\sin(\pi/4) \approx 0.70711$ ，比值約 0.54120 。

- q_0 貢獻：(0.54120, 0, 0, 0)
- q_1 貢獻： $0.54120 \cdot (0.70711, 0, 0, 0.70711) \approx (0.38268, 0, 0, 0.38268)$
- 總和：(0.92388, 0, 0, 0.38268)

檢核： $0.92388^2 + 0.38268^2 \approx 0.85355 + 0.14645 = 1$ 。角度約 $\arccos(0.92388) \cdot 2 \approx 22.5^\circ \cdot 2 = 45^\circ$ ，正好是 90° 的一半，符合預期。

例 3 (雙覆蓋與短弧)。 $q_0 = (1, 0, 0, 0)$ ， $q_1 = (-\frac{\sqrt{2}}{2}, 0, 0, -\frac{\sqrt{2}}{2})$ ，則 $d = -\frac{\sqrt{2}}{2} < 0$ 。

- 不做短弧調整： $\Omega = \arccos(-\frac{\sqrt{2}}{2}) = 3\pi/4$ ，SLERP 走長弧；軌跡對應的旋轉在此區段中先朝負方向旋轉約 270° ，與「走向最短 90° 旋轉」的動畫直覺相悖。
- 做短弧調整： $q_1 \leftarrow (-1) \cdot (-\frac{\sqrt{2}}{2}, 0, 0, -\frac{\sqrt{2}}{2}) = (\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2})$ ，回到例 2 的情形，SLERP 走短弧 90° 。

動畫通常選短弧。這正是 `q_slerp` 中 `if d<0` 段落的作用。

實作與程式

以下程式只依賴 **Python 3.10+ 標準庫**，讀者可自行複製執行。

```
# 檔案：quat.py 只依賴標準庫
import math

def q_identity():
    return (1.0, 0.0, 0.0, 0.0)

def q_from_axis_angle(axis, theta):
    """axis 為單位向量 (nx,ny,nz)；theta 弧度；回傳單位四元數 (w,x,y,z)。”
    nx, ny, nz = axis
    s = math.sin(0.5 * theta)
    return (math.cos(0.5 * theta), s * nx, s * ny, s * nz)

def q_mul(a, b):
    """Hamilton 乘法；a、b 為 (w,x,y,z)。”
    wa, xa, ya, za = a
    wb, xb, yb, zb = b
    return (
        wa * wb - xa * xb - ya * yb - za * zb,
        wa * xb + xa * wb + ya * zb - za * yb,
        wa * yb - xa * zb + ya * wb + za * xb,
        wa * zb + xa * yb - ya * xb + za * wb,
    )

def q_conj(q):
    w, x, y, z = q
    return (w, -x, -y, -z)

def q_norm(q):
    w, x, y, z = q
    return math.sqrt(w * w + x * x + y * y + z * z)

def q_normalize(q):
```

```

n = q_norm(q)
if n == 0.0:
    raise ValueError("zero quaternion")
w, x, y, z = q
return (w / n, x / n, y / n, z / n)

def q_rotate_vector(q, v):
    """v = (vx,vy,vz) 為主動旋轉：v' = q (θ,v) q*。"""
    p = (0.0, v[0], v[1], v[2])
    r = q_mul(q_mul(q, p), q_conj(q))
    return (r[1], r[2], r[3])

def q_dot(a, b):
    return a[0] * b[0] + a[1] * b[1] + a[2] * b[2] + a[3] * b[3]

def q_slerp(q0, q1, t, sin_eps=1e-6):
    """短弧球面線性插值；近共線時退化為 nlerp。
    輸入無須預先標準化，函式內部會正規化。"""
    q0 = q_normalize(q0)
    q1 = q_normalize(q1)
    d = q_dot(q0, q1)
    if d < 0.0:
        q1 = (-q1[0], -q1[1], -q1[2], -q1[3])
        d = -d
    # 短弧調整後 d ∈ [0,1]；浮點誤差可能使 d 略大於 1，需截斷。
    if d > 1.0:
        d = 1.0
    Omega = math.acos(d)
    sin_Omega = math.sin(Omega)
    if sin_Omega < sin_eps:
        # 近共線：d ≈ 1, Omega ≈ 0，改走 nlerp。
        w = (1.0 - t) * q0[0] + t * q1[0]
        x = (1.0 - t) * q0[1] + t * q1[1]
        y = (1.0 - t) * q0[2] + t * q1[2]
        z = (1.0 - t) * q0[3] + t * q1[3]
        return q_normalize((w, x, y, z))
    a = math.sin((1.0 - t) * Omega) / sin_Omega
    b = math.sin(t * Omega) / sin_Omega
    return (
        a * q0[0] + b * q1[0],
        a * q0[1] + b * q1[1],
        a * q0[2] + b * q1[2],
        a * q0[3] + b * q1[3],
    )

```

要點：

- 四元數一律用 (w, x, y, z) 的 tuple 表示；乘法公式以 $(w, \mathbf{v})$ 形式展開後直接對應到四個分量。
- `q_rotate_vector` 用 $q(0, \mathbf{v})q^*$ ；等價於「以 $R(q)$ 乘 $\mathbf{v}$ 」的矩陣形式。此處以兩次 `q_mul` 表達，是為了清晰；若要效率可改為展開後的 3×3 矩陣乘法，但不在本章重點。
- `q_slerp` 內部先正規化、再做短弧調整、再做近共線退化，呼叫端不必重複處理。
- `sin_eps` 依應用設定：對一般動畫 10^{-6} 已足夠；若要求更緊，可調至 10^{-12} 但需留意 $\arccos$ 在 $d \rightarrow 1$ 的數值放大。
- 短弧調整後 d 理論上 ≥ 0 ；`if d>1.0: d=1.0` 用來吸收浮點誤差，避免 `math.acos` 拋例外。

測試與預期結果

以下測試僅用標準庫 `math.isclose` 進行近似比較，不引入第三方套件。

測試 1：恆等與逆。

```
q = q_from_axis_angle((0.0, 0.0, 1.0), math.pi / 2)
r = q_rotate_vector(q, (1.0, 0.0, 0.0))
assert math.isclose(r[0], 0.0, abs_tol=1e-12)
assert math.isclose(r[1], 1.0, abs_tol=1e-12)
assert math.isclose(r[2], 0.0, abs_tol=1e-12)

r_id = q_rotate_vector(q_identity(), (0.3, -0.7, 1.2))
for a, b in zip(r_id, (0.3, -0.7, 1.2)):
    assert math.isclose(a, b, abs_tol=1e-12)
```

預期：全部通過。

測試 2：複合次序。設 q_x 為繞 X 軸 90° 、 q_z 為繞 Z 軸 90° 。

```
qx = q_from_axis_angle((1.0, 0.0, 0.0), math.pi / 2)
qz = q_from_axis_angle((0.0, 0.0, 1.0), math.pi / 2)
v = (0.0, 1.0, 0.0)
r1 = q_rotate_vector(q_mul(qz, qx), v)  # 先 qx 後 qz
r2 = q_rotate_vector(q_mul(qx, qz), v)  # 先 qz 後 qx
# 預期 r1 != r2
```

預期： $r_1 \neq r_2$ ，且兩者皆為單位向量。具體數值讀者可用手算驗證。

測試 3：雙覆蓋與短弧。

```
q0 = (1.0, 0.0, 0.0, 0.0)
q1_neg = (-math.sqrt(0.5), 0.0, 0.0, -math.sqrt(0.5))
q1_pos = (math.sqrt(0.5), 0.0, 0.0, math.sqrt(0.5))
s_neg = q_slerp(q0, q1_neg, 0.5)
s_pos = q_slerp(q0, q1_pos, 0.5)
# 預期 s_neg 與 s_pos 相等（短弧調整使兩者收斂到同一路徑）
for a, b in zip(s_neg, s_pos):
    assert math.isclose(a, b, abs_tol=1e-12)
# 預期 s_neg 為單位四元數
assert math.isclose(q_norm(s_neg), 1.0, abs_tol=1e-12)
```

預期：通過。若在 `q_slerp` 內移除短弧調整（if `d<0` 該段），則 `s_neg` 會與 `s_pos` 不同（前者走長弧）。

測試 4：近共線。 $q_0 = q_1 = (1, 0, 0, 0)$ ，`d=1.0`，`sin_0omega=0`。**預期：**`q_slerp` 走 `nlerp` 分支，對任意 t 回傳 `(1, 0, 0, 0)`（正規化後）；不得拋出除以零。

測試 5：插值等角速度。 q_0 與 q_1 相差 90° 繞 Z 。以 $t \in \{0, 0.25, 0.5, 0.75, 1\}$ 計算 `q_slerp`，對每個結果取角度 $\theta_t = 2 \arccos(w_t) \in [0, \pi]$ 。**預期：** θ_t 對 t 為線性，斜率 $\pi/2$ 。若改用 `nlerp`，此恆等姿態到 $+90^\circ$ 的例子中， $t = 0.5$ 恰為 45° ，與 SLERP 的差為零； $t = 0.25$ 約為 21.6° ，低於 22.5° ， $t = 0.75$ 約為 68.4° ，高於 67.5° 。前、後半段偏差符號相反，最大絕對偏差出現在中點兩側，不在中點。

以上具體數字需讀者執行後量測；本章未執行。

除錯與常見陷阱

- **旋轉方向：** q 與 q^* 分別對應旋轉與逆旋轉；將兩者寫反是常見錯誤，症狀是繞軸方向相反但角度相同。測試 1 與測試 2 可交叉驗證。
- **順序：** $q_a q_b$ 與 $q_b q_a$ 一般不同。動畫場景中「世界系旋轉後再局部旋轉」與「局部旋轉後再世界系旋轉」結果不同；務必明示複合次序。
- **未正規化：**四元數在迭代相乘或多次插值後會有浮點漂移，偏離單位長度。 $v' = q v q^*$ 與矩陣 $R(q)$ 皆假設 q 為單位；長時間累積後應定期 `q_normalize`（例如每 N 步一次，或每次插值後都做）。
- **雙覆蓋：** q 與 $-q$ 是同一旋轉。比較兩個四元數是否「相同姿態」時，應比較 $|d|$ 是否 ≈ 1 ，而非比較分量是否相等；這在動畫狀態機、關鍵影格查表時尤其重要。
- **SLERP 在 $d \rightarrow 1$ 的數值問題：** $\Omega = \arccos d$ 在 $d \rightarrow 1$ 附近對輸入非常敏感，且 $\sin \Omega \rightarrow 0$ 使除法不穩。`q_slerp` 的做法是先截斷 d 至 $[0, 1]$ 、再以 `sin_0mega < sin_eps` 決定是否走 `nlerp`。注意 $d = 0$ 不是奇異點；真正的退化只在 $d \rightarrow \pm 1$ （短弧調整後皆變為 $d \rightarrow 1$ ）。
- **弧度制：**所有角度參數為弧度；若使用者以角度輸入，須在邊界轉換。三角函式與 $\arccos$ 皆為弧度。
- **Euler 轉四元數的次序：**若從 UI 收到 `(yaw, pitch, roll)`，轉換公式依應用慣例（XYZ、ZYX 等）不同。務必在文件或函式名上明示，且旋轉軸對應哪個分量也須寫清楚。
- **插值非等弧長：**`nlerp` 速度略非等角；若動畫中對姿態速度敏感（例如機械臂同步），改用 SLERP。反之效能敏感處可接受 `nlerp`。
- **接近 180° 的兩姿態：** $d \approx 0$ ，SLERP 分母不零但接近，短弧方向由浮點符號決定；若應用要求穩定選擇，須明示 tie-breaking 規則（例如以軸向量字典序）。

養殖數位分身案例

魚體姿態在動畫中常用四元數表示：魚頭朝向由 look 方向與 up 方向決定（第 4 章相機模型同理）。若要保持「魚身側向擺動」的局部旋轉，可以魚體自己為父節點，魚鰭另加局部四元數；世界姿態為

$$q_{\text{fin,world}} = q_{\text{body,world}} \cdot q_{\text{fin,local}},$$

（依共同約定 $M_{\text{world}} = M_{\text{parent}} M_{\text{local}}$ ，四元數側為左乘父四元數。）

關鍵影格動畫：兩個關鍵時間分別存 $q(t_0) = q_0$ 、 $q(t_1) = q_1$ ；兩者之間以 `q_slerp(q0, q1, (t - t_0)/(t_1 - t_0))` 求任意時刻姿態。這保證魚頭在兩個姿態間沿**最短路徑**旋轉，翻滾不會突然加劇。

雙覆蓋在實務中的例子：若上游以 Euler 角分別記錄兩關鍵影格，轉成四元數後可能落在 S^3 的對徑半球 ($d < 0$)。若直接 SLERP 而未取短弧，魚在同一動作中會突然多轉一整圈；`q_slerp` 內的 `if d<0` 就是用來處理這件事。

若以 (x, y, z) 表魚頭朝向並希望有一確定「上方向」的魚身，可用 look-at 構造： $\hat{z} = -\text{look}$ 、 $\hat{y}$ 為指定 up 正交化、 $\hat{x} = \hat{y} \times \hat{z}$ (依第 4 章約定)，再轉為四元數。這與 Euler 角表示法不同但等價；對插值來說四元數更直接。

動畫資料為**合成資料**：本章產生的魚群軌跡與姿態不是實際觀測，僅用於視覺化與演算法驗證；不得當作生物行為或物理正確性的量測。

習題

習題 1 (手算)。設 $\hat{n} = \left(0, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$ (+Y 與 +Z 的中間方向)， $\theta = \pi/2$ 。(a) 寫出對應單位四元數 q 。(b) 以 $v' = q v q^*$ 計算 $v = (1, 0, 0)$ 的像，說明結果。(c) 設 $q_0 = (1, 0, 0, 0)$ ， $q_1 = q$ ，以 $t = 0.5$ 求 $\text{slerp}(q_0, q_1, t)$ ，並驗證其對應的旋轉角度為原角的一半。

習題 2 (程式測試)。使用本章 `quat.py`：(a) 驗證 `q_mul` 對單位四元數保持單位長度 (誤差 $< 10^{-12}$)，測 100 組隨機輸入。(b) 驗證 `q_rotate_vector(q_mul(a,b), v) == q_rotate_vector(a, q_rotate_vector(b, v))` 對若干 (a, b, v) 成立 (誤差 $< 10^{-12}$)。(c) 對 `q_slerp` 的 $t \in [0, 1]$ 做 11 等分，驗證 $\|q_t\| = 1$ 。角度線性性須在**相對起點旋轉距離**上檢驗，而非每個結果相對恆等旋轉的角度。實作建議採下列兩種之一：

1. 固定 $q_0 = (1, 0, 0, 0)$ ，並選 q_1 為繞**單一指定軸**的短弧旋轉；此時 $\theta_t = 2 \arccos(\text{clamp}(w_t, -1, 1))$ 對 t 線性。
2. 對一般 q_0, q_1 ，計算相對起點的角度

$$\phi_t = 2 \arccos(\text{clamp}(|q_0 \cdot q_t|, 0, 1)),$$

應等於 t 乘兩端的最短旋轉距離；取絕對值與短弧調整一致，避免 q_t 與 $-q_t$ 造成的符號跳動。**預期：** ϕ_t 對 t 線性；若將 `q_slerp` 換成 `nlerp`，此 90° 例子的 ϕ_t 在前半段低於線性值、後半段高於線性值，中點恰等於線性值；最大絕對偏差在中點兩側，量級約 10^{-2} 弧度。

習題 3 (反例／除錯)。以下片段聲稱做 SLERP：

```
def bad_slerp(q0, q1, t):
    d = q_dot(q0, q1)
    Omega = math.acos(d)
    a = math.sin((1-t)*Omega) / math.sin(Omega)
    b = math.sin(t*Omega) / math.sin(Omega)
    return tuple(a*q0[i] + b*q1[i] for i in range(4))
```

(a) 指出至少兩個數值／數學缺陷。(b) 給一組具體輸入使該函式拋出 `ZeroDivisionError` 或回傳非單位四元數。(c) 給一組具體輸入使該函式走「長弧」而呼叫者期待短弧。

習題 4 (整合應用)。使用 `quat.py` 設計一個魚頭姿態的關鍵影格函式 `pose_at(q_list, times, t)`：輸入關鍵影格四元數清單與對應時間，線性搜尋所在區段後以 SLERP 求 $q(t)$ 。另外設計驗證：

- 對每個關鍵時間 t_i ，`pose_at` 回傳 q_i （允許 q_i 或 $-q_i$ ，以 $\|q(t_i) - q_i\| < 10^{-9}$ 或 $\|q(t_i) + q_i\| < 10^{-9}$ 之一成立）。
- 對一時變場景（例如魚頭 yaw 從 -90° 到 $+90^\circ$ ），以逐步掃描 t 檢查相鄰姿態的四元數內積絕對值接近 1（無翻滾突變）。

說明若上游關鍵影格是 Euler 角（yaw 從 -90° 到 $+90^\circ$ ）且直接線性插值角度，會有什麼問題；並說明本案以 SLERP 取代有何改善。

習題解答

習題 1。

(a) $\hat{n}$ 三分量為 $(0, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ ， $\theta/2 = \pi/4$ ， $\cos = \sin = \frac{\sqrt{2}}{2}$ 。 $s = \frac{\sqrt{2}}{2}$ ，故

$$q = \left(\frac{\sqrt{2}}{2}, 0, \frac{1}{2}, \frac{1}{2} \right).$$

檢核 $\frac{1}{2} + \frac{1}{4} + \frac{1}{4} = 1$ 。

(b) 用 $q v q^*$ 。也可用 Rodrigues 公式驗證：

$$v' = v \cos \theta + (\hat{n} \times v) \sin \theta + \hat{n}(\hat{n} \cdot v)(1 - \cos \theta).$$

$\hat{n} \cdot v = 0$ 。叉積 $\hat{n} \times v$ ：

$$\hat{n} \times v = \left(0 \cdot 0 - \frac{1}{\sqrt{2}} \cdot 0, \frac{1}{\sqrt{2}} \cdot 1 - 0 \cdot 0, 0 \cdot 0 - \frac{1}{\sqrt{2}} \cdot 1 \right) = \left(0, \frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}} \right).$$

$v \cos \theta = (0, 0, 0)$ ； $(\hat{n} \times v) \sin \theta = \left(0, \frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}} \right)$ ；第三項為零。故

$$v' = \left(0, \frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}} \right),$$

長度為 1。此向量垂直 $+X$ ，且落在 $+Y$ 、 $-Z$ 之間，符合繞 $\hat{n}$ 轉 90° 的直覺。

(c) $d = q_0 \cdot q_1 = \frac{\sqrt{2}}{2} > 0$ ， $\Omega = \arccos \frac{\sqrt{2}}{2} = \pi/4$ 。

$t = 0.5$ 時， $a = b = \frac{\sin(\pi/8)}{\sin(\pi/4)} \approx 0.54120$ 。

$$\text{slerp} = (0.54120)(1, 0, 0, 0) + (0.54120) \left(\frac{\sqrt{2}}{2}, 0, \frac{1}{2}, \frac{1}{2} \right).$$

$\frac{\sqrt{2}}{2} \approx 0.70711$ ：前半 $= 0.54120$ ，後半 $0.54120 \cdot 0.70711 \approx 0.38268$ ，中段 $0.54120 \cdot 0.5 \approx 0.27060$ 。

$$\text{slerp} \approx (0.92388, 0, 0.27060, 0.27060).$$

角度： $2 \arccos(0.92388) \approx 2 \cdot 22.5^\circ = 45^\circ$ ，為原 90° 的一半，符合預期。長度檢核： $0.92388^2 + 2 \cdot 0.27060^2 \approx 0.85355 + 0.14645 = 1$ 。

習題 2。

(a) 取 $N = 100$ 組可控隨機（例如 `random.Random(0)`）生成的四元數，先 `q_normalize`，再算 `q_mul`；對結果檢查 `abs(q_norm - 1) < 1e-12`。**預期**：全部通過。若某組雜訊太大，可改用更嚴格的 `1e-14` 並記錄；不應放寬到 10^{-6} 之類鬆散值，那將失去測試意義。

(b) 對若干組 a, b, v ：左邊對 v 施加 $q_a q_b$ ；右邊先施加 q_b 再施加 q_a 。因 $(q_a q_b) v (q_a q_b)^* = q_a (q_b v q_b^*) q_a^*$ ，兩者理論上相等。**預期**：逐分量差 $< 10^{-12}$ 。這同時驗證複合次序：`q_mul(a, b)` 表示「先 b 後 a 」。

(c) 生成 q_0, q_1 兩姿態， $t = k/10$ 。對每個 k ：

- $|\|q_t\| - 1| < 10^{-12}$ ；
- 對一般 q_0, q_1 ，以題目指定的相對起點角度計算：

$$\phi_k = 2 \arccos(\text{clamp}(|q_0 \cdot q_k|, 0, 1)),$$

其中 q_k 是 $t = k/10$ 的插值結果。**預期**： ϕ_k 對 k 線性； ϕ_{10} 應等於兩端的最短旋轉距離，取絕對值可避免 q_k 與 $-q_k$ 造成的符號跳動。

- 如果只想檢查單軸短弧的簡單情形，可將 q_0 固定為 $(1, 0, 0, 0)$ ，並選 q_1 為繞單一指定軸的短弧旋轉；此時 $\theta_k = 2 \arccos(\text{clamp}(w_k, -1, 1))$ 對 k 線性。不滿足此兩條件的其他 q_0 ， $\theta_k = 2 \arccos(w_k)$ 可能因為 $q_0 \neq (1, 0, 0, 0)$ 而對 k 非線性，屬於規範問題，不能以此判定 `q_slerp` 錯誤。

預期：SLERP 的 ϕ_k 偏離線性量級在 10^{-14} 附近；若以 `nlerp` 取代，同一組 t 的 ϕ_k 與 SLERP 中間值最大差約 10^{-2} 弧度（ 90° 旋轉量級）。

習題 3。

(a) 缺陷：

1. `q_dot(q0, q1)` 未做短弧調整； $d < 0$ 時走長弧。
2. 未處理 $\sin \Omega = 0$ （ $d = \pm 1$ ，尤其 $q_0 = q_1$ 時 `sin(0*omega)=0` 直接除以零）。
3. 未對輸入正規化；若輸入非單位四元數， d 不再等於 $\cos \Omega$ ， $\Omega = \arccos d$ 也失效（甚至可能因為 $d > 1$ 拋出例外）。
4. 未保證輸出為單位四元數（因缺陷 1-3，輸出可能偏離）。

(b) 反例 1：`bad_slerp((1,0,0,0),(1,0,0,0),0.5)` 中 $\Omega = \arccos(1) = 0$ ， $\sin \Omega = 0$ ，除以零。反例 2：輸入兩者都非單位（例如 $(2, 0, 0, 0)$ 與 $(1, 0, 0, 0)$ ）， $d = 2$ ，`math.acos(2)` 拋 `ValueError`。

(c) 反例： $q_0 = (1, 0, 0, 0)$ ， $q_1 = (-\frac{\sqrt{2}}{2}, 0, 0, -\frac{\sqrt{2}}{2})$ 。 $d = -\frac{\sqrt{2}}{2} < 0$ ，函式 $\Omega = \arccos(-\frac{\sqrt{2}}{2}) = 3\pi/4$ ； $t = 0.5$ 得到中間值約在 135° 附近，而正確短弧應在 45° 。

習題 4。

```
def pose_at(q_list, times, t):
    """q_list、times 皆為序列，times 單調遞增。t 在區間外時夾取至相應端點姿態，不作外插。"""
    if t <= times[0]:
        return q_list[0]
    if t >= times[-1]:
        return q_list[-1]
    i = 0
    while i + 2 < len(times) and times[i + 1] <= t:
        i += 1
    t0, t1 = times[i], times[i + 1]
    u = (t - t0) / (t1 - t0) if t1 > t0 else 0.0
    return q_slerp(q_list[i], q_list[i + 1], u)
```

驗證 1（關鍵影格還原）：對每個 i ，`pose_at(q_list, times, times[i])` 應與 q_i 相等或差一符號。實作檢查 `min(||q(t_i)-q_i||, ||q(t_i)+q_i||) < 1e-9`，其中範數用四元數的歐氏長度。

驗證 2（無翻滾突變，輔助檢查）：生成 yaw 從 -90° 到 $+90^\circ$ 的關鍵影格， $N = 9$ 個等時距；以 t 掃描 $N \cdot 100$ 步，對相鄰樣本計算 $|\text{dot}(q_k, q_{k+1})|$ ，檢查值 > 0.99 。此檢查能偵測姿態序列內的**突然跳變**，是資料品質的輔助訊號。

需要強調：**密集取樣的相鄰內積無法可靠辨識長弧與短弧**。若實作走的是平滑長弧，相鄰樣本依然非常接近， $|\text{dot}|$ 仍接近 1；取樣密度越高，這個測試越看不出差別。

驗證 3（明確檢查中點或四分點）：要真的區分長弧與短弧，必須在**已知兩端**的情況下檢查特定參數位置的旋轉方向與相對角度。例如給定 $q_0 = (1, 0, 0, 0)$ 、 q_1 為繞 $+Z$ 軸旋轉 90° ，於 $t = 0.5$ 應得到繞 $+Z$ 軸旋轉 45° 的姿態（四元數的 w 分量約 $\cos(\pi/8) \approx 0.9239$ ）。若同一 q_0 搭配 $q_1 = (-\frac{\sqrt{2}}{2}, 0, 0, -\frac{\sqrt{2}}{2})$ （與前例代表**同一**旋轉，只是 S^3 上的對徑表示），正確的短弧調整後中點亦應得到同一 $w \approx 0.9239$ ；若無短弧調整，中點會走長弧， w 會落在完全不同的位置。反之，若誤寫成 $q_1 = (-\frac{\sqrt{2}}{2}, 0, 0, +\frac{\sqrt{2}}{2})$ ，該四元數代表繞 Z 軸 -90° 的旋轉，與前例不同；其短弧中點應對應 -45° ， w 仍為 $\cos(\pi/8)$ ，但旋轉方向相反。檢驗時務必確認所用四元數的末分量符號與軸向量符號一致。驗證應鎖定這類具體點，並搭配整體角度線性檢查。

驗證 2 與驗證 3 各司其職：前者偵測資料是否出現不連續；後者驗證短弧調整是否確實生效。

Euler 線性插值的問題：若直接對 yaw 角度 $-90^\circ \rightarrow +90^\circ$ 線性插值，視覺上通過 0° 且以此方式擺動似乎正確；但這是因為 yaw 只有一個分量變化。當關鍵影格同時改變 yaw 與 pitch（例如魚頭同時轉向與俯仰），Euler 三元組的獨立線性插值在旋轉流形上不是測地線，會出現中間姿態偏離、在 pitch 接近 $\pm 90^\circ$ 時出現萬向鎖抖動。四元數 SLERP 沿 S^3 的測地線走，配合短弧調整，對任意兩姿態都走一致的最短路徑，不受 Euler 次序與 gimbal lock 影響。

本章小結

- 單位四元數 $q = (\cos \frac{\theta}{2}, \sin \frac{\theta}{2} \hat{n})$ 表示繞軸 $\hat{n}$ 旋轉角度 θ ； (w, x, y, z) 為固定順序。
- Hamilton 乘法對應旋轉複合；非交換， $q_a q_b$ 表示「先 q_b 後 q_a 」對向量作用。
- 主動旋轉公式 $v' = q(0, v)q^*$ ；共軛 q^* 為逆旋轉。
- **雙覆蓋**： q 與 $-q$ 為同一旋轉。插值前以「內積符號翻轉」取最短弧。
- **SLERP** 沿 S^3 大圓弧插值，速度等角； $d \rightarrow 1$ 時 $\sin \Omega \rightarrow 0$ ，改走 nlerp。 $d = 0$ 不是奇異點。
- Euler 角仍有其位置（單一姿態輸入、單軸擺動），但插值路徑與複合應交由四元數處理。
- 養殖動畫中，魚體與魚鰭的姿態、關鍵影格插值、骨架蒙皮的局部旋轉，都以四元數為自然表示；所有合成動畫與姿態樣本僅為技術示範，不代表生物行為或物理準確性。

參考來源

- G1：PBRT 4 *Transformations*，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- G2：PBRT 4 *Reflection Models*，https://pbr-book.org/4ed/Reflection_Models
- G3：PBRT 4 *The Light Transport Equation*，https://pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation
- G4：*Ray Tracing in One Weekend*，<https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- G5：LearnOpenGL *Transformations*，<https://learnopengl.com/Getting-started/Transformations>
- G6：Blender Manual *Skinning Introduction*，<https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>
- G7：NumPy 線性代數參考，<https://numpy.org/doc/stable/reference/routines.linalg.html>
- G8：Khronos glTF 2.0 規格，<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

上述來源用於建構本章主題知識；本章文字與程式為自行撰寫，未逐字重製，亦未執行官方範例。SciPy 的 SLERP 相關文件查核未完成，故未列為來源；四元數乘法、SLERP 公式與雙覆蓋論述在本章以自足推導呈現，讀者可用 URL 回查以驗證等價性。

第 27 章 骨架、綁定姿勢與蒙皮

學習目標與先備知識

骨架動畫把複雜網格的變形控制濃縮成少量骨骼變換。本章以兩骨骼魚尾為例，建立從骨骼階層、綁定姿勢到線性混合蒙皮的完整資料流。

完成本章後，讀者應能：

1. 區分網格空間、骨骼局部空間與骨架全域空間。
2. 由父子階層計算骨骼全域矩陣。
3. 推導 inverse bind matrix 的作用。
4. 實作線性混合蒙皮（Linear Blend Skinning, LBS）。
5. 驗證綁定姿勢能還原原始頂點。
6. 檢查權重非負且總和為 1。
7. 拒絕奇異或過度病態的綁定矩陣。
8. 解釋 LBS 的糖紙效應與體積損失。
9. 知道雙四元數蒙皮的用途與限制，但不把它當成本章核心實作。

先備知識為齊次座標、 4×4 仿射矩陣、場景階層，以及旋轉矩陣或單位四元數的基本概念。本章使用 column vector，變換由右向左作用；長度單位為公尺，角度計算使用弧度。

問題與直覺

若逐一設定魚尾網格上數百個頂點的位置，動畫難以維護。骨架動畫改用少量骨骼描述大尺度運動，再令每個頂點由附近骨骼共同控制。

一個頂點可能有如下權重：

- 軀幹骨骼： $w_0 = 0.25$ ；
- 尾部骨骼： $w_1 = 0.75$ 。

尾部骨骼旋轉時，該頂點主要跟隨尾部，但仍受軀幹影響，於關節附近形成平滑過渡。

困難在於：頂點原本儲存在網格空間，而骨骼旋轉通常定義於自身局部空間。若直接把骨骼目前的全域矩陣乘到頂點，綁定姿勢中的關節偏移會被重複套用。正確流程必須先把頂點從網格綁定空間帶回骨骼的綁定局部空間，再帶到骨骼目前的姿勢。

數學與幾何推導

1. 骨骼階層與空間

令骨骼 j 的父骨骼索引為 $\pi(j)$ 。根骨骼沒有父節點，記為 $\pi(j) = -1$ 。

令 $L_j(t)$ 為時間 t 時，從骨骼 j 的局部座標轉到父骨骼座標的**完整局部矩陣**。它可以同時包含：

- 關節相對父骨骼的靜態偏移；
- 目前局部旋轉；
- 必要時的局部尺度。

其全域矩陣 $G_j(t)$ 把骨骼局部座標轉到骨架所在的網格空間：

$$G_j(t) = \begin{cases} L_j(t), & \pi(j) = -1, \\ G_{\pi(j)}(t)L_j(t), & \pi(j) \neq -1. \end{cases}$$

這符合全書的階層慣例：

$$M_{\text{world}} = M_{\text{parent}}M_{\text{local}}.$$

例如尾骨關節相對父骨骼位於 $(1, 0, 0)$ ，且目前繞自身局部原點旋轉 θ 。若沒有其他尺度或預旋轉，其完整局部矩陣是

$$L_{\text{tail}}(t) = T(1, 0, 0)R_z(\theta).$$

這不表示「局部旋轉本身包含平移」，而是局部到父空間的完整映射同時包含關節偏移與動畫旋轉。對 column vector，右側的 R_z 先作用，故骨骼局部原點仍被 $T(1, 0, 0)$ 放在父空間的關節位置。

若只令

$$L_{\text{tail}}(t) = R_z(\theta),$$

關節靜態偏移便會消失，除非動畫系統另有明確的 rest-offset 組合步驟。兩種表示皆可，但不可混用。

本章假設骨架與頂點的綁定座標位於同一網格空間。若整個魚模型另有場景節點矩陣 M_{mesh} ，應在蒙皮完成後才套用：

$$\mathbf{p}_{\text{world}} = M_{\text{mesh}}\mathbf{p}_{\text{skinned}}.$$

不要同時把 M_{mesh} 烘入骨骼矩陣又在場景圖套用一次。

2. 綁定姿勢與 inverse bind

綁定姿勢是建立權重時網格與骨架的參考姿勢。令骨骼 j 在綁定姿勢中的全域矩陣為

$$G_j^{(0)}.$$

它把骨骼 j 的局部綁定座標轉到網格空間。其 inverse bind matrix 為

$$B_j = \left(G_j^{(0)}\right)^{-1}.$$

B_j 的方向是

網格綁定空間 $\longrightarrow$ 骨骼 j 的綁定局部空間。

骨骼在目前姿勢下的蒙皮矩陣為

$$S_j(t) = G_j(t)B_j.$$

對綁定空間頂點 $\mathbf{p}_h = (x, y, z, 1)^T$ ， B_j 先將它帶入骨骼 j 的綁定局部空間， $G_j(t)$ 再把它帶回目前姿勢的網格空間。

3. 可逆性與病態矩陣

inverse bind 存在的數學必要條件是 $G_j^{(0)}$ 可逆。對仿射矩陣

$$G = \begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix},$$

可逆性取決於左上角線性部分 $A \in \mathbb{R}^{3 \times 3}$ 是否可逆。若某一軸尺度恰為零，則 A 奇異，inverse bind 不存在。

浮點運算還要區分「數學上可逆」與「數值上可靠」。令 A 的奇異值為

$$\sigma_{\max} \geq \sigma_{\text{mid}} \geq \sigma_{\min} \geq 0.$$

定義相對最小奇異值

$$r = \frac{\sigma_{\min}}{\sigma_{\max}},$$

前提是 $\sigma_{\max} > 0$ 。若

$$r \leq \tau_{\text{svd}},$$

便把矩陣視為過度病態而拒絕。本章示例使用

$$\tau_{\text{svd}} = 10^{-12}.$$

這是工程數值門檻，不是唯一通用值，也不是物理安全閾值。它具有整體尺度不變性：若 A 的所有尺度同乘相同正數，奇異值比值不變；但極端非均勻尺度會使 r 很小。

只呼叫 `np.linalg.inv` 並等待 `LinAlgError`，不保證能辨識所有近奇異矩陣。某些病態矩陣仍會回傳數值很大的反矩陣，因此應先檢查奇異值比值，再求反矩陣。

4. 綁定姿勢還原

當目前姿勢就是綁定姿勢時，

$$G_j(t) = G_j^{(0)}.$$

因此

$$S_j(t) = G_j^{(0)} \left(G_j^{(0)} \right)^{-1} = I.$$

若頂點權重總和為 1，LBS 結果為

$$\mathbf{p}'_h = \sum_j w_j S_j \mathbf{p}_h = \sum_j w_j \mathbf{p}_h = \mathbf{p}_h.$$

這是最重要的單元測試：**綁定姿勢必須還原原始網格**。

若權重總和不是 1，即使每個 $S_j = I$ ，也會得到

$$\mathbf{p}'_h = \left(\sum_j w_j \right) \mathbf{p}_h,$$

造成位置縮放，且齊次分量不再是 1。

5. 線性混合蒙皮

令頂點 i 的綁定位置為 $\mathbf{p}_i$ ，對骨骼 j 的權重為 w_{ij} 。LBS 定義為

$$\mathbf{p}'_i(t) = \sum_{j=0}^{m-1} w_{ij} S_j(t) \mathbf{p}_i.$$

權重通常要求

$$w_{ij} \geq 0, \quad \sum_{j=0}^{m-1} w_{ij} = 1.$$

實務上只保存每個頂點最重要的少數骨骼索引與權重，例如四組。刪除小權重後必須重新正規化：

$$\hat{w}_{ij} = \frac{w_{ij}}{\sum_k w_{ik}},$$

前提是分母大於零。零總權重頂點沒有明確控制者，不應靜默接受。

LBS 是變換後位置的仿射混合，不等於對旋轉角度做線性插值，也不保證混合矩陣仍為剛體變換。

6. 法線與切向

位置使用齊次分量 $w = 1$ ；方向使用 $w = 0$ 。但法線不能在一般非均勻縮放下直接套用位置矩陣。

以下公式只是**忽略權重空間梯度的常見局部近似**，不是一般蒙皮表面的精確法線公式。令頂點 i 的混合線性部分為

$$A_i = \sum_j w_{ij} A_j,$$

其中 A_j 是 S_j 左上角的 3×3 部分。若把頂點鄰域近似為單一仿射變形，可用

$$\mathbf{n}'_i = \frac{A_i^{-T} \mathbf{n}_i}{\|A_i^{-T} \mathbf{n}_i\|}$$

處理法線，但要求 A_i 可逆。

一般網格的蒙皮權重會隨表面位置變化。精確變形 Jacobian 還包含權重梯度，所以 A_i^{-T} 不應被描述成普遍精確答案。即時著色常見做法包括：

- 混合各骨骼變換後的法線，再重新正規化；
- 對混合位置矩陣採逆轉置近似；
- 由變形後位置重新計算幾何法線；
- 使用針對特定蒙皮方法設計的法線處理。

7. 糖紙效應

LBS 對變換結果做線性混合。考慮方向向量

$$\mathbf{v} = (1, 0, 0)^T.$$

若兩骨骼分別旋轉 $+90^\circ$ 與 -90° ，且權重各為 $1/2$ ，則

$$R_z(90^\circ)\mathbf{v} = (0, 1, 0)^T,$$

$$R_z(-90^\circ)\mathbf{v} = (0, -1, 0)^T.$$

混合後

$$\frac{1}{2}(0, 1, 0)^T + \frac{1}{2}(0, -1, 0)^T = (0, 0, 0)^T.$$

截面方向完全塌縮。這種在扭轉區域縮細或凹陷的現象稱為**糖紙效應**。它不是浮點誤差，而是線性混合剛體變換的模型限制。

雙四元數蒙皮可更好地保留旋轉與體積，尤其適合扭轉，但尺度、剪切、反射及雙四元數符號一致性需要額外處理。本章只把它列為延伸方向。

逐步手算例題

例題一：單一尾骨繞關節旋轉

令根骨骼的綁定全域矩陣為

$$G_0^{(0)} = I.$$

尾骨關節位於 $(1, 0, 0)$ ，其綁定全域矩陣為

$$G_1^{(0)} = T(1, 0, 0).$$

因此

$$B_1 = \left(G_1^{(0)}\right)^{-1} = T(-1, 0, 0).$$

現在尾骨繞自身原點旋轉 90° 。尾骨局部到父空間的完整矩陣為

$$L_1 = T(1, 0, 0)R_z(90^\circ).$$

由於本例根骨為單位矩陣，

$$G_1 = G_0 L_1 = T(1, 0, 0)R_z(90^\circ).$$

所以蒙皮矩陣為

$$S_1 = T(1, 0, 0)R_z(90^\circ)T(-1, 0, 0).$$

對頂點 $\mathbf{p} = (2, 0, 0)$ ，先移到關節局部空間：

$$T(-1, 0, 0)\mathbf{p} = (1, 0, 0).$$

旋轉後：

$$R_z(90^\circ)(1, 0, 0)^T = (0, 1, 0)^T.$$

再移回網格空間：

$$S_1\mathbf{p} = (1, 1, 0).$$

因此尾骨不是繞世界原點旋轉，而是繞 $(1, 0, 0)$ 的關節旋轉。

例題二：兩骨骼權重混合

沿用上一例。根骨蒙皮矩陣為 $S_0 = I$ ，頂點 $\mathbf{p} = (2, 0, 0)$ 的權重為

$$w_0 = 0.25, \quad w_1 = 0.75.$$

根骨結果為 $(2, 0, 0)$ ，尾骨結果為 $(1, 1, 0)$ ，所以

$$\mathbf{p}' = 0.25(2, 0, 0) + 0.75(1, 1, 0) = (1.25, 0.75, 0).$$

若改回綁定姿勢，則 $S_0 = S_1 = I$ ，所以

$$\mathbf{p}' = (0.25 + 0.75)\mathbf{p} = \mathbf{p}.$$

例題三：非單位父骨骼

假設根骨目前再平移 $(0, 2, 0)$ ：

$$G_0 = T(0, 2, 0).$$

尾骨局部矩陣仍是

$$L_1 = T(1, 0, 0)R_z(90^\circ).$$

依階層規則，

$$G_1 = G_0L_1 = T(0, 2, 0)T(1, 0, 0)R_z(90^\circ).$$

對頂點 $(2, 0, 0)$ ，根骨分支給出 $(2, 2, 0)$ ，尾骨分支給出 $(1, 3, 0)$ 。權重為 $(0.25, 0.75)$ 時，

$$\mathbf{p}' = 0.25(2, 2, 0) + 0.75(1, 3, 0) = (1.25, 2.75, 0).$$

根骨的純平移同時出現在兩個分支中。由於權重總和為 1，最後結果等於原本變形位置再加上 (0, 2, 0)。

例題四：近奇異綁定尺度

考慮線性部分

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 10^{-15} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

其奇異值為

$$1, \quad 1, \quad 10^{-15}.$$

因此

$$r = \frac{\sigma_{\min}}{\sigma_{\max}} = 10^{-15}.$$

若 $\tau_{\text{svd}} = 10^{-12}$ ，則

$$10^{-15} \leq 10^{-12},$$

此矩陣雖可能被浮點反矩陣函式接受，仍應由本章的工程判準拒絕。其 y 軸 inverse scale 會達到 10^{15} ，很容易放大誤差。

實作與程式

以下程式建立兩骨骼魚尾，計算 inverse bind、執行 LBS，並測試綁定姿勢、非單位父變換、錯誤權重與近奇異綁定矩陣。程式只依賴 NumPy，不讀檔、不使用 GPU。

```
import numpy as np

def translation(x, y, z):
    M = np.eye(4, dtype=float)
    M[:3, 3] = [x, y, z]
    return M

def scaling(x, y, z):
    return np.diag([x, y, z, 1.0]).astype(float)

def rotation_z(theta):
```

```

c = np.cos(theta)
s = np.sin(theta)
return np.array([
    [c, -s, 0.0, 0.0],
    [s, c, 0.0, 0.0],
    [0.0, 0.0, 1.0, 0.0],
    [0.0, 0.0, 0.0, 1.0],
], dtype=float)

def global_matrices(parents, local_matrices):
    """
    local_matrices[j] 是骨骼 j 的完整「局部到父空間」
    矩陣，可包含靜態關節偏移與目前局部旋轉。
    """
    local_matrices = np.asarray(local_matrices, dtype=float)
    count = len(parents)

    if local_matrices.shape != (count, 4, 4):
        raise ValueError(
            "local_matrices 形狀必須是 (J, 4, 4)"
        )
    if not np.all(np.isfinite(local_matrices)):
        raise ValueError("骨骼矩陣不可含 NaN 或無限值")

    globals_out = np.empty_like(local_matrices)

    for j, parent in enumerate(parents):
        if parent == -1:
            globals_out[j] = local_matrices[j]
        elif 0 <= parent < j:
            globals_out[j] = (
                globals_out[parent] @ local_matrices[j]
            )
        else:
            raise ValueError(
                "父索引必須為 -1 或小於子索引"
            )

    return globals_out

def inverse_bind_matrices(
    bind_globals,
    relative_singular_tol=1e-12,
    affine_tol=1e-12
):
    """
    拒絕非仿射、奇異或相對最小奇異值過小的綁定矩陣。
    relative_singular_tol 是無因次工程門檻。
    """
    bind_globals = np.asarray(bind_globals, dtype=float)

    if bind_globals.ndim != 3:
        raise ValueError("bind_globals 必須是三維陣列")
    if bind_globals.shape[1:] != (4, 4):
        raise ValueError("bind_globals 形狀必須是 (J, 4, 4)")
    if not np.isfinite(relative_singular_tol):
        raise ValueError("奇異值門檻必須為有限值")

```

```

if not 0.0 <= relative_singular_tol < 1.0:
    raise ValueError("奇異值門檻必須位於 [0, 1)")
if not np.isfinite(affine_tol) or affine_tol < 0.0:
    raise ValueError("affine_tol 必須是有限非負數")

result = np.empty_like(bind_globals)
affine_row = np.array([0.0, 0.0, 0.0, 1.0])

for j, G in enumerate(bind_globals):
    if not np.all(np.isfinite(G)):
        raise ValueError(
            f"骨骼 {j} 的綁定矩陣含非有限值"
        )
    if not np.allclose(
        G[3], affine_row,
        atol=affine_tol, rtol=0.0
    ):
        raise ValueError(
            f"骨骼 {j} 的矩陣不是本章使用的仿射形式"
        )

    singular_values = np.linalg.svd(
        G[:3, :3], compute_uv=False
    )
    sigma_max = float(singular_values[0])
    sigma_min = float(singular_values[-1])

    if sigma_max == 0.0:
        raise ValueError(
            f"骨骼 {j} 的綁定矩陣線性部分為零"
        )

    relative_sigma = sigma_min / sigma_max
    if relative_sigma <= relative_singular_tol:
        raise ValueError(
            f"骨骼 {j} 的綁定矩陣奇異或過度病態"
        )

    try:
        result[j] = np.linalg.inv(G)
    except np.linalg.LinAlgError as exc:
        raise ValueError(
            f"骨骼 {j} 的綁定矩陣不可逆"
        ) from exc

return result

def validate_weights(weights, bone_count, atol=1e-12):
    if weights.ndim != 2 or weights.shape[1] != bone_count:
        raise ValueError("weights 形狀必須是 (N, J)")
    if not np.all(np.isfinite(weights)):
        raise ValueError("權重不可含 NaN 或無限值")
    if np.any(weights < -atol):
        raise ValueError("權重不可為負")

    sums = np.sum(weights, axis=1)
    if not np.allclose(
        sums, 1.0, atol=atol, rtol=0.0

```

```

):
    raise ValueError(
        "每個頂點的權重總和必須為 1"
    )

def linear_blend_skinning(vertices, weights,
                          pose_globals, inverse_bind):
    vertices = np.asarray(vertices, dtype=float)
    weights = np.asarray(weights, dtype=float)
    pose_globals = np.asarray(pose_globals, dtype=float)
    inverse_bind = np.asarray(inverse_bind, dtype=float)

    if vertices.ndim != 2 or vertices.shape[1] != 3:
        raise ValueError("vertices 形狀必須是 (N, 3)")
    if not np.all(np.isfinite(vertices)):
        raise ValueError("頂點不可含 NaN 或無限值")

    bone_count = len(pose_globals)
    if pose_globals.shape != (bone_count, 4, 4):
        raise ValueError("pose_globals 形狀錯誤")
    if inverse_bind.shape != (bone_count, 4, 4):
        raise ValueError("inverse_bind 形狀錯誤")
    if not np.all(np.isfinite(pose_globals)):
        raise ValueError("姿勢矩陣含非有限值")
    if not np.all(np.isfinite(inverse_bind)):
        raise ValueError("inverse bind 含非有限值")
    if len(weights) != len(vertices):
        raise ValueError("頂點數與權重列數不一致")

    validate_weights(weights, bone_count)

    # NumPy 的批次 @ 在最後兩個維度執行矩陣乘法。
    skin = pose_globals @ inverse_bind

    vertices_h = np.column_stack([
        vertices,
        np.ones(len(vertices), dtype=float)
    ])
    output_h = np.zeros((len(vertices), 4), dtype=float)

    for i, p_h in enumerate(vertices_h):
        for j in range(bone_count):
            output_h[i] += (
                weights[i, j] * (skin[j] @ p_h)
            )

    if not np.allclose(
        output_h[:, 3], 1.0, atol=1e-12, rtol=0.0
    ):
        raise ValueError("蒙皮後齊次分量不是 1")

    return output_h[:, :3]

def make_two_bone_tail():
    # 簡化魚尾中心線與尾端上下兩點，單位：公尺。
    vertices = np.array([
        [0.0, 0.0, 0.0],

```



```

        [1.0, 0.0, 0.0],
        [1.5, 0.0, 0.0],
        [2.0, 0.0, 0.0],
        [2.0, 0.2, 0.0],
        [2.0, -0.2, 0.0],
    ], dtype=float)

    # 欄 0 為根骨，欄 1 為尾骨。
    weights = np.array([
        [1.00, 0.00],
        [0.80, 0.20],
        [0.50, 0.50],
        [0.25, 0.75],
        [0.00, 1.00],
        [0.00, 1.00],
    ], dtype=float)

    parents = [-1, 0]

    # 綁定時尾骨局部原點位於父空間 x=1。
    bind_local = np.stack([
        np.eye(4),
        translation(1.0, 0.0, 0.0),
    ])

    return vertices, weights, parents, bind_local

if __name__ == "__main__":
    vertices, weights, parents, bind_local = (
        make_two_bone_tail()
    )

    bind_global = global_matrices(
        parents, bind_local
    )
    inverse_bind = inverse_bind_matrices(
        bind_global
    )

    # 測試一：綁定姿勢必須還原原始頂點。
    restored = linear_blend_skinning(
        vertices, weights,
        bind_global, inverse_bind
    )
    assert np.allclose(
        restored, vertices, atol=1e-12
    )

    # 測試二：尾骨完整局部矩陣為 T_joint @ R_local。
    pose_local = np.stack([
        np.eye(4),
        translation(1.0, 0.0, 0.0)
        @ rotation_z(np.pi / 2.0),
    ])
    pose_global = global_matrices(
        parents, pose_local
    )

```

```

deformed = linear_blend_skinning(
    vertices, weights,
    pose_global, inverse_bind
)

expected = np.array([
    [0.00, 0.00, 0.0],
    [1.00, 0.00, 0.0],
    [1.25, 0.25, 0.0],
    [1.25, 0.75, 0.0],
    [0.80, 1.00, 0.0],
    [1.20, 1.00, 0.0],
])
assert np.allclose(
    deformed, expected, atol=1e-12
)

# 測試三：非單位根骨；所有結果繼承 +Y 方向 2 m。
parent_pose_local = np.stack([
    translation(0.0, 2.0, 0.0),
    translation(1.0, 0.0, 0.0)
    @ rotation_z(np.pi / 2.0),
])
parent_pose_global = global_matrices(
    parents, parent_pose_local
)

deformed_with_parent = linear_blend_skinning(
    vertices, weights,
    parent_pose_global, inverse_bind
)
expected_with_parent = (
    expected + np.array([0.0, 2.0, 0.0])
)
assert np.allclose(
    deformed_with_parent,
    expected_with_parent,
    atol=1e-12
)
assert np.allclose(
    parent_pose_global[1],
    parent_pose_global[0] @ parent_pose_local[1],
    atol=1e-12
)

# 測試四：錯誤權重應被拒絕。
bad_weights = weights.copy()
bad_weights[2] = [0.5, 0.4]

try:
    linear_blend_skinning(
        vertices, bad_weights,
        pose_global, inverse_bind
    )
except ValueError:
    pass
else:
    raise AssertionError("未拒絕錯誤權重")

```

```
# 測試五：近奇異綁定矩陣應被工程門檻拒絕。
bad_bind_global = bind_global.copy()
bad_bind_global[1] = (
    translation(1.0, 0.0, 0.0)
    @ scaling(1.0, 1e-15, 1.0)
)

try:
    inverse_bind_matrices(
        bad_bind_global,
        relative_singular_tol=1e-12
    )
except ValueError:
    pass
else:
    raise AssertionError("未拒絕近奇異綁定矩陣")

print("bind pose restored:", restored)
print("deformed:", deformed)
print("deformed with parent:", deformed_with_parent)
```

測試與預期結果

上述程式未在此處執行。依矩陣推導，**預期**五組測試皆通過。

綁定姿勢測試

對每根骨骼，

$$G_j^{(0)} B_j = I.$$

因此 `restored` 預期等於原始 `vertices`，容差為 10^{-12} 。

尾骨旋轉測試

預期變形位置為：

```
[[0.00, 0.00, 0.0],
 [1.00, 0.00, 0.0],
 [1.25, 0.25, 0.0],
 [1.25, 0.75, 0.0],
 [0.80, 1.00, 0.0],
 [1.20, 1.00, 0.0]]
```

尾端索引 4 的原始位置為 (2, 0.2, 0)。相對關節的向量 (1, 0.2, 0) 經右手 +90° 旋轉後為 (-0.2, 1, 0)，加回關節得到 (0.8, 1, 0)。

尾端索引 5 的原始位置為 $(2, -0.2, 0)$ 。相對向量 $(1, -0.2, 0)$ 旋轉後為 $(0.2, 1, 0)$ ，加回關節得到 $(1.2, 1, 0)$ 。

非單位父變換測試

根骨目前矩陣為 $T(0, 2, 0)$ 。因為子骨骼全域矩陣使用

$$G_1 = G_0 L_1,$$

兩個骨骼分支都繼承同一根骨平移。由於每個頂點的權重總和為 1，最終結果等於前一組結果增加 $(0, 2, 0)$ 。例如索引 3 由

$$(1.25, 0.75, 0)$$

變成

$$(1.25, 2.75, 0).$$

錯誤權重測試

`bad_weights[2]` 的權重總和為

$$0.5 + 0.4 = 0.9,$$

因此預期拋出 `ValueError`。程式不會自動正規化，因為自動修復可能掩蓋資產匯出錯誤。

近奇異矩陣測試

測試矩陣的線性尺度為

$$(1, 10^{-15}, 1).$$

其相對最小奇異值為 10^{-15} ，低於門檻 10^{-12} ，因此預期拋出 `ValueError`。這項測試不依賴 `np.linalg.inv` 是否碰巧成功。

除錯與常見陷阱

忘記 inverse bind

若直接使用 $G_j(t)\mathbf{p}_h$ ，尾骨綁定時已有的關節偏移會再次作用。常見症狀是網格在第一幀就跳離原位。

正確矩陣為

$$S_j(t) = G_j(t) \left(G_j^{(0)} \right)^{-1}.$$

把 inverse bind 乘在錯誤一側

column vector 慣例下應寫成

$$G_j(t) B_j \mathbf{p}_h,$$

不是 $B_j G_j(t) \mathbf{p}_h$ 。矩陣一般不可交換。

只依賴反矩陣函式偵測奇異

`np.linalg.inv` 可拒絕部分奇異矩陣，但近奇異矩陣可能仍回傳巨大且不可靠的反矩陣。若工作流程承諾拒絕病態綁定，應明確指定奇異值或條件數門檻，並保存該門檻以便重現。

相對奇異值門檻也不是絕對真理。門檻過大可能拒絕有意使用的強烈非均勻尺度；門檻過小則可能保留會放大誤差的資產。

誤解局部姿勢矩陣

本章的 `pose_local[j]` 是局部到父空間的完整矩陣，不是只含動畫旋轉。尾骨使用

$$T_{\text{joint}} R_{\text{local}}$$

以保留關節偏移。如果引擎把 rest offset 與動畫旋轉分開保存，也必須在形成完整局部矩陣時明確組合。

混淆局部矩陣與全域矩陣

子骨骼局部矩陣只描述相對父骨骼的變換。必須沿階層累乘：

$$G_{\text{child}} = G_{\text{parent}} L_{\text{child}}.$$

只用單位根骨測試，可能無法發現遺漏父變換的錯誤。

權重總和錯誤

權重總和不為 1 時，綁定姿勢也無法還原。負權重雖可出現在某些廣義變形方法中，但標準角色蒙皮通常要求非負；本章程式直接拒絕。

重複套用網格節點變換

若骨骼全域矩陣已在網格空間，蒙皮後再套一次場景節點矩陣即可。不要把同一平移或縮放同時放進骨架與網格節點。

把 LBS 當成剛體插值

$$\sum_j w_j R_j$$

通常不是正交矩陣，因此可能縮放、剪切或塌縮。糖紙效應不是提高浮點精度就能消除。

把近似法線公式當成精確結果

權重通常會沿表面變化，精確 Jacobian 包含權重梯度。對混合線性矩陣使用逆轉置只是常見近似之一。若需要高品質幾何法線，可由變形後三角形重新計算，再依平滑規則累積。

把動畫效果當成生物驗證

魚尾看似自然，只表示視覺結果符合某種動畫意圖，不代表尾部肌肉、流體阻力或游動效率已獲物理或生物驗證。

養殖數位分身案例

在合成養殖場景中，可用兩至數根骨骼建立低成本魚尾動畫：

1. 根骨控制軀幹後段。
2. 尾骨局部原點放在尾柄關節。
3. 關節前方頂點提高根骨權重。
4. 關節後方逐漸提高尾骨權重。
5. 尾鰭頂點主要由尾骨控制。

可令尾骨角度為

$$\theta(t) = \theta_{\max} \sin(2\pi ft + \phi),$$

其中 $\theta_{\max}$ 是合成擺幅， f 的單位為 s^{-1} ， ϕ 是相位。這只是動畫參數，不應解讀成真實魚種的生理量。

若多條魚共用同一網格與骨架，可以共享：

- 綁定頂點；
- 骨骼父子關係；

- inverse bind matrices ;
- 蒙皮權重；
- 綁定矩陣病態判定門檻。

每個個體只需保存自身場景變換、相位與目前骨骼姿勢。為確保可重現，資料清單應記錄角度單位、時間單位、骨骼順序、父索引、綁定矩陣、權重及動畫參數。

驗收至少包括：

- 綁定姿勢還原；
- 權重總和；
- inverse bind 的有限性與病態門檻；
- 關節點是否保持；
- 非單位根骨下的階層繼承；
- 左右擺動是否符合右手正角；
- 極端角度是否產生自交或明顯體積損失。

兩骨骼模型適合低面數遠景魚或演算法測試，不足以描述完整脊柱、肌肉或柔性鰭條。

習題

習題 1：手算

尾骨綁定全域矩陣為 $T(2, 0, 0)$ ，目前矩陣為

$$T(2, 0, 0)R_z(90^\circ).$$

求頂點 $\mathbf{p} = (3, 0, 0)$ 完全受尾骨控制時的變形位置。

習題 2：程式測試

把範例中頂點 $(1.5, 0, 0)$ 的權重從 $(0.5, 0.5)$ 改成 $(0.2, 0.8)$ 。尾骨旋轉 90° 時，推導其預期位置，並寫出適合加入程式的斷言。

習題 3：反例與除錯

某程式在動畫姿勢中使用

$$S_j = B_j G_j(t).$$

由於在綁定姿勢中兩種乘法都得到 I ，開發者認為次序無關。請指出錯誤並給出一般理由。

習題 4：整合應用

某魚尾頂點由三根骨骼控制，原始權重為 $(0.6, 0.3, 0.1)$ 。為節省儲存空間，只保留最大的兩項。

1. 直接刪除第三項後，權重總和是多少？
2. 正規化後的兩項權重是多少？
3. 為何應記錄這項裁剪操作？

習題 5：病態矩陣與糖紙效應

1. 某綁定矩陣線性部分的奇異值為 $(2, 1, 2 \times 10^{-14})$ 。若門檻為 10^{-12} ，是否接受？
 2. 某方向 $\mathbf{v} = (0, 1, 0)$ 由兩根骨骼以相等權重控制。兩骨分別繞 x 軸旋轉 $+90^\circ$ 與 -90° 。求 LBS 後方向。
-

習題解答

解答 1

inverse bind 為

$$B = T(-2, 0, 0).$$

蒙皮矩陣為

$$S = T(2, 0, 0)R_z(90^\circ)T(-2, 0, 0).$$

先把頂點移到關節局部空間：

$$(3, 0, 0) - (2, 0, 0) = (1, 0, 0).$$

旋轉後為 $(0, 1, 0)$ ，再加回關節位置：

$$\mathbf{p}' = (2, 1, 0).$$

解答 2

根骨結果仍為

$$(1.5, 0, 0).$$

尾骨結果為：相對關節 $(1, 0, 0)$ 的向量是 $(0.5, 0, 0)$ ，旋轉後為 $(0, 0.5, 0)$ ，所以

$$S_1 \mathbf{p} = (1, 0.5, 0).$$

混合後

$$\mathbf{p}' = 0.2(1.5, 0, 0) + 0.8(1, 0.5, 0) = (1.1, 0.4, 0).$$

若該點仍是陣列索引 2，可加入：

```
assert np.allclose(
    deformed[2], [1.1, 0.4, 0.0], atol=1e-12
)
```

同時必須修改 `expected` 中對應的一列。

解答 3

正確順序為

$$S_j = G_j(t)B_j.$$

B_j 先把網格空間頂點轉到骨骼綁定局部空間， $G_j(t)$ 再帶到目前網格空間。矩陣作用方向由最右側開始，因此不能顛倒。

在綁定姿勢下，因為

$$B_j = (G_j^{(0)})^{-1},$$

左右兩種乘法都會得到單位矩陣：

$$G_j^{(0)} B_j = I, \quad B_j G_j^{(0)} = I.$$

但動畫姿勢使用的是 $G_j(t)$ ，一般而言

$$G_j(t)B_j \neq B_jG_j(t).$$

尤其旋轉與平移通常不交換。因此只測綁定姿勢無法發現次序錯誤，還必須測試非平凡動畫姿勢。

解答 4

刪除 0.1 後，剩餘總和為

$$0.6 + 0.3 = 0.9.$$

正規化後：

$$\hat{w}_0 = \frac{0.6}{0.9} = \frac{2}{3}, \quad \hat{w}_1 = \frac{0.3}{0.9} = \frac{1}{3}.$$

裁剪會改變頂點運動，即使重新正規化也不等於原始三骨骼結果。記錄骨骼數上限、裁剪門檻與正規化方式，才能重現資產處理流程並比較誤差。

解答 5

第一小題的相對最小奇異值為

$$r = \frac{2 \times 10^{-14}}{2} = 10^{-14}.$$

因為

$$10^{-14} \leq 10^{-12},$$

所以依本章工程門檻拒絕。

第二小題中，

$$R_x(90^\circ)(0, 1, 0)^T = (0, 0, 1)^T,$$

$$R_x(-90^\circ)(0, 1, 0)^T = (0, 0, -1)^T.$$

相等權重混合得到

$$\frac{1}{2}(0, 0, 1)^T + \frac{1}{2}(0, 0, -1)^T = (0, 0, 0)^T.$$

方向完全塌縮，無法再正規化。這是 LBS 混合相反旋轉造成的糖紙效應，而非單純的數值容差問題。

本章小結

骨架蒙皮的核心不是只把頂點乘上目前骨骼矩陣，而是維持清楚的空間轉換：

$$\text{網格綁定空間} \xrightarrow{B_j} \text{骨骼綁定局部空間} \xrightarrow{G_j(t)} \text{目前網格空間}.$$

因此每根骨骼的蒙皮矩陣為

$$S_j(t) = G_j(t) \left(G_j^{(0)} \right)^{-1},$$

LBS 頂點為

$$\mathbf{p}'_i = \sum_j w_{ij} S_j(t) \mathbf{p}_i.$$

骨骼局部矩陣 $L_j(t)$ 是局部到父空間的完整映射，可包含靜態關節偏移與目前局部旋轉；全域矩陣必須依

$$G_j = G_{\pi(j)} L_j$$

沿階層累乘。

可靠實作必須驗證：綁定矩陣為仿射且數值可逆、父子階層正確、權重非負且總和為 1、綁定姿勢可還原、非單位父變換能正確繼承，以及網格節點變換沒有重複套用。

LBS 計算簡單且適用廣泛，但混合矩陣不一定保持剛性，極端扭轉可能造成糖紙效應。雙四元數蒙皮可作延伸方法，但不能省略空間、綁定姿勢與權重語意的正確定義。

參考來源

1. Blender Manual，〈Skinning Introduction〉：骨架變形與蒙皮工作流程的概念背景。
<https://docs.blender.org/manual/en/latest/animation/armatures/skinning/introduction.html>
2. Khronos Group，glTF 2.0 Specification：skin、joint、inverse bind matrix 與資產交換概念。
<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>
3. PBRT 4，〈Transformations〉：仿射矩陣、逆變換與座標空間背景。
https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
4. NumPy 線性代數參考：矩陣乘法、反矩陣、SVD 與數值陣列介面。
<https://numpy.org/doc/stable/reference/routines.linalg.html>

以上來源供延伸查閱；不表示本章每項蒙皮推導均由來源逐條驗證。glTF 規格在此只作概念橋接，實際交換資產時仍須依讀者採用的規格內容核對節點與座標空間。

第28章 逆向運動學與限制

學習目標與先備知識

本章以兩連桿平面關節鏈說明逆向運動學（inverse kinematics，IK）。完成後，你應能從關節角計算末端位置，以解析式及有限差分取得位置 Jacobian，使用阻尼最小平方更新關節角，並區分數值停止、關節受限與目標不可達。

先備知識是向量、矩陣與基本 Python。關節角以弧度計，連桿長度及位置以公尺計。平面例子放在本書右手世界系的 XY 平面，正角為繞 $+Z$ 的右手旋轉； $+Y$ 向上，與影像像素向下的座標不可混用。這是一個動畫用的幾何模型，不把關節角解讀為實際魚類的解剖量測。

問題與直覺

正向運動學（forward kinematics，FK）的問題是：「已知各關節角，尾端在哪裡？」IK 則倒過來問：「希望尾端到某處，各關節應轉多少？」例如要讓合成魚尾的控制點靠近一條動畫路徑，便可指定末端目標，再求魚身與魚尾兩個旋轉關節的角度。

倒過來求解並不等於把 FK 公式直接取逆。同一個末端位置可能有「向上彎」與「向下彎」兩組姿態；目標也可能超出連桿所能到達的範圍。加上關節角上下限後，即使目標落在未受限連桿的可達圓環內，也未必存在合法姿態。因此 IK 求解器應回報殘差與停止原因，而非只交出一組看似合理的角度。

數學與幾何推導

兩連桿 FK 與可達範圍

設根關節固定在 $\mathbf{b} = (b_x, b_y)^T$ ，兩段長度分別為 $L_1, L_2 > 0$ 。 θ_1 是第一段相對世界 $+X$ 的角度； θ_2 是第二段相對第一段的關節角。末端位置 $\mathbf{f}(\boldsymbol{\theta})$ 為

$$\mathbf{f}(\boldsymbol{\theta}) = \mathbf{b} + \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}, \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}.$$

這亦可由場景階層推出：先把第一段沿自身 $+X$ 延伸 L_1 ，再在其末端施加局部旋轉 θ_2 ，把第二段沿自己的 $+X$ 延伸 L_2 。採直向量時，子節點的世界變換為 $M_{\text{parent}}M_{\text{local}}$ 。

令目標為 $\mathbf{g}$ ，其離根部距離為 $r = \|\mathbf{g} - \mathbf{b}\|$ 。**不設關節限制時**，三角不等式給出可達位置的必要且充分距離條件：

$$|L_1 - L_2| \leq r \leq L_1 + L_2.$$

在邊界處，連桿伸直或折疊，姿態可能退化；加入角度界限後，這個圓環條件仍只是必要條件，不再保證目標可達。上述平面位置條件也不處理姿態朝向、碰撞或魚身變形。

位置 Jacobian：從小角度變化到位置變化

Jacobian 是 FK 的局部變化率矩陣。在目前角度附近作一個小更新 $\Delta\theta$ ，末端位置可近似為

$$\mathbf{f}(\theta + \Delta\theta) \approx \mathbf{f}(\theta) + J(\theta)\Delta\theta.$$

這裡用到的微積分只需理解為「角度稍微增加時，末端座標改變多少」。J 的第 j 欄是末端位置對第 j 個角度的變化率；位置單位為公尺、角度以弧度計，故其數值可視為每弧度的公尺位移。對 FK 中的正弦、餘弦逐項求導，得到 2×2 位置 Jacobian：

$$J = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}.$$

求解器未必能取得解析導數。以有限差分估算第 j 欄，可用中央差分

$$J_{:,j} \approx \frac{\mathbf{f}(\theta + h\mathbf{e}_j) - \mathbf{f}(\theta - h\mathbf{e}_j)}{2h},$$

其中 $\mathbf{e}_j$ 是第 j 個單位座標向量， $h > 0$ 是以弧度計的角度擾動。 h 太大會使近似粗糙；太小則可能讓浮點相減誤差顯著。有限差分是在目前角度附近估計 FK 的導數，不能把角度限制的裁切混入這兩次 FK 評估，否則算到的是裁切函數的變化率。

當兩段幾乎伸直或折疊時， J 可能失去秩：某些末端移動方向無法由當下的一階小角度變化產生。這稱為奇異姿態；它不表示所有其他角度的姿態均不可達。

阻尼最小平方與關節界限

令位置誤差為 $\mathbf{e} = \mathbf{g} - \mathbf{f}(\theta)$ 。若直接解 $J\Delta\theta = \mathbf{e}$ ，奇異或近奇異時更新可能不穩定。阻尼最小平方（damped least squares，DLS）改為最小化局部目標

$$\|J\Delta\theta - \mathbf{e}\|^2 + \lambda^2 \|\Delta\theta\|^2, \quad \lambda > 0.$$

第一項要求線性化後的位置接近目標；第二項抑制過大的角度更新。對更新量求此二次式的最小值，可得

$$\Delta\theta = J^T(JJ^T + \lambda^2 I)^{-1}\mathbf{e}.$$

也可等價地解 $(J^T J + \lambda^2 I)\Delta\theta = J^T \mathbf{e}$ 。實作宜用線性方程求解器，而非顯式建立逆矩陣。 λ 的選擇影響穩定性與收斂速度；它不是誤差容差，也不保證全域最佳解。

角度界限寫為 $\ell \leq \theta \leq \mathbf{u}$ 。簡單作法是更新後逐角裁切至區間內，再用**實際裁切後**的角度重新計算 FK。裁切方便教學，但不是嚴格的受限最佳化；更新方向可能一再撞上界限，甚至停在仍有殘差的姿態。為減少過大步伐，可對 $\Delta\theta$ 限制最大範數，並試用逐步縮小的步長，只接受真正令位置誤差下降的更新。

停止時至少要分清：已達位置容差、改善太小、找不到下降步、達到迭代上限，以及事先可判定的圓環外目標。即使回報「不可達」，也應附上最後末端位置及殘差；數值求解未成功，不總能證明受限問題在數學上無解。

逐步手算例題

例一：FK、Jacobian 與阻尼更新

取根部 $\mathbf{b} = (0, 0)^T$ 、 $L_1 = L_2 = 1$ 公尺、 $\theta_1 = 0$ 、 $\theta_2 = \pi/2$ 。第一段末端是 $(1, 0)^T$ ；第二段朝 $+Y$ ，所以 FK 末端為

$$\mathbf{f}(0, \pi/2) = (1, 1)^T.$$

代入解析導數得

$$J = \begin{bmatrix} -1 & -1 \\ 1 & 0 \end{bmatrix}.$$

若目標為 $\mathbf{g} = (0, 2)^T$ ，目前誤差 $\mathbf{e} = (-1, 1)^T$ 。取阻尼 $\lambda = 1$ ，則

$$JJ^T + \lambda^2 I = \begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix}, \quad (JJ^T + \lambda^2 I)^{-1} \mathbf{e} = \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix}.$$

因此 DLS 的**未限制更新**為

$$\Delta\theta = J^T \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.2 \end{bmatrix} \text{ 弧度}.$$

這只是以目前姿態線性化所得的一步，不代表更新後剛好到達 $(0, 2)^T$ ；實際位置須再用 FK 計算。若設定單步最大角度變化或關節界限，也可能只採用其中一部分。

例二：距離可達，不代表受限可達

令 $L_1 = 1$ 、 $L_2 = 0.5$ 公尺，目標離根部 $r = 1.2$ 公尺。由

$$|1 - 0.5| = 0.5 \leq 1.2 \leq 1.5 = 1 + 0.5$$

可知它在**未受限**可達圓環內。現在限制兩關節都只能取零角，則唯一末端是 $(1.5, 0)^T$ （根部在原點），不可能抵達 $(1.2, 0)^T$ 。故距離檢查不能取代受限求解。

反過來，若目標為 $(2, 0)^T$ ，則 $r = 2 > 1.5$ ；不論關節如何轉動都不可達。最接近目標的未受限伸直姿態末端為 $(1.5, 0)^T$ ，至少留下 0.5 公尺殘差。這是幾何下界，與求解器的迭代次數無關。

實作與程式

以下程式只依賴 NumPy，讀者可在既有環境執行；本書未執行。它使用有限差分 Jacobian、阻尼更新、角度界限、最大步長及回退搜尋。回退搜尋逐次縮短步長，只接受實際 FK 殘差下降；它不保證找到全域解。

```
import numpy as np

def fk(q, lengths, base=(0.0, 0.0)):
    q = np.asarray(q, dtype=float)
    L = np.asarray(lengths, dtype=float)
    b = np.asarray(base, dtype=float)
    if q.shape != (2,) or L.shape != (2,) or b.shape != (2,):
        raise ValueError("q、lengths、base 均須有兩個元素")
    if not (np.all(np.isfinite(q)) and np.all(np.isfinite(L))
            and np.all(np.isfinite(b))) or np.any(L <= 0):
        raise ValueError("輸入須有限，連桿長度須為正")
    a1, a2 = q[0], q[0] + q[1]
    return b + np.array([
        L[0] * np.cos(a1) + L[1] * np.cos(a2),
        L[0] * np.sin(a1) + L[1] * np.sin(a2),
    ])

def jacobian_fd(q, lengths, base=(0.0, 0.0), h=1e-5):
    if not np.isfinite(h) or h <= 0:
        raise ValueError("有限差分步長須為正")
    q = np.asarray(q, dtype=float)
    J = np.empty((2, 2), dtype=float)
    for j in range(2):
        step = np.zeros(2)
        step[j] = h
        J[:, j] = (fk(q + step, lengths, base)
                   - fk(q - step, lengths, base)) / (2 * h)
    return J

def solve_ik(target, q0, lengths, lower, upper, base=(0.0, 0.0),
             damping=0.1, tolerance=1e-5, max_iter=200,
             max_step=0.25, h=1e-5):
    target = np.asarray(target, dtype=float)
    q = np.asarray(q0, dtype=float).copy()
    L = np.asarray(lengths, dtype=float)
    lo = np.asarray(lower, dtype=float)
    hi = np.asarray(upper, dtype=float)
    base = np.asarray(base, dtype=float)

    if (target.shape != (2,) or q.shape != (2,)
        or L.shape != (2,) or lo.shape != (2,)
        or hi.shape != (2,) or base.shape != (2,)):
```

```

        raise ValueError("所有位置、角度及長度均須有兩個元素")
if not all(np.all(np.isfinite(x))
            for x in (target, q, L, lo, hi, base)):
    raise ValueError("輸入須為有限數值")
# 純量參數也要逐項檢查有限性與範圍。
# 若不檢查，damping=nan 之類的輸入可能通過 <= 比較而產生
# 非有限更新或線性代數例外。
if not np.isfinite(damping) or damping <= 0:
    raise ValueError("damping 必須為正的有限值")
if not np.isfinite(tolerance) or tolerance <= 0:
    raise ValueError("tolerance 必須為正的有限值")
if not np.isfinite(max_step) or max_step <= 0:
    raise ValueError("max_step 必須為正的有限值")
if not np.isfinite(h) or h <= 0:
    raise ValueError("h 必須為正的有限值")
if not isinstance(max_iter, (int, np.integer)) or max_iter < 1:
    raise ValueError("max_iter 必須為正整數")
if np.any(L <= 0) or np.any(lo > hi):
    raise ValueError("長度須為正或關節界限須遞增")

q = np.clip(q, lo, hi)
radius = np.linalg.norm(target - base)
outside_ring = (radius > L.sum() + tolerance
                or radius < abs(L[0] - L[1]) - tolerance)
status = "迭代上限"

for iteration in range(max_iter):
    tip = fk(q, L, base)
    error = target - tip
    distance = np.linalg.norm(error)
    if distance <= tolerance:
        status = "已達位置容差"
        break

    J = jacobian_fd(q, L, base, h)
    delta = J.T @ np.linalg.solve(
        J @ J.T + damping**2 * np.eye(2), error
    )
    delta_norm = np.linalg.norm(delta)
    if not np.isfinite(delta_norm) or delta_norm < 1e-12:
        status = "更新太小"
        break
    if delta_norm > max_step:
        delta *= max_step / delta_norm

    accepted = False
    for k in range(16):
        candidate = np.clip(q + (0.5**k) * delta, lo, hi)
        if np.linalg.norm(target - fk(candidate, L, base)) \
            < distance - 1e-12:
            q = candidate
            accepted = True
            break
    if not accepted:
        status = "找不到下降步"
        break
else:
    iteration = max_iter

```



```

tip = fk(q, L, base)
residual = float(np.linalg.norm(target - tip))
if outside_ring and residual > tolerance:
    status = "目標在未受限可達圓環外;" + status
return q, tip, residual, status, iteration + (iteration < max_iter)

# 合成測試；下列斷言及輸出均供讀者自行執行。
L = (1.0, 1.0)
q_test = np.array([0.0, np.pi / 2])
assert np.allclose(fk(q_test, L), [1.0, 1.0], atol=1e-12)

J_expected = np.array([[ -1.0, -1.0], [1.0, 0.0]])
assert np.allclose(jacobian_fd(q_test, L), J_expected, atol=1e-8)

q, tip, residual, status, steps = solve_ik(
    target=(1.0, 1.0),
    q0=(0.3, 0.8),
    lengths=L,
    lower=(-np.pi, -np.pi),
    upper=(np.pi, np.pi),
)
assert np.all(q >= [-np.pi, -np.pi])
assert np.all(q <= [np.pi, np.pi])
assert np.allclose(tip, fk(q, L))
assert residual < 1e-5

_, _, far_residual, far_status, _ = solve_ik(
    target=(3.0, 0.0),
    q0=(0.0, 0.0),
    lengths=L,
    lower=(-np.pi, -np.pi),
    upper=(np.pi, np.pi),
)
assert far_residual >= 1.0 - 1e-12
assert "圓環外" in far_status
print("預期：FK、有限差分、界限及不可達測試通過")

```

程式的 `iteration` 計數只描述嘗試過的迭代，不代表成功程度；應同時讀取 `status` 與以公尺計的 `residual`。`tolerance` 是動畫位置誤差容差，不是物理安全閾值。若在其他尺度的場景沿用本程式，應重新選擇距離容差、阻尼與停止閾值；角度有限差分步長則仍以弧度指定。

測試與預期結果

手算例一給出一個獨立基準： $\text{fk}([0, \pi/2], [1, 1])$ 預期接近 $(1, 1)^T$ ，有限差分 Jacobian 預期接近手算矩陣。讀者亦可嘗試多個非奇異角度，將有限差分結果與解析式逐元素比較；減半 h 時誤差不保證永遠下降，因為浮點相減誤差終會變重要。

對目標 $(1, 1)^T$ ，程式的受限求解斷言要求殘差小於 10^{-5} 公尺，且返回角度落在界限內；這是**預期測試**，不是已跑過的紀錄。對兩條各一公尺的連桿，目標 $(3, 0)^T$ 超出最遠兩公尺範圍，故任何姿態的殘差至少一公尺。最後還應測試把上下限都設為零而目標設為 $(1, 1)^T$ ：求解器不得宣稱達標，返回角度必須仍為零，殘差應保持非零。

除錯與常見陷阱

****方向轉反了。****先檢查角度是否以弧度傳給正弦、餘弦，以及正角是否繞 $+Z$ 轉向 $+Y$ 。相機影像的垂直像素方向向下，不能直接當本章世界 Y 軸使用。

****Jacobian 和 FK 不一致。****常見錯誤是第二段使用 θ_2 作世界角，而 FK 使用的是 $\theta_1 + \theta_2$ 。先以中央差分比較解析 Jacobian，再排查更新公式；有限差分本身也須避免把角度裁切混進擾動評估。

****在伸直姿態停住。****伸直時 Jacobian 對某些移動方向的一階反應為零。阻尼可防止不穩定的大更新，卻不能保證從任何奇異起點走向任何目標。可改用非奇異初始角、數個初始姿態分別求解，並如實報告每次結果，而不是聲稱已找到全域解。

****角度合法卻始終有殘差。****可能是界限使目標不可達，也可能是裁切後找不到下降步。檢查未受限圓環條件、最終角度是否貼住界限，以及不同初始角所得殘差；僅憑一次局部求解失敗，不能證明受限問題全域無解。

****動畫突然跳到另一姿態。****同一位置可對應不同關節配置。逐影格求解時，可以上一影格合法角度作初始值，另限制每步更新與檢查殘差；這有助於連續性，但不會自動處理碰撞或姿態美術需求。

養殖數位分身案例

在合成養殖池的魚群動畫中，可把一條簡化魚尾視為兩段平面連桿：根關節掛在魚身場景節點上，末端追蹤設計好的尾擺控制點。若魚身本身在世界空間移動，須先將世界目標轉成魚尾根節點的局部平面目標，再交由本章 FK/IK 求解；也可在同一世界空間一致地計算根部及目標。不可把兩種空間的座標直接相減。

實作時可在每個固定時間步，以前一影格的關節角作初始值，求解後記錄目標、末端、殘差、角度界限與停止原因。對碰到界限或超出可達範圍的影格，不應悄悄將求解結果標成「追蹤成功」。這套程序產生的是可重現的合成姿態與標註；魚尾軌跡平滑或影像逼真，都不等於已驗證真實魚類的游動力學。

習題

- **手算。****根部在原點， $L_1 = 2$ 、 $L_2 = 1$ 公尺， $\theta_1 = \pi/2$ 、 $\theta_2 = -\pi/2$ 。求末端位置及解析位置 Jacobian。
- **程式測試。****為本章程加入一項測試：在 $\theta = (0.4, -0.7)^T$ 、 $L_1 = 1$ 、 $L_2 = 0.5$ 下，分別以 $h = 10^{-4}$ 與 $h = 10^{-5}$ 求有限差分 Jacobian，並與依本章公式寫出的解析矩陣比較。寫出核心測試程式與合理斷言。
- **反例／除錯。****兩條各一公尺連桿從 $\theta = (0, 0)^T$ 出發，目標是 $(2, 0.2)^T$ 。某程式只因 DLS 更新很小便印出「目標已到達」。指出至少兩個錯誤判斷，並求目標離根部的距離，判斷未受限位置是否可達。
- **整合應用。****合成魚尾根節點在世界平面位置 $(4, 1)^T$ ，其局部 $+X$ 軸相對世界旋轉 $\pi/2$ 。世界空間目標是 $(3, 2)^T$ ，兩段長度均為一公尺。求目標在根節點局部平面的座標，判斷未受限距離條件，並提出逐影格求解時至少三項應記錄的欄位。

習題解答

1. 第一段方向為 $+Y$ ，第二段的世界角為 $\theta_1 + \theta_2 = 0$ ，所以末端是 $(1, 2)^T$ 公尺。代入導數：

$$J = \begin{bmatrix} -2 \sin(\pi/2) - \sin 0 & -\sin 0 \\ 2 \cos(\pi/2) + \cos 0 & \cos 0 \end{bmatrix} = \begin{bmatrix} -2 & 0 \\ 1 & 1 \end{bmatrix}.$$

2. 先按解析式寫出矩陣，再比較兩個有限差分步長；例如：

```
q = np.array([0.4, -0.7])
lengths = (1.0, 0.5)
a, b = q[0], q.sum()
J_exact = np.array([
    [-np.sin(a) - 0.5*np.sin(b), -0.5*np.sin(b)],
    [ np.cos(a) + 0.5*np.cos(b),  0.5*np.cos(b)],
])
for h in (1e-4, 1e-5):
    assert np.allclose(
        jacobian_fd(q, lengths, h=h), J_exact,
        rtol=0, atol=1e-7
    )
```

預期兩者均通過；這個斷言不要求較小 h 在所有硬體上嚴格優於較大 h 。

3. 第一，應以實際 FK 殘差而非更新量判定到達；小更新可能來自奇異姿態或限制。第二，應先做可達距離檢查，並分開回報停止與成功。目標距離為 $\sqrt{2^2 + 0.2^2} = \sqrt{4.04} \approx 2.010$ 公尺，大於最大伸展長度 2 公尺，因此未受限也不可達，最小可能殘差至少約 0.010 公尺。從伸直姿態作局部更新，更不能把停滯當成功。
4. 世界目標相對根部為 $(-1, 1)^T$ 。局部座標須乘根部旋轉的逆矩陣，即旋轉 $-\pi/2$ ，得到 $(1, 1)^T$ 公尺。距離為 $\sqrt{2}$ ，落在兩段各一公尺的未受限可達範圍 $[0, 2]$ 內；若加關節界限，仍需檢查合法解。逐影格至少記錄目標位置及其座標空間、求得的關節角、FK 末端位置、殘差與停止原因；固定時間步的時間戳也應保留，以便重現序列。

本章小結

FK 將關節角映射到末端位置；位置 Jacobian 描述這項映射在目前姿態附近的變化。阻尼最小平方能讓局部更新在奇異附近較穩定，但不保證全域收斂。有限差分需與 FK 使用相同角度及座標慣例；角度界限、可達距離和停止原因則須分開檢查。對動畫而言，可靠的 IK 結果不只有關節角，還包括可供查核的末端位置與殘差。

參考來源

- G1 PBRT 4：Transformations：座標與變換慣例的延伸參考。

- G6 Blender Manual：Skinning Introduction：骨架與網格關係的背景閱讀；本章求解器不依賴 Blender。
- G7 NumPy 線性代數參考：`solve` 與範數運算的介面參考。

第29章 魚群動畫與合成資料標註

學習目標與先備知識

本章探討如何使用局部規則模擬魚群動態，並生成包含相機投影、深度資訊及物體識別（ID）的合成訓練資料。這些資料可用於訓練機器學習模型進行目標偵測或姿態估計，無需依賴昂貴且難以標準化的真實水下攝影資料。

先備知識橋接：

1. **向量與變換**：熟練使用 4×4 齊次變換矩陣進行座標轉換。理解 $p_{world} = M_{world}p_{local}$ 的鏈式運算。
2. **相機管線**：掌握從世界座標到視圖座標（View Space），再到裁剪座標（Clip Space）及正規化裝置座標（NDC）的流程。特別是深度緩衝值（Depth Buffer）的計算公式。
3. **Python 與 NumPy**：能使用 NumPy 陣列與基本線性代數。本章範例為可讀性採**明確的 Python 迴圈**：對魚對做 $O(N^2)$ 鄰居搜尋，對每個圓盤逐像素填色，**不宣稱**已向量化或適合大量資料。
4. **JSON 格式**：理解基本 JSON 結構，用於序列化標註資料。

本卷限制聲明：本章生成的「魚群行為」僅為基於局部規則的幾何運動，**不等於**真實生物生態學。所有資料均為合成（Synthetic），用於驗證圖學管線與標註流程，不可直接用於科學推論。

本模型輸出的是質點的**位置、速度、近似深度與識別碼**；它不包含完整的魚體姿態（朝向、翻滾、關節角度）。速度可以近似推導出「前向方向」，但零速度時方向未定義，也不能取代骨架動畫。若下游需要姿態，必須在質點狀態之上另建朝向模型，並處理零速度、速度與 up 平行的退化情形。

問題與直覺

在養殖場監控中，真實水下攝影面臨光線衰減、浮游生物干擾及魚體遮擋等挑戰。合成資料的核心價值在於**可控性與由模擬狀態直接產生的標註**。本模型的狀態裡，我們知道每一幀中每條魚的位置、速度、近似深度及識別碼；前向方向可從非零速度粗略推得，但完整魚體姿態不在本模型範圍內。

直覺模型：想像一個 $10\text{m} \times 10\text{m} \times 5\text{m}$ 的水箱。我們放置 N 條簡化為質點的魚。每條魚遵循三個基本局部規則：

1. **分離（Separation）**：避免與鄰近個體過近。
2. **對齊（Alignment）**：朝向鄰近個體的平均速度方向。
3. **聚合（Cohesion）**：向鄰近個體的中心移動。

此外，需加入邊界約束，防止魚游出水箱。這些規則每幀更新速度與位置。接著，我們將這些 3D 狀態投影至 2D 影像平面，並輸出像素級深度圖與 ID 圖，作為標註資料。

數學與幾何推導

1. 局部群聚規則與同步更新

設第 i 條魚在第 k 步的狀態為位置 $\mathbf{p}_i^k$ 與速度 $\mathbf{v}_i^k$ 。定義鄰居集合 $\mathcal{N}_i^{ali}$ 為距離小於 R_{ali} 的其他魚，用於對齊； $\mathcal{N}_i^{coh}$ 為距離小於 R_{coh} 的其他魚，用於聚合； $\mathcal{N}_i^{sep}$ 為距離小於 R_{sep} 的其他魚，用於分離。

採用**半隱式 Euler (Semi-implicit Euler) **積分方法，以避免順序相依：

1. 複製舊狀態 $\mathbf{p}^k, \mathbf{v}^k$ 。
2. 計算加速度 $\mathbf{a}_i^k$ 基於 $\mathbf{p}^k, \mathbf{v}^k$ 。
3. 更新速度： $\mathbf{v}_i^{k+1} = \mathbf{v}_i^k + \mathbf{a}_i^k \Delta t$ 。
4. 更新位置： $\mathbf{p}_i^{k+1} = \mathbf{p}_i^k + \mathbf{v}_i^{k+1} \Delta t$ 。

量綱分析： 為確保加速度單位為 m/s^2 ，各權重需具備特定量綱：

- 分離項基礎量 $\frac{\mathbf{p}_i - \mathbf{p}_j}{d_{ij}^2}$ 單位為 m^{-1} 。權重 w_s 單位應為 m^2/s^2 。
- 對齊項基礎量 $(\bar{\mathbf{v}} - \mathbf{v}_i)$ 單位為 m/s 。權重 w_a 單位應為 s^{-1} 。
- 聚合項基礎量 $(\bar{\mathbf{p}} - \mathbf{p}_i)$ 單位為 m 。權重 w_c 單位應為 s^{-2} 。

分離力：

$$\mathbf{a}_{sep,i} = w_s \sum_{j \in \mathcal{N}_i^{sep}} \frac{\mathbf{p}_i - \mathbf{p}_j}{d_{ij}^2 + \epsilon}$$

其中 ϵ 避免分母為零，單位為 m^2 。若兩魚位置完全重合 ($d = 0$ 且 $\mathbf{p}_i = \mathbf{p}_j$)，此項仍為零；epsilon 只能穩定分母，不能提供方向。初始化最小間距只能降低初始重合機率，無法保證模擬途中不重合。若後續遇到 d 小於依場景尺度設定的長度容差，可依魚 ID 順序選定一個確定性單位方向，對兩魚施加大小相等、方向相反的小擾動或分離加速度，並以舊狀態同步計算，避免依迴圈更新順序而異。

對齊力：

$$\mathbf{a}_{ali,i} = w_a \left(\frac{\sum_{j \in \mathcal{N}_i^{ali}} \mathbf{v}_j}{|\mathcal{N}_i^{ali}|} - \mathbf{v}_i \right) \quad (\text{若 } |\mathcal{N}_i^{ali}| > 0)$$

聚合力：

$$\mathbf{a}_{coh,i} = w_c \left(\frac{\sum_{j \in \mathcal{N}_i^{coh}} \mathbf{p}_j}{|\mathcal{N}_i^{coh}|} - \mathbf{p}_i \right) \quad (\text{若 } |\mathcal{N}_i^{coh}| > 0)$$

2. 邊界約束

採用**裁切並反轉法向速度 (Clamp and Reflect)**策略。若 $\mathbf{p}_i^{k+1}$ 超出邊界 B ，則將其投影回邊界表面 (Clamp)，並反轉法向速度分量。例如，若 $x_{new} > x_{max}$ ，則 $x_{new} = x_{max}$ 且 $v_x^{new} = -v_x^{new}$ 。此策略保證了位置的有效性，但丟棄了越界距離，非精確幾何反射。

3. 相機投影與深度標註

依本卷約定，相機局部座標系看向 $-Z$ 方向。

1. **座標變換**：若相機與世界座標軸對齊（相機在 $\mathbf{c}$ ，看向 $-\mathbf{z}$ ），相機座標 $\mathbf{p}_{cam} = \mathbf{p}_{world} - \mathbf{c}$ 。定義正前向距離 $z = -p_{cam,z}$ 。可見條件： $z_{near} \leq z \leq z_{far}$ 。
2. **透視投影**：

$$u = f_x \frac{x_{cam}}{z} + c_x$$

$$v = c_y - f_y \frac{y_{cam}}{z}$$

注意：影像 Y 軸向下，故 y_{cam} 前帶負號。

3. **OpenGL 深度映射**：標準 OpenGL 深度緩衝值 $d \in [0, 1]$ 計算如下：

$$d = \frac{z_{far}}{z_{far} - z_{near}} - \frac{z_{far} z_{near}}{(z_{far} - z_{near})z}$$

驗算：

- 當 $z = z_{near}$ 時， $d = 0$ 。
- 當 $z = z_{far}$ 時， $d = 1$ 。注意：此 d 為非線性緩衝值。標註中若需線性深度，應另存 z （單位：公尺）。

4. 像素級標註生成

為生成像素級標註，使用**圓盤代理 (Disk Proxy)**近似魚體在螢幕上的投影。

1. **螢幕空間半徑**： $r_{screen} = f_y \cdot r_{world} / z$ 。
2. **像素中心**：依全書約定，像素 (u_{pix}, v_{pix}) 的中心為 $(u_{pix} + 0.5, v_{pix} + 0.5)$ 。
3. **候選範圍**：使用 `floor` 與 `ceil` 確定需要檢查的像素範圍，避免 `int()` 對負數截斷的錯誤。

$$u_{min} = \max(0, \lceil u - r_{screen} - 0.5 \rceil)$$

$$u_{max} = \min(W - 1, \lfloor u + r_{screen} - 0.5 \rfloor)$$

v 軸同理。

4. 深度測試與 ID 更新：維護兩張矩陣：Depth_Map (float, 單位 m) 與 ID_Map (int)。初始化

Depth_Map 為 ∞ ，ID_Map 為 0。對於候選範圍內的每個像素：

- 計算距離圓心的螢幕距離： $dist = \sqrt{(u_{pix} + 0.5 - u)^2 + (v_{pix} + 0.5 - v)^2}$ 。
- 若 $dist \leq r_{screen}$ ，且 $z < Depth_Map[v_{pix}, u_{pix}]$ ，則更新：Depth_Map[v_{pix}, u_{pix}] = z ID_Map[v_{pix}, u_{pix}] = fish_id + 1 (1-based ID)

5. 索引順序：NumPy 陣列索引為 [row, col]，對應影像的 [v, u]。

逐步手算例題

例 1：雙魚分離力與同步更新

設定：

- 魚 A： $\mathbf{p}_A = (0, 0, 0)$, $\mathbf{v}_A = (1, 0, 0)$
- 魚 B： $\mathbf{p}_B = (0.5, 0, 0)$, $\mathbf{v}_B = (-1, 0, 0)$
- 參數： $R_{sep} = 1.0$, $w_s = 2.0 \text{ m}^2/\text{s}^2$, $\epsilon = 0.01 \text{ m}^2$, $\Delta t = 0.1 \text{ s}$ 。
- 無對齊與聚合。

計算：

1. 距離： $d_{AB} = 0.5 \text{ m}$ 。
2. 分離加速度： $\mathbf{p}_A - \mathbf{p}_B = (-0.5, 0, 0)$ 。 $\mathbf{a}_{sep,A} = 2.0 \cdot \frac{(-0.5, 0, 0)}{0.5^2 + 0.01} = \frac{(-1.0, 0, 0)}{0.26} \approx (-3.846, 0, 0) \text{ m/s}^2$ 。
同理， $\mathbf{a}_{sep,B} \approx (3.846, 0, 0) \text{ m/s}^2$ 。
3. 更新速度： $\mathbf{v}_A^{new} = (1, 0, 0) + (-3.846, 0, 0) \cdot 0.1 = (0.6154, 0, 0) \text{ m/s}$ 。 $\mathbf{v}_B^{new} = (-1, 0, 0) + (3.846, 0, 0) \cdot 0.1 = (-0.6154, 0, 0) \text{ m/s}$ 。
4. 更新位置： $\mathbf{p}_A^{new} = (0, 0, 0) + (0.6154, 0, 0) \cdot 0.1 = (0.06154, 0, 0) \text{ m}$ 。 $\mathbf{p}_B^{new} = (0.5, 0, 0) + (-0.6154, 0, 0) \cdot 0.1 = (0.43846, 0, 0) \text{ m}$ 。結果：新距離 $d^{new} = 0.43846 - 0.06154 = 0.37692 \text{ m}$ 。
◦ 結論：分離加速度方向正確（相背），但在此參數與單一步長下，兩魚仍相向移動，因此距離暫時縮短。分離規則不保證每一步的距離都增加，它僅施加減速或反向加速度。

例 2：相機投影與深度

設定：

- 相機位於 $\mathbf{c} = (0, 0, 5)$ ，看向 $-Z$ 。
- 魚位於 $\mathbf{p} = (1, 1, 4)$ 。
- 焦距 $f_x = f_y = 500$ ，主點 $(c_x, c_y) = (400, 300)$ 。
- $z_{near} = 0.1$, $z_{far} = 10.0$ 。

計算：

1. 相機座標： $\mathbf{p}_{cam} = (1, 1, 4) - (0, 0, 5) = (1, 1, -1)$ 。 $z = -(-1) = 1.0$ 。 $z_{near} \leq 1.0 \leq z_{far}$ ，通過前後裁切。
2. 2D 投影： $u = 500 \cdot \frac{1}{1} + 400 = 900$ 。 $v = 300 - 500 \cdot \frac{1}{1} = -200$ 。此點通過深度範圍，但投影中心位於 800×600 影像範圍外。若魚體半徑足夠大，其圓盤代理可能仍與影像邊界相交，需進行螢幕邊界測試。
3. 深度： $d = \frac{10}{10-0.1} - \frac{10 \cdot 0.1}{(10-0.1) \cdot 1} = \frac{10}{9.9} - \frac{1}{9.9} = \frac{9}{9.9} \approx 0.9091$ 。線性深度 $z = 1.0 \text{ m}$ 。

實作與程式

以下程式生成魚群動畫，並輸出像素級深度與 ID 標註檔案及 JSON 清單。

```
import numpy as np
import json
import os
from math import floor, ceil

class Fish:
    def __init__(self, id, pos, vel):
        self.id = id
        self.pos = pos
        self.vel = vel

class SchoolSimulation:
    def __init__(self, num_fish=5, bounds=None, seed=42):
        if bounds is None:
            self.bounds = np.array([10.0, 10.0, 5.0])
        else:
            self.bounds = np.array(bounds)
        self.rng = np.random.default_rng(seed)
        self.fish = []
        min_init_dist = 0.5 # 初始化時可接受的最小間距，單位公尺
        min_init_dist2 = min_init_dist ** 2
        max_attempts = 1000
        for i in range(num_fish):
            placed = False
            for _ in range(max_attempts):
                pos = self.rng.uniform(-self.bounds/2, self.bounds/2)
                ok = True
                for f in self.fish:
                    if np.sum((pos - f.pos) ** 2) < min_init_dist2:
                        ok = False
                        break
                if ok:
                    placed = True
                    break
            if not placed:
                raise RuntimeError(
                    f"無法在 {max_attempts} 次嘗試內為魚 {i} 找到足夠遠的位置"
                )
            vel = self.rng.uniform(-0.5, 0.5, size=3)
            self.fish.append(Fish(i, pos, vel))

        self.params = {
```

```

        'w_sep': 2.0, 'R_sep': 1.0, 'eps': 1e-2,
        'w_ali': 0.5, 'R_ali': 2.0,
        'w_coh': 0.5, 'R_coh': 2.0,
        'dt': 0.1,
        'v_max': 2.0
    }
    # Camera setup
    self.cam_pos = np.array([0.0, 0.0, 10.0])
    self.img_width = 800
    self.img_height = 600
    self.focal = 400.0 # f_x = f_y
    self.principal_point = np.array([400, 300])
    self.z_near = 0.1
    self.z_far = 20.0
    self.fish_radius = 0.2 # World units

def update(self):
    dt = self.params['dt']
    n = len(self.fish)
    old_pos = [f.pos.copy() for f in self.fish]
    old_vel = [f.vel.copy() for f in self.fish]

    new_vels = [np.zeros(3) for _ in range(n)]

    for i in range(n):
        p_i = old_pos[i]
        v_i = old_vel[i]

        acc_sep = np.zeros(3)
        acc_ali = np.zeros(3)
        acc_coh = np.zeros(3)

        n_ali = 0
        n_coh = 0
        sum_vel = np.zeros(3)
        sum_pos = np.zeros(3)

        for j in range(n):
            if i == j: continue
            d_vec = p_i - old_pos[j]
            d = np.linalg.norm(d_vec)

            if d < self.params['R_sep']:
                denom = d * d + self.params['eps']
                acc_sep += self.params['w_sep'] * d_vec / denom

            if d < self.params['R_ali']:
                sum_vel += old_vel[j]
                n_ali += 1

            if d < self.params['R_coh']:
                sum_pos += old_pos[j]
                n_coh += 1

        if n_ali > 0:
            acc_ali = self.params['w_ali'] * (sum_vel / n_ali - v_i)
        if n_coh > 0:
            acc_coh = self.params['w_coh'] * (sum_pos / n_coh - p_i)

```

```

total_acc = acc_sep + acc.ali + acc.coh

v_new = v_i + total_acc * dt

v_norm = np.linalg.norm(v_new)
if v_norm > self.params['v_max']:
    v_new = (v_new / v_norm) * self.params['v_max']

new_vels[i] = v_new

for i in range(n):
    p_new = old_pos[i] + new_vels[i] * dt
    # Clamp and Reflect
    for axis in range(3):
        limit = self.bounds[axis] / 2
        if p_new[axis] < -limit:
            p_new[axis] = -limit
            new_vels[i][axis] *= -1
        elif p_new[axis] > limit:
            p_new[axis] = limit
            new_vels[i][axis] *= -1
    self.fish[i].pos = p_new
    self.fish[i].vel = new_vels[i]

def get_annotations(self):
    # Depth in meters, background is inf
    depth_map = np.full((self.img_height, self.img_width), np.inf, dtype=np.float32)
    # ID map, background is 0
    id_map = np.zeros((self.img_height, self.img_width), dtype=np.int32)

    for fish in self.fish:
        p_cam = fish.pos - self.cam_pos
        z = -p_cam[2]

        if z < self.z_near or z > self.z_far:
            continue

        u = self.focal * (p_cam[0] / z) + self.principal_point[0]
        v = self.principal_point[1] - self.focal * (p_cam[1] / z)

        r_screen = self.focal * (self.fish_radius / z)

        # Pixel center offset
        u_min = max(0, ceil(u - r_screen - 0.5))
        u_max = min(self.img_width - 1, floor(u + r_screen - 0.5))
        v_min = max(0, ceil(v - r_screen - 0.5))
        v_max = min(self.img_height - 1, floor(v + r_screen - 0.5))

        if u_min > u_max or v_min > v_max:
            continue

        for v_pix in range(int(v_min), int(v_max) + 1):
            for u_pix in range(int(u_min), int(u_max) + 1):
                # Pixel center
                pu = u_pix + 0.5
                pv = v_pix + 0.5
                dx = pu - u
                dy = pv - v
                if dx*dx + dy*dy <= r_screen*r_screen:

```

```

        if z < depth_map[v_pix, u_pix]:
            depth_map[v_pix, u_pix] = z
            id_map[v_pix, u_pix] = fish.id + 1

    return depth_map, id_map

def run_simulation():
    sim = SchoolSimulation(num_fish=5, seed=42)
    frames = 5
    out_dir = "synthetic_fish_seq"
    if not os.path.exists(out_dir):
        os.makedirs(out_dir)

    sequence_manifest = []

    for t in range(frames):
        sim.update()
        depth_map, id_map = sim.get_annotations()

        depth_file = f"depth_{t:04d}.numpy"
        id_file = f"id_{t:04d}.numpy"

        np.save(os.path.join(out_dir, depth_file), depth_map)
        np.save(os.path.join(out_dir, id_file), id_map)

        entry = {
            "frame": t,
            "depth_file": depth_file,
            "id_file": id_file,
            "visible_pixels": int(np.sum(id_map > 0)),
            "fish_positions": [f.pos.tolist() for f in sim.fish],
            "fish_velocities": [f.vel.tolist() for f in sim.fish]
        }
        sequence_manifest.append(entry)

    # Save global manifest
    global_manifest = {
        "camera": {
            "position": sim.cam_pos.tolist(),
            "focal_length": sim.focal,
            "principal_point": sim.principal_point.tolist(),
            "near": sim.z_near,
            "far": sim.z_far,
            "width": sim.img_width,
            "height": sim.img_height
        },
        "depth_encoding": "camera_forward_distance_m",
        "depth_dtype": "float32",
        "background_depth": "infinity",
        "id_encoding": "fish.id + 1",
        "background_id": 0,
        "id_dtype": "int32",
        "camera_convention": "local axes aligned with world axes; looks toward -Z",
        "focal_length_note": "f_x = f_y = focal_length",
        "frame_state": "post_update",
        "seed": 42,
        "dt": sim.params['dt'],
        "frames": sequence_manifest
    }

```

```

with open(os.path.join(out_dir, "manifest.json"), "w", encoding="utf-8") as f:
    json.dump(global_manifest, f, indent=2)

print(f"Generated {frames} frames in {out_dir}. Manifest written.")

if __name__ == "__main__":
    run_simulation()

```

測試與預期結果

執行上述程式，預期產生 `synthetic_fish_seq` 目錄，包含 5 組 `.npz` 檔案及 `manifest.json`。檢查點：

1. 檔案存在：檢查是否生成了 `depth_0000.npz` 至 `depth_0004.npz` 及對應 ID 檔案。
2. 可見像素：`manifest.json` 中 `visible_pixels` 應為非負整數。
3. 深度範圍：載入 `depth_0000.npz`，非 `inf` 值應在 $[0.1, 20.0]$ 之間。
4. ID 一致性：`id_map` 中非零值應在 $[1, 5]$ 之間（5 條魚，ID 1-5）。
5. 可重現性：使用固定 seed，每次執行結果相同。

單魚確定性測試（手算／簡化程式）：若單條魚位於相機座標 $(0, 0, -2)$ ，且相機在世界座標原點並與世界軸對齊，則其世界位置也是 $(0, 0, -2)$ ；若相機不在原點，應先以相機外參將世界位置轉為此相機座標。令 $r_{world} = 0.1, f = 400, W = 800, H = 600, z = 2, u = 400(0)/2 + 400 = 400, v = 300 - 400(0)/2 = 300, r_{screen} = 400(0.1)/2 = 20$ 。

注意投影中心 $(u, v) = (400, 300)$ 本身不是某個像素的中心——像素中心位於整數座標加 0.5。與此投影中心最接近的四個像素中心為 $(399.5, 299.5)$ 、 $(400.5, 299.5)$ 、 $(399.5, 300.5)$ 、 $(400.5, 300.5)$ ，對應陣列索引 $(v, u) = (299, 399), (299, 400), (300, 399), (300, 400)$ 。因為螢幕半徑 $r_{screen} = 20$ 遠大於 1，這四個像素都應被覆蓋，深度皆為 2.0，ID 皆為 1。**預期斷言：**

```

assert id_map[299, 399] == 1
assert depth_map[299, 399] == 2.0
assert id_map[300, 400] == 1
assert depth_map[300, 400] == 2.0

```

雙魚遮擋測試：程式中的 `fish.pos` 是世界座標，故測試先依相機位置換算世界位置。設定魚 A 的內部 `fish.id=0`、相機座標為 $(0, 0, -2)$ ；魚 B 的內部 `fish.id=1`、相機座標為 $(0, 0, -4)$ 。像素圖寫入的是 `fish.id + 1`，因此 A、B 的 map ID 分別為 1、2。兩魚投影中心相同時，中心重疊像素應由較近的 A 佔據，**預期：**

```

fish_a.pos = sim.cam_pos + np.array([0.0, 0.0, -2.0])
fish_b.pos = sim.cam_pos + np.array([0.0, 0.0, -4.0])
sim.fish = [fish_a, fish_b]
depth_map, id_map = sim.get_annotations()
assert depth_map[299, 399] == 2.0
assert id_map[299, 399] == fish_a.id + 1 == 1

```

把 `sim.fish` 列表順序交換後，深度與 map ID 預期仍相同，因為較近者會覆蓋較遠者。若深度完全相等，現行嚴格比較 `z < depth_map[...]` 會由先處理者勝出；重現測試時應固定列表順序，或另行制定 tie-break 規則。另應測試 `z < z_near`、`z > z_far` 以及投影完全在影像外的魚，預期皆不產生非零 ID 像素。

除錯與常見陷阱

1. 分離力方向：

- 陷阱：誤用 $\mathbf{p}_j - \mathbf{p}_i$ ，導致吸引。
- 解決：確認 $\mathbf{p}_i - \mathbf{p}_j$ 指向遠離鄰居。

2. 順序相依：

- 陷阱：在迴圈中直接更新 `self.fish[i].vel`，導致後續魚讀取到部分更新後的狀態。
- 解決：使用 `old_pos` 和 `old_vel` 列表，一次性提交更新。

3. 深度索引：

- 陷阱：使用 `id_map[u, v]`。
- 解決：使用 `id_map[v, u]`，因為 NumPy 陣列第一維是 row (v)。

4. 像素中心：

- 陷阱：使用整數 u, v 作為中心。
- 解決：使用 $u + 0.5, v + 0.5$ 。

5. 量綱不一致：

- 陷阱：權重無因次，導致加速度單位錯誤。
- 解決：明確設定權重單位，或正規化 steering 向量。

養殖數位分身案例

在養殖場數位分身中，此合成資料可用於：

1. 檢測模型訓練：使用 `id_map` 生成 2D bounding box 標註，訓練目標偵測模型。
2. 深度估計：使用 `depth_map`（線性公尺深度）作為真值，訓練單目深度估計網路。
3. 數據增強：透過改變 `seed`、權重參數、相機位置，生成多樣化資料集。

注意：合成魚的游動姿態為質點運動，未模擬尾擺。若需更真實，需結合骨架動畫。這些深度與 ID 圖是此幾何代理模型下的參考真值；它們不對應任何水下成像感測器的實際輸出。真實相機的深度標註需要考慮折射、散射、感測器噪聲與校正誤差，這些都不在本章範圍內。

inf 背景的讀取方式：`.npy` 檔案中的 `float32 inf` 會在 `np.load` 之後原樣讀回，可用 `np.isinf(depth)` 或 `np.isfinite(depth)` 判定。若下游以影像方式顯示，需先以對數或反比映射將 `inf` 換成可視值；若下游作為訓練標註，應在讀取後顯式排除 `inf` 像素，或另存遮罩。把 `inf` 直接送進梯度計算會汙染整批資料。

可重現性測試的固定初始狀態：除了固定 `seed` 外，可重現實驗還應記錄：初始 `min_init_dist` 與 `max_attempts`、`dt`、`v_max`、各權重與半徑、相機內外參、程式版本及NumPy版本。不同初始位置集合會導致不同群聚軌跡。相同環境與設定下，狀態及標註應在明訂容差內可重現；檔案雜湊或逐位元一致可作為同一受控環境內的加強檢查，但不應保證跨平台、函式庫版本或浮點實作逐位元相同。

$O(N^2)$ 鄰居搜尋與圓盤代理的限制：本章每步對所有魚對計算距離，總運算量為 $O(N^2)$ ；若 N 大於數百條魚，需要空間雜湊或網格分桶。圓盤代理在螢幕空間是硬邊圓，與魚體實際剪影相差很大；對細長魚體，圓盤將高估垂直方向的覆蓋、低估前後方向的覆蓋。作為合成標註，圓盤足以驗證相機投影、z-buffer 與 ID 更新流程，但不可作為姿態估計題目的擬真輸入。

習題

1. **手算：**給定相機 $f = 400$, $(c_x, c_y) = (320, 240)$ 。物體在相機座標 $(-1, 2, -4)$ 。求 u, v 及 OpenGL 深度 d （假設 $z_{near} = 0.1, z_{far} = 10$ ）。
2. **程式修改：**修改 `update` 函式，增加一個「捕食者」魚（ID=0），其目標是追擊最近的獵物。驗收條件為「50 幀內，捕食者與當前最近獵物的距離最小值小於初始距離」；若獵物會切換，必須記錄對當前最近獵物的距離，不能把它誤當成固定追蹤對象的距離。
3. **反例：**若將 `w_sep` 設為 0，預期魚群行為為何？若將 `w_coh` 設為 10，預期行為為何？
4. **整合應用：**修改 `get_annotations`，額外輸出每條魚中心的 OpenGL 深度緩衝值 $d \in [0, 1]$ 至 JSON 清單中，格式：`"fish_depths": [{"id": 0, "u": ..., "v": ..., "depth_buffer": ...}, ...]`。`id` 為內部 0-based 的 `fish.id`（與深度與 ID 圖中 1-based 的像素值不同），`u`、`v` 為投影中心（連續像素座標，非整數索引）。

習題解答

1. **手算：** $x' = -1, y' = 2, z' = -4 \implies z = 4$ 。 $u = 400 \cdot \frac{-1}{4} + 320 = 320 - 100 = 220$ 。 $v = 240 - 400 \cdot \frac{2}{4} = 240 - 200 = 40$ 。 $d = \frac{10}{10-0.1} - \frac{10 \cdot 0.1}{(10-0.1) \cdot 4} = \frac{10}{9.9} - \frac{1}{39.6} \approx 1.0101 - 0.0253 = 0.9848$ 。
2. **程式修改：**在 `update` 的加速度迴圈中，對 `i == 0` 的捕食者加上追擊項。完整片段如下（放在計算 `acc_sep` / `acc_al` / `acc_coh` 之後、組合 `total_acc` 之前）：

```
k_chase = 1.5 # 反比於時間平方的權重，單位 1/s^2
if i == 0 and n > 1:
    prey_indices = range(1, n)
    target_j = min(
        prey_indices,
        key=lambda j: np.linalg.norm(old_pos[j] - p_i),
    )
    acc_chase = k_chase * (old_pos[target_j] - p_i)
    total_acc = acc_sep + acc_al + acc_coh + acc_chase
else:
    total_acc = acc_sep + acc_al + acc_coh
```

注意力施加在舊狀態 `old_pos` 上，順序不依賴其他魚的更新；速度更新與位置更新維持同步半隱式 Euler 的既有順序。`k_chase` 與其他權重一樣僅是數值參數，須依場景尺度與 `dt` 選定，不是物理常數。

驗證：在 `run_simulation` 的每一幀之後，記錄「捕食者與當前最近獵物的歐氏距離」，取50幀的最小值；若小於初始距離即符合題目驗收條件。距離不保證逐幀單調下降，因為慣性、速度上限、其他群聚力、邊界反射與獵物切換都可能使距離增加。結果取決於 `k_chase`、固定初始狀態及其他規則；若要重現驗收，須記錄這些設定，且實際結果仍需執行測試確認。

3. 反例：

- `w_sep=0`：預期魚群過度聚集，可能重疊或頻繁觸發速度上限。
- `w_coh=10`：預期產生較強聚合，可能造成振盪、頻繁觸發速度上限或邊界撞擊。

4. **整合應用：**完整實作如下。只輸出在 $z_{near} \leq z \leq z_{far}$ 範圍內的中心；畫面外但深度通過裁切的中心仍保留，並明示其 u 、 v 可能超出 $[0, W)$ 或 $[0, H)$ 。

```
def get_fish_depths(self):
    """回傳每條魚中心投影的 u, v 與 OpenGL 深度緩衝值。"""
    out = []
    for fish in self.fish:
        p_cam = fish.pos - self.cam_pos
        z = -p_cam[2]
        if z < self.z_near or z > self.z_far:
            continue
        u = self.focal * (p_cam[0] / z) + self.principal_point[0]
        v = self.principal_point[1] - self.focal * (p_cam[1] / z)
        depth_buffer = (
            self.z_far / (self.z_far - self.z_near)
            - (self.z_far * self.z_near)
            / ((self.z_far - self.z_near) * z)
        )
        out.append({
            "id": int(fish.id),
            "u": float(u),
            "v": float(v),
            "depth_buffer": float(depth_buffer),
            "z_m": float(z),
        })
    return out
```

在 `run_simulation` 中把 `sim.get_fish_depths()` 的結果放入該幀的 manifest 條目（例如欄位名 `fish_depths`）。`id` 欄位採 0-based 內部 ID；深度與 ID 圖中的像素值使用 1-based，兩者必須區分。

本章小結

本章展示了如何利用局部規則生成可控的魚群動畫，並通過相機投影生成像素級深度和 ID 的合成標註資料。重點在於理解同步半隱式 Euler 時間積分、正確的座標變換與量綱、OpenGL 深度映射、像素中心規則及像素級 Z-buffering。

本模型明確的簡化包括：魚為質點（不含完整姿態與骨架）、鄰居搜尋為 $O(N^2)$ 、魚體剪影以圓盤代理、相機局部軸與世界軸對齊（尚未使用一般 view matrix）、未模擬水體吸收散射與感測器噪聲。這些限制決定它能驗證什麼、不能推論什麼：它可以驗證由已知 3D 狀態產生深度與 ID 標註的管線是否自洽，不能推論任何真實水下成像品質或生物行為。

合成資料是驗證管線的強大工具，但需明確其簡化假設與真實生態的差距。所有輸出檔案與測試結果應標為「預期」，未實跑時不得宣稱已驗證。

參考來源

1. **G1** PBRT 4: Transformations — 支援一般座標變換概念；不特別支持群聚規則或 OpenGL 深度緩衝公式。
2. **G4** Ray Tracing in One Weekend — 提供相機與光線追蹤概念的入門背景；不等於本章 OpenGL 深度映射的直接依據。
3. **G7** NumPy 線性代數參考 — 支援 `np.linalg.norm`、`np.sum` 等 API 說明；不涉及本章群聚規則。

本章的局部群聚規則（分離／對齊／聚合）、量綱分析、初始化 rejection sampling、遮擋測試設計、manifest 欄位規格與 inf 深度背景讀取方式，皆為本章自行推導與整理，未逐條依賴上述來源。來源清單僅供延伸查閱，不表示相關論點已由外部獨立驗證。本章描述的檔案與測試結果均為預期；出版前仍須在指定環境人工執行並檢查輸出。

第 30 章 整合專題：可稽核養殖數位分身

學習目標與先備知識

本章把場景、動畫、渲染、標註與查詢串成一個可重現的合成資料專題。完成後，你應能：

- 用一致的影格編號、時間、物件ID、單位與座標系管理輸出。
- 產生合成影像及由同一影格狀態建立的標註。
- 保存完整生成設定，讓輸出能追溯到所用參數與生成器版本。
- 撰寫唯讀查詢，驗證manifest與影格標註一致，並附上查詢依據。
- 區分合成資料、實測資料與控制指令。

先備為本卷向量、矩陣、相機、渲染、動畫及合成資料概念，以及 Python 基本語法。核心實驗只使用 Python 3.10+ 標準庫，不要求GPU、Blender、網路或第三方套件。程式產生二維合成影像、ID遮罩、JSON標註及清單；它不是完整三維渲染器，也不模擬真實水下光學、魚類生態或養殖設備。

問題與直覺

若場景、影像、動畫狀態和標註各自生成，卻沒有共同的影格編號、時間戳及設定來源，影像就難以追溯：標記是哪個物件？位置來自哪個影格？影像和標註是否使用同一份狀態？

本章所稱的「可稽核」，是每個輸出都能透過manifest追溯到影格、時間、完整生成設定及生成器版本；標註欄位也明示單位、座標系與資料來源。這只表示資料流程可追蹤，不表示結果正確描述真實養殖場。

專題流程為：

1. **設定場景**：列出影像尺寸、座標範圍、物件初始狀態及顏色。
2. **取樣動畫**：以固定影格率計算每個物件的位置。
3. **生成觀測**：由同一影格狀態生成彩色影像、物件ID遮罩及標註。
4. **記錄清單**：在manifest保存設定快照、版本識別、影格時間和輸出路徑。
5. **唯讀查詢**：讀取manifest與標註，驗證相互一致後回傳物件資料及來源。

彩色影像是示意圖；ID遮罩是由生成狀態產生的合成通道；JSON記錄的是合成狀態標註。三者都不是攝影機或現場感測器的實測輸出。

數學與幾何推導

固定時間步與影格狀態

令影格編號為整數 k ，影格頻率為 f 影格／秒，影格時間為

$$t_k = \frac{k}{f}.$$

本章用固定影格率取樣。若物件 i 沿水平方向等速移動，其位置為

$$x_i(t) = x_{i,0} + v_i t,$$

其中 $x_{i,0}$ 的單位是公尺，速度 v_i 的單位是公尺／秒。這是程式定義的合成運動，不代表真實魚類行為。

世界座標映射至影像座標

世界座標採右手系， $+X$ 向右、 $+Y$ 向上、 $+Z$ 由畫面向觀者。影像原點在左上角，水平索引 u 向右增加，垂直索引 v 向下增加。影像寬、高分別為 W, H ，整數索引範圍為 $0 \leq u < W$ 、 $0 \leq v < H$ 。

本例不做透視投影，而將水平範圍 $[x_{\min}, x_{\max}]$ 和垂直範圍 $[y_{\min}, y_{\max}]$ 線性映射到像素索引：

$$u = \text{round} \left(\frac{x - x_{\min}}{x_{\max} - x_{\min}} (W - 1) \right),$$

$$v = \text{round} \left(\frac{y_{\max} - y}{y_{\max} - y_{\min}} (H - 1) \right).$$

要求 $x_{\max} > x_{\min}$ 、 $y_{\max} > y_{\min}$ 。第二式使用 $y_{\max} - y$ ，是因為世界 Y 向上而影像列索引向下。

本章將「世界範圍出界」定義為 x 或 y 超出設定範圍，並在取整、像素夾取之前判斷：

$$\text{outside} = (x < x_{\min}) \vee (x > x_{\max}) \vee (y < y_{\min}) \vee (y > y_{\max}).$$

像素索引仍會夾取至影像邊界，但標註同時保留未夾取前的世界位置和出界旗標。如此可避免把邊界像素位置誤認為物件真實世界位置。

同步生成與追溯

令生成設定為 C ，生成器版本識別為 V ，影格輸出影像為 I_k 、ID遮罩為 M_k 、狀態標註為 A_k 。三者應由相同的影格狀態生成：

$$(I_k, M_k, A_k) = G(C, V, k, t_k).$$

manifest需保存 C 的完整快照或可追溯設定檔及內容識別碼，也要保存 V 、影格時間、影像和標註路徑。若只記錄程式檔名，卻不識別其內容版本，無法確認日後執行的是同一份生成器。

影像色彩數值也必須明確。本章直接指定8-bit sRGB編碼值作為示意顏色，不做線性光照或色彩運算。ID遮罩中的整數是標籤，不是顏色；不可對ID值做sRGB轉換或插值。

逐步手算例題

例一：影格時間與合成位置

某合成物件的初始水平位置為 $x_0 = 0.20$ 公尺，速度為 $v = 0.10$ 公尺／秒，影格率為 $f = 5$ 影格／秒。求第3影格時間與位置。

1. 影格時間：

$$t_3 = \frac{3}{5} = 0.6 \text{ 秒}.$$

2. 合成位置：

$$x(0.6) = 0.20 + 0.10 \times 0.6 = 0.26 \text{ 公尺}.$$

例二：映射至像素索引

影像寬 $W = 16$ ，水平範圍為 $[-1, 1]$ 公尺。將 $x = 0.26$ 公尺映射到水平索引：

1. 正規化位置為

$$\frac{0.26 - (-1)}{1 - (-1)} = 0.63.$$

2. 乘上 $W - 1 = 15$ ，得到 9.45，因此

$$u = \text{round}(9.45) = 9.$$

索引9是第10個像素欄，因為索引從0開始。此像素位置是線性映射後的影像座標，不是世界位置的精確測量。

實作與程式

以下程式只使用 Python 3.10+ 標準庫。它產生：

- 8-bit sRGB示意彩色PPM影像。
- 以整數ID表示物件的PGM遮罩。
- 逐影格JSON標註。
- 含完整設定快照、生成器版本識別、影格路徑與時間的manifest。
- 驗證manifest、標註及物件來源一致性的唯讀查詢函式。

將程式存為 `digital_twin.py`。執行會在目前目錄建立 `output/`，不讀取外部檔案或網路。

```

import hashlib
import json
from pathlib import Path

WIDTH = 64
HEIGHT = 48
FPS = 4
FRAME_COUNT = 5
X_RANGE = (-1.0, 1.0)
Y_RANGE = (-0.75, 0.75)
DISC_RADIUS = 3

# 直接指定的8-bit sRGB示意編碼值；不做線性光照運算。
OBJECTS = [
    {"id": 1, "x0": -0.60, "y": -0.10, "vx": 0.20,
     "srgb8": [240, 150, 50]},
    {"id": 2, "x0": 0.35, "y": 0.25, "vx": -0.10,
     "srgb8": [70, 210, 170]},
]

GENERATOR_VERSION = "synthetic-demo-1"
OUTPUT = Path("output")

def configuration():
    """回傳本次生成使用的完整設定快照。"""
    return {
        "width": WIDTH,
        "height": HEIGHT,
        "fps": FPS,
        "frame_count": FRAME_COUNT,
        "x_range_m": list(X_RANGE),
        "y_range_m": list(Y_RANGE),
        "disc_radius_px": DISC_RADIUS,
        "objects": OBJECTS,
        "background_srgb8": [18, 36, 54],
        "color_semantics": "8-bit sRGB display codes; no lighting",
        "mask_semantics": "integer object IDs; 0 is background",
        "world_convention":
            "right-handed; +X right, +Y up, +Z toward viewer",
        "image_convention":
            "origin upper-left; integer pixel indices increase right/down",
    }

def config_id(config):
    """以穩定JSON序列化後的SHA-256識別設定內容。"""
    payload = json.dumps(
        config, sort_keys=True, separators=(",", ":"), ensure_ascii=True
    ).encode("utf-8")
    return hashlib.sha256(payload).hexdigest()

def map_to_pixel(x, y):
    xmin, xmax = X_RANGE
    ymin, ymax = Y_RANGE
    if not xmin < xmax or not ymin < ymax:

```

```

        raise ValueError("世界座標範圍必須遞增")

# 出界依世界座標判定，先於取整與像素夾取。
outside = not (
    xmin <= x <= xmax and ymin <= y <= ymax
)

raw_u = round((x - xmin) / (xmax - xmin) * (WIDTH - 1))
raw_v = round((ymax - y) / (ymax - ymin) * (HEIGHT - 1))

u = max(0, min(WIDTH - 1, raw_u))
v = max(0, min(HEIGHT - 1, raw_v))
return u, v, outside

def set_pixel(image, x, y, value):
    if 0 <= x < WIDTH and 0 <= y < HEIGHT:
        image[y][x] = value

def draw_disc(image, cx, cy, radius, value):
    for y in range(cy - radius, cy + radius + 1):
        for x in range(cx - radius, cx + radius + 1):
            if (x - cx) ** 2 + (y - cy) ** 2 <= radius ** 2:
                set_pixel(image, x, y, value)

def render_frame(frame_index):
    """由單一影格狀態同時產生影像、ID遮罩與狀態標註。"""
    time_s = frame_index / FPS
    background = (18, 36, 54)

    color_image = [
        [background for _ in range(WIDTH)]
        for _ in range(HEIGHT)
    ]
    id_mask = [
        [0 for _ in range(WIDTH)]
        for _ in range(HEIGHT)
    ]
    annotations = []

    for obj in OBJECTS:
        x = obj["x0"] + obj["vx"] * time_s
        y = obj["y"]
        u, v, outside = map_to_pixel(x, y)

        draw_disc(color_image, u, v, DISC_RADIUS, tuple(obj["srgb8"]))
        draw_disc(id_mask, u, v, DISC_RADIUS, obj["id"])

        annotations.append({
            "object_id": obj["id"],
            "position_world_m": [x, y, 0.0],
            "position_pixel_uv": [u, v],
            "outside_world_range": outside,
            "source": "simulated",
        })

    return time_s, color_image, id_mask, annotations

```

```

def write_ppm(path, image):
    with open(path, "w", encoding="ascii") as f:
        f.write(f"P3\n{WIDTH} {HEIGHT}\n255\n")
        for row in image:
            f.write(" ".join(
                f"{r} {g} {b}" for r, g, b in row
            ))
            f.write("\n")

def write_pgm(path, mask):
    with open(path, "w", encoding="ascii") as f:
        f.write(f"P2\n{WIDTH} {HEIGHT}\n255\n")
        for row in mask:
            f.write(" ".join(str(value) for value in row))
            f.write("\n")

def write_json(path, value):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)

def validate_frame_record(manifest, frame_index, output_dir=OUTPUT):
    """唯讀查詢：驗證指定影格並回傳具來源依據的物件資料。"""
    if manifest.get("data_status") != "simulated":
        raise ValueError("本查詢只接受明確標記為 simulated 的資料")

    snapshot = manifest.get("config_snapshot")
    if not isinstance(snapshot, dict) or not snapshot:
        raise ValueError("manifest 缺少有效設定快照")
    try:
        expected_id = config_id(snapshot)
    except (TypeError, ValueError) as exc:
        raise ValueError("設定快照無法計算摘要") from exc
    if manifest.get("config_id") != expected_id:
        raise ValueError("設定快照與 config_id 不一致")

    frames = manifest.get("frames")
    if not isinstance(frames, list):
        raise ValueError("manifest 缺少 frames 清單")

    matches = [
        item for item in frames
        if item.get("frame_index") == frame_index
    ]
    if len(matches) != 1:
        raise ValueError("影格不存在或影格索引重複")

    record = matches[0]
    annotation_path = output_dir / record["annotation"]
    with open(annotation_path, "r", encoding="utf-8") as f:
        annotation = json.load(f)

    if annotation.get("frame_index") != frame_index:
        raise ValueError("manifest 與標註的影格編號不一致")
    if annotation.get("time_s") != record.get("time_s"):

```

```

        raise ValueError("manifest 與標註的時間不一致")

objects = annotation.get("objects")
if not isinstance(objects, list):
    raise ValueError("標註缺少 objects 清單")

required = {
    "object_id",
    "position_world_m",
    "position_pixel_uv",
    "source",
}
seen_ids = set()
for obj in objects:
    if not isinstance(obj, dict) or not required.issubset(obj):
        raise ValueError("物件標註缺少必要欄位")
    if obj["source"] != "simulated":
        raise ValueError("物件來源不是 simulated")
    if obj["object_id"] in seen_ids:
        raise ValueError("影格內物件ID重複")
    seen_ids.add(obj["object_id"])

    world = obj["position_world_m"]
    pixel = obj["position_pixel_uv"]
    if not isinstance(world, list) or len(world) != 3:
        raise ValueError("世界位置須為三維座標")
    if not isinstance(pixel, list) or len(pixel) != 2:
        raise ValueError("像素位置須為二維索引")

return {
    "dataset_id": manifest["dataset_id"],
    "config_id": manifest["config_id"],
    "generator_version": manifest["generator_version"],
    "frame_index": frame_index,
    "time_s": record["time_s"],
    "annotation_file": record["annotation"],
    "fields_used": [
        "objects[].object_id",
        "objects[].position_world_m",
        "objects[].position_pixel_uv",
        "objects[].source",
    ],
    "objects": objects,
}

def test_logic():
    # 角落映射。
    assert map_to_pixel(X_RANGE[0], Y_RANGE[1]) == (0, 0, False)
    assert map_to_pixel(X_RANGE[1], Y_RANGE[0]) == (
        WIDTH - 1, HEIGHT - 1, False
    )

    # 世界座標略微超界，取整後即使仍接近邊界也必須標示出界。
    assert map_to_pixel(X_RANGE[0] - 0.001, 0.0)[2] is True

    # 同一狀態同時生成影像、遮罩與標註。
    time_s, color, mask, objects = render_frame(0)
    assert time_s == 0.0

```



```

assert len(objects) == 2
assert {obj["object_id"] for obj in objects} == {1, 2}
assert {value for row in mask for value in row} == {0, 1, 2}
assert all(
    0 <= channel <= 255
    for row in color
    for pixel in row
    for channel in pixel
)
assert all(obj["source"] == "simulated" for obj in objects)

def test_query_rejects_wrong_source(manifest, output_dir):
    """以複製的標註測試查詢拒絕非合成來源。"""
    record = manifest["frames"][0]
    path = output_dir / record["annotation"]
    with open(path, "r", encoding="utf-8") as f:
        original = json.load(f)

    altered = json.loads(json.dumps(original))
    altered["objects"][0]["source"] = "measured"
    write_json(path, altered)
    try:
        try:
            validate_frame_record(manifest, 0, output_dir)
        except ValueError as error:
            assert "不是 simulated" in str(error)
        else:
            raise AssertionError("查詢未拒絕非合成來源")
    finally:
        write_json(path, original)

def main():
    test_logic()
    OUTPUT.mkdir(parents=True, exist_ok=True)

    config = configuration()
    manifest = {
        "dataset_id": "synthetic-tank-demo-v1",
        "data_status": "simulated",
        "config_id": config_id(config),
        "config_snapshot": config,
        "generator_version": GENERATOR_VERSION,
        "units": {
            "position_world": "m",
            "time": "s",
            "velocity": "m/s",
            "pixel_position": "integer pixel index",
        },
        "color_space": "sRGB encoded 8-bit display codes",
        "mask_semantics": "integer object IDs; 0 is background",
        "fps": FPS,
        "frame_count": FRAME_COUNT,
        "frames": [],
    }

    for k in range(FRAME_COUNT):
        time_s, color, mask, objects = render_frame(k)

```

```

image_name = f"frame_{k:04d}.ppm"
mask_name = f"mask_{k:04d}.pgm"
annotation_name = f"frame_{k:04d}.json"

write_ppm(OUTPUT / image_name, color)
write_pgm(OUTPUT / mask_name, mask)
write_json(OUTPUT / annotation_name, {
    "frame_index": k,
    "time_s": time_s,
    "objects": objects,
})

manifest["frames"].append({
    "frame_index": k,
    "time_s": time_s,
    "image": image_name,
    "id_mask": mask_name,
    "annotation": annotation_name,
})

write_json(OUTPUT / "manifest.json", manifest)

# 由輸出的manifest再讀入，示範唯讀查詢。
with open(OUTPUT / "manifest.json", "r", encoding="utf-8") as f:
    saved_manifest = json.load(f)
    result = validate_frame_record(saved_manifest, 2)
    print(json.dumps(result, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

生成設定與版本識別

`config_snapshot` 保存本次生成使用的影像尺寸、影格率、範圍、物件初始狀態、顏色、標記半徑及座標約定。`config_id` 是將設定以排序鍵序列化後計算的SHA-256摘要；設定欄位改變時，摘要也會改變。查詢會拒絕缺失或非字典的快照，並重新計算摘要、核對 `config_id`。測試時可刪除 `config_snapshot`，或保留原摘要但改動快照的影像尺寸；兩者都應引發 `ValueError`。這只檢查資料內部一致性，不是數位簽章，不能證明資料沒有被連同摘要一起改寫。`generator_version` 是手動維護的生成器版本標籤；修改生成演算法時應更新它。它不是程式檔案的自動雜湊。

目錄預期如下：

```

project/
  README.md
  digital_twin.py
  output/
    manifest.json
    frame_0000.ppm
    mask_0000.pgm
    frame_0000.json
    ...

```

本例由 `configuration()` 建立設定快照並寫入manifest，不需要額外的 `config.json`。若改成外部設定檔，應記錄實際讀取的設定內容或內容摘要，並在manifest保存檔名及識別碼。

測試與預期結果

程式會呼叫 `test_logic()`，再生成五組影格產物。按程式邏輯，預期每個影格包括一張PPM、一張PGM ID遮罩及一份JSON標註，另有一份manifest。程式最後載入已寫出的manifest，查詢第2影格，並驗證標註影格編號、時間、必要欄位及逐物件來源標記。這些是依程式推得的預期，並非已執行結果。

`test_query_rejects_wrong_source()` 提供一個反例測試：它暫時把一筆標註來源改成 `measured`，確認查詢會拒絕，最後還原原始JSON。此測試只能在產生輸出後呼叫；若要手動執行，可在生成manifest及影格檔案後呼叫：

```
test_query_rejects_wrong_source(saved_manifest, OUTPUT)
```

手動驗收時確認：

1. manifest有完整 `config_snapshot`、`config_id` 及 `generator_version`。
2. 每個影格編號只有一筆manifest記錄，標註JSON時間與該記錄相同。
3. 彩色影像、ID遮罩與標註由同一個 `render_frame` 狀態生成。
4. PPM顏色是示意性8-bit sRGB編碼值；PGM遮罩使用整數ID，背景為0。
5. 查詢結果附有資料集ID、設定ID、生成器版本、影格、時間、標註檔名及欄位。
6. 出界物件保留原始世界座標，並以世界座標範圍判定 `outside_world_range`。

除錯與常見陷阱

- **只記錄生成器檔名**：相同檔名的程式內容可能改變。本例記錄手動維護的版本標籤；正式專案也可記錄程式碼提交識別或檔案雜湊。
- **設定快照與生成參數不一致**：若修改常數卻未更新 `configuration()`，manifest便不能代表實際生成狀態。應讓生成器從同一個設定物件讀值，並以該物件渲染及寫入快照。
- **時間或影格索引錯位**：本例第0影格時間為0秒，時間公式為 k/f 。若改用從1開始的索引，須同步修改公式、清單與測試。
- **影像Y軸方向顛倒**：世界Y向上、影像列索引向下，映射時需翻轉Y。
- **邊界夾取掩蓋出界狀態**：像素位置會夾在影像範圍，但 `outside_world_range` 在取整及夾取前按世界座標判定。不得把邊界像素當成物件原始位置。
- **混淆色彩與標籤**：PPM通道值是示意性sRGB編碼值；PGM中的ID是整數標籤，不應套用色彩轉換或插值。
- **把影像當成三維定位證據**：本例沒有相機模型或深度；二維像素位置不能唯一決定三維世界位置。
- **查詢只檢查整體資料來源**：manifest標成 `simulated` 不保證每一筆物件標註都來自模擬。查詢還須逐筆檢查 `source`、必要欄位及唯一ID；有任何不符就拒絕回答。

- **查詢越權或猜測**：查詢只讀資料並驗證欄位一致性。欄位缺漏、時間不同、影格重複或來源不符時，不應自行補猜。

養殖數位分身案例

本章以移動標記代表合成魚，示範如何連結影像、ID遮罩與狀態標註。若擴充到三維圖學場景，資料流可包括：

1. **場景資料**：池體尺寸、魚體資產、世界變換及材質識別碼；長度使用公尺。
2. **動畫資料**：固定影格率、時間及穩定物件ID的變換。動畫是合成設定，不表示實際魚類行為。
3. **相機資料**：影像尺寸、投影類型、內外參、近平面與遠平面；明示相機座標系。
4. **渲染輸出**：彩色影像、物件ID遮罩，以及若有實作才提供的深度通道。ID與深度是資料通道，不作sRGB色彩處理。
5. **標註與清單**：每影格記錄ID、時間、相機識別碼、輸出檔名及資料來源。由場景直接生成的標註應標成合成標註。
6. **唯讀查詢**：讀取manifest與標註，核對資料集ID、影格和時間，並逐筆確認來源標記後，回傳物件欄位與依據。

本程式實際生成彩色影像及ID遮罩，並提供合成狀態JSON和唯讀查詢；它沒有生成三維相機深度、真實感測資料或完整三維渲染結果。

執行順序與驗收

執行順序為：確認設定與單位，執行 `digital_twin.py`，檢查manifest快照與識別碼，再抽查一個影格的PPM、PGM和JSON。唯讀查詢應驗證影格存在、時間一致、物件欄位完整且每筆來源皆為 `simulated`。

專題驗收至少涵蓋：

- **設定可追溯**：快照包含實際使用的物件位置、速度、顏色、影像範圍與標記半徑。
- **輸出可對應**：同一影格的影像、遮罩及JSON使用相同索引與時間。
- **查詢可稽核**：回答附上資料集ID、設定ID、生成器版本、影格編號、時間、標註檔案及欄位。
- **失效可辨識**：設定缺失、影格重複、時間不符、必要欄位缺漏或物件來源不符時，查詢拒絕回答。
- **用途受限**：合成輸出只說明生成器設定，不代替現場觀測、物理驗證或生物判斷。

唯讀查詢只整理記錄，不連接真實設備、不發出控制指令，也不自行加藥、投餌或操作養殖系統。若問題超出資料欄位，應回答「資料未提供」，而不是由合成動畫推測現實狀態。

習題

習題一：手算時間與像素

影格率為10影格／秒，第7影格的合成物件位置為 $x = 0.25$ 公尺。水平範圍為 $[-1, 1]$ 公尺，影像寬 $W = 21$ 。計算時間與像素欄索引。

習題二：程式測試設計

將 `FRAME_COUNT` 改為8、`FPS` 改為2。列出三項檢查manifest與影格時間一致的測試。

習題三：反例與除錯

某生成器把影像、ID遮罩和JSON分開計算；同一物件在影像使用第2影格位置，JSON卻使用第3影格位置。說明為何檔名相同仍不足以證明它們一致，並提出修正方法。

習題四：整合應用

設計唯讀查詢以回答「資料集的第 k 影格有哪些合成物件？其世界位置與像素位置為何？」列出讀取資料、驗證步驟與答案應附的依據，並說明此回答不能推論什麼。

習題解答

解答一

影格時間為

$$t_7 = \frac{7}{10} = 0.7 \text{ 秒}.$$

像素索引為

$$u = \text{round} \left(\frac{0.25 - (-1)}{1 - (-1)} (21 - 1) \right) = \text{round}(12.5) = 12.$$

Python的 `round` 對中點採銀行家捨入，所以12.5得到12。若專案採其他取整規則，生成器與測試必須明確採用同一規則。

解答二

可測：

1. manifest的影格數為8，索引恰為0至7且無重複。

2. 每筆時間等於 `frame_index / FPS`；第0影格為0秒，第7影格為3.5秒。
3. 每個影格標註中的 `frame_index` 和 `time_s` 分別與manifest相符。
4. 每筆影格記錄均包含影像、遮罩及標註路徑。

檔案是否存在須在實際生成後於本機檢查；單靠程式邏輯不能證明檔案已成功寫出。

解答三

相同檔名只表示檔名一致，不代表生成時使用同一份狀態。若影像和JSON採用不同影格位置，兩者語意不一致，會造成錯誤標註。應由一個共用的影格狀態計算函式取得時間與物件位置，再將同一份狀態傳給影像、遮罩和JSON生成器；測試應核對三種輸出的影格編號、時間、ID及對應位置。

解答四

查詢讀取manifest及該影格的JSON標註。先確認資料集ID與整體 `data_status`，再檢查影格索引唯一存在、JSON中的影格編號與manifest相符、時間相等。接著確認 `objects` 是清單，每筆物件包含必要欄位、ID不重複，且每筆 `source` 都是 `simulated`；任一檢查失敗便拒絕回答。

答案應附資料集ID、設定ID、生成器版本、影格編號、時間、標註檔名及使用欄位。它只能回報合成資料中記錄的位置，不能推論現場魚數、真實三維位置、健康狀態、生態行為或感測器讀值。

本章小結

可稽核的合成專題需讓設定、生成器版本、影格、影像、遮罩與標註彼此可追溯。本章以固定時間步生成PPM影像、PGM ID遮罩及JSON合成狀態，將完整設定快照寫入manifest，並提供驗證影格、時間與逐筆來源的唯讀查詢。顏色是示意性8-bit sRGB編碼值，ID是整數標籤；兩者語意不同。合成輸出不等於實測，也不構成生物、物理或養殖控制驗證。

參考來源

以下資料供延伸閱讀；列出來源不表示本章已逐項獨立核對或執行官方範例。

- [G1] PBRT 4：Transformations，https://pbr-book.org/4ed/Geometry_and_Transformations/Transformations
- [G3] PBRT 4：The Light Transport Equation，https://pbr-book.org/4ed/Light_Transport_and_Surface_Reflection/The_Light_Transport_Equation
- [G5] LearnOpenGL：Transformations，<https://learnopengl.com/Getting-started/Transformations>
- [G8] Khronos glTF 2.0規格，<https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>

附錄：實作契約與驗收清單

建議目錄

- `chapters/`：章稿；`editorial/`：逐章及跨章意見、精準修訂與版本紀錄。
- `figures/`：本機SVG圖解；`data/scene.json`：合成場景契約。
- `examples/`：可獨立執行的起步程式與測試。
- `agents/`：40個角色各自的session、輸入及事件，不是作業系統安全沙箱。

座標與顏色

使用右手世界系、column向量、相機朝局部負Z、OpenGL式NDC深度 $[-1,1]$ 。其他API需明列轉換。像素原點左上，UV的v向上，兩者不可直接混用。材質與光照在線性RGB計算；顏色輸出才做sRGB轉換，法線與深度不是顏色。

實驗紀錄

每個實作應記錄Python與NumPy版本、種子、輸入檔雜湊、尺寸、輸出及容差。效能紀錄還需硬體、解析度、樣本數及計時方法。單一seed的好圖不構成統計可靠性；未執行就寫預期，不捏造時間或FPS。

整合驗收

座標往返、退化幾何、透視深度、共邊光柵化、法線正交、sRGB往返、射線最近交點、BVH與暴力法一致、抽樣PDF、全反射、四元數雙覆蓋、bind pose還原、IK不可達與資料時間切分，都應各有正例及反例。不是每章模型批准就等於這些程式已跑完。

安全與範圍

模型只生成文件，不安裝套件、不執行程式、不更動遠端服務。核心CPU實驗不需要真實設備或敏感資料。真實養殖操作、物理場標定與專業判斷不由圖學模型代替。

參考來源

G1～G6已取得頁面內容作為候選參考，不表示逐條論點已外部驗證。G7沿用第一卷來源，G8本次尚未逐節查核。SciPy SLERP頁面本次取用失敗，不列為已查閱來源。讀者可按URL回查；不得仿抄來源的長段文字、杜撰頁碼或聲稱已跑官方範例。

- [G1] PBRT 4：Transformations
- [G2] PBRT 4：Reflection Models
- [G3] PBRT 4：The Light Transport Equation
- [G4] Ray Tracing in One Weekend
- [G5] LearnOpenGL：Transformations
- [G6] Blender Manual：Skinning Introduction
- [G7] NumPy線性代數參考
- [G8] Khronos glTF 2.0規格

出版與字型授權

本版由既有Markdown排版，未執行章內程式碼。字型：Noto Sans CJK TC（SIL Open Font License 1.1）；數學排版：KaTeX（MIT）。全文來源SHA-256：b85d5b68a09eaecbaf2f366ada88bc9ce09847fd5dd0ebc7f8e8a5c161653185。

► 字型與數學排版授權全文